

AI Change Request Analyzer

Complete Source Code Listing

239 files · generated September 20, 2026

Table of Contents

Backend (Python / FastAPI)	(121 files)
backend/app/__init__.py	(1 lines)
backend/app/api/__init__.py	(1 lines)
backend/app/api/admin.py	(439 lines)
backend/app/api/analytics.py	(187 lines)
backend/app/api/auth.py	(103 lines)
backend/app/api/change_requests.py	(2373 lines)
backend/app/api/dashboard.py	(230 lines)
backend/app/api/deps.py	(78 lines)
backend/app/api/health.py	(36 lines)
backend/app/api/knowledge.py	(218 lines)
backend/app/api/my_work.py	(439 lines)
backend/app/api/notifications.py	(95 lines)
backend/app/api/reports.py	(174 lines)
backend/app/api/repository.py	(83 lines)
backend/app/api/users.py	(23 lines)
backend/app/core/__init__.py	(1 lines)
backend/app/core/config.py	(151 lines)
backend/app/core/security.py	(69 lines)
backend/app/database/__init__.py	(1 lines)
backend/app/database/cleanup_dummy_data.py	(255 lines)
backend/app/database/init_db.py	(199 lines)
backend/app/database/seed.py	(201 lines)
backend/app/database/seed_bulk.py	(194 lines)
backend/app/database/seed_demo.py	(693 lines)
backend/app/database/session.py	(55 lines)
backend/app/main.py	(103 lines)
backend/app/models/__init__.py	(68 lines)
backend/app/models/affected_component.py	(38 lines)
backend/app/models/analysis.py	(120 lines)
backend/app/models/approval.py	(60 lines)
backend/app/models/approval_rule.py	(50 lines)
backend/app/models/change_request.py	(98 lines)
backend/app/models/change_request_assignment.py	(35 lines)
backend/app/models/change_request_history.py	(55 lines)
backend/app/models/change_request_version.py	(43 lines)
backend/app/models/clarification_question.py	(38 lines)
backend/app/models/comment.py	(38 lines)
backend/app/models/dependency.py	(37 lines)
backend/app/models/enums.py	(565 lines)
backend/app/models/impact_assessment.py	(46 lines)
backend/app/models/implementation_task.py	(52 lines)
backend/app/models/indexed_file.py	(63 lines)
backend/app/models/knowledge_chunk.py	(49 lines)
backend/app/models/knowledge_document.py	(82 lines)
backend/app/models/knowledge_evidence.py	(70 lines)
backend/app/models/notification.py	(48 lines)
backend/app/models/repository_finding.py	(82 lines)
backend/app/models/repository_scan.py	(51 lines)
backend/app/models/requirement.py	(53 lines)
backend/app/models/risk.py	(37 lines)
backend/app/models/role_permission.py	(38 lines)
backend/app/models/security_finding.py	(57 lines)
backend/app/models/system_audit_log.py	(50 lines)
backend/app/models/system_setting.py	(35 lines)
backend/app/models/test_case.py	(54 lines)
backend/app/models/user.py	(38 lines)
backend/app/schemas/__init__.py	(1 lines)
backend/app/schemas/admin.py	(231 lines)
backend/app/schemas/affected_component.py	(30 lines)
backend/app/schemas/ai_analysis.py	(710 lines)
backend/app/schemas/analysis.py	(132 lines)
backend/app/schemas/analytics.py	(143 lines)
backend/app/schemas/approval.py	(125 lines)
backend/app/schemas/assignment.py	(30 lines)
backend/app/schemas/auth.py	(38 lines)
backend/app/schemas/change_request.py	(392 lines)
backend/app/schemas/clarification_question.py	(44 lines)
backend/app/schemas/comment.py	(49 lines)
backend/app/schemas/dashboard.py	(59 lines)
backend/app/schemas/dependency.py	(26 lines)
backend/app/schemas/health.py	(10 lines)
backend/app/schemas/impact_assessment.py	(23 lines)
backend/app/schemas/implementation_task.py	(88 lines)
backend/app/schemas/knowledge_chunk.py	(30 lines)
backend/app/schemas/knowledge_document.py	(31 lines)
backend/app/schemas/knowledge_evidence.py	(34 lines)
backend/app/schemas/knowledge_search.py	(20 lines)
backend/app/schemas/my_work.py	(106 lines)

backend/app/schemas/notification.py (22 lines)
backend/app/schemas/report.py (57 lines)
backend/app/schemas/repository_finding.py (98 lines)
backend/app/schemas/repository_scan.py (70 lines)
backend/app/schemas/requirement.py (49 lines)
backend/app/schemas/risk.py (29 lines)
backend/app/schemas/security_finding.py (46 lines)
backend/app/schemas/test_case.py (128 lines)
backend/app/schemas/user.py (36 lines)
backend/app/services/__init__.py (1 lines)
backend/app/services/admin_audit.py (39 lines)
backend/app/services/ai/__init__.py (1 lines)
backend/app/services/ai/base.py (62 lines)
backend/app/services/ai/factory.py (35 lines)
backend/app/services/ai/providers/__init__.py (1 lines)
backend/app/services/ai/providers/anthropic_provider.py (45 lines)
backend/app/services/ai/providers/gemini_provider.py (59 lines)
backend/app/services/ai/providers/ollama_provider.py (64 lines)
backend/app/services/ai/providers/openrouter_provider.py (104 lines)
backend/app/services/analysis_delta.py (158 lines)
backend/app/services/analysis_engine.py (747 lines)
backend/app/services/analytics.py (651 lines)
backend/app/services/approvals.py (202 lines)
backend/app/services/code_understanding.py (288 lines)
backend/app/services/comments.py (93 lines)
backend/app/services/deadlines.py (94 lines)
backend/app/services/history.py (44 lines)
backend/app/services/knowledge_chunker.py (245 lines)
backend/app/services/knowledge_documents.py (123 lines)
backend/app/services/knowledge_embeddings.py (166 lines)
backend/app/services/mentions.py (36 lines)
backend/app/services/notify.py (46 lines)
backend/app/services/permissions.py (70 lines)
backend/app/services/report_generator.py (1159 lines)
backend/app/services/report_registry.py (231 lines)
backend/app/services/repository_linkage.py (124 lines)
backend/app/services/repository_matcher.py (306 lines)
backend/app/services/repository_scanner.py (295 lines)
backend/app/services/system_settings.py (130 lines)
backend/app/services/traceability_linkage.py (136 lines)
backend/app/services/versioning.py (267 lines)
backend/app/services/workflow_rules.py (641 lines)
backend/app/utils/__init__.py (1 lines)

Frontend (React) (61 files)

frontend/src/App.css (17 lines)
frontend/src/App.jsx (40 lines)
frontend/src/api/admin.js (66 lines)
frontend/src/api/analysis.js (150 lines)
frontend/src/api/analytics.js (48 lines)
frontend/src/api/approvals.js (30 lines)
frontend/src/api/auth.js (24 lines)
frontend/src/api/changeRequests.js (64 lines)
frontend/src/api/client.js (53 lines)
frontend/src/api/comments.js (27 lines)
frontend/src/api/dashboard.js (22 lines)
frontend/src/api/knowledge.js (97 lines)
frontend/src/api/myWork.js (41 lines)
frontend/src/api/notifications.js (25 lines)
frontend/src/api/reports.js (77 lines)
frontend/src/api/repository.js (43 lines)
frontend/src/api/users.js (7 lines)
frontend/src/components/AppHeader.jsx (288 lines)
frontend/src/components/AuthLayout.jsx (21 lines)
frontend/src/components/BackendStatusBadge.jsx (26 lines)
frontend/src/components/ChangeRequestDetail.jsx (1468 lines)
frontend/src/components/DistributionChart.jsx (37 lines)
frontend/src/components/EmptyState.jsx (17 lines)
frontend/src/components/ImpactGraph.jsx (288 lines)
frontend/src/components/MetricCard.jsx (29 lines)
frontend/src/components/Modal.jsx (33 lines)
frontend/src/components/RecentRequestsTable.jsx (129 lines)
frontend/src/components/RiskTrendChart.jsx (63 lines)
frontend/src/components/auth-layout.css (222 lines)
frontend/src/components/header.css (354 lines)
frontend/src/context/AuthContext.jsx (104 lines)
frontend/src/index.css (9 lines)
frontend/src/main.jsx (11 lines)
frontend/src/pages/AdminPage.jsx (887 lines)
frontend/src/pages/AnalysisDashboardPage.jsx (2884 lines)
frontend/src/pages/AnalyticsPage.jsx (490 lines)
frontend/src/pages/AuthenticatedHome.jsx (87 lines)

frontend/src/pages/ChangeRequestsPage.jsx (380 lines)
frontend/src/pages/ComingSoonPage.jsx (17 lines)
frontend/src/pages/DashboardPage.jsx (325 lines)
frontend/src/pages/EditChangeRequestPage.jsx (504 lines)
frontend/src/pages/KnowledgeBasePage.jsx (374 lines)
frontend/src/pages/LoginPage.jsx (83 lines)
frontend/src/pages/MyWorkPage.jsx (647 lines)
frontend/src/pages/NewChangeRequestPage.jsx (387 lines)
frontend/src/pages/NotificationsPage.jsx (202 lines)
frontend/src/pages/RegisterPage.jsx (110 lines)
frontend/src/pages/ReportDetailPage.jsx (209 lines)
frontend/src/pages/ReportsPage.jsx (390 lines)
frontend/src/styles/admin.css (240 lines)
frontend/src/styles/analysis-dashboard.css (887 lines)
frontend/src/styles/analytics.css (231 lines)
frontend/src/styles/change-request-form.css (346 lines)
frontend/src/styles/change-requests-list.css (373 lines)
frontend/src/styles/dashboard.css (423 lines)
frontend/src/styles/home.css (25 lines)
frontend/src/styles/impact-graph.css (257 lines)
frontend/src/styles/knowledge-base.css (57 lines)
frontend/src/styles/modal.css (71 lines)
frontend/src/styles/my-work.css (36 lines)
frontend/src/styles/workflow.css (579 lines)

Tests (57 files)

tests/_init_.py (1 lines)
tests/conftest.py (37 lines)
tests/test_ai_analysis_phase3.py (384 lines)
tests/test_ai_analysis_phase4.py (289 lines)
tests/test_ai_analysis_v2.py (293 lines)
tests/test_analysis_delta.py (302 lines)
tests/test_analysis_engine.py (457 lines)
tests/test_auth.py (103 lines)
tests/test_change_request_editing.py (379 lines)
tests/test_change_request_management.py (286 lines)
tests/test_change_requests.py (124 lines)
tests/test_clarification_answer.py (247 lines)
tests/test_dashboard.py (323 lines)
tests/test_database.py (115 lines)
tests/test_health.py (30 lines)
tests/test_module14_phase2.py (284 lines)
tests/test_module14_phase3.py (164 lines)
tests/test_module14_phase4.py (230 lines)
tests/test_module14_phase5.py (240 lines)
tests/test_module14_phase6.py (285 lines)
tests/test_module15_phase1.py (181 lines)
tests/test_module15_phase2.py (232 lines)
tests/test_module15_phase3.py (336 lines)
tests/test_module15_phase4.py (242 lines)
tests/test_module15_phase5.py (322 lines)
tests/test_module15_phase7_security.py (215 lines)
tests/test_module16_phase1.py (192 lines)
tests/test_module16_phase2.py (211 lines)
tests/test_module16_phase3.py (241 lines)
tests/test_module16_phase4.py (275 lines)
tests/test_module16_phase6.py (154 lines)
tests/test_module16_phase7_security.py (296 lines)
tests/test_module17_phase1.py (253 lines)
tests/test_module17_phase2.py (282 lines)
tests/test_module17_phase3.py (214 lines)
tests/test_module17_phase4.py (269 lines)
tests/test_module17_phase6.py (284 lines)
tests/test_module17_phase7_sync.py (300 lines)
tests/test_module18_phase1.py (216 lines)
tests/test_module18_phase2.py (248 lines)
tests/test_module18_phase3.py (289 lines)
tests/test_module18_phase7_notifications.py (425 lines)
tests/test_module19_phase1.py (203 lines)
tests/test_module19_phase2.py (305 lines)
tests/test_module19_phase3.py (267 lines)
tests/test_module19_phase4.py (271 lines)
tests/test_module19_phase5.py (235 lines)
tests/test_module19_phase7.py (185 lines)
tests/test_module20_phase1.py (399 lines)
tests/test_module21_admin.py (483 lines)
tests/test_module22_e2e.py (463 lines)
tests/test_module24_reports.py (359 lines)
tests/test_seed_bulk.py (49 lines)
tests/test_workflow_approvals.py (465 lines)
tests/test_workflow_comments_notifications.py (582 lines)
tests/test_workflow_foundation.py (414 lines)

tests/test_workflow_status_assignments.py (299 lines)

backend/app/__init__.py

1 lines

backend/app/api/__init__.py

1 lines


```

"""Module 21 (Administration & Configuration): the admin section's API.

Every endpoint in this router requires require_admin (app/api/deps.py) -
an authenticated, active Admin account. Nothing here replaces the existing
authentication system (Module 1) or the per-change-request permission
helpers already in app/services/workflow_rules.py - this is purely
account/role/permission/approval-rule/system-setting administration, one
level above any single change request.
"""
from __future__ import annotations

import json
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query, Response, status
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session, selectinload

from app.api.deps import require_admin
from app.core.security import hash_password
from app.database.session import get_db
from app.models.approval_rule import ApprovalRule
from app.models.enums import AdminAuditAction, ApprovalType, Capability, UserRole
from app.models.role_permission import RolePermission
from app.models.system_audit_log import SystemAuditLog
from app.models.user import User
from app.schemas.admin import (
    AdminUserCreate,
    AdminUserUpdate,
    ApprovalRuleCreate,
    ApprovalRuleRead,
    ApprovalRuleUpdate,
    CrCategoriesUpdate,
    RolePermissionBulkUpdate,
    RolePermissionRead,
    SystemAuditLogRead,
    SystemAuditLogResponse,
    SystemSettingsRead,
)
from app.schemas.user import UserRead
from app.services import admin_audit, system_settings
from app.services.permissions import LOCKED_CAPABILITIES
from app.services.workflow_rules import APPROVAL_TYPE_LABELS, USER_ROLE_LABELS

router = APIRouter(prefix="/api/admin", tags=["admin"])

# --- User management (spec section 1) -----

@router.get("/users", response_model=list[UserRead])
def list_users(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> list[User]:
    return db.query(User).order_by(User.name).all()

@router.post("/users", response_model=UserRead, status_code=status.HTTP_201_CREATED)
def create_user(
    payload: AdminUserCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> User:
    existing = db.query(User).filter(User.email == payload.email).first()
    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="An account with this email already exists."
        )

    user = User(
        name=payload.name,
        email=payload.email,
        password_hash=hash_password(payload.password),
        role=payload.role,
        is_active=True,
    )
    db.add(user)
    try:
        db.flush()
    except IntegrityError:
        # Module 22: same narrow race as app/api/auth.py::register - two
        # admin-create-user calls (or one of these racing a self-service
        # /auth/register) for the same email can both pass the .first()
        # check above before either writes. Without this, the second
        # flush would raise an uncaught IntegrityError (the unique
        # constraint on User.email) instead of the same clean 409 the
        # normal duplicate-email path above already returns.
        db.rollback()
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="An account with this email already exists."
        )

    admin_audit.record(
        db,
        action=AdminAuditAction.USER_CREATED,
        actor_user_id=current_user.id,
        target_type="user",
        target_id=str(user.id),
        detail=f"{current_user.name} created {user.name} ({user.email}) as {USER_ROLE_LABELS.get(user.role, user.role.value)}.",
    )
    db.commit()
    db.refresh(user)
    return user

@router.patch("/users/{user_id}", response_model=UserRead)
def update_user(
    user_id: int,
    payload: AdminUserUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> User:
    user = db.get(User, user_id)
    if user is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")

    changes = payload.model_dump(exclude_unset=True)
    if not changes:
        return user

    if "is_active" in changes and changes["is_active"] is False and user.id == current_user.id:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="You can't deactivate your own account.",
        )

```

```

    )
    if (
        "role" in changes
        and changes["role"] != UserRole.ADMIN
        and user.id == current_user.id
        and user.role == UserRole.ADMIN
    ):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="You can't remove your own Admin role.",
        )

    if "role" in changes and changes["role"] != user.role:
        old_role_label = USER_ROLE_LABELS.get(user.role, user.role.value)
        user.role = changes["role"]
        new_role_label = USER_ROLE_LABELS.get(user.role, user.role.value)
        admin_audit.record(
            db,
            action=AdminAuditAction.USER_ROLE_CHANGED,
            actor_user_id=current_user.id,
            target_type="user",
            target_id=str(user.id),
            detail=f"{current_user.name} changed {user.name}'s role from {old_role_label} to {new_role_label}.",
        )

    if "is_active" in changes and changes["is_active"] != (user.is_active is not False):
        user.is_active = changes["is_active"]
        admin_audit.record(
            db,
            action=AdminAuditAction.USER_ACTIVATED if user.is_active else AdminAuditAction.USER_DEACTIVATED,
            actor_user_id=current_user.id,
            target_type="user",
            target_id=str(user.id),
            detail=f"{current_user.name} {'activated' if user.is_active else 'deactivated'} {user.name} ({user.email}).",
        )

    db.commit()
    db.refresh(user)
    return user

# --- System-level audit log (spec section 6) -----

@router.get("/audit-log", response_model=SystemAuditLogResponse)
def list_audit_log(
    actor_user_id: Optional[int] = Query(None),
    action: Optional[AdminAuditAction] = Query(None),
    limit: int = Query(50, ge=1, le=200),
    offset: int = Query(0, ge=0),
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> SystemAuditLogResponse:
    query = db.query(SystemAuditLog).options(selectinload(SystemAuditLog.actor))
    if actor_user_id is not None:
        query = query.filter(SystemAuditLog.actor_user_id == actor_user_id)
    if action is not None:
        query = query.filter(SystemAuditLog.action == action)

    total = query.count()
    rows = query.order_by(SystemAuditLog.created_at.desc()).offset(offset).limit(limit).all()
    items = [
        SystemAuditLogRead(
            id=row.id,
            actor_user_id=row.actor_user_id,
            actor_label=row.actor_label,
            actor_name=row.actor_name if row.actor else row.actor_label,
            action=row.action,
            target_type=row.target_type,
            target_id=row.target_id,
            detail=row.detail,
            created_at=row.created_at,
        )
        for row in rows
    ]
    return SystemAuditLogResponse(items=items, total=total, limit=limit, offset=offset)

# --- Permissions matrix (spec section 3) -----

@router.get("/permissions", response_model=list[RolePermissionRead])
def list_permissions(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> list[RolePermission]:
    """Every (role, capability) row - init_db seeds all of them on
    startup, so this is always the complete matrix, not a sparse list the
    frontend has to fill gaps in."""
    return db.query(RolePermission).order_by(RolePermission.role, RolePermission.capability).all()

@router.put("/permissions", response_model=list[RolePermissionRead])
def update_permissions(
    payload: RolePermissionBulkUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> list[RolePermission]:
    """Bulk-update a set of (role, capability) -> allowed pairs in one
    call - the frontend's permissions matrix saves every checkbox that
    changed at once, not one request per cell. Administration can never be
    changed through this endpoint, in either direction (see
    app/services/permissions.py::LOCKED_CAPABILITIES's own docstring) -
    it stays hardcoded True for Admin and False for everyone else, so an
    admin can never accidentally lock every admin out of the admin section."""
    locked = [p for p in payload.permissions if p.capability in LOCKED_CAPABILITIES]
    if locked:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Administration access can't be changed - it's always Admin-only.",
        )

    changed_rows: list[RolePermission] = []
    for entry in payload.permissions:
        row = (
            db.query(RolePermission)
            .filter(RolePermission.role == entry.role, RolePermission.capability == entry.capability)
            .first()
        )
        if row is None:
            row = RolePermission(role=entry.role, capability=entry.capability, allowed=entry.allowed)
            db.add(row)
            db.flush()
        elif row.allowed == entry.allowed:
            continue
        else:
            row.allowed = entry.allowed
            changed_rows.append(row)

    if changed_rows:
        role_label = lambda r: USER_ROLE_LABELS.get(r, r.value) # noqa: E731

```

```

summary = ""; ".join(
    f"{role_label(row.role)} / {row.capability.value.replace('_', ' ')} -> {'allowed' if row.allowed else 'blocked'}"
    for row in changed_rows
)
admin_audit.record(
    db,
    action=AdminAuditAction.PERMISSION_CHANGED,
    actor_user_id=current_user.id,
    target_type="role_permission",
    detail=f"{current_user.name} updated permissions: {summary}.",
)
db.commit()

return db.query(RolePermission).order_by(RolePermission.role, RolePermission.capability).all()

# --- Approval rules (spec section 4) -----

def _validate_approval_types(values: list[str]) -> None:
    valid = {t.value for t in ApprovalType}
    unknown = [v for v in values if v not in valid]
    if unknown:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"Unknown approval type(s): {' '.join(sorted(unknown))}. Valid types: {' '.join(sorted(valid))}."
        )

def _approval_rule_summary(rule: ApprovalRule) -> str:
    labels = "", ".join(
        APPROVAL_TYPE_LABELS.get(ApprovalType(v), v) for v in json.loads(rule.approval_types)
    )
    kind = "Risk level" if rule.rule_type.value == "risk" else "Category keyword"
    return f'{kind} "{rule.match_value}" -> {labels}'

@router.get("/approval-rules", response_model=list[ApprovalRuleRead])
def list_approval_rules(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> list[ApprovalRule]:
    return db.query(ApprovalRule).order_by(ApprovalRule.rule_type, ApprovalRule.match_value).all()

@router.post("/approval-rules", response_model=ApprovalRuleRead, status_code=status.HTTP_201_CREATED)
def create_approval_rule(
    payload: ApprovalRuleCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> ApprovalRule:
    validate_approval_types(payload.approval_types)
    if payload.rule_type.value == "risk" and payload.match_value not in {"low", "medium", "high", "critical"}:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="A risk rule's match value must be one of: low, medium, high, critical."
        )

    rule = ApprovalRule(
        rule_type=payload.rule_type,
        match_value=payload.match_value,
        approval_types=json.dumps(payload.approval_types),
        enabled=payload.enabled,
    )
    db.add(rule)
    db.flush()
    admin_audit.record(
        db,
        action=AdminAuditAction.APPROVAL_RULE_CREATED,
        actor_user_id=current_user.id,
        target_type="approval_rule",
        target_id=str(rule.id),
        detail=f"{current_user.name} added an approval rule: {_approval_rule_summary(rule)}."
    )
    db.commit()
    db.refresh(rule)
    return rule

@router.patch("/approval-rules/{rule_id}", response_model=ApprovalRuleRead)
def update_approval_rule(
    rule_id: int,
    payload: ApprovalRuleUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> ApprovalRule:
    rule = db.get(ApprovalRule, rule_id)
    if rule is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Approval rule not found.")

    changes = payload.model_dump(exclude_unset=True)
    if not changes:
        return rule

    if "approval_types" in changes:
        validate_approval_types(changes["approval_types"])
        rule.approval_types = json.dumps(changes["approval_types"])
    if "match_value" in changes:
        if rule.rule_type.value == "risk" and changes["match_value"] not in {"low", "medium", "high", "critical"}:
            raise HTTPException(
                status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
                detail="A risk rule's match value must be one of: low, medium, high, critical."
            )
        rule.match_value = changes["match_value"]
    if "enabled" in changes:
        rule.enabled = changes["enabled"]

    db.flush()
    admin_audit.record(
        db,
        action=AdminAuditAction.APPROVAL_RULE_UPDATED,
        actor_user_id=current_user.id,
        target_type="approval_rule",
        target_id=str(rule.id),
        detail=f"{current_user.name} updated an approval rule: {_approval_rule_summary(rule)} "
        f"{'(enabled' if rule.enabled else 'disabled')}'."
    )
    db.commit()
    db.refresh(rule)
    return rule

@router.delete("/approval-rules/{rule_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_approval_rule(
    rule_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> Response:
    rule = db.get(ApprovalRule, rule_id)
    if rule is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Approval rule not found.")

```

```

summary = _approval_rule_summary(rule)
db.delete(rule)
admin_audit.record(
    db,
    action=AdminAuditAction.APPROVAL_RULE_DELETED,
    actor_user_id=current_user.id,
    target_type="approval_rule",
    target_id=str(rule_id),
    detail=f"{current_user.name} deleted an approval rule: {summary}.",
)
db.commit()
return Response(status_code=status.HTTP_204_NO_CONTENT)

# --- System settings (spec section 5) -----

@router.get("/system-settings", response_model=SystemSettingsRead)
def get_system_settings(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> SystemSettingsRead:
    """cr_categories is genuinely editable (see the PUT endpoint below);
    ai_provider/repository/knowledge_base are read-only .env-derived info -
    see app/services/system_settings.py's own module docstring for why
    Priority values and workflow transitions aren't surfaced here at all."""
    return SystemSettingsRead(
        cr_categories=system_settings.get_cr_categories(db),
        ai_provider=system_settings.get_ai_provider_info(),
        repository=system_settings.get_repository_info(db),
        knowledge_base=system_settings.get_knowledge_base_info(db),
    )

@router.put("/system-settings/cr-categories", response_model=list[str])
def update_cr_categories(
    payload: CrCategoriesUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
) -> list[str]:
    system_settings.set_cr_categories(db, payload.categories, updated_by=current_user.id)
    admin_audit.record(
        db,
        action=AdminAuditAction.SYSTEM_SETTING_CHANGED,
        actor_user_id=current_user.id,
        target_type="system_setting",
        target_id=system_settings.CR_CATEGORIES_KEY,
        detail=f"{current_user.name} updated the change request categories list: {' '.join(payload.categories)}.",
    )
    db.commit()
    return payload.categories

```

```

"""Module 19 (Engineering Change Analytics): read-only reporting endpoints
built entirely on top of tables Modules 1-18 already own - no new tables,
no invented numbers (spec's own "DO NOT generate fake statistics"). Every
endpoint here shares the same filter set (spec section 8: date range,
status, priority, risk, category, owner) via `parse_filters` below, so
filtering to e.g. "High priority, Security category" gives a consistent
slice of the data no matter which section of the dashboard is asking.

Grows one endpoint per phase - this file currently has Phase 2's
executive + workflow metrics; later phases add risk analytics, approval
bottlenecks, change analytics, workload, and AI analytics endpoints
alongside it, all built on the same `parse_filters` dependency and
`app.services.analytics.filtered_change_requests` query helper.
"""

from __future__ import annotations

from datetime import datetime
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.enums import ChangeRequestStatus, Priority
from app.models.user import User
from app.schemas.analytics import (
    AIAalyticsRead,
    ApprovalBottlenecksRead,
    ChangeAnalyticsRead,
    ExecutiveMetricsRead,
    ExecutiveWorkflowRead,
    RiskAnalyticsRead,
    WorkflowMetricsRead,
    WorkloadAnalyticsRead,
)
from app.services import analytics as analytics_service

router = APIRouter(prefix="/api/analytics", tags=["analytics"])

_VALID_RISK_BUCKETS = {"low", "medium", "high", "critical"}
_VALID_CATEGORIES = set(analytics_service.CANONICAL_CATEGORIES) | {"Other"}

def parse_filters(
    date_from: Optional[datetime] = Query(
        None, description="Only change requests created on/after this date (ISO 8601)."
    ),
    date_to: Optional[datetime] = Query(
        None, description="Only change requests created on/before this date (ISO 8601)."
    ),
    status: Optional[ChangeRequestStatus] = Query(None, description="Exact ChangeRequestStatus value."),
    priority: Optional[Priority] = Query(None, description="Exact Priority value."),
    risk: Optional[str] = Query(None, description="One of: low, medium, high, critical."),
    category: Optional[str] = Query(
        None, description="One of the dashboard's canonical categories, or 'Other'."
    ),
    owner_id: Optional[int] = Query(None, description="Only change requests created by this user."),
) -> analytics_service.AnalyticsFilters:
    """The one shared filter parser every analytics endpoint depends on
    (spec section 8) - so "filtered to these criteria" means the exact same
    thing everywhere on the dashboard, not a slightly different
    interpretation per section. """
    if risk is not None and risk not in _VALID_RISK_BUCKETS:
        raise HTTPException(
            status_code=422, detail=f"risk must be one of {sorted(_VALID_RISK_BUCKETS)}"
        )
    if category is not None and category not in _VALID_CATEGORIES:
        raise HTTPException(
            status_code=422, detail=f"category must be one of {sorted(_VALID_CATEGORIES)}"
        )
    return analytics_service.AnalyticsFilters(
        date_from=date_from,
        date_to=date_to,
        status=status,
        priority=priority,
        risk=risk,
        category=category,
        owner_id=owner_id,
    )

@router.get("/executive", response_model=ExecutiveWorkflowRead)
def get_executive_and_workflow_metrics(
    filters: analytics_service.AnalyticsFilters = Depends(parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ExecutiveWorkflowRead:
    """Spec sections 1 (Executive Metrics) and 2 (Workflow Metrics),
    computed together over the same filtered set of change requests."""
    rows = analytics_service.filtered_change_requests(db, filters)
    executive = analytics_service.executive_metrics(rows)
    workflow = analytics_service.workflow_metrics(rows)
    return ExecutiveWorkflowRead(
        executive=ExecutiveMetricsRead.model_validate(executive),
        workflow=WorkflowMetricsRead.model_validate(workflow),
    )

@router.get("/risk", response_model=RiskAnalyticsRead)
def get_risk_analytics(
    filters: analytics_service.AnalyticsFilters = Depends(parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> RiskAnalyticsRead:
    """Spec section 3 - current risk distribution plus a monthly trend, both
    over the filtered set of change requests."""
    rows = analytics_service.filtered_change_requests(db, filters)
    result = analytics_service.risk_analytics(rows)
    return RiskAnalyticsRead(current_distribution=result.current_distribution, over_time=result.over_time)

@router.get("/approval-bottlenecks", response_model=ApprovalBottlenecksRead)
def get_approval_bottlenecks(
    filters: analytics_service.AnalyticsFilters = Depends(parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ApprovalBottlenecksRead:
    """Spec section 4 - pending approvals by type/person, average approval
    duration by type, and the approval types most often responsible for a
    Rejected/Changes Requested outcome, all over the filtered set of change
    requests. Person-level detail deliberately carries only a name and a
    count - see ApprovalBottlenecksRead's own docstring."""
    rows = analytics_service.filtered_change_requests(db, filters)

```

```

result = analytics_service.approval_bottlenecks(rows)
return ApprovalBottlenecksRead(
    pending_by_type=result.pending_by_type,
    pending_by_person=result.pending_by_person,
    avg_duration_hours_by_type=result.avg_duration_hours_by_type,
    most_common_blockers=result.most_common_blockers,
)

@router.get("/change", response_model=ChangeAnalyticsRead)
def get_change_analytics(
    filters: analytics_service.AnalyticsFilters = Depends(_parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeAnalyticsRead:
    """Spec section 5 - multi-revision counts, average versions per CR, the
    most frequently edited fields, and CRs still awaiting clarification on
    their latest analysis, over the filtered set of change requests."""
    rows = analytics_service.filtered_change_requests(db, filters)
    result = analytics_service.change_analytics(rows)
    return ChangeAnalyticsRead(
        crs_with_multiple_revisions=result.crs_with_multiple_revisions,
        avg_versions_per_cr=result.avg_versions_per_cr,
        most_changed_fields=result.most_changed_fields,
        crs_returned_for_clarification=result.crs_returned_for_clarification,
    )

@router.get("/workload", response_model=WorkloadAnalyticsRead)
def get_workload_analytics(
    filters: analytics_service.AnalyticsFilters = Depends(_parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> WorkloadAnalyticsRead:
    """Spec section 6 - CRs per owner, plus organization-wide pending
    reviews/approvals/overdue-task totals. Excludes the app's own bulk-seed
    load-test bot accounts (see filtered_change_requests's own
    'exclude_bulk_seed_owners' parameter) - confirmed with the user as the
    one section of this module where that exclusion applies."""
    rows = analytics_service.filtered_change_requests(db, filters, exclude_bulk_seed_owners=True)
    result = analytics_service.workload_analytics(rows)
    return WorkloadAnalyticsRead(
        crs_per_owner=result.crs_per_owner,
        pending_reviews=result.pending_reviews,
        pending_approvals=result.pending_approvals,
        overdue_tasks=result.overdue_tasks,
    )

@router.get("/ai", response_model=AIAnalyticsRead)
def get_ai_analytics(
    filters: analytics_service.AnalyticsFilters = Depends(_parse_filters),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> AIAnalyticsRead:
    """Spec section 7 - "AI recommendation statistics" (never "AI
    accuracy" - this app has no ground-truth outcome data to measure
    accuracy against, see AIAnalyticsRead's own docstring), over every
    analysis belonging to a change request in the filtered set."""
    rows = analytics_service.filtered_change_requests(db, filters)
    result = analytics_service.ai_analytics(rows)
    return AIAnalyticsRead.model_validate(result)

```

```

"""Authentication endpoints: register, login, current-user, logout."""
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.core.config import get_settings
from app.core.security import create_access_token, hash_password, verify_password
from app.database.session import get_db
from app.models.enums import UserRole
from app.models.user import User
from app.schemas.auth import LoginRequest, RegisterRequest, TokenResponse
from app.schemas.user import UserRead
from app.services.workflow_rules import is_active_user

router = APIRouter(prefix="/auth", tags=["auth"])

def _issue_token(user: User, remember_me: bool = False) -> TokenResponse:
    settings = get_settings()
    expires_minutes = (
        settings.remember_me_token_expire_minutes if remember_me else settings.access_token_expire_minutes
    )
    token = create_access_token(user_id=user.id, email=user.email, expires_minutes=expires_minutes)
    return TokenResponse(
        access_token=token,
        expires_in=expires_minutes * 60,
        user=UserRead.model_validate(user),
    )

@router.post("/register", response_model=TokenResponse, status_code=status.HTTP_201_CREATED)
def register(payload: RegisterRequest, db: Session = Depends(get_db)) -> TokenResponse:
    existing = db.query(User).filter(User.email == payload.email).first()
    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="An account with this email already exists."
        )

    user = User(
        name=payload.name,
        email=payload.email,
        password_hash=hash_password(payload.password),
        role=UserRole.ENGINEER,
    )
    db.add(user)
    try:
        db.commit()
    except IntegrityError:
        # Module 22: closes a narrow race - two registrations for the same
        # email arriving close enough together can both pass the .first()
        # check above before either commits. Without this, the second
        # commit would raise an uncaught IntegrityError (the unique
        # constraint on User.email) straight through to a generic 500
        # instead of the same clean 409 the normal duplicate-email path
        # already returns just above.
        db.rollback()
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="An account with this email already exists."
        )
    db.refresh(user)

    # Auto-login on register - a fresh account can go straight in rather
    # than bouncing the user to a separate login step.
    return _issue_token(user)

@router.post("/login", response_model=TokenResponse)
def login(payload: LoginRequest, db: Session = Depends(get_db)) -> TokenResponse:
    user = db.query(User).filter(User.email == payload.email).first()
    if user is None or not verify_password(payload.password, user.password_hash):
        # Deliberately the same error for "no such user" and "wrong password" -
        # don't leak which one it was.
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Incorrect email or password.")

    # Module 21: only reachable once the password has already been
    # verified correct - so this never leaks "this email exists but is
    # deactivated" to someone who doesn't already know the password.
    if not is_active_user(user):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="This account has been deactivated. Contact an administrator.",
        )

    return _issue_token(user, remember_me=payload.remember_me)

@router.get("/me", response_model=UserRead)
def read_current_user(current_user: User = Depends(get_current_user)) -> UserRead:
    """A protected endpoint - requires a valid Authorization: Bearer <token> header."""
    return UserRead.model_validate(current_user)

@router.post("/logout")
def logout(current_user: User = Depends(get_current_user)) -> dict:
    # Tokens are stateless JWTs, so there's nothing to invalidate server-side
    # without a token blacklist (deliberately not added - extra
    # infrastructure this project doesn't need yet). The frontend discards
    # its stored token immediately after calling this; this endpoint mainly
    # exists so "logout" is a real authenticated action, not just a
    # frontend-only concept.
    return {"message": "Logged out. Discard the access token client-side."}

```

backend/app/api/change_requests.py

2373 lines

```
"""Change request creation, listing, filtering, detail, and AI-analysis
endpoints (Modules 4, 5, and 6).

Create / list (search/filter/sort) / get-one-with-detail (Modules 4-5) stay
read-only with respect to AI: the "category" / "risk" / "complexity" /
"effective_status" values they compute are derived from whatever analyses
already exist - they never call the AI provider themselves. The Module 6
section at the bottom of this file (analyze / get-analysis) is the only
place that does. No repository/RAG analysis yet - that's a later module.

Every route requires a logged-in user (Depends(get_current_user)).
"""
import json
from datetime import datetime
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query, Response, status
from sqlalchemy import String, cast, or_
from sqlalchemy.orm import Session, selectinload

from app.api.deps import check_capability, get_current_user
from app.database.session import get_db
from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.approval_rule import ApprovalRule
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.change_request_history import ChangeRequestHistory
from app.models.change_request_version import ChangeRequestVersion
from app.models.clarification_question import ClarificationQuestion
from app.models.comment import ChangeRequestComment
from app.models.enums import (
    ApprovalStatus,
    Capability,
    ChangeRequestStatus,
    HistoryAction,
    NotificationType,
    Priority,
    ReviewStatus,
)
from app.models.implementation_task import ImplementationTask
from app.models.repository_finding import RepositoryFinding
from app.models.repository_scan import RepositoryScan
from app.models.requirement import Requirement
from app.models.security_finding import SecurityFinding
from app.models.test_case import TestCase
from app.models.user import User
from app.services import approvals as approvals_service
from app.services import comments as comments_service
from app.services import history as history_service
from app.services import notify as notify_service
from app.services.analysis_delta import compare_analyses
from app.services.mentions import extract_mentioned_users
from app.services import repository_linkage
from app.services import traceability_linkage
from app.services.repository_matcher import RepositoryMatchError, run_repository_match
from app.services.workflow_rules import build_snapshot, risk_bucket
from app.schemas.analysis import AnalysisDeltaResponse, AnalysisRead, AnalysisSummaryRead
from app.schemas.approval import ApprovalRead, ApprovalRequestCreate, ApprovalRespond, RecommendedApprovalType
from app.schemas.assignment import AssignmentCreate, ChangeRequestAssignmentRead
from app.schemas.clarification_question import ClarificationQuestionAnswer, ClarificationQuestionRead
from app.schemas.comment import CommentCreate, CommentRead, CommentResolve
from app.schemas.implementation_task import ImplementationTaskRead, ImplementationTaskUpdate
from app.schemas.repository_finding import RepositoryFindingRead
from app.schemas.requirement import RequirementRead, RequirementReviewUpdate
from app.schemas.security_finding import SecurityFindingRead, SecurityFindingStatusUpdate
from app.schemas.test_case import TestCaseRead, TestCaseUpdate
from app.schemas.change_request import (
    ChangeRequestCreate,
    ChangeRequestDetail,
    ChangeRequestHistoryRead,
    ChangeRequestListItem,
    ChangeRequestListResponse,
    ChangeRequestRead,
    ChangeRequestUpdate,
    ChangeRequestUpdateResponse,
    ChangeRequestVersionDetail,
    ChangeRequestVersionRead,
    ReportableVersionRead,
    ChangeStatusRequest,
    FieldComparisonRead,
    LatestAnalysisRead,
    StatusTransitionOption,
    VersionCompareResponse,
)
from app.services import workflow_rules
from app.services.analysis_engine import AnalysisError, run_analysis
from app.services.report_generator import (
    ReportGenerationError,
    ReportVersionError,
    generate_report_pdf,
    resolve_report_context,
)
from app.services import report_registry
from app.schemas.report import ReportListItem, ReportListResponse
from app.services.versioning import apply_change_request_update, compare_snapshots

router = APIRouter(prefix="/api/change-requests", tags=["change-requests"])

# Same category vocabulary/normalization and risk buckets as the dashboard
# (app/api/dashboard.py) - kept as a separate copy rather than a shared
# import so this module can't accidentally change dashboard behavior.
CANONICAL_CATEGORIES = ["Feature", "Bug Fix", "Security", "Database", "API", "Infrastructure", "Integration"]

_CATEGORY_ALIASES = {
    "feature": "Feature",
    "new feature": "Feature",
    "enhancement": "Feature",
    "bug fix": "Bug Fix",
    "bugfix": "Bug Fix",
    "bug": "Bug Fix",
    "fix": "Bug Fix",
    "security": "Security",
    "database": "Database",
    "db": "Database",
    "api": "API",
    "infrastructure": "Infrastructure",
    "infra": "Infrastructure",
    "integration": "Integration",
}

# The status vocabulary the "Filter by status" dropdown offers. Most of
# these are derived, not stored - see effective_status() below. "analyzing"
# has no real backing yet (no AI pipeline exists to be "in progress" -
```



```

# that's a later module), so filtering by it will always return zero rows
# today; it's included so the filter is ready for when that module lands.
EFFECTIVE_STATUS_VALUES = {
    "draft",
    "pending_analysis",
    "analyzing",
    "requires_clarification",
    "completed",
    "approved",
}

def _normalize_category(raw: Optional[str]) -> str:
    if not raw:
        return "Other"
    return _CATEGORY_ALIASES.get(raw.strip().lower(), "Other")

def latest_analysis(change_request: ChangeRequest) -> Optional[Analysis]:
    if not change_request.analyses:
        return None
    return max(change_request.analyses, key=lambda a: a.created_at)

def load_approval_rules(db: Session) -> list[ApprovalRule]:
    """Module 21: every ApprovalRule row (enabled and disabled alike -
    workflow rules.required_approval_types itself skips disabled ones),
    loaded once per request and handed to that pure rule-logic function -
    same "caller owns the query" convention as every other permission
    helper in workflow_rules.py."""
    return db.query(ApprovalRule).all()

def _effective_status(change_request: ChangeRequest, latest: Optional[Analysis]) -> str:
    """Human-facing status: raw DB status plus what analysis state (if any)
    already exists. Never stored - computed fresh every time from real rows."""
    if change_request.status == ChangeRequestStatus.APPROVED:
        return "approved"
    if change_request.status == ChangeRequestStatus.DRAFT:
        return "draft"
    if latest is None:
        return "pending_analysis"
    if any(not question.resolved for question in latest.clarification_questions):
        return "requires_clarification"
    return "completed"

@router.post("", response_model=ChangeRequestRead, status_code=status.HTTP_201_CREATED)
def create_change_request(
    payload: ChangeRequestCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequest:
    check_capability(db, current_user, Capability.CR_CREATE)

    change_request = ChangeRequest(
        title=payload.title,
        description=payload.description,
        business_objective=payload.business_objective,
        priority=payload.priority,
        requested_by=payload.requested_by,
        target_system=payload.target_system,
        desired_deadline=payload.desired_deadline,
        business_impact=payload.business_impact,
        technical_impact=payload.technical_impact,
        customer_impact=payload.customer_impact,
        environment=payload.environment,
        dependencies_note=payload.dependencies_note,
        compliance_requirements=payload.compliance_requirements,
        tags=json.dumps(payload.tags) if payload.tags else None,
        status=ChangeRequestStatus.PENDING_ANALYSIS,
        created_by=current_user.id,
        current_version=1,
    )
    db.add(change_request)
    db.flush()

    db.add(
        ChangeRequestVersion(
            change_request_id=change_request.id,
            version_number=1,
            changed_by=current_user.id,
            change_summary="Change request created.",
            snapshot=build_snapshot(change_request),
        )
    )
    history_service.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.CREATED,
        user_id=current_user.id,
        version_number=1,
    )

    db.commit()
    db.refresh(change_request)
    return change_request

@router.get("", response_model=ChangeRequestListResponse)
def list_change_requests(
    search: Optional[str] = Query(
        None, description="Matches title, description, requested by, target system, or ticket number (e.g. '#3021')."
    ),
    status_filter: Optional[str] = Query(
        None,
        alias="status",
        description=f"One of: {'', '.join(sorted(EFFECTIVE_STATUS_VALUES))}."
    ),
    priority: Optional[Priority] = Query(None),
    assigned_to_me: bool = Query(
        False, description="Module 12 Phase 6: only change requests the current user is assigned to, in any role."
    ),
    sort: str = Query("newest", pattern="^(newest|oldest)$"),
    limit: int = Query(25, ge=1, le=200),
    offset: int = Query(0, ge=0),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestListResponse:
    if status_filter is not None and status_filter not in EFFECTIVE_STATUS_VALUES:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"status must be one of: {'', '.join(sorted(EFFECTIVE_STATUS_VALUES))}."
        )

    query = db.query(ChangeRequest).options(
        selectinload(ChangeRequest.analyses).selectinload(Analysis.clarification_questions)
    )

    if search:
        raw = search.strip()

```

```

pattern = f"%{raw}%"
filters = [
    ChangeRequest.title.ilike(pattern),
    ChangeRequest.description.ilike(pattern),
    ChangeRequest.requested_by.ilike(pattern),
    ChangeRequest.target_system.ilike(pattern),
]
# Ticket-number search: "#3021", "3021", or a partial number like
# "30" all match against the numeric id, in addition to the text
# fields above - a leading "#" (how the UI displays ids) is
# stripped before matching.
ticket_query = raw.lstrip("#").strip()
if ticket_query:
    filters.append(cast(ChangeRequest.id, String).ilike(f"%{ticket_query}%"))
query = query.filter(or_(*filters))

if priority is not None:
    query = query.filter(ChangeRequest.priority == priority)

if assigned to me:
    query = query.filter(
        ChangeRequest.assignments.any(ChangeRequestAssignment.user_id == current_user.id)
    )

query = query.order_by(
    ChangeRequest.created_at.desc() if sort == "newest" else ChangeRequest.created_at.asc()
)

rows: list[ChangeRequestListItem] = []
for change_request in query.all():
    latest = _latest_analysis(change_request)
    effective = _effective_status(change_request, latest)

    if status_filter is not None and effective != status_filter:
        continue

    rows.append(
        ChangeRequestListItem(
            id=change_request.id,
            title=change_request.title,
            priority=change_request.priority,
            status=change_request.status,
            effective_status=effective,
            category=normalize_category(latest.category) if latest else None,
            risk=risk_bucket(latest.risk_score) if latest else None,
            complexity=latest.complexity.value if latest else None,
            created_at=change_request.created_at,
            current_version=change_request.current_version or 1,
        )
    )

total = len(rows)
page = rows[offset : offset + limit]

return ChangeRequestListItemResponse(items=page, total=total, limit=limit, offset=offset)

@router.get("/{change_request_id}", response_model=ChangeRequestDetail)
def get_change_request(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestDetail:
    change_request = (
        db.query(ChangeRequest)
        .options(selectinload(ChangeRequest.analysis).selectinload(Analysis.clarification_questions))
        .filter(ChangeRequest.id == change_request_id)
        .first()
    )
    if change_request is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Change request not found.")

    return _build_detail(change_request)

def get_change_request_or_404(db: Session, change_request_id: int) -> ChangeRequest:
    change_request = db.get(ChangeRequest, change_request_id)
    if change_request is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Change request not found.")
    return change_request

def build_detail(change_request: ChangeRequest) -> ChangeRequestDetail:
    """Shared by every endpoint that returns a full ChangeRequestDetail
    (get, update, status change) so the shape can't drift between them."""
    latest = _latest_analysis(change_request)
    return ChangeRequestDetail(
        id=change_request.id,
        title=change_request.title,
        description=change_request.description,
        business_objective=change_request.business_objective,
        priority=change_request.priority,
        requested_by=change_request.requested_by,
        target_system=change_request.target_system,
        desired_deadline=change_request.desired_deadline,
        business_impact=change_request.business_impact,
        technical_impact=change_request.technical_impact,
        customer_impact=change_request.customer_impact,
        environment=change_request.environment,
        dependencies_note=change_request.dependencies_note,
        compliance_requirements=change_request.compliance_requirements,
        tags=change_request.tags,
        status=change_request.status,
        status_label=workflow_rules.STATUS_LABELS.get(change_request.status, change_request.status.value),
        effective_status=_effective_status(change_request, latest),
        created_by=change_request.created_by,
        created_at=change_request.created_at,
        updated_at=change_request.updated_at,
        current_version=change_request.current_version or 1,
        latest_analysis=LatestAnalysisRead.model_validate(latest) if latest else None,
        is_analysis_outdated=workflow_rules.is_analysis_outdated(change_request, latest),
        available_transitions=[
            StatusTransitionOption(
                status=s,
                label=workflow_rules.STATUS_LABELS.get(s, s.value),
                requires_reason=workflow_rules.reason_required(s),
            )
            for s in workflow_rules.available_transitions(change_request.status)
        ],
    )

def assignment_read(assignment: ChangeRequestAssignment) -> ChangeRequestAssignmentRead:
    return ChangeRequestAssignmentRead(
        id=assignment.id,
        user_id=assignment.user_id,
        user_name=assignment.user.name,
        user_email=assignment.user.email,
        role=assignment.role,
        role_label=workflow_rules.ROLE_LABELS.get(assignment.role, assignment.role.value),
        assigned_by=assignment.assigned_by,
    )

```

```

        assigned_by_name=assignment.assigned_by_user.name,
        assigned_at=assignment.assigned_at,
    )

def approval_read(db: Session, approval: Approval, change_request: ChangeRequest) -> ApprovalRead:
    # Module 13 Phase 4: the analysis this approval was actually requested
    # against (its own cr_version), not necessarily the CR's *current*
    # analysis - matches the architecture-lock rule that every artifact
    # stays tied to the CR version it was produced for.
    relevant_analysis = (
        db.query(Analysis)
        .filter(
            Analysis.change_request_id == change_request.id,
            Analysis.change_request_version == approval.cr_version,
        )
        .order_by(Analysis.created_at.desc())
        .first()
    )
    ai_recommendation_label = None
    override = False
    if relevant_analysis is not None:
        ai_recommendation_label = workflow_rules.RECOMMENDATION_LABELS.get(relevant_analysis.recommendation)
        if approval.status in workflow_rules.APPROVAL_RESPONSE_STATUSES:
            override = workflow_rules.overrides_ai_recommendation(relevant_analysis.recommendation, approval.status)

    return ApprovalRead(
        id=approval.id,
        change_request_id=approval.change_request_id,
        approval_type=approval.approval_type,
        approval_type_label=workflow_rules.APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value),
        status=approval.status,
        status_label=workflow_rules.APPROVAL_STATUS_LABELS.get(approval.status, approval.status.value),
        approver_id=approval.approver_id,
        approver_name=approval.approver.name,
        approver_email=approval.approver.email,
        requested_by=approval.requested_by,
        requested_by_name=approval.requested_by_user.name,
        requested_at=approval.requested_at,
        responded_at=approval.responded_at,
        comment=approval.comment,
        cr_version=approval.cr_version,
        due_date=approval.due_date,
        due_status=approvals_service.approval_due_status(approval),
        is_outdated=approvals_service.is_approval_outdated(change_request, approval),
        ai_recommendation=ai_recommendation_label,
        override_ai_recommendation=override,
    )

# --- Module 12: editing, versioning, audit history -----

@router.put("/{change_request_id}", response_model=ChangeRequestUpdateResponse)
def update_change_request(
    change_request_id: int,
    payload: ChangeRequestUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestUpdateResponse:
    """Full Change Request editing (spec section 2). Only fields actually
    present in the request body are touched (partial update) - see
    ChangeRequestUpdate's docstring. If anything changed, this creates a
    new version and field-level history events (app/services/versioning.py)
    rather than a vague "updated" log line; if nothing changed (identical
    values re-submitted), no version/history noise is created at all.
    """
    change_request = get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.CR_EDIT)

    if not workflow_rules.can_edit_change_request(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to edit this change request.",
        )

    update_data = payload.model_dump(exclude_unset=True)
    changes = apply_change_request_update(db, change_request, update_data, user=current_user)
    db.commit()
    db.refresh(change_request)

    detail = _build_detail(change_request)

    change_summaries = [
        FieldComparisonRead(
            field=c.field,
            label=c.label,
            old_value=c.old_value,
            new_value=c.new_value,
            status="removed" if c.new_value is None else ("added" if c.old_value is None else "changed"),
        )
        for c in changes
    ]

    return ChangeRequestUpdateResponse(
        change_request=detail,
        changes=change_summaries,
        new_version=change_request.current_version if changes else None,
    )

@router.get("/{change_request_id}/history", response_model=list[ChangeRequestHistoryRead])
def get_change_request_history(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ChangeRequestHistory]:
    """The append-only Activity/Audit timeline (spec section 17), newest
    first. """
    _get_change_request_or_404(db, change_request_id)
    return (
        db.query(ChangeRequestHistory)
        .filter(ChangeRequestHistory.change_request_id == change_request_id)
        .order_by(ChangeRequestHistory.created_at.desc(), ChangeRequestHistory.id.desc())
        .all()
    )

@router.get("/{change_request_id}/versions", response_model=list[ChangeRequestVersionRead])
def list_change_request_versions(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ChangeRequestVersion]:
    _get_change_request_or_404(db, change_request_id)
    return (
        db.query(ChangeRequestVersion)
        .filter(ChangeRequestVersion.change_request_id == change_request_id)
        .order_by(ChangeRequestVersion.version_number.desc())
        .all()
    )

```

```

@router.get("/{change_request_id}/versions/{version_number}", response_model=ChangeRequestVersionDetail)
def get_change_request_version(
    change_request_id: int,
    version_number: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestVersion:
    _get_change_request_or_404(db, change_request_id)
    version = (
        db.query(ChangeRequestVersion)
        .filter(
            ChangeRequestVersion.change_request_id == change_request_id,
            ChangeRequestVersion.version_number == version_number,
        )
        .first()
    )
    if version is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Version {version_number} not found.")
    return version

@router.get("/{change_request_id}/compare", response_model=VersionCompareResponse)
def compare_change_request_versions(
    change_request_id: int,
    from_version: int = Query(..., alias="from"),
    to_version: int = Query(..., alias="to"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> VersionCompareResponse:
    """Field-by-field diff between two versions (spec section 5)."""
    _get_change_request_or_404(db, change_request_id)

    def _get_version(number: int) -> ChangeRequestVersion:
        version = (
            db.query(ChangeRequestVersion)
            .filter(
                ChangeRequestVersion.change_request_id == change_request_id,
                ChangeRequestVersion.version_number == number,
            )
            .first()
        )
        if version is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Version {number} not found.")
        return version

    version_a = _get_version(from_version)
    version_b = _get_version(to_version)
    fields = compare_snapshots(version_a.snapshot, version_b.snapshot)

    return VersionCompareResponse(
        from_version=from_version,
        to_version=to_version,
        fields=[
            FieldComparisonRead(field=f.field, label=f.label, old_value=f.old_value, new_value=f.new_value, status=f.status)
            for f in fields
        ],
    )

# --- Module 12 Phase 3: status workflow + assignments -----

@router.put("/{change_request_id}/status", response_model=ChangeRequestDetail)
def change_status(
    change_request_id: int,
    payload: ChangeStatusRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestDetail:
    """Move a CR to a new status along the workflow lifecycle (spec section 8). Only a legal next step (app/services/workflow_rules.STATUS_TRANSITIONS) is accepted, and moving into a "something went wrong" status (Changes Requested / Rejected / Cancelled) requires a non-empty reason - both enforced here, not just suggested by the frontend."""
    change_request = _get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.STATUS_CHANGE)

    if not workflow_rules.can_manage_workflow(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to change this change request's status.",
        )

    if not workflow_rules.can_transition(change_request.status, payload.status):
        current_label = workflow_rules.STATUS_LABELS.get(change_request.status, change_request.status.value)
        target_label = workflow_rules.STATUS_LABELS.get(payload.status, payload.status.value)
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"Can't move from {current_label} to {target_label}.",
        )

    # Module 22 (Final Integration, Security & Quality) spec section 5:
    # "prevent approval bypass." Before this check, nothing here ever
    # looked at the Approval table at all - the CR's owner/Technical Lead
    # could tag someone for Security Approval, get no response, and still
    # mark the change request Approved. This only blocks on a still-open
    # ASK (PENDING) - it deliberately does NOT require that any approval
    # was ever requested in the first place, and does NOT keep blocking
    # forever over an old Rejected/Changes Requested approval (the normal
    # recovery path for those is requesting a fresh one, which
    # create_approval_request above already allows even while an old
    # resolved row for the same type/approver exists) - "AI recommends,
    # humans decide" (app/models/approval.py's own docstring) still means
    # a human can approve a CR with zero formal sign-offs on it, or after
    # weighing a past rejection and moving on; they just can't ignore
    # someone they're actively waiting to hear back from.
    if payload.status == ChangeRequestStatus.APPROVED:
        pending_count = (
            db.query(Approval)
            .filter(Approval.change_request_id == change_request_id, Approval.status == ApprovalStatus.PENDING)
            .count()
        )
        if pending_count:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=(
                    f"This change request still has {pending_count} pending approval"
                    f"{'s' if pending_count != 1 else ''} - resolve or cancel "
                    "them before marking it Approved."
                ),
            )

    reason = (payload.reason or "").strip() or None
    if workflow_rules.reason_required(payload.status) and not reason:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"A reason is required to move this change request to "
            f"{workflow_rules.STATUS_LABELS.get(payload.status, payload.status.value)}.",
        )

```

```

old_status = change_request.status
change_request.status = payload.status
old_label = workflow.rules.STATUS_LABELS.get(old_status, old_status.value)
new_label = workflow.rules.STATUS_LABELS.get(payload.status, payload.status.value)
history_service.record_event(
    db,
    change_request_id=change_request.id,
    action=HistoryAction.STATUS_CHANGED,
    user_id=current_user.id,
    old_value=old_label,
    new_value=new_label,
    reason=reason,
    version_number=change_request.current_version or 1,
)

# Module 12 Phase 5: tell everyone with a stake in this CR - the
# creator plus everyone currently assigned - except whoever just made
# the change themselves.
interested_ids = {change_request.created_by} | {a.user_id for a in change_request.assignments}
interested_ids.discard(current_user.id)
for user_id in interested_ids:
    notify_service.notify(
        db,
        user_id=user_id,
        type=NotificationType.STATUS_CHANGED,
        title=f"CR-{change_request.id} status changed",
        message=f"{current_user.name} moved \"{change_request.title}\" from {old_label} to {new_label}.",
        change_request_id=change_request.id,
    )

db.commit()
db.refresh(change_request)

return _build_detail(change_request)

@router.get("/{change_request_id}/assignments", response_model=list[ChangeRequestAssignmentRead])
def list_assignments(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ChangeRequestAssignmentRead]:
    """Who's responsible for what on this CR (spec section 9) - Requester,
    Owner, Technical Lead, Reviewer(s), Approver(s), Security Reviewer, QA
    Owner, Implementation Owner."""
    _get_change_request_or_404(db, change_request_id)
    rows = (
        db.query(ChangeRequestAssignment)
        .options(
            selectinload(ChangeRequestAssignment.user),
            selectinload(ChangeRequestAssignment.assigned_by_user),
        )
        .filter(ChangeRequestAssignment.change_request_id == change_request_id)
        .order_by(ChangeRequestAssignment.assigned_at)
        .all()
    )
    return [_assignment_read(a) for a in rows]

@router.post(
   ("/{change_request_id}/assignments",
    response_model=ChangeRequestAssignmentRead,
    status_code=status.HTTP_201_CREATED,
)
def create_assignment(
    change_request_id: int,
    payload: AssignmentCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ChangeRequestAssignmentRead:
    """Assign a user to a role on this CR. Only the CR's Owner/Technical
    Lead/creator/Admin can do this (workflow.rules.can_assign_users) - the
    same "CR Owner's job" permission as changing status. REVIEWER and
    APPROVER may have more than one person; assigning the same user to the
    same role twice is rejected as a no-op rather than silently duplicated."""
    change_request = _get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.ASSIGNMENT)

    if not workflow.rules.can_assign_users(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to assign users on this change request.",
        )

    target_user = db.get(User, payload.user_id)
    if target_user is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")

    existing = (
        db.query(ChangeRequestAssignment)
        .filter(
            ChangeRequestAssignment.change_request_id == change_request_id,
            ChangeRequestAssignment.user_id == payload.user_id,
            ChangeRequestAssignment.role == payload.role,
        )
        .first()
    )
    if existing is not None:
        role_label = workflow.rules.ROLE_LABELS.get(payload.role, payload.role.value)
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"{target_user.name} is already assigned as {role_label} on this change request.",
        )

    assignment = ChangeRequestAssignment(
        change_request_id=change_request_id,
        user_id=payload.user_id,
        role=payload.role,
        assigned_by=current_user.id,
    )
    db.add(assignment)
    db.flush()

    role_label = workflow.rules.ROLE_LABELS.get(payload.role, payload.role.value)
    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.ASSIGNED,
        user_id=current_user.id,
        new_value=f"{target_user.name} as {role_label}",
        version_number=change_request.current_version or 1,
    )
    if target_user.id != current_user.id:
        notify_service.notify(
            db,
            user_id=target_user.id,
            type=NotificationType.ASSIGNED,
            title=f"You were assigned to CR-{change_request.id}",
            message=f"{current_user.name} assigned you as {role_label} on \"{change_request.title}\".",
            change_request_id=change_request_id,
        )

```

```

    )
    db.commit()
    db.refresh(assignment)

    return _assignment_read(assignment)

@router.delete("/{change_request_id}/assignments/{assignment_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_assignment(
    change_request_id: int,
    assignment_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    change_request = _get_change_request_or_404(db, change_request_id)

    if not workflow_rules.can_assign_users(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to change assignments on this change request.",
        )

    assignment = (
        db.query(ChangeRequestAssignment)
        .options(selectinload(ChangeRequestAssignment.user))
        .filter(
            ChangeRequestAssignment.id == assignment_id,
            ChangeRequestAssignment.change_request_id == change_request_id,
        )
        .first()
    )
    if assignment is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Assignment not found.")

    role_label = workflow_rules.ROLE_LABELS.get(assignment.role, assignment.role.value)
    removed_user_name = assignment.user.name
    db.delete(assignment)
    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.UNASSIGNED,
        user_id=current_user.id,
        old_value=f"({removed_user_name}) as {role_label}",
        version_number=change_request.current_version or 1,
    )
    db.commit()

    return Response(status_code=status.HTTP_204_NO_CONTENT)

# --- Module 12 Phase 4: approvals -----

def _get_approval_or_404(db: Session, change_request_id: int, approval_id: int) -> Approval:
    approval = (
        db.query(Approval)
        .options(selectinload(Approval.approver), selectinload(Approval.requested_by_user))
        .filter(Approval.id == approval_id, Approval.change_request_id == change_request_id)
        .first()
    )
    if approval is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Approval not found.")
    return approval

@router.get("/{change_request_id}/approvals", response_model=list[ApprovalRead])
def list_approvals(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ApprovalRead]:
    """Every sign-off requested on this CR (spec section 14), oldest first -
    who was tagged, what kind, and where it stands."""
    change_request = _get_change_request_or_404(db, change_request_id)
    rows = (
        db.query(Approval)
        .options(selectinload(Approval.approver), selectinload(Approval.requested_by_user))
        .filter(Approval.change_request_id == change_request_id)
        .order_by(Approval.requested_at)
        .all()
    )
    return [_approval_read(db, a, change_request) for a in rows]

@router.get("/{change_request_id}/approvals/recommended", response_model=list[RecommendedApprovalType])
def recommended_approvals(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[RecommendedApprovalType]:
    """An AI-derived suggestion only - "AI recommends, humans decide"
    (spec section 14/29, app/services/workflow_rules.py::
    required_approval_types). Empty until the CR has been analyzed at
    least once; never creates an Approval row by itself.

    Module 22: also flags is outdated (see RecommendedApprovalType's own
    docstring) whenever this is computed from an analysis that's no
    longer current - the same is analysis outdated check the main CR
    detail view already uses for its own warning banner."""
    change_request = _get_change_request_or_404(db, change_request_id)
    latest = latest_analysis(change_request)
    if latest is None:
        return []
    outdated = workflow_rules.is_analysis_outdated(change_request, latest)
    types = workflow_rules.required_approval_types(change_request, latest, _load_approval_rules(db))
    return [
        RecommendedApprovalType(
            approval_type=t, label=workflow_rules.APPROVAL_TYPE_LABELS.get(t, t.value), is_outdated=outdated
        )
        for t in sorted(types, key=lambda t: t.value)
    ]

@router.post("/{change_request_id}/approvals", response_model=ApprovalRead, status_code=status.HTTP_201_CREATED)
def create_approval_request(
    change_request_id: int,
    payload: ApprovalRequestCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ApprovalRead:
    """Tag a real person for a specific kind of sign-off. Only the CR's
    Owner/Technical Lead/creator/Admin can do this
    (workflow_rules.can_request_approval) - the same "CR Owner's job"
    permission as changing status/assigning users. Requesting the same
    type from the same approver while one is already pending is rejected
    as a duplicate, same pattern as create assignment."""
    change_request = _get_change_request_or_404(db, change_request_id)

    if not workflow_rules.can_request_approval(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,

```

```

        detail="You don't have permission to request approvals on this change request.",
    )

    approver = db.get(User, payload.approver_user_id)
    if approver is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")

    existing = (
        db.query(Approval)
        .filter(
            Approval.change_request_id == change_request_id,
            Approval.approver_id == payload.approver_user_id,
            Approval.approval_type == payload.approval_type,
            Approval.status == ApprovalStatus.PENDING,
        )
        .first()
    )
    if existing is not None:
        type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(payload.approval_type, payload.approval_type.value)
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"{approver.name} already has a pending {type_label} approval request on this change request.",
        )

    approval = approvals_service.request_approval(
        db,
        change_request,
        approval_type=payload.approval_type,
        approver=approver,
        requested_by=current_user,
        comment=payload.comment,
        due_date=payload.due_date,
    )
    if approver.id != current_user.id:
        type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(payload.approval_type, payload.approval_type.value)
        notify_service.notify(
            db,
            user_id=approver.id,
            type=NotificationType.APPROVAL_REQUESTED,
            title=f"{type_label} approval requested on CR-{change_request.id}",
            message=f"{current_user.name} asked you for {type_label} approval on \"{change_request.title}\".",
            change_request_id=change_request_id,
        )
    db.commit()
    db.refresh(approval)

    return _approval_read(db, approval, change_request)

@router.post("/{change_request_id}/approvals/{approval_id}/respond", response_model=ApprovalRead)
def respond_to_approval_request(
    change_request_id: int,
    approval_id: int,
    payload: ApprovalRespond,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ApprovalRead:
    """Only the tagged approver may respond - never the CR's owner, an
    admin, or anyone else (app/models/approval.py's own docstring: "never
    by anyone else or by the AI"). A comment is required when rejecting or
    requesting changes, the same "explain what went wrong" rule as moving
    a CR's status."""
    change_request = _get_change_request_or_404(db, change_request_id)
    approval = _get_approval_or_404(db, change_request_id, approval_id)

    if approval.approver_id != current_user.id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Only the person tagged for this approval can respond to it.",
        )

    if approval.status != ApprovalStatus.PENDING:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="This approval has already been responded to.",
        )

    if payload.status not in workflow_rules.APPROVAL_RESPONSE_STATUSES:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="status must be one of: approved, rejected, changes_requested.",
        )

    # Module 21 spec section 3: Approval and Rejection are two separate,
    # independently-configurable permissions - approving is
    # Capability.APPROVAL, while rejecting OR requesting changes (both
    # "this isn't good to go yet, and here's why" outcomes) are
    # Capability.REJECTION.
    check_capability(
        db,
        current_user,
        Capability.APPROVAL if payload.status == ApprovalStatus.APPROVED else Capability.REJECTION,
    )

    if payload.status in workflow_rules.APPROVAL_REASON_REQUIRED and not payload.comment:
        target_label = workflow_rules.APPROVAL_STATUS_LABELS.get(payload.status, payload.status.value)
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"A comment is required to mark this approval as {target_label}.",
        )

    approvals_service.respond_to_approval(
        db,
        change_request,
        approval,
        new_status=payload.status,
        comment=payload.comment,
        responder=current_user,
    )

    type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
    response_notification_type = {
        ApprovalStatus.APPROVED: NotificationType.APPROVED,
        ApprovalStatus.REJECTED: NotificationType.REJECTED,
        ApprovalStatus.CHANGES_REQUESTED: NotificationType.CHANGES_REQUESTED,
    }[payload.status]
    if approval.requested_by != current_user.id:
        status_label = workflow_rules.APPROVAL_STATUS_LABELS.get(payload.status, payload.status.value)
        notify_service.notify(
            db,
            user_id=approval.requested_by,
            type=response_notification_type,
            title=f"{type_label} approval {status_label.lower()} on CR-{change_request.id}",
            message=f"{current_user.name} marked the {type_label} approval you requested as {status_label}.",
            change_request_id=change_request_id,
        )
    db.commit()
    db.refresh(approval)

    return _approval_read(db, approval, change_request)

```



```

@router.delete("/{change_request_id}/approvals/{approval_id}", status_code=status.HTTP_204_NO_CONTENT)
def cancel_approval_request(
    change_request_id: int,
    approval_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    """Withdraw a still-pending approval request (e.g. tagged the wrong
    person) - same permission as requesting one. Soft-cancelled (status ->
    Cancelled) rather than deleted, so it stays on the audit trail."""
    change_request = get_change_request_or_404(db, change_request_id)
    approval = get_approval_or_404(db, change_request_id, approval_id)

    if not workflow_rules.can_request_approval(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to cancel approval requests on this change request.",
        )

    if approval.status != ApprovalStatus.PENDING:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Only a pending approval request can be cancelled.",
        )

    approvals_service.cancel_approval(db, change_request, approval, cancelled_by=current_user)
    db.commit()

    return Response(status_code=status.HTTP_204_NO_CONTENT)

@router.post("/{change_request_id}/approvals/{approval_id}/remind", status_code=status.HTTP_204_NO_CONTENT)
def remind_approval(
    change_request_id: int,
    approval_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    """A manual nudge for an approver who hasn't responded yet - same
    permission as requesting the approval in the first place. Doesn't
    change anything about the approval itself, just sends a fresh
    REMINDER notification and logs REMINDER SENT on the audit trail."""
    change_request = get_change_request_or_404(db, change_request_id)
    approval = get_approval_or_404(db, change_request_id, approval_id)

    if not workflow_rules.can_request_approval(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to send reminders on this change request.",
        )

    if approval.status != ApprovalStatus.PENDING:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Only a pending approval request can be reminded.",
        )

    type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.REMINDER_SENT,
        user_id=current_user.id,
        new_value=f"Reminder sent to {approval.approver.name} for {type_label} approval",
        version_number=change_request.current_version or 1,
    )
    notify_service.notify(
        db,
        user_id=approval.approver_id,
        type=NotificationType.REMINDER,
        title=f"Reminder: {type_label} approval requested on CR-{change_request_id}",
        message=f"{current_user.name} is still waiting on your {type_label} approval for \"{change_request.title}\".",
        change_request_id=change_request_id,
    )
    db.commit()

    return Response(status_code=status.HTTP_204_NO_CONTENT)

# --- Module 12 Phase 5: comments -----

def _comment_read(comment: ChangeRequestComment, all_users: list[User]) -> CommentRead:
    mentioned = extract_mentioned_users(comment.body, all_users, exclude_user_id=comment.user_id)
    return CommentRead(
        id=comment.id,
        change_request_id=comment.change_request_id,
        user_id=comment.user_id,
        user_name=comment.user.name,
        parent_id=comment.parent_id,
        body=comment.body,
        resolved=comment.resolved,
        created_at=comment.created_at,
        mentioned_user_ids=[u.id for u in mentioned],
    )

@router.get("/{change_request_id}/comments", response_model=list[CommentRead])
def list_comments(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[CommentRead]:
    """The discussion thread on this CR, oldest first - a flat two-level
    thread (top-level comments plus their direct replies), not a nested
    tree (spec: "reply where practical")."""
    _get_change_request_or_404(db, change_request_id)
    rows = (
        db.query(ChangeRequestComment)
        .options(selectinload(ChangeRequestComment.user))
        .filter(ChangeRequestComment.change_request_id == change_request_id)
        .order_by(ChangeRequestComment.created_at)
        .all()
    )
    all_users = db.query(User).all()
    return [_comment_read(c, all_users) for c in rows]

@router.post("/{change_request_id}/comments", response_model=CommentRead, status_code=status.HTTP_201_CREATED)
def create_comment(
    change_request_id: int,
    payload: CommentCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> CommentRead:
    """Post a comment (optionally a reply to one existing top-level
    comment). Open to anyone who can see this CR - unlike editing/status/
    approvals, commenting isn't gated to the Owner/Technical Lead/Admin,
    it's how everyone else participates. @mentions in the text and who
    else gets notified are handled by app/services/comments.py."""

```



```

change_request = _get_change_request_or_404(db, change_request_id)
check_capability(db, current_user, Capability.COMMENTS)

if payload.parent_id is not None:
    parent = (
        db.query(ChangeRequestComment)
        .filter(
            ChangeRequestComment.id == payload.parent_id,
            ChangeRequestComment.change_request_id == change_request_id,
        )
        .first()
    )
    if parent is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Comment being replied to not found.")
    if parent.parent_id is not None:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="Can't reply to a reply - this thread only goes one level deep.",
        )

comment = comments_service.create_comment(
    db,
    change_request,
    author=current_user,
    body=payload.body,
    parent_id=payload.parent_id,
)
db.commit()
db.refresh(comment)

all_users = db.query(User).all()
return _comment_read(comment, all_users)

@router.put("/{change_request_id}/comments/{comment_id}/resolve", response_model=CommentRead)
def resolve_comment(
    change_request_id: int,
    comment_id: int,
    payload: CommentResolve,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> CommentRead:
    """Mark a comment resolved (or reopen it) - either whoever posted it,
    or whoever manages this CR's workflow, can do this."""
    change_request = _get_change_request_or_404(db, change_request_id)
    comment = (
        db.query(ChangeRequestComment)
        .options(selectinload(ChangeRequestComment.user))
        .filter(ChangeRequestComment.id == comment_id, ChangeRequestComment.change_request_id == change_request_id)
        .first()
    )
    if comment is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Comment not found.")

    is_author = comment.user_id == current_user.id
    if not is_author and not workflow_rules.can_manage_workflow(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to resolve this comment.",
        )

    comment.resolved = payload.resolved
    db.commit()
    db.refresh(comment)

    all_users = db.query(User).all()
    return _comment_read(comment, all_users)

@router.delete("/{change_request_id}/comments/{comment_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_comment(
    change_request_id: int,
    comment_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    """Only the comment's own author (or an Admin) can delete it."""
    change_request = _get_change_request_or_404(db, change_request_id)
    comment = (
        db.query(ChangeRequestComment)
        .filter(ChangeRequestComment.id == comment_id, ChangeRequestComment.change_request_id == change_request_id)
        .first()
    )
    if comment is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Comment not found.")

    if comment.user_id != current_user.id and not workflow_rules.is_admin(current_user):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You can only delete your own comments.",
        )

    db.delete(comment)
    db.commit()

    return Response(status_code=status.HTTP_204_NO_CONTENT)

# --- Module 6: AI Analysis Engine -----
#
# The only two routes that call the AI. Everything above this stays exactly
# as Module 5 left it - list/detail keep reading whatever analyses already
# exist, unaware of how they got there.

_ANALYSIS_RELATIONSHIPS = (
    selectinload(Analysis.requirements),
    selectinload(Analysis.affected_components),
    selectinload(Analysis.dependencies),
    selectinload(Analysis.risks),
    # Module 14 Phase 4/5 - added here for the same eager-loading reason as
    # every relationship above (avoids an N+1 query per analysis fetched);
    # functionally these were already reachable via SQLAlchemy's default
    # lazy-load, just not eager-loaded until now.
    selectinload(Analysis.impact_assessments),
    selectinload(Analysis.security_findings),
    selectinload(Analysis.clarification_questions),
    selectinload(Analysis.test_cases),
    selectinload(Analysis.implementation_tasks),
    # Module 16 (Project Knowledge Base & RAG) Phase 4/5.
    selectinload(Analysis.knowledge_evidence),
)

def _repository_findings_for_analysis(db: Session, analysis_id: int) -> List[RepositoryFinding]:
    """Module 15 Phase 5 (Repository Intelligence - Integration): whatever
    repository findings already exist for this exact analysis - eager-
    loading each finding's own indexed file since
    RepositoryFinding.file_path (used by repository linkage.related_files)
    reads it via that relationship. Empty (never 404) when no repository
    scan/match has ever been run for this analysis - the same "empty is a

```

```

valid, honest outcome" rule the matcher and the outdated-check follow."""
return (
    db.query(RepositoryFinding)
    .options(selectinload(RepositoryFinding.indexed_file))
    .filter(RepositoryFinding.analysis_id == analysis_id)
    .all()
)

@router.post("/{change_request_id}/analyze", response_model=AnalysisRead, status_code=status.HTTP_201_CREATED)
def analyze_change_request(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> AnalysisRead:
    """Run the AI Analysis Engine on a change request and persist the
    result. A request can be analyzed more than once (e.g. after edits) -
    each call adds a new Analysis row rather than overwriting the last one,
    and the list/detail views already show whichever is newest."""
    change_request = _get_change_request_or_404(db, change_request_id)

    # Module 13 Phase 3: captured *before* run_analysis() so we still have
    # whatever was previously "the latest analysis" once a new one exists -
    # used below to detect a materially significant re-analysis.
    previous_analysis = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .first()
    )

    try:
        analysis = run_analysis(db, change_request)
    except AnalysisError as exc:
        # Module 19 (Engineering Change Analytics): the one place a failed
        # analysis attempt gets recorded at all - same actor_label="AI
        # Analyzer" / no user_id convention as a successful "AI
        # ANALYSIS_COMPLETED" event, so the new analytics dashboard's "AI
        # analysis failures" count is real data, not zero-by-omission.
        # Explicit commit here (not just flush) since we're about to raise -
        # get db's session.close() would otherwise roll this back.
        history_service.record_event(
            db,
            change_request_id=change_request_id,
            action=HistoryAction.AI_ANALYSIS_FAILED,
            actor_label="AI Analyzer",
            new_value=str(exc),
            version_number=change_request.current_version or 1,
        )
        db.commit()
        raise HTTPException(status_code=exc.status_code, detail=str(exc)) from exc

    # Re-fetch with every relationship eager-loaded so the response includes
    # the full set of requirements/risks/etc. in one query.
    analysis = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.id == analysis.id)
        .one()
    )

    # Module 24 (Reports): a report is never generated as a separate,
    # manual step - the moment an analysis completes for the CR's current
    # version, that's automatically a REPORT_GENERATED event too, so it
    # shows up on the Reports page immediately with no extra click. This is
    # the exact same event/aggregation mechanism app/services/
    # report_registry.py already reads for every other report source (the
    # Analysis Dashboard's own "Download Report" button) - just triggered
    # here instead of on a download/generate action. Always the CR's
    # current, non-historical version, since that's what was just analyzed.
    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.REPORT_GENERATED,
        user_id=current_user.id,
        new_value=f"Version {change_request.current_version or 1}",
        version_number=change_request.current_version or 1,
    )
    db.commit()

    # Module 13 Phase 3: re-analyzing a CR that already had an analysis, and
    # landing somewhere materially different (risk bucket, complexity,
    # recommendation, or recommended approvals all changed) is worth
    # telling people about, the same way editing a CR already notifies
    # people that its old analysis went stale (see the ANALYSIS_OUTDATED
    # block in app/services/versioning.py). Reuses Phase 2's computed diff
    # rather than inventing a second "did this change materially" check.
    interested_ids = {change_request.created_by} | {a.user_id for a in change_request.assignments}
    interested_ids.discard(current_user.id)
    notified_anyone = False

    if previous_analysis is None:
        # Module 18 Phase 2 (Notifications, My Work & Personal Engineering
        # Queue): the very first analysis this change request has ever
        # had - distinct from ANALYSIS_SIGNIFICANTLY_CHANGED/RISK_ESCALATED
        # below, both of which only ever fire on a *re*-analysis (there's
        # nothing to compare a first analysis against). Never fires again
        # for the same CR once it has a previous analysis to compare to.
        for user_id in interested_ids:
            notify_service.notify(
                db,
                user_id=user_id,
                type=NotificationType.ANALYSIS_COMPLETED,
                title=f"CR-{change_request.id} has been analyzed",
                message=f"{current_user.name} ran the AI analysis on \"{change_request.title}\".",
                change_request_id=change_request_id,
            )
            notified_anyone = True
    else:
        delta = compare_analyses(change_request, previous_analysis, analysis, _load_approval_rules(db))
        if delta.is_significant_change:
            reasons_text = " ".join(delta.significant_change_reasons)
            for user_id in interested_ids:
                notify_service.notify(
                    db,
                    user_id=user_id,
                    type=NotificationType.ANALYSIS_SIGNIFICANTLY_CHANGED,
                    title=f"CR-{change_request.id}'s re-analysis changed materially",
                    message=f"{current_user.name} re-analyzed \"{change_request.title}\" - {reasons_text}",
                    change_request_id=change_request_id,
                )
                notified_anyone = True

    # Module 18 Phase 2: risk escalation gets its own, more urgent
    # notification, separate from "something changed" above - fires
    # only when the risk bucket actually got WORSE (never on an
    # improvement, and never just because *something* changed - a
    # complexity or recommendation change alone doesn't escalate risk).
    # Can fire alongside ANALYSIS_SIGNIFICANTLY_CHANGED above (a risk
    # bucket move is itself one of that notification's own reasons) -

```

```

# that's deliberate, not a duplicate: one says "the numbers moved",
# the other says specifically "and it's now riskier."
if workflow_rules.risk_bucket_rank(delta.risk_bucket_after) > workflow_rules.risk_bucket_rank(
    delta.risk_bucket_before
):
    for user_id in interested_ids:
        notify_service.notify(
            db,
            user_id=user_id,
            type=NotificationType.RISK_ESCALATED,
            title=f"CR-{change_request.id}'s risk escalated to {delta.risk_bucket_after.capitalize()}",
            message=(
                f"{current_user.name}'s re-analysis of \"{change_request.title}\" moved risk from "
                f"{delta.risk_bucket_before.capitalize()} to {delta.risk_bucket_after.capitalize()}."
            ),
            change_request_id=change_request.id,
        )
        notified_anyone = True

if notified_anyone:
    db.commit()

response = AnalysisRead.model_validate(analysis)
response.is_outdated = workflow_rules.is_analysis_outdated(change_request, analysis)
# Module 16 (Project Knowledge Base & RAG) Phase 4/5: each piece of
# evidence was retrieved as part of THIS analysis run, so it shares
# this analysis's own outdated status exactly - see
# KnowledgeEvidenceRead's own docstring for why there's no separate
# per-evidence staleness computation.
for evidence in response.knowledge_evidence:
    evidence.is_outdated = response.is_outdated
# Module 17 Phase 1 (Test Cases & Implementation Plan 2.0): every test
# case/task belongs to exactly this one analysis, so both share its
# version and its outdated status exactly - same "no separate
# staleness computation" reasoning as knowledge_evidence above.
for test_case in response.test_cases:
    test_case.analysis_version = analysis.change_request_version
    test_case.is_outdated = response.is_outdated
for task in response.implementation_tasks:
    task.analysis_version = analysis.change_request_version
    task.is_outdated = response.is_outdated
# Module 13 Phase 4
response.workflow_recommendation = workflow_rules.recommendation_summary(change_request, analysis, _load_approval_rules(db))
# Module 15 Phase 5: a brand-new analysis has no repository findings of
# its own yet (matching runs separately, after analysis, against this
# exact analysis id) - so this is a no-op right after /analyze, and
# becomes meaningful once POST .../repository-findings has run for this
# analysis. Still called unconditionally, same as everywhere else in
# this app that computes something fresh rather than special-casing
# "nothing to compute yet."
repository_linkage.annotate_analysis_response(
    response, _repository_findings_for_analysis(db, analysis.id)
)
# Module 17 Phase 4: which Requirement(s)/Risk(s) each test case/task
# is traceable to - computed the same keyword-overlap way
# repository linkage above is, using this same response's own
# requirements/risks (never a separate query - see this module's own
# docstring for why an artifact can only ever trace to its own
# analysis's rows).
traceability_linkage.annotate_analysis_response(response)
return response

@router.get("/{change_request_id}/analysis", response_model=AnalysisRead)
def get_latest_analysis(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> AnalysisRead:
    """The most recent analysis for a change request, with every child
    table (requirements, affected components, dependencies, risks,
    clarification questions, test cases, implementation tasks) included."""
    change_request = _get_change_request_or_404(db, change_request_id)

    analysis = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .first()
    )
    if analysis is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="This change request hasn't been analyzed yet.",
        )
    response = AnalysisRead.model_validate(analysis)
    # Module 13 Phase 3: this is always "the latest analysis" for this CR
    # (never a historical one from the /analyses or /analysis/compare
    # endpoints), so "outdated" has one unambiguous meaning here: the CR
    # has been edited since this analysis ran.
    response.is_outdated = workflow_rules.is_analysis_outdated(change_request, analysis)
    # Module 16 Phase 4/5: see the identical loop in analyze_change_request
    # above.
    for evidence in response.knowledge_evidence:
        evidence.is_outdated = response.is_outdated
    # Module 17 Phase 1 (Test Cases & Implementation Plan 2.0): every test
    # case/task belongs to exactly this one analysis, so both share its
    # version and its outdated status exactly - same "no separate
    # staleness computation" reasoning as knowledge_evidence above.
    for test_case in response.test_cases:
        test_case.analysis_version = analysis.change_request_version
        test_case.is_outdated = response.is_outdated
    for task in response.implementation_tasks:
        task.analysis_version = analysis.change_request_version
        task.is_outdated = response.is_outdated
    # Module 13 Phase 4.
    response.workflow_recommendation = workflow_rules.recommendation_summary(change_request, analysis, _load_approval_rules(db))
    # Module 15 Phase 5: see the identical call in analyze_change_request
    # above for why this is unconditional.
    repository_linkage.annotate_analysis_response(
        response, _repository_findings_for_analysis(db, analysis.id)
    )
    # Module 17 Phase 4: which Requirement(s)/Risk(s) each test case/task
    # is traceable to - computed the same keyword-overlap way
    # repository linkage above is, using this same response's own
    # requirements/risks (never a separate query - see this module's own
    # docstring for why an artifact can only ever trace to its own
    # analysis's rows).
    traceability_linkage.annotate_analysis_response(response)
    return response

# --- Module 15 Phase 3: CR -> File Matching (Repository Intelligence) ---
#
# "Which source files may actually be affected by this change request?" -
# this module's own headline differentiator. Matches the CR's latest
# analysis against the most recent repository scan; see
# app/services/repository_matcher.py for how the shortlist/AI-call/
# persistence actually works. Deliberately not permission-gated beyond
# plain authentication, same precedent as /analyze itself (generating an

```

```

# AI-derived finding isn't a workflow action the way editing/approving/
# assigning are).

def _get_latest_analysis_or_404(db: Session, change_request_id: int) -> Analysis:
    analysis = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .first()
    )
    if analysis is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="This change request hasn't been analyzed yet.",
        )
    return analysis

def _get_latest_repository_scan_or_404(db: Session) -> RepositoryScan:
    scan = db.query(RepositoryScan).order_by(RepositoryScan.started_at.desc()).first()
    if scan is None:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="No repository scan has been run yet - trigger POST /api/repository/scan first.",
        )
    return scan

@router.post(
    "/{change_request_id}/repository-findings",
    response_model=list[RepositoryFindingRead],
    status_code=status.HTTP_201_CREATED,
)
def trigger_repository_match(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[RepositoryFindingRead]:
    change_request = _get_change_request_or_404(db, change_request_id)
    analysis = _get_latest_analysis_or_404(db, change_request_id)
    scan = _get_latest_repository_scan_or_404(db)

    try:
        findings = run_repository_match(db, change_request, analysis, scan)
    except RepositoryMatchError as exc:
        raise HTTPException(status_code=exc.status_code, detail=str(exc)) from exc

    # Module 15 Phase 4: computed fresh here, same as everywhere else this
    # app computes "is this still current" - practically always False
    # immediately after a fresh match (this analysis/scan pair IS "the
    # latest" the moment these rows were written), but still run through
    # the shared check rather than hardcoded, for the same reason
    # AnalysisRead.is_outdated is recomputed even right after a fresh
    # analyze call.
    return [
        finding_with_outdated_info(finding, change_request, latest_analysis_id=analysis.id, latest_scan_id=scan.id)
        for finding in findings
    ]

def finding_with_outdated_info(
    finding: RepositoryFinding,
    change_request: ChangeRequest,
    *,
    latest_analysis_id: Optional[int],
    latest_scan_id: Optional[int],
) -> RepositoryFindingRead:
    item = RepositoryFindingRead.model_validate(finding)
    item.outdated_reasons = workflow_rules.repository_finding_outdated_reasons(
        change_request,
        finding.change_request_version,
        finding.analysis_id,
        finding.repository_scan_id,
        latest_analysis_id=latest_analysis_id,
        latest_repository_scan_id=latest_scan_id,
    )
    item.is_outdated = bool(item.outdated_reasons)
    return item

@router.get("/{change_request_id}/repository-findings", response_model=list[RepositoryFindingRead])
def list_repository_findings(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[RepositoryFindingRead]:
    """Whatever was found the last time a match was run for this change
    request's *current* latest analysis - empty (not 404) if none has
    been run yet, the same "an empty result is a valid, honest answer"
    rule the matcher itself follows. Module 15 Phase 4: each finding also
    reports whether it may be outdated - the change request was edited, a
    newer analysis ran, or a newer repository scan is available - since
    the last time this exact match was generated."""
    change_request = _get_change_request_or_404(db, change_request_id)
    analysis = (
        db.query(Analysis)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .first()
    )
    if analysis is None:
        return []

    findings = (
        db.query(RepositoryFinding)
        .filter(
            RepositoryFinding.change_request_id == change_request.id,
            RepositoryFinding.analysis_id == analysis.id,
        )
        .order_by(RepositoryFinding.confidence.desc())
        .all()
    )

    latest_scan = db.query(RepositoryScan).order_by(RepositoryScan.started_at.desc()).first()
    latest_scan_id = latest_scan.id if latest_scan is not None else None

    return [
        finding_with_outdated_info(
            finding, change_request, latest_analysis_id=analysis.id, latest_scan_id=latest_scan_id
        )
        for finding in findings
    ]

# --- Module 14 Phase 6: human review of AI findings -----
#
# Two narrow, explicitly-scoped actions - marking a Requirement's review
# status and moving a Security finding's status forward - never a general
# "edit the AI's analysis" capability. Neither endpoint below touches the

```

```

# AI's own certainty/confidence/evidence/finding/severity columns; both
# record the change on the change request's audit history (architecture
# lock: "AI may recommend but never perform human approval" - these
# endpoints are exactly the human half of that split).

def get_requirement_or_404(db: Session, change_request_id: int, requirement_id: int) -> Requirement:
    requirement = (
        db.query(Requirement)
        .join(Analysis, Requirement.analysis_id == Analysis.id)
        .filter(Requirement.id == requirement_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if requirement is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Requirement not found on this change request.",
        )
    return requirement

def get_security_finding_or_404(db: Session, change_request_id: int, finding_id: int) -> SecurityFinding:
    finding = (
        db.query(SecurityFinding)
        .join(Analysis, SecurityFinding.analysis_id == Analysis.id)
        .filter(SecurityFinding.id == finding_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if finding is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Security finding not found on this change request.",
        )
    return finding

@router.patch("/{change_request_id}/requirements/{requirement_id}/review", response_model=RequirementRead)
def review_requirement(
    change_request_id: int,
    requirement_id: int,
    payload: RequirementReviewUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> RequirementRead:
    """An authorized human's verdict on this specific AI-extracted
    requirement - Confirmed or Needs Clarification. Never touches this
    row's certainty/confidence/evidence (those stay exactly what the AI
    produced); this is a separate, additive record of what a human then
    did about it. A comment is required for Needs Clarification (same
    "explain what needs fixing" rule ApprovalRespond applies to its own
    negative outcomes) and optional for Confirmed."""
    change_request = get_change_request_or_404(db, change_request_id)
    requirement = get_requirement_or_404(db, change_request_id, requirement_id)

    if not workflow_rules.can_review_analysis_findings(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to review requirements on this change request.",
        )

    if payload.review_status == ReviewStatus.NEEDS_CLARIFICATION and not payload.comment:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="A comment is required to mark a requirement as Needs Clarification.",
        )

    old_value = requirement.review_status.value if requirement.review_status else "unreviewed"

    requirement.review_status = payload.review_status
    requirement.review_comment = payload.comment
    requirement.reviewed_by = current_user.id
    requirement.reviewed_at = datetime.utcnow()

    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.REQUIREMENT_REVIEWED,
        user_id=current_user.id,
        field_name=f"Requirement #{requirement_id}",
        old_value=old_value,
        new_value=payload.review_status.value,
        reason=payload.comment,
        version_number=change_request.current_version or 1,
    )
    db.commit()
    db.refresh(requirement)
    return requirement

def get_clarification_question_or_404(
    db: Session, change_request_id: int, question_id: int
) -> ClarificationQuestion:
    question = (
        db.query(ClarificationQuestion)
        .join(Analysis, ClarificationQuestion.analysis_id == Analysis.id)
        .filter(ClarificationQuestion.id == question_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if question is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Clarification question not found on this change request.",
        )
    return question

@router.patch(
   ("/{change_request_id}/clarification-questions/{question_id}/answer",
    response_model=ClarificationQuestionRead,
)
def answer_clarification_question(
    change_request_id: int,
    question_id: int,
    payload: ClarificationQuestionAnswer,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ClarificationQuestion:
    """Supplies the missing information the AI itself asked for, right on
    the Missing Information tab - open to the same people who can comment
    on this change request (Capability.COMMENTS), not gated to a reviewer
    role, since answering is usually about knowing the answer (often the
    requester), not passing judgment on the analysis. Always marks the
    question resolved: an answer with no explanation would be
    indistinguishable from the question just having been ignored, and
    'resolved' is what the Dashboard's "Requires Clarification" tile and
    this CR's own effective status ( effective status above) already key
    off of - so this is genuinely the first thing that can ever set it True.
    Can be called again on an already-answered question to correct it."""
    change_request = get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.COMMENTS)
    question = get_clarification_question_or_404(db, change_request_id, question_id)

```

```

question.answer.text = payload.answer
question.answered_by = current.user.id
question.answered_at = datetime.utcnow()
question.resolved = True

history_service.record_event(
    db,
    change_request_id=change_request.id,
    action=HistoryAction.CLARIFICATION_ANSWERED,
    user_id=current.user.id,
    field_name=question.question,
    new_value=payload.answer,
    version_number=change_request.current_version or 1,
)
db.commit()
db.refresh(question)
return question

@router.patch("/{change_request_id}/security-findings/{finding_id}/status", response_model=SecurityFindingRead)
def update_security_finding_status(
    change_request_id: int,
    finding_id: int,
    payload: SecurityFindingStatusUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> SecurityFindingRead:
    """Moves a Security finding's Status - typically forward (Open ->
    Acknowledged -> Resolved), but a human may also reopen or correct one.
    The AI itself can never call this or set Acknowledged/Resolved on its
    own (see SecurityFindingItem.normalize_status) - this endpoint is the
    only place those two values are ever set."""
    change_request = get_change_request_or_404(db, change_request_id)
    finding = get_security_finding_or_404(db, change_request_id, finding_id)

    if not workflow_rules.can_review_analysis_findings(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to update security findings on this change request.",
        )

    old_value = finding.status.value

    finding.status = payload.status
    finding.review_comment = payload.comment
    finding.reviewed_by = current.user.id
    finding.reviewed_at = datetime.utcnow()

    history_service.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.SECURITY_FINDING_STATUS_CHANGED,
        user_id=current.user.id,
        field_name=f"Security Finding #{finding.id} ({finding.category.value})",
        old_value=old_value,
        new_value=payload.status.value,
        reason=payload.comment,
        version_number=change_request.current_version or 1,
    )
    db.commit()
    db.refresh(finding)
    return finding

# --- Module 17 Phase 6: human editing of AI-generated test cases and ---
# --- implementation tasks -----

def get_test_case_or_404(db: Session, change_request_id: int, test_case_id: int) -> TestCase:
    test_case = (
        db.query(TestCase)
        .join(Analysis, TestCase.analysis_id == Analysis.id)
        .filter(TestCase.id == test_case_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if test_case is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Test case not found on this change request.",
        )
    return test_case

def get_implementation_task_or_404(db: Session, change_request_id: int, task_id: int) -> ImplementationTask:
    task = (
        db.query(ImplementationTask)
        .join(Analysis, ImplementationTask.analysis_id == Analysis.id)
        .filter(ImplementationTask.id == task_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if task is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Implementation task not found on this change request.",
        )
    return task

def full_analysis_read(db: Session, change_request: ChangeRequest, analysis: Analysis) -> AnalysisRead:
    """Builds the exact same fully-computed AnalysisRead response
    GET .../analysis returns (analysis_version/is_outdated on every test
    case/task, workflow_recommendation, repository related files,
    traceability requirement/risk references) - shared by the edit
    endpoints below so an edited row's response carries those same
    computed fields rather than the schema defaults an edit-only code path
    would otherwise leave them at. Mirrors analyze_change_request/
    get_latest_analysis's own identical block above field-for-field on
    purpose - one more reason those computed fields must never be trusted
    from anywhere but this one shared computation."""
    response = AnalysisRead.model_validate(analysis)
    response.is_outdated = workflow_rules.is_analysis_outdated(change_request, analysis)
    for evidence in response.knowledge.evidence:
        evidence.is_outdated = response.is_outdated
    for test_case in response.test_cases:
        test_case.analysis_version = analysis.change_request_version
        test_case.is_outdated = response.is_outdated
    for task in response.implementation_tasks:
        task.analysis_version = analysis.change_request_version
        task.is_outdated = response.is_outdated
    response.workflow_recommendation = workflow_rules.recommendation_summary(change_request, analysis, _load_approval_rules(db))
    repository_linkage.annotate_analysis_response(
        response, _repository_findings_for_analysis(db, analysis.id)
    )
    traceability_linkage.annotate_analysis_response(response)
    return response

def apply_partial_update(row, updates: dict) -> list[tuple[str, object, object]]:
    """Sets only the fields actually present in 'updates' (already filtered
    to exclude_unset by the caller) onto 'row', returning (field, old, new)

```

```

for every field whose value actually changed - a no-op field (the human
resubmitted the same value) is never reported as a change, so it never
shows up in the audit trail or resets edited by/edited_at for nothing."""
changed: list[tuple[str, object, object]] = []
for field, new_value in updates.items():
    old_value = getattr(row, field)
    if old_value != new_value:
        changed.append((field, old_value, new_value))
    setattr(row, field, new_value)
return changed

def _stringify_field_value(value) -> str:
    if value is None:
        return "(none)"
    if isinstance(value, list):
        return " ".join(str(item) for item in value) or "(none)"
    if hasattr(value, "value"): # an enum member (Priority, TestType, ...)
        return str(value.value)
    return str(value)

def _change_summary(changed: list[tuple[str, object, object]]) -> tuple[str, str]:
    """(old value, new value) strings for the history row - a single
    changed field reads as just that field's own old/new value; several
    changed fields at once are each labeled with their own field name so
    nothing is ambiguous, mirroring apply_change_request update's own
    "list every field that changed" idea (see versioning.py) just
    formatted per-field rather than as prose."""
    if len(changed) == 1:
        field, old_value, new_value = changed[0]
        return _stringify_field_value(old_value), _stringify_field_value(new_value)
    old_parts = [f"{field}: { _stringify_field_value(old_value)}" for field, old_value, _new_value in changed]
    new_parts = [f"{field}: { _stringify_field_value(new_value)}" for field, _old_value, new_value in changed]
    return " ".join(old_parts), " ".join(new_parts)

@router.patch("/{change_request_id}/test-cases/{test_case_id}", response_model=TestCaseRead)
def update_test_case(
    change_request_id: int,
    test_case_id: int,
    payload: TestCaseUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> TestCaseRead:
    """Lets an authorized human correct/refine one AI-generated test case
    in place - Module 17 Phase 6's own "human editing" requirement. Same
    can review analysis findings permission gate Module 14 Phase 6 already
    established for reviewing Requirements/Security findings - never a
    second, parallel editing/permission system. The AI's own prior wording
    is never lost: it's preserved as this edit's own TEST CASE EDITED
    history row (old value), never a second "original" column on the row
    itself (see TestCase.edited by's own docstring). A no-op submission
    (nothing actually changed) is a cheap read, not an audit-trail entry -
    edited by/edited at are only ever set when something real changed."""
    change_request = _get_change_request_or_404(db, change_request_id)
    test_case = _get_test_case_or_404(db, change_request_id, test_case_id)

    if not workflow_rules.can_review_analysis_findings(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to edit test cases on this change request.",
        )

    updates = payload.model_dump(exclude_unset=True, exclude={"reason"})
    if updates.get("steps") is not None:
        updates["steps"] = json.dumps(updates["steps"])

    changed = _apply_partial_update(test_case, updates)
    analysis_id = test_case.analysis_id

    if changed:
        test_case.edited_by = current_user.id
        test_case.edited_at = datetime.utcnow()
        test_case.edit_reason = payload.reason
        old_value, new_value = _change_summary(changed)
        history_service.record_event(
            db,
            change_request_id=change_request_id,
            action=HistoryAction.TEST_CASE_EDITED,
            user_id=current_user.id,
            field_name=f"Test Case {test_case.test_id}",
            old_value=old_value,
            new_value=new_value,
            reason=payload.reason,
            version_number=change_request.current_version or 1,
        )
        db.commit()
        db.refresh(test_case)

    analysis = db.query(Analysis).options(* ANALYSIS_RELATIONSHIPS).filter(Analysis.id == analysis_id).one()
    response = full_analysis_read(db, change_request, analysis)
    return next(tc for tc in response.test_cases if tc.id == test_case.id)

@router.patch("/{change_request_id}/implementation-tasks/{task_id}", response_model=ImplementationTaskRead)
def update_implementation_task(
    change_request_id: int,
    task_id: int,
    payload: ImplementationTaskUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ImplementationTaskRead:
    """Same reasoning as update_test_case above, applied to
    ImplementationTask - see that endpoint's own docstring."""
    change_request = _get_change_request_or_404(db, change_request_id)
    task = _get_implementation_task_or_404(db, change_request_id, task_id)

    if not workflow_rules.can_review_analysis_findings(current_user, change_request, change_request.assignments):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You don't have permission to edit implementation tasks on this change request.",
        )

    updates = payload.model_dump(exclude_unset=True, exclude={"reason"})
    changed = _apply_partial_update(task, updates)
    analysis_id = task.analysis_id

    if changed:
        task.edited_by = current_user.id
        task.edited_at = datetime.utcnow()
        task.edit_reason = payload.reason
        old_value, new_value = _change_summary(changed)
        history_service.record_event(
            db,
            change_request_id=change_request_id,
            action=HistoryAction.IMPLEMENTATION_TASK_EDITED,
            user_id=current_user.id,
            field_name=f"Implementation Task #{task.id}",
            old_value=old_value,
            new_value=new_value,
        )

```



```

        reason=payload.reason,
        version_number=change_request.current_version or 1,
    )
    db.commit()
    db.refresh(task)

analysis = db.query(Analysis).options(* ANALYSIS_RELATIONSHIPS).filter(Analysis.id == analysis_id).one()
response = full_analysis_read(db, change_request, analysis)
return next(t for t in response.implementation_tasks if t.id == task.id)

@router.get("/{change_request_id}/analyses", response_model=list[AnalysisSummaryRead])
def list_analyses(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[Analysis]:
    """Every analysis ever run for this change request, newest first - a
    lightweight summary (no child tables) so a caller can see what exists
    and pick two ids to feed into the compare endpoint below."""
    _get_change_request_or_404(db, change_request_id)
    return (
        db.query(Analysis)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .all()
    )

def _get_analysis_or_404(db: Session, change_request_id: int, analysis_id: int) -> Analysis:
    analysis = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.id == analysis_id, Analysis.change_request_id == change_request_id)
        .first()
    )
    if analysis is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Analysis {analysis_id} not found on this change request.",
        )
    return analysis

@router.get("/{change_request_id}/analysis/compare", response_model=AnalysisDeltaResponse)
def compare_change_request_analyses(
    change_request_id: int,
    from_analysis_id: Optional[int] = Query(None, alias="from"),
    to_analysis_id: Optional[int] = Query(None, alias="to"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> AnalysisDeltaResponse:
    """A computed diff between two analyses of the same change request
    (spec sections 5/8 - "material change detection" / "analysis delta").
    Both 'from' and 'to' are optional: with neither given, this compares
    the two most recent analyses; with only 'to' given, 'from' defaults to
    whichever analysis immediately precedes it. See
    app/services/analysis_delta.py for what "computed, not AI-written"
    means here."""
    change_request = _get_change_request_or_404(db, change_request_id)

    all_analyses = (
        db.query(Analysis)
        .options(* ANALYSIS_RELATIONSHIPS)
        .filter(Analysis.change_request_id == change_request_id)
        .order_by(Analysis.created_at.desc())
        .all()
    )
    if len(all_analyses) < 2:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="This change request has fewer than two analyses - nothing to compare yet.",
        )

    if to_analysis_id is not None:
        analysis_b = _get_analysis_or_404(db, change_request_id, to_analysis_id)
    else:
        analysis_b = all_analyses[0] # newest

    if from_analysis_id is not None:
        analysis_a = _get_analysis_or_404(db, change_request_id, from_analysis_id)
    else:
        # Whichever analysis was run immediately before analysis_b.
        older = [a for a in all_analyses if a.created_at < analysis_b.created_at]
        if not older:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="There's no earlier analysis to compare this one against.",
            )
        analysis_a = older[0]

    if analysis_a.id == analysis_b.id:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="'from' and 'to' must refer to two different analyses.",
        )
    # Always compare in chronological order, regardless of which id the
    # caller passed as 'from' vs 'to'.
    if analysis_a.created_at > analysis_b.created_at:
        analysis_a, analysis_b = analysis_b, analysis_a

    delta = compare_analyses(change_request, analysis_a, analysis_b, _load_approval_rules(db))

    return AnalysisDeltaResponse(
        from_analysis_id=analysis_a.id,
        to_analysis_id=analysis_b.id,
        from_version=analysis_a.change_request_version,
        to_version=analysis_b.change_request_version,
        fields=[
            FieldComparisonRead(field=f.field, label=f.label, old_value=f.old_value, new_value=f.new_value, status=f.status)
            for f in delta.fields
        ],
        requirements_added=delta.requirements_added,
        requirements_removed=delta.requirements_removed,
        affected_components_added=delta.affected_components_added,
        affected_components_removed=delta.affected_components_removed,
        is_significant_change=delta.is_significant_change,
        significant_change_reasons=delta.significant_change_reasons,
    )

@router.get("/{change_request_id}/report/versions", response_model=list[ReportableVersionRead])
def list_reportable_versions(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ReportableVersionRead]:
    """Module 20: what the "generate a report for an earlier version"
    picker renders from - every version this change request has ever had,
    newest first, each flagged with whether it's the current one and
    whether an analysis was ever run against it specifically (a version

```



```

with has_analysis=false can still get a report - see download_report
below - just one with no AI sections filled in)."""
change_request = _get_change_request_or_404(db, change_request_id)
current_version = change_request.current_version or 1

analyzed_versions = {
    row[0]
    for row in db.query(Analysis.change_request_version)
    .filter(Analysis.change_request_id == change_request_id)
    .distinct()
    .all()
}

versions = sorted(change_request.versions, key=lambda v: v.version_number, reverse=True)
return [
    ReportableVersionRead(
        version_number=v.version_number,
        is_current=v.version_number == current_version,
        change_summary=v.change_summary,
        created_at=v.created_at,
        has_analysis=v.version_number in analyzed_versions,
    )
    for v in versions
]

@router.get("/{change_request_id}/report")
def download_report(
    change_request_id: int,
    version: Optional[int] = Query(
        None,
        ge=1,
        description="Generate a report for this specific past version instead of the current one. "
        "Omit for the current version's report.",
    ),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    """Module 11's report, made version-aware by Module 20. Never calls the
    AI provider and never creates a new Analysis row (report generator.py
    only formats what run_analysis already persisted) - the one write this
    endpoint does make is a REPORT GENERATED audit event on success, so the
    Audit section itself can show who pulled which report and when.

    Without `?version=`, this is "give me the current report": if the
    current version has actually been analyzed, it's built normally; if
    the change request has been edited since its last analysis, this is
    now a 409 (spec section 3 - "do not generate a misleading current
    report") rather than silently mixing today's CR fields with yesterday's
    analysis the way this endpoint used to. The 409 body names the version
    that WAS last analyzed, so the frontend can offer that as an explicit
    historical report instead of a dead end.

    With `?version=N`, this is an explicit historical report for version N
    - always allowed (an authorized user, i.e. anyone who can already view
    this change request - this app has never restricted viewing by role,
    only actions like editing/approving), always clearly labeled
    "HISTORICAL REPORT" in the PDF itself, and built entirely from that
    version's own data - never today's."""
    change_request = _get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.REPORTS)

    try:
        context = resolve_report_context(db, change_request, requested_version=version)
    except ReportVersionError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc)) from exc

    if not context.is_historical and not context.analysis_matches_version:
        last_analyzed_version = (
            db.query(Analysis.change_request_version)
            .filter(Analysis.change_request_id == change_request_id)
            .order_by(Analysis.created_at.desc())
            .limit(1)
            .scalar()
        )
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail={
                "message": "Current CR version has not been analyzed.",
                "cr_version": context.report_version,
                "last_analyzed_version": last_analyzed_version,
                "can_reanalyze": True,
            },
        )

    try:
        pdf_bytes = generate_report_pdf(change_request, context, generated_by=current_user.name)
    except ReportGenerationError as exc:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=str(exc)) from exc

    kind = "historical-report" if context.is_historical else "analysis-report"

    # HistoryAction.REPORT GENERATED has existed since Module 12 but was
    # never actually recorded anywhere until now - closing that gap here
    # is exactly what Module 20's own Audit section (spec section 6) needs
    # "report generated" events to show up at all.
    history_service.record_event(
        db,
        change_request_id=change_request_id,
        action=HistoryAction.REPORT_GENERATED,
        user_id=current_user.id,
        new_value=f"Version {context.report_version}" + (" (historical)" if context.is_historical else ""),
        version_number=change_request.current_version or 1,
    )
    db.commit()

    filename = f"CR-{change_request_id:04d}-{context.report_version}-{kind}.pdf"
    return Response(
        content=pdf_bytes,
        media_type="application/pdf",
        headers={"Content-Disposition": f'attachment; filename="{filename}"'},
    )

@router.get("/{change_request_id}/reports", response_model=ReportListResponse)
def list_reports_for_change_request(
    change_request_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ReportListResponse:
    """Module 24 (Reports) spec section 19: every report ever generated for
    this one change request (across every version), newest first - the same
    computed view the standalone Reports page reads from (see
    app/services/report_registry.py and app/api/reports.py), just
    pre-filtered to one change request. Powers a "Reports for this CR"
    listing without a second query mechanism."""
    _get_change_request_or_404(db, change_request_id)
    check_capability(db, current_user, Capability.REPORTS)
    rows = report_registry.list_reports_for_change_request(db, change_request_id)
    items = [
        ReportListItem(

```

```
        report_id=r.report_id,
        change_request_id=r.change_request_id,
        change_request_code=r.change_request_code,
        change_request_title=r.change_request_title,
        version=r.version,
        is_current_version=r.is_current_version,
        is_historical_pull=r.is_historical_pull,
        cr_status=r.cr_status,
        cr_status_label=r.cr_status_label,
        analysis_version=r.analysis_version,
        has_analysis=r.has_analysis,
        risk_score=r.risk_score,
        risk_bucket=r.risk_bucket_label,
        generated_by_id=r.generated_by_id,
        generated_by_name=r.generated_by_name,
        generated_at=r.generated_at,
        pull_count=r.pull_count,
    )
    for r in rows
]
return ReportListResponse(items=items, total=len(items))
```

```

"""Dashboard summary endpoint - metrics, risk/category distributions, and a
recent-requests list, all computed live from the database. No AI calls and
no repository analysis here yet (that's a later module); this only reads
what's already in SQLite.
"""
from datetime import date
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session, selectinload

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.analysis import Analysis
from app.models.change_request import ChangeRequest
from app.models.enums import ChangeRequestStatus
from app.models.user import User
from app.schemas.dashboard import (
    DashboardDrillDownResponse,
    DashboardMetrics,
    DashboardSummary,
    RecentChangeRequestItem,
)
from app.services.workflow_rules import risk_bucket

router = APIRouter(prefix="/dashboard", tags=["dashboard"])

RECENT_LIMIT = 10

# The dashboard's 5 metric tiles, and - for the drill-down endpoint below -
# the predicate that decides whether one _classify_change_request() result
# counts toward that tile. Kept right next to each other so a tile and its
# drill-down filter can never quietly drift apart.
_METRIC_PREDICATES = {
    "total": lambda c: True,
    "pending_analysis": lambda c: c["is_pending_analysis"],
    "high_risk": lambda c: c["is_high_risk"],
    "approved": lambda c: c["is_approved"],
    "requires_clarification": lambda c: c["is_requires_clarification"],
}

# Fixed vocabulary shown in the "change category overview" chart. Analysis.category
# is free-text (AI-generated, later module), so incoming values are normalized
# against this list; anything that doesn't match falls into "Other".
CANONICAL_CATEGORIES = [
    "Feature",
    "Bug Fix",
    "Security",
    "Database",
    "API",
    "Infrastructure",
    "Integration",
]

_CATEGORY_ALIASES = {
    "feature": "Feature",
    "new feature": "Feature",
    "enhancement": "Feature",
    "bug fix": "Bug Fix",
    "bugfix": "Bug Fix",
    "bug": "Bug Fix",
    "fix": "Bug Fix",
    "security": "Security",
    "database": "Database",
    "db": "Database",
    "api": "API",
    "infrastructure": "Infrastructure",
    "infra": "Infrastructure",
    "integration": "Integration",
}

def _normalize_category(raw: str | None) -> str:
    if not raw:
        return "Other"
    return _CATEGORY_ALIASES.get(raw.strip().lower(), "Other")

def _latest_analysis(change_request: ChangeRequest) -> Analysis | None:
    if not change_request.analyses:
        return None
    return max(change_request.analyses, key=lambda a: a.created_at)

def _classify_change_request(change_request: ChangeRequest) -> dict:
    """One change request's full classification for dashboard purposes -
    its risk bucket, normalized category, and which of the 5 metric tiles
    it counts toward. Both /summary (the tile numbers) and /drill-down (a
    clicked tile's actual list) are built from this exact same function, so
    a tile's count and its drill-down table can never show different change
    requests than what was actually counted."""
    latest = _latest_analysis(change_request)

    risk_label: str | None = None
    category_label: str | None = None
    is_requires_clarification = False

    if latest is not None:
        risk_label = risk_bucket(latest.risk_score)
        category_label = _normalize_category(latest.category)
        is_requires_clarification = any(not question.resolved for question in latest.clarification_questions)

    return {
        "change_request": change_request,
        "risk_label": risk_label,
        "category_label": category_label,
        "is_pending_analysis": latest is None,
        "is_high_risk": risk_label in ("high", "critical"),
        "is_approved": change_request.status == ChangeRequestStatus.APPROVED,
        "is_requires_clarification": is_requires_clarification,
    }

def _load_classified_change_requests(db: Session) -> list[dict]:
    change_requests = (
        db.query(ChangeRequest)
        .options(
            selectinload(ChangeRequest.analyses).selectinload(Analysis.clarification_questions)
        )
        .order_by(ChangeRequest.created_at.desc())
        .limit(RECENT_LIMIT)
        .all()
    )
    # Tie-break by id (desc) whenever two rows share the same created at.
    # created_at can have coarser resolution than how fast rows actually
    # get inserted (e.g. a fast test run, or two change requests filed
    # in the same second), and without a tiebreaker SQL doesn't promise
    # any particular order among ties - so "most recent first" wouldn't
    # reliably mean "most recently inserted first." id always increases

```

```

        # with insertion order, so this makes the ordering fully
        # deterministic, which both /summary's "recent" list and
        # /drill-down's pagination depend on to never skip or repeat a row.
        .order_by(ChangeRequest.created_at.desc(), ChangeRequest.id.desc())
        .all())
    )
    return [_classify_change_request(cr) for cr in change_requests]

def _to_recent_item(classified: dict) -> RecentChangeRequestItem:
    change_request = classified["change_request"]
    return RecentChangeRequestItem(
        id=change_request.id,
        title=change_request.title,
        category=classified["category_label"],
        risk=classified["risk_label"],
        status=change_request.status.value,
        created_at=change_request.created_at,
    )

@router.get("/summary", response_model=DashboardSummary)
def get_dashboard_summary(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> DashboardSummary:
    classified = _load_classified_change_requests(db)

    risk_distribution = {"low": 0, "medium": 0, "high": 0, "critical": 0, "not_analyzed": 0}
    category_breakdown = {category: 0 for category in CANONICAL_CATEGORIES}
    category_breakdown["Other"] = 0

    pending_analysis = 0
    high_risk = 0
    approved = 0
    requires_clarification = 0

    for entry in classified:
        if entry["is_pending_analysis"]:
            pending_analysis += 1
            risk_distribution["not_analyzed"] += 1
        else:
            risk_distribution[entry["risk_label"]] += 1
            category_breakdown[entry["category_label"]] += 1
            if entry["is_high_risk"]:
                high_risk += 1
            if entry["is_requires_clarification"]:
                requires_clarification += 1
            if entry["is_approved"]:
                approved += 1

    return DashboardSummary(
        metrics=DashboardMetrics(
            total_change_requests=len(classified),
            pending_analysis=pending_analysis,
            high_risk_changes=high_risk,
            approved_changes=approved,
            requires_clarification=requires_clarification,
        ),
        risk_distribution=risk_distribution,
        category_breakdown=category_breakdown,
        recent_change_requests=[_to_recent_item(entry) for entry in classified[:RECENT_LIMIT]],
    )

@router.get("/drill-down", response_model=DashboardDrillDownResponse)
def get_dashboard_drill_down(
    metric: str = Query(
        ...,
        description="Which dashboard tile was clicked.",
        pattern="^(total|pending_analysis|high_risk|approved|requires_clarification)$",
    ),
    date_from: Optional[date] = Query(None, description="Only change requests created on/after this date."),
    date_to: Optional[date] = Query(None, description="Only change requests created on/before this date."),
    page: int = Query(1, ge=1),
    page_size: int = Query(10, ge=1, le=10),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> DashboardDrillDownResponse:
    """Every change request behind one clicked dashboard tile, shown in the
    popup's table - never a separate/duplicated count, always the exact
    same classification /summary's own numbers come from (see
    _classify_change_request). date_from/date_to filter by the change
    request's created date (the same "Created" column the table already
    shows); page/page_size (capped at 10 - the popup never shows more than
    10 rows at once) paginate whatever's left after that filter."""
    predicate = _METRIC_PREDICATES.get(metric)
    if predicate is None:
        # Defensive only - the Query(pattern=...) above already rejects
        # anything else before this endpoint runs.
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Unknown dashboard metric.")

    classified = load_classified_change_requests(db)
    matching = [entry for entry in classified if predicate(entry)]

    if date_from is not None:
        matching = [entry for entry in matching if entry["change_request"].created_at.date() >= date_from]
    if date_to is not None:
        matching = [entry for entry in matching if entry["change_request"].created_at.date() <= date_to]

    total = len(matching)
    start = (page - 1) * page_size
    page_entries = matching[start : start + page_size]
    items = [_to_recent_item(entry) for entry in page_entries]
    return DashboardDrillDownResponse(metric=metric, items=items, total=total, page=page, page_size=page_size)

```

backend/app/api/deps.py

78 lines

```
"""Shared FastAPI dependencies for the API layer."""
import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.security import decode_access_token
from app.database.session import get_db
from app.models.enums import Capability
from app.models.user import User
from app.services.permissions import has_permission
from app.services.workflow_rules import is_active_user, is_admin

# auto_error=False so a missing token gives us our own consistent 401
# response below, instead of FastAPI's default 403.
_bearer_scheme = HTTPBearer(auto_error=False)

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(_bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    if credentials is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not authenticated")

    try:
        payload = decode_access_token(credentials.credentials)
    except jwt.ExpiredSignatureError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Session expired, please log in again"
        )
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid authentication token")

    user_id = payload.get("sub")
    user = db.get(User, int(user_id)) if user_id is not None else None
    if user is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User no longer exists")
    # Module 21: a deactivated account's existing token stops working on
    # its very next request - same "logged out" experience as an expired
    # token (app/context/AuthContext.jsx already treats any /auth/me
    # failure as "log this session out"), rather than only blocking future
    # logins.
    if not is_active_user(user):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="This account has been deactivated. Contact an administrator.",
        )
    return user

def require_admin(current_user: User = Depends(get_current_user)) -> User:
    """Module 21: gate for every /api/admin/* endpoint. is_active_user is
    already enforced by get_current_user above, so this only adds the
    role check on top."""
    if not is_admin(current_user):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN, detail="Admin access is required for this action."
        )
    return current_user

def check_capability(db: Session, user: User, capability: Capability) -> None:
    """Module 21 spec section 3: "Backend must enforce permissions." Raises
    403 if user's role doesn't have 'capability' per the permissions
    matrix (app/services/permissions.py). Called directly inside an
    endpoint body (matching this file's existing can edit change request-
    style inline checks) rather than as a Depends() - several call sites
    need this to run after the request body is parsed (e.g. responding to
    an approval is Capability.APPROVAL or Capability.REJECTION depending
    on the status in the payload), so every call site uses the same
    pattern rather than mixing two styles."""
    if not has_permission(db, user, capability):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail=f"Your role doesn't have permission to do this ({capability.value.replace('_', ' ')}).",
        )
```

backend/app/api/health.py

36 lines

```
"""Health-check endpoint - confirms the API is up, the DB is reachable,
and reports (without calling out to it) whether an AI provider is configured.
"""
from fastapi import APIRouter, Depends
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.database.session import get_db
from app.schemas.health import HealthResponse

router = APIRouter(tags=["health"])

@router.get("/health", response_model=HealthResponse)
def health_check(db: Session = Depends(get_db)) -> HealthResponse:
    settings = get_settings()

    try:
        db.execute(text("SELECT 1"))
        db_status = "connected"
    except Exception as exc: # pragma: no cover - defensive, surfaced in response
        db_status = f"error: {exc}"

    ai_configured = bool(
        settings.anthropic_api_key if settings.ai_provider == "anthropic" else settings.openai_api_key
    )

    return HealthResponse(
        status="ok",
        app_name=settings.app_name,
        app_env=settings.app_env,
        database=db_status,
        ai_provider_configured=ai_configured,
    )
```

```

"""Module 16 (Project Knowledge Base & RAG): upload, list, chunk, embed,
and search knowledge-base documents.

Phase 1 (Foundation): upload/list/get.
Phase 2 (Parsing + Chunking): upload now parses+chunks synchronously; a
reprocess endpoint retries a failed document; a chunks endpoint previews
the result.
Phase 3 (Embedding + Vector Search): an explicit embed endpoint (a
separate, network-calling step - see knowledge_embeddings.py's own
docstring for why it isn't chained into upload) and a search endpoint.

Any authenticated user can use any of this for now - the same starting
point Module 15's own api/repository.py set for its first phase ("per-CR
permission checks arrive once a scan's findings are actually tied to
one"). This module's own Phase 7 security pass is where real role-based
restriction (who may upload/delete vs who may only view) gets decided and
enforced, per this module's own spec section 9.
"""
from fastapi import APIRouter, Depends, File, HTTPException, Query, UploadFile, status
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.knowledge_chunk import KnowledgeChunk
from app.models.knowledge_document import KnowledgeDocument
from app.models.user import User
from app.schemas.knowledge_chunk import KnowledgeChunkRead
from app.schemas.knowledge_document import KnowledgeDocumentRead
from app.schemas.knowledge_search import KnowledgeSearchResult
from app.services.knowledge_chunker import process_document
from app.services.knowledge_documents import KnowledgeDocumentError, save_uploaded_document
from app.services.knowledge_embeddings import KnowledgeEmbeddingError, embed_document, search_chunks

router = APIRouter(prefix="/api/knowledge", tags=["knowledge"])

def get_document_or_404(db: Session, document_id: int) -> KnowledgeDocument:
    document = db.get(KnowledgeDocument, document_id)
    if document is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Knowledge document not found.")
    return document

@router.post("/documents", response_model=KnowledgeDocumentRead, status_code=status.HTTP_201_CREATED)
async def upload_document(
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    """Saves the uploaded file to disk, creates its KnowledgeDocument row,
    and immediately parses+chunks it (Module 16 Phase 2) - synchronous,
    since parsing/chunking is pure, fast, local text processing with no
    external call, matching this app's "no queue/worker" rule. The
    returned document's 'status' therefore already reflects the outcome:
    "ready" (chunked successfully - see chunk count) or "failed" (see
    error message) rather than always coming back "uploaded". Rejects
    unsupported extensions, empty files, and oversized files with a 400
    before any of that even starts (see knowledge_documents.py's own
    validation) - those are upload-time rejections, distinct from a
    parsing failure on an otherwise valid, accepted file."""
    content = await file.read()
    try:
        document = save_uploaded_document(
            db,
            original_filename=file.filename or "document",
            content=content,
            uploaded_by=current_user.id,
        )
    except KnowledgeDocumentError as exc:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(exc)) from exc
    return process_document(db, document)

@router.post("/documents/{document_id}/reprocess", response_model=KnowledgeDocumentRead)
def reprocess_document(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    """Re-runs parsing+chunking for one already-uploaded document - the
    deliberate, explicit retry action for a document that came back
    FAILED (e.g. after fixing whatever made it unparseable and re-
    uploading the file under the same title isn't practical), rather than
    ever silently retrying on its own."""
    document = get_document_or_404(db, document_id)
    return process_document(db, document)

@router.post("/documents/{document_id}/embed", response_model=KnowledgeDocumentRead)
def embed_document_endpoint(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    """Embeds (or re-embeds) every chunk of one document via the
    configured AI provider (Module 16 Phase 3) - a separate, explicitly-
    triggered step from upload/chunking, since this one makes a real
    network call to an AI provider rather than being pure local text
    processing (see knowledge_embeddings.py's own docstring for why).
    Returns the same KnowledgeDocumentRead shape as every other document
    endpoint - embeddings themselves are never exposed in any API
    response (see KnowledgeChunkRead's own docstring)."""
    document = get_document_or_404(db, document_id)
    try:
        return embed_document(db, document)
    except KnowledgeEmbeddingError as exc:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(exc)) from exc

@router.get("/search", response_model=list[KnowledgeSearchResult])
def search_knowledge_base(
    q: str = Query(..., min_length=1, description="The search query to find relevant knowledge-base chunks for."),
    top_k: int = Query(5, ge=1, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[dict]:
    """Embeds 'q' and ranks every already-embedded chunk (from a non-
    archived document) by cosine similarity to it - Module 16 Phase 3's
    own manual search endpoint, and the same retrieval Phase 4's CR
    analysis integration will call internally to gather grounding
    context."""
    try:
        return search_chunks(db, q, top_k=top_k)
    except KnowledgeEmbeddingError as exc:

```

```

        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(exc)) from exc

@router.get("/documents/{document_id}/chunks", response_model=list[KnowledgeChunkRead])
def list_document_chunks(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[KnowledgeChunk]:
    """Every chunk belonging to one document, in original document order
    - lets the frontend (Phase 6) preview a document's content, and gives
    a direct way to verify parsing/chunking without needing the retrieval
    step (Phase 3/4) built yet."""
    document = get_document_or_404(db, document_id)
    # is embedded is a computed field, not a real attribute on the
    # KnowledgeChunk ORM model (see knowledge_chunk.py's own schema
    # docstring) - automatic from attributes mapping would silently default
    # it to False for every chunk, so each KnowledgeChunkRead is built
    # explicitly here instead.
    return [
        KnowledgeChunkRead(
            id=chunk.id,
            document_id=chunk.document_id,
            chunk_index=chunk.chunk_index,
            section_label=chunk.section_label,
            content=chunk.content,
            char_count=chunk.char_count,
            is_embedded=chunk.embedding is not None,
        )
        for chunk in document.chunks
    ]

@router.get("/documents", response_model=list[KnowledgeDocumentRead])
def list_documents(
    include_archived: bool = Query(False),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[KnowledgeDocument]:
    """Every document, newest first - archived documents are left out
    unless 'include_archived=true' is passed (same 'hidden, not deleted'
    idea as ChangeRequest never being hard-deleted elsewhere in this app).
    The Knowledge Base page uses the default view; a future 'show
    archived' toggle there is what 'include_archived' is for."""
    query = db.query(KnowledgeDocument)
    if not include_archived:
        query = query.filter(KnowledgeDocument.archived.is_(False))
    return query.order_by(KnowledgeDocument.uploaded_at.desc()).all()

@router.post("/documents/{document_id}/archive", response_model=KnowledgeDocumentRead)
def archive_document(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    """Marks a document archived - it drops out of the default list and
    out of search/analysis retrieval (knowledge_embeddings.py::search_chunks
    already filters on KnowledgeDocument.archived), but its file, chunks,
    and embeddings are all left exactly as they are so unarchiving is
    instant and lossless. Never a hard delete - matches this app's
    established "archive, don't destroy" pattern (ChangeRequest, and this
    module's own KnowledgeEvidence rows, which snapshot a document's title
    specifically so they stay meaningful even after it's archived)."""
    document = get_document_or_404(db, document_id)
    document.archived = True
    db.commit()
    db.refresh(document)
    return document

@router.post("/documents/{document_id}/unarchive", response_model=KnowledgeDocumentRead)
def unarchive_document(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    """Reverses archive document - the document reappears in the default
    list and becomes eligible for search/analysis retrieval again."""
    document = get_document_or_404(db, document_id)
    document.archived = False
    db.commit()
    db.refresh(document)
    return document

@router.get("/documents/{document_id}", response_model=KnowledgeDocumentRead)
def get_document(
    document_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> KnowledgeDocument:
    return get_document_or_404(db, document_id)

```



```

"""Module 18 Phase 3 (Notifications, My Work & Personal Engineering
Queue): a personal dashboard - "what's on my plate right now" - built
entirely on top of tables Module 12 (assignments/approvals/notifications)
and this module's own Phase 1/2 additions already own. No new tables, no
parallel notification/assignment system (per the architecture lock) - this
is purely a set of read-only, differently-sliced views over existing data.

Every query below filters at the database level (an AssignmentRole/
ApprovalStatus/user_id condition in the SQL WHERE clause, or an '.any(...)'
correlated-subquery filter on the ORM relationship) rather than loading
every change request or every approval in the system and filtering in
Python - spec section 7's own "keep queries efficient" requirement. The
one exception is the small, already-narrowed result set each query
returns, where a per-row computation (risk bucket, due status, which
roles this one user holds) is cheap because it only ever runs over rows
that already matched the SQL filter.
"""

from typing import Optional

from fastapi import APIRouter, Depends, Query
from sqlalchemy import and
from sqlalchemy.orm import Session, selectinload

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.enums import ApprovalStatus, AssignmentRole, ChangeRequestStatus, NotificationType
from app.models.notification import Notification
from app.models.user import User
from app.schemas.my_work import (
    ChangesRequestedItem,
    MyApprovalItem,
    MyAssignmentItem,
    MyChangeRequestItem,
    MyReviewItem,
    MyWorkSummary,
)
from app.schemas.notification import NotificationRead
from app.services import approvals as approvals_service
from app.services.workflow_rules import APPROVAL_STATUS_LABELS, APPROVAL_TYPE_LABELS, risk_bucket

router = APIRouter(prefix="/api/my-work", tags=["my-work"])

REVIEW_ROLES = {
    AssignmentRole.REVIEWER,
    AssignmentRole.TECHNICAL_LEAD,
    AssignmentRole.SECURITY_REVIEWER,
    AssignmentRole.OWNER,
}

# --- shared per-row helpers (deliberately duplicated in spirit from
# app/api/change_requests.py's own _latest_analysis/_effective_status -
# each module owns its own small helpers rather than importing another
# module's underscore-prefixed ones, same convention
# repository_linkage.py/traceability_linkage.py already follow) -----

def _latest_analysis(change_request: ChangeRequest) -> Optional[Analysis]:
    if not change_request.analysis:
        return None
    return max(change_request.analysis, key=lambda a: a.created_at)

def _effective_status(change_request: ChangeRequest, latest: Optional[Analysis]) -> str:
    if change_request.status == ChangeRequestStatus.APPROVED:
        return "approved"
    if change_request.status == ChangeRequestStatus.DRAFT:
        return "draft"
    if latest is None:
        return "pending_analysis"
    if any(not question.resolved for question in latest.clarification_questions):
        return "requires_clarification"
    return "completed"

def _risk_for(change_request: ChangeRequest) -> Optional[str]:
    latest = _latest_analysis(change_request)
    return risk_bucket(latest.risk_score) if latest else None

def _cr_query(db: Session):
    return db.query(ChangeRequest).options(
        selectinload(ChangeRequest.analysis).selectinload(Analysis.clarification_questions),
        selectinload(ChangeRequest.assignments),
    )

@router.get("/summary", response_model=MyWorkSummary)
def my_work_summary(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> MyWorkSummary:
    """Cheap counts only (see MyWorkSummary's own docstring) - each number
    here is its own small COUNT-style query, not a byproduct of loading
    every row from the section endpoints below."""
    my_change_requests = (
        db.query(ChangeRequest).filter(ChangeRequest.created_by == current_user.id).count()
    )
    my_approvals_pending = (
        db.query(Approval)
        .filter(Approval.approver_id == current_user.id, Approval.status == ApprovalStatus.PENDING)
        .count()
    )
    my_reviews = (
        db.query(ChangeRequest)
        .filter(
            ChangeRequest.assignments.any(
                and(
                    ChangeRequestAssignment.user_id == current_user.id,
                    ChangeRequestAssignment.role.in_(REVIEW_ROLES),
                )
            )
        )
        .count()
    )
    my_assignments = (
        db.query(ChangeRequest)
        .filter(ChangeRequest.assignments.any(ChangeRequestAssignment.user_id == current_user.id))
        .count()
    )

```

```

changes_requested_from_me = (
    db.query(ChangeRequest)
    .filter(
        ChangeRequest.created_by == current_user.id,
        ChangeRequest.approvals.any(Approval.status == ApprovalStatus.CHANGES_REQUESTED),
    )
    .count()
)
unread_mentions = (
    db.query(Notification)
    .filter(
        Notification.user_id == current_user.id,
        Notification.type == NotificationType.MENTIONED,
        Notification.is_read.is_(False),
    )
    .count()
)
# Overdue is computed (never stored), so this can't be a single SQL
# COUNT the way the others above are - but it's still scoped at the
# database level to just this user's own still-PENDING approvals with
# a due date at all, the same cheap pre-filter
# app/services/deadlines.py already uses, before the per-row
# approval due status() check.
pending_with_due_date = (
    db.query(Approval)
    .filter(
        Approval.approver_id == current_user.id,
        Approval.status == ApprovalStatus.PENDING,
        Approval.due_date.isnot(None),
    )
    .all()
)
overdue_approvals = sum(
    1 for a in pending_with_due_date if approvals_service.approval_due_status(a) == "overdue"
)

return MyWorkSummary(
    my_change_requests=my_change_requests,
    my_approvals_pending=my_approvals_pending,
    my_reviews=my_reviews,
    my_assignments=my_assignments,
    changes_requested_from_me=changes_requested_from_me,
    unread_mentions=unread_mentions,
    overdue_approvals=overdue_approvals,
)

@router.get("/change-requests", response_model=list[MyChangeRequestItem])
def my_change_requests(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[MyChangeRequestItem]:
    """Change requests the current user personally created - filtered at
the database level (ChangeRequest.created_by == current_user.id), not
by loading every change request in the system."""
    rows = (
        cr.query(db)
        .filter(ChangeRequest.created_by == current_user.id)
        .order_by(ChangeRequest.created_at.desc())
        .all()
    )
    return [
        MyChangeRequestItem(
            id=cr.id,
            title=cr.title,
            status=effective_status(cr, _latest_analysis(cr)),
            priority=cr.priority,
            risk=_risk_for(cr),
            created_at=cr.created_at,
        )
        for cr in rows
    ]

@router.get("/approvals", response_model=list[MyApprovalItem])
def my_approvals(
    status_filter: Optional[str] = Query(
        "pending",
        alias="status",
        description="pending" (default - the actual queue of things waiting on you), or "all" for every approval you've ever been tagged on.',
    ),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[MyApprovalItem]:
    """Every sign-off tagged to the current user (spec section 4) -
filtered at the database level by Approval.approver_id, never by
loading every approval in the system. Defaults to PENDING only (the
actual personal queue); pass ?status=all for the full history."""
    query = (
        db.query(Approval)
        .options(
            selectinload(Approval.requested_by.user),
            selectinload(Approval.change_request).selectinload(ChangeRequest.analyses),
        )
        .filter(Approval.approver_id == current_user.id)
    )
    if status_filter != "all":
        query = query.filter(Approval.status == ApprovalStatus.PENDING)

    rows = query.order_by(Approval.requested_at.desc()).all()

    items: list[MyApprovalItem] = []
    for approval in rows:
        change_request = approval.change_request
        items.append(
            MyApprovalItem(
                id=approval.id,
                change_request_id=change_request.id,
                change_request_title=change_request.title,
                approval_type=approval.approval_type,
                approval_type_label=APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value),
                status=approval.status,
                status_label=APPROVAL_STATUS_LABELS.get(approval.status, approval.status.value),
                requested_by_name=approval.requested_by.user.name,
                requested_at=approval.requested_at,
                due_date=approval.due_date,
                due_status=approvals_service.approval_due_status(approval),
                risk=_risk_for(change_request),
            )
        )
    return items

@router.get("/reviews", response_model=list[MyReviewItem])
def my_reviews(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[MyReviewItem]:
    """Change requests where the current user holds at least one of
Reviewer/Technical Lead/Security Reviewer/Owner (spec section 5) -
filtered at the database level via a correlated ".any(...)" on the

```

```

assignments relationship, never by loading every change request."""
rows = (
    cr.query(db)
    .filter(
        ChangeRequest.assignments.any(
            and_(
                ChangeRequestAssignment.user_id == current_user.id,
                ChangeRequestAssignment.role.in_(REVIEW_ROLES),
            )
        )
    )
    .order_by(ChangeRequest.created_at.desc())
    .all()
)
return [
    MyReviewItem(
        change_request_id=cr.id,
        title=cr.title,
        status=effective_status(cr, _latest_analysis(cr)),
        priority=cr.priority,
        risk=_risk_for(cr),
        roles=sorted(
            (a.role for a in cr.assignments if a.user_id == current_user.id and a.role in REVIEW_ROLES),
            key=lambda r: r.value,
        ),
    )
    for cr in rows
]

@router.get("/assignments", response_model=list[MyAssignmentItem])
def my_assignments(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[MyAssignmentItem]:
    """Every change request the current user holds ANY assignment role on
    - broader than /reviews above. Same database-level .any(...) filter
    already established by list_change_requests's own assigned_to_me
    query, reused here rather than re-invented."""
    rows = (
        cr.query(db)
        .filter(ChangeRequest.assignments.any(ChangeRequestAssignment.user_id == current_user.id))
        .order_by(ChangeRequest.created_at.desc())
        .all()
    )
    return [
        MyAssignmentItem(
            change_request_id=cr.id,
            title=cr.title,
            status=effective_status(cr, _latest_analysis(cr)),
            priority=cr.priority,
            risk=_risk_for(cr),
            roles=sorted(
                (a.role for a in cr.assignments if a.user_id == current_user.id),
                key=lambda r: r.value,
            ),
        )
        for cr in rows
    ]

@router.get("/changes-requested", response_model=list[ChangesRequestedItem])
def changes_requested_from_me(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[ChangesRequestedItem]:
    """Change requests the current user OWNS (created) where at least one
    approver has responded Changes Requested - "someone is waiting on ME
    to revise" (confirmed interpretation - see this module's own planning
    discussion). Filtered at the database level by created_by plus a
    correlated .any(...) on the approvals relationship.

    Known simplification (disclosed, not a bug): this shows every CR with
    ANY Changes-Requested approval ever recorded, even one from an older
    CR version the owner may have already addressed with a later edit -
    there's no existing "has this feedback been addressed" flag anywhere
    else in this app to check against (a resolved approval's own
    is_outdated is always False, by design - see
    app/services/approvals.py::is_approval_outdated's own docstring)."""
    rows = (
        db.query(ChangeRequest)
        .options(
            selectinload(ChangeRequest.approvals).selectinload(Approval.approver),
        )
        .filter(
            ChangeRequest.created_by == current_user.id,
            ChangeRequest.approvals.any(Approval.status == ApprovalStatus.CHANGES_REQUESTED),
        )
        .order_by(ChangeRequest.created_at.desc())
        .all()
    )

    items: list[ChangesRequestedItem] = []
    for cr in rows:
        changes_requested_approvals = [a for a in cr.approvals if a.status == ApprovalStatus.CHANGES_REQUESTED]
        # A CR can have more than one Changes-Requested approval (e.g. two
        # different approval types both sent back) - one row per approval,
        # newest response first, so nothing is silently collapsed away.
        for approval in sorted(
            changes_requested_approvals, key=lambda a: a.responded_at or a.requested_at, reverse=True
        ):
            items.append(
                ChangesRequestedItem(
                    change_request_id=cr.id,
                    title=cr.title,
                    approval_type=approval.approval_type,
                    approval_type_label=APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value),
                    approver_name=approval.approver.name,
                    responded_at=approval.responded_at,
                    comment=approval.comment,
                )
            )
    return items

@router.get("/mentions", response_model=list[NotificationRead])
def my_mentions(
    unread_only: bool = Query(False),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[Notification]:
    """Every time the current user was @mentioned (spec section 3's
    "Mentions" section) - reuses the existing Notification row (type=
    MENTIONED) rather than a second, parallel mentions table; mirrors
    app/api/notifications.py::list_notifications's own shape, just
    pre-filtered to this one type."""
    query = db.query(Notification).filter(
        Notification.user_id == current_user.id, Notification.type == NotificationType.MENTIONED
    )
    if unread_only:

```

```

        query = query.filter(Notification.is_read.is (False))
    return query.order_by(Notification.created_at.desc()).limit(limit).all()

@router.get("/overdue", response_model=list[MyApprovalItem])
def overdue_items(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[MyApprovalItem]:
    """Every still-PENDING approval tagged to the current user whose own
    due_date has already passed (spec section 3's "Overdue Items") - the
    same computed approval_due_status() everything else in this module
    uses, filtered down to exactly "overdue". Today this only ever
    surfaces overdue approvals - the one kind of per-user deadline this
    app tracks (see app/models/approval.py's own due_date docstring)."""
    candidates = (
        db.query(Approval)
        .options(
            selectinload(Approval.requested_by_user),
            selectinload(Approval.change_request).selectinload(ChangeRequest.analyses),
        )
        .filter(
            Approval.approver_id == current_user.id,
            Approval.status == ApprovalStatus.PENDING,
            Approval.due_date.isnot(None),
        )
        .order_by(Approval.due_date.asc())
        .all()
    )

    items: list[MyApprovalItem] = []
    for approval in candidates:
        if approvals_service.approval_due_status(approval) != "overdue":
            continue
        change_request = approval.change_request
        items.append(
            MyApprovalItem(
                id=approval.id,
                change_request_id=change_request.id,
                change_request_title=change_request.title,
                approval_type=approval.approval_type,
                approval_type_label=APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value),
                status=approval.status,
                status_label=APPROVAL_STATUS_LABELS.get(approval.status, approval.status.value),
                requested_by_name=approval.requested_by_user.name,
                requested_at=approval.requested_at,
                due_date=approval.due_date,
                due_status="overdue",
                risk=_risk_for(change_request),
            )
        )
    return items

```

backend/app/api/notifications.py

95 lines

```
"""Module 12 Phase 5: a user's own in-app notification inbox - assigned to
a CR, tagged for approval (or reminded), an approval was responded to, a
CR's status changed, @mentioned or commented on, an AI analysis went
outdated, or a re-approval is needed (see app/models/notification.py and
app/services/notify.py, the only place that creates these rows). Module 18
Phase 1 added risk escalation, analysis completion, and a computed
deadline-approaching check (see app/services/deadlines.py) - same
notification rows, same one gatekeeper, no second system.

Every route is scoped to `current_user` - there is no way to read or mark
another user's notifications through this API.
@ 12-18

from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.notification import Notification
from app.models.user import User
from app.schemas.notification import NotificationRead
from app.services.deadlines import check_and_notify_approaching_deadlines

router = APIRouter(prefix="/api/notifications", tags=["notifications"])

@router.get("", response_model=list[NotificationRead])
def list_notifications(
    unread_only: bool = Query(False),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[Notification]:
    # Module 18 Phase 1: a lazy, opportunistic check - see
    # app/services/deadlines.py's own docstring for why this lives here
    # instead of a background scheduler.
    check_and_notify_approaching_deadlines(db, current_user)
    db.commit()

    query = db.query(Notification).filter(Notification.user_id == current_user.id)
    if unread_only:
        query = query.filter(Notification.is_read.is_(False))
    return query.order_by(Notification.created_at.desc()).limit(limit).all()

@router.get("/unread-count")
def unread_count(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> dict:
    # Module 18 Phase 1: same reasoning as list notifications above - this
    # endpoint is polled every 30s by the frontend's badge, so it's a
    # reliable, frequent hook for a deadline to actually get noticed.
    check_and_notify_approaching_deadlines(db, current_user)
    db.commit()

    count = (
        db.query(Notification)
        .filter(Notification.user_id == current_user.id, Notification.is_read.is_(False))
        .count()
    )
    return {"count": count}

@router.put("/{notification_id}/read", response_model=NotificationRead)
def mark_read(
    notification_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Notification:
    notification = (
        db.query(Notification)
        .filter(Notification.id == notification_id, Notification.user_id == current_user.id)
        .first()
    )
    if notification is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Notification not found.")
    notification.is_read = True
    db.commit()
    db.refresh(notification)
    return notification

@router.put("/read-all")
def mark_all_read(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> dict:
    updated = (
        db.query(Notification)
        .filter(Notification.user_id == current_user.id, Notification.is_read.is_(False))
        .update({"is_read": True})
    )
    db.commit()
    return {"updated": updated}
```

```

"""Module 24: Reports section.

An aggregation layer over data that already exists (see
app/services/report_registry.py) - no new database table, no second
PDF-building system, and no second audit mechanism. Every endpoint here is
read-only: a report is never generated through this router. A
REPORT_GENERATED history event - the same event this module's whole
Reports view is built from - is recorded automatically in exactly two
places, both pre-existing: app/api/change_requests.py's own
"analyze_change_request" (the instant an analysis completes for a change
request's current version) and its long-standing "download_report"
endpoint (the Analysis Dashboard's "Download Report" button). Reports are
explicitly NOT something a person generates as a separate step (confirmed
with the project owner) - they simply appear here once an analysis exists,
and this router only ever reads that.

Every endpoint requires a logged-in user with the existing REPORTS
capability - the same permission check the Analysis Dashboard's own
report download has always used (app/services/permissions.py). This app
has never restricted which change requests a logged-in user can VIEW (see
report_generator.py's own docstring) - only actions like editing or
approving - so Reports follows that same, already-established rule rather
than inventing a new visibility restriction just for this module.
"""

from __future__ import annotations

from datetime import date
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query, Response, status
from sqlalchemy.orm import Session

from app.api.deps import check_capability, get_current_user
from app.database.session import get_db
from app.models.change_request import ChangeRequest
from app.models.enums import Capability, ChangeRequestStatus
from app.models.user import User
from app.schemas.report import ReportDetail, ReportListItem, ReportListResponse, ReportStatsRead
from app.services.report_registry import (
    ReportGenerationError,
    ReportVersionError,
    generate_report_pdf,
    resolve_report_context,
)

router = APIRouter(prefix="/api/reports", tags=["reports"])

def get_change_request_or_404(db: Session, change_request_id: int) -> ChangeRequest:
    # Same trivial lookup app/api/change_requests.py's own (private, so not
    # imported across modules) helper does - not worth a shared import for
    # four lines with no business logic in them.
    change_request = db.get(ChangeRequest, change_request_id)
    if change_request is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Change request not found.")
    return change_request

def _to_item(row: report_registry.ReportRow) -> ReportListItem:
    return ReportListItem(
        report_id=row.report_id,
        change_request_id=row.change_request_id,
        change_request_code=row.change_request_code,
        change_request_title=row.change_request_title,
        version=row.version,
        is_current_version=row.is_current_version,
        is_historical_pull=row.is_historical_pull,
        cr_status=row.cr_status,
        cr_status_label=row.cr_status_label,
        analysis_version=row.analysis_version,
        has_analysis=row.has_analysis,
        risk_score=row.risk_score,
        risk_bucket=row.risk_bucket_label,
        generated_by_id=row.generated_by_id,
        generated_by_name=row.generated_by_name,
        generated_at=row.generated_at,
        pull_count=row.pull_count,
    )

@router.get("", response_model=ReportListResponse)
def list_reports(
    search: Optional[str] = Query(None, description="Matches CR id, CR code (e.g. CR-0042), or CR title."),
    status_filter: Optional[ChangeRequestStatus] = Query(None, alias="status"),
    risk: Optional[str] = Query(None, description="low / medium / high / critical"),
    version: Optional[int] = Query(None, ge=1),
    generated_by: Optional[int] = Query(None, description="User id of whoever generated the report."),
    date_from: Optional[date] = Query(None),
    date_to: Optional[date] = Query(None),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ReportListResponse:
    check_capability(db, current_user, Capability.REPORTS)
    rows = report_registry.list_reports(
        db,
        search=search,
        status=status_filter,
        risk=risk,
        version=version,
        generated_by=generated_by,
        date_from=date_from,
        date_to=date_to,
    )
    items = [_to_item(r) for r in rows]
    return ReportListResponse(items=items, total=len(items))

@router.get("/stats", response_model=ReportStatsRead)
def report_stats(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ReportStatsRead:
    check_capability(db, current_user, Capability.REPORTS)
    stats = report_registry.compute_stats(db)
    return ReportStatsRead(
        total_reports=stats.total_reports,
        change_requests_with_reports=stats.change_requests_with_reports,
        recent_reports=stats.recent_reports,
        outdated_reports=stats.outdated_reports,
    )

@router.get("/{report_id}", response_model=ReportDetail)

```

```

def get_report(
    report_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> ReportDetail:
    check_capability(db, current_user, Capability.REPORTS)
    row = report_registry.get_report(db, report_id)
    if row is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Report not found.")
    return ReportDetail(*_to_item(row).model_dump())

@router.get("/{report_id}/download")
def download_report_by_id(
    report_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> Response:
    """Re-serves the PDF for an already-generated report row. Never stores
    PDF bytes (see report_registry's module docstring) - this rebuilds them
    fresh, every time, from the exact same version-consistent data
    resolve_report context always uses, so what's downloaded here can never
    drift from what a fresh "Download Report" click would produce. Does NOT
    record a new REPORT_GENERATED event: that already happened automatically
    (see app/api/change_requests.py::analyze_change_request, or the Analysis
    Dashboard's own existing download button) - re-opening an existing
    report is not a new "pull" for audit purposes, it's the same one."""
    check_capability(db, current_user, Capability.REPORTS)
    row = report_registry.get_report(db, report_id)
    if row is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Report not found.")

    change_request = _get_change_request_or_404(db, row.change_request_id)
    try:
        context = resolve_report_context(db, change_request, requested_version=row.version)
    except ReportVersionError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc)) from exc
    try:
        pdf_bytes = generate_report_pdf(change_request, context, generated_by=row.generated_by_name)
    except ReportGenerationError as exc:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=str(exc)) from exc

    kind = "historical-report" if context.is_historical else "analysis-report"
    filename = f"{row.change_request_code}-{v[context.report_version]}-{kind}.pdf"
    return Response(
        content=pdf_bytes,
        media_type="application/pdf",
        headers={"Content-Disposition": f'attachment; filename="{filename}"'},
    )

```

backend/app/api/repository.py

83 lines

```
"""Module 15 Phase 1 (Repository Intelligence): trigger and inspect scans
of the configured local repository/folder.

Not scoped to any one change request - a scan indexes "the repository" in
general (see app/models/repository_scan.py's docstring for why). Any
authenticated user can trigger or view a scan for now, the same way
GET /api/users is a read-only directory open to anyone logged in - nothing
here exposes anything about a specific change request yet. Per-CR
permission checks arrive in Phase 3, once a scan's findings are actually
tied to one.
"""
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session, selectinload

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.repository_scan import RepositoryScan
from app.models.user import User
from app.schemas.repository_scan import RepositoryScanDetail, RepositoryScanRead
from app.services.repository_scanner import RepositoryScanError, scan_repository

router = APIRouter(prefix="/api/repository", tags=["repository"])

_SCAN_RELATIONSHIPS = (selectinload(RepositoryScan.files),)

@router.post("/scan", response_model=RepositoryScanRead, status_code=status.HTTP_201_CREATED)
def trigger_scan(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> RepositoryScan:
    """Scans the app's configured repository root and records the result
    as a new RepositoryScan. Never touches or deletes any prior scan - a
    failed or stale scan just sits there as history, exactly like a failed
    AI re-analysis leaves the previous good analysis untouched."""
    try:
        return scan_repository(db, triggered_by=current_user.id)
    except RepositoryScanError as exc:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(exc)) from exc

@router.get("/", response_model=list[RepositoryScanRead])
def list_scans(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[RepositoryScan]:
    """Every scan ever run, newest first - a lightweight history list (no
    file listing), same shape as GET /api/change-requests/{id}/analyses."""
    return db.query(RepositoryScan).order_by(RepositoryScan.started_at.desc()).all()

@router.get("/latest", response_model=RepositoryScanDetail)
def get_latest_scan(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> RepositoryScan:
    scan = (
        db.query(RepositoryScan)
        .options(*_SCAN_RELATIONSHIPS)
        .order_by(RepositoryScan.started_at.desc())
        .first()
    )
    if scan is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="No repository scan has been run yet.")
    return scan

@router.get("/{scan_id}", response_model=RepositoryScanDetail)
def get_scan(
    scan_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> RepositoryScan:
    scan = (
        db.query(RepositoryScan)
        .options(*_SCAN_RELATIONSHIPS)
        .filter(RepositoryScan.id == scan_id)
        .first()
    )
    if scan is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Repository scan not found.")
    return scan
```


backend/app/api/users.py

23 lines

```
"""A simple, read-only user directory (Module 12 Phase 3) - lets the
frontend populate the "assign a team member" picker on a change request.
No admin user-management here (invite/edit/deactivate) - out of scope for
this hackathon.
"""
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.database.session import get_db
from app.models.user import User
from app.schemas.user import UserRead

router = APIRouter(prefix="/api/users", tags=["users"])

@router.get("", response_model=list[UserRead])
def list_users(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> list[User]:
    return db.query(User).order_by(User.name).all()
```

backend/app/core/__init__.py

1 lines

```

"""
Centralized application configuration.

All configuration is loaded from environment variables (via a local .env
file during development). Nothing sensitive is ever hardcoded here -
copy backend/.env.example to backend/.env and fill in real values locally.
"""
from functools import lru_cache
from pathlib import Path
from typing import List

from pydantic import BaseModel
from pydantic_settings import BaseSettings, SettingsConfigDict

# Module 22 (Final Integration, Security & Quality): the one value below
# that must never be a real, working secret. Every other "unsafe" default
# in this file (blank AI provider keys, the local sqlite path) just leaves
# a feature turned off until configured - a weak SECRET_KEY instead
# silently accepts forged login tokens, so it gets its own named sentinel
# and its own startup guard (see Settings.forbid_placeholder_secret_key
# below) rather than just a comment asking a developer to remember to
# change it.
_PLACEHOLDER_SECRET_KEY = "hackathon-dev-secret-change-me"

class Settings(BaseSettings):
    # --- App ---
    app_name: str = "AI Change Request Analyzer"
    # Deliberately no 'debug' flag here (an earlier version of this file
    # had one that was never actually wired to FastAPI's own debug mode,
    # which would echo Python tracebacks straight into API responses on an
    # unhandled error - spec section 6's "do not expose stack traces in
    # production UI"). Removing the dead field is safer than fixing its
    # wiring, since this app never needs traceback-in-response behavior -
    # app/main.py's own exception handler is what should show on an
    # unhandled error, in every environment, always.
    app_env: str = "development"

    # --- Database ---
    database_url: str = "sqlite:///./data/app.db"

    # --- CORS ---
    cors_origins: str = "http://localhost:5173,http://127.0.0.1:5173"

    # --- AI provider ---
    ai_provider: str = "anthropic"
    anthropic_api_key: str = ""
    anthropic_model: str = "claude-sonnet-4.5"
    openai_api_key: str = ""
    openai_model: str = "gpt-4o"
    # Google Gemini - a free-tier alternative to Anthropic/OpenAI. A Gemini
    # API key (aistudio.google.com/apikey) doesn't require billing to get
    # started, unlike the other two providers. Set AI_PROVIDER=gemini to use it.
    gemini_api_key: str = ""
    gemini_model: str = "gemini-3.8-flash"
    # Ollama - runs a model entirely on this machine, no API key/signup/
    # internet at all once the model is pulled. Set AI_PROVIDER=ollama to
    # use it (requires Ollama installed and running locally - see
    # ollama.com). Slower than a hosted API on a laptop CPU, so consider
    # raising ai_request_timeout_seconds when using this.
    ollama_base_url: str = "http://localhost:11434"
    ollama_model: str = "llama3.2"
    # OpenRouter - one API key/endpoint that proxies many different models
    # (Anthropic, OpenAI, Meta, Google, etc). Get a key at
    # openrouter.ai/keys, then set AI_PROVIDER=openrouter to use it. Model
    # names are OpenRouter's own slugs (e.g. "openai/gpt-4o-mini",
    # "anthropic/claude-3.5-sonnet") - see openrouter.ai/models.
    openrouter_api_key: str = ""
    openrouter_model: str = "openai/gpt-4o-mini"
    # Module 16 (Project Knowledge Base & RAG): the model OpenRouter's
    # /api/v1/embeddings endpoint uses to turn document chunks and search
    # queries into vectors - a small, cheap, well-established embedding
    # model by default. Only used if AI_PROVIDER=openrouter; other
    # providers wired up here don't support embeddings (see
    # AIProvider.embed's own docstring). Override in backend/.env only if
    # you want a different OpenRouter-hosted embedding model.
    openrouter_embedding_model: str = "openai/text-embedding-3-small"
    # How long (seconds) the AI Analysis Engine (Module 6) waits for one
    # provider call before giving up and reporting a timeout error.
    ai_request_timeout_seconds: int = 60

    # --- Auth ---
    # Signs/verifies login tokens (JWT). The placeholder below only lets
    # the app boot before backend/.env exists at all (same "blank until
    # configured" convention as the AI provider keys above) - it is never
    # a usable production value, and forbid_placeholder_secret_key below
    # refuses to start on it outside local development. Set your own
    # SECRET_KEY in backend/.env (see .env.example for how to generate
    # one) - never one shared with, or committed alongside, this code.
    secret_key: str = _PLACEHOLDER_SECRET_KEY
    access_token_expire_minutes: int = 60 * 24 # 1 day - normal login
    remember_me_token_expire_minutes: int = 60 * 24 * 30 # 30 days - "remember me" checked

    # --- Repository Intelligence (Module 15) ---
    # Local filesystem path to the repository the AI should be aware of
    # when matching change requests to source files. Left blank by default,
    # which means "index this project's own repository" (see
    # repository_root_path below) - the simplest practical option per the
    # module's own spec ("do not introduce unnecessary infrastructure"): no
    # upload/archive handling, no separate storage - just a path on disk
    # this backend process can already read. Set REPOSITORY_ROOT in .env to
    # point at a different local folder instead.
    repository_root: str = ""

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
        extra="ignore",
    )

    @model_validator(mode="after")
    def forbid_placeholder_secret_key(self) -> "Settings":
        """Module 22: the placeholder SECRET_KEY is fine for local
        development (app env stays "development" unless APP_ENV is
        explicitly set otherwise in .env) - anyone running this app
        elsewhere with APP_ENV set to anything else is refused a boot
        instead of silently signing every login token with a value that
        ships in this project's own source. This never affects local
        hackathon use: app.env's own default is "development", so nothing
        changes unless APP_ENV is deliberately set."""
        if self.app_env.strip().lower() != "development" and self.secret_key == _PLACEHOLDER_SECRET_KEY:
            raise ValueError(
                "SECRET_KEY is still the placeholder value from config.py, but APP_ENV is "

```

```

        f"{self.app_env!r} (not 'development'). Set a real SECRET_KEY in backend/.env "
        "before running outside local development - see .env.example."
    )
    return self

@property
def cors_origins_list(self) -> List[str]:
    return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]

@property
def repository_root_path(self) -> Path:
    """Resolves repository_root to an absolute path. An explicit
    REPOSITORY_ROOT env var always wins; otherwise this defaults to the
    project's own root folder - computed from this file's own location
    (backend/app/core/config.py -> core -> app -> backend -> project
    root) rather than hardcoded, so it resolves correctly on any
    machine this app is checked out on."""
    if self.repository_root.strip():
        return Path(self.repository_root).expanduser().resolve()
    return Path(__file__).resolve().parents[3]

@lru_cache
def get_settings() -> Settings:
    """Settings are read once and cached - re-reading env vars every request is wasteful."""
    return Settings()

```

backend/app/core/security.py

69 lines

```
"""Password hashing and JWT helpers.

Password hashing uses PBKDF2-HMAC-SHA256 from Python's standard library
`hashlib` - not bcrypt/argon2. This is a deliberate choice for this
project: earlier modules hit real build failures on this machine because
some packages (pydantic-core) needed to compile native code for a very new
Python version, and bcrypt has the same class of risk (it ships prebuilt
wheels, but not always for the newest Python on day one). PBKDF2 avoids
that entirely - it's pure Python/stdlib, and it's a legitimate, widely used
algorithm (it was Django's own default for years) at a strong iteration
count.

Tokens are JWTs (HS256) via PyJWT, which is pure Python with no compiled
dependency either - same reasoning.
"""
import hashlib
import hmac
import secrets
from datetime import datetime, timedelta, timezone
from typing import Any

import jwt

from app.core.config import get_settings

PBKDF2_ALGORITHM = "sha256"
_PBKDF2_ITERATIONS = 260_000 # OWASP-recommended minimum (2023+) for PBKDF2-SHA256


def hash_password(password: str) -> str:
    """Returns a self-contained hash string: "pbkdf2_sha256$<iterations>$<salt>$<hash>" (hex)."""
    salt = secrets.token_hex(16)
    derived = hashlib.pbkdf2_hmac(
        _PBKDF2_ALGORITHM, password.encode("utf-8"), bytes.fromhex(salt), _PBKDF2_ITERATIONS
    )
    return f"pbkdf2_sha256${_PBKDF2_ITERATIONS}${salt}${derived.hex()}"


def verify_password(password: str, stored_hash: str) -> bool:
    """Constant-time comparison - never short-circuits on the first differing byte."""
    try:
        algorithm, iterations, salt, hash_hex = stored_hash.split("$")
        if algorithm != "pbkdf2_sha256":
            return False
        derived = hashlib.pbkdf2_hmac(
            _PBKDF2_ALGORITHM, password.encode("utf-8"), bytes.fromhex(salt), int(iterations)
        )
        return hmac.compare_digest(derived.hex(), hash_hex)
    except (ValueError, AttributeError):
        return False


def create_access_token(*, user_id: int, email: str, expires_minutes: int) -> str:
    settings = get_settings()
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id),
        "email": email,
        "iat": now,
        "exp": now + timedelta(minutes=expires_minutes),
    }
    return jwt.encode(payload, settings.secret_key, algorithm="HS256")


def decode_access_token(token: str) -> dict[str, Any]:
    """Raises jwt.ExpiredSignatureError / jwt.InvalidTokenError on failure - callers handle these."""
    settings = get_settings()
    return jwt.decode(token, settings.secret_key, algorithms=["HS256"])
```

backend/app/database/__init__.py

1 lines

backend/app/database/cleanup_dummy_data.py

255 lines

"""One-time cleanup for the dev database (backend/data/app.db): delete every account except the ONE you name, along with everything only the other accounts touched - the seeded demo accounts (seed_demo.py), seed_bulk.py's bot accounts, seed.py's background demo user, and anything you clicked around and created through the app under any other login. What's left afterward is exactly the account you named and whatever it created.

This is a different, broader tool than cleanup_test_users.py, which only ever targets pytest's own known email patterns (accounts like "ai-<hex>@example.com") and leaves every real account - demo or otherwise - alone. This one flips that around: you name the one account to keep, and every other account is gone, regardless of how it got there.

Runs in passes:

1. Delete every change request NOT created by the kept account. Each model's own SQLAlchemy relationship cascade (see app/models/change_request.py) takes its versions, history, analyses (and everything under an analysis - requirements, risks, test cases, security findings, implementation tasks, clarification questions, etc.), assignments, approvals, and comments down with it; this script only additionally clears the notifications that pointed at that change request, since Notification isn't one of those cascading relationships.
2. Scrub any lingering reference to a non-kept account from what the kept account's OWN change requests still hold - e.g. another account's comment, assignment, or approval on the kept account's CR, or as the reviewer/editor of one of its analysis's requirements/test cases/security findings/implementation tasks. A record that requires a valid user to exist (a comment, an assignment, an approval) is deleted outright; a record that only references one optionally (history's actor, a version's "changed by", a reviewed/edited-by field) has just that reference cleared, so the CR's own content and audit trail stay intact.
3. Delete every remaining non-kept account, after clearing their own outgoing notifications and any stray optional reference to them elsewhere in the database (a knowledge document they uploaded, a repository scan they triggered, an admin audit log entry, a system setting they last changed).

Usage (from inside backend/):

```
python -m app.database.cleanup_dummy_data --keep-email test@alight.com # dry run - report only, deletes nothing
python -m app.database.cleanup_dummy_data --keep-email test@alight.com --apply # actually delete everything else
```

```
"""
from __future__ import annotations
```

```
import argparse
```

```
from sqlalchemy import func, or_
from sqlalchemy.orm import Session
```

```
from app.database.session import SessionLocal, engine
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.change_request_history import ChangeRequestHistory
from app.models.change_request_version import ChangeRequestVersion
from app.models.comment import ChangeRequestComment
from app.models.implementation_task import ImplementationTask
from app.models.knowledge_document import KnowledgeDocument
from app.models.notification import Notification
from app.models.repository_scan import RepositoryScan
from app.models.requirement import Requirement
from app.models.security_finding import SecurityFinding
from app.models.system_audit_log import SystemAuditLog
from app.models.system_setting import SystemSetting
from app.models.test_case import TestCase
from app.models.user import User
```

```
def find_keep_user(db: Session, email: str) -> User:
    # Case-insensitive - registration doesn't normalize casing, and typing
    # the email slightly differently than you signed up with shouldn't
    # accidentally match nothing (or, far worse, nothing and then this
    # script has no "keep" account at all).
    user = db.query(User).filter(func.lower(User.email) == email.strip().lower()).first()
    if user is None:
        raise SystemExit(
            f"No account found with email {email!r}. Nothing was touched - "
            "double check the email against what you actually registered with."
        )
    return user
```

```
def cleanup_foreign_change_requests(db: Session, keep_user_id: int, apply: bool) -> None:
    crs = db.query(ChangeRequest).filter(ChangeRequest.created_by != keep_user_id).all()
    if not crs:
        print("No change requests to remove - everything left is already the kept account's own.\n")
        return

    verb = "Deleting" if apply else "Would delete"
    print(
        f"{verb} {len(crs)} change request(s) not created by the kept account (and everything under them)"
        f"{' if apply else ' (dry run - nothing deleted yet)}':"
    )
    for cr in crs:
        print(f" - CR #{cr.id}: {cr.title!r}")

    if not apply:
        print()
        return

    deleted_notifications = 0
    for cr in crs:
        deleted_notifications += db.query(Notification).filter(Notification.change_request_id == cr.id).delete()
        db.delete(cr)
    db.commit()
    print(f"Deleted {len(crs)} change request(s) and {deleted_notifications} notification(s) referencing them.\n")
```

```
def scrub_foreign_references_on_kept_data(db: Session, keep_user_id: int, foreign_ids: set[int], apply: bool) -> None:
    """After pass 1, the only change requests left are the kept account's
    own - but something on one of them can still point at an account
    that's about to be deleted (a comment, an assignment, an approval, a
    reviewer/editor field). Handled here so pass 3 never hits a
    foreign-key error, and so a NOT NULL column is never left dangling."""
    verb = "Would remove" if not apply else "Removing"

    comments = db.query(ChangeRequestComment).filter(ChangeRequestComment.user_id.in_(foreign_ids))
    n = comments.count()
    if n:
        print(f"{verb} {n} comment(s) on your own change request(s) authored by an account being deleted.")
```

```

        if apply:
            comments.delete(synchronize_session=False)

    assignments = db.query(ChangeRequestAssignment).filter(
        or_(ChangeRequestAssignment.user_id.in_(foreign_ids), ChangeRequestAssignment.assigned_by.in_(foreign_ids))
    )
    n = assignments.count()
    if n:
        print(f"{verb} {n} assignment(s) involving an account being deleted.")
        if apply:
            assignments.delete(synchronize_session=False)

    approvals = db.query(Approval).filter(
        or_(Approval.approver_id.in_(foreign_ids), Approval.requested_by.in_(foreign_ids))
    )
    n = approvals.count()
    if n:
        print(f"{verb} {n} approval(s) involving an account being deleted.")
        if apply:
            approvals.delete(synchronize_session=False)

    history = db.query(ChangeRequestHistory).filter(ChangeRequestHistory.user_id.in_(foreign_ids))
    n = history.count()
    if n:
        print(
            f"{verb} the actor on {n} history/audit event(s) - the events themselves "
            "are kept, just no longer attributed to a deleted account."
        )
        if apply:
            history.update({ChangeRequestHistory.user_id: None}, synchronize_session=False)

    versions = db.query(ChangeRequestVersion).filter(ChangeRequestVersion.changed_by.in_(foreign_ids))
    n = versions.count()
    if n:
        print(f"{verb} reattributing {n} version snapshot(s) to the kept account (changed_by can't be empty).")
        if apply:
            versions.update({ChangeRequestVersion.changed_by: keep_user_id}, synchronize_session=False)

    for model, column, label in [
        (SecurityFinding, SecurityFinding.reviewed_by, "security finding reviewer"),
        (TestCase, TestCase.edited_by, "test case editor"),
        (Requirement, Requirement.reviewed_by, "requirement reviewer"),
        (ImplementationTask, ImplementationTask.edited_by, "implementation task editor"),
    ]:
        q = db.query(model).filter(column.in_(foreign_ids))
        n = q.count()
        if n:
            print(f"{verb} the {label} field on {n} row(s) referencing an account being deleted.")
            if apply:
                q.update({column: None}, synchronize_session=False)

    if apply:
        db.commit()
    print()

def _clear_stray_optional_references(db: Session, foreign_ids: set[int], apply: bool) -> None:
    """Optional References to a soon-to-be-deleted account that live
    outside the change-request world entirely. The row itself (a
    knowledge document, a repo scan, an audit log entry, a system
    setting) is real content worth keeping even once whoever touched it
    is gone, so this only clears the reference, never the row."""
    verb = "Would clear" if not apply else "Clearing"
    for model, column, label in [
        (KnowledgeDocument, KnowledgeDocument.uploaded_by, "knowledge document uploader"),
        (RepositoryScan, RepositoryScan.triggered_by, "repository scan trigger"),
        (SystemAuditLog, SystemAuditLog.actor_user_id, "audit log actor"),
        (SystemSetting, SystemSetting.updated_by, "system setting editor"),
    ]:
        q = db.query(model).filter(column.in_(foreign_ids))
        n = q.count()
        if n:
            print(f"{verb} the {label} field on {n} row(s).")
            if apply:
                q.update({column: None}, synchronize_session=False)
    if apply:
        db.commit()
    print()

def _cleanup_foreign_users(db: Session, keep_user_id: int, apply: bool) -> None:
    foreign_users = db.query(User).filter(User.id != keep_user_id).all()
    if not foreign_users:
        print("No other accounts exist - nothing left to delete.")
        return

    foreign_ids = {u.id for u in foreign_users}
    _scrub_foreign_references_on_kept_data(db, keep_user_id, foreign_ids, apply)
    _clear_stray_optional_references(db, foreign_ids, apply)

    verb = "Deleting" if apply else "Would delete"
    print(f"{verb} {len(foreign_users)} account(s){' ' if apply else ' (dry run - nothing deleted yet)'}:")
    for user in foreign_users:
        print(f"  - #{user.id} {user.name} <{user.email}>")

    if not apply:
        print("\nRun again with --apply to actually delete these.")
        return

    deleted_notifications = 0
    for user in foreign_users:
        deleted_notifications += db.query(Notification).filter(Notification.user_id == user.id).delete()
        db.delete(user)
    db.commit()
    print(f"\nDone - deleted {len(foreign_users)} account(s) and {deleted_notifications} notification(s) sent to them.")

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument(
        "--keep-email", required=True, help="Email of the ONE account to keep - every other account is removed."
    )
    parser.add_argument("--apply", action="store_true", help="Actually delete data (default: dry run only).")
    args = parser.parse_args()

    print(f"Database: {engine.url}\n")

    db = SessionLocal()
    try:
        keep_user = _find_keep_user(db, args.keep_email)
        print(f"Keeping account: #{keep_user.id} {keep_user.name} <{keep_user.email}>\n")

        _cleanup_foreign_change_requests(db, keep_user.id, args.apply)
        _cleanup_foreign_users(db, keep_user.id, args.apply)
    finally:
        db.close()

if __name__ == "__main__":
    main()

```



```

"""Database initialization - creates all tables if they don't already exist,
and adds any columns a model gained since its table was first created.

Hackathon-simple: no migration framework (Alembic etc.). Base.metadata.
create_all() is idempotent and handles brand-new tables, but it does NOT add
columns to a table that already exists - so when a later module adds a
field to an existing model (e.g. ChangeRequest.target_system in Module 4),
the real SQLite file on disk needs those columns added too, or the app
breaks against anyone's existing database. _add_missing_columns() below
closes that one gap with plain ALTER TABLE statements, safe to run on every
startup.

Called automatically on API startup (see app/main.py), and can also be run
directly:

    python -m app.database.init_db
"""
import json

from sqlalchemy import inspect, text
from sqlalchemy.orm import Session

from app import models  # noqa: F401 - importing registers every model on Base.metadata
from app.database.session import Base, engine

def _add_missing_columns() -> None:
    inspector = inspect(engine)
    existing_tables = set(inspector.get_table_names())

    with engine.begin() as conn:
        for table in Base.metadata.sorted_tables:
            if table.name not in existing_tables:
                continue # brand new table - create_all() just created it with every column

            existing_columns = {col["name"] for col in inspector.get_columns(table.name)}
            for column in table.columns:
                if column.name in existing_columns:
                    continue
                column_type = column.type.compile(dialect=engine.dialect)
                conn.execute(text(f'ALTER TABLE "{table.name}" ADD COLUMN "{column.name}" {column_type}'))
                print(f" (migrated) added column: {table.name}.{column.name}")

def _backfill_workflow_data() -> None:
    """Module 12 (Enterprise Workflow): give every pre-existing row a
    sensible starting point instead of leaving the new columns NULL.

    - Every ChangeRequest with no current version gets current version=1,
      a Version 1 snapshot (change summary "Initial version imported."),
      and a matching CREATED history event - a real, honest baseline
      rather than inventing fake prior user actions (spec section 35: "do
      not invent fake user actions").
    - Every Analysis with no change_request version is associated with
      version 1 (the best-effort baseline - there's no way to know which
      historical CR version an analysis older than this column actually
      ran against).

    Idempotent: every check is "IS NULL", so this is a no-op on every
    startup after the first time it runs on a given database. It's also
    guarded per-row against a version-1 / CREATED row that already exists -
    since Module 12's create_change_request() now creates those itself for
    every brand-new change request, this only ever fills in what's
    genuinely missing (a real pre-Module-12 row) rather than ever
    duplicating what creation already wrote.
    """
    from app.models.analysis import Analysis
    from app.models.change_request import ChangeRequest
    from app.models.change_request_history import ChangeRequestHistory
    from app.models.change_request_version import ChangeRequestVersion
    from app.models.enums import HistoryAction
    from app.services.workflow_rules import build_snapshot

    with Session(engine) as session:
        change_requests = (
            session.query(ChangeRequest).filter(ChangeRequest.current_version.is_(None)).all()
        )
        for change_request in change_requests:
            change_request.current_version = 1

            has_version_1 = (
                session.query(ChangeRequestVersion)
                .filter(
                    ChangeRequestVersion.change_request_id == change_request.id,
                    ChangeRequestVersion.version_number == 1,
                )
                .first()
                is not None
            )
            if not has_version_1:
                session.add(
                    ChangeRequestVersion(
                        change_request_id=change_request.id,
                        version_number=1,
                        changed_by=change_request.created_by,
                        created_at=change_request.created_at,
                        change_summary="Initial version imported.",
                        snapshot=build_snapshot(change_request),
                    )
                )

            has_created_event = (
                session.query(ChangeRequestHistory)
                .filter(
                    ChangeRequestHistory.change_request_id == change_request.id,
                    ChangeRequestHistory.action == HistoryAction.CREATED,
                )
                .first()
                is not None
            )
            if not has_created_event:
                session.add(
                    ChangeRequestHistory(
                        change_request_id=change_request.id,
                        user_id=change_request.created_by,
                        action=HistoryAction.CREATED,
                        reason=None,
                        version_number=1,
                        created_at=change_request.created_at,
                    )
                )

    if change_requests:
        print(f" (migrated) backfilled version 1 for {len(change_requests)} existing change request(s)")

```

```

analyses = session.query(Analysis).filter(Analysis.change_request_version.is_(None)).all()
for analysis in analyses:
    analysis.change_request_version = 1
if analyses:
    print(f" (migrated) backfilled analysis version for {len(analyses)} existing analysis row(s)")

session.commit()

def _seed_role_permissions() -> None:
    """Module 21: give every (role, capability) pair a real row, seeded
    from app/services/permissions.py::DEFAULT_PERMISSIONS - the exact
    values the app already behaved with before this module existed (see
    that module's own docstring). Only ever INSERTs a pair that has no row
    yet; an admin's later edit through PUT /api/admin/permissions is never
    overwritten by a later server restart. """
    from app.models.role_permission import RolePermission
    from app.services.permissions import DEFAULT_PERMISSIONS

    with Session(engine) as session:
        existing = {(row.role, row.capability) for row in session.query(RolePermission).all()}
        added = 0
        for (role, capability), allowed in DEFAULT_PERMISSIONS.items():
            if (role, capability) in existing:
                continue
            session.add(RolePermission(role=role, capability=capability, allowed=allowed))
            added += 1
        if added:
            session.commit()
            print(f" (migrated) seeded {added} role permission default(s)")

def _seed_approval_rules() -> None:
    """Module 21 spec section 4: seed the admin-editable approval rules
    table from workflow_rules.DEFAULT_APPROVAL_RULE_SEEDS - the exact
    risk/category -> approval-type mapping the app already used before
    this module existed (see that constant's own docstring). Guarded by
    "only if the table is completely empty," not a per-row check like
    _seed_role_permissions - these rows are meant to be freely added to,
    edited, and DELETED by an admin, and a per-row re-seed would silently
    resurrect a rule an admin deliberately deleted."""
    import json

    from app.models.approval_rule import ApprovalRule
    from app.models.enums import ApprovalRuleType
    from app.services.workflow_rules import DEFAULT_APPROVAL_RULE_SEEDS

    with Session(engine) as session:
        if session.query(ApprovalRule).first() is not None:
            return
        for rule_type, match_value, approval_types in DEFAULT_APPROVAL_RULE_SEEDS:
            session.add(
                ApprovalRule(
                    rule_type=ApprovalRuleType(rule_type),
                    match_value=match_value,
                    approval_types=json.dumps(approval_types),
                    enabled=True,
                )
            )
        session.commit()
        print(f" (migrated) seeded {len(DEFAULT_APPROVAL_RULE_SEEDS)} default approval rule(s)")

def init_db() -> None:
    Base.metadata.create_all(bind=engine)
    add_missing_columns()
    backfill_workflow_data()
    _seed_role_permissions()
    _seed_approval_rules()

if __name__ == "__main__":
    init_db()
    print("Database initialized (tables created, and any new columns added, as needed).")

```

backend/app/database/seed.py

201 lines

```
"""Optional demo data - NOT run automatically on startup. Handy for manually
poking at the database (or the dashboard) without waiting on the AI module
to exist. Safe to run more than once - skips seeding if the demo user
already exists.

Run with (from backend/):

python -m app.database.seed
"""
from datetime import datetime, timedelta

from app.database.init_db import init_db
from app.database.session import SessionLocal
from app.models import Analysis, ChangeRequest, ClarificationQuestion, User
from app.models.enums import ApprovalRecommendation, ChangeRequestStatus, ComplexityLevel, Priority, UserRole

DEMO_EMAIL = "demo@alight.com"

def _days_ago(days: int) -> datetime:
    return datetime.utcnow() - timedelta(days=days)

def seed_demo_data() -> None:
    init_db()
    db = SessionLocal()
    try:
        existing = db.query(User).filter(User.email == DEMO_EMAIL).first()
        if existing:
            print("Demo data already present - skipping.")
            return

        demo_user = User(
            name="Demo User",
            email=DEMO_EMAIL,
            password_hash="not-a-real-hash", # placeholder - not meant to be logged into
            role=UserRole.ENGINEER,
        )
        db.add(demo_user)
        db.flush() # assigns demo_user.id without committing yet

        # A spread of requests across statuses, priorities, and (for most of
        # them) an analysis with a different risk/category/complexity - gives
        # the dashboard real variety to render instead of all-zero charts.
        change_requests = [
            ChangeRequest(
                title="Add dark mode to the POS dashboard",
                description=(
                    "Staff have asked for a dark theme option in the POS dashboard "
                    "to reduce eye strain during night shifts."
                ),
                business_objective="Improve staff comfort and reduce eye strain during night shifts.",
                priority=Priority.MEDIUM,
                status=ChangeRequestStatus.DRAFT,
                created_by=demo_user.id,
                created_at=_days_ago(1),
                updated_at=_days_ago(1),
            ),
            ChangeRequest(
                title="Migrate payment processing to new gateway",
                description="Replace the current payment gateway integration with the new provider's API.",
                business_objective="Lower per-transaction processing fees.",
                priority=Priority.HIGH,
                status=ChangeRequestStatus.IN_REVIEW,
                created_by=demo_user.id,
                created_at=_days_ago(6),
                updated_at=_days_ago(2),
            ),
            ChangeRequest(
                title="Fix inventory count rounding bug",
                description="Fractional inventory counts are rounding incorrectly on the stock report.",
                business_objective="Ensure stock reports match actual inventory.",
                priority=Priority.LOW,
                status=ChangeRequestStatus.APPROVED,
                created_by=demo_user.id,
                created_at=_days_ago(10),
                updated_at=_days_ago(9),
            ),
            ChangeRequest(
                title="Add role-based access control to admin panel",
                description="Restrict admin panel sections by user role instead of a single admin flag.",
                business_objective="Reduce risk of accidental or unauthorized changes by staff.",
                priority=Priority.HIGH,
                status=ChangeRequestStatus.SUBMITTED,
                created_by=demo_user.id,
                created_at=_days_ago(3),
                updated_at=_days_ago(3),
            ),
            ChangeRequest(
                title="Upgrade PostgreSQL to v16",
                description="Upgrade the production database engine to the latest major version.",
                business_objective="Stay on a supported database version.",
                priority=Priority.MEDIUM,
                status=ChangeRequestStatus.REJECTED,
                created_by=demo_user.id,
                created_at=_days_ago(14),
                updated_at=_days_ago(12),
            ),
            ChangeRequest(
                title="Add barcode scanner API endpoint",
                description="Expose an endpoint the handheld scanner hardware can call directly.",
                business_objective="Speed up receiving workflow.",
                priority=Priority.MEDIUM,
                status=ChangeRequestStatus.IMPLEMENTED,
                created_by=demo_user.id,
                created_at=_days_ago(20),
                updated_at=_days_ago(15),
            ),
        ],
        db.add_all(change_requests)
        db.flush() # assigns ids without committing yet

        payment_cr, inventory_cr, rbac_cr, postgres_cr, barcode_cr = change_requests[1:]

        analyses = [
            Analysis(
                change_request_id=payment_cr.id,
                summary="High-impact integration change touching payment flow and third-party API.",
                category="Integration",
                complexity=ComplexityLevel.HIGH,
                risk_score=82.0,
                confidence_score=74.0,
                recommendation=ApprovalRecommendation.NEEDS_MORE_INFO,
                created_at=_days_ago(2),
            )
        ]
```

```

    ),
    Analysis(
        change_request_id=inventory_cr.id,
        summary="Small, well-contained rounding fix in one report calculation.",
        category="Bug Fix",
        complexity=ComplexityLevel.LOW,
        risk_score=12.0,
        confidence_score=91.0,
        recommendation=ApprovalRecommendation.APPROVE,
        created_at=_days_ago(9),
    ),
    Analysis(
        change_request_id=rbac_cr.id,
        summary="Access-control change with meaningful security surface area.",
        category="Security",
        complexity=ComplexityLevel.HIGH,
        risk_score=61.0,
        confidence_score=68.0,
        recommendation=ApprovalRecommendation.APPROVE_WITH_CONDITIONS,
        created_at=_days_ago(3),
    ),
    Analysis(
        change_request_id=postgres_cr.id,
        summary="Infrastructure upgrade with moderate compatibility risk.",
        category="Infrastructure",
        complexity=ComplexityLevel.MEDIUM,
        risk_score=45.0,
        confidence_score=70.0,
        recommendation=ApprovalRecommendation.REJECT,
        created_at=_days_ago(12),
    ),
    Analysis(
        change_request_id=barcode_cr.id,
        summary="New, narrowly-scoped API endpoint with existing auth reused.",
        category="API",
        complexity=ComplexityLevel.MEDIUM,
        risk_score=33.0,
        confidence_score=80.0,
        recommendation=ApprovalRecommendation.APPROVE,
        created_at=_days_ago(15),
    ),
]
db.add_all(analyses)
db.flush()

payment_analysis = analyses[0]
db.add_all(
    [
        ClarificationQuestion(
            analysis_id=payment_analysis.id,
            question="Which fallback happens if the new gateway's API is unreachable mid-transaction?",
            priority=Priority.HIGH,
            reason="Payment failures need a defined fallback before this can be approved.",
            resolved=False,
        ),
        ClarificationQuestion(
            analysis_id=payment_analysis.id,
            question="Has the new gateway been confirmed PCI-DSS compliant?",
            priority=Priority.HIGH,
            reason="Compliance must be confirmed for anything touching card data.",
            resolved=True,
        ),
    ]
)

db.commit()
print(
    f"Seeded demo user (id={demo_user.id}), {len(change_requests)} change requests, "
    f"{len(analyses)} analyses, and 2 clarification questions."
)
finally:
    db.close()

if __name__ == "__main__":
    seed_demo_data()

```

backend/app/database/seed_bulk.py

194 lines

```
"""Bulk synthetic data generator - NOT run automatically. Creates thousands
of realistic-but-clearly-fake change requests (with analyses and
clarification questions) so the dashboard's metrics, risk buckets, category
breakdown, and recent-requests list can all be stress-tested against real
volume - not just the handful of rows seed.py adds.

Every synthetic row is created by one of 5 "Load Test Bot" users
(loadtest1@alight.com .. loadtest5@alight.com) so it's obvious in the data
which rows are load-test filler versus real accounts like yours.

Safe to run more than once - skips (and does nothing) if the bulk synthetic
users already exist. To reset and regenerate, stop the backend and delete
backend/data/app.db, then run init_db again.

Run with (from backend/):

    python -m app.database.seed_bulk          # 3000 change requests
    python -m app.database.seed_bulk 5000    # custom count

This can take anywhere from ~15 seconds to a couple of minutes depending on
your machine - it prints progress every 500 rows so you can see it's working.
"""
import random
import sys
from datetime import datetime, timedelta

from app.database.init_db import init_db
from app.database.session import SessionLocal
from app.models import Analysis, ChangeRequest, ClarificationQuestion, User
from app.models.enums import (
    ApprovalRecommendation,
    ChangeRequestStatus,
    ComplexityLevel,
    Priority,
    UserRole,
)

DEFAULT_COUNT = 3000
CHUNK_SIZE = 500

BULK_USER_EMAILS = [f"loadtest{i}@alight.com" for i in range(1, 6)]

VERBS = [
    "Add", "Update", "Fix", "Remove", "Refactor", "Migrate", "Optimize",
    "Integrate", "Deprecate", "Harden", "Redesign", "Simplify", "Automate",
    "Rebuild", "Extend", "Consolidate", "Streamline",
]
SUBJECTS = [
    "checkout flow", "inventory sync", "user authentication", "reporting dashboard",
    "payment gateway", "loyalty program", "barcode scanner integration", "admin panel",
    "notification service", "search indexing", "tax calculation", "refund workflow",
    "kiosk UI", "printer integration", "menu management", "order queue",
    "table management", "employee scheduling", "customer profiles", "email templates",
    "delivery tracking", "gift card system", "multi-location sync", "offline mode",
    "receipt printing", "self-checkout flow", "franchise reporting", "API rate limiting",
]

CANONICAL_CATEGORIES = ["Feature", "Bug Fix", "Security", "Database", "API", "Infrastructure", "Integration"]
UNUSUAL_CATEGORIES = ["Documentation", "UX Research", "Performance Tuning", "Localization", "Cleanup"]

QUESTION_TEMPLATES = [
    "What is the rollback plan if this fails in production?",
    "Has this been tested against the largest customer's data volume?",
    "Does this require a database migration, and if so, is it reversible?",
    "Are there any third-party dependencies this introduces?",
    "What is the expected impact on existing API consumers?",
    "Has security reviewed the data this change touches?",
]

def _random_datetime_within(days_back: int) -> datetime:
    return datetime.utcnow() - timedelta(days=random.uniform(0, days_back), hours=random.uniform(0, 23))

def seed_bulk_data(count: int = DEFAULT_COUNT) -> None:
    init_db()
    db = SessionLocal()
    try:
        existing = db.query(User).filter(User.email == BULK_USER_EMAILS[0]).first()
        if existing:
            print(
                "Bulk synthetic data already present - skipping. "
                "(Delete backend/data/app.db and restart the backend to reset everything.)"
            )
            return
        users = [
            User(name=f"Load Test Bot {i}", email=email, password_hash="not-a-real-hash", role=UserRole.ENGINEER)
            for i, email in enumerate(BULK_USER_EMAILS, start=1)
        ]
        db.add_all(users)
        db.flush()
        user_ids = [u.id for u in users]

        statuses = list(ChangeRequestStatus)
        priorities = list(Priority)
        complexities = list(ComplexityLevel)
        recommendations = list(ApprovalRecommendation)

        total_analyses = 0
        total_questions = 0

        for start in range(0, count, CHUNK_SIZE):
            batch_end = min(start + CHUNK_SIZE, count)
            batch_change_requests = []

            for _ in range(start, batch_end):
                verb = random.choice(VERBS)
                subject = random.choice(SUBJECTS)
                created_at = _random_datetime_within(365)
                cr = ChangeRequest(
                    title=f"{verb} {subject}",
                    description=(
                        f"{verb} {subject} to improve reliability, performance, or usability "
                        f"across the platform. (Synthetic load-test data.)"
                    ),
                    business_objective=f"Support the {subject} initiative.",
                    priority=random.choice(priorities),
                    status=random.choice(statuses),
                    created_by=random.choice(user_ids),
                    created_at=created_at,
                    updated_at=created_at,
                )
                batch_change_requests.append(cr)

            batch_change_requests.append(cr)
```

```

db.add_all(batch_change_requests)
db.flush() # assigns ids for the FKs below

batch_analyses = []
for cr in batch_change_requests:
    # ~80% of requests have been analyzed at least once; the rest
    # stay "pending analysis" - both states need real coverage.
    if random.random() >= 0.8:
        continue

    num_analyses = 2 if random.random() < 0.1 else 1 # ~10% re-analyzed
    analysis_time = cr.created_at
    for _ in range(num_analyses):
        analysis_time = analysis_time + timedelta(hours=random.uniform(1, 72))
        category = (
            random.choice(CANONICAL_CATEGORIES)
            if random.random() < 0.85
            else random.choice(UNUSUAL_CATEGORIES)
        )
        recommendation = random.choice(recommendations) if random.random() < 0.8 else None

        batch_analyses.append(
            Analysis(
                change_request_id=cr.id,
                summary=f"Synthetic analysis of: {cr.title}.",
                category=category,
                complexity=random.choice(complexities),
                risk_score=round(random.uniform(0, 100), 1),
                confidence_score=round(random.uniform(40, 99), 1),
                recommendation=recommendation,
                created_at=analysis_time,
            )
        )

db.add_all(batch_analyses)
db.flush() # assigns ids for the clarification questions below
total_analyses += len(batch_analyses)

batch_questions = []
for analysis in batch_analyses:
    num_questions = random.choices([0, 1, 2, 3], weights=[40, 30, 20, 10])[0]
    for _ in range(num_questions):
        batch_questions.append(
            ClarificationQuestion(
                analysis_id=analysis.id,
                question=random.choice(QUESTION_TEMPLATES),
                priority=random.choice(priorities),
                reason="Needs confirmation before this can be safely approved.",
                resolved=random.random() < 0.5,
            )
        )

db.add_all(batch_questions)
total_questions += len(batch_questions)

db.commit()
print(f"...inserted {batch_end}/{count} change requests so far")

print(
    f"Done. Added {count} change requests, {total_analyses} analyses, and "
    f"{total_questions} clarification questions, owned by 5 synthetic 'Load Test Bot' users."
)
finally:
    db.close()

if __name__ == "__main__":
    requested_count = int(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_COUNT
    seed_bulk_data(requested_count)

```

```

"""Module 23: hackathon demo data.

Distinct from `seed.py` (background-only, unloggable-into demo user) and
`seed.bulk.py` (3000 anonymous bot accounts for dashboard-at-scale load
testing) - this script's job is different: it creates 5 REAL, LOGGABLE-IN
demo accounts covering this app's five practical roles, one flagship change
request ("Add OTP-based authentication for customers") left deliberately
FRESH and unanalyzed so a presenter runs its AI analysis live, and a handful
of supporting change requests spanning different statuses/priorities/risks
so the Dashboard, CR list, Analytics, and Admin sections all have real
variety to show - see DEMO.md for the actual presentation script.

Every event this script writes (change requests, versions, audit history,
approvals, assignments, comments, notifications) is created through this
project's own real service-layer functions wherever one exists
(app/services/history.py, notify.py, approvals.py, comments.py,
versioning.py) - the exact same code path the live app itself uses - so
none of it is a fabricated shortcut. The one place this script deliberately
does NOT go through the live app's own rules is setting a change request's
`status` field directly rather than through the guarded PUT .../status
endpoint - the same convention `seed.py`/`seed.bulk.py` already use for
background data, since there's no "seed this CR straight into Approved"
service function and there doesn't need to be one.

Safe to run more than once - skips seeding entirely if the demo Admin
account already exists.

Run with (from backend/):

    python -m app.database.seed_demo

Demo credentials (see DEMO.md for the full table): every seeded demo
account shares the password below. These are local, throwaway demo
credentials only - never real ones.
"""
from datetime import datetime, timedelta

from app.core.security import hash_password
from app.database.init_db import init_db
from app.database.session import SessionLocal
from app.models import (
    AffectedComponent,
    Analysis,
    ChangeRequest,
    ChangeRequestVersion,
    Dependency,
    ImpactAssessment,
    ImplementationTask,
    Requirement,
    Risk,
    SecurityFinding,
    TestCase,
    User,
)
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.enums import (
    ApprovalRecommendation,
    ApprovalStatus,
    ApprovalType,
    AssignmentRole,
    Certainty,
    ChangeRequestStatus,
    ComplexityLevel,
    ComponentType,
    DependencyType,
    HistoryAction,
    ImpactCategory,
    ImpactLevel,
    NotificationType,
    Priority,
    RequirementType,
    RiskCategory,
    SecurityCategory,
    SecurityFindingStatus,
    Severity,
    TestType,
    UserRole,
)
from app.services import approvals as approvals_service
from app.services import comments as comments_service
from app.services import history as history_service
from app.services import notify as notify_service
from app.services import versioning
from app.services import workflow_rules

DEMO_PASSWORD = "ALightDemo1231" # shared by every seeded demo account - local/throwaway only
ADMIN_EMAIL = "admin@alight.com"

def days_ago(days: int, hours: int = 0) -> datetime:
    return datetime.utcnow() - timedelta(days=days, hours=hours)

def _make_user(db, *, name: str, email: str, role: UserRole) -> User:
    user = User(name=name, email=email, password_hash=hash_password(DEMO_PASSWORD), role=role)
    db.add(user)
    db.flush()
    return user

def _create_cr(db, creator: User, *, created_at: datetime, status: ChangeRequestStatus, **fields) -> ChangeRequest:
    """Mirrors app/api/change_requests.py::create_change_request exactly -
    a version 1 snapshot plus a real CREATED history event - so every
    seeded change request has an honest audit trail from the moment it
    exists, not just a bare row with no history behind it."""
    cr = ChangeRequest(
        created_by=creator.id,
        status=status,
        current_version=1,
        created_at=created_at,
        updated_at=created_at,
        **fields,
    )
    db.add(cr)
    db.flush()
    db.add(
        ChangeRequestVersion(
            change_request_id=cr.id,
            version_number=1,
            changed_by=creator.id,
            change_summary="Change request created.",
            snapshot=workflow_rules.build_snapshot(cr),
            created_at=created_at,
        )
    )

```

```

    )
    event = history_service.record_event(
        db,
        change_request_id=cr.id,
        action=HistoryAction.CREATED,
        user_id=creator.id,
        version_number=1,
    )
    event.created_at = created_at
    return cr

def transition_status(
    db, cr: ChangeRequest, actor: User, new_status: ChangeRequestStatus, *, at: datetime, reason: str | None = None
) -> None:
    """Mirrors app/api/change_requests.py::change_status's own history +
    notification side effects exactly (minus the legality/permission
    checks, which don't apply to seed data - same convention seed.py /
    seed bulk.py already use for setting status directly)."""
    old_label = workflow_rules.STATUS_LABELS.get(cr.status, cr.status.value)
    new_label = workflow_rules.STATUS_LABELS.get(new_status, new_status.value)
    cr.status = new_status
    event = history_service.record_event(
        db,
        change_request_id=cr.id,
        action=HistoryAction.STATUS_CHANGED,
        user_id=actor.id,
        old_value=old_label,
        new_value=new_label,
        reason=reason,
        version_number=cr.current_version or 1,
    )
    event.created_at = at
    interested_ids = {cr.created_by} | {a.user_id for a in cr.assignments}
    interested_ids.discard(actor.id)
    for user_id in interested_ids:
        notify_service.notify(
            db,
            user_id=user_id,
            type=NotificationType.STATUS_CHANGED,
            title=f"CR-{cr.id} moved to {new_label}",
            message=f'{actor.name} moved "{cr.title}" to {new_label}.',
            change_request_id=cr.id,
        )

def assign(db, cr: ChangeRequest, *, target: User, role: AssignmentRole, assigned_by: User, at: datetime) -> None:
    """Mirrors the real POST .../assignments endpoint's own history +
    notification side effects."""
    db.add(ChangeRequestAssignment(change_request_id=cr.id, user_id=target.id, role=role, assigned_by=assigned_by.id))
    db.flush()
    role_label = workflow_rules.ROLE_LABELS.get(role, role.value)
    event = history_service.record_event(
        db,
        change_request_id=cr.id,
        action=HistoryAction.ASSIGNED,
        user_id=assigned_by.id,
        new_value=f'{target.name} as {role_label}',
        version_number=cr.current_version or 1,
    )
    event.created_at = at
    if target.id != assigned_by.id:
        notify_service.notify(
            db,
            user_id=target.id,
            type=NotificationType.ASSIGNED,
            title=f"You were assigned to CR-{cr.id}",
            message=f'{assigned_by.name} assigned you as {role_label} on "{cr.title}".',
            change_request_id=cr.id,
        )

def request_approval(
    db, cr: ChangeRequest, *, approval_type: ApprovalType, approver: User, requested_by: User, at: datetime
):
    """Mirrors the real POST .../approvals endpoint's own notification."""
    approval = approvals_service.request_approval(
        db, cr, approval_type=approval_type, approver=approver, requested_by=requested_by
    )
    approval.requested_at = at
    type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(approval_type, approval_type.value)
    if approver.id != requested_by.id:
        notify_service.notify(
            db,
            user_id=approver.id,
            type=NotificationType.APPROVAL_REQUESTED,
            title=f'{type_label} approval requested on CR-{cr.id}',
            message=f'{requested_by.name} asked you for {type_label} approval on "{cr.title}".',
            change_request_id=cr.id,
        )
    return approval

def respond_approval(db, cr: ChangeRequest, approval, *, new_status: ApprovalStatus, comment: str, responder: User, at: datetime):
    """Mirrors the real POST .../approvals/{id}/respond endpoint's own
    notification."""
    approvals_service.respond_to_approval(db, cr, approval, new_status=new_status, comment=comment, responder=responder)
    approval.responded_at = at
    type_label = workflow_rules.APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
    response_type = {
        ApprovalStatus.APPROVED: NotificationType.APPROVED,
        ApprovalStatus.REJECTED: NotificationType.REJECTED,
        ApprovalStatus.CHANGES_REQUESTED: NotificationType.CHANGES_REQUESTED,
    }[new_status]
    if approval.requested_by != responder.id:
        status_label = workflow_rules.APPROVAL_STATUS_LABELS.get(new_status, new_status.value)
        notify_service.notify(
            db,
            user_id=approval.requested_by,
            type=response_type,
            title=f'{type_label} approval {status_label.lower()} on CR-{cr.id}',
            message=f'{responder.name} marked the {type_label} approval you requested as {status_label}.',
            change_request_id=cr.id,
        )

def comment(db, cr: ChangeRequest, *, author: User, body: str, at: datetime):
    """comments service.create_comment already records history and sends
    every notification (mentions + interested parties) - see its own
    docstring in app/services/comments.py."""
    comment = comments_service.create_comment(db, cr, author=author, body=body)
    comment.created_at = at
    return comment

def seed_analysis(
    db,
    cr: ChangeRequest,
    *,
    summary: str,
    category: str,

```



```

complexity: ComplexityLevel,
risk score: float,
confidence score: float,
recommendation: ApprovalRecommendation,
at: datetime,
requirements: list[dict],
risks: list[dict],
impacts: list[dict],
security_findings: list[dict],
test_cases: list[dict],
tasks: list[dict],
affected_components: list[dict] | None = None,
dependencies: list[dict] | None = None,
) -> Analysis:
    """Directly seeds an Analysis and its child rows for background/demo
    variety (no real AI call - this is clearly-labeled seed data, same
    spirit as the pre-existing seed.py). Mirrors
    app/services/analysis_engine.py's own two side effects after a real
    analysis completes: an AI ANALYSIS_COMPLETED history event attributed
    to the system actor "AI Analyzer" (never a human), and auto-advancing
    a Pending Analysis change request straight to Analyzed."""
    analysis = Analysis(
        change_request_id=cr.id,
        change_request_version=cr.current_version or 1,
        summary=summary,
        category=category,
        complexity=complexity,
        risk_score=risk_score,
        confidence_score=confidence_score,
        recommendation=recommendation,
        created_at=at,
    )
    db.add(analysis)
    db.flush()

    for r in requirements:
        db.add(Requirement(analysis_id=analysis.id, **r))
    for r in risks:
        db.add(Risk(analysis_id=analysis.id, **r))
    for i in impacts:
        db.add(ImpactAssessment(analysis_id=analysis.id, **i))
    for s in security_findings:
        db.add(SecurityFinding(analysis_id=analysis.id, **s))
    for t in test_cases:
        db.add(TestCase(analysis_id=analysis.id, **t))
    for t in tasks:
        db.add(ImplementationTask(analysis_id=analysis.id, **t))
    for c in affected_components or []:
        db.add(AffectedComponent(analysis_id=analysis.id, **c))
    for d in dependencies or []:
        db.add(Dependency(analysis_id=analysis.id, **d))

    event = history_service.record_event(
        db,
        change_request_id=cr.id,
        action=HistoryAction.AI_ANALYSIS_COMPLETED,
        actor_label="AI Analyzer",
        new_value=f"Risk score: {round(risk_score)/100}, Complexity: {complexity.value}",
        version_number=cr.current_version or 1,
    )
    event.created_at = at
    if cr.status == ChangeRequestStatus.PENDING_ANALYSIS:
        cr.status = ChangeRequestStatus.ANALYZED
    return analysis

def seed_demo_data() -> None:
    init_db()
    db = SessionLocal()
    try:
        existing = db.query(User).filter(User.email == ADMIN_EMAIL).first()
        if existing:
            print("Demo data already present - skipping.")
            return

        # --- 1. The five demo accounts (spec section 4) -----
        admin = _make_user(db, name="Ava Chen", email=ADMIN_EMAIL, role=UserRole.ADMIN)
        manager = _make_user(db, name="Jordan Blake", email="manager@alight.com", role=UserRole.PRODUCT_MANAGER)
        engineer = _make_user(db, name="Liam Carter", email="engineer@alight.com", role=UserRole.ENGINEER)
        security_reviewer = _make_user(
            db, name="Priya Nair", email="security@alight.com", role=UserRole.SECURITY_REVIEWER
        )
        requester = _make_user(db, name="Sofia Reyes", email="requester@alight.com", role=UserRole.REQUESTER)

        # --- 2. The flagship demo CR - fresh, unanalyzed (spec section 1) --
        # Deliberately NOT analyzed here: the presenter runs AI Analysis on
        # this one live, on stage, so the audience sees a real model call -
        # never a canned response standing in for one (spec section 7).
        otp_cr = _create_cr(
            db,
            requester,
            created_at=days_ago(0, hours=1),
            status=ChangeRequestStatus.PENDING_ANALYSIS,
            title="Add OTP-based authentication for customers",
            description=(
                "Add one-time-password (OTP) verification to customer login on the loyalty portal. "
                "The OTP is sent by SMS to the customer's registered mobile number and must be entered "
                "within a defined expiry window. Requirements: OTP sent to the registered mobile number; "
                "the OTP expires after a defined period; a maximum number of retry attempts before the "
                "code is invalidated; OTP values are never stored in plain text; rate limiting on both "
                "OTP requests and verification attempts to prevent abuse; and every OTP request, "
                "verification attempt, and outcome is written to the audit log."
            ),
            business_objective="Reduce account-takeover fraud on customer logins without adding a hard password-reset burden.",
            priority=Priority.HIGH,
            requested_by="Customer Experience team",
            target_system="Customer Loyalty Portal",
            customer_impact="Customers will see a new verification step at login when signing in from a new device.",
            compliance_requirements="Must not log or store the raw OTP value anywhere, including application logs.",
        )

        # --- 3. Supporting CRs (spec section 3: several CRs, different
        # statuses/priorities/risks, pending + completed + changes-requested
        # approvals, comments, notifications, version history, closed CRs) -

        # A: Analyzed, HIGH risk, a PENDING Technical approval, two versions
        # (a real seeded edit), an assignment, and a comment with an
        # @mention - the richest supporting CR, good for a quick detour if
        # a judge asks "show me another one."
        ledger_cr = _create_cr(
            db,
            requester,
            created_at=days_ago(8),
            status=ChangeRequestStatus.PENDING_ANALYSIS,
            title="Migrate loyalty points ledger to new database schema",
            description=(
                "Migrate the loyalty points ledger table to a new normalized schema to support "
                "multi-currency point balances. Requires a one-time backfill of existing balances "
                "and a brief read-only maintenance window during cutover."
            ),
            business_objective="Unblock multi-currency loyalty support requested by three regional teams.",
        )

```

```

priority=Priority.HIGH,
requested_by="Loyalty Platform team",
target_system="Loyalty Service",
technical_impact="Touches the production loyalty database directly during cutover.",
)
_seed_analysis(
db,
ledger_cr,
at=days_ago(8),
summary="Direct schema migration on the production loyalty ledger with a live cutover window.",
category="Database",
complexity=ComplexityLevel.HIGH,
risk_score=72.0,
confidence_score=76.0,
recommendation=ApprovalRecommendation.APPROVE_WITH_CONDITIONS,
requirements=[
dict(
requirement_type=RequirementType.FUNCTIONAL,
description="Existing point balances must be backfilled into the new schema with zero data loss.",
priority=Priority.HIGH,
certainty=Certainty.KNOWN,
confidence=90.0,
),
dict(
requirement_type=RequirementType.NON_FUNCTIONAL,
description="Cutover maintenance window must be read-only, not a full outage.",
priority=Priority.HIGH,
certainty=Certainty.INFERRED,
confidence=70.0,
),
],
risks=[
dict(
category=RiskCategory.DATA,
description="A failed backfill could corrupt or double-count customer point balances.",
severity=Severity.HIGH,
probability=0.3,
score=72.0,
mitigation="Run the backfill against a full production snapshot first and diff the totals before cutover.",
),
],
impacts=[
dict(category=ImpactCategory.TECHNICAL, impact_level=ImpactLevel.HIGH, description="Schema change touches the core ledger table.",
certainty=Certainty.KNOWN, confidence=90.0),
dict(category=ImpactCategory.DATA, impact_level=ImpactLevel.HIGH, description="Requires a full backfill of historical balances.",
certainty=Certainty.KNOWN, confidence=85.0),
dict(category=ImpactCategory.CUSTOMER, impact_level=ImpactLevel.MEDIUM, description="Brief read-only window during cutover.",
certainty=Certainty.INFERRED, confidence=65.0),
],
security_findings=[
dict(category=SecurityCategory.DATA_PROTECTION, finding="Backfill script will handle customer balance data at rest.", severity=Severity.MEDIUM,
evidence="Migration touches the loyalty ledger table directly.", recommendation="Run the backfill inside the existing encrypted database connection, no new
export step.", status=SecurityFindingStatus.OPEN),
dict(category=SecurityCategory.AUDIT_LOGGING, finding="Cutover should be logged as a distinct operational event.", severity=Severity.LOW,
evidence="No dedicated migration-event logging exists today.", recommendation="Log migration start/end and row counts to the ops audit log.",
status=SecurityFindingStatus.OPEN),
],
test_cases=[
dict(test_id="TC-101", title="Backfilled balances match pre-migration totals", test_type=TestType.INTEGRATION, priority=Priority.HIGH,
description="Sum balances before and after migration.", expected_result="Totals match exactly, per customer and in aggregate."),
dict(test_id="TC-102", title="Writes are rejected during the read-only window", test_type=TestType.INTEGRATION, priority=Priority.MEDIUM,
description="Attempt a balance write during cutover.", expected_result="Write is cleanly rejected, not silently dropped."),
],
tasks=[
dict(task="Write and dry-run the backfill script", description="Backfill existing balances into the new schema against a snapshot.",
component="Loyalty Service", priority=Priority.HIGH, estimated_effort="2 days"),
dict(task="Add pre/post migration balance reconciliation", description="Automated diff of total balances before and after cutover.",
component="Loyalty Service", priority=Priority.HIGH, estimated_effort="1 day"),
],
affected_components=[
dict(component_name="Loyalty Service", component_type=ComponentType.BACKEND, impact_level=ImpactLevel.HIGH, reason="Owns the ledger table being
migrated.", confidence=92.0),
dict(component_name="Loyalty Database", component_type=ComponentType.DATABASE, impact_level=ImpactLevel.HIGH, reason="Schema change and backfill
run directly against it.", confidence=95.0),
],
dependencies=[
dict(dependency_name="Loyalty Database", dependency_type=DependencyType.DATABASE, impact_level=ImpactLevel.HIGH, reason="Migration must complete
before any dependent read/write path is safe."),
],
)
# A real seeded edit -> version 2, so version history + the
# outdated-analysis flag are both visible without touching this CR.
versioning.apply_change_request_update(
db,
ledger_cr,
{"description": ledger_cr.description + " Also adds a rollback script in case cutover needs to be reverted."},
user=engineer,
)
assign(db, ledger_cr, target=engineer, role=AssignmentRole.OWNER, assigned_by=requester, at=days_ago(6))
assign(db, ledger_cr, target=security_reviewer, role=AssignmentRole.SECURITY_REVIEWER, assigned_by=requester, at=days_ago(6))
_comment(
db, ledger_cr, author=engineer, at=days_ago(5),
body="@Priya Nair can you take a look at the backfill script before we request approval? It touches customer balance data directly.",
)
request_approval(db, ledger_cr, approval_type=ApprovalType.TECHNICAL, approver=manager, requested_by=engineer, at=days_ago(4))

# B: fully Approved - a completed approval, closed-out lifecycle so
# far, MEDIUM risk.
currency_cr = _create_cr(
db,
requester,
created_at=days_ago(12),
status=ChangeRequestStatus.PENDING_ANALYSIS,
title="Add multi-currency support to POS checkout",
description="Allow POS checkout to accept and total payments in a customer's local currency, not just USD.",
business_objective="Support three new regional store openings this quarter.",
priority=Priority.MEDIUM,
requested_by="Regional Expansion team",
target_system="POS Checkout",
)
_seed_analysis(
db, currency_cr, at=days_ago(12),
summary="Adds currency selection and conversion display to an existing, well-tested checkout flow.",
category="Feature",
complexity=ComplexityLevel.MEDIUM,
risk_score=44.0,
confidence_score=82.0,
recommendation=ApprovalRecommendation.APPROVE,
requirements=[
dict(requirement_type=RequirementType.FUNCTIONAL, description="Checkout totals display in the customer's selected currency.",
priority=Priority.HIGH, certainty=Certainty.KNOWN, confidence=88.0),
],
risks=[
dict(category=RiskCategory.BUSINESS, description="Incorrect exchange-rate rounding could under- or over-charge customers.",
severity=Severity.MEDIUM, probability=0.25, score=44.0, mitigation="Use the existing finance-approved rounding library, not a new implementation."),
],
impacts=[
dict(category=ImpactCategory.CUSTOMER, impact_level=ImpactLevel.MEDIUM, description="Visible pricing change at checkout.",
certainty=Certainty.KNOWN, confidence=85.0),
dict(category=ImpactCategory.BUSINESS, impact_level=ImpactLevel.MEDIUM, description="Unblocks three regional store openings.",
certainty=Certainty.KNOWN, confidence=80.0),
],

```

```

    ],
    security_findings=[
        dict(category=SecurityCategory.API_SECURITY, finding="New currency-rate lookup calls an external rates API.", severity=Severity.LOW,
evidence="No existing outbound integration for exchange rates.", recommendation="Cache rates and fail closed to the last known-good rate on lookup failure.",
status=SecurityFindingStatus.OPEN),
    ],
    test_cases=[
        dict(test_id="TC-201", title="Checkout total converts correctly to a selected currency", test_type=TestType.INTEGRATION, priority=Priority.HIGH,
description="Select a non-USD currency at checkout.", expected_result="Total matches the current exchange rate within rounding tolerance."),
    ],
    tasks=[
        dict(task="Add currency selector to checkout UI", description="Let staff select the transaction currency at checkout.", component="POS
Checkout", priority=Priority.MEDIUM, estimated_effort="3 days"),
    ],
    )
    approval_b = _request_approval(db, currency_cr, approval_type=ApprovalType.PRODUCT, approver=manager, requested_by=requester, at=_days_ago(10))
    _respond_approval(db, currency_cr, approval_b, new_status=ApprovalStatus.APPROVED, comment="Looks good - rounding approach matches finance's
requirement", responder=manager, at=_days_ago(9))
    _transition_status(db, currency_cr, manager, ChangeRequestStatus.APPROVED, at=_days_ago(9))

    # C: Changes Requested - shows the "request changes" state (spec
    # section 3) explicitly, with the reviewer's comment attached.
    sync_cr = _create_cr(
        db,
        requester,
        created_at=_days_ago(7),
        status=ChangeRequestStatus.PENDING_ANALYSIS,
        title="Refactor inventory sync job for the new warehouse integration",
        description="Rework the nightly inventory sync job so it can also pull stock levels from the new third-party warehouse system.",
        business_objective="Support the new outsourced warehouse partner going live next month.",
        priority=Priority.MEDIUM,
        requested_by="Warehouse Operations team",
        target_system="Inventory Sync Service",
    )
    _seed_analysis(
        db, sync_cr, at=_days_ago(7),
        summary="Adds a second inbound data source to an existing nightly batch job.",
        category="Integration",
        complexity=ComplexityLevel.MEDIUM,
        risk_score=54.0,
        confidence_score=71.0,
        recommendation=ApprovalRecommendation.NEEDS_MORE_INFO,
        requirements=[
            dict(requirement_type=RequirementType.FUNCTIONAL, description="Nightly job reconciles stock levels from both the existing and new warehouse
systems.", priority=Priority.HIGH, certainty=Certainty.KNOWN, confidence=84.0),
        ],
        risks=[
            dict(category=RiskCategory.OPERATIONAL, description="A malformed feed from the new partner could silently zero out stock counts.",
severity=Severity.HIGH, probability=0.35, score=54.0, mitigation="Reject and alert on any feed with an implausible swing in total stock, rather than applying
it."),
        ],
        impacts=[
            dict(category=ImpactCategory.OPERATIONAL, impact_level=ImpactLevel.MEDIUM, description="Adds a new dependency to the nightly batch job.",
certainty=Certainty.KNOWN, confidence=80.0),
        ],
        security_findings=[
            dict(category=SecurityCategory.API_SECURITY, finding="New partner feed is pulled over an external SFTP connection.", severity=Severity.MEDIUM,
evidence="No existing external SFTP integration on this job today.", recommendation="Confirm the partner's SFTP endpoint uses key-based auth, not a shared
password.", status=SecurityFindingStatus.OPEN),
        ],
        test_cases=[
            dict(test_id="TC-301", title="Malformed partner feed is rejected, not applied", test_type=TestType.INTEGRATION, priority=Priority.HIGH,
description="Feed the job a feed with an implausible stock swing.", expected_result="Job rejects the feed and alerts, existing stock levels unchanged."),
        ],
        tasks=[
            dict(task="Add feed sanity-check before applying stock updates", description="Reject a feed whose totals move implausibly versus the prior
run.", component="Inventory Sync Service", priority=Priority.HIGH, estimated_effort="2 days"),
        ],
    )
    approval_c = _request_approval(db, sync_cr, approval_type=ApprovalType.TECHNICAL, approver=engineer, requested_by=requester, at=_days_ago(5))
    _respond_approval(
        db, sync_cr, approval_c, new_status=ApprovalStatus.CHANGES_REQUESTED,
        comment="Please add the feed sanity-check as a hard gate before this goes further, not just a task on the plan.",
        responder=engineer, at=_days_ago(4),
    )
    _transition_status(db, sync_cr, engineer, ChangeRequestStatus.CHANGES_REQUESTED, at=_days_ago(4))

    # D: fully Closed - complete lifecycle, LOW risk, older.
    legacy_cr = _create_cr(
        db,
        requester,
        created_at=_days_ago(30),
        status=ChangeRequestStatus.PENDING_ANALYSIS,
        title="Decommission legacy reporting service",
        description="Retire the old nightly reporting service now that Analytics (Module 19) fully replaces its reports.",
        business_objective="Reduce infrastructure cost and maintenance burden of an unused service.",
        priority=Priority.LOW,
        requested_by="Platform team",
        target_system="Legacy Reporting Service",
    )
    _seed_analysis(
        db, legacy_cr, at=_days_ago(30),
        summary="Removal of an already-unused internal service with no remaining consumers.",
        category="Maintenance",
        complexity=ComplexityLevel.LOW,
        risk_score=14.0,
        confidence_score=93.0,
        recommendation=ApprovalRecommendation.APPROVE,
        requirements=[
            dict(requirement_type=RequirementType.TECHNICAL, description="Confirm no remaining consumers before shutdown.", priority=Priority.MEDIUM,
certainty=Certainty.KNOWN, confidence=90.0),
        ],
        risks=[
            dict(category=RiskCategory.OPERATIONAL, description="An undiscovered consumer could still depend on this service.", severity=Severity.LOW,
probability=0.1, score=14.0, mitigation="Leave the service reachable but logging-only for two weeks before final shutdown."),
        ],
        impacts=[
            dict(category=ImpactCategory.TECHNICAL, impact_level=ImpactLevel.LOW, description="Removes one internal service and its scheduled job.",
certainty=Certainty.KNOWN, confidence=90.0),
        ],
        security_findings=[
            dict(category=SecurityCategory.COMPLIANCE, finding="Service holds historical report data that must be retained per policy.",
severity=Severity.LOW, evidence="Reports database has not been reviewed for a retention policy.", recommendation="Archive the historical reports database before
decommissioning the service itself.", status=SecurityFindingStatus.OPEN),
        ],
        test_cases=[
            dict(test_id="TC-401", title="No remaining callers after shutdown", test_type=TestType.MANUAL, priority=Priority.MEDIUM, description="Monitor
access logs for two weeks post-shutdown.", expected_result="Zero requests to the decommissioned service."),
        ],
        tasks=[
            dict(task="Archive historical reports database", description="Export and archive report history before final shutdown.", component="Legacy
Reporting Service", priority=Priority.MEDIUM, estimated_effort="1 day"),
        ],
    )
    approval_d = _request_approval(db, legacy_cr, approval_type=ApprovalType.ENGINEERING_MANAGER, approver=manager, requested_by=requester,
at=_days_ago(28))
    _respond_approval(db, legacy_cr, approval_d, new_status=ApprovalStatus.APPROVED, comment="Approved - archive the report history first as noted.",
responder=manager, at=_days_ago(27))
    for status, at in (
        (ChangeRequestStatus.APPROVED, _days_ago(27)),
        (ChangeRequestStatus.IMPLEMENTATION_PLANNED, _days_ago(20)),
    )

```

```

        (ChangeRequestStatus.IN_PROGRESS, _days_ago(15)),
        (ChangeRequestStatus.IMPLEMENTED, _days_ago(5)),
        (ChangeRequestStatus.VALIDATED, _days_ago(3)),
        (ChangeRequestStatus.CLOSED, _days_ago(2)),
    ):
        _transition_status(db, legacy_cr, engineer, status, at=at)

# E: simple Draft, LOW priority/risk - quick background variety,
# no analysis yet.
_create_cr(
    db,
    requester,
    created_at=_days_ago(1),
    status=ChangeRequestStatus.DRAFT,
    title="Add dark mode toggle to staff dashboard",
    description="Staff have asked for a dark theme option in the internal dashboard to reduce eye strain during night shifts.",
    business_objective="Improve staff comfort during night-shift use.",
    priority=Priority.LOW,
    requested_by="Store Operations team",
    target_system="Staff Dashboard",
)

# F: mid-lifecycle, In Progress, MEDIUM risk - shows implementation
# in flight rather than only start/end states.
sso_cr = _create_cr(
    db,
    requester,
    created_at=_days_ago(15),
    status=ChangeRequestStatus.PENDING_ANALYSIS,
    title="Enable SSO login for admin portal",
    description="Replace the admin portal's separate login with single sign-on against the corporate identity provider.",
    business_objective="Reduce credential sprawl and simplify offboarding for admin accounts.",
    priority=Priority.MEDIUM,
    requested_by="IT Security team",
    target_system="Admin Portal",
)

_seed_analysis(
    db, sso_cr, at=_days_ago(15),
    summary="Replaces a Self-contained login form with an external identity provider integration.",
    category="Security",
    complexity=ComplexityLevel.MEDIUM,
    risk_score=41.0,
    confidence_score=79.0,
    recommendation=ApprovalRecommendation.APPROVE,
    requirements=[
        dict(requirement_type=RequirementType.FUNCTIONAL, description="Admin accounts log in through the corporate identity provider instead of a local password.", priority=Priority.HIGH, certainty=Certainty.KNOWN, confidence=88.0),
    ],
    risks=[
        dict(category=RiskCategory.SECURITY, description="A misconfigured SSO integration could lock out all admin accounts at once.", severity=Severity.MEDIUM, probability=0.2, score=41.0, mitigation="Keep the existing local-login path available for one admin break-glass account during rollout."),
    ],
    impacts=[
        dict(category=ImpactCategory.SECURITY, impact_level=ImpactLevel.MEDIUM, description="Centralizes admin authentication through the corporate IdP.", certainty=Certainty.KNOWN, confidence=85.0),
    ],
    security_findings=[
        dict(category=SecurityCategory.AUTHENTICATION, finding="Admin login currently uses a locally-stored password, not SSO.", severity=Severity.MEDIUM, evidence="Admin portal has its own login form today.", recommendation="Migrate to the corporate IdP and keep one documented break-glass account.", status=SecurityFindingStatus.OPEN),
    ],
    test_cases=[
        dict(test_id="TC-501", title="Admin can log in via SSO", test_type=TestType.E2E, priority=Priority.HIGH, description="Log in to the admin portal via the corporate IdP.", expected_result="Successful login lands on the admin dashboard."),
    ],
    tasks=[
        dict(task="Integrate corporate IdP for admin login", description="Replace the local login form with an SSO redirect flow.", component="Admin Portal", priority=Priority.MEDIUM, estimated_effort="3 days"),
    ],
)
assign(db, sso_cr, target=engineer, role=AssignmentRole.TECHNICAL_LEAD, assigned_by=requester, at=_days_ago(13))
for status, at in (
    (ChangeRequestStatus.APPROVAL_REQUIRED, _days_ago(12)),
    (ChangeRequestStatus.APPROVED, _days_ago(11)),
    (ChangeRequestStatus.IMPLEMENTATION_PLANNED, _days_ago(9)),
    (ChangeRequestStatus.IN_PROGRESS, _days_ago(3)),
):
    _transition_status(db, sso_cr, engineer, status, at=at)

db.commit()
print(
    "Seeded 5 demo accounts (admin/manager/engineer/security/requester@alight.com, "
    f"password '{DEMO_PASSWORD}') and 7 change requests, including the flagship "
    f"'{otp_cr.title}' (CR-{otp_cr.id}), left unanalyzed for a live demo. See DEMO.md."
)
finally:
    db.close()

if __name__ == "__main__":
    seed_demo_data()

```

backend/app/database/session.py

55 lines

```
"""
SQLAlchemy engine/session setup for SQLite.

Kept intentionally simple for the hackathon: a single SQLite file under
backend/data/, created automatically on first run. Later modules (models,
schemas, services) will import `Base` and `get_db` from here.
"""
import os
from pathlib import Path
from typing import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from app.core.config import get_settings

settings = get_settings()

# Ensure the SQLite file's parent directory exists (e.g. backend/data/)
if settings.database_url.startswith("sqlite:///./"):
    relative_path = settings.database_url.replace("sqlite:///./", "")
    db_path = Path(_file_).resolve().parent.parent.parent / relative_path
    db_path.parent.mkdir(parents=True, exist_ok=True)

connect_args = {"check_same_thread": False} if settings.database_url.startswith("sqlite") else {}

engine = create_engine(settings.database_url, connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    """Shared declarative base for all ORM models."""
    pass

def get_db() -> Generator[Session, None, None]:
    """FastAPI dependency that yields a DB session and always closes it.

    Module 22: the `except` below rolls back on the way out whenever the
    request raised (an HTTPException, or a genuinely unexpected error) -
    previously only `close()` ran, which still ends the session but
    leaves an already-open failed transaction to SQLAlchemy's own
    implicit handling instead of explicitly closing it out. This never
    changes behavior for the success path (nothing here runs unless an
    exception propagated through the `yield`), and the endpoint's own
    `db.commit()` calls are completely unaffected either way."""
    db = SessionLocal()
    try:
        yield db
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

backend/app/main.py

103 lines

```
"""
FastAPI application entrypoint.

Run with: uvicorn app.main:app --reload (from inside backend/)
"""
import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.api import (
    admin,
    analytics,
    auth,
    change_requests,
    dashboard,
    health,
    knowledge,
    my_work,
    notifications,
    reports,
    repository,
    users,
)
from app.core.config import get_settings
from app.database.init_db import init_db

settings = get_settings()
logger = logging.getLogger("app")

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Hackathon-simple: create any missing tables on startup instead of a
    # separate migration step. init_db() is idempotent - safe to call every
    # time the app starts.
    init_db()
    yield

app = FastAPI(
    title=settings.app_name,
    description="AI-powered engineering assistant that analyzes software change requests.",
    version="0.1.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(health.router)
app.include_router(auth.router)
app.include_router(dashboard.router)
app.include_router(change_requests.router)
app.include_router(users.router)
app.include_router(notifications.router)
app.include_router(repository.router)
app.include_router(knowledge.router)
app.include_router(my_work.router)
app.include_router(analytics.router)
app.include_router(admin.router)
app.include_router(reports.router)

@app.exception_handler(Exception)
async def unhandled_exception_handler(request: Request, exc: Exception) -> JSONResponse:
    """Module 22 (Final Integration, Security & Quality) spec section 6:
    "show useful user-facing errors, do not expose stack traces in
    production UI." Every expected failure in this app already raises its
    own HTTPException with a clean message (bad AI response, missing CR,
    permission denied, and so on) - those are handled by FastAPI's own
    built-in HTTPException handler and never reach this one (Starlette
    picks the most specific registered handler for an exception's type,
    and HTTPException already has one). This one only catches whatever's
    left: a genuinely unexpected bug. Before this handler existed,
    Starlette's own default already returned a generic 500 with no
    traceback in the response body - so nothing was ever actually leaking
    to a client - but it never logged anything server-side either, making
    an unexpected failure invisible except as a mysterious 500 in the
    browser. This just adds that logging, with the real traceback, so a
    real bug is diagnosable instead of an opaque dead end."""
    logger.exception("Unhandled exception on %s %s", request.method, request.url.path, exc_info=exc)
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={"detail": "Something went wrong on our end. Please try again."},
    )

@app.get("/")
def root() -> dict:
    return {
        "message": f"({settings.app_name}) API is running.",
        "docs": "/docs",
        "health": "/health",
    }
```

backend/app/models/__init__.py

68 lines

```
"""Importing this package registers every ORM model on Base.metadata, so
Base.metadata.create_all() (see app/database/init_db.py) knows about all of
them. Import new model modules here as they're added.
"""
from app.models.affected_component import AffectedComponent
from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.approval_rule import ApprovalRule
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.change_request_history import ChangeRequestHistory
from app.models.change_request_version import ChangeRequestVersion
from app.models.clarification_question import ClarificationQuestion
from app.models.comment import ChangeRequestComment
from app.models.dependency import Dependency
from app.models.implementation_task import ImplementationTask
from app.models.impact_assessment import ImpactAssessment
from app.models.indexed_file import IndexedFile
from app.models.knowledge_chunk import KnowledgeChunk
from app.models.knowledge_document import KnowledgeDocument
from app.models.knowledge_evidence import KnowledgeEvidence
from app.models.notification import Notification
from app.models.repository_finding import RepositoryFinding
from app.models.repository_scan import RepositoryScan
from app.models.requirement import Requirement
from app.models.risk import Risk
from app.models.role_permission import RolePermission
from app.models.security_finding import SecurityFinding
from app.models.system_audit_log import SystemAuditLog
from app.models.system_setting import SystemSetting
from app.models.test_case import TestCase
from app.models.user import User

__all__ = [
    "User",
    "ChangeRequest",
    "Analysis",
    "Requirement",
    "AffectedComponent",
    "Dependency",
    "Risk",
    "ClarificationQuestion",
    "TestCase",
    "ImplementationTask",
    "ImpactAssessment",
    "SecurityFinding",
    # Module 12: Enterprise Workflow
    "ChangeRequestVersion",
    "ChangeRequestHistory",
    "ChangeRequestAssignment",
    "Approval",
    "ChangeRequestComment",
    "Notification",
    # Module 15: Repository Intelligence
    "RepositoryScan",
    "IndexedFile",
    "RepositoryFinding",
    # Module 16: Project Knowledge Base & RAG
    "KnowledgeDocument",
    "KnowledgeChunk",
    "KnowledgeEvidence",
    # Module 21: Administration & Configuration
    "RolePermission",
    "ApprovalRule",
    "SystemSetting",
    "SystemAuditLog",
]
```

backend/app/models/affected_component.py

38 lines

```
"""AffectedComponent model - a system component the change is expected to touch."""
from typing import Optional

from sqlalchemy import Float, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Certainty, ComponentType, ImpactLevel, sa_enum

class AffectedComponent(Base):
    __tablename__ = "affected_components"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    component_name: Mapped[str] = mapped_column(String(255), nullable=False)
    component_type: Mapped[ComponentType] = mapped_column(
        sa_enum(ComponentType, "affected_component_type"), nullable=False
    )
    impact_level: Mapped[ImpactLevel] = mapped_column(
        sa_enum(ImpactLevel, "affected_component_impact_level"), nullable=False
    )
    reason: Mapped[str] = mapped_column(Text, nullable=False)
    confidence: Mapped[float] = mapped_column(Float, nullable=False) # 0-100
    # Module 13 (AI Analysis 2.0) - nullable for the same reason as
    # Requirement.certainty above (pre-existing rows predate this column).
    certainty: Mapped[Optional[Certainty]] = mapped_column(
        sa_enum(Certainty, "affected_component_certainty"), nullable=True
    )
    # Module 14 (Analysis & Impact Intelligence) - nullable for the same
    # reason as `certainty` above: pre-existing rows predate this column.
    evidence: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="affected_components")

    def __repr__(self) -> str:
        return f"<AffectedComponent id={self.id} name={self.component_name!r}>"
```


backend/app/models/analysis.py

120 lines

```
"""Analysis model - one AI-generated analysis run for a change request.

A change request can be (re-)analyzed more than once (e.g. after it's
edited), so this is one-to-many from ChangeRequest, not one-to-one.
"""
from datetime import datetime
from typing import List, Optional

from sqlalchemy import DateTime, Float, ForeignKey, Integer, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import ApprovalRecommendation, ComplexityLevel, ConfidenceLevel, sa_enum

class Analysis(Base):
    __tablename__ = "analyses"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(ForeignKey("change_requests.id"), nullable=False)
    summary: Mapped[str] = mapped_column(Text, nullable=False)
    # Free text on purpose: AI-generated categorization (e.g. "feature",
    # "bugfix", "infrastructure") is open-ended, not a fixed vocabulary the
    # app enforces.
    category: Mapped[str] = mapped_column(Text, nullable=False)
    complexity: Mapped[ComplexityLevel] = mapped_column(
        sa_enum(ComplexityLevel, "analysis_complexity"), nullable=False
    )
    risk_score: Mapped[float] = mapped_column(Float, nullable=False) # 0-100
    confidence_score: Mapped[float] = mapped_column(Float, nullable=False) # 0-100
    recommendation: Mapped[Optional[ApprovalRecommendation]] = mapped_column(
        sa_enum(ApprovalRecommendation, "analysis_recommendation"), nullable=True
    )
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)

    # --- Module 6 (AI Analysis Engine) -----
    # These hold narrative AI output that doesn't have its own relational
    # table (unlike requirements/risks/etc. below, which do). All nullable
    # so existing rows created before this module stay valid.
    classification_reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # The AI's stated reasoning for the complexity level (Module 7's Overview
    # tab needs this for "technical impact") - was validated but silently
    # dropped during persistence until now.
    complexity_reasoning: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # JSON-encoded {"concerns": [str, ...], "summary": str} - structured
    # enough to be worth keeping as one blob rather than a whole new table.
    security_analysis: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # Human-readable range, e.g. "Backend: 2-3 days, Frontend: 1-2 days,
    # Testing: 2 days, Total: 5-8 developer-days" - deliberately a string,
    # not a number, per the spec ("a range, not fake precision").
    effort_estimate: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    recommendation_reasoning: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    # --- Module 14 (Analysis & Impact Intelligence) -----
    # A coarse Low/Medium/High confidence in the complexity level and the
    # effort estimate respectively - both nullable: rows from before this
    # module ran have neither recorded. Deliberately separate from
    # 'confidence_score' above (that's the AI's confidence in its overall
    # classification, not specifically in these two estimates).
    complexity_confidence: Mapped[Optional[ConfidenceLevel]] = mapped_column(
        sa_enum(ConfidenceLevel, "analysis_complexity_confidence"), nullable=True
    )
    effort_confidence: Mapped[Optional[ConfidenceLevel]] = mapped_column(
        sa_enum(ConfidenceLevel, "analysis_effort_confidence"), nullable=True
    )
    # The full validated AI JSON response, verbatim, for audit/debugging -
    # not surfaced through the API. A safety net in case a future module
    # needs a field that didn't get its own column.
    raw_ai_response: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    # --- Module 12 (Enterprise Workflow) -----
    # Which ChangeRequest.current version this analysis was run against.
    # Nullable for the same reason as ChangeRequest.current version above
    # (ALTER-added column on an existing table); init db backfills existing
    # rows to 1. "Is the current analysis outdated?" is computed, never
    # stored, by comparing this to the CR's current version - see
    # app/services/workflow_rules.py::is_analysis_outdated().
    change_request_version: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="analyses")

    requirements: Mapped[List["Requirement"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    affected_components: Mapped[List["AffectedComponent"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    dependencies: Mapped[List["Dependency"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    risks: Mapped[List["Risk"]] = relationship(back_populates="analysis", cascade="all, delete-orphan")
    # Module 14 Phase 4
    impact_assessments: Mapped[List["ImpactAssessment"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    # Module 14 Phase 5
    security_findings: Mapped[List["SecurityFinding"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    clarification_questions: Mapped[List["ClarificationQuestion"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    test_cases: Mapped[List["TestCase"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    implementation_tasks: Mapped[List["ImplementationTask"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )
    # Module 16 (Project Knowledge Base & RAG) Phase 4/5 - which knowledge-
    # base chunks were retrieved and given to the AI as grounding context
    # for this specific analysis run (empty for any analysis run before
    # this module existed, or when the knowledge base had nothing
    # plausibly relevant)
    knowledge_evidence: Mapped[List["KnowledgeEvidence"]] = relationship(
        back_populates="analysis", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<Analysis id={self.id} change_request_id={self.change_request_id}>"
```

backend/app/models/approval.py

60 lines

```
"""Approval model (Module 12) - one sign-off request on a change request,
e.g. "Priya, Security Approval, Pending". Created when someone with
permission tags an approver (POST ../approvals/request); moved to
Approved/Rejected/Changes Requested by that approver, never by anyone
else or by the AI (see app/services/workflow_rules.py - "AI recommends,
humans decide").

cr version records which CR version this approval was requested against.
If the change request is edited again after that (current version moves
past cr_version), the approval is stale with respect to the latest edit -
see approval invalidation logic in app/services/approvals.py.

Module 18 Phase 1 (Notifications, My Work & Personal Engineering Queue):
due_date is an optional target date the requester can set when tagging an
approver (e.g. "please respond by Friday") - nullable so every approval
requested before this module rolled out simply has no due date, never a
fabricated one. Purely informational, like everything else on this row -
it never blocks a response and is never required. Whether a still-pending
approval counts as "overdue" or "due soon" is never stored here; it's
computed fresh at read time (see app/services/approvals.py::
approval_due_status), the same "never a stored boolean that can drift"
rule is outdated/is_analysis_outdated already follow elsewhere in this app.
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, Integer, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import ApprovalStatus, ApprovalType, sa_enum


class Approval(Base):
    __tablename__ = "change_request_approvals"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(
        ForeignKey("change_requests.id"), nullable=False, index=True
    )
    approver_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    approval_type: Mapped[ApprovalType] = mapped_column(sa_enum(ApprovalType, "approval_type"), nullable=False)
    status: Mapped[ApprovalStatus] = mapped_column(
        sa_enum(ApprovalStatus, "approval_status"), default=ApprovalStatus.PENDING, nullable=False, index=True
    )
    requested_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    requested_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    responded_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
    comment: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    cr_version: Mapped[int] = mapped_column(Integer, nullable=False)
    # Module 18 Phase 1 - see this model's own docstring above.
    due_date: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="approvals")
    approver: Mapped["User"] = relationship(foreign_keys=[approver_id])
    requested_by_user: Mapped["User"] = relationship(foreign_keys=[requested_by])

    def __repr__(self) -> str:
        return f"<Approval cr={self.change_request_id} type={self.approval_type} status={self.status}>"
```

backend/app/models/approval_rule.py

50 lines

```
"""ApprovalRule model (Module 21 spec section 4) - the admin-editable
version of what used to be the hardcoded RISK BASELINE /
CATEGORY KEYWORD APPROVALS tuples in app/services/workflow_rules.py.
Still "a simple, readable config, not a rule engine" (same phrase that
module's own docstring already used) - just database rows now instead of
Python constants, so an admin can view/add/edit/disable them without a
code change.

RISK rows: match_value is one of "low"/"medium"/"high"/"critical" (the
same bucket labels workflow_rules.risk_bucket() already returns).
CATEGORY rows: match_value is a lowercase keyword matched against the
analysis category / target system / title (same haystack the old
hardcoded keyword list matched against).

required_approval_types() in workflow_rules.py takes the currently-enabled
rules as a plain list (the same "caller owns the query, this module stays
pure rule logic" convention every other per-CR permission helper in that
file already follows) rather than querying the database itself.
"""
from datetime import datetime

from sqlalchemy import Boolean, DateTime, String
from sqlalchemy.orm import Mapped, mapped_column

from app.database.session import Base
from app.models.enums import ApprovalRuleType, sa_enum

class ApprovalRule(Base):
    __tablename__ = "approval_rules"

    id: Mapped[int] = mapped_column(primary_key=True)
    rule_type: Mapped[ApprovalRuleType] = mapped_column(
        sa_enum(ApprovalRuleType, "approval_rule_type"), nullable=False, index=True
    )
    match_value: Mapped[str] = mapped_column(String(255), nullable=False)
    # JSON-encoded list of ApprovalType values (e.g.
    # ["security", "engineering_manager"]) - same "store as a JSON
    # string column" convention as ChangeRequest.tags, not a separate
    # many-to-many join table for what's a short, rarely-changed list.
    approval_types: Mapped[str] = mapped_column(String(500), nullable=False)
    enabled: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False
    )

    def __repr__(self) -> str:
        return f"<ApprovalRule {self.rule_type}={self.match_value!r}>"
```

backend/app/models/change_request.py

98 lines

```
"""ChangeRequest model - the software change request as submitted by a user."""
from datetime import date, datetime
from typing import List, Optional

from sqlalchemy import Date, DateTime, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import ChangeRequestStatus, Priority, sa_enum

class ChangeRequest(Base):
    __tablename__ = "change_requests"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    business_objective: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    priority: Mapped[Priority] = mapped_column(
        sa_enum(Priority, "change_request_priority"), default=Priority.MEDIUM, nullable=False
    )
    status: Mapped[ChangeRequestStatus] = mapped_column(
        sa_enum(ChangeRequestStatus, "change_request_status"),
        default=ChangeRequestStatus.PENDING_ANALYSIS,
        nullable=False,
    )
    # Who the change was requested by. Often the submitter, but not always
    # (e.g. an engineer filing it on behalf of a store manager) - free text
    # on purpose, not every requester has an account here. Nullable so the
    # thousands of rows created before this column existed stay valid; the
    # creation form (Module 4) requires it for anything submitted from now on.
    requested_by: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    # Which system/service this change targets, e.g. "POS Backend", "Mobile App".
    target_system: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    desired_deadline: Mapped[Optional[date]] = mapped_column(Date, nullable=True)
    created_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False
    )

    # --- Module 12 (Enterprise Workflow) -----
    # Nullable (not nullable=False) even though every CR conceptually has a
    # version >= 1: this column was added to an EXISTING table, and the
    # project's home-grown migration (init db.py:: add missing columns)
    # only ALTERs in the column's type, not a NOT NULL/DEFAULT clause - so
    # SQLite would leave every pre-Module-12 row's new column NULL. init_db
    # backfills every existing row to 1 on startup (see
    # 'backfill workflow data), and application code should still read this
    # as 'change request.current_version or 1' to stay correct even before
    # that backfill has run once.
    current_version: Mapped[Optional[int]] = mapped_column(Integer, default=1, nullable=True)

    # Additional enterprise CR fields requested by Module 12's editing spec
    # (section 2). "Category" is deliberately NOT duplicated here - the app
    # already has a working, AI-derived category (Analysis.category, used
    # throughout the dashboard/list filters); adding a second, manually-
    # editable category on the CR itself would just create two
    # disagreeing sources of truth for the same concept.
    business_impact: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    technical_impact: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    customer_impact: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # e.g. "Production", "Staging" - free text, not an enum: environments
    # vary too much per org to hard-code a fixed list here.
    environment: Mapped[Optional[str]] = mapped_column(String(100), nullable=True)
    # Free-text dependency note as stated by the requester/owner - distinct
    # from Analysis.dependencies (the AI-derived, structured Dependency
    # rows on the latest analysis).
    dependencies_note: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    compliance_requirements: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # JSON-encoded list of strings, e.g. '["payments", "mobile"]' - same
    # "one text column, not a new table" pattern already used for
    # Analysis.security_analysis.
    tags: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    creator: Mapped["User"] = relationship(back_populates="change_requests")
    analyses: Mapped[List["Analysis"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan"
    )
    versions: Mapped[List["ChangeRequestVersion"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan", order_by="ChangeRequestVersion.version_number"
    )
    history: Mapped[List["ChangeRequestHistory"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan", order_by="ChangeRequestHistory.created_at"
    )
    assignments: Mapped[List["ChangeRequestAssignment"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan"
    )
    approvals: Mapped[List["Approval"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan"
    )
    comments: Mapped[List["ChangeRequestComment"]] = relationship(
        back_populates="change_request", cascade="all, delete-orphan", order_by="ChangeRequestComment.created_at"
    )

    def __repr__(self) -> str:
        return f"<ChangeRequest id={self.id} title={self.title!r}>"
```

backend/app/models/change_request_assignment.py

35 lines

```
"""ChangeRequestAssignment model (Module 12) - "who's responsible for what"
on a change request: Requester, Owner, Technical Lead, Reviewer, Approver,
Security Reviewer, QA Owner, Implementation Owner (app.models.enums.
AssignmentRole). A CR is not required to have every role filled, and
REVIEWER/APPROVER can have more than one person - so this is a plain list
of (user, role) rows per CR rather than fixed columns on ChangeRequest.
"""
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import AssignmentRole, sa_enum

class ChangeRequestAssignment(Base):
    __tablename__ = "change_request_assignments"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(
        ForeignKey("change_requests.id"), nullable=False, index=True
    )
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    role: Mapped[AssignmentRole] = mapped_column(sa_enum(AssignmentRole, "assignment_role"), nullable=False)
    assigned_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    assigned_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="assignments")
    user: Mapped["User"] = relationship(foreign_keys=[user_id])
    assigned_by_user: Mapped["User"] = relationship(foreign_keys=[assigned_by])

    def __repr__(self) -> str:
        return f"<ChangeRequestAssignment cr={self.change_request_id} user={self.user_id} role={self.role}>"
```

backend/app/models/change_request_history.py

55 lines

```
"""ChangeRequestHistory model (Module 12) - the append-only audit trail.

This is the backbone of the "Activity" tab: every meaningful thing that
happens to a change request gets one row here, in enough detail to answer
"who did it, what did they change, when, from what, to what, and why" -
never a vague "Change Request updated." Rows are never edited or deleted
by normal application code (see app/services/history.py, the only place
that's allowed to insert into this table) - that's the "audit data
integrity" requirement (spec section 24).

user_id is nullable to allow one specific system actor: the AI Analyzer
itself recording "AI Analysis Completed" / "AI Analysis Invalidated"
events. When user_id is None, actor_label carries a human-readable name
for that actor (e.g. "AI Analyzer") instead.
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import HistoryAction, sa_enum


class ChangeRequestHistory(Base):
    __tablename__ = "change_request_history"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(
        ForeignKey("change_requests.id"), nullable=False, index=True
    )
    user_id: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    # Set only when user_id is None (a system actor, e.g. "AI Analyzer").
    actor_label: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    action: Mapped[HistoryAction] = mapped_column(sa_enum(HistoryAction, "history_action"), nullable=False, index=True)
    # Set for FIELD_CHANGED events (e.g. "priority", "target system"); None
    # for events that aren't about one specific field (status changes use
    # their own old/new below with field_name="status", but STATUS_CHANGED
    # is its own action so the timeline can render it with a distinct icon).
    field_name: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    old_value: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    new_value: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # The CR version this event happened at/produced, where relevant (e.g.
    # a FIELD_CHANGED event's new value belongs to the version it created).
    version_number: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False, index=True)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="history")
    user: Mapped[Optional["User"]] = relationship()

    def __repr__(self) -> str:
        return f"<ChangeRequestHistory cr={self.change_request_id} action={self.action}>"
```

backend/app/models/change_request_version.py

43 lines

```
"""ChangeRequestVersion model (Module 12) - a snapshot taken every time a
change request's editable fields are meaningfully modified.

Version 1 is created the moment a CR is created (or, for CRs that already
existed before this module, backfilled by init_db.py as "Initial version
imported"). Every subsequent save that changes at least one editable field
creates Version 2, 3, 4, ... - never overwrites a previous one, so the
Compare Versions view (GET /api/change-requests/{id}/compare) always has
two real snapshots to diff.
"""
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Integer, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base


class ChangeRequestVersion(Base):
    __tablename__ = "change_request_versions"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(
        ForeignKey("change_requests.id"), nullable=False, index=True
    )
    version_number: Mapped[int] = mapped_column(Integer, nullable=False)
    changed_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False, index=True)
    # Short human-readable summary, e.g. "Priority changed from Medium to
    # High" or "Initial version imported."
    change_summary: Mapped[str] = mapped_column(Text, nullable=False)
    # JSON object of the CR's editable fields as of this version - what
    # Compare Versions actually diffs. Not a full relational history; a
    # flat field-name -> value dict is enough per the spec ("a clean
    # field-by-field comparison is sufficient").
    snapshot: Mapped[str] = mapped_column(Text, nullable=False)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="versions")
    changed_by_user: Mapped["User"] = relationship()

    def __repr__(self) -> str:
        return f"<ChangeRequestVersion cr={self.change_request_id} v={self.version_number}>"
```

backend/app/models/clarification_question.py

38 lines

```
"""ClarificationQuestion model - a question worth answering before the change can be safely approved."""
from datetime import datetime
from typing import Optional

from sqlalchemy import Boolean, DateTime, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Priority, sa_enum

class ClarificationQuestion(Base):
    __tablename__ = "clarification_questions"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    question: Mapped[str] = mapped_column(Text, nullable=False)
    priority: Mapped[Priority] = mapped_column(
        sa_enum(Priority, "clarification_question_priority"), default=Priority.MEDIUM, nullable=False
    )
    reason: Mapped[str] = mapped_column(Text, nullable=False)
    resolved: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)

    # A human answering the AI's question directly, right where it's asked -
    # rather than "resolved" being an unreachable dead-end flag nothing ever
    # set. Nullable: None means nobody has answered yet. Answering also sets
    # resolved=True (see app/api/change_requests.py::answer_clarification
    # question); the two are always kept in sync from that one endpoint, so
    # nothing else needs to check both.
    answer_text: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    answered_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    answered_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="clarification_questions")

    def __repr__(self) -> str:
        return f"<ClarificationQuestion id={self.id} resolved={self.resolved}>"
```


backend/app/models/comment.py

38 lines

```
"""ChangeRequestComment model (Module 12) - discussion thread on a change
request. Supports one level of replies via parent id (spec: "Reply where
practical" - a flat two-level thread is practical here, a full nested tree
is not needed) and @mentions are parsed out of `body` at write time by
app/services/mentions.py, which is also what creates the resulting
notifications - nothing about mentions is stored structurally beyond the
plain text itself.
"""
from datetime import datetime
from typing import Optional

from sqlalchemy import Boolean, DateTime, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base


class ChangeRequestComment(Base):
    __tablename__ = "change_request_comments"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(
        ForeignKey("change_requests.id"), nullable=False, index=True
    )
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    parent_id: Mapped[Optional[int]] = mapped_column(
        ForeignKey("change_request_comments.id"), nullable=True
    )
    body: Mapped[str] = mapped_column(Text, nullable=False)
    resolved: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False, index=True)

    change_request: Mapped["ChangeRequest"] = relationship(back_populates="comments")
    user: Mapped["User"] = relationship()

    def __repr__(self) -> str:
        return f"<ChangeRequestComment cr={self.change_request_id} user={self.user_id}>"
```

backend/app/models/dependency.py

37 lines

```
"""Dependency model - something this change depends on (a service, library, another component, etc.)."""
from typing import Optional

from sqlalchemy import ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import DependencyRelationship, DependencyType, ImpactLevel, Severity, sa_enum

class Dependency(Base):
    __tablename__ = "dependencies"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    dependency_name: Mapped[str] = mapped_column(String(255), nullable=False)
    dependency_type: Mapped[DependencyType] = mapped_column(
        sa_enum(DependencyType, "dependency_type"), nullable=False
    )
    impact_level: Mapped[ImpactLevel] = mapped_column(
        sa_enum(ImpactLevel, "dependency_impact_level"), nullable=False
    )
    reason: Mapped[str] = mapped_column(Text, nullable=False)
    # Module 14 (Analysis & Impact Intelligence) - both nullable: rows from
    # before this module ran have neither recorded.
    relationship_type: Mapped[Optional[DependencyRelationship]] = mapped_column(
        sa_enum(DependencyRelationship, "dependency_relationship_type"), nullable=True
    )
    risk_severity: Mapped[Optional[Severity]] = mapped_column(
        sa_enum(Severity, "dependency_risk_severity"), nullable=True
    )

    analysis: Mapped["Analysis"] = relationship(back_populates="dependencies")

    def __repr__(self) -> str:
        return f"<Dependency id={self.id} name={self.dependency_name}>"
```

```

"""Shared controlled-vocabulary enums used across the ORM models.

Kept as plain Python str-Enums (not free-text columns) for the fields the
app itself relies on for logic/filtering. Fields that hold AI-generated,
open-ended text (e.g. Analysis.category) are deliberately left as plain
strings instead - see the comment on that column.
"""
import enum

from sqlalchemy import Enum as SAEnum

def sa_enum(enum_cls: type[enum.Enum], name: str) -> SAEnum:
    """Build a SQLAlchemy Enum column type that stores the member's *value*
    (e.g. "medium") rather than its name (e.g. "MEDIUM"), and gives the
    underlying CHECK constraint / DB type a unique name.
    """
    return SAEnum(enum_cls, name=name, values_callable=lambda obj: [e.value for e in obj])

class UserRole(str, enum.Enum):
    ADMIN = "admin"
    ENGINEER = "engineer"
    PRODUCT_MANAGER = "product_manager" # displayed as "Manager" - see
    # app/services/workflow_rules.py::USER_ROLE_LABELS. Kept as the
    # existing wire value (never renamed - see this file's own module
    # docstring) rather than adding a new, confusingly-similar "manager"
    # member for Module 21's practical role list.
    REVIEWER = "reviewer"
    # --- Module 21 (Administration & Configuration) - additive only, same
    # convention as every other enum extension in this app. These three
    # complete the spec's practical role list (Admin/Requester/Engineer/
    # Reviewer/Security Reviewer/Approver/Manager) - the other four already
    # existed above.
    REQUESTER = "requester"
    SECURITY_REVIEWER = "security_reviewer"
    APPROVER = "approver"

class Priority(str, enum.Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class ChangeRequestStatus(str, enum.Enum):
    # Every new change request starts here (Module 4) and stays until a
    # later module runs an AI analysis on it and/or a reviewer acts on it.
    PENDING_ANALYSIS = "pending_analysis"
    DRAFT = "draft"
    SUBMITTED = "submitted"
    IN_REVIEW = "in_review" # "Under Review" in the Module 12 workflow UI
    APPROVED = "approved"
    REJECTED = "rejected"
    IMPLEMENTED = "implemented"
    # --- Module 12 (Enterprise Workflow) - additive only, see
    # app/services/workflow_rules.py for the allowed-transition graph that
    # ties these (and the ones above) together into one lifecycle:
    # Draft -> Submitted -> Pending Analysis -> Analyzed -> Under Review ->
    # Changes Requested / Approval Required -> Approved/Rejected ->
    # Implementation Planned -> In Progress -> Implemented -> Validated ->
    # Closed, with Cancelled reachable from most non-terminal states.
    ANALYZED = "analyzed"
    CHANGES_REQUESTED = "changes_requested"
    APPROVAL_REQUIRED = "approval_required"
    IMPLEMENTATION_PLANNED = "implementation_planned"
    IN_PROGRESS = "in_progress"
    VALIDATED = "validated"
    CLOSED = "closed"
    CANCELLED = "cancelled"

class ComplexityLevel(str, enum.Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    VERY_HIGH = "very_high"

class ApprovalRecommendation(str, enum.Enum):
    APPROVE = "approve"
    APPROVE_WITH_CONDITIONS = "approve_with_conditions"
    NEEDS_MORE_INFO = "needs_more_info"
    REJECT = "reject"
    # Added for Module 6 (AI Analysis Engine) - the spec's exact wording for
    # "there are open questions that need answers before this can be
    # approved" is "REQUIRES CLARIFICATION", which is distinct enough from
    # the older NEEDS_MORE_INFO to keep both rather than rename (renaming
    # would change the meaning of NEEDS_MORE_INFO values already sitting in
    # the 3000+ seeded rows).
    REQUIRES_CLARIFICATION = "requires_clarification"

class RequirementType(str, enum.Enum):
    FUNCTIONAL = "functional"
    NON_FUNCTIONAL = "non_functional"
    TECHNICAL = "technical"
    BUSINESS = "business"

class ComponentType(str, enum.Enum):
    FRONTEND = "frontend"
    BACKEND = "backend"
    DATABASE = "database"
    API = "api"
    INFRASTRUCTURE = "infrastructure"
    THIRD_PARTY = "third_party"
    OTHER = "other"

class ImpactLevel(str, enum.Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class DependencyType(str, enum.Enum):
    INTERNAL = "internal"
    EXTERNAL = "external"
    SERVICE = "service"
    LIBRARY = "library"
    DATABASE = "database"

```

```

API = "api"

# --- Module 14 (Analysis & Impact Intelligence) -----

class DependencyRelationship(str, enum.Enum):
    """How directly this change actually relies on the dependency -
    separate from 'DependencyType' above, which is a *category* (service,
    library, ...) rather than a directness rating. DIRECT: this change
    itself calls/uses it. INDIRECT: something this change touches relies
    on it, one step removed. POTENTIAL: plausible but not confidently
    established from the change request as written - see
    analysis_engine.py's "do not invent dependencies" prompt rule; this is
    the AI's way of saying "maybe" instead of asserting a dependency it
    isn't sure about."""
    DIRECT = "direct"
    INDIRECT = "indirect"
    POTENTIAL = "potential"

class ConfidenceLevel(str, enum.Enum):
    """A coarse Low/Medium/High confidence rating - deliberately not the
    finer-grained 0-100 'confidence' float used elsewhere (Requirement,
    Risk, AffectedComponent), because Complexity and Effort are already
    plain-language estimates on purpose ("a range, not fake precision" -
    see Analysis.effort estimate); pairing that with a fake-precise numeric
    confidence would undercut the same point. Three values only (no
    CRITICAL, unlike Severity) - a confidence rating has no "critical"
    equivalent."""
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"

class ImpactCategory(str, enum.Enum):
    """Module 14 Phase 4: the 7 fixed lenses every analysis assesses impact
    through (see app/models/impact_assessment.py). Distinct from
    'RiskCategory' below - a Risk is a specific thing that could go wrong
    ("the migration could corrupt data"), while an ImpactAssessment
    describes the change's general effect through one lens regardless of
    whether anything goes wrong ("this touches customer-facing billing
    flows"). Also distinct from 'ImpactLevel' - this says *what kind* of
    impact, 'ImpactLevel' says *how much*."""
    BUSINESS = "business"
    TECHNICAL = "technical"
    CUSTOMER = "customer"
    OPERATIONAL = "operational"
    SECURITY = "security"
    DATA = "data"
    PERFORMANCE = "performance"

class SecurityCategory(str, enum.Enum):
    """Module 14 Phase 5: the 9 fixed lenses a Security finding is
    classified under - replaces the old free-text 'Analysis.security analysis'
    concerns list (kept, nullable, for analyses that predate this table)
    with a real per-category structured finding (see
    app/models/security_finding.py)."""
    AUTHENTICATION = "authentication"
    AUTHORIZATION = "authorization"
    DATA_PROTECTION = "data_protection"
    SECRETS = "secrets"
    API_SECURITY = "api_security"
    RATE_LIMITING = "rate_limiting"
    PRIVACY = "privacy"
    AUDIT_LOGGING = "audit_logging"
    COMPLIANCE = "compliance"

class KnowledgeDocumentStatus(str, enum.Enum):
    """Module 16 (Project Knowledge Base & RAG): a document's lifecycle
    from upload to searchable. Mirrors RepositoryScanStatus's own
    philosophy - a failed parse/chunk/embed pass is a real, visible,
    permanently recorded state (FAILED + error_message on the document),
    never a silently-dropped upload and never quietly retried in place.

    UPLOADED: the file is saved to disk and a row exists, but parsing/
    chunking/embedding hasn't run yet (or is queued to run next).
    PROCESSING: parsing/chunking/embedding is actively running for this
    document right now.
    READY: fully parsed, chunked, and embedded - eligible for retrieval.
    FAILED: processing was attempted and did not complete; error_message
    on the document explains why. A failed document is never silently
    retried - re-processing it is a deliberate, explicit action.
    """
    UPLOADED = "uploaded"
    PROCESSING = "processing"
    READY = "ready"
    FAILED = "failed"

class SecurityFindingStatus(str, enum.Enum):
    """Module 14 Phase 5: OPEN/NOT_APPLICABLE are the only two values the AI
    itself is ever allowed to set (see
    app/schemas/ai_analysis.py::SecurityFindingItem.normalize status) -
    ACKNOWLEDGED/RESOLVED are reserved for a human reviewer to set later
    (Module 14 Phase 6's "same reviewed-by-a-human pattern" applied to
    Requirement certainty, applied here to this field instead)."""
    OPEN = "open"
    NOT_APPLICABLE = "not_applicable"
    ACKNOWLEDGED = "acknowledged"
    RESOLVED = "resolved"

class ReviewStatus(str, enum.Enum):
    """Module 14 Phase 6: a human reviewer's verdict on an AI-generated
    Requirement - entirely separate from the AI's own 'certainty' (Known/
    Inferred/Unknown) on that same row, which reviewing never touches
    (architecture lock: "AI may recommend but never perform human
    approval" - the AI's certainty/confidence/evidence stay the permanent
    record of what the AI said; this is the separate record of what a
    human then did about it). No "unreviewed" member on purpose: a NULL
    'Requirement.review_status' already means "no human has reviewed this
    yet" without needing a dedicated value for it (same nullable-for-
    pre-existing-rows reasoning as 'certainty' itself)."""
    CONFIRMED = "confirmed"
    NEEDS_CLARIFICATION = "needs_clarification"

class RiskCategory(str, enum.Enum):
    TECHNICAL = "technical"
    SECURITY = "security"
    BUSINESS = "business"

```

```

OPERATIONAL = "operational"
COMPLIANCE = "compliance"
PERFORMANCE = "performance"
# Added for Module 6 (AI Analysis Engine) - the spec's risk-analysis
# categories are security/data/performance/availability/integration/
# regression/compliance/operational. TECHNICAL and BUSINESS above predate
# that spec and are kept (additive-only enum changes) in case anything
# already stored used them.
DATA = "data"
AVAILABILITY = "availability"
INTEGRATION = "integration"
REGRESSION = "regression"

class Severity(str, enum.Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class TestType(str, enum.Enum):
    UNIT = "unit"
    INTEGRATION = "integration"
    E2E = "e2e"
    REGRESSION = "regression"
    PERFORMANCE = "performance"
    SECURITY = "security"
    MANUAL = "manual"
    # --- Module 17 (Test Cases & Implementation Plan 2.0) -----
    # The spec's own list of test types to draw from - additive-only, same
    # reasoning as every other enum extension in this app (existing seeded
    # TestCase rows using the 7 values above stay perfectly valid). FUNCTIONAL
    # is deliberately not added: MANUAL/UNIT/INTEGRATION/E2E already cover
    # "this is a normal functional test" better than one more generic value
    # would, and the spec's own list is a menu to draw from ("do not
    # generate irrelevant tests"), not a mandate to add every named type.
    NEGATIVE = "negative"
    BOUNDARY = "boundary"
    API = "api"
    UI = "ui"
    DATA_VALIDATION = "data_validation"

# --- Module 12 (Enterprise Workflow) -----
#
# A per-CR "who's involved" role, distinct from the account-wide UserRole
# above. UserRole answers "what kind of account is this"; AssignmentRole
# answers "what is this person responsible for on THIS change request" -
# the same user can hold different assignment roles on different CRs.

class AssignmentRole(str, enum.Enum):
    REQUESTER = "requester"
    OWNER = "owner"
    TECHNICAL_LEAD = "technical_lead"
    REVIEWER = "reviewer"
    APPROVER = "approver"
    SECURITY_REVIEWER = "security_reviewer"
    QA_OWNER = "qa_owner"
    IMPLEMENTATION_OWNER = "implementation_owner"

class ApprovalType(str, enum.Enum):
    """What kind of sign-off an Approval record represents. Deliberately a
    short, generic list (not one entry per possible reviewer title) - the
    approval matrix in workflow_rules.py maps risk/category to a subset of
    these rather than inventing new types per rule."""
    TECHNICAL = "technical"
    SECURITY = "security"
    PRODUCT = "product"
    ENGINEERING_MANAGER = "engineering_manager"
    DIRECTOR = "director"
    QA = "qa"
    DBA = "dba"
    RELEASE = "release"
    GENERAL = "general"

class ApprovalStatus(str, enum.Enum):
    PENDING = "pending"
    APPROVED = "approved"
    REJECTED = "rejected"
    CHANGES_REQUESTED = "changes_requested"
    CANCELLED = "cancelled"

class HistoryAction(str, enum.Enum):
    """One entry per kind of event the Activity/Audit timeline can show -
    see app/models/change_request_history.py."""
    CREATED = "created"
    FIELD_CHANGED = "field_changed"
    STATUS_CHANGED = "status_changed"
    ASSIGNED = "assigned"
    UNASSIGNED = "unassigned"
    APPROVAL_REQUESTED = "approval_requested"
    APPROVED = "approved"
    REJECTED = "rejected"
    CHANGES_REQUESTED = "changes_requested"
    APPROVAL_INVALIDATED = "approval_invalidated"
    # Module 12 Phase 4: a still-pending approval request withdrawn by
    # whoever could have requested it (Owner/Technical Lead/Admin) -
    # distinct from APPROVAL_INVALIDATED (the CR changed under it) and from
    # REJECTED (the approver said no) - nobody weighed in, it was pulled.
    APPROVAL_CANCELLED = "approval_cancelled"
    COMMENT_ADDED = "comment_added"
    MENTIONED = "mentioned"
    AI_ANALYSIS_COMPLETED = "ai_analysis_completed"
    AI_ANALYSIS_INVALIDATED = "ai_analysis_invalidated"
    VERSION_CREATED = "version_created"
    REPORT_GENERATED = "report_generated"
    REMINDER_SENT = "reminder_sent"
    # Module 13 Phase 4: recorded ALONGSIDE the normal APPROVED/REJECTED/
    # CHANGES_REQUESTED event (never instead of it) whenever that decision
    # didn't match what the AI's own recommendation on the relevant
    # analysis would suggest - see
    # app/services/workflow_rules.py::overrides_ai_recommendation. Purely
    # a record of the fact, never a block on the decision itself - "AI
    # recommends, humans decide" means a human is never wrong for
    # disagreeing with the AI, but it should still be visible on the
    # audit trail when they did.
    AI_RECOMMENDATION_OVERRIDDEN = "ai_recommendation_overridden"
    # Module 14 Phase 6: a human reviewer marked a Requirement Confirmed/
    # Needs Clarification, or moved a Security finding's Status forward
    # (e.g. Acknowledged/Resolved) - see app/api/change_requests.py's
    # review_requirement / update_security_finding_status endpoints.
    REQUIREMENT_REVIEWED = "requirement_reviewed"
    SECURITY_FINDING_STATUS_CHANGED = "security_finding_status_changed"
    # Module 17 (Test Cases & Implementation Plan 2.0) Phase 6: an

```

```

# authorized human edited one AI-generated test case or implementation
# task. Recorded the same way every other human edit in this app is -
# old value/new value on this append-only row - which is what actually
# satisfies "never overwrite AI provenance": the AI's original wording
# stays permanently readable here even after the live row is updated to
# the human's edit, exactly like a normal ChangeRequest field edit.
TEST_CASE_EDITED = "test_case_edited"
IMPLEMENTATION_TASK_EDITED = "implementation_task_edited"
# Someone answered one of the AI's own clarification questions right on
# the Missing Information tab - see app/api/change_requests.py's
# answer clarification question. This is what actually flips that
# question's resolved flag; before this existed resolved could never be
# set to True by anything.
CLARIFICATION_ANSWERED = "clarification_answered"
# Module 19 (Engineering Change Analytics): a POST ../analyze call that
# raised AnalysisError (provider not configured, provider call failed,
# malformed/invalid AI response, ...) - recorded the same way a
# successful analysis is (actor label="AI Analyzer", no user_id), so
# "AI analysis failures" on the new analytics dashboard is a real,
# traceable count rather than an invented statistic. Failures from
# before this module existed are simply not counted - there was nowhere
# for them to have been recorded.
AI_ANALYSIS_FAILED = "ai_analysis_failed"

# --- Module 13 (AI Analysis 2.0) -----
#
# Every AI finding that could plausibly be wrong should say how sure it is,
# not just how much (confidence is the "how much"; Certainty is the "what
# kind of statement is this" - see analysis_engine.py's system prompt for
# the exact definitions given to the AI).

class Certainty(str, enum.Enum):
    KNOWN = "known" # explicitly stated in the change request, or directly implied by it
    INFERRED = "inferred" # a reasonable conclusion the AI drew, not stated outright
    UNKNOWN = "unknown" # cannot be determined from the change request as written

# --- Module 15 (Repository Intelligence) -----

class RepositoryScanStatus(str, enum.Enum):
    """A RepositoryScan's outcome (app/models/repository_scan.py). FAILED is
    reserved for the walk itself blowing up partway through (e.g. the
    configured path became unreadable mid-scan) - an individual file that
    can't be read/decoded is just skipped and counted in 'skipped_count',
    never enough on its own to fail the whole scan. Mirrors the "existing
    good data is never silently destroyed by a failed run" rule Module 13
    Phase 3 already applies to AI analysis failures: a FAILED scan is its
    own new row, and whatever the last COMPLETED scan indexed stays exactly
    as it was."""
    COMPLETED = "completed"
    FAILED = "failed"

class FileMatchLabel(str, enum.Enum):
    """Module 15 Phase 3 (CR -> File Matching): the hedged, code-enforced
    headline for a RepositoryFinding - computed purely from its numeric
    'confidence' score (see from_confidence() below), never trusted from
    the AI's own wording. The module's own spec is explicit: "never claim
    certainty without evidence" - use "potentially affected"/"likely
    affected"/"possibly related", never "definitely affected" unless
    there's strong evidence. The simplest way to actually guarantee that,
    rather than just asking the AI nicely, is to never give this
    vocabulary a "definitely affected" value at all - no code path can
    ever emit one, regardless of how confident the AI's own reasoning
    text sounds."""
    POSSIBLY_RELATED = "possibly_related"
    POTENTIALLY_AFFECTED = "potentially_affected"
    LIKELY_AFFECTED = "likely_affected"

    @classmethod
    def from_confidence(cls, confidence: float) -> "FileMatchLabel":
        if confidence >= 75:
            return cls.LIKELY_AFFECTED
        if confidence >= 45:
            return cls.POTENTIALLY_AFFECTED
        return cls.POSSIBLY_RELATED

class NotificationType(str, enum.Enum):
    ASSIGNED = "assigned"
    APPROVAL_REQUESTED = "approval_requested"
    APPROVED = "approved"
    REJECTED = "rejected"
    CHANGES_REQUESTED = "changes_requested"
    MENTIONED = "mentioned"
    STATUS_CHANGED = "status_changed"
    ANALYSIS_OUTDATED = "analysis_outdated"
    # Module 13 Phase 3: distinct from ANALYSIS_OUTDATED above. That one
    # fires the moment a CR is *edited* ("your last analysis no longer
    # matches the current version"). This one fires after a *re-analysis*
    # completes and the new result is materially different from the one it
    # replaced (bigger risk bucket, new recommendation, etc - see
    # app/services/analysis_delta.py::compare_analyses's
    # is significant change) - "the numbers actually got worse/better", not
    # just "the CR changed shape underneath the old analysis."
    ANALYSIS_SIGNIFICANTLY_CHANGED = "analysis_significantly_changed"
    REAPPROVAL_REQUIRED = "reapproval_required"
    REMINDER = "reminder"
    COMMENT_ADDED = "comment_added"
    # --- Module 18 Phase 1 (Notifications, My Work & Personal Engineering
    # Queue) -----
    # Fires once, ever, per Approval - the moment a re-analysis lands the
    # change request in a strictly higher risk bucket than it was in
    # before (see app/services/analysis_delta.py). Distinct from
    # ANALYSIS_SIGNIFICANTLY_CHANGED above: that one is a general "the
    # numbers moved" notice; this one is specifically "the risk got
    # worse", which is worth its own, more urgent-reading event.
    RISK_ESCALATED = "risk_escalated"
    # Fires the first time a change request is ever analyzed (never on a
    # later re-analysis - that's what ANALYSIS_SIGNIFICANTLY_CHANGED /
    # RISK_ESCALATED already cover) - "your change request has been
    # analyzed" for whoever's watching it, not just the person who
    # clicked Analyze.
    ANALYSIS_COMPLETED = "analysis_completed"
    # A still-PENDING approval's own due date is close (see
    # app/services/approvals.py:approval_due_status) or has already
    # passed. Distinct from REMINDER above, which is a human deliberately
    # nudging someone via the "remind" button - this one is a computed,
    # automatic notice, fired at most once per approval (see
    # app/services/deadlines.py).
    DEADLINE_APPROACHING = "deadline_approaching"

# --- Module 21 (Administration & Configuration) -----
#
# Distinct from HistoryAction above: HistoryAction is scoped to ONE change

```

```
# request (its own change_request_id column is required); the events below
# are system-level - who an admin is, what role someone got, whether a
# rule changed - and belong to no single change request. See
# app/models/system_audit_log.py.
```

```
class AdminAuditAction(str, enum.Enum):
    USER_CREATED = "user_created"
    USER_ROLE_CHANGED = "user_role_changed"
    USER_ACTIVATED = "user_activated"
    USER_DEACTIVATED = "user_deactivated"
    PERMISSION_CHANGED = "permission_changed"
    APPROVAL_RULE_CREATED = "approval_rule_created"
    APPROVAL_RULE_UPDATED = "approval_rule_updated"
    APPROVAL_RULE_DELETED = "approval_rule_deleted"
    SYSTEM_SETTING_CHANGED = "system_setting_changed"
```

```
class Capability(str, enum.Enum):
    """The 9 actions the spec's permissions matrix (section 3) names -
    what app/services/permissions.py checks a role against before an
    endpoint proceeds. Deliberately this short, fixed list - not one
    capability per API route - matching the spec's own "do not
    overengineer" framing for section 4's approval rules, applied the same
    way here."""
```

```
    CR_CREATE = "cr_create"
    CR_EDIT = "cr_edit"
    STATUS_CHANGE = "status_change"
    ASSIGNMENT = "assignment"
    APPROVAL = "approval"
    REJECTION = "rejection"
    COMMENTS = "comments"
    REPORTS = "reports"
    ADMINISTRATION = "administration"
```

```
class ApprovalRuleType(str, enum.Enum):
    """What an ApprovalRule matches against - see
    app/models/approval_rule.py. RISK matches a risk bucket
    (low/medium/high/critical, the same buckets workflow rules.risk bucket
    already computes); CATEGORY matches a keyword found in the analysis
    category / target system / title (the same haystack the old hardcoded
    _CATEGORY_KEYWORD_APPROVALS matched against)."""
```

```
    RISK = "risk"
    CATEGORY = "category"
```

backend/app/models/impact_assessment.py

46 lines

```
"""ImpactAssessment model - Module 14 Phase 4.

One structured finding per fixed lens (Business/Technical/Customer/
Operational/Security/Data/Performance - see app.models.enums.ImpactCategory)
for a given analysis. A brand-new table (not an ALTER on an existing one),
so every column here is NOT NULL from day one - unlike 'certainty'/
'confidence' on Requirement/AffectedComponent/Risk, there's no pre-existing
data that predates this column to worry about.

Replaces the frontend-only "Business Impact"/"Technical Impact" text that
AnalysisDashboardPage.jsx's OverviewTab used to assemble itself out of
other fields (business-type Requirements, complexity reasoning, affected
component names) - that text was never a real, AI-assessed judgment about
business/technical impact specifically, just a repackaging of other
findings. This table is real, per-category AI output instead.
"""

from sqlalchemy import Float, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Certainty, ImpactCategory, ImpactLevel, sa_enum

class ImpactAssessment(Base):
    __tablename__ = "impact_assessments"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    category: Mapped[ImpactCategory] = mapped_column(
        sa_enum(ImpactCategory, "impact_assessment_category"), nullable=False
    )
    impact_level: Mapped[ImpactLevel] = mapped_column(
        sa_enum(ImpactLevel, "impact_assessment_level"), nullable=False
    )
    description: Mapped[str] = mapped_column(Text, nullable=False)
    # Same certainty/confidence pair every other AI finding in this app
    # carries (see Requirement/AffectedComponent/Risk) - NOT NULL here since
    # this table postdates Module 13, so there's no legacy row without them.
    certainty: Mapped[Certainty] = mapped_column(sa_enum(Certainty, "impact_assessment_certainty"), nullable=False)
    confidence: Mapped[float] = mapped_column(Float, nullable=False) # 0-100

    analysis: Mapped["Analysis"] = relationship(back_populates="impact_assessments")

    def __repr__(self) -> str:
        return f"<ImpactAssessment id={self.id} category={self.category}>"
```


backend/app/models/implementation_task.py

52 lines

```
"""ImplementationTask model - one task in the suggested implementation plan."""
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import AssignmentRole, Priority, sa_enum

class ImplementationTask(Base):
    __tablename__ = "implementation_tasks"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    task: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    component: Mapped[str] = mapped_column(String(255), nullable=False)
    priority: Mapped[Priority] = mapped_column(
        sa_enum(Priority, "implementation_task_priority"), default=Priority.MEDIUM, nullable=False
    )
    # Free-form on purpose (e.g. "4h", "2d", "3 story points") - AI-generated
    # estimates don't fit one numeric unit cleanly.
    estimated_effort: Mapped[str] = mapped_column(String(50), nullable=False)
    # Free text / comma-separated references to other tasks this depends on.
    dependencies: Mapped[Optional[str]] = mapped_column(Text, nullable=True, default="")

    # --- Module 17 Phase 1 (Test Cases & Implementation Plan 2.0) -----
    # Nullable: an ALTER-added column on a table that already has rows from
    # every analysis run before this module existed. Reuses the existing
    # per-CR AssignmentRole vocabulary (Module 12) rather than inventing a
    # second one - this is only ever a *suggestion* of who's suited to a
    # task, never an actual assignment (that stays Module 12's own
    # assignments system, per the architecture lock: no parallel ownership
    # system). None means the AI didn't have enough to suggest one.
    owner_suggestion: Mapped[Optional[AssignmentRole]] = mapped_column(
        sa_enum(AssignmentRole, "implementation_task_owner_suggestion"), nullable=True
    )

    # --- Module 17 Phase 6 (human editing) -----
    # Same "reviewed by/reviewed at, no separate boolean flag" pattern as
    # TestCase.edited by above and Requirement/SecurityFinding before it.
    edited_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    edited_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
    edit_reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="implementation_tasks")

    def __repr__(self) -> str:
        return f"<ImplementationTask id={self.id} task={self.task!r}>"
```

backend/app/models/indexed_file.py

63 lines

```
"""IndexedFile model - Module 15 Phase 1 (Repository Intelligence).

One row per source file a RepositoryScan kept (everything ignored -
node_modules/.git/build/binaries/secrets/oversized files/etc - is simply
never written here at all, not stored-then-hidden). Phase 1 only records
what's knowable just from walking the filesystem: the path, a guessed
language, and basic size/line stats. Phase 2 (code understanding) adds the
deeper extraction - imports/functions/classes/API routes/database and
config references - as new nullable columns on this same table, same
reasoning as every other phased column addition in this app: by the time
Phase 2 ships, this table already has real rows from Phase 1 testing, so
init db's plain 'ALTER TABLE ADD COLUMN' (no DEFAULT) would otherwise leave
them NULL despite a NOT NULL claim.
"""

from typing import Optional

from sqlalchemy import ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base


class IndexedFile(Base):
    __tablename__ = "indexed_files"

    id: Mapped[int] = mapped_column(primary_key=True)
    scan_id: Mapped[int] = mapped_column(ForeignKey("repository_scans.id"), nullable=False)
    # Relative to the scan's root_path, forward-slash separated regardless
    # of OS (see repository_scanner.py's use of PurePosixPath) - so a path
    # recorded on Windows reads/matches identically on any other machine.
    file_path: Mapped[str] = mapped_column(String(1024), nullable=False)
    # A best-effort guess from the file extension (e.g. "python",
    # "javascript") - None for a recognized-but-unmapped extension. Not an
    # enum: the set of languages a real repository can contain is open-
    # ended, and this is derived deterministically from the extension
    # rather than something the app filters/branches logic on.
    language: Mapped[Optional[str]] = mapped_column(String(50), nullable=True)
    size_bytes: Mapped[int] = mapped_column(Integer, nullable=False)
    line_count: Mapped[int] = mapped_column(Integer, nullable=False)

    # --- Module 15 Phase 2 (Code Understanding) -----
    # Each of these is a JSON-encoded list of strings (same "one text
    # column, not a new table" pattern already used for
    # ChangeRequest.tags/Analysis.security_analysis) - see
    # app/services/code_understanding.py for what actually populates them
    # and app/schemas/repository_scan.py for how they're read back out as
    # real lists. All nullable: an ALTER-added column on a table that
    # already has real rows from Phase 1 testing (same reasoning as every
    # other phased column addition in this app), and also simply absent
    # for a file type Phase 2 doesn't attempt to understand (e.g.
    # markdown, CSS) rather than ever being invented.
    imports: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    functions: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    classes: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    api_routes: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    database_references: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    config_references: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    scan: Mapped["RepositoryScan"] = relationship(back_populates="files")

    def __repr__(self) -> str:
        return f"<IndexedFile id={self.id} file_path={self.file_path!r}>"
```

backend/app/models/knowledge_chunk.py

49 lines

```
"""KnowledgeChunk model - Module 16 Phase 1 (Project Knowledge Base &
RAG).

One row per chunk a KnowledgeDocument is split into (Phase 2 does the
actual parsing/chunking - this table just holds the result). `embedding`
is defined now, in Phase 1, but stays unpopulated (nullable) until Phase 3
adds the real AI-embeddings call - declaring the column up front avoids a
second ALTER TABLE later, and it's nullable exactly like every other
"phase 2/3 fills this in" column elsewhere in this app (e.g.
IndexedFile.imports et al from Module 15 Phase 1 -> Phase 2).
"""
from typing import Optional

from sqlalchemy import ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base


class KnowledgeChunk(Base):
    __tablename__ = "knowledge_chunks"

    id: Mapped[int] = mapped_column(primary_key=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("knowledge_documents.id"), nullable=False)
    # 0-based position of this chunk within its document - lets the UI/API
    # show chunks in original document order regardless of insertion order.
    chunk_index: Mapped[int] = mapped_column(Integer, nullable=False)
    content: Mapped[str] = mapped_column(Text, nullable=False)
    # A best-effort heading/section this chunk fell under (e.g. "OTP
    # Authentication"), when the source format has one (markdown headings,
    # PDF section text) - None when the source has no such structure (e.g.
    # a plain .txt file). Used for Phase 5's source-attribution "Section:
    # ..." line from the module spec.
    section_label: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    char_count: Mapped[int] = mapped_column(Integer, nullable=False)
    # JSON-encoded list[float] - same "one text column, not a new table/
    # extension" pattern already used throughout this app (ChangeRequest.
    # tags, IndexedFile's Phase-2 columns) - populated by Module 16 Phase 3
    # (Embedding + Vector Search). Nullable: every chunk starts without an
    # embedding, and a chunk whose embedding call failed stays this way
    # rather than the whole document being silently marked READY without
    # actually being searchable.
    embedding: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    document: Mapped["KnowledgeDocument"] = relationship(back_populates="chunks")

    def __repr__(self) -> str:
        return f"<KnowledgeChunk id={self.id} document_id={self.document_id} chunk_index={self.chunk_index}>"
```

backend/app/models/knowledge_document.py

82 lines

```
"""KnowledgeDocument model - Module 16 Phase 1 (Project Knowledge Base &
RAG).

One row per uploaded knowledge-source document (architecture docs, API
docs, engineering guidelines, security policies, etc - see the module's
own spec section 1). This table is brand new (no pre-existing rows from an
earlier phase), so its columns can be NOT NULL from day one wherever the
value is always knowable at upload time - only genuinely optional fields
(error_message, processed_at) are nullable, following the same "new table
vs ALTER-added column" distinction already established for
RepositoryScan/IndexedFile in Module 15.

Deliberately separate from IndexedFile/RepositoryFinding (Module 15) - a
knowledge document is project "documentation" a human uploaded, not a
"source file" the app discovered by scanning a repository. Nothing here
duplicates or replaces Repository Intelligence; the two systems both feed
CR analysis but from different kinds of material (see
app/services/knowledge_documents.py's own docstring for the storage
layout, and why backend/data/knowledge_documents/ living under the same
data/ folder that Module 15's own scanner already ignores means uploaded
knowledge documents can never be accidentally re-indexed as source code).
"""

from datetime import datetime
from typing import List, Optional

from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import KnowledgeDocumentStatus, sa_enum


class KnowledgeDocument(Base):
    __tablename__ = "knowledge_documents"

    id: Mapped[int] = mapped_column(primary_key=True)
    # A human-readable title - defaults to the original filename at upload
    # time (see knowledge_documents.py::save_uploaded_document) but kept as
    # its own column rather than always deriving it, so a later phase could
    # let someone rename a document without touching the file on disk.
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    original_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    # The actual filename on disk under backend/data/knowledge_documents/
    # UUID-suffixed to avoid collisions between two uploads that happen to
    # share a name (see _safe_stem in the service module). Never shown to
    # the user directly; original filename/title are what the UI displays.
    stored_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    # Lowercase, includes the leading dot (".pdf", ".md", ".txt") - matches
    # the ALLOWED_EXTENSIONS the upload endpoint validates against.
    extension: Mapped[str] = mapped_column(String(20), nullable=False)
    size_bytes: Mapped[int] = mapped_column(Integer, nullable=False)
    status: Mapped[KnowledgeDocumentStatus] = mapped_column(
        sa_enum(KnowledgeDocumentStatus, "knowledge_document_status"),
        nullable=False,
        default=KnowledgeDocumentStatus.UPLOADED,
    )
    error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    # Denormalized count of this document's own chunks, kept in sync by
    # whatever phase actually creates them (Phase 2) - lets the Knowledge
    # Base UI (Phase 6) show "12 chunks" without a separate COUNT query.
    chunk_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    # Soft-delete/hide, not a hard delete - matches the module spec's
    # "Delete/archive if supported" wording and this app's existing
    # preference for keeping history (nothing in this codebase hard-
    # deletes a change request either). An archived document is excluded
    # from retrieval and from the default document list, but its row (and
    # any analysis that already cited it) still exists.
    archived: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    uploaded_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    uploaded_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    processed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    chunks: Mapped[List["KnowledgeChunk"]] = relationship(
        back_populates="document",
        cascade="all, delete-orphan",
        order_by="KnowledgeChunk.chunk_index",
    )
    uploader: Mapped[Optional["User"]] = relationship()

    def __repr__(self) -> str:
        return f"<KnowledgeDocument id={self.id} title={self.title!r} status={self.status}>"
```

backend/app/models/knowledge_evidence.py

70 lines

```
"""KnowledgeEvidence model - Module 16 (Project Knowledge Base & RAG),
Phase 4/5 (CR Analysis Integration + Source Attribution & Version
Awareness).

One row per knowledge-base chunk that was retrieved and handed to the AI
as grounding context for one specific Analysis run - the module's own
spec section 4 ("Record evidence") and section 7 ("RAG context should be
associated with the analysis that used it... Record useful metadata:
Document, Chunk, Retrieved, Analysis version").

Deliberately snapshots document_title/section_label/content/score at
retrieval time rather than relying only on a live foreign key to
KnowledgeChunk - a document's chunks are replaced wholesale whenever it's
reprocessed (see app/services/knowledge_chunker.py::process_document,
which deletes every existing chunk before creating a fresh set), so a
chunk id recorded here could otherwise end up pointing at a row that no
longer exists by the time someone looks at a past analysis's evidence.
`chunk_id` is kept anyway as a plain, unenforced column (not a foreign
key) purely as an informational "which chunk, if it still exists" pointer
- every field a reader actually needs to display is snapshotted directly.
`document_id` IS a real foreign key, since KnowledgeDocument rows are
never hard-deleted (only archived) - this mirrors exactly the choice
app/models/repository_finding.py already made for Module 15: store what
was actually true at the time directly, don't assume a live join stays
valid forever.

`change_request_version` is duplicated here from Analysis.change_request_
version (rather than only reachable by joining back to Analysis) for the
same reason RepositoryFinding does the same thing - a reader can tell
"was this evidence retrieved against an outdated version of the change
request" without an extra join. Since evidence is always generated fresh
as part of one specific analysis run (never independently re-retrieved
later, unlike a repository scan which can be re-run on its own), there is
only one staleness axis here - not three like RepositoryFinding's own
version-awareness - so the API layer simply reuses whatever
is_analysis_outdated() already computed for the parent Analysis rather
than a second bespoke computation (see app/api/change_requests.py).
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, Float, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base


class KnowledgeEvidence(Base):
    __tablename__ = "knowledge_evidence"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    document_id: Mapped[int] = mapped_column(ForeignKey("knowledge_documents.id"), nullable=False)
    # NOT a foreign key - see module docstring for why (the source chunk
    # may no longer exist by the time this row is read). Purely
    # informational.
    chunk_id: Mapped[int] = mapped_column(Integer, nullable=False)
    document_title: Mapped[str] = mapped_column(String(255), nullable=False)
    section_label: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    content_snippet: Mapped[str] = mapped_column(Text, nullable=False)
    similarity_score: Mapped[float] = mapped_column(Float, nullable=False)
    change_request_version: Mapped[int] = mapped_column(Integer, nullable=False)
    retrieved_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)

    analysis: Mapped["Analysis"] = relationship(back_populates="knowledge_evidence")
    document: Mapped["KnowledgeDocument"] = relationship()

    def __repr__(self) -> str:
        return f"<KnowledgeEvidence id={self.id} analysis_id={self.analysis_id} document_id={self.document_id}>"
```

backend/app/models/notification.py

48 lines

```
"""Notification model (Module 12) - lightweight in-app notifications only
(spec: "Do not implement email unless the existing application already
supports it" - it doesn't, so this is in-app only). Created by
app/services/notify.py whenever something happens that a user should know
about: assigned to a CR, tagged for approval, their CR's status changed,
their approval was requested/reminded, they were @mentioned, their AI
analysis went outdated, or a re-approval is needed.

Module 18 Phase 1 (Notifications, My Work & Personal Engineering Queue)
added approval_id: an optional link back to the specific Approval this
notification is about. Nullable, and only ever set today for a
DEADLINE_APPROACHING notification (app/services/deadlines.py) - it's what
lets that check ask "has THIS exact approval already been notified about?"
and never fire twice for the same one, rather than fuzzy-matching on
title/message text. Every other notification type is free to leave it
unset; nothing about them requires it.
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import Boolean, DateTime, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import NotificationType, sa_enum


class Notification(Base):
    __tablename__ = "notifications"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    type: Mapped[NotificationType] = mapped_column(sa_enum(NotificationType, "notification_type"), nullable=False)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    message: Mapped[str] = mapped_column(Text, nullable=False)
    change_request_id: Mapped[Optional[int]] = mapped_column(ForeignKey("change_requests.id"), nullable=True)
    # Module 18 Phase 1 - see this model's own docstring above.
    approval_id: Mapped[Optional[int]] = mapped_column(
        ForeignKey("change_request_approvals.id"), nullable=True
    )
    is_read: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False, index=True)

    user: Mapped["User"] = relationship()

    def __repr__(self) -> str:
        return f"<Notification user={self.user_id} type={self.type} read={self.is_read}>"
```

backend/app/models/repository_finding.py

82 lines

```
"""RepositoryFinding model - Module 15 Phase 3 (Repository Intelligence -
CR -> File Matching).

One row per (change request, source file) pair the matcher decided is
plausibly affected. Deliberately carries its own change request version,
analysis_id, and repository_scan_id rather than only pointing at "the
change request" - this is what makes a finding version-aware from day
one: it's tied to the *exact* CR version, AI analysis, and repository
scan it was generated against, so Phase 4 can compute whether any of
those three have since moved on without this table needing to change at
all (the same "tied to a CR + its version, never presented as current
once the CR has moved on" rule the POST-MODULE-12 architecture lock
requires of every future module).

A brand-new table, so every column here is NOT NULL from day one - same
reasoning as ImpactAssessment/SecurityFinding before it.
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, Float, ForeignKey, Integer, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import FileMatchLabel, ImpactLevel, sa_enum

class RepositoryFinding(Base):
    __tablename__ = "repository_findings"

    id: Mapped[int] = mapped_column(primary_key=True)
    change_request_id: Mapped[int] = mapped_column(ForeignKey("change_requests.id"), nullable=False)
    # The CR's own current version at the moment this finding was
    # generated - NOT a live/computed value, so it stays exactly what it
    # was even after the CR is edited again later (see Phase 4).
    change_request_version: Mapped[int] = mapped_column(Integer, nullable=False)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    repository_scan_id: Mapped[int] = mapped_column(ForeignKey("repository_scans.id"), nullable=False)
    indexed_file_id: Mapped[int] = mapped_column(ForeignKey("indexed_files.id"), nullable=False)
    impact_level: Mapped[ImpactLevel] = mapped_column(
        sa_enum(ImpactLevel, "repository_finding_impact_level"), nullable=False
    )
    confidence: Mapped[float] = mapped_column(Float, nullable=False) # 0-100
    # Computed server-side from 'confidence' (FileMatchLabel.from confidence)
    # - see that classmethod's own docstring for why this is never trusted
    # from the AI's raw response.
    match_label: Mapped[FileMatchLabel] = mapped_column(
        sa_enum(FileMatchLabel, "repository_finding_match_label"), nullable=False
    )
    reason: Mapped[str] = mapped_column(Text, nullable=False)
    evidence: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)

    change_request: Mapped["ChangeRequest"] = relationship()
    analysis: Mapped["Analysis"] = relationship()
    repository_scan: Mapped["RepositoryScan"] = relationship()
    indexed_file: Mapped["IndexedFile"] = relationship()

    # --- Read-only convenience for the API layer -----
    # RepositoryFindingRead (app/schemas/repository_finding.py) reads these
    # straight off the ORM object via from_attributes - avoids either a
    # manual join in every endpoint or duplicating file_path/language onto
    # this table (which would just be a second, drifttable copy of what
    # IndexedFile already owns).
    @property
    def file_path(self) -> str:
        return self.indexed_file.file_path

    @property
    def language(self) -> Optional[str]:
        return self.indexed_file.language

    @property
    def repository_scan_number(self) -> int:
        return self.repository_scan.scan_number

    def __repr__(self) -> str:
        return (
            f"<RepositoryFinding id={self.id} change_request_id={self.change_request_id} "
            f"indexed_file_id={self.indexed_file_id} match_label={self.match_label}>"
        )
```

backend/app/models/repository_scan.py

51 lines

```
"""RepositoryScan model - Module 15 Phase 1 (Repository Intelligence).

Records one pass of indexing a local repository/folder. Deliberately NOT
tied to a specific ChangeRequest - a scan indexes "the repository" in
general, the same way `git log` isn't scoped to any one change. Module 15
Phase 3 ties a "specific change request's" affected-file findings back to
one of these scans (plus the CR's own version and its analysis's version),
so the version-awareness the rest of the app relies on ("never present a
finding as current once something it depended on has moved on") applies
here too, without this table itself needing to know about change requests
at all.
15-04-19

from datetime import datetime
from typing import List, Optional

from sqlalchemy import DateTime, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import RepositoryScanStatus, sa_enum

class RepositoryScan(Base):
    __tablename__ = "repository_scans"

    id: Mapped[int] = mapped_column(primary_key=True)
    # Absolute path on disk that was scanned - kept as a string (not
    # resolved again at read time) so a scan's own record stays accurate
    # even if the app's configured REPOSITORY_ROOT changes later.
    root_path: Mapped[str] = mapped_column(String(1024), nullable=False)
    # The Nth scan of this exact root path, 1-based - lets the UI/API say
    # "Scan #3" instead of only an opaque database id, and gives Phase 3's
    # "repository scan version" its actual meaning.
    scan_number: Mapped[int] = mapped_column(Integer, nullable=False)
    status: Mapped[RepositoryScanStatus] = mapped_column(
        sa_enum(RepositoryScanStatus, "repository_scan_status"), nullable=False
    )
    error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    file_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    skipped_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    triggered_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    started_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    completed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    files: Mapped[List["IndexedFile"]] = relationship(
        back_populates="scan", cascade="all, delete-orphan", order_by="IndexedFile.file_path"
    )

    def __repr__(self) -> str:
        return f"<RepositoryScan id={self.id} root_path={self.root_path!r} scan_number={self.scan_number}>"
```


backend/app/models/requirement.py

53 lines

```
"""Requirement model - one extracted requirement belonging to an analysis."""
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, Float, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Certainty, Priority, RequirementType, ReviewStatus, sa_enum

class Requirement(Base):
    __tablename__ = "requirements"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    requirement_type: Mapped[RequirementType] = mapped_column(
        sa_enum(RequirementType, "requirement_type"), nullable=False
    )
    description: Mapped[str] = mapped_column(Text, nullable=False)
    priority: Mapped[Priority] = mapped_column(
        sa_enum(Priority, "requirement_priority"), default=Priority.MEDIUM, nullable=False
    )

    # --- Module 13 (AI Analysis 2.0) -----
    # Nullable: an ALTER-added column on an existing table, same reasoning
    # as ChangeRequest.current_version - rows from before this module ran
    # simply have no epistemic status recorded, and the UI treats that as
    # "not evaluated" rather than guessing one.
    certainty: Mapped[Optional[Certainty]] = mapped_column(
        sa_enum(Certainty, "requirement_certainty"), nullable=True
    )
    confidence: Mapped[Optional[float]] = mapped_column(Float, nullable=True) # 0-100
    evidence: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    # --- Module 14 Phase 6 (human review) -----
    # A human reviewer's verdict, entirely separate from 'certainty' /
    # 'confidence' / 'evidence' above - reviewing NEVER touches those AI
    # columns (see ReviewStatus's docstring). All nullable: an ALTER-added
    # column on an already-populated table, and NULL here specifically
    # means "no human has reviewed this yet."
    review_status: Mapped[Optional[ReviewStatus]] = mapped_column(
        sa_enum(ReviewStatus, "requirement_review_status"), nullable=True
    )
    review_comment: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    reviewed_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    reviewed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="requirements")

    def __repr__(self) -> str:
        return f"<Requirement id={self.id} type={self.requirement_type}>"
```

backend/app/models/risk.py

37 lines

```
"""Risk model - one identified risk for the change."""
from typing import Optional

from sqlalchemy import Float, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Certainty, RiskCategory, Severity, sa_enum

class Risk(Base):
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    category: Mapped[RiskCategory] = mapped_column(sa_enum(RiskCategory, "risk_category"), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    severity: Mapped[Severity] = mapped_column(sa_enum(Severity, "risk_severity"), nullable=False)
    probability: Mapped[float] = mapped_column(Float, nullable=False) # 0.0-1.0 - likelihood this risk occurs
    score: Mapped[float] = mapped_column(Float, nullable=False) # 0-100 combined risk score
    mitigation: Mapped[str] = mapped_column(Text, nullable=False)

    # --- Module 13 (AI Analysis 2.0) -----
    # Distinct from 'probability': probability is "how likely is this risk
    # to materialize", confidence is "how sure is the AI about this
    # assessment at all" - a risk can be rated low-probability with high
    # confidence, or high-probability with low confidence. Nullable for the
    # same reason as the other Module 13 additions (pre-existing rows).
    certainty: Mapped[Optional[Certainty]] = mapped_column(sa_enum(Certainty, "risk_certainty"), nullable=True)
    confidence: Mapped[Optional[float]] = mapped_column(Float, nullable=True) # 0-100
    evidence: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="risks")

    def __repr__(self) -> str:
        return f"<Risk id={self.id} category={self.category} severity={self.severity}>"
```

backend/app/models/role_permission.py

38 lines

```
"""RolePermission model (Module 21 spec section 3) - a plain (role,
capability) -> allowed lookup table, not a rule engine. Seeded on startup
(see app/database/init_db.py:: seed_role_permissions) with defaults that
match how the app already behaved before this module existed, so shipping
this never silently changes what anyone could already do; from then on an
admin can flip individual rows via PUT /api/admin/permissions.

One row per (role, capability) pair that's been explicitly set. A pair
with no row at all is treated as "not allowed" by
app/services/permissions.py::has_permission - but init_db seeds every
combination on first run, so in practice every pair always has a row.
12-04-19

from datetime import datetime

from sqlalchemy import Boolean, DateTime, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

from app.database.session import Base
from app.models.enums import Capability, UserRole, sa_enum

class RolePermission(Base):
    __tablename__ = "role_permissions"
    __table_args__ = (UniqueConstraint("role", "capability", name="uq_role_permission"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    role: Mapped[UserRole] = mapped_column(sa_enum(UserRole, "role_permission_role"), nullable=False, index=True)
    capability: Mapped[Capability] = mapped_column(
        sa_enum(Capability, "role_permission_capability"), nullable=False, index=True
    )
    allowed: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False
    )

    def __repr__(self) -> str:
        return f"<RolePermission role={self.role} capability={self.capability} allowed={self.allowed}>"
```

backend/app/models/security_finding.py

57 lines

```
"""SecurityFinding model - Module 14 Phase 5.

One structured finding per fixed lens (Authentication/Authorization/Data
Protection/Secrets/API Security/Rate Limiting/Privacy/Audit Logging/
Compliance - see app.models.enums.SecurityCategory) for a given analysis.
Replaces the old flat 'Analysis.security_analysis' JSON blob
({"concerns": [...], "summary": "..."}) as the primary security content
going forward - that column is kept (nullable, untouched) purely so
analyses created before this module still render something in the
Security tab, per the "never silently break old data" rule this app
follows throughout.

A brand-new table, so every column here is NOT NULL from day one - same
reasoning as ImpactAssessment (app/models/impact_assessment.py).
"""

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import SecurityCategory, SecurityFindingStatus, Severity, sa_enum

class SecurityFinding(Base):
    __tablename__ = "security_findings"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    category: Mapped[SecurityCategory] = mapped_column(
        sa_enum(SecurityCategory, "security_finding_category"), nullable=False
    )
    finding: Mapped[str] = mapped_column(Text, nullable=False)
    severity: Mapped[Severity] = mapped_column(sa_enum(Severity, "security_finding_severity"), nullable=False)
    evidence: Mapped[str] = mapped_column(Text, nullable=False)
    recommendation: Mapped[str] = mapped_column(Text, nullable=False)
    # AI sets OPEN/NOT APPLICABLE only; a human moves it to ACKNOWLEDGED/
    # RESOLVED later (Module 14 Phase 6 - not yet built).
    status: Mapped[SecurityFindingStatus] = mapped_column(
        sa_enum(SecurityFindingStatus, "security_finding_status"), nullable=False
    )

    # --- Module 14 Phase 6 (human review) -----
    # Who last changed `status` and when, plus their optional note -
    # nullable: an ALTER-added column on a table that already has rows
    # from Phase 5 (this app's real, populated database), same reasoning
    # as Requirement.reviewed by/reviewed at.
    review_comment: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    reviewed_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    reviewed_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="security_findings")

    def __repr__(self) -> str:
        return f"<SecurityFinding id={self.id} category={self.category}>"
```

backend/app/models/system_audit_log.py

50 lines

```
"""SystemAuditLog model (Module 21) - the system-level counterpart to
ChangeRequestHistory. ChangeRequestHistory is always scoped to one change
request; this table is for admin/configuration actions that belong to no
single change request at all - a role change, an account deactivation, an
edited approval rule, a changed permission or system setting.

Append-only, same as ChangeRequestHistory - see app/services/admin_audit.py,
the only place that's allowed to insert into this table (spec section 6:
"Do not allow normal users to edit audit history").

actor_user_id is nullable for the same reason ChangeRequestHistory.user_id
is: a rare system-generated entry with no human actor would use
actor_label instead, though in practice every Module 21 action so far is
always something an admin did.
@ 10-19

from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import AdminAuditAction, sa_enum

class SystemAuditLog(Base):
    __tablename__ = "system_audit_log"

    id: Mapped[int] = mapped_column(primary_key=True)
    actor_user_id: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    actor_label: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    action: Mapped[AdminAuditAction] = mapped_column(
        sa_enum(AdminAuditAction, "admin_audit_action"), nullable=False, index=True
    )
    # What this action was about - e.g. target_type="user", target_id="42"
    # for a role change on user #42; target_type="approval_rule",
    # target_id="7" for an edited rule. Loose (plain strings, not a
    # foreign key) on purpose - the target can be a user, a permission
    # row, a rule, or a system setting key, and this log must still be
    # readable even after the target itself is later deleted.
    target_type: Mapped[Optional[str]] = mapped_column(String(50), nullable=True)
    target_id: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    detail: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False, index=True)

    actor: Mapped[Optional["User"]] = relationship()

    def __repr__(self) -> str:
        return f"<SystemAuditLog action={self.action} target={self.target_type}:{self.target_id}>"
```

backend/app/models/system_setting.py

35 lines

```
"""SystemSetting model (Module 21 spec section 5) - a small admin-editable
key/value store, deliberately scoped to the handful of settings that are
actually safe to change at runtime (see app/services/system_settings.py
for exactly which keys exist and why). NOT a general configuration system:
Priority values, the workflow status graph, and AI provider / repository /
knowledge base configuration all stay code- or .env-driven, for the
reasons explained where each is surfaced to the admin UI (read-only there,
never written through this table).

value_json is a JSON-encoded value (usually a list of strings, e.g. the CR
categories list) so one table can hold different setting shapes without a
column per setting.
"""
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.database.session import Base


class SystemSetting(Base):
    __tablename__ = "system_settings"

    key: Mapped[str] = mapped_column(String(100), primary_key=True)
    value_json: Mapped[str] = mapped_column(Text, nullable=False)
    updated_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False
    )

    def __repr__(self) -> str:
        return f"<SystemSetting {self.key}>"
```

backend/app/models/test_case.py

54 lines

```
"""TestCase model - a suggested test case for verifying the change."""
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import Priority, TestType, sa_enum

class TestCase(Base):
    __tablename__ = "test_cases"

    id: Mapped[int] = mapped_column(primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("analyses.id"), nullable=False)
    test_id: Mapped[str] = mapped_column(String(50), nullable=False) # human-readable code, e.g. "TC-001"
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    test_type: Mapped[TestType] = mapped_column(sa_enum(TestType, "test_case_type"), nullable=False)
    priority: Mapped[Priority] = mapped_column(
        sa_enum(Priority, "test_case_priority"), default=Priority.MEDIUM, nullable=False
    )
    description: Mapped[str] = mapped_column(Text, nullable=False)
    expected_result: Mapped[str] = mapped_column(Text, nullable=False)

    # --- Module 17 Phase 1 (Test Cases & Implementation Plan 2.0) -----
    # Nullable: ALTER-added columns on a table that already has rows from
    # every analysis run before this module existed - same reasoning as
    # Requirement.certainty/confidence/evidence. `steps` is stored as a
    # JSON-encoded list[str] (one step per element, in order) - the same
    # "one text column, not a new table" pattern already used for
    # ChangeRequest.tags and KnowledgeChunk.embedding - parsed back into a
    # real list by TestCaseRead's own field validator.
    preconditions: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    steps: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    # --- Module 17 Phase 6 (human editing) -----
    # Who last edited this test case's own content and when, plus their
    # optional reason - exactly the same "reviewed by/reviewed at, no
    # separate boolean flag" pattern Requirement and SecurityFinding already
    # use (NULL edited by means "the AI's original content, never touched by
    # a human"). Editing updates title/description/etc. in place; the AI's
    # prior wording is never lost because it's permanently preserved as the
    # old value on the TEST CASE EDITED history row this creates - never a
    # second copy kept on this row itself.
    edited_by: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"), nullable=True)
    edited_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
    edit_reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

    analysis: Mapped["Analysis"] = relationship(back_populates="test_cases")

    def __repr__(self) -> str:
        return f"<TestCase id={self.id} test_id={self.test_id!r}>"
```

backend/app/models/user.py

38 lines

```
"""User model - an account that can submit and review change requests."""
from datetime import datetime
from typing import List, Optional

from sqlalchemy import Boolean, DateTime, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database.session import Base
from app.models.enums import UserRole, sa_enum

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[UserRole] = mapped_column(
        sa_enum(UserRole, "user_role"), default=UserRole.ENGINEER, nullable=False
    )
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, nullable=False)
    # Module 21 (Administration & Configuration): nullable (not
    # nullable=False) on purpose, same reasoning as ChangeRequest.
    # current version - a plain ALTER TABLE ADD COLUMN on everyone's
    # existing database leaves every pre-existing row NULL, and there's no
    # honest "was this account active before this column existed" answer
    # other than "yes." So NULL and True both mean active; only an
    # explicit False (set by an admin deactivating the account) means
    # inactive - see app/services/workflow_rules.py::is_active_user(),
    # the one place that reads this column.
    is_active: Mapped[Optional[bool]] = mapped_column(Boolean, default=True, nullable=True)

    change_requests: Mapped[List["ChangeRequest"]] = relationship(back_populates="creator")

    def __repr__(self) -> str:
        return f"<User id={self.id} email={self.email!r}>"
```


backend/app/schemas/__init__.py

1 lines

```
"""Pydantic schemas for the admin API (Module 21). User listing itself
reuses the existing UserRead schema (app/schemas/user.py) - no need for a
parallel "admin view of a user" shape when the plain one already excludes
password_hash and already carries is_active.
"""
```

```
from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator

from app.models.enums import AdminAuditAction, ApprovalRuleType, Capability, UserRole
```

```
class AdminUserCreate(BaseModel):
    """POST /api/admin/users body - an admin creating an account directly,
    as opposed to /auth/register (self-service signup). Every field
    required; role defaults to Engineer the same way self-registration
    always has, but an admin can set any role immediately instead of
    creating-then-editing.
```

```
    email is a plain str (not pydantic's EmailStr) - same convention as
    RegisterRequest/LoginRequest in app/schemas/auth.py, which never added
    the extra email-validator dependency EmailStr requires."""
```

```
    name: str
    email: str
    password: str
    role: UserRole = UserRole.ENGINEER
```

```
@field_validator("email")
@classmethod
def email_not_blank(cls, value: str) -> str:
    value = value.strip()
    if "@" not in value or len(value) < 5:
        raise ValueError("A valid email address is required.")
    return value
```

```
@field_validator("name")
@classmethod
def name_not_blank(cls, value: str) -> str:
    value = value.strip()
    if len(value) < 2:
        raise ValueError("Name is required.")
    return value
```

```
@field_validator("password")
@classmethod
def password_length(cls, value: str) -> str:
    if len(value) < 8:
        raise ValueError("Password must be at least 8 characters.")
    return value
```

```
class AdminUserUpdate(BaseModel):
    """PATCH /api/admin/users/{id} body - partial update semantics (only
    fields actually present in the request are touched), same convention
    as ChangeRequestUpdate. role and is_active can be changed independently
    or together in one call."""
```

```
    role: Optional[UserRole] = None
    is_active: Optional[bool] = None
```

```
class SystemAuditLogRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
    id: int
    actor_user_id: Optional[int] = None
    actor_label: Optional[str] = None
    # Not an ORM column - the API layer fills this in from the actor
    # relationship (or actor_label, for a system actor with no user_id)
    # before constructing this schema, so the frontend never has to make a
    # second call just to show a name in the audit log.
    actor_name: Optional[str] = None
    action: AdminAuditAction
    target_type: Optional[str] = None
    target_id: Optional[str] = None
    detail: Optional[str] = None
    created_at: datetime
```

```
@field_validator("action", mode="before")
@classmethod
def _action_value(cls, value):
    return value.value if hasattr(value, "value") else value
```

```
class SystemAuditLogResponse(BaseModel):
    items: List[SystemAuditLogRead]
    total: int
    limit: int
    offset: int
```

```
class RolePermissionRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
    role: UserRole
    capability: Capability
    allowed: bool
```

```
class RolePermissionUpdate(BaseModel):
    """One entry in the PUT /api/admin/permissions bulk-update body."""
```

```
    role: UserRole
    capability: Capability
    allowed: bool
```

```
class RolePermissionBulkUpdate(BaseModel):
    permissions: List[RolePermissionUpdate]
```

```
class ApprovalRuleRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
    id: int
    rule_type: ApprovalRuleType
    match_value: str
    approval_types: List[str]
    enabled: bool
    created_at: datetime
    updated_at: datetime
```

```

@field_validator("approval_types", mode="before")
@classmethod
def _parse_approval_types(cls, value):
    import json

    if isinstance(value, str):
        try:
            return json.loads(value)
        except (TypeError, ValueError):
            return []
    return value

class ApprovalRuleCreate(BaseModel):
    rule_type: ApprovalRuleType
    match_value: str
    approval_types: List[str]
    enabled: bool = True

    @field_validator("match_value")
    @classmethod
    def match_value_not_blank(cls, value: str) -> str:
        value = value.strip().lower()
        if not value:
            raise ValueError("This field can't be blank.")
        return value

    @field_validator("approval_types")
    @classmethod
    def approval_types_not_empty(cls, value: List[str]) -> List[str]:
        if not value:
            raise ValueError("At least one required approval type must be selected.")
        return value

class ApprovalRuleUpdate(BaseModel):
    match_value: Optional[str] = None
    approval_types: Optional[List[str]] = None
    enabled: Optional[bool] = None

    @field_validator("match_value")
    @classmethod
    def match_value_not_blank(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip().lower()
        if not value:
            raise ValueError("This field can't be blank.")
        return value

# --- System settings (spec section 5) -----

class AIProviderInfo(BaseModel):
    provider: str
    model: str
    api_key_configured: bool
    embeddings_supported: bool

class RepositoryInfo(BaseModel):
    repository_root: str
    last_scan_status: Optional[str] = None
    last_scan_file_count: Optional[int] = None
    last_scan_at: Optional[datetime] = None

class KnowledgeBaseInfo(BaseModel):
    embedding_model: str
    embeddings_supported: bool
    document_count: int

class SystemSettingsRead(BaseModel):
    """GET /api/admin/system-settings - cr_categories is the one genuinely
    editable setting (see PUT ../system-settings/cr-categories); the rest
    are read-only .env-derived info panels - see
    app/services/system_settings.py's own module docstring for why."""

    cr_categories: List[str]
    ai_provider: AIProviderInfo
    repository: RepositoryInfo
    knowledge_base: KnowledgeBaseInfo

class CrCategoriesUpdate(BaseModel):
    categories: List[str]

    @field_validator("categories")
    @classmethod
    def categories_not_empty(cls, value: List[str]) -> List[str]:
        cleaned = [v.strip() for v in value if v and v.strip()]
        if not cleaned:
            raise ValueError("At least one category is required.")
        # De-duplicate while preserving order - a category list is a
        # simple reference vocabulary, not something meant to hold the
        # same name twice.
        seen = set()
        deduped = []
        for item in cleaned:
            if item.lower() in seen:
                continue
            seen.add(item.lower())
            deduped.append(item)
        return deduped

```

backend/app/schemas/affected_component.py

30 lines

```
"""Pydantic schema for AffectedComponent (read-only for now - populated by the AI module, built later)."""
from typing import List, Optional

from pydantic import BaseModel, ConfigDict

from app.models.enums import Certainty, ComponentType, ImpactLevel


class AffectedComponentRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    component_name: str
    component_type: ComponentType
    impact_level: ImpactLevel
    reason: str
    confidence: float
    # Module 13 (AI Analysis 2.0) - nullable: rows from before this module
    # ran have no epistemic status recorded.
    certainty: Optional[Certainty] = None
    # Module 14 (Analysis & Impact Intelligence) - nullable, same reason.
    evidence: Optional[str] = None
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: the file path of any of this CR's own repository findings
    # that plausibly relate to this component. Empty (the default) when no
    # repository scan/match has run yet, or none of it overlaps.
    related_files: List[str] = []
```

```

"""Module 6 - AI Analysis Engine: the schema for the AI's structured JSON output.

This is what the raw text response from the AI provider gets parsed and
validated against ('AIAnalysisResult.model.validate_json(...)', *before*
any of it is trusted or persisted. If the AI's response doesn't match this
shape, validation fails loudly and the analysis engine reports a clean
error instead of saving garbage.

Field names mirror the exact JSON structure specified for this module, and
enum-typed fields reuse the app's existing controlled vocabularies
(app.models.enums) wherever one already fits, so persistence (mapping this
onto Analysis + its child tables) is a straightforward field-by-field copy.

A smaller/local model (e.g. Ollama) occasionally writes a "no answer"
placeholder - "None", "Not provided", "N/A" - into a field that's supposed
to be a real enum value or number, instead of actually picking one or
omitting the field. Rather than hard-failing on every one of those, the
helpers below substitute a conservative default and keep going - that's
still "handle validation errors gracefully" (the spec's own words), just
with fewer needless failures. A response that's wrong in some *other* way
still fails validation as before.
"""

import re
from typing import List, Optional

from pydantic import BaseModel, Field, field_validator, model_validator

from app.models.enums import (
    AssignmentRole,
    Certainty,
    ComplexityLevel,
    ComponentType,
    ConfidenceLevel,
    DependencyRelationship,
    DependencyType,
    ImpactCategory,
    ImpactLevel,
    Priority,
    RequirementType,
    RiskCategory,
    SecurityCategory,
    SecurityFindingStatus,
    Severity,
    TestType,
)

# Module 14 Phase 4 - the fixed set (and display order) every analysis must
# cover; see AIAnalysisResult.fill_missing_impact_categories below for why
# a missing one is filled in rather than left out.
IMPACT_CATEGORIES_ORDER = ["business", "technical", "customer", "operational", "security", "data", "performance"]

# Module 14 Phase 5 - same idea as IMPACT_CATEGORIES_ORDER above, but for
# Security findings; see AIAnalysisResult.fill_missing_security_categories.
SECURITY_CATEGORIES_ORDER = [
    "authentication",
    "authorization",
    "data_protection",
    "secrets",
    "api_security",
    "rate_limiting",
    "privacy",
    "audit_logging",
    "compliance",
]

_NULLISH = {
    "none",
    "n a",
    "na",
    "not provided",
    "not applicable",
    "unknown",
    "null",
    "",
    "not specified",
    "none specified",
    "tbd",
}

# Test-type near-misses actually seen in real AI responses (or trivially
# likely, given the same pattern) - each maps a plausible-but-not-exact name
# onto the TestType member closest to what it obviously meant, rather than
# failing the whole analysis over a naming choice. Every other unrecognized
# value still fails loudly, as intended (see this file's own docstring).
TEST_TYPE_VALUES = {member.value for member in TestType}
TEST_TYPE_SYNONYMS = {
    # DATA VALIDATION is the only member of this enum that isn't a single
    # obvious word - seen in practice shortened to just "validation".
    "validation": "data validation",
    "input validation": "data validation",
    "data validate": "data validation",
    # This enum deliberately has no separate FUNCTIONAL member (see
    # TestType's own docstring) - MANUAL is what a "functional" test (also
    # seen in practice), or a "smoke"/"sanity"/"acceptance"/"uat" one,
    # becomes here.
    "functional": "manual",
    "smoke": "manual",
    "sanity": "manual",
    "acceptance": "manual",
    "uat": "manual",
    "end to end": "e2e",
    "load": "performance",
    "stress": "performance",
}

def _normalize(value):
    """Lowercase/underscore-normalize an incoming string for enum matching.
    Returns None for a "no answer" placeholder (see module docstring) so
    callers can substitute a field-appropriate default instead of failing.
    Also treats a literal "[abc]" echoed back verbatim (a smaller model
    sometimes copies the pipe-separated option list from the prompt instead
    of picking one) as "no answer" too, for the same reason."""
    if isinstance(value, str):
        cleaned = value.strip().lower()
        if "|" in cleaned:
            return None
        cleaned = cleaned.replace(" ", "_").replace("-", "_").replace("/", "_")
        cleaned = "_".join(cleaned.split())
        if cleaned in _NULLISH:
            return None
        return cleaned.replace("_", "-")
    return value

```

```

def _clean_effort_text(value):
    """Effort-estimate fields are free text (no fixed vocabulary), so
    _normalize()'s enum handling doesn't apply - but the same weaker/local
    model habit shows up here differently: the JSON shape given to it uses
    illustrative example text like "e.g. 5-8 developer-days" for these
    fields, and a smaller model sometimes echoes that "e.g. ..." formatting
    back verbatim (with the numbers changed) instead of just answering
    directly. Stripping a leading "e.g. /"eg" recovers the real estimate
    without discarding it; an empty result after stripping falls back to
    the field's own "Insufficient information." default."""
    if isinstance(value, str):
        cleaned = re.sub(r"^\s*e\.?g\.?\s*[:,-]?s*", "", value, flags=re.IGNORECASE).strip()
        return cleaned or None
    return value

def _normalize_certainty(value):
    """Certainty gets its own normalizer rather than reusing _normalize()
    above: "unknown" is a genuine, meaningful value here (the AI is
    explicitly allowed to say "I can't determine this"), whereas everywhere
    else in this file "unknown" is treated as a placeholder for "no answer
    given." Reusing _normalize() would silently turn a real "unknown"
    certainty into the inferred default below - the opposite of what
    Module 13 needs."""
    if isinstance(value, str):
        cleaned = value.strip().lower().replace("-", " ").replace(" ", "_")
        if cleaned in ("known", "inferred", "unknown"):
            return cleaned
    return None

def _normalize_security_status(value):
    """SecurityFindingStatus gets its own normalizer for the same reason
    _normalize_certainty() above exists: "not applicable" is a genuine,
    meaningful value here (Module 14 Phase 5 - a category can genuinely
    not apply to a given change), but "not applicable" is ALSO a member of
    the generic _NULLISH placeholder set above (used everywhere else in
    this file to mean "no real answer given"). Reusing _normalize() would
    silently turn a real "not applicable" status into the "open" default
    below - discovered via test module14_phase5.py's
    test_pre_module14_phase5_shaped_response_fills_all_categories, which
    failed for exactly this reason until this dedicated normalizer was
    added."""
    if isinstance(value, str):
        cleaned = value.strip().lower().replace("-", " ").replace(" ", "_")
        if cleaned in ("open", "not_applicable", "acknowledged", "resolved"):
            return cleaned
    return None

def _safe_float(value, default: float):
    """Same idea as _normalize but for numeric fields (probability, score,
    confidence) - a stray "Not provided" string becomes 'default' instead
    of a validation error."""
    if isinstance(value, str):
        cleaned = value.strip().lower()
        if cleaned in _NULLISH:
            return default
        try:
            return float(cleaned)
        except ValueError:
            return default
    return value

class ClassificationResult(BaseModel):
    category: str = Field(..., description="One of the Module 6 classification categories.")
    confidence: float = Field(..., ge=0, le=1)
    reason: str

    @field_validator("confidence", mode="before")
    @classmethod
    def safe_confidence(cls, value):
        return _safe_float(value, default=0.5)

class RequirementItem(BaseModel):
    # "business_objective" and "acceptance_criteria" don't have their own
    # RequirementType value - they're stored as BUSINESS / FUNCTIONAL
    # respectively (see analysis_engine.py) with a label kept in the text.
    category: str = Field(
        ..., description="business_objective | functional | non_functional | constraint | assumption | acceptance_criteria"
    )
    description: str
    priority: Priority = Priority.MEDIUM
    # --- Module 13 (AI Analysis 2.0) ---
    certainty: Certainty = Certainty.INFERRED
    confidence: float = Field(70.0, ge=0, le=100)
    evidence: str = ""

    @field_validator("category", mode="before")
    @classmethod
    def normalize_category(cls, value):
        return _normalize(value) or "functional"

    @field_validator("priority", mode="before")
    @classmethod
    def normalize_priority(cls, value):
        return _normalize(value) or "medium"

    @field_validator("certainty", mode="before")
    @classmethod
    def normalize_certainty(cls, value):
        return _normalize_certainty(value) or "inferred"

    @field_validator("confidence", mode="before")
    @classmethod
    def safe_confidence(cls, value):
        return _safe_float(value, default=70.0)

class AffectedComponentItem(BaseModel):
    name: str
    type: ComponentType
    impact_level: ImpactLevel
    reason: str
    confidence: float = Field(..., ge=0, le=100)
    # Module 13 (AI Analysis 2.0)
    certainty: Certainty = Certainty.INFERRED
    # Module 14 (Analysis & Impact Intelligence) - what specifically in the
    # change request grounds this component being affected at all, same
    # role "evidence" already plays for Requirement/Risk.
    evidence: str = ""

    @field_validator("type", mode="before")
    @classmethod
    def normalize_type(cls, value):
        return _normalize(value) or "other"

    @field_validator("impact_level", mode="before")

```

```

@classmethod
def normalize_impact(cls, value):
    return _normalize(value) or "low"

@field_validator("confidence", mode="before")
@classmethod
def safe_confidence(cls, value):
    return _safe_float(value, default=50.0)

@field_validator("certainty", mode="before")
@classmethod
def normalize_certainty(cls, value):
    return _normalize_certainty(value) or "inferred"

class DependencyItem(BaseModel):
    name: str
    type: DependencyType
    impact_level: ImpactLevel
    reason: str
    # --- Module 14 (Analysis & Impact Intelligence) ---
    # How directly this change actually relies on it (see
    # app.models.enums.DependencyRelationship) - distinct from `type` above,
    # which is a category (service/library/...), not a directness rating.
    # Defaults to "direct" only because that's this field's most common,
    # least-surprising value absent other information - NOT a license to
    # guess a dependency into existence in the first place; see the "do not
    # invent dependencies" system-prompt rule in analysis_engine.py, which
    # governs whether an item belongs in this list at all.
    relationship: DependencyRelationship = DependencyRelationship.DIRECT
    risk: Severity = Severity.LOW

    @field_validator("type", mode="before")
    @classmethod
    def normalize_type(cls, value):
        return _normalize(value) or "internal"

    @field_validator("impact_level", mode="before")
    @classmethod
    def normalize_impact(cls, value):
        return _normalize(value) or "low"

    @field_validator("relationship", mode="before")
    @classmethod
    def normalize_relationship(cls, value):
        return _normalize(value) or "direct"

    @field_validator("risk", mode="before")
    @classmethod
    def normalize_risk(cls, value):
        return _normalize(value) or "low"

class RiskItem(BaseModel):
    category: RiskCategory
    description: str
    severity: Severity
    probability: float = Field(..., ge=0, le=1)
    score: float = Field(..., ge=0, le=100)
    explanation: str = ""
    mitigation: str = "Insufficient information."
    # --- Module 13 (AI Analysis 2.0) ---
    # Distinct from `probability`: probability is "how likely is this risk
    # to happen", confidence is "how sure is the AI about this assessment
    # at all."
    certainty: Certainty = Certainty.INFERRED
    confidence: float = Field(70.0, ge=0, le=100)

    @field_validator("category", mode="before")
    @classmethod
    def normalize_category(cls, value):
        return _normalize(value) or "operational"

    @field_validator("severity", mode="before")
    @classmethod
    def normalize_severity(cls, value):
        return _normalize(value) or "medium"

    @field_validator("probability", mode="before")
    @classmethod
    def safe_probability(cls, value):
        # Defaults to a mid-range "genuinely uncertain" value rather than
        # 0 - a risk the model couldn't quantify shouldn't silently read as
        # "no risk."
        return _safe_float(value, default=0.5)

    @field_validator("score", mode="before")
    @classmethod
    def safe_score(cls, value):
        return _safe_float(value, default=50.0)

    @field_validator("certainty", mode="before")
    @classmethod
    def normalize_certainty(cls, value):
        return _normalize_certainty(value) or "inferred"

    @field_validator("confidence", mode="before")
    @classmethod
    def safe_confidence(cls, value):
        return _safe_float(value, default=70.0)

class ImpactAssessmentItem(BaseModel):
    """Module 14 Phase 4: one structured finding for one of the 7 fixed
    impact lenses (see app.models.enums.ImpactCategory). Distinct from a
    RiskItem - this describes the change's general effect through this
    lens, not a specific thing that could go wrong."""
    category: ImpactCategory
    impact_level: ImpactLevel
    description: str
    certainty: Certainty = Certainty.INFERRED
    confidence: float = Field(70.0, ge=0, le=100)

    @field_validator("category", mode="before")
    @classmethod
    def normalize_category(cls, value):
        return _normalize(value) or "technical"

    @field_validator("impact_level", mode="before")
    @classmethod
    def normalize_impact(cls, value):
        return _normalize(value) or "medium"

    @field_validator("certainty", mode="before")
    @classmethod
    def normalize_certainty(cls, value):
        return _normalize_certainty(value) or "inferred"

    @field_validator("confidence", mode="before")
    @classmethod

```

```

def safe_confidence(cls, value):
    return _safe_float(value, default=70.0)

class SecurityFindingItem(BaseModel):
    """Module 14 Phase 5: one structured finding for one of the 9 fixed
    security lenses (see app.models.enums.SecurityCategory). Replaces the
    old flat concerns-list approach (SecurityAnalysis below, kept only for
    analyses that predate this) with a real per-category finding."""
    category: SecurityCategory
    finding: str
    severity: Severity
    evidence: str = "Insufficient information."
    recommendation: str = "Insufficient information."
    # Only the AI-settable subset - see SecurityFindingStatus's docstring.
    status: SecurityFindingStatus = SecurityFindingStatus.OPEN

    @field_validator("category", mode="before")
    @classmethod
    def normalize_category(cls, value):
        return _normalize(value) or "compliance"

    @field_validator("severity", mode="before")
    @classmethod
    def normalize_severity(cls, value):
        return _normalize(value) or "medium"

    @field_validator("status", mode="before")
    @classmethod
    def normalize_status(cls, value):
        normalized = _normalize_security_status(value) or "open"
        # The AI is never allowed to claim "acknowledged"/"resolved" -
        # those are reserved for a human reviewer (Module 14 Phase 6).
        return normalized if normalized in ("open", "not_applicable") else "open"

class SecurityAnalysis(BaseModel):
    concerns: List[str] = []
    summary: str = ""

class ComplexityResult(BaseModel):
    level: ComplexityLevel
    reasoning: str
    # Module 14 (Analysis & Impact Intelligence) - a coarse Low/Medium/High
    # rating of how sure the AI is in 'level' itself, distinct from the
    # overall classification confidence.
    confidence: ConfidenceLevel = ConfidenceLevel.MEDIUM

    @field_validator("level", mode="before")
    @classmethod
    def normalize_level(cls, value):
        return _normalize(value) or "medium"

    @field_validator("confidence", mode="before")
    @classmethod
    def normalize_confidence(cls, value):
        return _normalize(value) or "medium"

class EffortEstimate(BaseModel):
    backend: str = "Insufficient information."
    frontend: str = "Insufficient information."
    testing: str = "Insufficient information."
    total: str = "Insufficient information."
    # Module 14 (Analysis & Impact Intelligence) - same role as
    # ComplexityResult.confidence above, but for the effort estimate.
    confidence: ConfidenceLevel = ConfidenceLevel.MEDIUM

    @field_validator("backend", "frontend", "testing", "total", mode="before")
    @classmethod
    def clean_effort(cls, value):
        return _clean_effort_text(value) or "Insufficient information."

    @field_validator("confidence", mode="before")
    @classmethod
    def normalize_confidence(cls, value):
        return _normalize(value) or "medium"

class MissingInformationItem(BaseModel):
    question: str
    # Module 6 spec's own wording ("Critical / Important / Nice to have")
    # rather than the app's Priority enum - mapped onto Priority in
    # analysis_engine.py (critical->CRITICAL, important->HIGH,
    # nice_to_have->LOW) since there's no dedicated column for it.
    priority: str = Field(..., description="critical | important | nice_to_have")
    reason: str

    @field_validator("priority", mode="before")
    @classmethod
    def normalize(cls, value):
        value = _normalize(value) or "important"
        if value in ("nice_to_have", "nice-to-have", "nicetohave", "nice_to_have."):
            return "nice_to_have"
        return value

def _coerce_steps(value):
    """steps must end up as a List[str] here so analysis_engine.py can
    JSON-encode it straight into TestCase.steps (see that column's own
    docstring for why it's stored as JSON text). A weaker/local model
    occasionally returns one newline- or numbered-list-separated string
    instead of a real JSON array - split that into individual step
    strings (stripping any "1." / "2") numbering it added) rather than
    persisting one giant blob as a single "step". A genuine empty/missing
    answer becomes an empty list, never a validation error."""
    if value is None:
        return []
    if isinstance(value, list):
        return [str(item).strip() for item in value if str(item).strip()]
    if isinstance(value, str):
        lines = [re.sub(r"^\s*\d+[\.\.])\s*", "", line).strip() for line in value.splitlines()]
        return [line for line in lines if line]
    return []

class TestCaseItem(BaseModel):
    id: str
    title: str
    type: TestType
    priority: Priority = Priority.MEDIUM
    description: str
    expected_result: str = "Insufficient information."
    # --- Module 17 (Test Cases & Implementation Plan 2.0) Phase 2 -----
    # Both optional: a change too simple/vague to need real setup steps
    # still gets an honest "None." rather than an invented precondition,
    # and 'steps' legitimately empty means the AI didn't have enough to
    # give concrete steps (same "never invent, say so instead" rule as
    # everywhere else in this file).

```



```

preconditions: Optional[str] = None
steps: List[str] = Field(default_factory=list)

@field_validator("type", mode="before")
@classmethod
def normalize_type(cls, value):
    normalized = _normalize(value) or "manual"
    if normalized in TEST_TYPE_VALUES:
        return normalized
    # A trailing " test" ("unit_test", "negative_test", "boundary_test",
    # ...) is a very common way the AI names these that this enum's own
    # values never include - strip it and try again before falling back
    # to the explicit synonym table above.
    if normalized.endswith(" test"):
        stripped = normalized[:-len(" test")]
        if stripped in TEST_TYPE_VALUES:
            return stripped
        normalized = stripped
    return _TEST_TYPE_SYNONYMS.get(normalized, normalized)

@field_validator("priority", mode="before")
@classmethod
def normalize_priority(cls, value):
    return _normalize(value) or "medium"

@field_validator("preconditions", mode="before")
@classmethod
def normalize_preconditions(cls, value):
    # Reuses _clean_effort_text: same "strip a stray leading 'e.g.'"
    # and treat an empty result as no answer" behavior, just applied
    # to a free-text field other than an effort estimate.
    return _clean_effort_text(value)

@field_validator("steps", mode="before")
@classmethod
def coerce_steps(cls, value):
    return _coerce_steps(value)

class ImplementationTaskItem(BaseModel):
    task: str
    description: str
    component: str = "Unspecified"
    priority: Priority = Priority.MEDIUM
    estimated_effort: str = "Insufficient information."
    dependencies: Optional[str] = None
    # --- Module 17 (Test Cases & Implementation Plan 2.0) Phase 3 -----
    # A suggestion only, never an actual assignment - see
    # ImplementationTask.owner_suggestion's own docstring for the
    # architecture-lock reasoning. None means the AI had no well-founded
    # suggestion, never a guess just to fill the field.
    owner_suggestion: Optional[AssignmentRole] = None

    @field_validator("priority", mode="before")
    @classmethod
    def normalize_priority(cls, value):
        return _normalize(value) or "medium"

    @field_validator("estimated_effort", mode="before")
    @classmethod
    def clean_effort(cls, value):
        return _clean_effort_text(value) or "Insufficient information."

    @field_validator("owner_suggestion", mode="before")
    @classmethod
    def normalize_owner_suggestion(cls, value):
        # Reuses The generic placeholder-detection _normalize() already
        # relies on everywhere else in this file, but - unlike every other
        # _normalize * validator here - has NO fallback default: an
        # unrecognized or placeholder value becomes None (no suggestion)
        # rather than a fabricated role, since this is Module 12's own
        # controlled AssignmentRole vocabulary, not a free-text guess.
        normalized = _normalize(value)
        if normalized in {role.value for role in AssignmentRole}:
            return normalized
        return None

class RecommendationResult(BaseModel):
    # APPROVE | APPROVE WITH CONDITIONS | REQUIRES CLARIFICATION - kept as a
    # plain string (validated against the allowed set below) rather than the
    # ApprovalRecommendation enum directly, because that enum also carries
    # older values (NEEDS MORE INFO, REJECT) this module never emits.
    decision: str
    reasoning: str

    @field_validator("decision", mode="before")
    @classmethod
    def normalize(cls, value):
        # No safe default here reads as "approve" - if the model genuinely
        # couldn't decide, "requires clarification" is the only safe
        # fallback (never silently approves something).
        return _normalize(value) or "requires_clarification"

    @field_validator("decision")
    @classmethod
    def decision_allowed(cls, value: str) -> str:
        allowed = {"approve", "approve_with_conditions", "requires_clarification"}
        if value not in allowed:
            raise ValueError(f"recommendation.decision must be one of {sorted(allowed)}, got {value!r}")
        return value

class AIAnalysisResult(BaseModel):
    """The full structured output the AI must return for one change request."""

    summary: str
    classification: ClassificationResult
    requirements: List[RequirementItem] = []
    affected_components: List[AffectedComponentItem] = []
    dependencies: List[DependencyItem] = []
    risks: List[RiskItem] = []
    # Module 14 Phase 4
    impact_assessments: List[ImpactAssessmentItem] = []
    # Module 14 Phase 5 - security analysis (the old flat blob) is kept
    # alongside the new structured list below, purely as a short executive-
    # summary sentence the frontend can still show above the structured
    # cards - not a second source of the same finding-level detail.
    security_findings: List[SecurityFindingItem] = []
    security_analysis: SecurityAnalysis = SecurityAnalysis()
    complexity: ComplexityResult
    effort: EffortEstimate = EffortEstimate()
    missing_information: List[MissingInformationItem] = []
    test_cases: List[TestCaseItem] = []
    implementation_plan: List[ImplementationTaskItem] = []
    recommendation: RecommendationResult

    @model_validator(mode="after")
    def drop_placeholder_list_items(self):
        """A weaker/local model sometimes says "there are none" by adding a
        single list item whose *name* is itself the placeholder text (e.g.

```

```

a dependency literally named "None specified") instead of just
returning an empty list. name/description fields aren't enum-typed
so nothing above catches this - left as-is, it persists as a fake
dependency/component and renders as a real node in Module 8's
impact graph. Drop any item whose name is itself a "no answer"
placeholder rather than a real name, reusing the same normalize()
placeholder detection used everywhere else in this file."""
self.affected_components = [c for c in self.affected_components if _normalize(c.name) is not None]
self.dependencies = [d for d in self.dependencies if _normalize(d.name) is not None]
return self

@model_validator(mode="after")
def fill_missing_impact_categories(self):
    """Module 14 Phase 4: the spec calls for a card per lens - Business,
    Technical, Customer, Operational, Security, Data, Performance -
    every time, not just whichever ones the AI happened to mention. If
    a weaker/local model (or a genuinely uneventful change) leaves one
    out, fill it with an honest "no assessment given" placeholder
    (unknown certainty, 0 confidence, medium impact) rather than
    silently rendering six cards instead of seven - same "avoid false
    precision, never invent" rule this file follows everywhere else,
    just applied to *which categories exist* instead of *what a field
    says*. A duplicate category from the AI is left as a second entry
    rather than dropped - the frontend only reads the first match per
    category, so it's harmless, and dropping data the AI actually gave
    wouldn't fit "AI correctness" (never discard a real finding)."""
    seen = {item.category.value for item in self.impact_assessments}
    for category in IMPACT_CATEGORIES_ORDER:
        if category not in seen:
            self.impact_assessments.append(
                ImpactAssessmentItem(
                    category=category,
                    impact_level="medium",
                    description="Insufficient information.",
                    certainty="unknown",
                    confidence=0.0,
                )
            )
    return self

@model_validator(mode="after")
def fill_missing_security_categories(self):
    """Module 14 Phase 5: same reasoning as fill missing impact categories
    above, applied to the 9 fixed Security lenses - a category the AI
    left out gets an honest "nothing flagged" placeholder (severity
    low, status not applicable) instead of silently rendering fewer
    than 9 cards."""
    seen = {item.category.value for item in self.security_findings}
    for category in SECURITY_CATEGORIES_ORDER:
        if category not in seen:
            self.security_findings.append(
                SecurityFindingItem(
                    category=category,
                    finding="No security concern was identified for this category.",
                    severity="low",
                    evidence="Insufficient information.",
                    recommendation="No specific recommendation.",
                    status="not_applicable",
                )
            )
    return self

```

```

"""Pydantic schema for Analysis and its related AI-derived tables.

Read-only for now - nothing creates Analysis rows yet (that's the AI
module, built later). Defining the shape now means the API layer can
serialize these tables as soon as something starts writing to them.
"""

from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict

from app.models.enums import ApprovalRecommendation, ComplexityLevel, ConfidenceLevel
from app.schemas.affected_component import AffectedComponentRead
from app.schemas.change_request import FieldComparisonRead
from app.schemas.clarification_question import ClarificationQuestionRead
from app.schemas.dependency import DependencyRead
from app.schemas.implementation_task import ImplementationTaskRead
from app.schemas.impact_assessment import ImpactAssessmentRead
from app.schemas.knowledge_evidence import KnowledgeEvidenceRead
from app.schemas.requirement import RequirementRead
from app.schemas.risk import RiskRead
from app.schemas.security_finding import SecurityFindingRead
from app.schemas.test_case import TestCaseRead


class AnalysisRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    change_request_id: int
    summary: str
    category: str
    complexity: ComplexityLevel
    risk_score: float
    confidence_score: float
    recommendation: Optional[ApprovalRecommendation] = None
    created_at: datetime

    # Module 6 - narrative AI output that doesn't have its own table.
    # security analysis is stored as a JSON string; left as-is here (the
    # frontend can parse it, or a later module can give it its own schema).
    classification_reason: Optional[str] = None
    complexity_reasoning: Optional[str] = None
    security_analysis: Optional[str] = None
    effort_estimate: Optional[str] = None
    recommendation_reasoning: Optional[str] = None

    requirements: List[RequirementRead] = []
    affected_components: List[AffectedComponentRead] = []
    dependencies: List[DependencyRead] = []
    risks: List[RiskRead] = []
    # Module 14 Phase 4
    impact_assessments: List[ImpactAssessmentRead] = []
    # Module 14 Phase 5
    security_findings: List[SecurityFindingRead] = []
    clarification_questions: List[ClarificationQuestionRead] = []
    test_cases: List[TestCaseRead] = []
    implementation_tasks: List[ImplementationTaskRead] = []
    # Module 16 (Project Knowledge Base & RAG) Phase 4/5 - the knowledge-
    # base chunks (if any) retrieved and given to the AI as grounding
    # context for this analysis. is_outdated on each entry is set by the
    # API layer to match this analysis's own is_outdated below (see
    # KnowledgeEvidenceRead's own docstring for why there's only one
    # staleness axis here).
    knowledge_evidence: List[KnowledgeEvidenceRead] = []

    # Module 13 (AI Analysis 2.0) - which CR version this analysis ran
    # against (None only for the handful of pre-Module-12 rows init db
    # hasn't backfilled yet on a database that's never been started up
    # since - see app/database/init_db.py::backfill_workflow_data).
    change_request_version: Optional[int] = None

    # Module 13 Phase 3: computed fresh by the endpoint (never stored -
    # same rule as ChangeRequestDetail.is_analysis_outdated), true when the
    # change request has been edited since this was the CR's *latest*
    # analysis. Endpoints that return an existing analysis (GET
    # .../analysis, POST .../analyze) set this explicitly after building
    # the response; it defaults to False so nothing breaks if a caller
    # forgets to set it or feeds this schema a plain historical row.
    is_outdated: bool = False

    # Module 13 Phase 4: a single computed (not AI-written) sentence
    # combining 'recommendation' with what it actually takes to move this
    # change request forward - see
    # app/services/workflow_rules.py::recommendation_summary. Same
    # "computed, never stored" rule as is_outdated above.
    workflow_recommendation: Optional[str] = None

    # Module 14 (Analysis & Impact Intelligence) - both nullable: rows from
    # before this module ran have neither recorded.
    complexity_confidence: Optional[ConfidenceLevel] = None
    effort_confidence: Optional[ConfidenceLevel] = None


class AnalysisSummaryRead(BaseModel):
    """Module 13 Phase 2: a lightweight, one-line-per-analysis summary for
    GET /{id}/analyses - lets a caller (the frontend's future analysis
    history picker, or a person poking at the API directly) discover which
    analysis ids exist and pick two to compare, without pulling every
    child table for every past analysis."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    change_request_version: Optional[int] = None
    created_at: datetime
    category: str
    complexity: ComplexityLevel
    risk_score: float
    confidence_score: float
    recommendation: Optional[ApprovalRecommendation] = None


class AnalysisDeltaResponse(BaseModel):
    """Module 13 Phase 2: a computed (not AI-written) diff between two
    analyses for the same change request - see
    app/services/analysis_delta.py for why computed was chosen over an
    AI-written explanation, and for the honest limitation on the
    added/removed lists below (text-matched, not semantically matched)."""

    from_analysis_id: int
    to_analysis_id: int
    from_version: Optional[int] = None
    to_version: Optional[int] = None

```

```
fields: List[FieldComparisonRead]
requirements_added: List[str]
requirements_removed: List[str]
affected_components_added: List[str]
affected_components_removed: List[str]
is_significant_change: bool
significant_change_reasons: List[str]
```

```

"""Module 19 (Engineering Change Analytics) response shapes. Grows one
section at a time as each phase adds its own endpoint - see
app/services/analytics.py for the dataclasses these are built from and
app/api/analytics.py for the endpoints that return them.
"""
from __future__ import annotations
from typing import Optional
from pydantic import BaseModel, ConfigDict

class ExecutiveMetricsRead(BaseModel):
    """Spec section 1 - see app/services/analytics.py::ExecutiveMetrics for
    what each bucket means and why Cancelled sits outside all of them."""
    model_config = ConfigDict(from_attributes=True)

    total: int
    open: int
    pending_approval: int
    approved: int
    rejected: int
    in_progress: int
    closed: int

class WorkflowMetricsRead(BaseModel):
    """Spec section 2 - see app/services/analytics.py::WorkflowMetrics.
    Every avg_*_hours field is null, not 0, when nothing in the filtered
    set has reached that milestone yet."""
    model_config = ConfigDict(from_attributes=True)

    avg_time_to_analysis_hours: Optional[float] = None
    avg_approval_time_hours: Optional[float] = None
    avg_time_to_implementation_hours: Optional[float] = None
    avg_time_to_closure_hours: Optional[float] = None
    crs_waiting_for_approval: int
    crs_with_requested_changes: int

class ExecutiveWorkflowRead(BaseModel):
    """GET /api/analytics/executive's response - executive and workflow
    metrics are returned together since the frontend's Phase 6 dashboard
    header shows both at once, computed from the exact same filtered set of
    change requests in the same request."""
    executive: ExecutiveMetricsRead
    workflow: WorkflowMetricsRead

class RiskOverTimePoint(BaseModel):
    """One calendar month's risk-bucket counts, e.g.
    {"period": "2026-03", "low": 2, "medium": 1, "high": 0, "critical": 0}."""
    model_config = ConfigDict(extra="allow")

    period: str
    low: int = 0
    medium: int = 0
    high: int = 0
    critical: int = 0

class RiskAnalyticsRead(BaseModel):
    """GET /api/analytics/risk's response - spec section 3."""
    current_distribution: dict[str, int]
    over_time: list[RiskOverTimePoint]

class PendingByPersonEntry(BaseModel):
    approver_name: str
    count: int

class ApprovalBlockerEntry(BaseModel):
    approval_type_label: str
    count: int

class ApprovalBottlenecksRead(BaseModel):
    """GET /api/analytics/approval-bottlenecks's response - spec section 4.
    Person-level detail is deliberately limited to a name and a count (no
    email or other account details) per the spec's own "do not expose
    unnecessary sensitive information" instruction."""
    pending_by_type: dict[str, int]
    pending_by_person: list[PendingByPersonEntry]
    avg_duration_hours_by_type: dict[str, float]
    most_common_blockers: list[ApprovalBlockerEntry]

class MostChangedFieldEntry(BaseModel):
    field_label: str
    count: int

class ChangeAnalyticsRead(BaseModel):
    """GET /api/analytics/change's response - spec section 5."""
    model_config = ConfigDict(from_attributes=True)

    crs_with_multiple_revisions: int
    avg_versions_per_cr: Optional[float] = None
    most_changed_fields: list[MostChangedFieldEntry]
    crs_returned_for_clarification: int

class CrsPerOwnerEntry(BaseModel):
    owner_name: str
    count: int

class WorkloadAnalyticsRead(BaseModel):
    """GET /api/analytics/workload's response - spec section 6. Excludes
    the app's own bulk-seed load-test bot accounts (see
    app/services/analytics.py::BULK_SEED_EMAIL_PATTERN) - the one section
    of this module where that exclusion applies, per this module's own
    planning discussion with the user."""
    crs_per_owner: list[CrsPerOwnerEntry]
    pending_reviews: int

```

```
pending_approvals: int
overdue_tasks: int
```

```
class AIAnalyticsRead(BaseModel):
    """GET /api/analytics/ai's response - spec section 7, labeled "AI
    recommendation statistics" throughout (never "AI accuracy") since this
    app has no ground-truth outcome data - see
    app/services/analytics.py::AIAnalyticsMetrics for the full rationale."""

    model_config = ConfigDict(from_attributes=True)

    total_analyses: int
    re_analysis_rate: Optional[float] = None
    average_confidence: Optional[float] = None
    risk_changes_after_edits: int
    ai_recommended_approvals: int
    ai_analysis_failures: int
```

```

"""Pydantic schemas for Approval (Module 12 Phase 4).

Mirrors the shape of app/schemas/assignment.py: reads are built manually by
the API layer (not from attributes) since approver/requester names come
from the related User rows, not directly off the Approval row itself.
"""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import ApprovalStatus, ApprovalType


class ApprovalRequestCreate(BaseModel):
    """POST /{id}/approvals body - tag a real person for a specific kind of
    sign-off. `comment` here is an optional note from the requester (e.g.
    "please double check the OTP rate limiting") - stored on the audit
    trail, not on the Approval row itself (that `comment` column is the
    approver's own response - see app/models/approval.py)."""
    approval_type: ApprovalType
    approver_user_id: int
    comment: Optional[str] = None
    # Module 18 Phase 1: entirely optional - "please respond by ..." - see
    # app/models/approval.py's own docstring for why this is never required
    # or defaulted to anything.
    due_date: Optional[datetime] = None

    @field_validator("comment")
    @classmethod
    def blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None


class ApprovalRespond(BaseModel):
    """POST /{id}/approvals/{approval_id}/respond body - only the tagged
    approver may call this (see app/api/change_requests.py::respond to approval).
    `status` must be one of Approved/Rejected/Changes Requested - never
    Pending or Cancelled (validated in the endpoint against
    workflow rules.APPROVAL_RESPONSE_STATUSES so the 422 message can name
    exactly what went wrong)."""
    status: ApprovalStatus
    comment: Optional[str] = None

    @field_validator("comment")
    @classmethod
    def blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None


class ApprovalRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    change_request_id: int
    approval_type: ApprovalType
    approval_type_label: str
    status: ApprovalStatus
    status_label: str
    approver_id: int
    approver_name: str
    approver_email: str
    requested_by: int
    requested_by_name: str
    requested_at: datetime
    responded_at: Optional[datetime] = None
    comment: Optional[str] = None
    cr_version: int
    # Module 18 Phase 1 - see this model's own docstring for why this is
    # optional and never a required/defaulted value.
    due_date: Optional[datetime] = None
    # Computed fresh at read time from due_date (app/services/approvals.py
    # ::approval_due_status) - "overdue" / "due soon" / None. Never stored,
    # never meaningful once the approval has actually been decided.
    due_status: Optional[str] = None
    # True only for a still-PENDING approval whose cr_version has fallen
    # behind the CR's current_version (the CR was edited since this
    # approval was requested) - a decision already recorded is a historical
    # fact and never reported as outdated, no matter how the CR changes
    # after it. See app/services/approvals.py::is_approval_outdated.
    is_outdated: bool = False
    # Module 13 Phase 4: what the AI itself recommended on the analysis
    # this approval was requested against (approval.cr_version) - None if
    # no analysis exists for that version. Purely informational context,
    # never a constraint on the approver's decision.
    ai_recommendation: Optional[str] = None
    # True only once this approval has actually been decided (Approved /
    # Rejected / Changes Requested) AND that decision didn't match
    # ai recommendation above - see
    # app/services/workflow_rules.py::overrides_ai_recommendation. Always
    # False while still Pending or Cancelled; never implies the decision
    # was wrong, only that it diverged from the AI's own suggestion.
    overrode_ai_recommendation: bool = False


class RecommendedApprovalType(BaseModel):
    """One entry in GET /{id}/approvals/recommended - an AI-derived
    suggestion only (spec's "AI recommends, humans decide" - see
    app/services/workflow_rules.py::required_approval_types). Never creates
    an Approval row by itself.

    is_outdated (Module 22): true when this recommendation was computed
    from an analysis that no longer matches the change request's current
    version (app/services/workflow_rules.py::is_analysis_outdated) - the
    same check the main CR detail view already surfaces its own "this
    analysis may be outdated" banner from (Module 13). Before this field
    existed, this endpoint had no way to say so at all: editing a CR after
    analyzing it left recommended approvals silently suggesting approvals
    off stale data, with none of the "may be outdated" signal the rest of
    the app already gives elsewhere. Repeated on every item (rather than
    a separate top-level flag) so existing callers that treat this
    endpoint's response as a plain list of items keep working unchanged."""
    approval_type: ApprovalType
    label: str
    is_outdated: bool = False

```

backend/app/schemas/assignment.py

30 lines

```
"""Pydantic schemas for ChangeRequestAssignment (Module 12 Phase 3)."""
from datetime import datetime

from pydantic import BaseModel, ConfigDict

from app.models.enums import AssignmentRole


class AssignmentCreate(BaseModel):
    user_id: int
    role: AssignmentRole


class ChangeRequestAssignmentRead(BaseModel):
    """Built manually by the API layer (not from attributes) since
    user name/user email/assigned by name come from the related User rows,
    not directly off the ChangeRequestAssignment row itself."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    user_id: int
    user_name: str
    user_email: str
    role: AssignmentRole
    role_label: str
    assigned_by: int
    assigned_by_name: str
    assigned_at: datetime
```



```
"""Pydantic schemas for the authentication endpoints."""
from pydantic import BaseModel, field_validator, model_validator
from app.schemas.user import UserRead

class RegisterRequest(BaseModel):
    name: str
    email: str
    password: str
    confirm_password: str

    @field_validator("password")
    @classmethod
    def password_min_length(cls, value: str) -> str:
        if len(value) < 8:
            raise ValueError("Password must be at least 8 characters long.")
        return value

    @model_validator(mode="after")
    def passwords_match(self) -> "RegisterRequest":
        if self.password != self.confirm_password:
            raise ValueError("Passwords do not match.")
        return self

class LoginRequest(BaseModel):
    email: str
    password: str
    remember_me: bool = False

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int # seconds
    user: UserRead
```

```

"""Pydantic schemas for ChangeRequest."""
import json
from datetime import date, datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator

from app.models.enums import ApprovalRecommendation, ChangeRequestStatus, ComplexityLevel, Priority


def _parse_tags(value):
    """tags is stored as a JSON-encoded string column; every schema that
    reads it from the ORM needs to turn that back into a real list."""
    if value is None:
        return None
    if isinstance(value, list):
        return value
    try:
        return json.loads(value)
    except (TypeError, ValueError):
        return None


class ChangeRequestBase(BaseModel):
    title: str
    description: str
    business_objective: Optional[str] = None
    priority: Priority = Priority.MEDIUM
    requested_by: Optional[str] = None
    target_system: Optional[str] = None
    desired_deadline: Optional[date] = None
    # --- Module 12 (Enterprise Workflow) additional CR fields -----
    business_impact: Optional[str] = None
    technical_impact: Optional[str] = None
    customer_impact: Optional[str] = None
    environment: Optional[str] = None
    dependencies_note: Optional[str] = None
    compliance_requirements: Optional[str] = None
    tags: Optional[List[str]] = None

    @field_validator("tags", mode="before")
    @classmethod
    def _tags_from_json(cls, value):
        return _parse_tags(value)


class ChangeRequestCreate(BaseModel):
    """What the "New Change Request" form submits. created_by/status are NOT
    accepted from the client - created_by comes from the authenticated user
    and status is always set server-side to PENDING_ANALYSIS."""
    title: str
    description: str
    business_objective: Optional[str] = None
    priority: Priority = Priority.MEDIUM
    requested_by: str
    target_system: str
    desired_deadline: Optional[date] = None
    business_impact: Optional[str] = None
    technical_impact: Optional[str] = None
    customer_impact: Optional[str] = None
    environment: Optional[str] = None
    dependencies_note: Optional[str] = None
    compliance_requirements: Optional[str] = None
    tags: Optional[List[str]] = None

    @field_validator("title")
    @classmethod
    def title_length(cls, value: str) -> str:
        value = value.strip()
        if len(value) < 5:
            raise ValueError("Title must be at least 5 characters.")
        if len(value) > 255:
            raise ValueError("Title must be 255 characters or fewer.")
        return value

    @field_validator("description")
    @classmethod
    def description_length(cls, value: str) -> str:
        value = value.strip()
        if len(value) < 20:
            raise ValueError(
                "Description must be at least 20 characters - describe the change in enough detail to be useful."
            )
        if len(value) > 4000:
            raise ValueError("Description must be 4000 characters or fewer.")
        return value

    @field_validator("business_objective")
    @classmethod
    def business_objective_normalize(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None

    @field_validator("requested_by")
    @classmethod
    def requested_by_required(cls, value: str) -> str:
        value = value.strip()
        if len(value) < 2:
            raise ValueError("Requested By is required.")
        if len(value) > 255:
            raise ValueError("Requested By must be 255 characters or fewer.")
        return value

    @field_validator("target_system")
    @classmethod
    def target_system_required(cls, value: str) -> str:
        value = value.strip()
        if len(value) < 2:
            raise ValueError("Target System is required.")
        if len(value) > 255:
            raise ValueError("Target System must be 255 characters or fewer.")
        return value

    @field_validator("desired_deadline")
    @classmethod
    def deadline_not_in_past(cls, value: Optional[date]) -> Optional[date]:
        if value is not None and value < date.today():
            raise ValueError("Desired deadline can't be in the past.")
        return value

```

```

# --- Module 12: editing -----
#
# Every field is Optional with no default spelled out as "unset" - the API
# layer reads payload.model_dump(exclude_unset=True) so a field the client
# never included in the request body is left completely untouched (true
# partial-update / PATCH-like semantics on a PUT route), while a field
# explicitly sent as null clears it. Reuses ChangeRequestCreate's own
# validators where the same rule applies (title/description length, etc),
# but every validator here is skipped when the field is simply absent -
# only ChangeRequestCreate enforces "required at creation time".

class ChangeRequestUpdate(BaseModel):
    title: Optional[str] = None
    description: Optional[str] = None
    business_objective: Optional[str] = None
    priority: Optional[Priority] = None
    requested_by: Optional[str] = None
    target_system: Optional[str] = None
    desired_deadline: Optional[date] = None
    business_impact: Optional[str] = None
    technical_impact: Optional[str] = None
    customer_impact: Optional[str] = None
    environment: Optional[str] = None
    dependencies_note: Optional[str] = None
    compliance_requirements: Optional[str] = None
    tags: Optional[List[str]] = None

    @field_validator("title")
    @classmethod
    def title_length(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        if len(value) < 5:
            raise ValueError("Title must be at least 5 characters.")
        if len(value) > 255:
            raise ValueError("Title must be 255 characters or fewer.")
        return value

    @field_validator("description")
    @classmethod
    def description_length(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        if len(value) < 20:
            raise ValueError(
                "Description must be at least 20 characters - describe the change in enough detail to be useful."
            )
        if len(value) > 4000:
            raise ValueError("Description must be 4000 characters or fewer.")
        return value

    @field_validator("requested_by", "target_system")
    @classmethod
    def not_blank_if_provided(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        if len(value) < 2:
            raise ValueError("This field can't be blank.")
        return value

class ChangeRequestRead(ChangeRequestBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
    status: ChangeRequestStatus
    created_by: int
    created_at: datetime
    updated_at: datetime
    current_version: int = 1

    @field_validator("current_version", mode="before")
    @classmethod
    def default_version(cls, value):
        return value or 1

# --- Module 5: Change Request Management (list page + detail view) -----
#
# Everything below is read-only display shaping - no new AI logic, just
# presenting data that already exists (the raw `status` column, plus
# whatever the latest analysis/clarification questions already say).

class ChangeRequestListItem(BaseModel):
    """One row in the Change Requests table."""
    model_config = ConfigDict(from_attributes=True)

    id: int
    title: str
    priority: Priority
    status: ChangeRequestStatus
    # A human-facing status layered on top of the raw `status` column plus
    # whether/how the request has been analyzed - see
    # app/api/change_requests.py::effective_status(). One of: "draft",
    # "pending analysis", "analyzing", "requires clarification", "completed",
    # "approved". Never stored - always computed fresh from real rows.
    effective_status: str
    category: Optional[str] = None
    risk: Optional[str] = None
    complexity: Optional[str] = None
    created_at: datetime
    current_version: int = 1

    @field_validator("current_version", mode="before")
    @classmethod
    def default_version(cls, value):
        return value or 1

class ChangeRequestListResponse(BaseModel):
    items: List[ChangeRequestListItem]
    total: int
    limit: int
    offset: int

class ClarificationQuestionRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    question: str
    priority: Priority
    reason: str
    resolved: bool

```

```

class LatestAnalysisRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    summary: str
    category: str
    complexity: ComplexityLevel
    risk_score: float
    confidence_score: float
    recommendation: Optional[ApprovalRecommendation] = None
    created_at: datetime
    change_request_version: int = 1
    clarification_questions: List[ClarificationQuestionRead] = []

    @field_validator("change_request_version", mode="before")
    @classmethod
    def _default_version(cls, value):
        return value or 1

class StatusTransitionOption(BaseModel):
    """One legal next step from the CR's current status - what the
    "Change Status" picker offers (Module 12 Phase 3)."""

    status: ChangeRequestStatus
    label: str
    requires_reason: bool

class ChangeRequestDetail(ChangeRequestRead):
    effective_status: str
    latest_analysis: Optional[LatestAnalysisRead] = None
    # Module 12: computed fresh every request from current version vs
    # latest_analysis.change_request_version - never stored (see
    # app/services/workflow_rules.py::is_analysis_outdated).
    is_analysis_outdated: bool = False
    # Module 12 Phase 3: the real workflow lifecycle. `status` above is the
    # raw enum value (e.g. "in review"); status_label is its human label,
    # and available_transitions lists exactly what this CR can legally move
    # to next - both computed fresh from app/services/workflow_rules.py, so
    # the frontend never has to duplicate the status graph.
    status_label: str = ""
    available_transitions: List[StatusTransitionOption] = []

class ChangeStatusRequest(BaseModel):
    """PUT /{id}/status body (Module 12 Phase 3)."""

    status: ChangeRequestStatus
    reason: Optional[str] = None

# --- Module 12: versions, history, compare -----

class ChangeRequestVersionRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    version_number: int
    changed_by: int
    created_at: datetime
    change_summary: str

class ChangeRequestVersionDetail(ChangeRequestVersionRead):
    snapshot: dict

    @field_validator("snapshot", mode="before")
    @classmethod
    def _parse_snapshot(cls, value):
        if isinstance(value, str):
            return json.loads(value)
        return value

# --- Module 20: version-aware reports -----

class ReportableVersionRead(BaseModel):
    """One row of GET /{id}/report/versions - what the "generate a report
    for an earlier version" picker on the frontend renders from. Not an
    ORM-backed model (this is a computed view combining version + analysis
    data), so no from_attributes config - the API layer builds these by
    hand."""

    version_number: int
    is_current: bool
    change_summary: str
    created_at: datetime
    has_analysis: bool

class ChangeRequestHistoryRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    user_id: Optional[int] = None
    actor_label: Optional[str] = None
    action: str
    field_name: Optional[str] = None
    old_value: Optional[str] = None
    new_value: Optional[str] = None
    reason: Optional[str] = None
    version_number: Optional[int] = None
    created_at: datetime

    @field_validator("action", mode="before")
    @classmethod
    def _action_value(cls, value):
        return value.value if hasattr(value, "value") else value

class FieldComparisonRead(BaseModel):
    field: str
    label: str
    old_value: Optional[str] = None
    new_value: Optional[str] = None
    status: str

class VersionCompareResponse(BaseModel):
    from_version: int
    to_version: int
    fields: List[FieldComparisonRead]

class ChangeRequestUpdateResponse(BaseModel):
    """What PUT /change-requests/{id} returns: the updated CR plus a plain
    summary of what changed, so the frontend can show "here's what

```

changed" without re-deriving it from the history endpoint."

```
change_request: ChangeRequestDetail
changes: List[FieldComparisonRead]
new_version: Optional[int] = None
```

backend/app/schemas/clarification_question.py

44 lines

```
"""Pydantic schemas for ClarificationQuestion. Populated by the AI module;
answering one (see app/api/change_requests.py::answer_clarification_question)
is the one human-writable action on it - everything else stays exactly what
the AI produced."""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import Priority

class ClarificationQuestionRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    question: str
    priority: Priority
    reason: str
    resolved: bool
    # None until someone answers it (see ClarificationQuestionAnswer below).
    answer_text: Optional[str] = None
    answered_by: Optional[int] = None
    answered_at: Optional[datetime] = None

class ClarificationQuestionAnswer(BaseModel):
    """PATCH ../clarification-questions/{id}/answer body - the actual
    missing information, typed in by whoever has it. Answering always marks
    the question resolved; there's no separate "just mark resolved with no
    text" path, since an unexplained resolved=True would be indistinguishable
    from the AI's question having simply been ignored."""

    answer: str

    @field_validator("answer")
    @classmethod
    def _not_blank(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("An answer is required.")
        return value
```

backend/app/schemas/comment.py

49 lines

```
"""Pydantic schemas for ChangeRequestComment (Module 12 Phase 5)."""
from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator

class CommentCreate(BaseModel):
    body: str
    # One level of replies only (spec: "reply where practical" - a flat
    # two-level thread, not a full nested tree) - the API layer checks this
    # points at a top-level comment (parent_id is itself None) on the same
    # change request.
    parent_id: Optional[int] = None

    @field_validator("body")
    @classmethod
    def body_length(cls, value: str) -> str:
        value = value.strip()
        if len(value) < 1:
            raise ValueError("Comment can't be empty.")
        if len(value) > 4000:
            raise ValueError("Comment must be 4000 characters or fewer.")
        return value

class CommentResolve(BaseModel):
    resolved: bool

class CommentRead(BaseModel):
    """Built manually by the API layer (not from attributes) - user_name
    comes from the related User row, and mentioned_user_ids is recomputed
    fresh from 'body' at read time (mentions aren't stored structurally,
    see app/models/comment.py), same "compute, don't store" pattern as
    is_analysis_outdated / is_approval_outdated elsewhere in Module 12."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    change_request_id: int
    user_id: int
    user_name: str
    parent_id: Optional[int] = None
    body: str
    resolved: bool
    created_at: datetime
    mentioned_user_ids: List[int] = []
```

```

"""Response schemas for the dashboard summary endpoint.

Everything here is derived from real rows in the database at request time -
there is no hardcoded/sample data baked into these schemas or the endpoint
that fills them.
"""
from datetime import datetime
from typing import Dict, List, Optional

from pydantic import BaseModel


class DashboardMetrics(BaseModel):
    total_change_requests: int
    pending_analysis: int
    high_risk_changes: int
    approved_changes: int
    requires_clarification: int


class RecentChangeRequestItem(BaseModel):
    id: int
    title: str
    category: Optional[str] = None # None = not analyzed yet
    risk: Optional[str] = None # "low" | "medium" | "high" | "critical" | None
    status: str
    created_at: datetime


class DashboardSummary(BaseModel):
    metrics: DashboardMetrics
    # Keys: low, medium, high, critical, not_analyzed -> count of change requests
    risk_distribution: Dict[str, int]
    # Keys: the fixed category list (Feature, Bug Fix, Security, Database, API,
    # Infrastructure, Integration) plus "Other" -> count of change requests
    category_breakdown: Dict[str, int]
    recent_change_requests: List[RecentChangeRequestItem]


class DashboardDrillDownResponse(BaseModel):
    """The change requests behind one clicked dashboard tile, shown as a
    popup (see app/api/dashboard.py's own /dashboard/drill-down). Reuses
    RecentChangeRequestItem - same row shape the "Recent Change Requests"
    table already renders - rather than a second item schema, and is built
    from the exact same per-CR classification the summary endpoint's
    counts come from, so a tile's *unfiltered* count and this endpoint's
    with-no-date-filter total can never disagree.

    `total` is the count AFTER any date from/date to filter (so pagination
    reflects what's actually being paged through), which is why it isn't
    guaranteed to equal the tile's own number once a date range is applied
    - that's the filter working as intended, not a mismatch."""

    metric: str
    items: List[RecentChangeRequestItem]
    total: int
    page: int
    page_size: int

```


backend/app/schemas/dependency.py

26 lines

```
"""Pydantic schema for Dependency (read-only for now - populated by the AI module, built later)."""
from typing import List, Optional

from pydantic import BaseModel, ConfigDict

from app.models.enums import DependencyRelationship, DependencyType, ImpactLevel, Severity

class DependencyRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    dependency_name: str
    dependency_type: DependencyType
    impact_level: ImpactLevel
    reason: str
    # Module 14 (Analysis & Impact Intelligence) - both nullable: rows from
    # before this module ran have neither recorded.
    relationship_type: Optional[DependencyRelationship] = None
    risk_severity: Optional[Severity] = None
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: see AffectedComponentRead.related_files for the full rule.
    related_files: List[str] = []
```

backend/app/schemas/health.py

10 lines

```
from pydantic import BaseModel

class HealthResponse(BaseModel):
    status: str
    app_name: str
    app_env: str
    database: str
    ai_provider_configured: bool
```

backend/app/schemas/impact_assessment.py

23 lines

```
"""Pydantic read schema for ImpactAssessment - Module 14 Phase 4."""
from typing import List

from pydantic import BaseModel, ConfigDict

from app.models.enums import Certainty, ImpactCategory, ImpactLevel

class ImpactAssessmentRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    category: ImpactCategory
    impact_level: ImpactLevel
    description: str
    certainty: Certainty
    confidence: float
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: see AffectedComponentRead.related_files for the full rule.
    related_files: List[str] = []
```

backend/app/schemas/implementation_task.py

88 lines

```
"""Pydantic schemas for ImplementationTask."""
from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import AssignmentRole, Priority

class ImplementationTaskRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    task: str
    description: str
    component: str
    priority: Priority
    estimated_effort: str
    dependencies: Optional[str] = None
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: see AffectedComponentRead.related_files for the full rule.
    related_files: List[str] = []

    # --- Module 17 Phase 1 (Test Cases & Implementation Plan 2.0) -----
    # Nullable: rows from before this module ran have no suggestion
    # recorded. A suggestion only, never an actual assignment - see
    # models/implementation_task.py's own docstring.
    owner_suggestion: Optional[AssignmentRole] = None

    # Module 17 Phase 4 - which Requirement(s) this task actually
    # implements, computed the same keyword-overlap way related_files
    # above already is (never AI-generated - see TestCaseRead's own
    # docstring for why). Empty when nothing overlaps meaningfully.
    requirement_references: List[str] = []

    # Module 17 Phase 5 - see TestCaseRead's own docstring for both fields.
    analysis_version: Optional[int] = None
    is_outdated: bool = False

    # Module 17 Phase 6 (human editing) - see TestCaseRead's own docstring.
    edited_by: Optional[int] = None
    edited_at: Optional[datetime] = None
    edit_reason: Optional[str] = None

class ImplementationTaskUpdate(BaseModel):
    """PATCH /{change_request_id}/implementation-tasks/{task_id} body -
    see TestCaseUpdate's own docstring for the same "every field optional,
    reason optional, AI's prior wording preserved on the history row"
    reasoning, applied here to ImplementationTask instead."""

    task: Optional[str] = None
    description: Optional[str] = None
    component: Optional[str] = None
    priority: Optional[Priority] = None
    estimated_effort: Optional[str] = None
    dependencies: Optional[str] = None
    # A human may set this to any real AssignmentRole (or explicitly clear
    # it back to null) - unlike the AI, which can only ever suggest one via
    # ImplementationTaskItem's own normalizer, a human reviewer isn't
    # restricted to "clearly calls for it" - see workflow rules.
    # can review analysis findings for the permission gate itself.
    owner_suggestion: Optional[AssignmentRole] = None
    reason: Optional[str] = None

    @field_validator("task", "description", "component", "estimated_effort")
    @classmethod
    def _required_not_blank(cls, value: Optional[str]) -> Optional[str]:
        # task/description/component/estimated_effort are NOT NULL columns -
        # unlike dependencies/reason below, blanking one out here is a
        # rejected edit, not a legitimate "clear this field."
        if value is None:
            return None
        value = value.strip()
        if not value:
            raise ValueError("This field can't be blank.")
        return value

    @field_validator("dependencies", "reason")
    @classmethod
    def _optional_blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None
```

backend/app/schemas/knowledge_chunk.py

30 lines

```
"""Pydantic read schema for KnowledgeChunk - Module 16 Phase 2 (Project
Knowledge Base & RAG - Parsing + Chunking). Deliberately excludes
`embedding` itself (an internal JSON-encoded vector, meaningless to
display and not something any caller of this API needs) - only content a
person or the AI-analysis integration (Phase 4) would actually want to
read.

`is_embedded` (Phase 6 - Frontend) is a computed boolean, not a raw
passthrough of the `embedding` column - it answers "has this chunk been
embedded yet?" (for the Knowledge Base page's own status display) without
ever exposing the vector itself. Because it isn't a real attribute on the
KnowledgeChunk model, api/knowledge.py::list_document_chunks builds these
objects explicitly rather than relying on automatic from_attributes
mapping for this one field."""
from typing import Optional

from pydantic import BaseModel, ConfigDict


class KnowledgeChunkRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    document_id: int
    chunk_index: int
    section_label: Optional[str] = None
    content: str
    char_count: int
    is_embedded: bool = False
```

backend/app/schemas/knowledge_document.py

31 lines

```
"""Pydantic read schema for KnowledgeDocument - Module 16 Phase 1 (Project
Knowledge Base & RAG). Mirrors the shape of app/schemas/repository_scan.py's
RepositoryScanRead - a plain from attributes read model, no raw file
content ever included (the document's actual text lives on disk / in its
chunks, never in this response, for the same "never expose raw content in
an API response schema" reason Module 15 Phase 7 locked down for
IndexedFileRead)."""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict

from app.models.enums import KnowledgeDocumentStatus

class KnowledgeDocumentRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    title: str
    original_filename: str
    extension: str
    size_bytes: int
    status: KnowledgeDocumentStatus
    error_message: Optional[str] = None
    chunk_count: int
    archived: bool
    uploaded_by: Optional[int] = None
    uploaded_at: datetime
    processed_at: Optional[datetime] = None
```

backend/app/schemas/knowledge_evidence.py

34 lines

```
"""Pydantic read schema for KnowledgeEvidence - Module 16 (Project
Knowledge Base & RAG) Phase 4/5 (CR Analysis Integration + Source
Attribution & Version Awareness).

This is the "Evidence Used" list the module's own spec section 8 asks the
Analysis Dashboard to show - one entry per knowledge-base chunk that was
retrieved and given to the AI as context for a specific analysis.
"""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict


class KnowledgeEvidenceRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    document_id: int
    document_title: str
    section_label: Optional[str] = None
    content_snippet: str
    similarity_score: float
    change_request_version: int
    retrieved_at: datetime

    # Computed fresh by the API layer (app/api/change_requests.py), never
    # stored - same rule as AnalysisRead.is_outdated, and in fact always
    # set to exactly that same value: this evidence was retrieved as part
    # of one specific analysis run, so it's outdated exactly when that
    # analysis itself is (the change request has been edited since), never
    # independently.
    is_outdated: bool = False
```

backend/app/schemas/knowledge_search.py

20 lines

```
"""Pydantic read schema for one knowledge-base search result - Module 16
Phase 3 (Project Knowledge Base & RAG - Embedding + Vector Search).

Not from_attributes - app/services/knowledge_embeddings.py::search_chunks
returns plain dicts (each carrying a computed similarity `score` alongside
chunk/document data), not ORM objects, so Pydantic validates them as
ordinary dicts here."""
from typing import Optional

from pydantic import BaseModel


class KnowledgeSearchResult(BaseModel):
    score: float
    chunk_id: int
    document_id: int
    document_title: str
    section_label: Optional[str] = None
    content: str
```



```

"""Module 18 Phase 3 (Notifications, My Work & Personal Engineering Queue):
response shapes for GET /api/my-work/* - a personal dashboard built
entirely from data Module 12 (assignments/approvals/history) and Module 18
Phase 1/2 (due dates, notifications) already own. No new tables here -
every one of these is a read, never a write.
"""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel

from app.models.enums import ApprovalStatus, ApprovalType, AssignmentRole, Priority


class MyChangeRequestItem(BaseModel):
    """One row of "My Change Requests" - CRs the current user personally
    created. status is the same computed effective status
    (pending analysis/requires clarification/completed/approved/draft)
    list_change_requests already returns, not the raw DB column."""
    id: int
    title: str
    status: str
    priority: Priority
    risk: Optional[str] = None
    created_at: datetime


class MyApprovalItem(BaseModel):
    """One row of "My Approvals" (spec section 4) - every sign-off ever
    tagged to the current user, across every change request, regardless of
    which CR it's on. due_date/due_status mirror ApprovalRead's own fields
    (app/services/approvals.py::approval_due_status) - computed fresh, never
    stored."""
    id: int
    change_request_id: int
    change_request_title: str
    approval_type: ApprovalType
    approval_type_label: str
    status: ApprovalStatus
    status_label: str
    requested_by_name: str
    requested_at: datetime
    due_date: Optional[datetime] = None
    due_status: Optional[str] = None
    risk: Optional[str] = None


class MyReviewItem(BaseModel):
    """One row of "My Reviews" (spec section 5) - a change request where
    the current user holds at least one of Reviewer/Technical Lead/
    Security Reviewer/Owner. `roles` lists every role they hold on this
    specific CR (a person can hold more than one)."""
    change_request_id: int
    title: str
    status: str
    priority: Priority
    risk: Optional[str] = None
    roles: list[AssignmentRole]


class MyAssignmentItem(BaseModel):
    """One row of "My Assignments" - every change request the current user
    holds ANY assignment role on (broader than My Reviews above, which is
    limited to the four review-flavored roles)."""
    change_request_id: int
    title: str
    status: str
    priority: Priority
    risk: Optional[str] = None
    roles: list[AssignmentRole]


class ChangesRequestedItem(BaseModel):
    """One row of "Changes Requested From Me" - a change request the
    current user personally OWNS (created) where an approver has responded
    Changes Requested. This is "someone is waiting on ME to revise and
    resubmit" - the opposite direction from My Approvals, where the
    current user is the one being asked to decide on someone else's work."""
    change_request_id: int
    title: str
    approval_type: ApprovalType
    approval_type_label: str
    approver_name: str
    responded_at: Optional[datetime] = None
    comment: Optional[str] = None


class MyWorkSummary(BaseModel):
    """Cheap counts only, for an overview strip at the top of the My Work
    page - the actual rows for each section come from that section's own
    dedicated endpoint, fetched only once its tab is opened (spec section 7:
    efficient, never "load everything up front")."""
    my_change_requests: int
    my_approvals_pending: int
    my_reviews: int
    my_assignments: int
    changes_requested_from_me: int
    unread_mentions: int
    overdue_approvals: int

```

backend/app/schemas/notification.py

22 lines

```
"""Pydantic schema for Notification (Module 12 Phase 5)."""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict
from app.models.enums import NotificationType

class NotificationRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    type: NotificationType
    title: str
    message: str
    change_request_id: Optional[int] = None
    # Module 18 Phase 1 - see app/models/notification.py's own docstring.
    approval_id: Optional[int] = None
    is_read: bool
    created_at: datetime
```

backend/app/schemas/report.py

57 lines

```
"""Pydantic schemas for the Reports module (Module 24).

Reports are not their own database table - see
app/services/report_registry.py's module docstring for why. Every schema
here is a plain computed view built by hand in app/api/reports.py (and the
one small addition to app/api/change_requests.py), never from attributes
off a single ORM row, since a "report" is assembled from ChangeRequest +
Analysis + ChangeRequestHistory together.
"""

from __future__ import annotations

from datetime import datetime
from typing import Optional

from pydantic import BaseModel

from app.models.enums import ChangeRequestStatus


class ReportListItem(BaseModel):
    report_id: int
    change_request_id: int
    change_request_code: str
    change_request_title: str
    version: int
    is_current_version: bool
    is_historical_pull: bool
    cr_status: ChangeRequestStatus
    cr_status_label: str
    analysis_version: Optional[int] = None
    has_analysis: bool
    risk_score: Optional[float] = None
    risk_bucket: Optional[str] = None
    generated_by_id: Optional[int] = None
    generated_by_name: str
    generated_at: datetime
    pull_count: int


class ReportListResponse(BaseModel):
    items: list[ReportListItem]
    total: int


class ReportDetail(ReportListItem):
    """Same fields as a list row - the actual report CONTENT is the PDF
    itself (GET /api/reports/{id}/download), shown embedded on the report
    view page rather than re-implemented a second time as HTML (confirmed
    with the project owner before implementing)."""


class ReportStatsRead(BaseModel):
    total_reports: int
    change_requests_with_reports: int
    recent_reports: int
    outdated_reports: int
```

backend/app/schemas/repository_finding.py

98 lines

```
"""Module 15 Phase 3 (Repository Intelligence - CR -> File Matching).

`AffectedFileMatch`/`RepositoryMatchResult` are what the AI's raw JSON
response gets parsed and validated against (mirrors
app/schemas/ai_analysis.py's own pattern exactly) - a response that
doesn't match this shape fails validation loudly instead of persisting
something invented. `RepositoryFindingRead` is the read schema for an
already-persisted finding.
"""
from datetime import datetime

from pydantic import BaseModel, ConfigDict, Field, field_validator

from app.models.enums import FileMatchLabel, ImpactLevel

_NULLISH = {"none", "n a", "na", "not provided", "null", "", "not specified", "unknown", "tbd"}


def normalize_impact(value):
    """Same "no answer given" placeholder handling as
    app/schemas/ai_analysis.py::normalize, kept local rather than
    cross-imported (each schema file in this app is self-contained) -
    just enough of it for the one enum field this schema actually has."""
    if isinstance(value, str):
        cleaned = value.strip().lower().replace("-", "_").replace(" ", "_")
        if cleaned in _NULLISH:
            return None
        return cleaned
    return value


def _safe_confidence(value, default: float = 50.0):
    if isinstance(value, str):
        cleaned = value.strip().lower()
        if cleaned in _NULLISH:
            return default
        try:
            return float(cleaned)
        except ValueError:
            return default
    return value


class AffectedFileMatch(BaseModel):
    # Must exactly match one of the candidate file path values the AI was
    # given - app/services/repository_matcher.py drops (never persists)
    # any match whose file path isn't actually one of those candidates,
    # so a hallucinated path here is simply discarded, never invented
    # into a real finding.
    file_path: str
    impact_level: ImpactLevel = ImpactLevel.LOW
    confidence: float = Field(50.0, ge=0, le=100)
    reason: str = ""
    evidence: str = ""

    @field_validator("impact_level", mode="before")
    @classmethod
    def normalize_impact(cls, value):
        return _normalize_impact(value) or "low"

    @field_validator("confidence", mode="before")
    @classmethod
    def safe_confidence(cls, value):
        return _safe_confidence(value, default=50.0)


class RepositoryMatchResult(BaseModel):
    matches: list[AffectedFileMatch] = []


class RepositoryFindingRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    change_request_id: int
    change_request_version: int
    analysis_id: int
    repository_scan_id: int
    repository_scan_number: int
    indexed_file_id: int
    file_path: str
    language: str | None = None
    impact_level: ImpactLevel
    confidence: float
    match_label: FileMatchLabel
    reason: str
    evidence: str
    created_at: datetime
    # Module 15 Phase 4 (version awareness) - computed fresh on every
    # request by the API layer (app/services/workflow_rules.py::
    # repository_finding_outdated_reasons), never stored on the row
    # itself: this finding's own change request version/analysis id/
    # repository scan id above never change once written, only whether
    # they still match "the latest" of each does. Defaults below only
    # matter if a caller builds this schema without setting them.
    is_outdated: bool = False
    outdated_reasons: list[str] = []
```

backend/app/schemas/repository_scan.py

70 lines

```
"""Pydantic read schemas for RepositoryScan/IndexedFile - Module 15
Phases 1-2 (Repository Intelligence)."""
import json
from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator

from app.models.enums import RepositoryScanStatus


def _parse_json_list(value):
    """Imports/functions/classes/api_routes/database_references/
    config_references are each stored as a JSON-encoded string column
    (Module 15 Phase 2) - same pattern as ChangeRequest.tags
    (app/schemas/change_request.py::parse_tags), turned back into a real
    list here so the API always returns [] instead of null for a file
    with no findings of that kind."""
    if value is None:
        return []
    if isinstance(value, list):
        return value
    try:
        return json.loads(value)
    except (TypeError, ValueError):
        return []


class IndexedFileRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    file_path: str
    language: Optional[str] = None
    size_bytes: int
    line_count: int
    imports: List[str] = []
    functions: List[str] = []
    classes: List[str] = []
    api_routes: List[str] = []
    database_references: List[str] = []
    config_references: List[str] = []

    @field_validator(
        "imports", "functions", "classes", "api_routes", "database_references", "config_references",
        mode="before",
    )
    @classmethod
    def _lists_from_json(cls, value):
        return _parse_json_list(value)


class RepositoryScanRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    root_path: str
    scan_number: int
    status: RepositoryScanStatus
    error_message: Optional[str] = None
    file_count: int
    skipped_count: int
    triggered_by: Optional[int] = None
    started_at: datetime
    completed_at: Optional[datetime] = None


class RepositoryScanDetail(RepositoryScanRead):
    files: List[IndexedFileRead] = []
```

```
"""Pydantic schemas for Requirement."""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import Certainty, Priority, RequirementType, ReviewStatus

class RequirementRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    requirement_type: RequirementType
    description: str
    priority: Priority
    # Module 13 (AI Analysis 2.0) - all nullable: rows from before this
    # module ran have no epistemic status recorded.
    certainty: Optional[Certainty] = None
    confidence: Optional[float] = None
    evidence: Optional[str] = None
    # Module 14 Phase 6 (human review) - None means "no human has reviewed
    # this yet"; never set by the AI, never overwrites certainty/confidence/
    # evidence above.
    review_status: Optional[ReviewStatus] = None
    review_comment: Optional[str] = None
    reviewed_by: Optional[int] = None
    reviewed_at: Optional[datetime] = None

class RequirementReviewUpdate(BaseModel):
    """PATCH /{id}/requirements/{requirement_id}/review body - see
    app/api/change_requests.py::review_requirement. A comment is required
    when marking Needs Clarification (validated in the endpoint, same
    pattern as ApprovalRespond's required-comment-for-negative-outcomes
    rule), optional for Confirmed."""

    review_status: ReviewStatus
    comment: Optional[str] = None

    @field_validator("comment")
    @classmethod
    def blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None
```

backend/app/schemas/risk.py

29 lines

```
"""Pydantic schema for Risk (read-only for now - populated by the AI module, built later)."""
from typing import List, Optional

from pydantic import BaseModel, ConfigDict

from app.models.enums import Certainty, RiskCategory, Severity


class RiskRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    category: RiskCategory
    description: str
    severity: Severity
    probability: float
    score: float
    mitigation: str
    # Module 13 (AI Analysis 2.0) - all nullable: rows from before this
    # module ran have no epistemic status recorded.
    certainty: Optional[Certainty] = None
    confidence: Optional[float] = None
    evidence: Optional[str] = None
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: see AffectedComponentRead.related_files for the full rule.
    related_files: List[str] = []
```

backend/app/schemas/security_finding.py

46 lines

```
"""Pydantic schemas for SecurityFinding."""
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import SecurityCategory, SecurityFindingStatus, Severity

class SecurityFindingRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    category: SecurityCategory
    finding: str
    severity: Severity
    evidence: str
    recommendation: str
    status: SecurityFindingStatus
    # Module 14 Phase 6 (human review) - all nullable: no human has
    # touched `status` yet until these are set.
    review_comment: Optional[str] = None
    reviewed_by: Optional[int] = None
    reviewed_at: Optional[datetime] = None

class SecurityFindingStatusUpdate(BaseModel):
    """PATCH /{id}/security-findings/{finding_id}/status body - see
    app/api/change_requests.py::update_security_finding_status. Any of the
    4 status values may be set here (unlike the AI, which is restricted to
    open/not_applicable - see SecurityFindingItem.normalize_status) - a
    human reviewer has full authority to move a finding forward
    (Acknowledged/Resolved) or back (e.g. reopening a Resolved finding)."""

    status: SecurityFindingStatus
    comment: Optional[str] = None

    @field_validator("comment")
    @classmethod
    def blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None
```



```

"""Pydantic schemas for TestCase."""
import json
from datetime import datetime
from typing import List, Optional

from pydantic import BaseModel, ConfigDict, field_validator

from app.models.enums import Priority, TestType


def _parse_steps(value):
    """Steps is stored as a JSON-encoded list[str] column (see
    models/test_case.py's own docstring for why) - turn it back into a
    real list here, the same pattern ChangeRequest.tags already uses."""
    if value is None:
        return []
    if isinstance(value, list):
        return value
    try:
        parsed = json.loads(value)
    except (TypeError, ValueError):
        return []
    return parsed if isinstance(parsed, list) else []


class TestCaseRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    analysis_id: int
    test_id: str
    title: str
    test_type: TestType
    priority: Priority
    description: str
    expected_result: str
    # Module 15 Phase 5 (Repository Intelligence - Integration) - computed
    # fresh by the API layer (app/services/repository_linkage.py), never
    # stored: see AffectedComponentRead.related_files for the full rule.
    related_files: List[str] = []

    # --- Module 17 Phase 1 (Test Cases & Implementation Plan 2.0) -----
    # All nullable/empty-default: rows from before this module ran simply
    # have none of this recorded.
    preconditions: Optional[str] = None
    steps: List[str] = []

    @field_validator("steps", mode="before")
    @classmethod
    def _steps_from_json(cls, value):
        return _parse_steps(value)

    # Module 17 Phase 4 - which Requirement(s)/Risk(s) this test case
    # actually verifies, computed fresh by the API layer the same
    # keyword-overlap way related_files above already is (never AI-
    # generated - see services/traceability_linkage.py's own docstring for
    # why asking the AI to cite not-yet-existing IDs would be a
    # hallucination risk). Empty when nothing overlaps meaningfully.
    requirement_references: List[str] = []
    risk_references: List[str] = []

    # Module 17 Phase 5 - which analysis version this test case belongs to,
    # and whether that analysis (and therefore this test case) is stale -
    # both populated by the API layer from the parent Analysis, the same
    # "computed fresh, never stored" rule KnowledgeEvidenceRead.is_outdated
    # already follows (one staleness axis: has the CR been edited since).
    analysis_version: Optional[int] = None
    is_outdated: bool = False

    # Module 17 Phase 6 (human editing) - None means "the AI's original
    # content, never touched by a human" - same pattern as
    # Requirement.reviewed_by/SecurityFinding.reviewed_by.
    edited_by: Optional[int] = None
    edited_at: Optional[datetime] = None
    edit_reason: Optional[str] = None


class TestCaseUpdate(BaseModel):
    """PATCH /{change_request_id}/test-cases/{test_case_id} body - Module 17
    Phase 6 (human editing). Every field is optional - only ones actually
    present in the request are changed (see the endpoint's own
    model_dump(exclude_unset=True) call), so correcting one field never
    requires resending the whole test case. 'reason' is optional context
    for the audit trail, not required the way a Requirement's "Needs
    Clarification" comment is (see RequirementReviewUpdate) - editing an
    AI-generated test case isn't a rejection that demands justification.
    The AI's own prior wording is never lost - it's preserved as the
    old value on this edit's own TEST_CASE_EDITED history row (see
    app/api/change_requests.py::update_test_case), never a second column
    on this row itself."""

    title: Optional[str] = None
    description: Optional[str] = None
    test_type: Optional[TestType] = None
    priority: Optional[Priority] = None
    preconditions: Optional[str] = None
    steps: Optional[List[str]] = None
    expected_result: Optional[str] = None
    reason: Optional[str] = None

    @field_validator("title", "description", "expected_result")
    @classmethod
    def _required_not_blank(cls, value: Optional[str]) -> Optional[str]:
        # title/description/expected_result are NOT NULL columns - unlike
        # preconditions/reason below, blanking one out here is a rejected
        # edit, not a legitimate "clear this field."
        if value is None:
            return None
        value = value.strip()
        if not value:
            raise ValueError("This field can't be blank.")
        return value

    @field_validator("preconditions", "reason")
    @classmethod
    def _optional_blank_to_none(cls, value: Optional[str]) -> Optional[str]:
        if value is None:
            return None
        value = value.strip()
        return value or None

    @field_validator("steps")
    @classmethod
    def _clean_steps(cls, value: Optional[List[str]]) -> Optional[List[str]]:

```

```
if value is None:  
    return None  
return [str(item).strip() for item in value if str(item).strip()]
```

backend/app/schemas/user.py

36 lines

```
"""Pydantic schemas for User."""
from datetime import datetime

from pydantic import BaseModel, ConfigDict, field_validator
from app.models.enums import UserRole

class UserBase(BaseModel):
    name: str
    email: str
    role: UserRole = UserRole.ENGINEER

class UserCreate(UserBase):
    # Plaintext password - hashing happens in the auth module, which
    # doesn't exist yet. Nothing calls this schema yet; it just defines the
    # shape for that later module to build against.
    password: str

class UserRead(UserBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
    created_at: datetime
    # Module 21: NULL (every account predating this column) reads as
    # active - same convention as ChangeRequestRead.current version's own
    # "value or default" validator just below it in that file.
    is_active: bool = True

    @field_validator("is_active", mode="before")
    @classmethod
    def _default_active(cls, value):
        return True if value is None else value
```

backend/app/services/__init__.py

1 lines

backend/app/services/admin_audit.py

39 lines

```
"""Module 21: the only place allowed to write SystemAuditLog rows - same
"one writer, append-only" convention as app/services/history.py for
ChangeRequestHistory (spec section 6: "do not allow normal users to edit
audit history"). Every admin action that changes a user, a permission, an
approval rule, or a system setting calls record() rather than constructing
a SystemAuditLog row directly.
"""
from __future__ import annotations

from typing import Optional

from sqlalchemy.orm import Session

from app.models.enums import AdminAuditAction
from app.models.system_audit_log import SystemAuditLog


def record(
    db: Session,
    *,
    action: AdminAuditAction,
    actor_user_id: Optional[int] = None,
    actor_label: Optional[str] = None,
    target_type: Optional[str] = None,
    target_id: Optional[str] = None,
    detail: Optional[str] = None,
) -> SystemAuditLog:
    event = SystemAuditLog(
        actor_user_id=actor_user_id,
        actor_label=actor_label,
        action=action,
        target_type=target_type,
        target_id=target_id,
        detail=detail,
    )
    db.add(event)
    db.flush()
    return event
```

backend/app/services/ai/__init__.py

1 lines

```

"""
Provider-agnostic AI interface.

Every concrete provider (Anthropic, OpenAI, ...) implements this interface.
Nothing else in the codebase should import a specific provider's SDK directly -
always go through `get_ai_provider()` in `factory.py`, so swapping providers
later is a one-line config change (AI_PROVIDER env var), not a code change.
"""
from abc import ABC, abstractmethod

class AIProvider(ABC):
    """Minimal contract the rest of the app relies on.

    Analysis modules built in later steps (requirement extraction, risk
    analysis, etc.) will call `.complete()` with a prompt and get text back.
    Keeping the interface to a single method is deliberate: it's the
    smallest surface that still lets us swap providers freely.
    """

    @abstractmethod
    def complete(
        self,
        prompt: str,
        *,
        system: str | None = None,
        max_tokens: int = 1024,
        timeout: float | None = None,
    ) -> str:
        """Send a prompt to the underlying model and return its text response.

        Implementations should let timeouts and provider/network errors
        propagate as ordinary exceptions - callers (the analysis engine,
        Module 6) are responsible for catching them and turning them into
        a user-facing error, not this layer.
        """
        raise NotImplementedError

    @abstractmethod
    def is_configured(self) -> bool:
        """Whether this provider has the credentials it needs to actually run."""
        raise NotImplementedError

    def embed(self, texts: list[str], *, timeout: float | None = None) -> list[list[float]]:
        """Returns one embedding vector (a list of floats) per input text,
        in the same order as `texts`. Used by Module 16 (Project
        Knowledge Base & RAG) to turn a document's chunks - and a search
        query - into vectors for similarity search.

        Deliberately NOT an @abstractmethod: not every provider wired up
        in this app exposes an embeddings endpoint (Anthropic/Gemini/
        Ollama, as configured here, don't), and none of them should have
        to stub out a method they can't meaningfully implement just to
        remain instantiable. The default here raises a clear, catchable
        error instead - only OpenRouterProvider overrides it for real
        (OpenRouter has a native embeddings endpoint using the same API
        key already configured for chat completions)."""
        raise NotImplementedError(
            f"{type(self).__name__} does not support embeddings - "
            "set AI_PROVIDER=openrouter in backend/.env to use the knowledge base's search."
        )

```

backend/app/services/ai/factory.py

35 lines

```
"""Resolves the configured AI provider by name.

Adding a new provider later is: write a class implementing AIProvider,
register it in PROVIDERS below, and set AI_PROVIDER in .env - nothing
else in the app needs to change.
"""
from functools import lru_cache

from app.core.config import get_settings
from app.services.ai.base import AIProvider
from app.services.ai.providers.anthropic_provider import AnthropicProvider
from app.services.ai.providers.gemini_provider import GeminiProvider
from app.services.ai.providers.ollama_provider import OllamaProvider
from app.services.ai.providers.openrouter_provider import OpenRouterProvider

_PROVIDERS: dict[str, type[AIProvider]] = {
    "anthropic": AnthropicProvider,
    "gemini": GeminiProvider,
    "ollama": OllamaProvider,
    "openrouter": OpenRouterProvider,
    # "openai": OpenAIProvider, # add when that module is built
}

@lru_cache
def get_ai_provider() -> AIProvider:
    settings = get_settings()
    provider_cls = _PROVIDERS.get(settings.ai_provider)
    if provider_cls is None:
        raise ValueError(
            f"Unknown AI_PROVIDER '{settings.ai_provider}'. "
            f"Available: {'', '.join(_PROVIDERS)}"
        )
    return provider_cls()
```


backend/app/services/ai/providers/__init__.py

1 lines

backend/app/services/ai/providers/anthropic_provider.py

45 lines

```
"""Anthropic (Claude) implementation of the AIProvider interface."""
from app.core.config import get_settings
from app.services.ai.base import AIProvider

class AnthropicProvider(AIProvider):
    def __init__(self) -> None:
        settings = get_settings()
        self._api_key = settings.anthropic_api_key
        self._model = settings.anthropic_model
        self._client = None # created lazily, only once we know a key is present

    def is_configured(self) -> bool:
        return bool(self._api_key)

    def get_client(self):
        if self._client is None:
            import anthropic # imported lazily so the package is only required when actually used
            self._client = anthropic.Anthropic(api_key=self._api_key)
        return self._client

    def complete(
        self,
        prompt: str,
        *,
        system: str | None = None,
        max_tokens: int = 1024,
        timeout: float | None = None,
    ) -> str:
        if not self.is_configured():
            raise RuntimeError(
                "AnthropicProvider is not configured - set ANTHROPIC_API_KEY in backend/.env"
            )
        client = self.get_client()
        response = client.messages.create(
            model=self._model,
            max_tokens=max_tokens,
            system=system or "",
            messages=[{"role": "user", "content": prompt}],
            timeout=timeout,
        )
        return "".join(block.text for block in response.content if block.type == "text")
```

backend/app/services/ai/providers/gemini_provider.py

59 lines

```
"""Google Gemini implementation of the AIProvider interface.

Added as a free-tier alternative to Anthropic: a Gemini API key (created at
https://aistudio.google.com/apikey) doesn't require billing/a credit card
to get started, unlike an Anthropic key. Implemented as a plain REST call
via httpx (already a project dependency) rather than pulling in Google's
SDK, so switching AI_PROVIDER=gemini in backend/.env is the only change
needed - no new packages to install.
"""
import httpx

from app.core.config import get_settings
from app.services.ai.base import AIProvider

_API_BASE = "https://generativelanguage.googleapis.com/v1beta"

class GeminiProvider(AIProvider):
    def __init__(self) -> None:
        settings = get_settings()
        self._api_key = settings.gemini_api_key
        self._model = settings.gemini_model

    def is_configured(self) -> bool:
        return bool(self._api_key)

    def complete(
        self,
        prompt: str,
        *,
        system: str | None = None,
        max_tokens: int = 1024,
        timeout: float | None = None,
    ) -> str:
        if not self.is_configured():
            raise RuntimeError(
                "GeminiProvider is not configured - set GEMINI_API_KEY in backend/.env"
            )

        url = f"{_API_BASE}/models/{self._model}:generateContent"
        payload: dict = {
            "contents": [{"role": "user", "parts": [{"text": prompt}]}],
            "generationConfig": {"maxOutputTokens": max_tokens},
        }
        if system:
            payload["systemInstruction"] = {"parts": [{"text": system}]}

        response = httpx.post(url, params={"key": self._api_key}, json=payload, timeout=timeout)
        response.raise_for_status() # non-2xx (e.g. bad key, quota) becomes httpx.HTTPStatusError
        data = response.json()

        candidates = data.get("candidates") or []
        if not candidates:
            block_reason = (data.get("promptFeedback") or {}).get("blockReason")
            raise RuntimeError(f"Gemini returned no response (blockReason={block_reason!r}).")

        parts = candidates[0].get("content", {}).get("parts", [])
        return "".join(part.get("text", "") for part in parts)
```

backend/app/services/ai/providers/ollama_provider.py

64 lines

```
"""Ollama implementation of the AIProvider interface - runs a model
entirely on this machine via a local Ollama install (ollama.com). No API
key, no signup, no internet needed once a model is pulled - the only
"local" option among the three providers.

Implemented as a plain REST call via httpx (already a project dependency)
against Ollama's /api/chat endpoint, same pattern as the other providers.
Uses Ollama's "format: "json"" mode so the model is constrained to return
syntactically valid JSON - the analysis engine's Pydantic validation still
checks it actually matches the required shape.
"""
import httpx

from app.core.config import get_settings
from app.services.ai.base import AIProvider


class OllamaProvider(AIProvider):
    def __init__(self) -> None:
        settings = get_settings()
        self._base_url = settings.ollama_base_url.rstrip("/")
        self._model = settings.ollama_model

    def is_configured(self) -> bool:
        # No API key to check - Ollama either has a model pulled and
        # running or it doesn't. A model name being set is enough to try.
        return bool(self._model)

    def complete(
        self,
        prompt: str,
        *,
        system: str | None = None,
        max_tokens: int = 1024,
        timeout: float | None = None,
    ) -> str:
        if not self.is_configured():
            raise RuntimeError("OllamaProvider has no model configured - set OLLAMA_MODEL in backend/.env")

        messages = []
        if system:
            messages.append({"role": "system", "content": system})
        messages.append({"role": "user", "content": prompt})

        payload = {
            "model": self._model,
            "messages": messages,
            "stream": False,
            "format": "json",
            "options": {"num_predict": max_tokens},
        }

        try:
            response = httpx.post(f"{self._base_url}/api/chat", json=payload, timeout=timeout)
        except httpx.ConnectError as exc:
            raise RuntimeError(
                f"Couldn't reach Ollama at {self._base_url} - make sure the Ollama app is running "
                f"and you've run `ollama pull {self._model}`."
            ) from exc

        response.raise_for_status()
        data = response.json()
        return data.get("message", {}).get("content", "")
```

```

"""OpenRouter implementation of the AIProvider interface.

OpenRouter (openrouter.ai) is a single API that proxies many different
models (Anthropic, OpenAI, Meta, Google, and more) behind one OpenAI-
compatible endpoint and one API key - handy when you already have an
OpenRouter key and don't want to sign up for a model provider directly.

Implemented as a plain REST call via httpx (already a project dependency),
same pattern as gemini_provider.py / ollama_provider.py - no new package to
install. Set AI_PROVIDER=openrouter in backend/.env to use it.
"""
import httpx

from app.core.config import get_settings
from app.services.ai.base import AIProvider

API_URL = "https://openrouter.ai/api/v1/chat/completions"
# Module 16 (Project Knowledge Base & RAG): OpenRouter's own embeddings
# endpoint - same base API, same API key, an OpenAI-compatible request/
# response shape (see embed() below).
_EMBEDDINGS_API_URL = "https://openrouter.ai/api/v1/embeddings"

class OpenRouterProvider(AIProvider):
    def __init__(self) -> None:
        settings = get_settings()
        self._api_key = settings.openrouter_api_key
        self._model = settings.openrouter_model
        self._embedding_model = settings.openrouter_embedding_model

    def is_configured(self) -> bool:
        return bool(self._api_key)

    def _headers(self) -> dict:
        return {
            "Authorization": f"Bearer {self._api_key}",
            "Content-Type": "application/json",
            # Optional but recommended by OpenRouter for their own request
            # attribution/leaderboards - harmless to include, not required
            # for the API call to work.
            "HTTP-Referer": "http://localhost",
            "X-Title": "AI Change Request Analyzer",
        }

    def embed(self, texts: list[str], *, timeout: float | None = None) -> list[list[float]]:
        if not self.is_configured():
            raise RuntimeError(
                "OpenRouterProvider is not configured - set OPENROUTER_API_KEY in backend/.env"
            )
        if not texts:
            return []

        payload = {"model": self._embedding_model, "input": texts}
        response = httpx.post(_EMBEDDINGS_API_URL, headers=self._headers(), json=payload, timeout=timeout)
        response.raise_for_status()
        data = response.json()

        items = data.get("data") or []
        if len(items) != len(texts):
            error = data.get("error")
            raise RuntimeError(
                f"OpenRouter embeddings returned {len(items)} vectors for {len(texts)} inputs (error={error!r})."
            )
        # The OpenAI-compatible embeddings response includes each item's
        # own "index" - sort by it rather than trusting response list
        # order, so a vector always lines up with the text it belongs to
        # even if a provider ever returns them out of order.
        items_sorted = sorted(items, key=lambda item: item.get("index", 0))
        return [item["embedding"] for item in items_sorted]

    def complete(
        self,
        prompt: str,
        *,
        system: str | None = None,
        max_tokens: int = 1024,
        timeout: float | None = None,
    ) -> str:
        if not self.is_configured():
            raise RuntimeError(
                "OpenRouterProvider is not configured - set OPENROUTER_API_KEY in backend/.env"
            )

        messages = []
        if system:
            messages.append({"role": "system", "content": system})
        messages.append({"role": "user", "content": prompt})

        payload = {
            "model": self._model,
            "messages": messages,
            "max_tokens": max_tokens,
        }
        response = httpx.post(API_URL, headers=self._headers(), json=payload, timeout=timeout)
        response.raise_for_status() # non-2xx (bad key, model not found, out of credits) -> httpx.HTTPStatusError
        data = response.json()

        choices = data.get("choices") or []
        if not choices:
            error = data.get("error")
            raise RuntimeError(f"OpenRouter returned no response (error={error!r}).")

        return choices[0].get("message", {}).get("content", "") or ""

```

```

"""Module 13 Phase 2 (AI Analysis 2.0): a computed - not AI-written - diff
between two Analysis rows for the same change request.

Computed over AI-written was his own choice (Phase 2 planning): no extra
AI call/cost on every re-analysis, and it's never wrong about the numbers.
The honest tradeoff of that choice: the added/removed requirement and
affected-component lists below match by normalized text, not by meaning -
a requirement the AI worded slightly differently on the next run shows up
as one "removed" + one "added", not one "changed". That's a real
limitation of a computed diff (see PROJECT_REPORT.md's Known Limitations
once this phase is folded in), not a bug to fix here.

Reuses FieldComparison (app/services/versioning.py) - the exact same
field/label/old_value/new_value/status shape the CR-version "Compare
Versions" feature already uses - rather than inventing a second diff
shape for the same idea (architecture-lock rule 17: avoid duplicate
concepts).
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable, Optional

from app.models.analysis import Analysis
from app.models.approval_rule import ApprovalRule
from app.models.change_request import ChangeRequest
from app.services.versioning import FieldComparison
from app.services.workflow_rules import format_approval_types, required_approval_types, risk_bucket

def _normalize_text(value: str) -> str:
    return " ".join(value.strip().lower().split())

def _field(field: str, label: str, old, new) -> FieldComparison:
    old_s = "" if old is None else str(old)
    new_s = "" if new is None else str(new)
    return FieldComparison(
        field=field,
        label=label,
        old_value=old_s or None,
        new_value=new_s or None,
        status="unchanged" if old_s == new_s else "changed",
    )

@dataclass
class AnalysisDelta:
    fields: list[FieldComparison]
    requirements_added: list[str]
    requirements_removed: list[str]
    affected_components_added: list[str]
    affected_components_removed: list[str]
    is_significant_change: bool
    significant_change_reasons: list[str]
    # Module 18 Phase 2 (Notifications, My Work & Personal Engineering
    # Queue): the same two bucket values already computed below for the
    # "Risk" row in `fields` - broken out as their own typed fields so a
    # caller (app/api/change_requests.py's RISK_ESCALATED check) can
    # compare them directly rather than re-parsing significant change
    # reasons' own prose, or re-deriving risk_bucket() from the raw scores
    # a second time.
    risk_bucket_before: str
    risk_bucket_after: str

def compare_analyses(
    change_request: ChangeRequest,
    analysis_a: Analysis,
    analysis_b: Analysis,
    rules: Optional[Iterable[ApprovalRule]] = None,
) -> AnalysisDelta:
    """analysis_a is the earlier analysis, analysis_b the later one - both
    are/were real, persisted analyses (this never compares against
    anything hypothetical or in-progress). `rules` is passed straight
    through to required_approval_types (Module 21) - see that function's
    own docstring."""
    approvals_a = required_approval_types(change_request, analysis_a, rules)
    approvals_b = required_approval_types(change_request, analysis_b, rules)

    bucket_a = risk_bucket(analysis_a.risk_score)
    bucket_b = risk_bucket(analysis_b.risk_score)

    fields = [
        _field("risk_bucket", "Risk", bucket_a.capitalize(), bucket_b.capitalize()),
        _field("risk_score", "Risk score (0-100)", round(analysis_a.risk_score), round(analysis_b.risk_score)),
        _field(
            "complexity",
            "Complexity",
            analysis_a.complexity.value.replace(" ", "").title(),
            analysis_b.complexity.value.replace("_", "").title(),
        ),
        _field(
            "confidence_score",
            "Classification confidence (0-100)",
            round(analysis_a.confidence_score),
            round(analysis_b.confidence_score),
        ),
        _field("category", "Classification", analysis_a.category, analysis_b.category),
        _field(
            "recommendation",
            "Recommendation",
            analysis_a.recommendation.value if analysis_a.recommendation else None,
            analysis_b.recommendation.value if analysis_b.recommendation else None,
        ),
        _field("effort_estimate", "Effort estimate", analysis_a.effort_estimate, analysis_b.effort_estimate),
        _field("recommended_approvals", "Recommended approvals", format_approval_types(approvals_a), format_approval_types(approvals_b)),
        _field("requirement_count", "Requirements", len(analysis_a.requirements), len(analysis_b.requirements)),
        _field(
            "affected_component_count",
            "Affected components",
            len(analysis_a.affected_components),
            len(analysis_b.affected_components),
        ),
        _field("risk_count", "Risks identified", len(analysis_a.risks), len(analysis_b.risks)),
        _field("test_case_count", "Test cases", len(analysis_a.test_cases), len(analysis_b.test_cases)),
        _field(
            "implementation_task_count",
            "Implementation tasks",
            len(analysis_a.implementation_tasks),
            len(analysis_b.implementation_tasks),
        ),
    ],

```

```

reqs_a = { normalize_text(r.description): r.description for r in analysis_a.requirements}
reqs_b = { normalize_text(r.description): r.description for r in analysis_b.requirements}
requirements_added = sorted(reqs_b[k] for k in reqs_b.keys() - reqs_a.keys())
requirements_removed = sorted(reqs_a[k] for k in reqs_a.keys() - reqs_b.keys())

comps_a = { normalize_text(c.component_name): c.component_name for c in analysis_a.affected_components}
comps_b = { normalize_text(c.component_name): c.component_name for c in analysis_b.affected_components}
components_added = sorted(comps_b[k] for k in comps_b.keys() - comps_a.keys())
components_removed = sorted(comps_a[k] for k in comps_a.keys() - comps_b.keys())

reasons: list[str] = []
if bucket_a != bucket_b:
    reasons.append(f"Risk moved from {bucket_a} to {bucket_b}.")
if analysis_a.complexity != analysis_b.complexity:
    reasons.append(f"Complexity changed from {analysis_a.complexity.value} to {analysis_b.complexity.value}.")
if analysis_a.recommendation != analysis_b.recommendation:
    old_rec = analysis_a.recommendation.value if analysis_a.recommendation else "none"
    new_rec = analysis_b.recommendation.value if analysis_b.recommendation else "none"
    reasons.append(f"Recommendation changed from {old_rec} to {new_rec}.")
if approvals_a != approvals_b:
    reasons.append(f"Recommended approvals changed from [{format_approval_types(approvals_a)}] to [{format_approval_types(approvals_b)}].")

return AnalysisDelta(
    fields=fields,
    requirements_added=requirements_added,
    requirements_removed=requirements_removed,
    affected_components_added=components_added,
    affected_components_removed=components_removed,
    is_significant_change=bool(reasons),
    significant_change_reasons=reasons,
    risk_bucket_before=bucket_a,
    risk_bucket_after=bucket_b,
)

```

```

"""Module 6 - AI Analysis Engine.

The one place that: builds a prompt from a change request's *real* stored
fields (nothing invented), calls the configured AI provider, parses and
validates the response as JSON matching 'AIAnalysisResult'
(app/schemas/ai_analysis.py), and persists it across 'Analysis' and its
child tables (Requirement, AffectedComponent, Dependency, Risk,
ClarificationQuestion, TestCase, ImplementationTask - all pre-existing
tables from Module 1).

Deliberately does NOT touch repository files, GitHub, or RAG - the prompt
only ever includes what's already in the 'change_requests' row.

Module 13 (AI Analysis 2.0) Phase 1 additions: Requirement/AffectedComponent/
Risk findings now each carry a 'certainty' (known/inferred/unknown) label
and a per-finding 'confidence'; Requirement and Risk also carry an
'evidence' string. See app/schemas/ai_analysis.py's 'normalize certainty'
for why "unknown" gets its own normalizer instead of reusing the generic
placeholder-detection every other enum field here relies on.

Module 14 (Analysis & Impact Intelligence) Phase 2 additions:
AffectedComponent also carries 'evidence' now (the same role it plays on
Requirement/Risk); Dependency carries 'relationship' (direct/indirect/
potential - how directly this change actually relies on it) and 'risk'
(a severity rating of relying on it), plus an explicit "do not invent
dependencies" system-prompt rule.

Module 14 Phase 3 additions: Analysis carries a coarse Low/Medium/High
'complexity confidence' and 'effort confidence' - deliberately not a
finer-grained numeric confidence, since Complexity/Effort are already
plain-language estimates on purpose ("a range, not fake precision").

Module 14 Phase 4 additions: a new ImpactAssessment child table - one
structured finding per fixed lens (Business/Technical/Customer/
Operational/Security/Data/Performance, see app.models.enums.ImpactCategory)
with its own impact_level/certainty/confidence, replacing the frontend's
old habit of assembling a "Business Impact"/"Technical Impact" narrative
out of other fields.

Module 14 Phase 5 additions: a new SecurityFinding child table - one
structured finding per fixed lens (Authentication/Authorization/Data
Protection/Secrets/API Security/Rate Limiting/Privacy/Audit Logging/
Compliance, see app.models.enums.SecurityCategory) with its own severity/
evidence/recommendation/status, alongside (not replacing) the older flat
'security_analysis' blob - that column stays purely so pre-Phase-5
analyses keep rendering something in the Security tab.

Module 17 (Test Cases & Implementation Plan 2.0) Phase 2 additions: each
TestCase now also carries 'preconditions' (free text, "None." if nothing
beyond a normal starting state is required) and 'steps' (an ordered list,
JSON-encoded in storage - see TestCase.steps's own docstring). The system
prompt now also tells the AI to ground each test case in this same
analysis's own requirements/impact/risks/security findings rather than
generic boilerplate, and to use only the test types genuinely relevant to
this change (the type menu was widened to negative/boundary/api/ui/
data_validation in Phase 1, but nothing requires using all of them).

Module 17 Phase 3 additions: each ImplementationTask now also carries
'owner_suggestion' - reusing Module 12's own AssignmentRole vocabulary as
a *suggestion* only (never a real assignment - that stays Module 12's
own assignments system, per the architecture lock). The system prompt now
also tells the AI to break the plan into phase-oriented tasks only where a
phase is genuinely relevant to this change, and to suggest an owner only
when the task itself clearly calls for a particular kind of person.
"""

import json
import re
from datetime import datetime
from typing import Optional

from pydantic import ValidationError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models.analysis import Analysis
from app.models.affected_component import AffectedComponent
from app.models.change_request import ChangeRequest
from app.models.clarification_question import ClarificationQuestion
from app.models.dependency import Dependency
from app.models.enums import (
    ApprovalRecommendation,
    AssignmentRole,
    ChangeRequestStatus,
    HistoryAction,
    Priority,
    RequirementType,
)
from app.models.impact_assessment import ImpactAssessment
from app.models.implementation_task import ImplementationTask
from app.models.knowledge_evidence import KnowledgeEvidence
from app.models.requirement import Requirement
from app.models.risk import Risk
from app.models.security_finding import SecurityFinding
from app.models.test_case import TestCase
from app.schemas.ai_analysis import AIAnalysisResult
from app.services import history
from app.services.ai.factory import get_ai_provider
from app.services.knowledge_embeddings import KnowledgeEmbeddingError, search_chunks

class AnalysisError(Exception):
    """Raised for any failure running an AI analysis. 'status_code' tells
    the API layer which HTTP status to respond with:
    - 503: provider isn't configured (no API key)
    - 504: the provider call timed out
    - 502: the provider call failed for some other reason
    - 422: the provider responded, but the response wasn't valid JSON, or
    didn't match the required schema
    """

    def __init__(self, message: str, status_code: int = 502):
        super().__init__(message)
        self.status_code = status_code

CLASSIFICATION_CATEGORIES = [
    "Feature Enhancement",
    "Bug Fix",
    "Security",
    "Database",
    "API",
    "UI/UX",
    "Infrastructure",
    "Configuration",

```



```

    "Compliance",
    "Performance",
    "Integration",
]

RISK_CATEGORIES = [
    "security",
    "data",
    "performance",
    "availability",
    "integration",
    "regression",
    "compliance",
    "operational",
]

AFFECTED_COMPONENT_TYPES = ["frontend", "backend", "database", "api", "infrastructure", "third_party", "other"]
DEPENDENCY_TYPES = ["internal", "external", "service", "library", "database", "api"]
DEPENDENCY_RELATIONSHIPS = ["direct", "indirect", "potential"]
# Module 17 (Test Cases & Implementation Plan 2.0) Phase 2 added the last
# five values (negative/boundary/api/ui/data.validation) - a menu to draw
# from, not a mandate to use every one on every change (see the "never pad
# with irrelevant types" prompt guidance below, and TestType's own
# docstring in app/models/enums.py).
TEST_TYPES = [
    "unit",
    "integration",
    "e2e",
    "regression",
    "performance",
    "security",
    "manual",
    "negative",
    "boundary",
    "api",
    "ui",
    "data_validation",
]

IMPACT_CATEGORIES = ["business", "technical", "customer", "operational", "security", "data", "performance"]
SECURITY_CATEGORIES = [
    "authentication",
    "authorization",
    "data protection",
    "secrets",
    "api security",
    "rate limiting",
    "privacy",
    "audit logging",
    "compliance",
]

# Module 17 Phase 3 - Module 12's own per-CR role vocabulary (see
# app.models.enums.AssignmentRole), reused here as a menu of *suggestions*
# only - see ImplementationTask.owner_suggestion's own docstring for why
# this never becomes a second, parallel ownership system.
OWNER_SUGGESTIONS = [role.value for role in AssignmentRole]

SYSTEM_PROMPT = f"""You are an AI change-request analyst for a software engineering team. You are \
given ONE change request as submitted by a user. You may ALSO be given a "Relevant Documentation" \
section below the change request (Module 16 - Project Knowledge Base & RAG): excerpts retrieved from \
the team's own project knowledge base (architecture docs, API docs, engineering guidelines, security \
policies, deployment docs, etc.) - real project documentation, not something you invented, included only \
when the team's knowledge base actually had something plausibly related to this specific change. Beyond \
the change request and that optional documentation section, you have nothing else - no repository, no \
codebase, no architecture diagrams beyond what a provided excerpt itself contains, no other tickets.

Critical rule - do not hallucinate: beyond the change request and any Relevant Documentation section \
actually provided below, you have NOT been given any repository files, source code, API definitions, \
database schemas, or system architecture. Never invent or assume specifics about them (file names, \
endpoint paths, table names, class names, etc.) beyond what a provided documentation excerpt itself \
states. Where an assessment would require that kind of information and neither the change request nor \
the provided documentation covers it, say so explicitly (e.g. "Insufficient information.") rather than \
guessing. In your writing, clearly distinguish three kinds of statements: facts provided in the request \
(or in a provided documentation excerpt), your own inference/judgment based on those facts, and things \
that are simply unknown.

When a Relevant Documentation section is provided and a specific excerpt in it genuinely supports part \
of your assessment, you may mark that part "known" (per the certainty rules below) and should cite the \
excerpt's own "Source" and "Section" (exactly as given) directly in the relevant field's own evidence/ \
reason text - for example: "Per Security Guidelines.pdf, Section: OTP Authentication, rate-limiting is \
required for OTP attempts." Never treat an excerpt as confirming something it doesn't actually say, and \
never cite a Source/Section that wasn't actually given to you in this exact prompt. If no Relevant \
Documentation section is provided at all, or nothing in it is actually relevant to a given point, that \
point remains "inferred" or "unknown" exactly as it would with no documentation at all - never invent \
supporting documentation that wasn't given to you.

Respond with ONLY a single valid JSON object - no markdown code fences, no commentary before or after \
it. It must match this exact shape:

{{
  "summary": "2-4 sentence plain-language summary of the change request",
  "classification": {{"category": one of {CLASSIFICATION_CATEGORIES}, "confidence": 0.0-1.0, "reason": "why"}},
  "requirements": [
    {{"category": "business objective|functional|non functional|constraint|assumption|acceptance_criteria",
      "description": "...", "priority": "low|medium|high|critical",
      "certainty": "known|inferred|unknown", "confidence": 0-100, "evidence": "..."}},
  ],
  "affected_components": [
    {{"name": "...", "type": one of {AFFECTED_COMPONENT_TYPES}, "impact_level": "low|medium|high|critical",
      "reason": "...", "confidence": 0-100, "certainty": "known|inferred|unknown", "evidence": "..."}},
  ],
  "dependencies": [
    {{"name": "...", "type": one of {DEPENDENCY_TYPES}, "impact_level": "low|medium|high|critical", "reason": "...",
      "relationship": one of {DEPENDENCY_RELATIONSHIPS}, "risk": "low|medium|high|critical"}},
  ],
  "risks": [
    {{"category": one of {RISK_CATEGORIES}, "description": "...", "severity": "low|medium|high|critical",
      "probability": 0.0-1.0, "score": 0-100, "explanation": "...", "mitigation": "...",
      "certainty": "known|inferred|unknown", "confidence": 0-100}},
  ],
  "impact_assessments": [
    {{"category": one of {IMPACT_CATEGORIES}, "impact_level": "low|medium|high|critical", "description": "...",
      "certainty": "known|inferred|unknown", "confidence": 0-100}},
  ],
  "security_findings": [
    {{"category": one of {SECURITY_CATEGORIES}, "finding": "...", "severity": "low|medium|high|critical",
      "evidence": "...", "recommendation": "...", "status": "open|not_applicable"}},
  ],
  "security_analysis": {{"concerns": [...], "summary": "..."},
  "complexity": {{"level": "low|medium|high|very_high", "reasoning": "...", "confidence": "low|medium|high"}},
  "effort": {{"backend": "<your estimate>", "frontend": "<your estimate>", "testing": "<your estimate>",
    "total": "<your estimate>", "confidence": "low|medium|high"}},
  "missing_information": [
    {{"question": "...", "priority": "critical|important|nice_to_have", "reason": "..."}},
  ],
  "test_cases": [
    {{"id": "TC-001", "title": "...", "type": one of {TEST_TYPES}, "priority": "low|medium|high|critical",
      "description": "...", "preconditions": "... or None.", "steps": ["step 1", "step 2", "..."],
      "expected_result": "..."}},
  ],
  "implementation_plan": [
    {{"task": "...", "description": "...", "component": "...", "priority": "low|medium|high|critical",

```

```

    "estimated_effort": "<your estimate>", "dependencies": "optional, references to earlier tasks or null",
    "owner_suggestion": one of {OWNER_SUGGESTIONS}, or null if you can't make a well-founded suggestion}}
  ],
  "recommendation": {{ "decision": "approve|approve_with_conditions|requires_clarification", "reasoning": "..."}}
}}

```

Wherever this shape shows options separated by "|" (for example "low|medium|high|critical"), that is a \ list of your choices, not literal text - pick exactly ONE of those words as your answer. Never output \ the "|" character itself, and never output placeholder text like "none", "n/a", "not specified", or \ "tbd" for a field that requires picking one of a fixed set of options - always pick the closest real \ option instead.

Wherever this shape asks for "certainty", pick exactly one of: "known" (explicitly stated in the change \ request, or a direct, unambiguous implication of what's stated), "inferred" (a reasonable judgment call \ you made that isn't stated outright), or "unknown" (you genuinely cannot determine this from the change \ request as written - this is a legitimate answer, not a failure to answer). Wherever this shape also asks \ for "confidence" (a separate 0-100 number), that measures how sure you are in your own assessment itself, \ independent of the certainty label - for example a component you're not sure is even affected can still \ carry a stated confidence about that guess. Do not mark something "known" just because you're confident \ in it; a well-reasoned inference is still "inferred" no matter how confident you are in it.

"impact assessments" must contain exactly one entry for EACH of the {len(IMPACT_CATEGORIES)} categories listed \ above ({", ".join(IMPACT_CATEGORIES)}). - never fewer, never more than one per category. Each entry describes \ the change's general effect through that one lens specifically, independent of the risks list above: a risk is a \ specific thing that could go wrong, while an impact assessment describes what the change touches or affects \ through that lens even if nothing goes wrong (for example, "customer" impact describes how customers experience \ this change, not a risk of a customer-facing outage). If a given lens genuinely has little or no impact for this \ change, say so honestly (a "low" impact level with a description explaining why) rather than omitting that \ category or inflating its impact to seem more thorough.

"security findings" must contain exactly one entry for EACH of the {len(SEcurity_CATEGORIES)} categories listed \ above ({", ".join(SEcurity_CATEGORIES)}). - never fewer, never more than one per category. "finding" states what you \ observed for that category from the change request as written (or the honest absence of a concern - never invent \ a vulnerability that isn't grounded in the request). "evidence" is what specifically in the change request grounds \ this finding, the same role it plays for requirements/risks/components elsewhere in this shape. "recommendation" is \ a concrete, actionable suggestion, not a restatement of the finding. "status" must be exactly "open" (a real, \ unresolved concern exists for this category) or "not applicable" (this category genuinely doesn't apply to this \ change) - never any other value; a human reviewer, not you, is the one who later marks a finding "acknowledged" or \ "resolved".

Only list a dependency if it is a well-founded conclusion from the change request as written - never \ invent a dependency with no plausible connection to it just to fill out the list. For each dependency, \ "relationship" says how directly this change actually relies on it: "direct" (this change itself uses \ it), "indirect" (something this change touches relies on it, one step removed), or "potential" (plausible \ but not confidently established - use this rather than asserting certainty you don't have). "risk" is a \ separate low/medium/high/critical rating of how risky relying on this specific dependency is, independent \ of "impact level" (which rates how much *this change* would be affected if this dependency were wrong or \ unavailable). "evidence" on an affected component works the same way it does for requirements and risks - \ what in the change request specifically grounds this component being affected at all.

For "test cases": ground each one in the requirements, impact assessments, risks, and security findings \ you identified above (and any Relevant Documentation actually provided) - a real concern from this \ specific analysis, never generic boilerplate a change like this "usually" gets. Use whichever of the test \ types listed above are genuinely relevant to this change - unlike impact assessments/security findings \ above, this list has no fixed required size or one-per-category rule, so never pad it with irrelevant \ types just to appear thorough, and never omit a type (e.g. "security") that a risk or finding above \ clearly calls for. "preconditions" is any setup or starting state the steps below assume (login state, \ existing data, feature flags) - state "None." if nothing beyond a normal starting state is required, \ never invent a specific one. "steps" is an ordered list of short, concrete, literally-followable actions \ (never a single paragraph); leave it an empty list only if the change is too vague to describe any.

For "implementation plan": break the work into phase-oriented tasks (for example: requirement \ clarification, architecture/design, backend changes, database changes, frontend changes, security, \ testing, deployment, validation) only where a phase is actually relevant to this specific change - never \ emit a task for a phase this change doesn't genuinely touch (a change with no data-model impact gets no \ database task). "owner suggestion" is only ever a suggestion of what kind of person is best suited to a \ task, never an actual assignment (a human always makes the real assignment separately, through the \ project's own review process) - pick one of {OWNER_SUGGESTIONS} only when the task itself clearly calls \ for that kind of owner (for example a security-hardening task suggesting security reviewer, a task that \ is itself testing suggesting qa owner), and leave it null rather than guessing when nothing about the \ task points to a particular kind of owner.

Wherever this shape shows "<your estimate>", replace it with your own honest plain-language estimate - \ never output the literal text "<your estimate>". Effort estimates must be ranges in plain language \ (such as a number of days or weeks), never fake single-number precision, and never prefixed with the \ words "e.g." or "eg" - just state the range directly. If the change request is too vague to assess a \ section meaningfully, still return that section with an honest, minimal answer (an empty list, or \ "Insufficient information." as the reasoning/estimate) rather than omitting the key or inventing detail.

The "confidence" on "complexity" and on "effort" is a coarse low/medium/high rating of how sure you are \ in that specific level/estimate, not the same thing as the overall classification confidence above - a \ vague change request should usually get a lower complexity/effort confidence even if you're still willing \ to give your best-guess level and range. Never let a low confidence talk you into a fake-precise number \ instead of a range - state your honest range and rate your confidence in it separately.""

```

def build_user_prompt(change_request: ChangeRequest, *, knowledge_chunks: Optional[list] = None) -> str:
    """Everything the AI is allowed to know about this change - the
    change request's own stored fields, verbatim, plus (Module 16
    Project Knowledge Base & RAG) an optional "Relevant Documentation"
    section built from "knowledge_chunks" - whatever
    knowledge embeddings.search_chunks() retrieved for this change
    request, if anything. Nothing else, ever."""
    lines = [
        f"Title: {change_request.title}",
        f"Description: {change_request.description}",
        f"Business Objective: {change_request.business_objective or 'Not provided.'}",
        f"Priority: {change_request.priority.value}",
        f"Requested By: {change_request.requested_by or 'Not provided.'}",
        f"Target System: {change_request.target_system or 'Not provided.'}",
        f"Desired Deadline: {change_request.desired_deadline.isoformat() if change_request.desired_deadline else 'Not provided.'}",
    ]
    prompt = "Analyze this change request:\n\n" + "\n".join(lines)

    if knowledge_chunks:
        doc_lines = [
            "\n\nRelevant Documentation (retrieved from the team's own project knowledge base - use only "
            "where genuinely relevant; if it doesn't confirm something, that thing is still \"unknown\" or "
            "\"inferred\", never fabricated):"
        ]
        for chunk in knowledge_chunks:
            doc_lines.append(
                f"\nSource: {chunk['document title']}\n"
                f"Section: {chunk['section_label'] or 'N/A'}\n"
                f"{chunk['content']}"
            )
        prompt += "\n".join(doc_lines)

    return prompt

def extract_json(text: str) -> str:
    """Strip a ```json ... ``` (or bare ```...) fence if the model wrapped
    its response in one despite being told not to - cheap robustness that
    doesn't change what we accept as valid."""
    text = text.strip()
    if text.startswith("```"):
        text = re.sub(r"```([a-zA-Z]*)\s*\n?", "", text)
        text = re.sub(r"\n```\s*$", "", text)
    return text.strip()

```

```

_MISSING_INFO_PRIORITY = {
    "critical": Priority.CRITICAL,
    "important": Priority.HIGH,
    "nice_to_have": Priority.LOW,
}

_REQUIREMENT_CATEGORY_MAP = {
    "business_objective": (RequirementType.BUSINESS, "Business Objective"),
    "functional": (RequirementType.FUNCTIONAL, None),
    "non_functional": (RequirementType.NON_FUNCTIONAL, None),
    "constraint": (RequirementType.TECHNICAL, "Constraint"),
    "assumption": (RequirementType.TECHNICAL, "Assumption"),
    "acceptance_criteria": (RequirementType.FUNCTIONAL, "Acceptance Criteria"),
}

_RECOMMENDATION_MAP = {
    "approve": ApprovalRecommendation.APPROVE,
    "approve_with_conditions": ApprovalRecommendation.APPROVE_WITH_CONDITIONS,
    "requires_clarification": ApprovalRecommendation.REQUIRES_CLARIFICATION,
}

def overall_risk_score(result: AIAnalysisResult) -> float:
    """One 0-100 number to represent the change's overall risk (used by the
    dashboard and the change-requests list). Uses the single worst
    individual risk rather than an average, so one severe risk can't get
    diluted into an unremarkable-looking overall score."""
    if not result.risks:
        return 5.0
    return max(risk.score for risk in result.risks)

def _persist_result(
    db: Session,
    change_request: ChangeRequest,
    result: AIAnalysisResult,
    raw_text: str,
    *,
    knowledge_chunks: Optional[list] = None,
) -> Analysis:
    analysis = Analysis(
        change_request_id=change_request.id,
        summary=result.summary,
        category=result.classification.category,
        complexity=result.complexity.level,
        risk_score=overall_risk_score(result),
        confidence_score=result.classification.confidence * 100,
        recommendation=_RECOMMENDATION_MAP.get(result.recommendation.decision),
        classification_reason=result.classification.reason,
        complexity_reasoning=result.complexity.reasoning,
        complexity_confidence=result.complexity.confidence,
        security_analysis=json.dumps(result.security_analysis.model_dump()),
        effort_estimate=(
            f"Backend: {result.effort.backend}, Frontend: {result.effort.frontend}, "
            f"Testing: {result.effort.testing}, Total: {result.effort.total}"
        ),
        effort_confidence=result.effort.confidence,
        recommendation_reasoning=result.recommendation.reasoning,
        raw_ai_response=raw_text,
        # Module 12: which CR version this analysis covers - lets the UI
        # detect "the CR changed since this analysis ran" (see
        # app/services/workflow_rules.py::is_analysis_outdated).
        change_request_version=change_request.current_version or 1,
    )
    db.add(analysis)
    db.flush() # assigns analysis.id without committing yet

    for item in result.requirements:
        req_type, label = _REQUIREMENT_CATEGORY_MAP.get(item.category, (RequirementType.FUNCTIONAL, None))
        description = f"[{label}] {item.description}" if label else item.description
        db.add(
            Requirement(
                analysis_id=analysis.id,
                requirement_type=req_type,
                description=description,
                priority=item.priority,
                certainty=item.certainty,
                confidence=item.confidence,
                evidence=item.evidence or None,
            )
        )

    for item in result.affected_components:
        db.add(
            AffectedComponent(
                analysis_id=analysis.id,
                component_name=item.name,
                component_type=item.type,
                impact_level=item.impact_level,
                reason=item.reason,
                confidence=item.confidence,
                certainty=item.certainty,
                evidence=item.evidence or None,
            )
        )

    for item in result.dependencies:
        db.add(
            Dependency(
                analysis_id=analysis.id,
                dependency_name=item.name,
                dependency_type=item.type,
                impact_level=item.impact_level,
                reason=item.reason,
                relationship_type=item.relationship,
                risk_severity=item.risk,
            )
        )

    for item in result.risks:
        description = f"{item.description} {item.explanation}".strip() if item.explanation else item.description
        db.add(
            Risk(
                analysis_id=analysis.id,
                category=item.category,
                description=description,
                severity=item.severity,
                probability=item.probability,
                score=item.score,
                mitigation=item.mitigation,
                certainty=item.certainty,
                confidence=item.confidence,
                # `explanation` is the AI's stated reasoning for this risk -
                # already folded into `description` above for backward
                # compatibility with Module 6, and stored again here,
                # verbatim, as this risk's evidence.
                evidence=item.explanation or None,
            )
        )

```

```

for item in result.impact_assessments:
    db.add(
        ImpactAssessment(
            analysis_id=analysis.id,
            category=item.category,
            impact_level=item.impact_level,
            description=item.description,
            certainty=item.certainty,
            confidence=item.confidence,
        )
    )

for item in result.security_findings:
    db.add(
        SecurityFinding(
            analysis_id=analysis.id,
            category=item.category,
            finding=item.finding,
            severity=item.severity,
            evidence=item.evidence,
            recommendation=item.recommendation,
            status=item.status,
        )
    )

for item in result.missing_information:
    db.add(
        ClarificationQuestion(
            analysis_id=analysis.id,
            question=item.question,
            priority=MISSING_INFO_PRIORITY.get(item.priority, Priority.MEDIUM),
            reason=item.reason,
            resolved=False,
        )
    )

for item in result.test_cases:
    db.add(
        TestCase(
            analysis_id=analysis.id,
            test_id=item.id,
            title=item.title,
            test_type=item.type,
            priority=item.priority,
            description=item.description,
            expected_result=item.expected_result,
            # Module 17 Phase 2 - preconditions is free text (already
            # normalized to None for "no answer" - see TestCaseItem's
            # own validator); steps is JSON-encoded the same way
            # ChangeRequest.tags already is (see TestCase.steps's own
            # docstring), an empty list persisting as "[]", never null.
            preconditions=item.preconditions,
            steps=json.dumps(item.steps),
        )
    )

for item in result.implementation_plan:
    db.add(
        ImplementationTask(
            analysis_id=analysis.id,
            task=item.task,
            description=item.description,
            component=item.component,
            priority=item.priority,
            estimated_effort=item.estimated_effort,
            dependencies=item.dependencies,
            # Module 17 Phase 3 - already normalized to a real
            # AssignmentRole or None by ImplementationTaskItem's own
            # validator; never a fabricated value.
            owner_suggestion=item.owner_suggestion,
        )
    )

# Module 12: record the event on the Activity timeline as its own
# system actor ("AI Analyzer"), not attributed to whichever human
# clicked Analyze/Re-analyze - matches the spec's own timeline example
# ("AI Analyzer - Analysis completed - Risk: High"). Also: the very
# first successful analysis moves a CR out of Pending Analysis into
# Analyzed automatically (the natural next step in the lifecycle);
# re-analyzing a CR that's already further along (in review, changes
# requested, etc) does NOT force its status backwards.
history.record_event(
    db,
    change_request_id=change_request.id,
    action=HistoryAction.AI_ANALYSIS_COMPLETED,
    actor_label="AI Analyzer",
    new_value=f"Risk score: {round(analysis.risk_score)}/100, Complexity: {analysis.complexity.value}",
    version_number=change_request.current_version or 1,
)

if change_request.status == ChangeRequestStatus.PENDING_ANALYSIS:
    change_request.status = ChangeRequestStatus.ANALYZED

# Module 16 (Project Knowledge Base & RAG) - Phase 4/5: record exactly
# which knowledge-base chunks were retrieved and handed to the AI as
# context for this specific analysis run, so "Evidence Used" (the
# module's own spec section 8) can show real sources rather than the
# AI's own unverifiable say-so. Snapshots document_title/section_label/
# content/score at retrieval time - see
# app/models/knowledge_evidence.py's own docstring for why this isn't
# only a live foreign key to KnowledgeChunk.
now = datetime.utcnow()
for chunk in knowledge_chunks or []:
    db.add(
        KnowledgeEvidence(
            analysis_id=analysis.id,
            document_id=chunk["document_id"],
            chunk_id=chunk["chunk_id"],
            document_title=chunk["document_title"],
            section_label=chunk["section_label"],
            content_snippet=chunk["content"],
            similarity_score=chunk["score"],
            change_request_version=change_request.current_version or 1,
            retrieved_at=now,
        )
    )

db.commit()
db.refresh(analysis)
return analysis

```

```

def retrieve_knowledge_context(db: Session, change_request: ChangeRequest) -> list:
    """Module 16 (Project Knowledge Base & RAG) Phase 4: retrieves
    knowledge-base chunks plausibly relevant to this change request, to
    ground the AI's analysis in real project documentation rather than
    only what the change request itself states.

```

```

    "Extract relevant concepts" (the module's own spec section 4, step 1)
    is done the simplest reliable way per that same spec's own instruction
    - the change request's own title/description/business objective ARE
    the concepts, fed directly into the same semantic search built in

```

Phase 3, rather than a separate NLP extraction step.

Never lets a knowledge-base problem - nothing embedded yet, the configured AI provider doesn't support embeddings, a network hiccup - fail the analysis itself: retrieval is optional grounding, not a hard dependency of the core analysis feature. Returns [] in any of those cases, exactly as if the knowledge base were simply empty."""
query_parts = [change_request.title, change_request.description, change_request.business_objective]
query = " ".join(part for part in query_parts if part)
try:
 # min score: only genuinely relevant chunks become grounding
 # context - see search_chunks's own docstring for why this
 # differs from the manual search endpoint's default of 0.0.
 return search_chunks(db, query, top_k=5, min_score=0.2)
except KnowledgeEmbeddingError:
 return []
except Exception: # noqa: BLE001 - retrieval is optional grounding, never a reason to fail the analysis
 return []

def run_analysis(db: Session, change_request: ChangeRequest) -> Analysis:

"""Analyze one change request end-to-end: call the AI, validate its response, persist it, and return the new Analysis row. Raises AnalysisError (with an appropriate status code) on any failure - the caller decides how to surface that to the client.

Module 13 Phase 3 (AI-failure safety): nothing about a failed run touches the database - _persist_result() is the only place that writes an Analysis row, and every raise below happens before it's ever called. So if this change request already had an analysis, that analysis is still exactly as it was: still the latest one, still what every other endpoint (GET .../analysis, the report, the dashboard) reads. _prior_note() below just says so in the error message itself, rather than leaving a failed re-analyze click looking like it might have silently wiped something out."""

prior_analysis = (
 db.query(Analysis)
 .filter(Analysis.change_request_id == change_request.id)
 .order_by(Analysis.created_at.desc())
 .first()
)

def _prior_note() -> str:
 if prior_analysis is None:
 return ""
 version = prior_analysis.change_request_version or 1
 return f" Its existing analysis (from Version {version}) is untouched and still available."

provider = get_ai_provider()
if not provider.is_configured():
 raise AnalysisError(
 "The AI provider isn't configured yet - set ANTHROPIC_API_KEY in backend/.env and "
 f"restart the backend.{_prior_note()}",
 status_code=503,
)

settings = get_settings()
knowledge_chunks = _retrieve_knowledge_context(db, change_request)
user_prompt = _build_user_prompt(change_request, knowledge_chunks=knowledge_chunks)

try:
 raw_text = provider.complete(
 user_prompt,
 system=SYSTEM_PROMPT,
 max_tokens=4096,
 timeout=settings.ai_request_timeout_seconds,
)
except AnalysisError:
 raise
except Exception as exc: # noqa: BLE001 - deliberately broad: any SDK/network error becomes a clean AnalysisError
 exc_name = type(exc).__name__
 if "Timeout" in exc_name or "timeout" in str(exc).lower():
 raise AnalysisError(
 f"The AI provider took too long to respond (over {settings.ai_request_timeout_seconds}s). "
 f"Please try again.{_prior_note()}",
 status_code=504,
) from exc
 raise AnalysisError(f"The AI provider call failed: {exc}.{_prior_note()}", status_code=502) from exc

cleaned = _extract_json(raw_text)

try:
 parsed = json.loads(cleaned)
except json.JSONDecodeError as exc:
 raise AnalysisError(
 f"The AI response wasn't valid JSON ({exc}). This usually clears up on retry.{_prior_note()}",
 status_code=422,
) from exc

try:
 result = AIAnalysisResult.model_validate(parsed)
except ValidationError as exc:
 raise AnalysisError(
 f"The AI response didn't match the expected analysis format: {exc}.{_prior_note()}",
 status_code=422,
) from exc

return _persist_result(db, change_request, result, raw_text, knowledge_chunks=knowledge_chunks)

```

"""Module 19 (Engineering Change Analytics): shared filtering
infrastructure every analytics metric function builds on, plus (as each
phase adds them) the metric functions themselves. Real database queries
only - no invented numbers, no placeholder/demo data (the spec's own "DO
NOT generate fake statistics" instruction, and this project's own standing
"every metric should be traceable to real database records" rule).

Filters (spec section 8: date/status/priority/risk/category/owner) are
pushed down to SQL wherever there's a real column to filter on
(created at/status/priority/created by); risk and category have no direct
column of their own - risk comes from the latest Analysis's risk score,
category from its own free-text field normalized against a fixed
vocabulary - so those two are applied in Python over the already
SQL-filtered result. This mirrors the exact pattern app/api/dashboard.py
already established at this same ~3000-row scale (its own docstring: "no
AI calls ... this only reads what's already in SQLite"), not a new one
invented for this module.

CANONICAL_CATEGORIES/ normalize category are intentionally duplicated from
dashboard.py rather than imported - a services/ module importing from an
api/ module would invert this project's usual dependency direction, and
duplicating one small, stable helper matches the same "each module owns
its own small helpers" convention app/api/my_work.py already established
for _latest_analysis/_effective_status/_risk_for.
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Iterable, Optional

from sqlalchemy.orm import Session, selectinload

from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.enums import (
    ApprovalRecommendation,
    ApprovalStatus,
    ApprovalType,
    AssignmentRole,
    ChangeRequestStatus,
    HistoryAction,
    Priority,
)
from app.models.user import User
from app.services.approvals import approval_due_status
from app.services.workflow_rules import APPROVAL_TYPE_LABELS, FIELD_LABELS, STATUS_LABELS, risk_bucket

# Module 19: the app's own bulk-synthetic-data seed script
# (app/database/seed_bulk.py) creates every load-test row under one of 5
# accounts named exactly this way. Excluded from workload/ownership
# breakdowns only (per this module's own planning discussion with the
# user) - never from overall totals, which still count every real row in
# the database, synthetic or not.
BULK_SEED_EMAIL_PATTERN = "loadtest%alight.com"

# Mirrors app/api/dashboard.py's own CANONICAL_CATEGORIES/ CATEGORY_ALIASES
# exactly - see this module's own docstring for why it's duplicated here
# rather than imported.
CANONICAL_CATEGORIES = [
    "Feature",
    "Bug Fix",
    "Security",
    "Database",
    "API",
    "Infrastructure",
    "Integration",
]

CATEGORY_ALIASES = {
    "feature": "Feature",
    "new feature": "Feature",
    "enhancement": "Feature",
    "bug fix": "Bug Fix",
    "bugfix": "Bug Fix",
    "bug": "Bug Fix",
    "fix": "Bug Fix",
    "security": "Security",
    "database": "Database",
    "db": "Database",
    "api": "API",
    "infrastructure": "Infrastructure",
    "infra": "Infrastructure",
    "integration": "Integration",
}

def normalize_category(raw: Optional[str]) -> str:
    if not raw:
        return "Other"
    return _CATEGORY_ALIASES.get(raw.strip().lower(), "Other")

@dataclass
class AnalyticsFilters:
    """One shared filter set (spec section 8) applied consistently across
    every analytics endpoint - a person filtering to "High priority,
    Security category" should see that same slice everywhere on the
    dashboard, not a different interpretation per section."""

    date_from: Optional[datetime] = None
    date_to: Optional[datetime] = None
    status: Optional[ChangeRequestStatus] = None
    priority: Optional[Priority] = None
    risk: Optional[str] = None # "low" | "medium" | "high" | "critical"
    category: Optional[str] = None # one of CANONICAL_CATEGORIES, or "Other"
    owner_id: Optional[int] = None

def latest_analysis(change_request: ChangeRequest) -> Optional[Analysis]:
    if not change_request.analyses:
        return None
    return max(change_request.analyses, key=lambda a: a.created_at)

def first_analysis(change_request: ChangeRequest) -> Optional[Analysis]:
    if not change_request.analyses:
        return None
    return min(change_request.analyses, key=lambda a: a.created_at)

def risk_for(change_request: ChangeRequest) -> Optional[str]:

```

```

latest = latest_analysis(change_request)
return risk_bucket(latest.risk_score) if latest else None

def category_for(change_request: ChangeRequest) -> Optional[str]:
    latest = latest_analysis(change_request)
    return _normalize_category(latest.category) if latest else None

def filtered_change_requests(
    db: Session, filters: AnalyticsFilters, *, exclude_bulk_seed_owners: bool = False
) -> list[ChangeRequest]:
    """Every change request matching `filters`, database-filtered on every
    column that actually has one (date range, status, priority, owner);
    risk and category are applied afterward in Python since neither is a
    direct column (see this module's own docstring). Always eager-loads
    `analyses` - nearly every analytics metric needs the latest one for
    risk/category/confidence, so loading it once here avoids an N+1 in
    every caller.

    `exclude_bulk_seed_owners` is opt-in, used only by the workload/
    ownership metrics - every other section counts the app's own bulk
    synthetic data like any other real row, per this module's own
    "overall totals still include everyone" design note.

    Also eager-loads `approvals`, `history`, `assignments` and each
    analysis's `clarification questions` alongside `analyses` - between
    the executive/workflow, risk, bottleneck, change-analytics and workload
    metrics this module ends up computing, nearly every one of them needs
    at least one of these, so loading them all once here (instead of each
    phase adding its own selectinload) keeps this the one place that
    decides how a change request is fetched for analytics."""
    query = db.query(ChangeRequest).options(
        selectinload(ChangeRequest.analyses).selectinload(Analysis.clarification_questions),
        selectinload(ChangeRequest.approvals).selectinload(Approval.approver),
        selectinload(ChangeRequest.history),
        selectinload(ChangeRequest.assignments).selectinload(ChangeRequestAssignment.user),
        selectinload(ChangeRequest.creator),
    )

    if filters.date_from is not None:
        query = query.filter(ChangeRequest.created_at >= filters.date_from)
    if filters.date_to is not None:
        query = query.filter(ChangeRequest.created_at <= filters.date_to)
    if filters.status is not None:
        query = query.filter(ChangeRequest.status == filters.status)
    if filters.priority is not None:
        query = query.filter(ChangeRequest.priority == filters.priority)
    if filters.owner_id is not None:
        query = query.filter(ChangeRequest.created_by == filters.owner_id)
    if exclude_bulk_seed_owners:
        query = query.join(User, ChangeRequest.created_by == User.id).filter(
            ~User.email.like(BULK_SEED_EMAIL_PATTERN)
        )

    rows = query.order_by(ChangeRequest.created_at.desc()).all()

    if filters.risk is not None:
        rows = [cr for cr in rows if risk_for(cr) == filters.risk]
    if filters.category is not None:
        rows = [cr for cr in rows if category_for(cr) == filters.category]

    return rows

# --- Module 19 Phase 2: Executive Metrics + Workflow Metrics (spec 1/2) ---

def average_hours(durations: Iterable[Optional[timedelta]]) -> Optional[float]:
    """Average of a list of timedeltas, in hours, ignoring every None entry
    (a None means "hasn't reached that milestone yet", never coerced to a
    0-hour duration - doing that would silently pull the average toward
    zero and understate it). Returns None, not 0, when there is nothing to
    average at all, so a caller can render "No data yet" instead of a
    misleading "0.0 hours"."""
    values = [d.total_seconds() / 3600 for d in durations if d is not None]
    if not values:
        return None
    return sum(values) / len(values)

def approval_duration(approval) -> Optional[timedelta]:
    """How long a single approval sat PENDING before it was answered - None
    for one that's still pending (there's no end time yet to measure)."""
    if approval.responded_at is None:
        return None
    return approval.responded_at - approval.requested_at

def time_to_status(change_request: ChangeRequest, target_status: ChangeRequestStatus) -> Optional[timedelta]:
    """How long after creation `change_request` first reached
    `target_status`, per the real STATUS_CHANGED history trail (whose
    new value stores the human-readable STATUS LABELS text, not the raw
    enum - see app/services/history.py's own callers) - None if it never
    has. Uses the EARLIEST matching event, so a CR that later moved away
    from that status and came back (e.g. In Progress -> Cancelled is not
    reachable, but Approval Required -> Changes Requested -> Approval
    Required again is) is still timed from the first time it got there,
    not the most recent."""
    label = STATUS_LABELS.get(target_status, target_status.value)
    matching = [
        event.created_at
        for event in change_request.history
        if event.action == HistoryAction.STATUS_CHANGED and event.new_value == label
    ]
    if not matching:
        return None
    return min(matching) - change_request.created_at

# The 6 named Executive-Metrics buckets (spec section 1) are mutually
# exclusive by construction - every ChangeRequestStatus member appears in
# at most one set below. CANCELLED is deliberately not claimed by any of
# them: a cancelled change request is genuinely not "still open", not
# "approved", not "rejected", not "in progress" and not "closed" - forcing
# it into one of those would misrepresent it. It's still counted in
# `total` (this dashboard's own "use actual database data" rule applies to
# the total exactly like everything else), just not double-counted into a
# bucket that doesn't actually describe it.
_OPEN_STATUSES: frozenset[ChangeRequestStatus] = frozenset(
    {
        ChangeRequestStatus.DRAFT,
        ChangeRequestStatus.SUBMITTED,
        ChangeRequestStatus.PENDING_ANALYSIS,
        ChangeRequestStatus.ANALYZED,
        ChangeRequestStatus.IN_REVIEW,
        ChangeRequestStatus.CHANGES_REQUESTED,
    }
)
_IN_PROGRESS_STATUSES: frozenset[ChangeRequestStatus] = frozenset(
    {

```



```

        ChangeRequestStatus.IMPLEMENTATION_PLANNED,
        ChangeRequestStatus.IN_PROGRESS,
        ChangeRequestStatus.IMPLEMENTED,
        ChangeRequestStatus.VALIDATED,
    }
)

@dataclass
class ExecutiveMetrics:
    """Spec section 1. `open` counts every pre-approval-pipeline status
    (Draft through Changes Requested); `in progress` counts every
    post-approval, pre-closure status (Implementation Planned through
    Validated) - see the OPEN_STATUSES/IN_PROGRESS_STATUSES comment
    above for why Cancelled sits outside every named bucket."""

    total: int
    open: int
    pending_approval: int
    approved: int
    rejected: int
    in_progress: int
    closed: int

def executive_metrics(rows: list[ChangeRequest]) -> ExecutiveMetrics:
    return ExecutiveMetrics(
        total=len(rows),
        open=sum(1 for cr in rows if cr.status in OPEN_STATUSES),
        pending_approval=sum(1 for cr in rows if cr.status == ChangeRequestStatus.APPROVAL_REQUIRED),
        approved=sum(1 for cr in rows if cr.status == ChangeRequestStatus.APPROVED),
        rejected=sum(1 for cr in rows if cr.status == ChangeRequestStatus.REJECTED),
        in_progress=sum(1 for cr in rows if cr.status in IN_PROGRESS_STATUSES),
        closed=sum(1 for cr in rows if cr.status == ChangeRequestStatus.CLOSED),
    )

@dataclass
class WorkflowMetrics:
    """Spec section 2. Every avg * hours field is None (never 0) when
    nothing in the filtered set has reached that milestone yet - see
    average_hours()'s own docstring for why. `crs waiting for approval`
    counts change requests with at least one currently-PENDING Approval
    row (regardless of the CR's own `status` column) - a broader, real-time
    view of "who's actually waiting on a decision right now" than
    ExecutiveMetrics.pending_approval, which only reflects the CR's status
    column having been explicitly moved to Approval Required.
    `crs with requested changes` reflects the CR's *current* status - a CR
    already moved on from Changes Requested (e.g. back to Under Review) no
    longer counts here, same as ExecutiveMetrics' own status-based buckets."""

    avg_time_to_analysis_hours: Optional[float]
    avg_approval_time_hours: Optional[float]
    avg_time_to_implementation_hours: Optional[float]
    avg_time_to_closure_hours: Optional[float]
    crs_waiting_for_approval: int
    crs_with_requested_changes: int

def workflow_metrics(rows: list[ChangeRequest]) -> WorkflowMetrics:
    analysis_durations = []
    for cr in rows:
        fa = first_analysis(cr)
        if fa is not None:
            analysis_durations.append(fa.created_at - cr.created_at)

    approval_durations = [approval.duration(a) for cr in rows for a in cr.approvals]
    implementation_durations = [time_to_status(cr, ChangeRequestStatus.IN_PROGRESS) for cr in rows]
    closure_durations = [time_to_status(cr, ChangeRequestStatus.CLOSED) for cr in rows]

    return WorkflowMetrics(
        avg_time_to_analysis_hours=average_hours(analysis_durations),
        avg_approval_time_hours=average_hours(approval_durations),
        avg_time_to_implementation_hours=average_hours(implementation_durations),
        avg_time_to_closure_hours=average_hours(closure_durations),
        crs_waiting_for_approval=sum(
            1 for cr in rows if any(a.status == ApprovalStatus.PENDING for a in cr.approvals)
        ),
        crs_with_requested_changes=sum(
            1 for cr in rows if cr.status == ChangeRequestStatus.CHANGES_REQUESTED
        ),
    )

# --- Module 19 Phase 3: Risk Analytics + Approval Bottlenecks (spec 3/4) ---
RISK_BUCKET_LABELS: tuple[str, ...] = ("low", "medium", "high", "critical")

@dataclass
class RiskAnalytics:
    """Spec section 3. `current distribution` mirrors the existing
    dashboard's own risk distribution shape (low/medium/high/critical/
    not_analyzed) but computed over this module's own filtered set, not the
    whole database. `over time` is one row per calendar month (oldest
    first), each counting - among the CRs in the filtered set whose latest
    analysis happened that month - how many landed in each risk bucket.
    This is deliberately "risk profile of change requests analyzed each
    month", not a per-CR history of every risk score it ever had (this app
    only stores each analysis's own risk score, not a separate time series)
    - the most honest trend this data actually supports."""

    current_distribution: dict[str, int]
    over_time: list[dict]

def risk_analytics(rows: list[ChangeRequest]) -> RiskAnalytics:
    current_distribution = {bucket: 0 for bucket in RISK_BUCKET_LABELS}
    current_distribution["not_analyzed"] = 0

    # month ("YYYY-MM") -> {bucket: count}
    by_month: dict[str, dict[str, int]] = {}

    for cr in rows:
        latest = latest_analysis(cr)
        if latest is None:
            current_distribution["not_analyzed"] += 1
            continue
        bucket = risk_bucket(latest.risk_score)
        current_distribution[bucket] += 1

        month_key = latest.created_at.strftime("%Y-%m")
        month_counts = by_month.setdefault(month_key, {b: 0 for b in RISK_BUCKET_LABELS})
        month_counts[bucket] += 1

    over_time = [
        {"period": month, **counts} for month, counts in sorted(by_month.items(), key=lambda item: item[0])
    ]

    return RiskAnalytics(current_distribution=current_distribution, over_time=over_time)

```



```

# Module 19's own planning discussion with the user: this app has no real
# "team" concept anywhere - ApprovalType (Technical/Security/Product/
# Engineering Manager/Director/QA/DBA/Release/General) is the closest
# existing stand-in for "team", used here exactly like workflow_rules.py's
# own approval matrix already treats it. Person-level breakdown is
# deliberately limited to a name and a count - no email, no other account
# details - per the spec's own "do not expose unnecessary sensitive
# information" instruction.
_BOTTLENECK_LABEL_LIMIT = 10 # top N people, so one prolific approver's
# history doesn't turn this into a full account directory

@dataclass
class ApprovalBottlenecks:
    """Spec section 4. `pending by type`/`pending by person` count only
    currently-PENDING approvals (the actual queue right now);
    `avg duration hours by type` and `most common blockers` look at every
    RESOLVED approval instead (there's nothing to measure duration or an
    outcome for on one that's still pending)."""

    pending_by_type: dict[str, int]
    pending_by_person: list[dict]
    avg_duration_hours_by_type: dict[str, Optional[float]]
    most_common_blockers: list[dict]

def approval_bottlenecks(rows: list[ChangeRequest]) -> ApprovalBottlenecks:
    all_approvals: list[Approval] = [a for cr in rows for a in cr.approvals]
    pending = [a for a in all_approvals if a.status == ApprovalStatus.PENDING]
    resolved = [a for a in all_approvals if a.status != ApprovalStatus.PENDING]

    pending_by_type: dict[str, int] = {}
    for approval in pending:
        label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        pending_by_type[label] = pending_by_type.get(label, 0) + 1

    pending_by_person_counts: dict[str, int] = {}
    for approval in pending:
        name = approval.approver.name
        pending_by_person_counts[name] = pending_by_person_counts.get(name, 0) + 1
    pending_by_person = [
        {"approver_name": name, "count": count}
        for name, count in sorted(pending_by_person_counts.items(), key=lambda item: item[1], reverse=True)
    ][:_BOTTLENECK_LABEL_LIMIT]

    durations_by_type: dict[str, list[float]] = {}
    for approval in resolved:
        duration = approval_duration(approval)
        if duration is None:
            continue
        label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        durations_by_type.setdefault(label, []).append(duration.total_seconds() / 3600)
    avg_duration_hours_by_type = {
        label: (sum(values) / len(values)) for label, values in durations_by_type.items()
    }

    blocker_counts: dict[str, int] = {}
    for approval in resolved:
        if approval.status not in (ApprovalStatus.REJECTED, ApprovalStatus.CHANGES_REQUESTED):
            continue
        label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        blocker_counts[label] = blocker_counts.get(label, 0) + 1
    most_common_blockers = [
        {"approval_type_label": label, "count": count}
        for label, count in sorted(blocker_counts.items(), key=lambda item: item[1], reverse=True)
    ]

    return ApprovalBottlenecks(
        pending_by_type=pending_by_type,
        pending_by_person=pending_by_person,
        avg_duration_hours_by_type=avg_duration_hours_by_type,
        most_common_blockers=most_common_blockers,
    )

# --- Module 19 Phase 4: Change Analytics + Workload (spec 5/6) -----

MOST_CHANGED_FIELDS_LIMIT = 10

@dataclass
class ChangeAnalytics:
    """Spec section 5. `crs with multiple revisions`/`avg versions per cr`
    read ChangeRequest.current version directly (the same "read this
    column, don't recount ChangeRequestVersion rows" shortcut the rest of
    the app already takes - current version IS the version count, they're
    kept in lockstep by every edit). `crs returned for clarification` counts
    a CR once if its LATEST analysis has any still-unresolved
    ClarificationQuestion - the same is-it-resolved check
    app/api/my work.py: effective status already uses for its own
    "requires clarification" bucket, applied here as a count instead of a
    per-CR status label."""

    crs_with_multiple_revisions: int
    avg_versions_per_cr: Optional[float]
    most_changed_fields: list[dict]
    crs_returned_for_clarification: int

def change_analytics(rows: list[ChangeRequest]) -> ChangeAnalytics:
    versions = [cr.current_version or 1 for cr in rows]
    crs_with_multiple_revisions = sum(1 for v in versions if v > 1)
    avg_versions_per_cr = (sum(versions) / len(versions)) if versions else None

    field_counts: dict[str, int] = {}
    for cr in rows:
        for event in cr.history:
            if event.action != HistoryAction.FIELD_CHANGED or not event.field_name:
                continue
            label = FIELD_LABELS.get(event.field_name, event.field_name)
            field_counts[label] = field_counts.get(label, 0) + 1
    most_changed_fields = [
        {"field_label": label, "count": count}
        for label, count in sorted(field_counts.items(), key=lambda item: item[1], reverse=True)
    ][:_MOST_CHANGED_FIELDS_LIMIT]

    crs_returned_for_clarification = 0
    for cr in rows:
        latest = latest_analysis(cr)
        if latest is not None and any(not q.resolved for q in latest.clarification_questions):
            crs_returned_for_clarification += 1

    return ChangeAnalytics(
        crs_with_multiple_revisions=crs_with_multiple_revisions,
        avg_versions_per_cr=avg_versions_per_cr,
        most_changed_fields=most_changed_fields,
        crs_returned_for_clarification=crs_returned_for_clarification,
    )

```

```

# Which assignment roles count as "reviewing" for workload purposes -
# mirrors app/api/my_work.py::REVIEW_ROLES exactly (Reviewer/Technical
# Lead/Security Reviewer/Owner) rather than inventing a second definition
# of "who reviews" for this module.
_REVIEW_ROLES = frozenset([AssignmentRole] = frozenset(
    {
        AssignmentRole.REVIEWER,
        AssignmentRole.TECHNICAL_LEAD,
        AssignmentRole.SECURITY_REVIEWER,
        AssignmentRole.OWNER,
    }
)

@dataclass
class WorkloadAnalytics:
    """Spec section 6 - crs per owner` is the one per-person breakdown the
    spec actually asks for here ("CRs per owner"); `pending_reviews` /
    `pending_approvals` / `overdue_tasks` are plain totals across the
    filtered set - the spec's own wording for those three has no "per
    owner/person" qualifier, unlike Approval Bottlenecks' own
    pending by person (Phase 3), which does. Callers should pass
    `exclude_bulk_seed_owners=True` to filtered_change_requests() before
    computing this - see that function's own docstring for why."""
    crs_per_owner: list[dict]
    pending_reviews: int
    pending_approvals: int
    overdue_tasks: int

def workload_analytics(rows: list[ChangeRequest]) -> WorkloadAnalytics:
    owner_counts: dict[str, int] = {}
    for cr in rows:
        owner_counts[cr.creator.name] = owner_counts.get(cr.creator.name, 0) + 1
    crs_per_owner = [
        {"owner_name": name, "count": count}
        for name, count in sorted(owner_counts.items(), key=lambda item: item[1], reverse=True)
    ]

    pending_reviews = sum(
        1 for cr in rows for a in cr.assignments if a.role in _REVIEW_ROLES
    )
    all_approvals = [a for cr in rows for a in cr.approvals]
    pending_approvals = sum(1 for a in all_approvals if a.status == ApprovalStatus.PENDING)
    overdue_tasks = sum(
        1
        for a in all_approvals
        if a.status == ApprovalStatus.PENDING and approval_due_status(a) == "overdue"
    )

    return WorkloadAnalytics(
        crs_per_owner=crs_per_owner,
        pending_reviews=pending_reviews,
        pending_approvals=pending_approvals,
        overdue_tasks=overdue_tasks,
    )

# --- Module 19 Phase 5: AI Analytics (spec 7) -----

_AI_RECOMMENDED_APPROVAL_DECISIONS = frozenset([ApprovalRecommendation] = frozenset(
    {ApprovalRecommendation.APPROVE, ApprovalRecommendation.APPROVE_WITH_CONDITIONS}
)

@dataclass
class AIAalyticsMetrics:
    """Spec section 7 - deliberately called "AI recommendation statistics"
    everywhere in this module's API/docs, never "AI accuracy": this app has
    no ground-truth record of whether an AI recommendation was actually
    *right* in hindsight (nothing here compares an AI recommendation to a
    real-world outcome), so nothing computed below claims or implies an
    accuracy rate - only counts of what the AI actually said and did.

    Every field is computed over every Analysis row belonging to a change
    request in the filtered set (not just each CR's latest one) - "Number
    of analyses" and "Average confidence" are both about analysis RUNS, not
    change requests, so they use the same granularity throughout.
    `re_analysis_rate` is the one exception (necessarily per-CR: "did this
    CR get re-analyzed" isn't a per-analysis question) and is None (not 0)
    when nothing in the filtered set has been analyzed at all - nothing to
    compute a rate over yet."""
    total_analyses: int
    re_analysis_rate: Optional[float]
    average_confidence: Optional[float]
    risk_changes_after_edits: int
    ai_recommended_approvals: int
    ai_analysis_failures: int

def ai_analytics(rows: list[ChangeRequest]) -> AIAalyticsMetrics:
    all_analyses: list[Analysis] = [a for cr in rows for a in cr.analyses]
    total_analyses = len(all_analyses)

    analyzed_crs = [cr for cr in rows if cr.analyses]
    re_analysis_rate = (
        sum(1 for cr in analyzed_crs if len(cr.analyses) > 1) / len(analyzed_crs)
        if analyzed_crs
        else None
    )

    average_confidence = (sum(a.confidence_score for a in all_analyses) / total_analyses) if total_analyses else None

    # How many times, across every CR's own analysis history in order, did
    # a re-analysis land in a DIFFERENT risk bucket than the one right
    # before it - not just "went up" (see RISK_ESCALATED's own, stricter,
    # notification-worthy definition in workflow_rules.py) but any change
    # at all, since this is a descriptive count, not an alert.
    risk_changes_after_edits = 0
    for cr in rows:
        ordered = sorted(cr.analyses, key=lambda a: a.created_at)
        for previous, current in zip(ordered, ordered[1:]):
            if risk_bucket(previous.risk_score) != risk_bucket(current.risk_score):
                risk_changes_after_edits += 1

    ai_recommended_approvals = sum(
        1 for a in all_analyses if a.recommendation in _AI_RECOMMENDED_APPROVAL_DECISIONS
    )

    ai_analysis_failures = sum(
        1 for cr in rows for event in cr.history if event.action == HistoryAction.AI_ANALYSIS_FAILED
    )

    return AIAalyticsMetrics(
        total_analyses=total_analyses,
        re_analysis_rate=re_analysis_rate,
        average_confidence=average_confidence,
        risk_changes_after_edits=risk_changes_after_edits,
        ai_recommended_approvals=ai_recommended_approvals,

```

```
    ai_analysis_failures=ai_analysis_failures,  
)
```

backend/app/services/approvals.py

202 lines

```
"""Module 12 Phase 4 (Enterprise Workflow - Approvals): where an AI-derived
recommendation (app/services/workflow_rules.py::required_approval_types)
turns into a real Approval row against a real, tagged person.

"AI recommends, humans decide" (spec section 14/29): nothing in this file
is ever called by the AI Analysis Engine itself - a human always chooses
the approval type and the approver, via the two entry points below. Every
state change here is written through app/services/history.py so it lands
on the same append-only audit trail as everything else in Module 12.
"""
from __future__ import annotations

from datetime import datetime, timedelta
from typing import Optional

from sqlalchemy.orm import Session

from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.enums import ApprovalStatus, ApprovalType, HistoryAction
from app.models.user import User
from app.services import history
from app.services.workflow_rules import (
    APPROVAL_STATUS_LABELS,
    APPROVAL_TYPE_LABELS,
    RECOMMENDATION_LABELS,
    overrides_ai_recommendation,
)

def is_approval_outdated(change_request: ChangeRequest, approval: Approval) -> bool:
    """True only for a still-PENDING approval requested against an older CR
    version than the one it's now sitting on - mirrors
    workflow_rules.is_analysis_outdated. A decision an approver already made
    is a historical fact, not something later edits can retroactively make
    "outdated", so this is always False once the approval is resolved."""
    if approval.status != ApprovalStatus.PENDING:
        return False
    cr_version = change_request.current_version or 1
    return approval.cr_version != cr_version

# Module 18 Phase 1: how close to (or past) its own due date counts as
# "due soon" - a simple, adjustable window (mirrors Module 16's own
# min_score=0.2 cutoff in spirit: a deliberately plain heuristic, not a
# tuned value). Approaching-vs-overdue is a strict boundary at "now";
# "due soon" is anything inside this window that hasn't crossed it yet.
DUE_SOON_WINDOW = timedelta(days=2)

def approval_due_status(approval: Approval, *, now: Optional[datetime] = None) -> Optional[str]:
    """overdue" / "due soon" / None (not due soon, or no due date at all) -
    computed fresh at read time from approval.due_date, never stored, the
    same "never a boolean that can silently go stale" rule
    is approval_outdated/is_analysis_outdated already follow elsewhere in
    this app. Only ever meaningful for a still-PENDING approval - once an
    approval has actually been decided, its due date no longer means
    anything (mirrors is_approval_outdated's own "only PENDING" rule)."""
    if approval.status != ApprovalStatus.PENDING or approval.due_date is None:
        return None
    now = now or datetime.utcnow()
    if approval.due_date < now:
        return "overdue"
    if approval.due_date <= now + DUE_SOON_WINDOW:
        return "due soon"
    return None

def request_approval(
    db: Session,
    change_request: ChangeRequest,
    *,
    approval_type: ApprovalType,
    approver: User,
    requested_by: User,
    comment: Optional[str] = None,
    due_date: Optional[datetime] = None,
) -> Approval:
    """Creates the Approval row and records APPROVAL_REQUESTED. Duplicate-
    pending-request checking (same type + approver already PENDING) is the
    API layer's job, same as create_assignment's duplicate check - this
    function just does the write once callers have already decided it's OK.
    due_date (Module 18 Phase 1) is entirely optional - most approvals
    still have none, same as before this module existed."""
    approval = Approval(
        change_request_id=change_request.id,
        approver_id=approver.id,
        approval_type=approval_type,
        status=ApprovalStatus.PENDING,
        requested_by=requested_by.id,
        cr_version=change_request.current_version or 1,
        due_date=due_date,
    )
    db.add(approval)
    db.flush()

    type_label = APPROVAL_TYPE_LABELS.get(approval_type, approval_type.value)
    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.APPROVAL_REQUESTED,
        user_id=requested_by.id,
        new_value=f"{type_label} approval requested from {approver.name}",
        reason=comment,
        version_number=change_request.current_version or 1,
    )
    return approval

def respond_to_approval(
    db: Session,
    change_request: ChangeRequest,
    approval: Approval,
    *,
    new_status: ApprovalStatus,
    comment: Optional[str],
    responder: User,
) -> Approval:
    """Moves a PENDING approval to Approved/Rejected/Changes Requested.
    Callers (the API layer) are responsible for checking that `responder`
    is the tagged approver and that `approval.status` is still PENDING -
    this function assumes both are already true."""
    type_label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
```

```

old_label = APPROVAL_STATUS_LABELS.get(approval.status, approval.status.value)
new_label = APPROVAL_STATUS_LABELS.get(new_status, new_status.value)

approval.status = new_status
approval.responded_at = datetime.utcnow()
approval.comment = comment

action = {
    ApprovalStatus.APPROVED: HistoryAction.APPROVED,
    ApprovalStatus.REJECTED: HistoryAction.REJECTED,
    ApprovalStatus.CHANGES_REQUESTED: HistoryAction.CHANGES_REQUESTED,
}[new_status]

history.record_event(
    db,
    change_request_id=change_request.id,
    action=action,
    user_id=responder.id,
    old_value=f"{type_label} - {old_label}",
    new_value=f"{type_label} - {new_label}",
    reason=comment,
    version_number=change_request.current_version or 1,
)

# Module 13 Phase 4 (human-override recording): compare this decision
# against what the AI itself recommended on the analysis this approval
# was actually requested against (approval.cr version - not
# necessarily the CR's *current* analysis, which may have moved on
# since). Recorded as its own additional event, never in place of the
# one above - "AI recommends, humans decide" means disagreeing is
# never wrong, it should just stay visible on the record.
relevant_analysis = (
    db.query(Analysis)
    .filter(
        Analysis.change_request_id == change_request.id,
        Analysis.change_request_version == approval.cr_version,
    )
    .order_by(Analysis.created_at.desc())
    .first()
)

if relevant_analysis is not None and overrides_ai_recommendation(relevant_analysis.recommendation, new_status):
    rec_label = RECOMMENDATION_LABELS.get(
        relevant_analysis.recommendation,
        relevant_analysis.recommendation.value if relevant_analysis.recommendation else "No recommendation",
    )
    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.AI_RECOMMENDATION_OVERRIDDEN,
        user_id=responder.id,
        old_value=f"AI recommended: {rec_label}",
        new_value=f"{responder.name} marked {type_label} as {new_label}",
        reason=comment,
        version_number=change_request.current_version or 1,
    )

return approval

def cancel_approval(db: Session, change_request: ChangeRequest, approval: Approval, *, cancelled_by: User) -> Approval:
    """Withdraws a still-PENDING approval request (e.g. tagged the wrong
    person). Never used on an already-resolved approval - that's the API
    layer's 409 to raise, not this function's job."""
    type_label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)

    approval.status = ApprovalStatus.CANCELLED
    approval.responded_at = datetime.utcnow()

    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.APPROVAL_CANCELLED,
        user_id=cancelled_by.id,
        old_value=f"{type_label} - Pending",
        version_number=change_request.current_version or 1,
    )

    return approval

```

```

"""Module 15 Phase 2 (Repository Intelligence - Code Understanding).

For each indexed file, extracts a lightweight structural summary: imports,
function names, class names, API routes, and database/config references.
Deliberately not a full static analyzer ("keep implementation lightweight"
per the module's own spec):

* Python gets real, exact answers via the stdlib `ast` module - no extra
  dependency, and no risk of the regex-style false positives/negatives
  every other language here is subject to.
* JavaScript/TypeScript/JSX/TSX get a best-effort regex pass - good
  enough to find the common patterns this project's own frontend
  actually uses (import/require, `apiRequest(...)`/`fetch(...)`, calls,
  `import.meta.env.*`), but not a real parser.
* JSON/YAML/TOML/INI config files get their top-level keys read as
  config references (JSON via the stdlib parser; the rest via a simple
  line pattern, not a real parser).
* SQL files get table names read off FROM/INTO/UPDATE/TABLE keywords.
* Anything else (markdown, CSS, HTML, shell scripts, ...) yields empty
  lists - not every file has to have findings, and inventing some would
  be worse than none.

A file that can't be understood this way (a syntax error, a JSON parse
failure, ...) never fails the scan over it - understand_file() always
returns a result, falling back to empty lists, the same way an unreadable
file is just skipped rather than aborting the whole walk
(repository_scanner.py).
"""

from __future__ import annotations

import ast
import json
import re
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class CodeUnderstanding:
    imports: list[str] = field(default_factory=list)
    functions: list[str] = field(default_factory=list)
    classes: list[str] = field(default_factory=list)
    api_routes: list[str] = field(default_factory=list)
    database_references: list[str] = field(default_factory=list)
    config_references: list[str] = field(default_factory=list)

def dedupe(values: list[str]) -> list[str]:
    seen: dict[str, None] = {}
    for value in values:
        seen.setdefault(value, None)
    return list(seen.keys())

def understand_file(content: str, language: Optional[str]) -> CodeUnderstanding:
    try:
        if language == "python":
            return understand_python(content)
        if language in ("javascript", "typescript"):
            return understand_js(content)
        if language == "json":
            return understand_json_config(content)
        if language in ("yaml", "toml", "ini"):
            return understand_line_based_config(content)
        if language == "sql":
            return understand_sql(content)
    except Exception: # noqa: BLE001 - a file this app can't parse is just empty, never a scan failure
        pass
    return CodeUnderstanding()

# --- Python (stdlib ast - exact, no dependency) -----

HTTP_DECORATOR_METHODS = {"get", "post", "put", "patch", "delete", "websocket"}
_FORWARD_REF_RE = re.compile(r"^[A-Z][A-Za-z0-9_]*$")

def literal_str(node) -> Optional[str]:
    if isinstance(node, ast.Constant) and isinstance(node.value, str):
        return node.value
    return None

def kwarg_str(call: ast.Call, name: str) -> Optional[str]:
    for kw in call.keywords:
        if kw.arg == name:
            return literal_str(kw.value)
    return None

def dotted_name(node) -> Optional[str]:
    if isinstance(node, ast.Name):
        return node.id
    if isinstance(node, ast.Attribute):
        base = dotted_name(node.value)
        return f"{base}.{node.attr}" if base else node.attr
    return None

def route_from_decorator(decorator: ast.expr, router_prefixes: dict[str, str]) -> Optional[str]:
    if not isinstance(decorator, ast.Call):
        return None
    func = decorator.func
    if not isinstance(func, ast.Attribute) or func.attr not in _HTTP_DECORATOR_METHODS:
        return None
    if not isinstance(func.value, ast.Name):
        return None
    path = literal_str(decorator.args[0]) if decorator.args else None
    if path is None:
        return None
    prefix = router_prefixes.get(func.value.id, "")
    return f"{func.attr.upper()} {prefix}{path}"

def forward_ref_strings(annotation: ast.expr) -> list[str]:
    """Pulls string forward-references out of a `Mapped[...]` style type
    annotation, e.g. `Mapped["User"]` or `Mapped[List["Analysis"]]` - this
    project's own SQLAlchemy models declare every relationship's target
    exactly this way (see app/models/change_request.py), so this is a real,
    precise way to find ORM relationships, not a guess."""
    found = []
    for node in ast.walk(annotation):
        if isinstance(node, ast.Constant) and isinstance(node.value, str) and _FORWARD_REF_RE.match(node.value):
            found.append(node.value)

```

```

return found

def _understand_python(source: str) -> CodeUnderstanding:
    imports: list[str] = []
    functions: list[str] = []
    classes: list[str] = []
    api_routes: list[str] = []
    database_references: list[str] = []
    config_references: list[str] = []
    router_prefixes: dict[str, str] = {}

    try:
        tree = ast.parse(source)
    except SyntaxError:
        return CodeUnderstanding()

    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            imports.extend(alias.name for alias in node.names)
        elif isinstance(node, ast.ImportFrom):
            module = node.module or ""
            imports.extend(f"{module}.{alias.name}" if module else alias.name for alias in node.names)
        elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            functions.append(node.name)
            for decorator in node.decorator_list:
                route = _route_from_decorator(decorator, router_prefixes)
                if route:
                    api_routes.append(route)
        elif isinstance(node, ast.ClassDef):
            classes.append(node.name)
        elif isinstance(node, ast.Assign) and node.annotation is not None:
            if isinstance(node.target, ast.Name) and node.target.id == "__tablename__":
                value = _literal_str(node.value) if node.value is not None else None
                if value:
                    database_references.append(f"table:{value}")
            for ref in _forward_ref_strings(node.annotation):
                database_references.append(f"relationship:{ref}")
        elif isinstance(node, ast.Assign):
            for target in node.targets:
                if isinstance(target, ast.Name) and target.id == "__tablename__":
                    value = _literal_str(node.value)
                    if value:
                        database_references.append(f"table:{value}")
            if (
                len(node.targets) == 1
                and isinstance(node.targets[0], ast.Name)
                and isinstance(node.value, ast.Call)
                and _dotted_name(node.value.func) == "APIRouter"
            ):
                prefix = kwarg_str(node.value, "prefix") or ""
                router_prefixes[node.targets[0].id] = prefix
        elif isinstance(node, ast.Call):
            call_name = _dotted_name(node.func)
            if call_name == "ForeignKey" and node.args:
                target = _literal_str(node.args[0])
                if target:
                    database_references.append(f"foreign_key:{target}")
            elif call_name in ("os.getenv", "os.environ.get") and node.args:
                value = _literal_str(node.args[0])
                if value:
                    config_references.append(f"env:{value}")
            elif call_name and call_name.split(".")[1] == "query" and node.args:
                if isinstance(node.args[0], ast.Name):
                    database_references.append(f"query:{node.args[0].id}")
        elif isinstance(node, ast.Subscript):
            if _dotted_name(node.value) == "os.environ":
                key_node = node.slice
                value = _literal_str(key_node)
                if value:
                    config_references.append(f"env:{value}")
        elif isinstance(node, ast.Attribute):
            if isinstance(node.value, ast.Name) and node.value.id == "settings":
                config_references.append(f"settings:{node.attr}")

    return CodeUnderstanding(
        imports=dedupe(imports),
        functions=dedupe(functions),
        classes=dedupe(classes),
        api_routes=dedupe(api_routes),
        database_references=dedupe(database_references),
        config_references=dedupe(config_references),
    )

# --- JavaScript / TypeScript (regex best-effort) -----
_JS_IMPORT_FROM_RE = re.compile(r"^(?:import|export)\s+([^\s;]*)(?:\s+from\s+([^\s;]*))?(?:\s+as\s+([^\s;]*))?"
_JS_BARE_IMPORT_RE = re.compile(r"^(?:import)\s+([^\s;]*)(?:\s+as\s+([^\s;]*))?"
_JS_REQUIRE_RE = re.compile(r"^(?:require)\s+([^\s;]*)(?:\s+as\s+([^\s;]*))?"
_JS_DYNAMIC_IMPORT_RE = re.compile(r"^(?:import)\s+([^\s;]*)(?:\s+from\s+([^\s;]*))?(?:\s+as\s+([^\s;]*))?"
_JS_FUNCTION_DECL_RE = re.compile(r"^(?:function)\s+([A-Za-z_$][\w$]*)(?:\s+([^\s;]*))?"
_JS_CONST_ARROW_RE = re.compile(r"^(?:const)\s+([A-Za-z_$][\w$]*)\s*=\s*(?:async\s*)?(?:[^\s;]*)"
_JS_CONST_FUNC_RE = re.compile(r"^(?:const)\s+([A-Za-z_$][\w$]*)\s*=\s*(?:async\s*)?function\s*"
_JS_CLASS_RE = re.compile(r"^(?:class)\s+([A-Za-z_$][\w$]*)"
_JS_ROUTE_CALL_RE = re.compile(
    r"^(?:apiRequest|fetch|axios)\.([a-z_]{1,10})(?:\s+([a-z_]{1,10}))?"
)
_JS_ENV_RE = re.compile(r"^(?:import)\.meta\.env|process\.env)\.([A-Za-z_]{1,10}[A-Za-z0-9_]{0,9})"

def _understand_js(source: str) -> CodeUnderstanding:
    imports = (
        _JS_IMPORT_FROM_RE.findall(source)
        + _JS_BARE_IMPORT_RE.findall(source)
        + _JS_REQUIRE_RE.findall(source)
        + _JS_DYNAMIC_IMPORT_RE.findall(source)
    )
    functions = (
        _JS_FUNCTION_DECL_RE.findall(source)
        + _JS_CONST_ARROW_RE.findall(source)
        + _JS_CONST_FUNC_RE.findall(source)
    )
    classes = _JS_CLASS_RE.findall(source)
    api_routes = _JS_ROUTE_CALL_RE.findall(source)
    config_references = [f"env:{name}" for name in _JS_ENV_RE.findall(source)]

    return CodeUnderstanding(
        imports=dedupe(imports),
        functions=dedupe(functions),
        classes=dedupe(classes),
        api_routes=dedupe(api_routes),
        database_references=[],
        config_references=dedupe(config_references),
    )

# --- Config files (JSON exact, YAML/TOML/INI best-effort) -----

def _understand_json_config(source: str) -> CodeUnderstanding:

```

```

try:
    data = json.loads(source)
except ValueError:
    return CodeUnderstanding()
if isinstance(data, dict):
    keys = [f"key:{key}" for key in data.keys()]
    return CodeUnderstanding(config_references=_dedupe(keys))
return CodeUnderstanding()

_CONFIG_LINE_KEY_RE = re.compile(r"^[^s*]([A-Za-z_][A-Za-z0-9_\.~]*)\s*[:=]"")

def _understand_line_based_config(source: str) -> CodeUnderstanding:
    keys = []
    for line in source.splitlines():
        if line.strip().startswith(("#", ";", "[")):
            continue
        match = _CONFIG_LINE_KEY_RE.match(line)
        if match:
            keys.append(f"key:{match.group(1)}")
    return CodeUnderstanding(config_references=_dedupe(keys))

# --- SQL (keyword-based table references) -----
_SQL_TABLE_RE = re.compile(r"""\b(?:FROM|INTO|UPDATE|TABLE)\s+["'`]?([A-Za-z_][\w]*)""", re.IGNORECASE)

def _understand_sql(source: str) -> CodeUnderstanding:
    tables = [f"table:{name}" for name in _SQL_TABLE_RE.findall(source)]
    return CodeUnderstanding(database_references=_dedupe(tables))

```



```

"""Module 12 Phase 5: posting a comment - creates the row, records it on
the audit trail, and figures out who to notify. One comment notifies each
recipient exactly once: someone explicitly @mentioned gets the more
specific MENTIONED notification; the CR's creator and everyone currently
assigned to it (its "interested parties") get a plain COMMENT_ADDED one
instead, unless they were already covered by a mention. The author never
notifies themselves.
"""
from __future__ import annotations
from typing import Optional
from sqlalchemy.orm import Session

from app.models.change_request import ChangeRequest
from app.models.comment import ChangeRequestComment
from app.models.enums import HistoryAction, NotificationType
from app.models.user import User
from app.services import history
from app.services import notify as notify_service
from app.services.mentions import extract_mentioned_users

_PREVIEW_LIMIT = 200

def _preview(body: str) -> str:
    body = body.strip()
    return body if len(body) <= _PREVIEW_LIMIT else body[: _PREVIEW_LIMIT - 3] + "..."

def create_comment(
    db: Session,
    change_request: ChangeRequest,
    *,
    author: User,
    body: str,
    parent_id: Optional[int] = None,
) -> ChangeRequestComment:
    comment = ChangeRequestComment(
        change_request_id=change_request.id,
        user_id=author.id,
        parent_id=parent_id,
        body=body,
    )
    db.add(comment)
    db.flush()

    preview = _preview(body)
    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.COMMENT_ADDED,
        user_id=author.id,
        new_value=preview,
        version_number=change_request.current_version or 1,
    )

    all_users = db.query(User).all()
    mentioned = extract_mentioned_users(body, all_users, exclude_user_id=author.id)
    mentioned_ids = {u.id for u in mentioned}

    for user in mentioned:
        history.record_event(
            db,
            change_request_id=change_request.id,
            action=HistoryAction.MENTIONED,
            user_id=author.id,
            new_value=f"{user.name} was mentioned",
            version_number=change_request.current_version or 1,
        )
        notify_service.notify(
            db,
            user_id=user.id,
            type=NotificationType.MENTIONED,
            title=f"You were mentioned on CR-{change_request.id}",
            message=f"{author.name} mentioned you: \"{preview}\"",
            change_request_id=change_request.id,
        )

    interested_ids = {change_request.created_by} | {a.user_id for a in change_request.assignments}
    interested_ids.discard(author.id)
    for user_id in interested_ids - mentioned_ids:
        notify_service.notify(
            db,
            user_id=user_id,
            type=NotificationType.COMMENT_ADDED,
            title=f"New comment on CR-{change_request.id}",
            message=f"{author.name} commented: \"{preview}\"",
            change_request_id=change_request.id,
        )

    return comment

```

backend/app/services/deadlines.py

94 lines

```
"""Module 18 Phase 1 (Notifications, My Work & Personal Engineering Queue):
turns a PENDING approval's own due date into a real DEADLINE APPROACHING
notification - without a background job scheduler (per this project's own
"keep architecture minimal" rule, no Docker/Redis/Kubernetes cron runner is
being added just for this).

Instead, this is a lazy, opportunistic check: check_and_notify_approaching_
deadlines() is called from the notification-inbox endpoints themselves
(app/api/notifications.py), which the frontend already polls every 30
seconds for the unread-count badge - so a deadline is noticed the next
time its own approver's notifications are read, same responsiveness as
everything else in this app's notification system, with zero new moving
parts.

Scoped to exactly one user's own PENDING approvals (never a full-table
scan across every user in the system) - cheap enough to run on every
notification read. Fires at most once, ever, per Approval - idempotency is
enforced by checking for an existing DEADLINE APPROACHING notification
carrying that exact approval_id (see Notification.approval_id's own
docstring) before creating a new one, so a due-soon approval that later
becomes overdue is never re-notified a second time for the same deadline.
"""

from __future__ import annotations

from datetime import datetime

from sqlalchemy.orm import Session

from app.models.approval import Approval
from app.models.enums import ApprovalStatus, NotificationType
from app.models.notification import Notification
from app.models.user import User
from app.services import notify as notify_service
from app.services.approvals import DUE_SOON_WINDOW, approval_due_status
from app.services.workflow_rules import APPROVAL_TYPE_LABELS


def check_and_notify_approaching_deadlines(db: Session, user: User) -> None:
    """Looks only at user's own still-PENDING approvals with a due date
    that's already within the "due soon" window (or already overdue), and
    creates one DEADLINE APPROACHING notification per approval that
    doesn't already have one. Callers are responsible for committing - see
    app/api/notifications.py."""
    now = datetime.utcnow()
    candidates = (
        db.query(Approval)
        .filter(
            Approval.approver_id == user.id,
            Approval.status == ApprovalStatus.PENDING,
            Approval.due_date.isnot(None),
            Approval.due_date <= now + DUE_SOON_WINDOW,
        )
        .all()
    )
    if not candidates:
        return

    for approval in candidates:
        # approval.due_status re-checks PENDING/due date itself - the query
        # above is just a cheap pre-filter to avoid loading every approval
        # this user has ever been tagged on.
        if approval.due_status(approval, now=now) is None:
            continue

        already_notified = (
            db.query(Notification)
            .filter(
                Notification.type == NotificationType.DEADLINE_APPROACHING,
                Notification.approval_id == approval.id,
            )
            .first()
        )
        if already_notified is not None:
            continue

        type_label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        due_status = approval.due_status(approval, now=now)
        if due_status == "overdue":
            title = f"{type_label} approval on CR-{approval.change_request_id} is overdue"
            message = f"Your {type_label} approval was due {approval.due_date.strftime('%b %d, %Y')} and is still pending."
        else:
            title = f"{type_label} approval on CR-{approval.change_request_id} is due soon"
            message = f"Your {type_label} approval is due {approval.due_date.strftime('%b %d, %Y')}."

        notify_service.notify(
            db,
            user_id=user.id,
            type=NotificationType.DEADLINE_APPROACHING,
            title=title,
            message=message,
            change_request_id=approval.change_request_id,
            approval_id=approval.id,
        )
```

backend/app/services/history.py

44 lines

```
"""Module 12: the only place allowed to write ChangeRequestHistory rows
(spec section 24, "audit data integrity" - append-only). Every other
service/endpoint that needs to record something calls record_event()
rather than constructing a ChangeRequestHistory row directly, so there's
exactly one place that could ever violate append-only-ness to audit.
"""
from __future__ import annotations

from typing import Optional

from sqlalchemy.orm import Session

from app.models.change_request_history import ChangeRequestHistory
from app.models.enums import HistoryAction


def record_event(
    db: Session,
    *,
    change_request_id: int,
    action: HistoryAction,
    user_id: Optional[int] = None,
    actor_label: Optional[str] = None,
    field_name: Optional[str] = None,
    old_value: Optional[str] = None,
    new_value: Optional[str] = None,
    reason: Optional[str] = None,
    version_number: Optional[int] = None,
) -> ChangeRequestHistory:
    event = ChangeRequestHistory(
        change_request_id=change_request_id,
        user_id=user_id,
        actor_label=actor_label,
        action=action,
        field_name=field_name,
        old_value=old_value,
        new_value=new_value,
        reason=reason,
        version_number=version_number,
    )
    db.add(event)
    db.flush()
    return event
```

```

"""Module 16 Phase 2 (Project Knowledge Base & RAG - Parsing + Chunking).

Turns an uploaded document's file on disk into an ordered list of
KnowledgeChunk rows: extract plain text (format-specific), split it into
labeled sections wherever the format has natural structure, then split
each section into deterministic, size-bounded chunks. No AI is involved in
this step - it's pure text processing, so it runs synchronously right
after upload (see process_document()'s only caller,
app/api/knowledge.py:upload_document) - no queue/worker needed, matching
this app's "no unnecessary infrastructure" rule.

Per-format extraction (extract_sections):
* .txt: read as one unlabeled section - a plain text file has no
  natural internal structure to label sections by.
* .md: split on markdown headings (# through #####) - each heading's
  own text becomes that section's label; any content before the first
  heading is one leading unlabeled section. A markdown file with no
  headings at all falls back to being treated as one plain section,
  same as .txt.
* .pdf: each page becomes one section labeled "Page N" (via pypdf).

Chunking (chunk_sections / _chunk_section_text): a section's text is split
into paragraph-accumulated chunks capped at MAX_CHUNK_CHARS - paragraphs
(blank-line-separated) are never split across two chunks unless a single
paragraph alone exceeds the cap, in which case it's hard-split into fixed-
size pieces. This keeps chunks topically coherent without needing a
heavier NLP-aware splitter, matching the module's own spec instruction to
"use the simplest reliable architecture."

Failure (a corrupt/unreadable PDF, an unreadable text encoding, a document
with no extractable text at all) is caught by process_document() and
recorded as KnowledgeDocumentStatus.FAILED with error_message set -
mirroring app/services/repository_scanner.py's own "a per-item problem is
a recorded state, never an unhandled crash" pattern. A document's 'status'
only ever describes parsing/chunking here in Phase 2 - Module 16 Phase 3
(Embedding) fills in each chunk's own 'embedding' field afterwards without
needing a further document-level status of its own.
"""

import re
from datetime import datetime
from pathlib import Path
from typing import List, Optional, Tuple

from sqlalchemy.orm import Session

from app.models.enums import KnowledgeDocumentStatus
from app.models.knowledge_chunk import KnowledgeChunk
from app.models.knowledge_document import KnowledgeDocument
from app.services.knowledge_documents import document_file_path

# A chunk's content is capped at this many characters - generous enough to
# keep a paragraph or two of real documentation together (useful, coherent
# context for the AI in later phases), small enough that Phase 3's
# embedding calls and Phase 4's prompt payloads stay cheap and fast.
MAX_CHUNK_CHARS = 1200

_MARKDOWN_HEADING_RE = re.compile(r"^(#{1,6})\s+(\.*\S)\s*$")

class KnowledgeParsingError(Exception):
    """Raised for a document that can't be parsed at all (corrupt PDF,
    unreadable file, no extractable text). Always caught by
    process_document() and recorded as a FAILED document with this
    message as error_message - never raised out to the API layer
    directly."""

def _read_text_file(path: Path) -> str:
    try:
        return path.read_text(encoding="utf-8", errors="replace")
    except OSError as exc:
        raise KnowledgeParsingError(f"Could not read file: {exc}") from exc

def _extract_markdown_sections(text: str) -> List[Tuple[Optional[str], str]]:
    """Splits markdown text on headings - one section per heading, plus a
    leading unlabeled section for any content before the first heading.
    Sections with no non-whitespace content (e.g. a heading immediately
    followed by another heading) are dropped rather than producing an
    empty chunk."""
    lines = text.splitlines()
    raw_sections: List[Tuple[Optional[str], List[str]]] = []
    current_label: Optional[str] = None
    current_lines: List[str] = []

    for line in lines:
        match = _MARKDOWN_HEADING_RE.match(line)
        if match:
            raw_sections.append((current_label, current_lines))
            current_label = match.group(2).strip()
            current_lines = []
        else:
            current_lines.append(line)
    raw_sections.append((current_label, current_lines))

    sections = []
    for label, body_lines in raw_sections:
        body = "\n".join(body_lines).strip()
        if body:
            sections.append((label, body))
    return sections

def _extract_pdf_sections(path: Path) -> List[Tuple[Optional[str], str]]:
    try:
        from pypdf import PdfReader
    except ImportError as exc: # pragma: no cover - depends on an installed extra
        raise KnowledgeParsingError(
            "PDF support requires the 'pypdf' package - run `pip install -r requirements.txt`."
        ) from exc

    try:
        reader = PdfReader(str(path))
    except Exception as exc: # noqa: BLE001 - pypdf raises several distinct error types for a bad/corrupt PDF
        raise KnowledgeParsingError(f"Could not open PDF: {exc}") from exc

    sections: List[Tuple[Optional[str], str]] = []
    for page_number, page in enumerate(reader.pages, start=1):
        try:
            text = (page.extract_text() or "").strip()
        except Exception: # noqa: BLE001 - one unreadable page shouldn't fail the whole document
            text = ""
        if text:
            sections.append(("Page {page_number}", text))

```

```

if not sections:
    raise KnowledgeParsingError("No extractable text found in this PDF (it may be a scanned image).")

return sections

def extract_sections(path: Path, extension: str) -> List[Tuple[Optional[str], str]]:
    """Returns an ordered list of (section_label, section_text) pairs for
    the given file - the unit chunk_sections() splits within, and the
    unit Module 16 Phase 5's source-attribution "Section: ..." line reads
    directly from a resulting chunk's own section_label."""
    if extension == ".pdf":
        return _extract_pdf_sections(path)

    text = read_text_file(path)
    if extension == ".md":
        sections = extract_markdown_sections(text)
        if sections:
            return sections
        # A markdown file with no headings at all is just one big section,
        # same handling as a plain .txt file.
        stripped = text.strip()
        return [(None, stripped)] if stripped else []

    stripped = text.strip()
    return [(None, stripped)] if stripped else []

def _split_paragraph(paragraph: str) -> List[str]:
    """Hard-splits a single paragraph longer than MAX_CHUNK_CHARS into
    fixed-size pieces - only reached for the rare paragraph that alone
    exceeds the cap (e.g. one long unbroken block of text with no blank
    lines). Together these pieces reconstruct the paragraph exactly, with
    nothing dropped."""
    return [paragraph[i : i + MAX_CHUNK_CHARS] for i in range(0, len(paragraph), MAX_CHUNK_CHARS)]

def _chunk_section_text(text: str) -> List[str]:
    """Splits one section's text into paragraph-accumulated chunks capped
    at MAX_CHUNK_CHARS - a paragraph (blank-line-separated) is never split
    across two chunks unless it alone exceeds the cap."""
    paragraphs = [p.strip() for p in re.split(r"\n\s*\n", text) if p.strip()]
    chunks: List[str] = []
    current = ""

    for paragraph in paragraphs:
        pieces = _split_paragraph(paragraph) if len(paragraph) > MAX_CHUNK_CHARS else [paragraph]
        for piece in pieces:
            candidate = f"{current}\n\n{piece}" if current else piece
            if len(candidate) > MAX_CHUNK_CHARS and current:
                chunks.append(current)
                current = piece
            else:
                current = candidate

    if current:
        chunks.append(current)

    return chunks

def chunk_sections(sections: List[Tuple[Optional[str], str]]) -> List[Tuple[Optional[str], str]]:
    """Expands each (section_label, section_text) pair into one or more
    (section_label, chunk_text) pairs, in original order - a long section
    yields several chunks that all share its own section_label."""
    result: List[Tuple[Optional[str], str]] = []
    for label, text in sections:
        for chunk_text in _chunk_section_text(text):
            result.append((label, chunk_text))
    return result

def process_document(db: Session, document: KnowledgeDocument) -> KnowledgeDocument:
    """Parses, chunks, and persists chunks for one document, updating its
    status along the way (UPLOADED/FAILED -> PROCESSING -> READY/FAILED).
    Safe to call again on any document, including one that already has
    chunks (e.g. a manual re-process after fixing a bad upload) - existing
    chunks are replaced, never left to accumulate duplicates alongside a
    fresh set, and never removed unless a full new set is ready to take
    their place."""
    document.status = KnowledgeDocumentStatus.PROCESSING
    document.error_message = None
    db.commit()

    try:
        path = document_file_path(document)
        sections = extract_sections(path, document.extension)
        chunk_pairs = chunk_sections(sections)
        if not chunk_pairs:
            raise KnowledgeParsingError("No text content could be extracted from this document.")

        for existing in list(document.chunks):
            db.delete(existing)
        db.flush()

        for index, (label, content) in enumerate(chunk_pairs):
            db.add(
                KnowledgeChunk(
                    document_id=document.id,
                    chunk_index=index,
                    content=content,
                    section_label=label,
                    char_count=len(content),
                )
            )

        document.status = KnowledgeDocumentStatus.READY
        document.chunk_count = len(chunk_pairs)
        document.processed_at = datetime.utcnow()
        db.commit()
        db.refresh(document)
        return document
    except Exception as exc: # noqa: BLE001 - any parsing/chunking failure is a recorded FAILED state, never a crash
        db.rollback()
        document.status = KnowledgeDocumentStatus.FAILED
        document.error_message = str(exc)
        document.processed_at = datetime.utcnow()
        db.commit()
        db.refresh(document)
        return document

```

```

"""Module 16 Phase 1 (Project Knowledge Base & RAG): validating and
storing an uploaded knowledge-source document.

Files are saved to plain local disk under backend/data/knowledge_documents/
- the same "no unnecessary infrastructure" choice this app has made
everywhere else (SQLite instead of Postgres, no Docker, and per Module 15's
own repository scanner, no separate blob store for indexed source files
either). This also has a useful cross-module safety property: backend/
data/ is already excluded from Module 15's own repository scanner via its
IGNORED_DIR_NAMES set (which includes "data"), so an uploaded knowledge
document can never accidentally get re-indexed as a project source file by
Repository Intelligence.

This module only handles Phase 1's slice: validate -> save to disk ->
create the KnowledgeDocument row with status UPLOADED. Parsing/chunking
(Phase 2) and embedding (Phase 3) are separate services that pick a
document up from there.
"""
import uuid
from pathlib import Path
from typing import Optional

from sqlalchemy.orm import Session

from app.models.knowledge_document import KnowledgeDocument
from app.models.enums import KnowledgeDocumentStatus

# .txt/.md/.pdf only for now, per this module's own spec section 1
# ("practical technical documents") - deliberately not html/docx/etc,
# matching the "simplest reliable architecture" instruction in spec
# section 2. Extending this set later is a one-line change here plus
# whatever Phase 2 parsing that new format needs.
ALLOWED_EXTENSIONS = {".txt", ".md", ".pdf"}

# 10 MB - generous for a technical doc/spec/policy at hackathon scale,
# small enough that parsing/chunking/embedding it stays fast and cheap.
MAX_DOCUMENT_BYTES = 10 * 1024 * 1024


class KnowledgeDocumentError(Exception):
    """Raised for any reason an upload can't be accepted - unsupported
    extension, empty file, oversized file. The API layer turns this into a
    400 with the message as-is (same pattern as RepositoryScanError in
    Module 15)."""


def storage_root() -> Path:
    """backend/data/knowledge_documents/ - resolved from this file's own
    location (services -> app -> backend) rather than hardcoded, same
    reasoning as Settings.repository_root_path in core/config.py, so this
    resolves correctly on any machine this app is checked out on. Created
    on first use rather than assumed to exist."""
    root = Path(file).resolve().parents[2] / "data" / "knowledge_documents"
    root.mkdir(parents=True, exist_ok=True)
    return root


def safe_stem(original_filename: str) -> str:
    """A filesystem-safe stem derived from the original filename, kept
    short and free of path separators - the actual uniqueness guarantee
    comes from the UUID suffix added in save_uploaded_document, this just
    keeps the stored filename recognizable at a glance in the data/
    folder."""
    stem = Path(original_filename).stem.strip()
    safe = "".join(ch if ch.isalnum() or ch in ("-", "_") else "_" for ch in stem)
    safe = safe.strip("_") or "document"
    return safe[:80]


def save_uploaded_document(
    db: Session,
    *,
    original_filename: str,
    content: bytes,
    uploaded_by: Optional[int],
) -> KnowledgeDocument:
    """Validates and saves one uploaded file, then creates+commits+
    refreshes its KnowledgeDocument row with status UPLOADED (chunking/
    embedding happen later, in Phase 2/3). Raises KnowledgeDocumentError
    for any validation failure - nothing is written to disk or the
    database in that case."""
    if not original_filename or not original_filename.strip():
        raise KnowledgeDocumentError("A filename is required.")

    extension = Path(original_filename).suffix.lower()
    if extension not in ALLOWED_EXTENSIONS:
        allowed = ", ".join(sorted(ALLOWED_EXTENSIONS))
        raise KnowledgeDocumentError(f"Unsupported file type {extension!r} - allowed types: {allowed}.")

    if not content:
        raise KnowledgeDocumentError("The uploaded file is empty.")

    if len(content) > MAX_DOCUMENT_BYTES:
        max_mb = MAX_DOCUMENT_BYTES // (1024 * 1024)
        raise KnowledgeDocumentError(f"The uploaded file is too large - the limit is {max_mb} MB.")

    stored_filename = f"{safe_stem(original_filename)}_{uuid.uuid4().hex[:12]}{extension}"
    destination = storage_root() / stored_filename
    destination.write_bytes(content)

    document = KnowledgeDocument(
        title=Path(original_filename).stem.strip() or original_filename,
        original_filename=original_filename,
        stored_filename=stored_filename,
        extension=extension,
        size_bytes=len(content),
        status=KnowledgeDocumentStatus.UPLOADED,
        chunk_count=0,
        archived=False,
        uploaded_by=uploaded_by,
    )
    db.add(document)
    db.commit()
    db.refresh(document)
    return document


def document_file_path(document: KnowledgeDocument) -> Path:
    """The absolute path to a document's stored file on disk - used by
    Phase 2's parsing step, and kept here rather than duplicated so
    storage layout only ever lives in one place."""
    return storage_root() / document.stored_filename

```

```

"""Module 16 Phase 3 (Project Knowledge Base & RAG - Embedding + Vector
Search): turns a document's already-chunked text into real AI embeddings
via this app's existing swappable AIProvider abstraction, and does brute-
force cosine-similarity search over them.

Deliberately NOT triggered automatically by upload/chunking (Phase 2) -
embedding needs a network call to an AI provider, unlike parsing/chunking
which is pure local text processing. Keeping it a separate, explicitly-
triggered step means:
* Phase 1/2's own tests stay network-free - they never call a real AI
provider, matching this app's established testing pattern (the
analysis engine and repository-matching tests always monkeypatch
get_ai_provider rather than hitting a real API - see
tests/test_module15_phase3.py and tests/test_module16_phase3.py for
this phase's own version of that same pattern).
* A slow/failed/rate-limited embedding call never blocks or corrupts an
otherwise-successful upload+chunk - a document already sitting at
"ready" (chunked - see knowledge_chunker.py) stays exactly that
regardless of what happens here.

No vector database - each chunk's embedding is stored as a JSON-encoded
list[float] on KnowledgeChunk.embedding (same "one text column, not a new
table" pattern used throughout this app - ChangeRequest.tags,
IndexedFile's Phase 2 columns), and search is brute-force cosine
similarity computed in pure Python. Perfectly fine at hackathon scale (a
handful of documents, at most a few hundred chunks) and avoids the
"unnecessary infrastructure" this module's own spec explicitly warns
against (no Pinecone/pgvector/FAISS/etc).
"""
import json
import math
from typing import List

from sqlalchemy.orm import Session

from app.models.knowledge_chunk import KnowledgeChunk
from app.models.knowledge_document import KnowledgeDocument
from app.services.ai.factory import get_ai_provider


class KnowledgeEmbeddingError(Exception):
    """Raised when embeddings can't be produced right now - the
    configured AI provider isn't set up, doesn't support embed() at all,
    or a network/provider call failed. Caught by the API layer and turned
    into a 400 with a plain-language reason, never a bare 500."""


def embed_document(db: Session, document: KnowledgeDocument) -> KnowledgeDocument:
    """Embeds every chunk of one document - re-embedding replaces each
    chunk's prior vector, if any, rather than leaving old and new vectors
    both sitting around. Raises KnowledgeEmbeddingError on any failure;
    nothing about the document or its chunks is changed in that case, so
    a failed embed attempt never leaves a document half-embedded (the
    chunk loop below only assigns vectors after the provider call has
    already returned a complete, correctly-sized result for all of
    them)."""
    chunks: List[KnowledgeChunk] = list(document.chunks)
    if not chunks:
        raise KnowledgeEmbeddingError("This document has no chunks to embed yet - process it first.")

    provider = get_ai_provider()
    try:
        vectors = provider.embed([chunk.content for chunk in chunks])
    except NotImplementedError as exc:
        raise KnowledgeEmbeddingError(str(exc)) from exc
    except KnowledgeEmbeddingError:
        raise
    except Exception as exc:
        # noqa: BLE001 - any provider/network failure becomes one clean, catchable error
        raise KnowledgeEmbeddingError(f"Embedding failed: {exc}") from exc

    if len(vectors) != len(chunks):
        raise KnowledgeEmbeddingError(
            f"Embedding provider returned {len(vectors)} vectors for {len(chunks)} chunks."
        )

    for chunk, vector in zip(chunks, vectors):
        chunk.embedding = json.dumps(vector)
    db.commit()
    db.refresh(document)
    return document


def _cosine_similarity(a: List[float], b: List[float]) -> float:
    if not a or not b or len(a) != len(b):
        return 0.0
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return dot / (norm_a * norm_b)


def search_chunks(db: Session, query: str, *, top_k: int = 5, min_score: float = 0.0) -> List[dict]:
    """Embeds `query` with the configured AI provider, then ranks every
    embedded, non-archived document's chunk by cosine similarity to it -
    brute force, fine at hackathon scale (see module docstring). A chunk
    belonging to an archived document, or one that hasn't been embedded
    yet, is silently excluded from ranking (never an error) - same as an
    archived document already being excluded from the default document
    list in api/knowledge.py::list_documents.

    `min_score` (default 0.0 - no filtering) drops any result scoring
    below it before truncating to `top_k`. The manual search endpoint
    (api/knowledge.py::search_knowledge_base) leaves this at 0.0 - a
    person searching may still want to see the closest results even if
    weak. app/services/analysis_engine.py::retrieve_knowledge_context
    passes a real threshold instead: injecting a barely-related chunk into
    the AI's prompt as "Relevant Documentation" risks exactly the
    fabrication this module's own spec warns against (section 6 - "Do not
    fabricate project facts"), so grounding context should only ever be
    genuinely relevant, not merely "the least-bad thing on file." A
    deliberately simple, adjustable heuristic - not a scientifically tuned
    cutoff.

    Returns the top `top_k` results (after the `min_score` filter) as
    plain dicts (not ORM objects, since each carries a computed similarity
    score alongside the chunk/document data), ordered by similarity,
    highest first. Returns [] for a blank query rather than embedding an
    empty string."""
    if not query or not query.strip():
        return []

    provider = get_ai_provider()

```

```

try:
    query_vector = provider.embed([query])[0]
except NotImplementedError as exc:
    raise KnowledgeEmbeddingError(str(exc)) from exc
except Exception as exc: # noqa: BLE001
    raise KnowledgeEmbeddingError(f"Embedding the search query failed: {exc}") from exc

candidates = (
    db.query(KnowledgeChunk)
    .join(KnowledgeDocument, KnowledgeChunk.document_id == KnowledgeDocument.id)
    .filter(KnowledgeDocument.archived.is_(False))
    .filter(KnowledgeChunk.embedding.isnot(None))
    .all()
)

scored = []
for chunk in candidates:
    try:
        vector = json.loads(chunk.embedding)
    except (TypeError, ValueError):
        continue
    score = _cosine_similarity(query_vector, vector)
    if score >= min_score:
        scored.append((score, chunk))

scored.sort(key=lambda pair: pair[0], reverse=True)

results = []
for score, chunk in scored[:top_k]:
    document = chunk.document
    results.append(
        {
            "score": score,
            "chunk_id": chunk.id,
            "document_id": document.id,
            "document_title": document.title,
            "section_label": chunk.section_label,
            "content": chunk.content,
        }
    )
return results

```


backend/app/services/mentions.py

36 lines

```
"""Module 12 Phase 5: parses @mentions out of a comment's raw text.

Deliberately simple - not a tokenizer/parser, just "does '@<user's full
name>' appear in this text", checked against every registered user (a small
team, no pagination concerns), longest name first so a short name (e.g.
"Al") can't shadow-match ahead of a longer one that starts the same way
(e.g. "Alice Tester"). Good enough for a small team; the frontend's mention
picker (see frontend/src/components/ChangeRequestDetail.jsx) inserts the
exact "@Full Name" text so what's typed always matches exactly what's
stored - nothing here tries to guess partial or misspelled names.
"""
from __future__ import annotations

from typing import Iterable, List, Optional

from app.models.user import User


def extract_mentioned_users(
    body: str, candidates: Iterable[User], *, exclude_user_id: Optional[int] = None
) -> List[User]:
    """Every candidate user whose "@Full Name" appears in `body`, longest
    name first, deduplicated, excluding `exclude_user_id` (the comment's
    own author - mentioning yourself doesn't notify yourself)."""
    lowered = body.lower()
    matched: List[User] = []
    seen_ids: set[int] = set()
    for candidate in sorted(candidates, key=lambda u: len(u.name), reverse=True):
        if candidate.id == exclude_user_id or candidate.id in seen_ids:
            continue
        needle = f"@{candidate.name}".lower()
        if needle in lowered:
            matched.append(candidate)
            seen_ids.add(candidate.id)
    return matched
```

backend/app/services/notify.py

46 lines

"""Module 12 Phase 5: the one place that creates Notification rows - same "one gatekeeper" pattern as app/services/history.py for the audit trail, so there's exactly one place that could ever get a notification's shape wrong.

In-app only (spec: "Do not implement email unless the existing application already supports it" - it doesn't). Every event listed on app/models/notification.py's own docstring is raised from here: assigned to a CR (app/api/change_requests.py::create_assignment), tagged for approval / reminded (create_approval_request / remind_approval), their CR's status changed (change_status), their approval was responded to (respond_to_approval_request), they were @mentioned or someone commented (app/services/comments.py), their AI analysis went outdated, or a re-approval is needed (both app/services/versioning.py).

```
from __future__ import annotations
from typing import Optional
from sqlalchemy.orm import Session

from app.models.enums import NotificationType
from app.models.notification import Notification

def notify(
    db: Session,
    *,
    user_id: int,
    type: NotificationType,
    title: str,
    message: str,
    change_request_id: Optional[int] = None,
    approval_id: Optional[int] = None,
) -> Notification:
    notification = Notification(
        user_id=user_id,
        type=type,
        title=title,
        message=message,
        change_request_id=change_request_id,
        approval_id=approval_id,
    )
    db.add(notification)
    db.flush()
    return notification
```

backend/app/services/permissions.py

70 lines

```
"""Module 21 spec section 3: role -> capability permissions. A plain
lookup table (RolePermission rows), not a rule engine - the same "simple,
readable config" philosophy workflow_rules.py's approval matrix already
uses for section 4.

DEFAULT PERMISSIONS below is what every role could already do before this
module existed: shipping this never silently takes anything away on day
one (init db seeds exactly these values - see
app/database/init_db.py::seed_role_permissions). From there, an admin can
tighten (or loosen, except Administration) individual role/capability
pairs through PUT /api/admin/permissions.

Administration is deliberately excluded from what an admin can edit
through that endpoint (see app/api/admin.py) - it's hardcoded True for
Admin and False for everyone else, always, so an admin can never
accidentally lock every admin account out of the admin section itself.

This module never talks to the database except has_permission(), which
takes an already-open Session the same way every other permission helper
in this app does (see workflow_rules.py's own module docstring) - callers
own the session, this stays a small, readable rule check.
"""
from __future__ import annotations

from sqlalchemy.orm import Session

from app.models.enums import Capability, UserRole
from app.models.role_permission import RolePermission
from app.models.user import User
from app.services.workflow_rules import is_admin

# Every (role, capability) pair, and what it defaults to - matches how the
# app already behaved before Module 21: none of these 9 actions were ever
# gated by ACCOUNT role before now (they were gated by per-change-request
# ownership/assignment instead, which still applies underneath - see each
# endpoint's existing can_edit_change_request / can_manage_workflow /
# approval-ownership checks, none of which this module replaces).
# Administration is the one exception - only Admin could ever reach
# /api/admin/* (it didn't exist before this module), so False is the only
# honest default for every other role.
DEFAULT_PERMISSIONS: dict[tuple[UserRole, Capability], bool] = {
    (role, capability): (capability != Capability.ADMINISTRATION or role == UserRole.ADMIN)
    for role in UserRole
    for capability in Capability
}

# Never editable through PUT /api/admin/permissions, in either direction -
# see this module's own docstring above.
LOCKED_CAPABILITIES: set[Capability] = {Capability.ADMINISTRATION}

def has_permission(db: Session, user: User, capability: Capability) -> bool:
    """Admin always has every capability (same "admin is a superuser"
    convention as workflow_rules.is_admin's use everywhere else in this
    app), regardless of what the matrix says - so an admin can never be
    accidentally locked out of the app by a permissions row. Every other
    role reads its stored row; a role/capability pair with no stored row
    at all (shouldn't happen - init db seeds every combination) falls back
    to the same default the matrix was seeded with, never a silent deny."""
    if is_admin(user):
        return True
    row = (
        db.query(RolePermission)
        .filter(RolePermission.role == user.role, RolePermission.capability == capability)
        .first()
    )
    if row is not None:
        return row.allowed
    return DEFAULT_PERMISSIONS.get((user.role, capability), True)
```

```
backend/app/services/report_generator.py
```

```
"""Module 11: Report Generation, made version-aware by Module 20.
```

Builds a professional PDF from an EXISTING ChangeRequest + (usually) an Analysis, for engineering manager / CAB review. This module never calls an AI provider and never creates an Analysis row - it only reads and formats data that Module 6's analysis engine already generated and persisted, plus version/approval/history data Module 12 already tracks.

Module 20's core rule (spec section 7): "Do not mix Version 3 CR data with Version 4 analysis." Before this module, a report always read the CR's LIVE field values alongside whatever the latest analysis happened to be - if the CR had been edited since that analysis ran, the report silently contained data from two different versions (the outdated-analysis banner told the reader "you might be looking at old analysis," but the CR fields displayed were still always today's, not the version the analysis actually ran against). resolve_report_context() below is what fixes that: it resolves BOTH the change request fields AND the analysis to one single, consistent version before generate_report.pdf ever touches them - either the live current version (normal case) or a past version reconstructed from that version's ChangeRequestVersion.snapshot (a historical report, clearly labeled as such - never presented as "current"). The API layer (app/api/change_requests.py::download_report) is what turns "current version has no matching analysis" into a 409 instead of letting this module quietly build a misleading report.

Every string that came from the AI (summary, reasoning, descriptions, mitigations, ...) is routed through `_sanitize()` before being handed to `fpdf2`: the AI providers can and do produce Unicode punctuation (smart quotes, em dashes, ellipses, the odd emoji) that the PDF's core fonts can't encode. Rather than letting one such character blow up report generation for an otherwise-fine analysis, common Unicode punctuation is transliterated to its ASCII equivalent and anything else unmappable is replaced rather than raising - "handle PDF generation errors gracefully" starts here, not just in the `try/except` at the API layer.

```
from __future__ import annotations
```

```
import json
from dataclasses import dataclass, field
from datetime import date, datetime
from typing import Iterable, Optional
```

```
from fpdf import FPDF
from sqlalchemy.orm import Session, selectinload
```

```
from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_history import ChangeRequestHistory
from app.models.change_request_version import ChangeRequestVersion
from app.models.enums import ApprovalStatus, HistoryAction
from app.services.workflow_rules import (
    APPROVAL_STATUS_LABELS,
    APPROVAL_TYPE_LABELS,
    FIELD_LABELS,
    STATUS_LABELS,
)
```

```
PAGE_MARGIN = 18
ACCENT_COLOR = (67, 56, 202) # the app's #4338ca purple
TEXT_COLOR = (16, 24, 40)
MUTED_COLOR = (102, 112, 133)
BORDER_COLOR = (208, 213, 221)
HISTORICAL_COLOR = (30, 64, 175) # a neutral blue - "labeled", not "warning"
HISTORICAL_FILL = (239, 246, 255)
WARNING_COLOR = (146, 64, 14)
WARNING_FILL = (255, 244, 229)
RISK_COLORS = {
    "low": (5, 150, 105),
    "medium": (202, 138, 4),
    "high": (217, 119, 6),
    "critical": (220, 38, 38),
}
```

```
class ReportGenerationError(Exception):
    """Raised when the PDF can't be built. Caught by the API layer and
    turned into a clean error response instead of a raw traceback."""
```

```
class ReportVersionError(Exception):
    """Raised when a requested version number doesn't exist for this
    change request - never below 1, never above the CR's current
    version. """
```

```
def sanitize(value) -> str:
    """Every AI-written string passes through here before reaching pdf2's
    core (non-Unicode) fonts. Known "smart" punctuation is transliterated
    to ASCII; anything else outside Latin-1 is replaced rather than
    crashing the whole report over a single stray character."""
    if value is None:
        return ""
    text = str(value)
    for src, dst in UNICODE_REPLACEMENTS.items():
        text = text.replace(src, dst)
    return text.encode("latin-1", errors="replace").decode("latin-1")
```

```
def title_case(value: Optional[str]) -> str:
    if not value:
        return ""
    return sanitize(value).replace(" ", "-").title()
```

```
def fmt_date(value) -> str:
    if value is None:
        return "Not provided"
```

```

try:
    return value.strftime("%B %d, %Y")
except AttributeError:
    return _sanitize(value)

def fmt_datetime(value) -> str:
    if value is None:
        return "-"
    try:
        return value.strftime("%B %d, %Y %H:%M UTC")
    except AttributeError:
        return _sanitize(value)

def _enum_value(member) -> Optional[str]:
    return member.value if member is not None else None

def _priority_label(value) -> str:
    """Priority arrives as a real Priority enum member when read straight
    off a live ChangeRequest, but as a plain string once it's come back out
    of a ChangeRequestVersion.snapshot (see workflow rules.normalize_field_
    value) - this reads either shape the same way."""
    if value is None:
        return "-"
    raw = value.value if hasattr(value, "value") else value
    return _title_case(raw)

# --- Version-aware field/context resolution (Module 20) -----
#
# ResolvedFields is the single shape generate_report_pdf renders from - it
# never reads change_request.title/description/... directly, so it can't
# accidentally mix a historical version's CR data with the live row's.

@dataclass
class ResolvedFields:
    title: str
    description: str
    business_objective: Optional[str]
    priority_label: str
    requested_by: Optional[str]
    target_system: Optional[str]
    desired_deadline: Optional[date]
    business_impact: Optional[str]
    technical_impact: Optional[str]
    customer_impact: Optional[str]
    environment: Optional[str]
    dependencies_note: Optional[str]
    compliance_requirements: Optional[str]

def _resolve_fields_live(change_request: ChangeRequest) -> ResolvedFields:
    return ResolvedFields(
        title=change_request.title,
        description=change_request.description,
        business_objective=change_request.business_objective,
        priority_label=_priority_label(change_request.priority),
        requested_by=change_request.requested_by,
        target_system=change_request.target_system,
        desired_deadline=change_request.desired_deadline,
        business_impact=change_request.business_impact,
        technical_impact=change_request.technical_impact,
        customer_impact=change_request.customer_impact,
        environment=change_request.environment,
        dependencies_note=change_request.dependencies_note,
        compliance_requirements=change_request.compliance_requirements,
    )

def _resolve_fields_from_snapshot(snapshot_json: str, *, fallback: ResolvedFields) -> ResolvedFields:
    """'fallback' (the live row's fields) only ever backfills title/
    description - the two fields a report can't sensibly render blank -
    for the edge case of a Version 1 snapshot backfilled by init db before
    every column existed. Every other field is read strictly from the
    snapshot: if it wasn't set as of that version, it wasn't set, and the
    report should say so rather than silently borrowing today's value."""
    try:
        data = json.loads(snapshot_json) if snapshot_json else {}
    except (TypeError, ValueError):
        data = {}

    deadline = None
    deadline_raw = data.get("desired_deadline")
    if deadline_raw:
        try:
            deadline = date.fromisoformat(deadline_raw)
        except (TypeError, ValueError):
            deadline = None

    return ResolvedFields(
        title=data.get("title") or fallback.title,
        description=data.get("description") or fallback.description,
        business_objective=data.get("business_objective"),
        priority_label=_priority_label(data.get("priority")),
        requested_by=data.get("requested_by"),
        target_system=data.get("target_system"),
        desired_deadline=deadline,
        business_impact=data.get("business_impact"),
        technical_impact=data.get("technical_impact"),
        customer_impact=data.get("customer_impact"),
        environment=data.get("environment"),
        dependencies_note=data.get("dependencies_note"),
        compliance_requirements=data.get("compliance_requirements"),
    )

# Mirrors app/api/change_requests.py:: ANALYSIS RELATIONSHIPS - duplicated
# rather than imported (same api/ vs services/ dependency-direction
# convention already used for CANONICAL CATEGORIES in analytics.py/
# dashboard.py) so this module never depends on the API layer.
_REPORT_ANALYSIS_RELATIONSHIPS = (
    selectinload(Analysis.requirements),
    selectinload(Analysis.affected_components),
    selectinload(Analysis.dependencies),
    selectinload(Analysis.risks),
    selectinload(Analysis.impact_assessments),
    selectinload(Analysis.security_findings),
    selectinload(Analysis.clarification_questions),
    selectinload(Analysis.test_cases),
    selectinload(Analysis.implementation_tasks),
)

@dataclass
class ReportContext:
    """Everything generate_report_pdf needs, already resolved to one
    consistent change-request version - the piece that makes "don't mix
    Version 3 CR data with Version 4 analysis" actually true rather than

```

```

just documented. `report_version` is the version this ENTIRE report
represents; `analysis` (if not None) is guaranteed to have been run
against exactly that version - never a newer or older one."""

report_version: int
is_historical: bool
fields: ResolvedFields
analysis: Optional[Analysis]
approvals: list[Approval] = field(default_factory=list)
versions: list[ChangeRequestVersion] = field(default_factory=list)
history_events: list[ChangeRequestHistory] = field(default_factory=list)

@property
def analysis_matches_version(self) -> bool:
    return self.analysis is not None

def resolve_report_context(
    db: Session,
    change_request: ChangeRequest,
    *,
    requested_version: Optional[int] = None,
) -> ReportContext:
    """Resolves which version this report represents (the CR's current
    version by default, or `requested_version` for an explicit historical
    report), then reconstructs every piece of data a report needs - CR
    fields, the analysis (if any) run against exactly that version,
    approvals requested against that version or earlier, the full version
    history, and the audit trail up through that version - from that one
    version, consistently.

    Approvals/history are scoped with "at or before this version" rather
    than "only exactly this version" - an approval or history event from
    an earlier version is still part of this version's story (nothing has
    un-happened), but one from a LATER version hasn't occurred yet as far
    as a report representing an earlier point in time is concerned. For a
    report of the CR's current version this scoping is a no-op (nothing
    in this change request's own history can be dated after its own
    current version), so today's "current report" behavior is unchanged.
    """
    current_version = change_request.current_version or 1
    target_version = requested_version if requested_version is not None else current_version

    if target_version < 1 or target_version > current_version:
        raise ReportVersionError(
            f"Version {target_version} does not exist for this change request "
            f"(current version is {current_version})."
        )

    is_historical = target_version != current_version
    live_fields = _resolve_fields_live(change_request)

    if is_historical:
        version_row = next(
            (v for v in change_request.versions if v.version_number == target_version), None
        )
        if version_row is None:
            # Defensive - versions are created sequentially and never
            # deleted, so every version from 1..current_version should
            # always have a row.
            raise ReportVersionError(f"No version record found for version {target_version}.")
        fields = _resolve_fields_from_snapshot(version_row.snapshot, fallback=live_fields)
    else:
        fields = live_fields

    analysis = (
        db.query(Analysis)
        .options(*_REPORT_ANALYSIS_RELATIONSHIPS)
        .filter(
            Analysis.change_request_id == change_request.id,
            Analysis.change_request_version == target_version,
        )
        .order_by(Analysis.created_at.desc())
        .first()
    )

    approvals = sorted(
        (a for a in change_request.approvals if (a.cr_version or 1) <= target_version),
        key=lambda a: a.requested_at,
    )
    # Same "at or before this version" scoping as approvals/history_events
    # above - a historical report must never show a LATER version's row in
    # its own Change History table. Each ChangeRequestVersion's
    # change summary embeds the actual old/new field text for that edit
    # (e.g. "Description changed from ... to <the new text>"), so an
    # unfiltered list would leak a future version's field values into an
    # earlier version's report exactly the way spec section 7 forbids.
    versions = sorted(
        (v for v in change_request.versions if v.version_number <= target_version),
        key=lambda v: v.version_number,
    )
    history_events = sorted(
        (h for h in change_request.history if (h.version_number or 1) <= target_version),
        key=lambda h: h.created_at,
    )

    return ReportContext(
        report_version=target_version,
        is_historical=is_historical,
        fields=fields,
        analysis=analysis,
        approvals=approvals,
        versions=versions,
        history_events=history_events,
    )

class ReportPDF(FPDF):
    def __init__(self, cr_code: str, cr_title: str):
        super().__init__(orientation="P", unit="mm", format="A4")
        self.cr_code = cr_code
        self.cr_title = _sanitize(cr_title)
        self.set_auto_page_break(auto=True, margin=20)
        self.set_margins(PAGE_MARGIN, 14, PAGE_MARGIN)
        # Computed AFTER set_margins() - the constructor's own default
        # margins (10mm) are not the ones this report actually uses, and
        # computing this too early silently produced a page width ~16mm
        # wider than what's actually printable.
        self.page_width = self.w - self.l_margin - self.r_margin
        self.set_title(f"{cr_code} Analysis Report")

    def header(self):
        if self.page_no() == 1:
            return
        self.set_font("Helvetica", "", 8)
        self.set_text_color(MUTED_COLOR)
        self.set_xy(self.l_margin, 8)
        self.cell(self.page_width - 20, 5, f"{self.cr_code} - {self.cr_title}", new_x="RIGHT", new_y="TOP")
        self.set_xy(self.w - self.r_margin - 20, 8)
        self.cell(20, 5, f"Page {self.page_no()}", align="R", new_x="LMARGIN", new_y="NEXT")
        self.set_y(15)

```

```

self.set_draw_color(*BORDER_COLOR)
self.line(self.l_margin, self.get_y(), self.w - self.r_margin, self.get_y())
self.ln(6)

def footer(self):
    self.set_y(-14)
    self.set_font("Helvetica", "I", 7)
    self.set_text_color(*MUTED_COLOR)
    self.cell(
        self._page_width,
        8,
        "Generated by AI Change Request Analyzer - contains AI-generated analysis, for human review",
        align="C",
        new_x="LMARGIN",
        new_y="NEXT",
    )

def section_heading(self, number: int, title: str):
    if self.get_y() > self.h - 35:
        self.add_page()
    self.ln(4)
    self.set_font("Helvetica", "B", 13)
    self.set_text_color(*ACCENT_COLOR)
    self.cell(self._page_width, 8, f"{number}. {title}", new_x="LMARGIN", new_y="NEXT")
    self.ln(1)
    self.set_draw_color(*ACCENT_COLOR)
    self.set_line_width(0.6)
    self.line(self.l_margin, self.get_y(), self.l_margin + 28, self.get_y())
    self.set_line_width(0.2)
    self.ln(3)
    self.set_text_color(*TEXT_COLOR)

def body_paragraph(self, text: Optional[str], *, empty: str = "No information available."):
    self.set_font("Helvetica", "", 10)
    self.set_text_color(*TEXT_COLOR)
    clean = _sanitize(text).strip()
    self.multi_cell(self._page_width, 5.5, clean if clean else empty, new_x="LMARGIN", new_y="NEXT")
    self.ln(1)

def label_value(self, label: str, value: str):
    self.set_font("Helvetica", "B", 9.5)
    self.set_text_color(*MUTED_COLOR)
    label_w = 42
    self.cell(label_w, 5.8, _sanitize(label), new_x="RIGHT", new_y="TOP")
    self.set_font("Helvetica", "", 9.5)
    self.set_text_color(*TEXT_COLOR)
    self.multi_cell(
        self._page_width - label_w,
        5.8,
        _sanitize(value) if value else "-",
        new_x="LMARGIN",
        new_y="NEXT",
    )

def item_card(self, title: str, fields: Iterable[tuple[str, str]], body_label: str = None, body_text: str = None):
    """One bordered-ish list entry: a bold title line, label:value pairs, optionally one longer body paragraph, then a divider."""
    if self.get_y() > self.h - 30:
        self.add_page()
    self.set_font("Helvetica", "B", 10)
    self.set_text_color(*TEXT_COLOR)
    self.multi_cell(self._page_width, 5.5, _sanitize(title), new_x="LMARGIN", new_y="NEXT")
    self.set_font("Helvetica", "", 9)
    for label, value in fields:
        self.set_text_color(*MUTED_COLOR)
        self.write(5, f"{_sanitize(label)}: ")
        self.set_text_color(*TEXT_COLOR)
        self.write(5, f"{_sanitize(value)} ")
    self.ln(6)
    if body_label:
        self.set_font("Helvetica", "B", 8.5)
        self.set_text_color(*MUTED_COLOR)
        self.cell(self._page_width, 4.5, _sanitize(body_label), new_x="LMARGIN", new_y="NEXT")
    if body_text:
        self.set_font("Helvetica", "", 9)
        self.set_text_color(*TEXT_COLOR)
        self.multi_cell(self._page_width, 5, _sanitize(body_text), new_x="LMARGIN", new_y="NEXT")
    self.ln(1.5)
    self.set_draw_color(*BORDER_COLOR)
    self.line(self.l_margin, self.get_y(), self.w - self.r_margin, self.get_y())
    self.ln(3)

def table(self, columns: list[tuple[str, float]], rows: list[list[str]], *, empty: str = "No records."):
    """A simple bordered table: 'columns' is [(header, width fraction), ...] with widths as fractions of the page width (so callers never have to hand-compute mm). Row cells wrap onto multiple lines when needed (multi_cell), and every row in the same table is drawn at the same height as its tallest cell so nothing overlaps - the "avoid broken tables and overflow" requirement (spec section 8)."""
    if self.get_y() > self.h - 40:
        self.add_page()
    widths = [self._page_width * frac for _, frac in columns]
    line_h = 5

    def _row_height(cells: list[str]) -> float:
        max_lines = 1
        for cell_text, w in zip(cells, widths):
            text = _sanitize(cell_text) or "-"
            # fpdf2 doesn't expose a line-count helper directly, so
            # this counts characters-per-line via get_string_width -
            # good enough for a report table (never pixel-perfect
            # typesetting), and always at least 1 line.
            words = text.split(" ")
            lines = 1
            current = ""
            for word in words:
                trial = f"{current} {word}".strip()
                if self.get_string_width(trial) > w - 2 and current:
                    lines += 1
                    current = word
                else:
                    current = trial
            max_lines = max(max_lines, lines)
        return max_lines * line_h + 2

    # Header row
    self.set_font("Helvetica", "B", 8.5)
    self.set_fill_color(243, 244, 246)
    self.set_text_color(*TEXT_COLOR)
    self.set_draw_color(*BORDER_COLOR)
    y0 = self.get_y()
    x0 = self.l_margin
    for (label, _), w in zip(columns, widths):
        self.set_xy(x0, y0)
        self.cell(w, 7, _sanitize(label), border=1, fill=True, align="L")
        x0 += w
    self.set_xy(self.l_margin, y0 + 7)

    if not rows:
        self.set_font("Helvetica", "", 9)
        self.set_text_color(*MUTED_COLOR)

```

```

        self.cell(sum(widths), 8, _sanitize(empty), border=1, new_x="LMARGIN", new_y="NEXT")
        self.ln(2)
        return

    self.set_font("Helvetica", "", 8.5)
    for row_cells in rows:
        if self.get_y() > self.h - 25:
            self.add_page()
            y0 = self.get_y()
            x0 = self.l_margin
            self.set_font("Helvetica", "B", 8.5)
            self.set_fill_color(243, 244, 246)
            for (label, _) in zip(columns, widths):
                self.set_xy(x0, y0)
                self.cell(w, 7, _sanitize(label), border=1, fill=True, align="L")
                x0 += w
            self.set_xy(self.l_margin, y0 + 7)
            self.set_font("Helvetica", "", 8.5)

            row_h = row_height(row_cells)
            y0 = self.get_y()
            x0 = self.l_margin
            self.set_text_color(*TEXT_COLOR)
            for cell_text, w in zip(row_cells, widths):
                self.set_xy(x0, y0)
                self.multi_cell(w, line_h, _sanitize(cell_text) if cell_text else "-", border=1)
                x0 += w
            self.set_xy(self.l_margin, y0 + row_h)
        self.ln(3)

def _parse_security_analysis(raw: Optional[str]) -> tuple[str, list[str]]:
    if not raw:
        return "", []
    try:
        data = json.loads(raw)
    except (TypeError, ValueError):
        return "", []
    return data.get("summary") or "", list(data.get("concerns") or [])

def generate_report_pdf(
    change_request: ChangeRequest, context: ReportContext, *, generated_by: Optional[str] = None
) -> bytes:
    """Builds the full PDF and returns its raw bytes, entirely from
    `context` (see resolve_report_context) plus a small number of purely
    structural, never-version-specific facts read straight off
    `change_request` (its id, its creator, and its overall creation date -
    none of which a past version could ever have differed on). Every
    version-sensitive value - title, description, priority, and whether an
    analysis exists at all - comes from `context`, never from
    `change_request` directly, so a historical report can never
    accidentally show today's data.

    `generated_by` is the name of whoever clicked "Download Report" - it
    has no bearing on what's IN the report, it's just printed on the
    metadata line so two people looking at two PDFs of the same change
    request can tell them apart."""
    try:
        fields = context.fields
        analysis = context.analysis
        cr_code = f"CR-{change_request.id:04d}"
        pdf = ReportPDF(cr_code, fields.title)
        pdf.add_page()
        pw = pdf.page_width # noqa: SLF001 - internal helper, same module

        # --- Cover -----
        pdf.ln(18)
        pdf.set_font("Helvetica", "B", 21)
        pdf.set_text_color(*ACCENT_COLOR)
        pdf.multi_cell(pw, 10, "Change Request Analysis Report", align="C", new_x="LMARGIN", new_y="NEXT")
        pdf.ln(2)
        pdf.set_font("Helvetica", "B", 14)
        pdf.set_text_color(*TEXT_COLOR)
        pdf.multi_cell(pw, 8, f"[cr_code] - {_sanitize(fields.title)}", align="C", new_x="LMARGIN", new_y="NEXT")
        pdf.ln(1)
        if context.is_historical:
            pdf.set_font("Helvetica", "B", 11)
            pdf.set_text_color(*HISTORICAL_COLOR)
            pdf.multi_cell(
                pw, 6, f"HISTORICAL REPORT - Version {context.report_version}", align="C", new_x="LMARGIN", new_y="NEXT"
            )
        pdf.ln(2)
        pdf.set_font("Helvetica", "", 10)
        pdf.set_text_color(*MUTED_COLOR)
        pdf.multi_cell(
            pw,
            6,
            f"Generated {datetime.utcnow().strftime('%B %d, %Y')} - for engineering manager / CAB review",
            align="C",
            new_x="LMARGIN",
            new_y="NEXT",
        )
        pdf.ln(6)

        # --- Report metadata (Module 13 Phase 3, made version-safe by
        # Module 20) -----
        # Spec section 3: never generate a misleadingly "current" report.
        # By the time this function is called, the API layer has already
        # confirmed either (a) this IS the current version and it DOES
        # have a matching analysis, or (b) this is a deliberately-
        # requested historical report - so the only two states this PDF
        # ever has to render are "current and consistent" or "historical
        # and clearly labeled," never "stale but pretending to be current."
        cr_version = change_request.current_version or 1
        report_version = context.report_version

        pdf.set_font("Helvetica", "", 9)
        pdf.set_text_color(*MUTED_COLOR)
        meta_bits = [f"Change Request Version {report_version}"]
        if context.is_historical:
            meta_bits.append(f"Current Version is {cr_version}")
        meta_bits.append(
            f"Analysis Version {analysis.change_request_version}" if analysis is not None else "No analysis recorded at this version"
        )
        meta_bits.append(f"Generated {datetime.utcnow().strftime('%B %d, %Y %H:%M')} UTC")
        if generated_by:
            meta_bits.append(f"Generated by {_sanitize(generated_by)}")
        pdf.multi_cell(pw, 5, " | ".join(meta_bits), align="C", new_x="LMARGIN", new_y="NEXT")
        pdf.ln(3)

        if context.is_historical:
            pdf.set_fill_color(*HISTORICAL_FILL)
            pdf.set_draw_color(*HISTORICAL_COLOR)
            pdf.set_text_color(*HISTORICAL_COLOR)
            pdf.set_line_width(0.3)
            pdf.set_font("Helvetica", "B", 9.5)
            note = (
                f"This report represents Version {report_version} of this change request, not its current "
                f"Version {cr_version}. The Change Request fields below reflect Version {report_version} "
                "exactly as it stood then; the analysis (if shown) is the one that ran against that same "
            )

```



```

        "version - never a newer or older one."
    )
    pdf.multi_cell(pw, 6, _sanitize(note), align="C", border=1, fill=True, new_x="LMARGIN", new_y="NEXT")
    pdf.set_line_width(0.2)
    pdf.set_text_color(*TEXT_COLOR)
    pdf.ln(4)
else:
    pdf.ln(4)

no_analysis_note = (
    f"No analysis was performed against Version {report_version} of this change request - "
    "the sections below that depend on AI analysis are not available for this version."
)

# --- 1. Executive Summary -----
pdf.section_heading(1, "Executive Summary")
if analysis is not None:
    pdf.body_paragraph(analysis.summary)
    pdf.label_value("Risk Score", f"{analysis.risk_score / 10:.1f} / 10")
    pdf.label_value("Complexity", title_case(_enum_value(analysis.complexity)))
    pdf.label_value("AI Confidence", f"{round(analysis.confidence_score)}%")
    pdf.label_value("Recommendation", title_case(_enum_value(analysis.recommendation)))
else:
    pdf.body_paragraph(None, empty=no_analysis_note)

# --- 2. Change Request -----
pdf.section_heading(2, "Change Request")
pdf.label_value("Title", fields.title)
# Module 23: previously the only way to see this change request is
# Closed was to read through the Audit table at the end of the
# report - a one-line headline here means the reader doesn't have
# to. Always the change request's real, current status (not
# whatever it was at a historical report's own version), which is
# exactly why this line lives outside the historical-vs-current
# fields above.
pdf.label_value("Current Status", STATUS_LABELS.get(change_request.status, change_request.status.value))
pdf.label_value("Priority", fields.priority_label)
pdf.label_value("Requested By", fields.requested_by or "Not provided")
pdf.label_value("Target System", fields.target_system or "Not provided")
pdf.label_value("Desired Deadline", _fmt_date(fields.desired_deadline))
pdf.label_value("Environment", fields.environment or "Not provided")
pdf.label_value("Created", _fmt_date(change_request.created_at))
pdf.ln(1)
pdf.set_font("Helvetica", "B", 9.5)
pdf.set_text_color(*MUTED_COLOR)
pdf.cell(pw, 5.5, "Description", new_x="LMARGIN", new_y="NEXT")
pdf.body_paragraph(fields.description)

# --- 3. Business Objective -----
pdf.section_heading(3, "Business Objective")
pdf.body_paragraph(fields.business_objective, empty="No business objective was provided.")

# --- 4. Business Impact -----
pdf.section_heading(4, "Business Impact")
pdf.body_paragraph(fields.business_impact, empty="No business impact was recorded for this change.")
if fields.customer_impact:
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)
    pdf.cell(pw, 5.5, "Customer Impact", new_x="LMARGIN", new_y="NEXT")
    pdf.body_paragraph(fields.customer_impact)

# --- 5. Classification -----
pdf.section_heading(5, "Classification")
if analysis is not None:
    pdf.label_value("Category", analysis.category or "-")
    pdf.label_value("Confidence", f"{round(analysis.confidence_score)}%")
    pdf.ln(1)
    pdf.body_paragraph(analysis.classification_reason, empty="No classification reasoning was provided.")
else:
    pdf.body_paragraph(None, empty=no_analysis_note)

# --- 6. Requirements -----
pdf.section_heading(6, "Requirements")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.requirements:
    pdf.body_paragraph(None, empty="No requirements were extracted for this change.")
else:
    for req in analysis.requirements:
        pdf.item_card(
            req.description,
            [
                ("Type", title_case(req.requirement_type.value if req.requirement_type else None)),
                ("Priority", title_case(_enum_value(req.priority))),
            ],
        )

# --- 7. Technical Impact -----
pdf.section_heading(7, "Technical Impact")
pdf.body_paragraph(fields.technical_impact, empty="No technical impact was recorded for this change.")
if analysis is not None and analysis.complexity_reasoning:
    pdf.ln(1)
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)
    pdf.cell(pw, 5.5, "AI Technical Impact Reasoning", new_x="LMARGIN", new_y="NEXT")
    pdf.body_paragraph(analysis.complexity_reasoning)
if analysis is not None and analysis.affected_components:
    names = ", ".join(
        f"{c.component_name} ({title_case(c.component_type.value if c.component_type else None)})"
        for c in analysis.affected_components
    )
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)
    pdf.cell(28, 5.5, "Affects:", new_x="RIGHT", new_y="TOP")
    pdf.set_font("Helvetica", "", 9.5)
    pdf.set_text_color(*TEXT_COLOR)
    pdf.multi_cell(pw - 28, 5.5, _sanitize(names), new_x="LMARGIN", new_y="NEXT")

# --- 8. Affected Components -----
pdf.section_heading(8, "Affected Components")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.affected_components:
    pdf.body_paragraph(None, empty="No affected components were identified.")
else:
    for comp in analysis.affected_components:
        pdf.item_card(
            f"{comp.component_name} ({title_case(comp.component_type.value if comp.component_type else None)})",
            [
                ("Impact", title_case(_enum_value(comp.impact_level))),
                ("Confidence", f"{round(comp.confidence)}%"),
            ],
            body_label="Reason",
            body_text=comp.reason,
        )

# --- 9. Dependencies -----
pdf.section_heading(9, "Dependencies")
if fields.dependencies_note:
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)

```

```

pdf.cell(pw, 5.5, "Noted by the requester/owner", new_x="LMARGIN", new_y="NEXT")
pdf.body_paragraph(fields.dependencies_note)
pdf.ln(1)
if analysis is None:
    if not fields.dependencies_note:
        pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.dependencies:
    pdf.body_paragraph(None, empty="No dependencies were identified by AI analysis.")
else:
    for dep in analysis.dependencies:
        pdf.item_card(
            f"{dep.dependency_name} ({_title_case(dep.dependency_type.value if dep.dependency_type else None)})",
            [{"Impact", _title_case(_enum_value(dep.impact_level))}],
            body_label="Reason",
            body_text=dep.reason,
        )

# --- 10. Security Assessment -----
pdf.section_heading(10, "Security")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
else:
    summary, concerns = _parse_security_analysis(analysis.security_analysis)
    security_risks = [r for r in analysis.risks if _enum_value(r.category) == "security"]
    if security_risks:
        worst = min(
            security_risks,
            key=lambda r: {"critical": 0, "high": 1, "medium": 2, "low": 3}.get(_enum_value(r.severity), 9),
        )
        pdf.label_value("Security Risk Level", _title_case(_enum_value(worst.severity)))
    else:
        pdf.body_paragraph(
            None, empty="The AI didn't score a specific security risk level for this change."
        )
    if summary:
        pdf.ln(1)
        pdf.body_paragraph(summary)
        pdf.set_font("Helvetica", "B", 9.5)
        pdf.set_text_color(*MUTED_COLOR)
        pdf.cell(pw, 5.5, "Potential Concerns", new_x="LMARGIN", new_y="NEXT")
    if concerns:
        for concern in concerns:
            pdf.set_font("Helvetica", "", 9.5)
            pdf.set_text_color(*TEXT_COLOR)
            pdf.multi_cell(pw, 5.2, f"- {_sanitize(concern)}", new_x="LMARGIN", new_y="NEXT")
    else:
        pdf.body_paragraph(None, empty="No specific concerns were flagged.")
    mitigations = [r.mitigation for r in security_risks if r.mitigation]
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)
    pdf.cell(pw, 5.5, "Recommended Mitigations", new_x="LMARGIN", new_y="NEXT")
    if mitigations:
        for mitigation in mitigations:
            pdf.set_font("Helvetica", "", 9.5)
            pdf.set_text_color(*TEXT_COLOR)
            pdf.multi_cell(pw, 5.2, f"- {_sanitize(mitigation)}", new_x="LMARGIN", new_y="NEXT")
    else:
        pdf.body_paragraph(None, empty="No specific mitigations beyond the concerns above were provided.")

# --- 11. Risk -----
pdf.section_heading(11, "Risk")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
else:
    risk_bucket = _risk_bucket_label(analysis.risk_score)
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*MUTED_COLOR)
    pdf.cell(42, 5.8, "Overall Risk Score", new_x="RIGHT", new_y="TOP")
    pdf.set_font("Helvetica", "B", 9.5)
    pdf.set_text_color(*RISK_COLORS.get(risk_bucket.lower(), TEXT_COLOR))
    pdf.cell(
        pw - 42,
        5.8,
        _sanitize(f"analysis.risk_score / 10: .1f} / 10 ({risk_bucket}")),
        new_x="LMARGIN",
        new_y="NEXT",
    )
    pdf.set_text_color(*TEXT_COLOR)
    pdf.ln(1)
    if not analysis.risks:
        pdf.body_paragraph(None, empty="No risks were identified for this change.")
    for risk in analysis.risks:
        pdf.item_card(
            f"{_title_case(_enum_value(risk.category))} risk",
            [
                ("Severity", _title_case(_enum_value(risk.severity))),
                ("Probability", f"{round(risk.probability * 100)}%"),
                ("Score", f"{round(risk.score)/100}"),
            ],
            body_label="Explanation",
            body_text=risk.description,
        )
        if risk.mitigation:
            pdf.set_font("Helvetica", "B", 8.5)
            pdf.set_text_color(*MUTED_COLOR)
            pdf.cell(pw, 4.5, "Mitigation", new_x="LMARGIN", new_y="NEXT")
            pdf.set_font("Helvetica", "", 9)
            pdf.set_text_color(*TEXT_COLOR)
            pdf.multi_cell(pw, 5, _sanitize(risk.mitigation), new_x="LMARGIN", new_y="NEXT")
            pdf.ln(2)

# --- 12. Complexity -----
pdf.section_heading(12, "Complexity")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
else:
    pdf.label_value("Level", _title_case(_enum_value(analysis.complexity)))
    pdf.ln(1)
    pdf.body_paragraph(analysis.complexity_reasoning, empty="No complexity reasoning was provided.")

# --- 13. Effort Estimate -----
pdf.section_heading(13, "Effort")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
else:
    pdf.body_paragraph(analysis.effort_estimate, empty="No effort estimate was provided.")

# --- 14. Missing Information (clarification questions) -----
pdf.section_heading(14, "Missing Information")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.clarification_questions:
    pdf.body_paragraph(None, empty="No open clarification items - nothing blocking this change.")
else:
    for question in analysis.clarification_questions:
        pdf.item_card(
            question.question,
            [
                ("Priority", _title_case(_enum_value(question.priority))),
                ("Status", "Resolved" if question.resolved else "Open"),
            ],
        )

```

```

        body_label="Why it matters",
        body_text=question.reason,
    )

# --- 15. Test Strategy -----
pdf.section_heading(15, "Test Strategy")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.test_cases:
    pdf.body_paragraph(None, empty="No test cases were suggested.")
else:
    for test in analysis.test_cases:
        pdf.item_card(
            f"{test.test_id} - {test.title}",
            [
                ("Type", _title_case(test.test_type.value if test.test_type else None)),
                ("Priority", _title_case(_enum_value(test.priority))),
            ],
            body_label="Description",
            body_text=test.description,
        )
        if test.expected_result:
            pdf.set_font("Helvetica", "B", 8.5)
            pdf.set_text_color(*MUTED_COLOR)
            pdf.cell(pw, 4.5, "Expected Result", new_x="LMARGIN", new_y="NEXT")
            pdf.set_font("Helvetica", "", 9)
            pdf.set_text_color(*TEXT_COLOR)
            pdf.multi_cell(pw, 5, _sanitize(test.expected_result), new_x="LMARGIN", new_y="NEXT")
            pdf.ln(2)

# --- 16. Implementation Plan -----
pdf.section_heading(16, "Implementation Plan")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
elif not analysis.implementation_tasks:
    pdf.body_paragraph(None, empty="No implementation plan was suggested.")
else:
    for index, task in enumerate(analysis.implementation_tasks, start=1):
        fields_row = [
            ("Component", task.component or "Unspecified"),
            ("Priority", _title_case(_enum_value(task.priority))),
            ("Estimated Effort", task.estimated_effort or "Insufficient information."),
        ]
        pdf.item_card(f"Step {index}: {task.task}", fields_row, body_label="Description", body_text=task.description)
        if task.dependencies:
            pdf.set_font("Helvetica", "B", 8.5)
            pdf.set_text_color(*MUTED_COLOR)
            pdf.cell(pw, 4.5, "Depends On", new_x="LMARGIN", new_y="NEXT")
            pdf.set_font("Helvetica", "", 9)
            pdf.set_text_color(*TEXT_COLOR)
            pdf.multi_cell(pw, 5, _sanitize(task.dependencies), new_x="LMARGIN", new_y="NEXT")
            pdf.ln(2)

# --- 17. Approval Status (Module 20) -----
pdf.section_heading(17, "Approval Status")
_render_approvals_section(pdf, pw, context.approvals)

# --- 18. Change History (Module 20) -----
pdf.section_heading(18, "Change History")
_render_version_history_section(pdf, pw, context.versions, report_version, cr_version)

# --- 19. Audit (Module 20) -----
pdf.section_heading(19, "Audit")
_render_audit_section(pdf, pw, context.history_events)

# --- 20. Recommendation -----
pdf.section_heading(20, "Recommendation")
if analysis is None:
    pdf.body_paragraph(None, empty=no_analysis_note)
else:
    pdf.label_value("Recommendation", _title_case(_enum_value(analysis.recommendation)))
    pdf.ln(1)
    pdf.body_paragraph(analysis.recommendation_reasoning, empty="No recommendation reasoning was provided.")

raw = pdf.output()
# fpdf2's output() has returned a plain str (classic PyFPDF, needing
# a latin-1 encode) in some versions and a bytearray in others -
# handle both rather than assuming one.
if isinstance(raw, (bytes, bytearray)):
    return bytes(raw)
return raw.encode("latin-1")
except Exception as exc: # noqa: BLE001 - deliberately broad: any PDF-
    # building failure should surface as a clean, catchable error
    # rather than an unhandled 500 with a raw fpdf2 traceback.
    raise ReportGenerationError(f"Could not build the PDF report: {exc}") from exc

def risk_bucket_label(risk_score: Optional[float]) -> str:
    """Same low/medium/high/critical cutoffs the rest of the app already
    uses (see app/api/dashboard.py / app/services/analytics.py) - kept as
    its own tiny copy here rather than a cross-import, matching this
    project's established api/ vs services/ duplication convention."""
    if risk_score is None:
        return "Not Analyzed"
    if risk_score >= 75:
        return "Critical"
    if risk_score >= 50:
        return "High"
    if risk_score >= 25:
        return "Medium"
    return "Low"

def _render_approvals_section(pdf: _ReportPDF, pw: float, approvals: list[Approval]) -> None:
    """Spec section 4: required/completed/pending/rejected counts, plus a
    full table with timestamps and approver comments. Approver identity is
    shown by name only - never an email address or other account detail,
    matching the same "don't expose unnecessary sensitive information"
    rule Module 19's Approval Bottlenecks section already follows."""
    if not approvals:
        pdf.body_paragraph(None, empty="No approvals have been requested for this change request.")
        return

    completed_statuses = {ApprovalStatus.APPROVED, ApprovalStatus.REJECTED, ApprovalStatus.CHANGES_REQUESTED}
    total = len(approvals)
    completed = sum(1 for a in approvals if a.status in completed_statuses)
    pending = sum(1 for a in approvals if a.status == ApprovalStatus.PENDING)
    rejected = sum(1 for a in approvals if a.status == ApprovalStatus.REJECTED)
    approved = sum(1 for a in approvals if a.status == ApprovalStatus.APPROVED)
    changes_requested = sum(1 for a in approvals if a.status == ApprovalStatus.CHANGES_REQUESTED)

    pdf.label_value("Required Approvals", str(total))
    pdf.label_value("Completed", f"{completed} ({approved} approved, {rejected} rejected, {changes_requested} changes requested)")
    pdf.label_value("Pending", str(pending))
    pdf.ln(2)

    rows = []
    for approval in approvals:
        approver_name = approval.approver.name if approval.approver else "Unknown"
        type_label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        status_label = APPROVAL_STATUS_LABELS.get(approval.status, approval.status.value)

```

```

        rows.append(
            [
                type_label,
                approver_name,
                status_label,
                _fmt_datetime(approval.requested_at),
                _fmt_datetime(approval.responded_at) if approval.responded_at else "-",
                approval.comment or "-",
            ]
        )
    pdf.table(
        [
            ("Type", 0.14),
            ("Approver", 0.16),
            ("Status", 0.14),
            ("Requested", 0.18),
            ("Responded", 0.18),
            ("Comment", 0.20),
        ],
        rows,
    )

def _render_version_history_section(
    pdf: ReportPDF, pw: float, versions: list[ChangeRequestVersion], report_version: int, current_version: int
) -> None:
    """Spec section 5: version/date/changed-by/summary, current version
    clearly marked. 'report version' (not just 'current version') is also
    flagged, since on a historical report those two can differ."""
    if not versions:
        pdf.body_paragraph(None, empty="No version history recorded.")
        return

    rows = []
    for version_row in versions:
        tags = []
        if version_row.version_number == current_version:
            tags.append("CURRENT")
        if version_row.version_number == report_version and report_version != current_version:
            tags.append("THIS REPORT")
        version_label = f"Version {version_row.version_number}" + (f" ({', '.join(tags)})" if tags else "")
        changed_by_name = version_row.changed_by_user.name if version_row.changed_by_user else "Unknown"
        rows.append(
            [
                version_label,
                _fmt_date(version_row.created_at),
                changed_by_name,
                version_row.change_summary,
            ]
        )
    pdf.table(
        [
            ("Version", 0.18),
            ("Date", 0.16),
            ("Changed By", 0.18),
            ("Summary", 0.48),
        ],
        rows,
    )

# History actions rendered on the report's Audit section, with a short
# human-readable label - deliberately a curated subset of HistoryAction
# covering spec section 6's own list (created / edited / status changes /
# approval decisions / analysis / implementation) rather than every single
# action type this app tracks internally (e.g. MENTIONED, REMINDER SENT
# are operational noise for a CAB-facing PDF, not workflow milestones).
_AUDIT_ACTION_LABELS: dict[HistoryAction, str] = {
    HistoryAction.CREATED: "Change request created",
    HistoryAction.FIELD_CHANGED: "Field edited",
    HistoryAction.VERSION_CREATED: "New version created",
    HistoryAction.STATUS_CHANGED: "Status changed",
    HistoryAction.ASSIGNED: "Person assigned",
    HistoryAction.UNASSIGNED: "Person unassigned",
    HistoryAction.APPROVAL_REQUESTED: "Approval requested",
    HistoryAction.APPROVED: "Approved",
    HistoryAction.REJECTED: "Rejected",
    HistoryAction.CHANGES_REQUESTED: "Changes requested",
    HistoryAction.APPROVAL_INVALIDATED: "Approval invalidated by later edit",
    HistoryAction.APPROVAL_CANCELLED: "Approval request cancelled",
    HistoryAction.AI_ANALYSIS_COMPLETED: "AI analysis completed",
    HistoryAction.AI_ANALYSIS_INVALIDATED: "AI analysis marked outdated",
    HistoryAction.REPORT_GENERATED: "Report generated",
}

def _render_audit_section(pdf: ReportPDF, pw: float, history_events: list[ChangeRequestHistory]) -> None:
    """Spec section 6: created / edited / status changes / approval
    decisions / analysis / implementation. Filtered to the curated
    milestone list above - not every internal event type - and actor
    identity is shown by name only (never an email), same rule as the
    Approval Status section above."""
    relevant = [h for h in history_events if h.action in _AUDIT_ACTION_LABELS]
    if not relevant:
        pdf.body_paragraph(None, empty="No audit events recorded.")
        return

    rows = []
    for event in relevant:
        actor = event.user.name if event.user else (event.actor_label or "System")
        label = _AUDIT_ACTION_LABELS.get(event.action, _title_case(event.action.value))
        detail_bits = []
        if event.field_name:
            detail_bits.append(_FIELD_LABELS.get(event.field_name, event.field_name.replace("_", " ").title()))
        if event.new_value:
            detail_bits.append(event.new_value if len(event.new_value) < 120 else event.new_value[:117] + "...")
        detail = " - ".join(detail_bits) if detail_bits else "-"
        rows.append([label, actor, _fmt_datetime(event.created_at), detail])
    pdf.table(
        [
            ("Event", 0.20),
            ("By", 0.16),
            ("When", 0.20),
            ("Detail", 0.44),
        ],
        rows,
    )

```

```

"""Module 24 (Reports): a read-only aggregation layer over data that
already exists elsewhere in this app - no new database table, and no
manual "generate" step either.

A REPORT_GENERATED ChangeRequestHistory event, with its new value
formatted as "Version {N}" or "Version {N} (historical)", is recorded
automatically in exactly two places: the instant an analysis completes for
a change request's current version (app/api/change_requests.py::
analyze_change_request), and the long-standing GET /api/change-requests/
{id}/report endpoint the Analysis Dashboard's "Download Report" button
already used (Module 11/20). Reports are deliberately never generated as a
separate action a person takes (confirmed with the project owner: "the
reports should not be generated separately, once the analysis is complete,
it should reflect in the reports section") - this module only ever reads
that event stream. One (change_request_id, version) pair that has at least
one REPORT_GENERATED event against it is one "report" as far as the
Reports page is concerned. When the same pair has more than one event
(analyzed more than once, downloaded more than once, or both), the most
recent one's actor and timestamp are what the Reports page shows for that
row - every individual event is still visible, in full, on that change
request's own Activity tab; nothing is hidden or discarded, there is just
one row per report on the Reports page rather than one per event.

This is a deliberate architecture choice (confirmed with the project owner
before implementing): it keeps the Module 24 spec's own instruction -
"do not duplicate all of these databases just to create reports" - literally
true. There is no second store anywhere of "which reports exist"; this
module only ever reads ChangeRequestHistory, ChangeRequest, and Analysis,
exactly the tables that already existed.
"""
from __future__ import annotations

import re
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from typing import Optional

from sqlalchemy.orm import Session, selectinload

from app.models.analysis import Analysis
from app.models.change_request import ChangeRequest
from app.models.change_request_history import ChangeRequestHistory
from app.models.enums import ChangeRequestStatus, HistoryAction
from app.services.workflow_rules import STATUS_LABELS, risk_bucket

# Matches the new_value format both REPORT_GENERATED sources write:
# "Version 3" or "Version 3 (historical)".
_VERSION_RE = re.compile(r"Version (\d+)")

@dataclass
class ReportRow:
    """One row of the Reports page - a computed view, never its own
    database row. 'report_id' is the id of the most recent
    ChangeRequestHistory REPORT_GENERATED event for this (change request,
    version) pair, used as this "report"'s identity for GET/download."""
    report_id: int
    change_request_id: int
    change_request_code: str
    change_request_title: str
    version: int
    is_current_version: bool
    is_historical_pull: bool
    cr_status: ChangeRequestStatus
    cr_status_label: str
    analysis_version: Optional[int]
    has_analysis: bool
    risk_score: Optional[float]
    risk_bucket_label: Optional[str]
    generated_by_id: Optional[int]
    generated_by_name: str
    generated_at: datetime
    pull_count: int

@dataclass
class ReportStats:
    total_reports: int
    change_requests_with_reports: int
    recent_reports: int
    outdated_reports: int

def _parse_report_version(event: ChangeRequestHistory) -> Optional[int]:
    if not event.new_value:
        return None
    match = _VERSION_RE.search(event.new_value)
    return int(match.group(1)) if match else None

def _all_report_events(db: Session) -> list[ChangeRequestHistory]:
    return (
        db.query(ChangeRequestHistory)
        .options(selectinload(ChangeRequestHistory.user))
        .filter(ChangeRequestHistory.action == HistoryAction.REPORT_GENERATED)
        .order_by(ChangeRequestHistory.created_at.asc())
        .all()
    )

def _build_rows(db: Session) -> list[ReportRow]:
    events = _all_report_events(db)
    if not events:
        return []

    grouped: dict[tuple[int, int], list[ChangeRequestHistory]] = {}
    for event in events:
        version = _parse_report_version(event)
        if version is None:
            # A report pulled before this module existed (or anything
            # otherwise unparseable) - fall back to the CR's version at the
            # time of the event rather than guessing or dropping it.
            version = event.version number or 1
        grouped.setdefault((event.change_request_id, version), []).append(event)

    cr_ids = {cr_id for cr_id, _ in grouped}
    change_requests = {
        cr_id: cr for cr_id, cr in db.query(ChangeRequest).filter(ChangeRequest.id.in_(cr_ids)).all()
    }

    analyses = db.query(Analysis).filter(Analysis.change_request_id.in_(cr_ids)).all()
    analysis_index: dict[tuple[int, int], Analysis] = {}
    for analysis in analyses:

```

```

        key = (analysis.change_request_id, analysis.change_request_version or 1)
        existing = analysis_index.get(key)
        if existing is None or analysis.created_at > existing.created_at:
            analysis_index[key] = analysis

rows: List[ReportRow] = []
for (cr_id, version), pulls in grouped.items():
    change_request = change_requests.get(cr_id)
    if change_request is None:
        # Defensive only - this app never deletes change requests, so
        # every event's change_request_id always resolves to a real row.
        continue
    latest = max(pulls, key=lambda e: e.created_at)
    analysis = analysis_index.get((cr_id, version))
    current_version = change_request.current_version or 1
    bucket = risk_bucket(analysis.risk_score) if analysis is not None else None
    rows.append(
        ReportRow(
            report_id=latest.id,
            change_request_id=cr_id,
            change_request_code=f"CR-{cr_id:04d}",
            change_request_title=change_request.title,
            version=version,
            is_current_version=version == current_version,
            is_historical_pull="(historical)" in (latest.new_value or ""),
            cr_status=change_request.status,
            cr_status_label=STATUS_LABELS.get(change_request.status, change_request.status.value),
            analysis_version=analysis.change_request_version if analysis is not None else None,
            has_analysis=analysis is not None,
            risk_score=analysis.risk_score if analysis is not None else None,
            risk_bucket_label=bucket.capitalize() if bucket else None,
            generated_by_id=latest.user_id,
            generated_by_name=latest.user.name if latest.user else (latest.actor_label or "System"),
            generated_at=latest.created_at,
            pull_count=len(pulls),
        )
    )

rows.sort(key=lambda r: r.generated_at, reverse=True)
return rows

def list_reports(
    db: Session,
    *,
    search: Optional[str] = None,
    status: Optional[ChangeRequestStatus] = None,
    risk: Optional[str] = None,
    version: Optional[int] = None,
    generated_by: Optional[int] = None,
    date_from: Optional[date] = None,
    date_to: Optional[date] = None,
) -> List[ReportRow]:
    """The Reports page's list/search/filter, all applied in Python over
    the already-small result of _build_rows() - a brute-force approach
    that's genuinely fine at this project's scale (see
    app/services/knowledge_embeddings.py's own search_chunks() for the same
    reasoning elsewhere in this codebase)."""
    rows = _build_rows(db)

    if search:
        needle = search.strip().lower()
        if needle:
            rows = [
                r
                for r in rows
                if needle in r.change_request_title.lower()
                or needle in r.change_request_code.lower()
                or needle in str(r.change_request_id)
            ]

    if status is not None:
        rows = [r for r in rows if r.cr_status == status]
    if risk is not None:
        rows = [r for r in rows if (r.risk_bucket_label or "").lower() == risk.lower()]
    if version is not None:
        rows = [r for r in rows if r.version == version]
    if generated_by is not None:
        rows = [r for r in rows if r.generated_by_id == generated_by]
    if date_from is not None:
        rows = [r for r in rows if r.generated_at.date() >= date_from]
    if date_to is not None:
        rows = [r for r in rows if r.generated_at.date() <= date_to]

    return rows

def list_reports_for_change_request(db: Session, change_request_id: int) -> List[ReportRow]:
    return [r for r in _build_rows(db) if r.change_request_id == change_request_id]

def get_report(db: Session, report_id: int) -> Optional[ReportRow]:
    for row in _build_rows(db):
        if row.report_id == report_id:
            return row
    return None

def compute_stats(db: Session) -> ReportStats:
    rows = _build_rows(db)
    recent_cutoff = datetime.utcnow() - timedelta(days=7)
    return ReportStats(
        total_reports=len(rows),
        change_requests_with_reports=len({r.change_request_id for r in rows}),
        recent_reports=sum(1 for r in rows if r.generated_at >= recent_cutoff),
        outdated_reports=sum(1 for r in rows if not r.is_current_version),
    )

```

```

"""Module 15 Phase 5 (Repository Intelligence - Integration).

The spec's own instruction for this phase: repository findings should
"feed" the existing Impact Analysis / Dependencies / Risk / Implementation
Plan / Test Cases sections - and explicitly "do not create duplicate
impact systems." Combined with the POST-MODULE-12 architecture lock's own
priority order (data integrity > version correctness > permissions >
auditability > workflow correctness > AI correctness > UI polish), the
safest reading of "feed" is the most conservative one: never invent a new
table, a new impact model, or a second AI call - only *group* repository
findings that already exist (already vetted by Module 15 Phase 3's
hallucination guard, already carrying their own honest, code-computed
confidence label) underneath whichever existing analysis item they
actually seem to be about.

"Seem to be about" is deliberately dumb and auditable rather than clever:
the exact same kind of keyword-overlap heuristic
app/services/repository_matcher.py already uses to shortlist candidate
files before ever calling the AI (no embeddings, no vector database, no
RAG - per the module's own "do not introduce unnecessary infrastructure"
rule). A small tokenizer is duplicated here rather than importing
repository_matcher's own private `tokenize` - this app's established
convention (see e.g. every app/schemas/*.py file) is that a module's
underscores-prefixed helpers are its own business, and a second, small,
locally-tuned copy is preferred over reaching into another module's
internals.

Computed fresh on every response, never stored anywhere - same rule as
is outdated/workflow recommendation/repository_finding_outdated_reasons
elsewhere in this app. An analysis run before any repository scan/match
existed, or one that shares no real words with anything the matcher found,
ends up with every related files list empty - the same honest "nothing
found" outcome this module treats as valid everywhere else, not an error.
"""
from __future__ import annotations

import re
from typing import Iterable

from app.models.repository_finding import RepositoryFinding

_STOPWORDS = {
    "the", "a", "an", "and", "or", "but", "for", "with", "this", "that", "from", "into",
    "when", "then", "than", "have", "has", "had", "will", "would", "should", "could",
    "can", "not", "are", "was", "were", "been", "being", "its", "their", "them", "they",
    "you", "your", "our", "ours", "also", "each", "any", "all", "some", "other", "more",
    "most", "use", "used", "using", "via", "per", "let", "need", "needs", "needed",
    "add", "adds", "added", "new", "make", "makes", "made", "get", "gets", "set", "sets",
}

_TOKEN_RE = re.compile(r"^[A-Za-z0-9]+")

# Sharing just one word (often a generic one like "user" or "data") is too
# weak a signal to present as "this file relates to this analysis item" -
# two independent shared words is a meaningfully stronger, still-cheap bar.
_MIN_SHARED_TOKENS = 2

def _tokenize(*texts: str | None) -> set[str]:
    tokens: set[str] = set()
    for text in texts:
        if not text:
            continue
        for raw in _TOKEN_RE.split(text):
            word = raw.lower()
            if len(word) < 3 or word in _STOPWORDS:
                continue
            tokens.add(word)
    return tokens

def _finding_tokens(finding: RepositoryFinding) -> set[str]:
    return _tokenize(finding.file_path, finding.reason, finding.evidence)

def related_files(item_text: str, findings: Iterable[RepositoryFinding]) -> list[str]:
    """The file path of every repository finding whose own file
    path/reason/evidence shares at least `MIN_SHARED_TOKENS` meaningful
    words with `item_text`, most-overlapping (then most-confident) first.
    Empty list - the common case - is an honest "nothing in the repository
    scan looks connected to this," never an error."""
    item_tokens = _tokenize(item_text)
    if not item_tokens:
        return []
    scored: list[tuple[RepositoryFinding, int]] = []
    for finding in findings:
        overlap = len(item_tokens & _finding_tokens(finding))
        if overlap >= _MIN_SHARED_TOKENS:
            scored.append((finding, overlap))
    scored.sort(key=lambda pair: (pair[1], pair[0].confidence), reverse=True)
    return [finding.file_path for finding, _overlap in scored]

def annotate_analysis_response(response, findings: list[RepositoryFinding]) -> None:
    """Mutates an already-built AnalysisRead in place, setting every child
    item's `related_files` - mirrors this app's own established "build the
    Pydantic object, then set computed fields on it directly" pattern
    (AnalysisRead.is outdated/workflow recommendation,
    RepositoryFindingRead.is outdated/outdated_reasons). A no-op when
    `findings` is empty (no scan/match has ever run for this change
    request's current analysis yet) - every related_files list simply
    stays at its schema default of [].

    Deliberately does not touch requirements, security findings, or
    clarification questions - the spec names Impact Analysis, Dependencies,
    Risk, Implementation Plan, and Test Cases specifically; affected
    components are included too since "which components does this touch"
    is the most direct analog of this module's own headline question."""
    if not findings:
        return

    for component in response.affected_components:
        component.related_files = related_files(f"{component.component_name} {component.reason}", findings)
    for assessment in response.impact_assessments:
        assessment.related_files = related_files(assessment.description, findings)
    for dependency in response.dependencies:
        dependency.related_files = related_files(f"{dependency.dependency_name} {dependency.reason}", findings)
    for risk in response.risks:
        risk.related_files = related_files(f"{risk.description} {risk.mitigation}", findings)
    for test_case in response.test_cases:
        test_case.related_files = related_files(f"{test_case.title} {test_case.description}", findings)
    for task in response.implementation_tasks:
        task.related_files = related_files(f"{task.task} {task.description}", findings)

```



```

"""Module 15 Phase 3 (Repository Intelligence - CR -> File Matching).

Answers this module's own headline question - "which source files may
actually be affected by this change request?" - in two stages, both kept
deliberately lightweight (no vector database, no embeddings, no RAG
infrastructure, per the module's own spec: "do not introduce unnecessary
infrastructure"):

1. A pure-Python, deterministic keyword-overlap scoring pass shortlists
the most plausibly relevant files out of however many a scan indexed
(see _score_candidates below). This is what keeps the AI call itself
cheap and bounded - a repository with hundreds of indexed files is
never sent to the AI wholesale, only its most plausible ~25.
2. The shortlisted candidates' own extracted structure (Module 15 Phase
2's imports/functions/classes/api_routes/database_references/
config_references - never raw file contents) is handed to the AI
alongside the change request's own text, asking it to name which of
THOSE specific candidates are actually affected, with a reason and
evidence grounded in what was actually extracted.

Mirrors app/services/analysis_engine.py's own shape closely on purpose
(build prompt -> call provider -> parse/validate JSON -> persist), so
this reads as "the same kind of AI call the app already makes," not a new
pattern to learn. The one deliberate difference: a persisted finding's
headline "how sure" label (FileMatchLabel) is computed by code from the
numeric confidence score, never trusted from the AI's own wording - see
that enum's own docstring for why.
"""
from __future__ import annotations

import json
import re
from datetime import datetime
from typing import Optional

from pydantic import ValidationError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models.analysis import Analysis
from app.models.change_request import ChangeRequest
from app.models.enums import FileMatchLabel
from app.models.indexed_file import IndexedFile
from app.models.repository_finding import RepositoryFinding
from app.models.repository_scan import RepositoryScan
from app.schemas.repository_finding import RepositoryMatchResult
from app.services.ai.factory import get_ai_provider

MAX_CANDIDATE_FILES = 25
MAX_SYMBOLS_PER_FIELD = 12

_STOPWORDS = {
    "the", "a", "an", "and", "or", "but", "for", "with", "this", "that", "from", "into",
    "when", "then", "than", "have", "has", "had", "will", "would", "should", "could",
    "can", "not", "are", "was", "were", "been", "being", "its", "their", "them", "they",
    "you", "your", "our", "ours", "also", "each", "any", "all", "some", "other", "more",
    "most", "use", "used", "using", "via", "per", "let", "need", "needs", "needed",
    "add", "adds", "added", "new", "make", "makes", "made", "get", "gets", "set", "sets",
}

_TOKEN_RE = re.compile(r"^[A-Za-z0-9]+")

class RepositoryMatchError(Exception):
    """Raised for any failure running a repository match. `status_code`
    mirrors AnalysisError's own convention (503 not configured, 504
    timeout, 502 other provider failure, 422 bad response shape)."""

    def __init__(self, message: str, status_code: int = 502):
        super().__init__(message)
        self.status_code = status_code

def tokenize(*texts: Optional[str]) -> set[str]:
    tokens: set[str] = set()
    for text in texts:
        if not text:
            continue
        for raw in _TOKEN_RE.split(text):
            word = raw.lower()
            if len(word) < 3 or word in _STOPWORDS:
                continue
            tokens.add(word)
    return tokens

def load_list(column: Optional[str]) -> list[str]:
    if not column:
        return []
    try:
        return json.loads(column)
    except (TypeError, ValueError):
        return []

def file_tokens(indexed_file: IndexedFile) -> set[str]:
    parts = [indexed_file.file_path, indexed_file.language or ""]
    for column in (
        indexed_file.imports, indexed_file.functions, indexed_file.classes,
        indexed_file.api_routes, indexed_file.database_references, indexed_file.config_references,
    ):
        parts.extend(load_list(column))
    return _tokenize(*parts)

def query_tokens(change_request: ChangeRequest, analysis: Analysis) -> set[str]:
    parts = [change_request.title, change_request.description, change_request.business_objective,
        analysis.summary, analysis.category]
    parts.extend(requirement.description for requirement in analysis.requirements)
    return _tokenize(*parts)

def _score_candidates(query_tokens: set[str], indexed_files: list[IndexedFile]) -> list[IndexedFile]:
    """Ranks every indexed file by how many meaningful tokens it shares
    with the change request's own text - a cheap, fully deterministic
    stand-in for semantic search that needs no embeddings/vector store.
    Only files with at least one shared token are candidates at all; a
    change request that shares nothing with anything in the repository
    yields no candidates, and that's a valid, honest outcome (see
    run_repository_match's early return below), not an error."""
    scored = []
    for indexed_file in indexed_files:
        overlap = len(query_tokens & _file_tokens(indexed_file))
        if overlap > 0:

```



```

        scored.append((indexed_file, overlap))
    scored.sort(key=lambda pair: pair[1], reverse=True)
    return [indexed_file for indexed_file, _score in scored[:MAX_CANDIDATE_FILES]]

def _candidate_digest(indexed_file: IndexedFile) -> dict:
    return {
        "file_path": indexed_file.file_path,
        "language": indexed_file.language,
        "imports": load_list(indexed_file.imports)[:MAX_SYMBOLS_PER_FIELD],
        "functions": load_list(indexed_file.functions)[:MAX_SYMBOLS_PER_FIELD],
        "classes": load_list(indexed_file.classes)[:MAX_SYMBOLS_PER_FIELD],
        "api_routes": load_list(indexed_file.api_routes)[:MAX_SYMBOLS_PER_FIELD],
        "database_references": load_list(indexed_file.database_references)[:MAX_SYMBOLS_PER_FIELD],
        "config_references": load_list(indexed_file.config_references)[:MAX_SYMBOLS_PER_FIELD],
    }

SYSTEM_PROMPT = """You are assisting a software change-request analyzer. You will be given a change \
Request's own description/requirements and a shortlist of candidate source files, each with its \
extracted imports/functions/classes/API routes/database and config references - never its raw source \
code. Decide which of these SPECIFIC candidate files are plausibly affected by the change request, \
and respond with ONLY a JSON object of this exact shape:

{"matches": [{"file_path": "<one of the given candidate file path values, EXACTLY as given>", \
"impact_level": "low|medium|high|critical", "confidence": "<0-100>", \
"reason": "<one sentence, grounded in the change request>", \
"evidence": "<the specific import/function/class/route/reference that justifies this, or \
'file name relevance' if the match is only by naming>"}]}

Rules:
- Only include a file if there is a real, checkable connection to the change request: a shared \
concept in its own imports/functions/classes/routes/references, or a strong file-name/path match. \
Never include a file just because it seems plausible in general.
- 'file_path' must be copied EXACTLY from the candidates you were given - never invent a path that \
wasn't in the candidate list.
- Do not claim more certainty than the evidence supports. If the connection is only suggestive, say \
so honestly rather than overstating it - a lower confidence score is expected and fine. Never assert \
a file is "definitely" affected.
- If none of the candidates are genuinely relevant, respond with {"matches": []} - an empty result is \
a valid, honest answer, not a failure.
- Respond with ONLY the JSON object - no markdown fences, no commentary."""

def _extract_json(text: str) -> str:
    text = text.strip()
    if text.startswith("```"):
        text = re.sub(r"```[a-zA-Z]*\s*\n?", "", text)
        text = re.sub(r"\n?\s*\s*$", "", text)
    return text.strip()

def _build_user_prompt(change_request: ChangeRequest, analysis: Analysis, candidates: list[IndexedFile]) -> str:
    payload = {
        "change_request": {
            "title": change_request.title,
            "description": change_request.description,
            "business_objective": change_request.business_objective,
        },
        "analysis_summary": analysis.summary,
        "analysis_category": analysis.category,
        "requirements": [r.description for r in analysis.requirements],
        "candidate_files": [_candidate_digest(f) for f in candidates],
    }
    return json.dumps(payload, indent=2)

def run_repository_match(
    db: Session,
    change_request: ChangeRequest,
    analysis: Analysis,
    scan: RepositoryScan,
) -> list[RepositoryFinding]:
    """Matches 'change_request' (via its given 'analysis') against the
    files 'scan' indexed, persists the result as new RepositoryFinding
    rows, and returns them.

    Re-running this for the exact same (change request, analysis, scan)
    triple replaces that triple's own prior findings (an idempotent
    re-run, not an ever-growing pile of duplicates) - but never touches a
    finding tied to a DIFFERENT analysis or scan, which stays exactly as
    it was, the same "old data is real history, never silently
    destroyed" rule Module 13 Phase 3 applies to AI re-analysis.

    Raises RepositoryMatchError on any AI-call failure - nothing is
    persisted or deleted before a successful, validated response comes
    back, mirroring analysis_engine.run_analysis()'s own failure
    handling exactly.
    """
    indexed_files = list(scan.files)
    query_tokens = query_tokens(change_request, analysis)
    candidates = _score_candidates(query_tokens, indexed_files)

    if not candidates:
        _replace_findings(db, change_request, analysis, scan, [])
        return []

    provider = get_ai_provider()
    if not provider.is_configured():
        raise RepositoryMatchError(
            "The AI provider isn't configured yet - set ANTHROPIC_API_KEY in backend/.env and restart the backend.",
            status_code=503,
        )

    settings = get_settings()
    user_prompt = _build_user_prompt(change_request, analysis, candidates)

    try:
        raw_text = provider.complete(
            user_prompt,
            system=SYSTEM_PROMPT,
            max_tokens=settings.ai_request_timeout_seconds,
            timeout=settings.ai_request_timeout_seconds,
        )
    except RepositoryMatchError:
        raise
    except Exception as exc:
        # noqa: BLE001 - any SDK/network error becomes a clean RepositoryMatchError
        exc_name = type(exc).__name__
        if "Timeout" in exc_name or "timeout" in str(exc).lower():
            raise RepositoryMatchError(
                f"The AI provider took too long to respond (over {settings.ai_request_timeout_seconds}s). "
                "Please try again.",
                status_code=504,
            ) from exc
        raise RepositoryMatchError(f"The AI provider call failed: {exc}.", status_code=502) from exc

    cleaned = _extract_json(raw_text)
    try:
        parsed = json.loads(cleaned)
    except json.JSONDecodeError as exc:
        raise RepositoryMatchError(
            f"The AI response wasn't valid JSON ({exc}). This usually clears up on retry.", status_code=422

```

```

    ) from exc

try:
    result = RepositoryMatchResult.model_validate(parsed)
except ValidationError as exc:
    raise RepositoryMatchError(
        f"The AI response didn't match the expected format: {exc}", status_code=422
    ) from exc

candidates_by_path = {f.file_path: f for f in candidates}
kept_matches = [(candidates_by_path[m.file_path], m) for m in result.matches if m.file_path in candidates_by_path]

return _replace_findings(db, change_request, analysis, scan, kept_matches)

def _replace_findings(
    db: Session,
    change_request: ChangeRequest,
    analysis: Analysis,
    scan: RepositoryScan,
    kept_matches: List,
) -> List[RepositoryFinding]:
    db.query(RepositoryFinding).filter(
        RepositoryFinding.change_request_id == change_request.id,
        RepositoryFinding.analysis_id == analysis.id,
        RepositoryFinding.repository_scan_id == scan.id,
    ).delete()

    findings: List[RepositoryFinding] = []
    for indexed_file, match in kept_matches:
        finding = RepositoryFinding(
            change_request_id=change_request.id,
            change_request_version=change_request.current_version or 1,
            analysis_id=analysis.id,
            repository_scan_id=scan.id,
            indexed_file_id=indexed_file.id,
            impact_level=match.impact_level,
            confidence=match.confidence,
            match_label=FileMatchLabel.from_confidence(match.confidence),
            reason=match.reason,
            evidence=match.evidence,
            created_at=datetime.utcnow(),
        )
        db.add(finding)
        findings.append(finding)

    db.commit()
    for finding in findings:
        db.refresh(finding)
    return findings

```

```

"""Module 15 Phase 1 (Repository Intelligence): safely walks a local
repository/folder and records what it found as a RepositoryScan +
IndexedFile rows.

Deliberately lightweight, per the module's own spec ("do not introduce
unnecessary infrastructure"): no upload/archive handling, no external
indexing service, no third-party gitignore-parsing dependency - just
os.walk with an allow-list of source-like extensions, a deny-list of
directories/files that should never be touched, and a small best-effort
.gitignore reader for the common cases (comments, blank lines, a plain
"name" or "name/" or "*.ext" pattern; a leading "/" anchors it to the
scanned root; negation ("!pattern") is not supported - "respect .gitignore
where practical" per the spec, not a full gitignore-spec implementation).

This module NEVER reads the contents of anything on the hard-excluded
list below - secrets/env files are skipped before they are ever opened,
not read-then-redacted.

Module 15 Phase 2 (Code Understanding): every file kept here also gets
passed to app.services.code_understanding.understand_file(), and its
imports/functions/classes/api_routes/database references/
config_references are stored alongside the basic path/language/size/line
stats recorded in Phase 1.
"""
from __future__ import annotations

import fnmatch
import json
import os
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path, PurePosixPath
from typing import Optional

from sqlalchemy import func
from sqlalchemy.orm import Session

from app.models.indexed_file import IndexedFile
from app.models.repository_scan import RepositoryScan
from app.models.enums import RepositoryScanStatus
from app.services.code_understanding import understand_file

# --- Directories never descended into, regardless of .gitignore -----
# Dependency/build/VCS/cache noise, plus this app's own SQLite data folder
# (binary database files, not source).
_IGNORED_DIR_NAMES = {
    "node_modules", ".git", ".hg", ".svn", ".pycache", ".venv", "venv",
    ".env", ".env.d", "build", "dist", ".next", ".nuxt", "out", "target",
    "bin", "obj", ".cache", "coverage", ".nyc_output", ".pytest_cache",
    ".mypy_cache", ".ruff_cache", ".tox", ".idea", ".vscode", ".parcel-cache",
    "site-packages", "data",
}

# --- Exact filenames that are always skipped - secrets first, then large
# generated/lockfiles that carry no hand-written meaning of their own. ---
_ALWAYS_IGNORED_FILENAMES = {
    ".env", ".npmrc", ".pyprc", ".netrc",
    "package-lock.json", "yarn.lock", "pnpm-lock.yaml", "poetry.lock", "Pipfile.lock",
    "credentials.json", "service-account.json",
}
_ALWAYS_IGNORED_FILENAME_PREFIXES = (".env.",)

# --- Extensions that are always skipped - secrets/keys, then binaries and
# other formats that aren't meaningfully "source". ---
_ALWAYS_IGNORED_EXTENSIONS = {
    ".pem", ".key", ".crt", ".cer", ".p12", ".pfx", ".keystore", ".jks", ".war",
    ".pyc", ".pyo", ".so", ".dll", ".dylib", ".exe", ".class", ".jar", ".war",
    ".db", ".sqlite", ".sqlite3",
    ".zip", ".tar", ".gz", ".tgz", ".7z", ".rar",
    ".png", ".jpg", ".jpeg", ".gif", ".bmp", ".ico", ".webp", ".svg",
    ".mp4", ".mov", ".avi", ".mp3", ".wav", ".pdf",
    ".woff", ".woff2", ".ttf", ".eot", ".otf",
    ".whl", ".egg", ".map", ".lock",
}

# --- The only extensions actually indexed - an allow-list is safer than
# trying to enumerate every possible binary format. ---
_INDEXABLE_EXTENSIONS = {
    ".py", ".js", ".jsx", ".ts", ".tsx", ".mjs", ".cjs",
    ".json", ".yaml", ".yml", ".toml", ".ini", ".cfg",
    ".html", ".css", ".scss", ".less",
    ".md", ".sql", ".sh", ".bat", ".ps1",
}

_EXTENSION_LANGUAGE_MAP = {
    ".py": "python",
    ".js": "javascript", ".jsx": "javascript", ".mjs": "javascript", ".cjs": "javascript",
    ".ts": "typescript", ".tsx": "typescript",
    ".json": "json", ".yaml": "yaml", ".yml": "yaml", ".toml": "toml",
    ".ini": "ini", ".cfg": "ini",
    ".html": "html", ".css": "css", ".scss": "scss", ".less": "less",
    ".md": "markdown", ".sql": "sql",
    ".sh": "shell", ".bat": "batch", ".ps1": "powershell",
}

# Files larger than this are skipped as "large irrelevant files" per the
# module spec, regardless of extension - a 5MB generated JSON fixture isn't
# useful context for matching a change request to source code.
MAX_INDEXABLE_FILE_BYTES = 512 * 1024

class RepositoryScanError(Exception):
    """Raised for a scan that can't even start (bad/missing configured
    path) - distinct from a per-file problem during the walk, which is
    just skipped and counted, never enough to fail the whole scan."""

@dataclass
class WalkResult:
    indexed: list[dict] = field(default_factory=list)
    skipped_count: int = 0

def load_gitignore_patterns(root: Path) -> list[str]:
    gitignore_path = root / ".gitignore"
    if not gitignore_path.is_file():
        return []
    patterns: list[str] = []
    try:
        text = gitignore_path.read_text(encoding="utf-8", errors="ignore")
    except OSError:
        return []
    for line in text.splitlines():
        stripped = line.strip()

```

```

        if not stripped or stripped.startswith("#") or stripped.startswith("!"):
            continue # blank/comment/negation - negation isn't supported, see module docstring
        patterns.append(stripped)
    return patterns

def matches_gitignore(rel_posix: str, basename: str, patterns: list[str]) -> bool:
    for pattern in patterns:
        pattern = pattern.rstrip("/") # a trailing slash just marks "directory-only"
        if not pattern:
            continue
        if pattern.startswith("/"):
            anchored = pattern.lstrip("/")
            if fnmatch.fnmatch(rel_posix, anchored) or fnmatch.fnmatch(rel_posix, f"{anchored}/*"):
                return True
        else:
            if (
                fnmatch.fnmatch(basename, pattern)
                or fnmatch.fnmatch(rel_posix, pattern)
                or fnmatch.fnmatch(rel_posix, f"*/{pattern}")
                or fnmatch.fnmatch(rel_posix, f"{pattern}/*")
                or fnmatch.fnmatch(rel_posix, f"*/{pattern}/*")
            ):
                return True
    return False

def should_skip_file(rel_posix: str, basename: str, ext: str, size_bytes: int, gitignore_patterns: list[str]) -> bool:
    if basename in _ALWAYS_IGNORED_FILENAMES:
        return True
    if any(basename.startswith(prefix) for prefix in _ALWAYS_IGNORED_FILENAME_PREFIXES):
        return True
    if ext in _ALWAYS_IGNORED_EXTENSIONS:
        return True
    if ext not in _INDEXABLE_EXTENSIONS:
        return True
    if size_bytes > MAX_INDEXABLE_FILE_BYTES:
        return True
    if matches_gitignore(rel_posix, basename, gitignore_patterns):
        return True
    return False

def walk(root: Path) -> WalkResult:
    result = WalkResult()
    gitignore_patterns = _load_gitignore_patterns(root)

    for dirpath, dirnames, filenames in os.walk(root):
        # Prune ignored directories in place so os.walk never descends into
        # them at all - not "descend then discard".
        kept_dirnames = []
        for d in dirnames:
            if d in _IGNORED_DIR_NAMES:
                continue
            rel_dir_posix = str(PurePosixPath(*Path(dirpath, d).relative_to(root).parts))
            if matches_gitignore(rel_dir_posix, d, gitignore_patterns):
                continue
            kept_dirnames.append(d)
        dirnames[:] = kept_dirnames

        for filename in filenames:
            full_path = Path(dirpath) / filename
            rel_parts = full_path.relative_to(root).parts
            rel_posix = str(PurePosixPath(*rel_parts))
            ext = full_path.suffix.lower()

            try:
                size_bytes = full_path.stat().st_size
            except OSError:
                result.skipped_count += 1
                continue

            if should_skip_file(rel_posix, filename, ext, size_bytes, gitignore_patterns):
                result.skipped_count += 1
                continue

            try:
                content = full_path.read_text(encoding="utf-8", errors="ignore")
            except OSError:
                result.skipped_count += 1
                continue

            language = _EXTENSION_LANGUAGE_MAP.get(ext)
            # Module 15 Phase 2: Reuses the content already read above for
            # line-counting - never a second file read. A file this can't
            # make sense of (bad syntax, etc.) just comes back with empty
            # lists, never a reason to skip indexing the file itself.
            understanding = understand_file(content, language)

            result.indexed.append(
                {
                    "file_path": rel_posix,
                    "language": language,
                    "size_bytes": size_bytes,
                    "line_count": content.count("\n") + (1 if content and not content.endswith("\n") else 0),
                    "imports": json.dumps(understanding.imports) if understanding.imports else None,
                    "functions": json.dumps(understanding.functions) if understanding.functions else None,
                    "classes": json.dumps(understanding.classes) if understanding.classes else None,
                    "api_routes": json.dumps(understanding.api_routes) if understanding.api_routes else None,
                    "database_references": (
                        json.dumps(understanding.database_references) if understanding.database_references else None
                    ),
                    "config_references": (
                        json.dumps(understanding.config_references) if understanding.config_references else None
                    ),
                }
            )

    return result

def scan_repository(
    db: Session,
    *,
    root_path: Optional[Path] = None,
    triggered_by: Optional[int] = None,
) -> RepositoryScan:
    """Walks 'root_path' (or the app's configured repository root if not
    given) and records the result as a new RepositoryScan + IndexedFile
    rows. Never mutates or deletes a prior scan - each call is a new, fully
    independent row, so an old scan's findings are always still exactly
    what they were (same non-destructive rule Module 13 Phase 3 applies to
    AI re-analysis)."""
    if root_path is None:
        from app.core.config import get_settings

        root_path = get_settings().repository_root_path

    root_path = Path(root_path).expanduser().resolve()
    if not root_path.exists() or not root_path.is_dir():
        raise RepositoryScanError()

```

```

        f"Configured repository path does not exist or is not a folder: {root_path}"
    )

scan_number = (
    db.query(func.max(RepositoryScan.scan_number))
    .filter(RepositoryScan.root_path == str(root_path))
    .scalar()
    or 0
) + 1

scan = RepositoryScan(
    root_path=str(root_path),
    scan_number=scan_number,
    status=RepositoryScanStatus.COMPLETED,
    triggered_by=triggered_by,
    started_at=datetime.utcnow(),
)
db.add(scan)
db.flush()

try:
    walk_result = _walk(root_path)
except Exception as exc: # noqa: BLE001 - a genuinely unexpected walk failure
    scan.status = RepositoryScanStatus.FAILED
    scan.error_message = str(exc)
    scan.completed_at = datetime.utcnow()
    db.commit()
    db.refresh(scan)
    return scan

for entry in walk_result.indexed:
    db.add(IndexedFile(scan_id=scan.id, **entry))

scan.file_count = len(walk_result.indexed)
scan.skipped_count = walk_result.skipped_count
scan.completed_at = datetime.utcnow()
db.commit()
db.refresh(scan)
return scan

```

```

"""Module 21 spec section 5 (System Settings) - deliberately scoped down
from the spec's full list. Two different things live here:

1. A genuinely admin-editable setting (cr_categories) - stored as a
SystemSetting row, read/written through get_cr_categories/
set_cr_categories below.
2. Read-only info panels for AI provider / repository / knowledge base
configuration - these stay .env-driven (see app/core/config.py) and are
only ever DISPLAYED here, never written through this module. Changing
them still goes through the guided .env edit this project has always
used, not a form that saves directly - see get_ai_provider_info /
get_repository_info / get_knowledge_base_info, none of which ever
return an actual API key or secret.

Priority values and the workflow status transition graph are NOT exposed
here at all, editable or otherwise - both are load-bearing across risk
scoring, sorting, and the status machine throughout this app (see
app/services/workflow_rules.py), and making them admin-editable at
runtime would be a real risk of quietly breaking other modules rather
than a simple settings toggle. That's a deliberate, disclosed scope limit
(see PROJECT_REPORT.md's Module 21 section), not an oversight.

cr_categories itself is ALSO scoped down from what it might look like it
does: it's a plain editable reference list (add/rename/remove a category
name), stored and returned as-is - it is NOT yet wired into the Dashboard
(app/api/dashboard.py) or Analytics (app/services/analytics.py) category
groupings, which each keep their own separate, already-tested
CANONICAL CATEGORIES/ CATEGORY ALIASES copies (a pre-existing, deliberate
duplication - see either file's own comments). Properly synchronizing an
editable alias-keyword mapping into both of those without risking a
regression in already-shipped reporting is a larger change than this
module takes on; this list is a maintained reference for now.
"""

from __future__ import annotations

import json
from typing import List, Optional

from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models.knowledge_document import KnowledgeDocument
from app.models.repository_scan import RepositoryScan
from app.models.system_setting import SystemSetting

CR_CATEGORIES_KEY = "cr_categories"

# Same default vocabulary app/api/change_requests.py, app/api/dashboard.py,
# and app/services/analytics.py each already hardcode their own copy of.
DEFAULT_CR_CATEGORIES: List[str] = [
    "Feature",
    "Bug Fix",
    "Security",
    "Database",
    "API",
    "Infrastructure",
    "Integration",
]

def get_cr_categories(db: Session) -> List[str]:
    row = db.get(SystemSetting, CR_CATEGORIES_KEY)
    if row is None:
        return list(DEFAULT_CR_CATEGORIES)
    try:
        value = json.loads(row.value_json)
    except (TypeError, ValueError):
        return list(DEFAULT_CR_CATEGORIES)
    return value if isinstance(value, list) else list(DEFAULT_CR_CATEGORIES)

def set_cr_categories(db: Session, categories: List[str], *, updated_by: Optional[int]) -> None:
    row = db.get(SystemSetting, CR_CATEGORIES_KEY)
    value_json = json.dumps(categories)
    if row is None:
        db.add(SystemSetting(key=CR_CATEGORIES_KEY, value_json=value_json, updated_by=updated_by))
    else:
        row.value_json = value_json
        row.updated_by = updated_by
    db.flush()

def get_ai_provider_info() -> dict:
    """Never returns an API key - only which provider is active, which
    model it's configured to use, and whether a key is present at all
    (True/False, never the value)."""
    settings = get_settings()
    provider = settings.ai_provider
    model_by_provider = {
        "anthropic": settings.anthropic_model,
        "openai": settings.openai_model,
        "gemini": settings.gemini_model,
        "ollama": settings.ollama_model,
        "openrouter": settings.openrouter_model,
    }
    key_configured_by_provider = {
        "anthropic": bool(settings.anthropic_api_key),
        "openai": bool(settings.openai_api_key),
        "gemini": bool(settings.gemini_api_key),
        "ollama": True, # no key needed - runs locally
        "openrouter": bool(settings.openrouter_api_key),
    }
    return {
        "provider": provider,
        "model": model_by_provider.get(provider, "unknown"),
        "api_key_configured": key_configured_by_provider.get(provider, False),
        "embeddings_supported": provider == "openrouter",
    }

def get_repository_info(db: Session) -> dict:
    settings = get_settings()
    latest_scan = db.query(RepositoryScan).order_by(RepositoryScan.started_at.desc()).first()
    return {
        "repository_root": str(settings.repository_root_path),
        "last_scan_status": latest_scan.status.value if latest_scan else None,
        "last_scan_file_count": latest_scan.file_count if latest_scan else None,
        "last_scan_at": latest_scan.completed_at or latest_scan.started_at if latest_scan else None,
    }

def get_knowledge_base_info(db: Session) -> dict:
    settings = get_settings()
    document_count = db.query(KnowledgeDocument).count()

```

```
return {  
  "embedding_model": settings.openrouter_embedding_model,  
  "embeddings_supported": settings.ai_provider == "openrouter",  
  "document_count": document_count,  
}
```

```

"""Module 17 Phase 4 (Test Cases & Implementation Plan 2.0 - Traceability).

The spec's own instruction: provide Requirement -> Implementation Task ->
Test Case traceability. Per this module's own Phase 1 decision (see
TestCaseRead/ImplementationTaskRead's own docstrings), requirement references
and risk references are deliberately NOT AI-generated - asking the AI to
cite a Requirement/Risk ID at generation time would mean citing IDs that
don't exist yet (they're only assigned once _persist_result flushes each
row), the exact hallucination-by-construction risk Module 15's own
"AI never invents a file path, code enforces the label" rule exists to
avoid. Instead this mirrors app/services/repository_linkage.py's own
established pattern exactly: a small, locally-tuned keyword-overlap
tokenizer, computed fresh on every response, never stored anywhere.

Unlike repository linkage (which links against RepositoryFinding rows that
live outside the analysis being built), the Requirement and Risk rows here
are already sitting right on the same AnalysisRead response being
annotated - no separate query is needed, since a test case/task can only
ever be traceable to a requirement/risk from its own analysis (tying an
artifact to a requirement from some OTHER analysis/version would violate
the architecture lock's own "never tied to the wrong version" rule).

A small tokenizer is duplicated here rather than importing
repository_linkage's own private `tokenize` - this app's established
convention (see that module's own docstring) is that a module's
underscore-prefixed helpers are its own business, and a second, small,
locally-tuned copy is preferred over reaching into another module's
internals.

Computed fresh on every response, never stored. An analysis with no
requirements/risks at all (or one where nothing shares meaningful words
with a given test case/task) ends up with empty reference lists - the
same honest "nothing found" outcome repository_linkage treats as valid.
"""
from _future_ import annotations

import re
from typing import Iterable

from app.schemas.requirement import RequirementRead
from app.schemas.risk import RiskRead

_STOPWORDS = {
    "the", "a", "an", "and", "or", "but", "for", "with", "this", "that", "from", "into",
    "when", "then", "than", "have", "has", "had", "will", "would", "should", "could",
    "can", "not", "are", "was", "were", "been", "being", "its", "their", "them", "they",
    "you", "your", "our", "ours", "also", "each", "any", "all", "some", "other", "more",
    "most", "use", "used", "using", "via", "per", "let", "need", "needs", "needed",
    "add", "adds", "added", "new", "make", "makes", "made", "get", "gets", "set", "sets",
}

_TOKEN_RE = re.compile(r"^[A-Za-z0-9]+")

# Same reasoning as repository_linkage._MIN_SHARED_TOKENS: one shared word
# (often a generic one like "user" or "checkout") is too weak a signal to
# present as a real traceability link.
_MIN_SHARED_TOKENS = 2

def _tokenize(*texts: str | None) -> set[str]:
    tokens: set[str] = set()
    for text in texts:
        if not text:
            continue
        for raw in _TOKEN_RE.split(text):
            word = raw.lower()
            if len(word) < 3 or word in _STOPWORDS:
                continue
            tokens.add(word)
    return tokens

def _requirement_tokens(requirement: RequirementRead) -> set[str]:
    return _tokenize(requirement.description, requirement.evidence)

def _risk_tokens(risk: RiskRead) -> set[str]:
    return _tokenize(risk.description, risk.mitigation, risk.evidence)

def requirement_references(item_text: str, requirements: Iterable[RequirementRead]) -> list[str]:
    """REQ-<id> for every requirement whose own description/evidence
    shares at least _MIN_SHARED_TOKENS meaningful words with `item_text`,
    most-overlapping first. The "REQ-" prefix mirrors TestCase.test_id's
    own "TC-001" human-readable style - <id> is the requirement's real
    database id, so a "REQ-7" reference points at exactly one row, never
    an ambiguous label.
    """
    item_tokens = _tokenize(item_text)
    if not item_tokens:
        return []
    scored: list[tuple[RequirementRead, int]] = []
    for requirement in requirements:
        overlap = len(item_tokens & _requirement_tokens(requirement))
        if overlap >= _MIN_SHARED_TOKENS:
            scored.append((requirement, overlap))
    scored.sort(key=lambda pair: pair[1], reverse=True)
    return [f"REQ-{requirement.id}" for requirement, _overlap in scored]

def risk_references(item_text: str, risks: Iterable[RiskRead]) -> list[str]:
    """Same as requirement_references above, but against this analysis's
    own Risk rows - "RISK-<id>".
    """
    item_tokens = _tokenize(item_text)
    if not item_tokens:
        return []
    scored: list[tuple[RiskRead, int]] = []
    for risk in risks:
        overlap = len(item_tokens & _risk_tokens(risk))
        if overlap >= _MIN_SHARED_TOKENS:
            scored.append((risk, overlap))
    scored.sort(key=lambda pair: pair[1], reverse=True)
    return [f"RISK-{risk.id}" for risk, _overlap in scored]

def annotate_analysis_response(response) -> None:
    """Mutates an already-built AnalysisRead in place, setting every test
    case's requirement_references/risk_references and every implementation
    task's requirement_references - mirrors repository linkage.
    annotate_analysis_response's own "build the Pydantic object, then set
    computed fields on it directly" pattern. A no-op when this analysis has
    neither requirements nor risks of its own (every reference list simply
    stays at its schema default of [])."""
    requirements = response.requirements
    risks = response.risks

```



```
if not requirements and not risks:
    return

for test_case in response.test_cases:
    text = f"{test_case.title} {test_case.description} {test_case.expected_result}"
    test_case.requirement_references = requirement_references(text, requirements)
    test_case.risk_references = risk_references(text, risks)

for task in response.implementation_tasks:
    text = f"{task.task} {task.description}"
    task.requirement_references = requirement_references(text, requirements)
```

```

"""Module 12: turns a CR edit into "what changed, field by field" plus a
new version - the alternative to the forbidden "Change Request updated."
apply_change_request_update() is the one place that:

1. diffs the incoming payload against the CR's current values (using
workflow rules.normalize_field_value so an enum-vs-string or
date-vs-isoformat mismatch never looks like a false change)
2. applies only the fields that actually changed
3. if anything changed: bumps current version, writes a
ChangeRequestVersion snapshot, and writes one FIELD_CHANGED history
event per changed field via app.services.history (never a single
vague "updated" event)
4. returns the list of changes made - the API response uses this for
the "here's what changed" confirmation the edit form shows.

"""
from __future__ import annotations

import json as _json
from dataclasses import dataclass
from typing import Any, Optional

from sqlalchemy.orm import Session

from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_version import ChangeRequestVersion
from app.models.enums import ApprovalStatus, HistoryAction, NotificationType
from app.models.user import User
from app.services import history
from app.services import notify as notify_service
from app.services.workflow_rules import (
    APPROVAL_TYPE_LABELS,
    EDITABLE_FIELDS,
    FIELD_LABELS,
    build_snapshot,
    normalize_field_value,
)

@dataclass
class FieldChange:
    field: str
    label: str
    old_value: Optional[str]
    new_value: Optional[str]

def display(value: Any, field: Optional[str] = None) -> Optional[str]:
    if value is None:
        return None
    if isinstance(value, list):
        return ", ".join(str(v) for v in value) if value else None
    if field == "priority" and isinstance(value, str):
        return value.replace("_", " ").title()
    return str(value)

def diff_change_request(change_request: ChangeRequest, payload: dict[str, Any]) -> list[FieldChange]:
    """payload is {field name: new_value} for whichever editable fields the
client sent (partial-update semantics - a field absent from payload is
left untouched, so this only reports/applies fields actually present)."""
    changes: list[FieldChange] = []
    for field in EDITABLE_FIELDS:
        if field not in payload:
            continue
        old_norm = normalize_field_value(field, getattr(change_request, field))
        new_norm = normalize_field_value(field, payload[field])
        old_norm = old_norm if old_norm not in (None, "", []) else None
        new_norm = new_norm if new_norm not in (None, "", []) else None
        if old_norm == new_norm:
            continue
        changes.append(
            FieldChange(
                field=field,
                label=FIELD_LABELS[field],
                old_value=display(old_norm, field),
                new_value=display(new_norm, field),
            )
        )
    return changes

def apply_field(change_request: ChangeRequest, field: str, raw_value: Any) -> None:
    if field == "tags":
        setattr(change_request, field, _json.dumps(raw_value) if raw_value else None)
    else:
        setattr(change_request, field, raw_value)

def apply_change_request_update(
    db: Session,
    change_request: ChangeRequest,
    payload: dict[str, Any],
    *,
    user: User,
) -> list[FieldChange]:
    """Applies payload to change request. If (and only if) something
actually changed: bumps current version, records the version snapshot,
and records field-level history. Returns the changes made (empty if
the save was a no-op)."""
    changes = diff_change_request(change_request, payload)
    if not changes:
        return changes

    changed_fields = {c.field for c in changes}
    for field in changed_fields:
        apply_field(change_request, field, payload[field])

    new_version = (change_request.current_version or 1) + 1
    change_request.current_version = new_version
    db.flush() # so build_snapshot() below sees the fields just set above

    if len(changes) == 1:
        only = changes[0]
        if only.old_value and only.new_value:
            summary = f"{only.label} changed from {only.old_value} to {only.new_value}"
        elif only.new_value:
            summary = f"{only.label} set to {only.new_value}"
        else:
            summary = f"{only.label} cleared"
    else:
        summary = ", ".join(c.label for c in changes) + " updated"

```

```

db.add(
    ChangeRequestVersion(
        change_request_id=change_request.id,
        version_number=new_version,
        changed_by=user.id,
        change_summary=summary,
        snapshot=build_snapshot(change_request),
    )
)
history.record_event(
    db,
    change_request_id=change_request.id,
    action=HistoryAction.VERSION_CREATED,
    user_id=user.id,
    new_value=f"Version {new_version}",
    version_number=new_version,
)
for change in changes:
    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.FIELD_CHANGED,
        user_id=user.id,
        field_name=change.field,
        old_value=change.old_value,
        new_value=change.new_value,
        version_number=new_version,
    )

# Module 12 section 6: editing a CR that already has an analysis makes
# that analysis outdated - record it on the timeline rather than
# silently leaving the old analysis looking current (the "is this
# outdated" banner itself is computed fresh at read time by
# workflow_rules.is_analysis_outdated; this is just the audit trail
# entry marking the moment it happened).
latest_analysis = (
    db.query(Analysis)
    .filter(Analysis.change_request_id == change_request.id)
    .order_by(Analysis.created_at.desc())
    .first()
)
if latest_analysis is not None and (latest_analysis.change_request_version or 1) != new_version:
    history.record_event(
        db,
        change_request_id=change_request.id,
        action=HistoryAction.AI_ANALYSIS_INVALIDATED,
        actor_label="System",
        new_value=f"Analysis from version {latest_analysis.change_request_version or 1} is now outdated",
        version_number=new_version,
    )
    interested_ids = {change_request.created_by} | {a.user_id for a in change_request.assignments}
    interested_ids.discard(user.id)
    for user_id in interested_ids:
        notify_service.notify(
            db,
            user_id=user_id,
            type=NotificationType.ANALYSIS_OUTDATED,
            title=f"CR-{change_request.id}'s AI analysis is outdated",
            message=f"{user.name} edited \"{change_request.title}\" - its last AI analysis no longer reflects the current version.",
            change_request_id=change_request.id,
        )

# Module 12 Phase 4: the same idea, applied to any approval still
# awaiting a response - it was requested against an earlier version of
# this CR, and this edit just moved the CR past it (see
# app/services/approvals.py::is_approval_outdated, computed fresh at
# read time; this is just the audit trail entry for the moment it
# happened). An approval that's already been responded to is a
# historical fact and is never touched here.
pending_approvals = (
    db.query(Approval)
    .filter(Approval.change_request_id == change_request.id, Approval.status == ApprovalStatus.PENDING)
    .all()
)
for approval in pending_approvals:
    if (approval.cr_version or 1) != new_version:
        type_label = APPROVAL_TYPE_LABELS.get(approval.approval_type, approval.approval_type.value)
        history.record_event(
            db,
            change_request_id=change_request.id,
            action=HistoryAction.APPROVAL_INVALIDATED,
            actor_label="System",
            new_value=f"{type_label} approval from version {approval.cr_version or 1} is now outdated",
            version_number=new_version,
        )
        if approval.approver_id != user.id:
            notify_service.notify(
                db,
                user_id=approval.approver_id,
                type=NotificationType.REAPPROVAL_REQUIRED,
                title=f"{type_label} approval on CR-{change_request.id} may need a re-check",
                message=f"{user.name} edited \"{change_request.title}\" after your {type_label} "
                    f"approval was requested - it may be worth a fresh look.",
                change_request_id=change_request.id,
            )

return changes

@dataclass
class FieldComparison:
    field: str
    label: str
    old_value: Optional[str]
    new_value: Optional[str]
    status: str # "unchanged" | "added" | "removed" | "changed"

def compare_snapshots(snapshot_a: str, snapshot_b: str) -> list[FieldComparison]:
    """Field-by-field diff between two ChangeRequestVersion snapshots, in
    EDITABLE_FIELDS order - what Compare Versions renders. "A clean
    field-by-field comparison is sufficient" (spec section 5) - this isn't
    trying to be a pixel-perfect text diff."""
    data_a: dict[str, Any] = _json.loads(snapshot_a) if snapshot_a else {}
    data_b: dict[str, Any] = _json.loads(snapshot_b) if snapshot_b else {}

    results: list[FieldComparison] = []
    for field in EDITABLE_FIELDS:
        old_norm = normalize_field_value(field, data_a.get(field))
        new_norm = normalize_field_value(field, data_b.get(field))
        old_norm = old_norm if old_norm not in (None, "", []) else None
        new_norm = new_norm if new_norm not in (None, "", []) else None

        if old_norm == new_norm:
            status = "unchanged"
        elif old_norm is None:
            status = "added"
        elif new_norm is None:
            status = "removed"
        else:
            status = "changed"

```

```
        results.append(  
            FieldComparison(  
                field=field,  
                label=FIELD_LABELS[field],  
                old_value=display(old_norm, field),  
                new_value=display(new_norm, field),  
                status=status,  
            )  
        )  
    return results
```

```

"""Module 12 (Enterprise Workflow) - the rules that keep the workflow from
being "anything can change to anything, whenever, by anyone":

* the status lifecycle's allowed transitions (STATUS_TRANSITIONS)
* which transitions require a typed reason (REASON_REQUIRED_STATUSES)
* the approval matrix (required approval types) - a simple, readable
  config, not a rule *engine* (The spec explicitly says not to
  overengineer this)
* "is the current AI analysis outdated" (is_analysis_outdated)
* per-CR permission helpers built on top of the account-wide UserRole
  plus ChangeRequestAssignment rows (assignment_roles_for /
  can_edit_change_request / can_manage_workflow / can_assign_users /
  can_request_approval)

Nothing here talks to the database except assignment roles for(), which
takes an already-loaded list of ChangeRequestAssignment rows rather than
querying itself - callers own the query/session, this module stays pure
rule logic that's easy to unit test.
"""

from __future__ import annotations

import json
from typing import Any, Iterable, Optional

from app.models.analysis import Analysis
from app.models.approval_rule import ApprovalRule
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.enums import (
    ApprovalRecommendation,
    ApprovalRuleType,
    AssignmentRole,
    ApprovalStatus,
    ChangeRequestStatus,
    ApprovalType,
    UserRole,
)
from app.models.user import User

CRS = ChangeRequestStatus

# --- Status lifecycle -----
#
# Draft -> Submitted -> Pending Analysis -> Analyzed -> Under Review ->
# {Changes Requested, Approval Required} -> Approved/Rejected ->
# Implementation Planned -> In Progress -> Implemented -> Validated ->
# Closed. Cancelled is reachable from every non-terminal state.
#
# IN REVIEW is this app's existing enum member for what the spec calls
# "Under Review" (kept, not renamed, to avoid breaking any CR already
# stored with that value - Module 12 only adds new status values, never
# renames existing ones).
STATUS_TRANSITIONS: dict[ChangeRequestStatus, set[ChangeRequestStatus]] = {
    CRS.DRAFT: {CRS.SUBMITTED, CRS.CANCELLED},
    CRS.SUBMITTED: {CRS.PENDING_ANALYSIS, CRS.CANCELLED},
    CRS.PENDING_ANALYSIS: {CRS.ANALYZED, CRS.CANCELLED},
    CRS.ANALYZED: {CRS.IN_REVIEW, CRS.CANCELLED},
    CRS.IN_REVIEW: {CRS.CHANGES_REQUESTED, CRS.APPROVAL_REQUIRED, CRS.CANCELLED},
    # After feedback is addressed: either straight back under review, or
    # through a re-analysis first if the edit was substantial enough to
    # warrant one (both are valid next steps, not a fixed sequence).
    CRS.CHANGES_REQUESTED: {CRS.IN_REVIEW, CRS.ANALYZED, CRS.CANCELLED},
    CRS.APPROVAL_REQUIRED: {CRS.APPROVED, CRS.REJECTED, CRS.CHANGES_REQUESTED, CRS.CANCELLED},
    CRS.APPROVED: {CRS.IMPLEMENTATION_PLANNED, CRS.CANCELLED},
    # A rejected CR isn't necessarily dead - it can be revised and
    # resubmitted through the same Changes Requested path as any other
    # feedback loop.
    CRS.REJECTED: {CRS.CHANGES_REQUESTED, CRS.CANCELLED},
    CRS.IMPLEMENTATION_PLANNED: {CRS.IN_PROGRESS, CRS.CANCELLED},
    CRS.IN_PROGRESS: {CRS.IMPLEMENTED, CRS.CANCELLED},
    CRS.IMPLEMENTED: {CRS.VALIDATED, CRS.CANCELLED},
    CRS.VALIDATED: {CRS.CLOSED},
    CRS.CLOSED: set(),
    CRS.CANCELLED: set(),
}

# Transitions INTO one of these statuses must carry a non-empty `reason` -
# these are exactly the "something went wrong / needs explanation" turns
# (spec section 8's own example is Under Review -> Changes Requested).
REASON_REQUIRED_STATUSES: set[ChangeRequestStatus] = {
    CRS.CHANGES_REQUESTED,
    CRS.REJECTED,
    CRS.CANCELLED,
}

def can_transition(current: ChangeRequestStatus, target: ChangeRequestStatus) -> bool:
    if current == target:
        return False
    return target in STATUS_TRANSITIONS.get(current, set())

def reason_required(target: ChangeRequestStatus) -> bool:
    return target in REASON_REQUIRED_STATUSES

def available_transitions(current: ChangeRequestStatus) -> list[ChangeRequestStatus]:
    """The statuses `current` can legally move to next, in a stable
    (alphabetical) order - what the "Change Status" picker offers."""
    return sorted(STATUS_TRANSITIONS.get(current, set()), key=lambda s: s.value)

# Human-facing labels for the raw status column (distinct from the older,
# derived "effective_status" used elsewhere - see app/api/change_requests.py
# :: effective_status). Every ChangeRequestStatus member must have one.
STATUS_LABELS: dict[ChangeRequestStatus, str] = {
    CRS.DRAFT: "Draft",
    CRS.SUBMITTED: "Submitted",
    CRS.PENDING_ANALYSIS: "Pending Analysis",
    CRS.ANALYZED: "Analyzed",
    CRS.IN_REVIEW: "Under Review",
    CRS.CHANGES_REQUESTED: "Changes Requested",
    CRS.APPROVAL_REQUIRED: "Approval Required",
    CRS.APPROVED: "Approved",
    CRS.REJECTED: "Rejected",
    CRS.IMPLEMENTATION_PLANNED: "Implementation Planned",
    CRS.IN_PROGRESS: "In Progress",
    CRS.IMPLEMENTED: "Implemented",
    CRS.VALIDATED: "Validated",
    CRS.CLOSED: "Closed",
    CRS.CANCELLED: "Cancelled",
}

# Human-facing labels for AssignmentRole - used in history lines ("assigned

```

```

# Priya Shah as Technical Lead") and by the frontend's assignment picker.
ROLE_LABELS: dict[AssignmentRole, str] = {
    AssignmentRole.REQUESTER: "Requester",
    AssignmentRole.OWNER: "Owner",
    AssignmentRole.TECHNICAL_LEAD: "Technical Lead",
    AssignmentRole.REVIEWER: "Reviewer",
    AssignmentRole.APPROVER: "Approver",
    AssignmentRole.SECURITY_REVIEWER: "Security Reviewer",
    AssignmentRole.QA_OWNER: "QA Owner",
    AssignmentRole.IMPLEMENTATION_OWNER: "Implementation Owner",
}

# Module 21 (Administration & Configuration): human-facing labels for the
# account-wide UserRole - PRODUCT_MANAGER displays as "Manager" (the spec's
# practical role list) rather than adding a second, confusingly-similar
# enum member for the same real-world role.
USER_ROLE_LABELS: dict[UserRole, str] = {
    UserRole.ADMIN: "Admin",
    UserRole.REQUESTER: "Requester",
    UserRole.ENGINEER: "Engineer",
    UserRole.REVIEWER: "Reviewer",
    UserRole.SECURITY_REVIEWER: "Security Reviewer",
    UserRole.APPROVER: "Approver",
    UserRole.PRODUCT_MANAGER: "Manager",
}

# --- AI analysis version awareness -----

def is_analysis_outdated(change_request: ChangeRequest, latest_analysis: Optional[Analysis]) -> bool:
    """True if there's no analysis yet, or the CR has been edited (its
    version bumped) since the most recent analysis was run."""
    if latest_analysis is None:
        return False # "no analysis" and "outdated analysis" are different banners
    cr_version = change_request.current_version or 1
    analysis_version = latest_analysis.change_request_version or 1
    return analysis_version != cr_version

def repository_finding_outdated_reasons(
    change_request: ChangeRequest,
    finding_change_request_version: int,
    finding_analysis_id: int,
    finding_repository_scan_id: int,
    *,
    latest_analysis_id: Optional[int],
    latest_repository_scan_id: Optional[int],
) -> list[str]:
    """Module 15 Phase 4: why (if at all) a RepositoryFinding may no longer
    reflect reality. Three independent things can have moved on since it
    was generated - the change request itself was edited, a newer AI
    analysis was run, or a newer repository scan was taken - and any ONE
    of them is enough to warn about. Returns every reason that actually
    applies (an empty list means still current) rather than collapsing to
    a single boolean, the same "computed fresh per request, never stored"
    rule is analysis_outdated() above already follows - a finding's own
    row never changes once written, only what's said about it."""
    reasons: list[str] = []
    cr_version = change_request.current_version or 1
    if finding_change_request_version != cr_version:
        reasons.append("The change request has been edited since this finding was generated.")
    if latest_analysis_id is not None and finding_analysis_id != latest_analysis_id:
        reasons.append("A newer AI analysis has been run for this change request.")
    if latest_repository_scan_id is not None and finding_repository_scan_id != latest_repository_scan_id:
        reasons.append("A newer repository scan is available.")
    return reasons

# --- Approval matrix -----
#
# A plain, readable config - not a rule engine. Risk sets a baseline
# ("who always needs to sign off at this risk level"); category/keyword
# matches add more approvers on top when the change touches something
# specific (security, database, customer-facing, etc). This only computes
# a *recommendation* - see app/services/approvals.py for where a human
# turns that into actual Approval rows by tagging real people.
#
# Module 21 (Administration & Configuration) made this admin-editable: the
# risk-bucket and category-keyword mappings below are now the SEEDED
# DEFAULTS for the real, database-backed app.models.approval_rule.py rows
# (see app/database/init db.py:: seed_approval_rules) rather than the only
# source of truth. required_approval_types() takes the currently-enabled
# ApprovalRule rows as a plain argument - the same "caller owns the query,
# this module stays pure rule logic" convention every other permission
# helper in this file already follows (see assignment_roles for's own
# comment) - and falls back to these exact constants when no rules are
# passed at all, so any caller not yet updated to load rules keeps
# behaving exactly as before.
_RISK_BASELINE: tuple[tuple[float, set[ApprovalType]], ...] = (
    (75, {ApprovalType.ENGINEERING_MANAGER, ApprovalType.SECURITY, ApprovalType.DIRECTOR}), # critical
    (50, {ApprovalType.ENGINEERING_MANAGER, ApprovalType.SECURITY}), # high
    (25, {ApprovalType.ENGINEERING_MANAGER}), # medium
)
_LOW_RISK_BASELINE = {ApprovalType.TECHNICAL} # "Team Lead approval" - low risk

_CATEGORY_KEYWORD_APPROVALS: tuple[tuple[str, ApprovalType], ...] = (
    ("security", ApprovalType.SECURITY),
    ("database", ApprovalType.DBA),
    ("db", ApprovalType.DBA),
    ("infrastructure", ApprovalType.RELEASE),
    ("production", ApprovalType.RELEASE),
    ("api", ApprovalType.TECHNICAL),
    # Customer-facing: no dedicated column exists for this, so it's
    # inferred from the target system / category naming - a reasonable,
    # non-overengineered heuristic rather than a new required field.
    ("customer", ApprovalType.PRODUCT),
    ("mobile app", ApprovalType.PRODUCT),
    ("storefront", ApprovalType.PRODUCT),
    ("checkout", ApprovalType.PRODUCT),
    ("pos frontend", ApprovalType.PRODUCT),
)

# Every DEFAULT APPROVAL RULE SEEDS entry becomes one ApprovalRule row on
# first startup (see init db.py) - "risk" rows built from _RISK_BASELINE/
# _LOW_RISK_BASELINE, "category" rows built from _CATEGORY_KEYWORD_APPROVALS,
# one row per keyword so an admin can edit/disable them individually.
DEFAULT_APPROVAL_RULE_SEEDS: tuple[tuple[str, str, list[str]], ...] = (
    ("risk", "critical", sorted(t.value for t in _RISK_BASELINE[0][1])),
    ("risk", "high", sorted(t.value for t in _RISK_BASELINE[1][1])),
    ("risk", "medium", sorted(t.value for t in _RISK_BASELINE[2][1])),
    ("risk", "low", sorted(t.value for t in _LOW_RISK_BASELINE)),
) + tuple(("category", keyword, [approval_type.value] for keyword, approval_type in _CATEGORY_KEYWORD_APPROVALS))

def risk_baseline(risk_score: float) -> set[ApprovalType]:
    for threshold, approvals in _RISK_BASELINE:
        if risk_score >= threshold:
            return set(approvals)
    return set(_LOW_RISK_BASELINE)

```

```

# Module 13 (AI Analysis 2.0): moved here from app/api/change_requests.py
# (where it started as a private, list-endpoint-only helper) so
# app/services/analysis_delta.py can share the exact same thresholds - one
# definition of what "high risk" means, not two that could drift apart.
_RISK_BUCKETS: tuple[tuple[float, str], ...] = (
    (75, "critical"),
    (50, "high"),
    (25, "medium"),
)

def risk_bucket(score: float) -> str:
    for threshold, label in _RISK_BUCKETS:
        if score >= threshold:
            return label
    return "low"

# Module 18 Phase 2 (Notifications, My Work & Personal Engineering Queue):
# a plain ordering over the same four bucket labels risk_bucket() already
# returns, so "did risk get WORSE" can be answered with a simple integer
# comparison rather than re-deriving/hardcoding the low->critical order a
# second time somewhere else.
_RISK_BUCKET_RANK: dict[str, int] = {"low": 0, "medium": 1, "high": 2, "critical": 3}

def risk_bucket_rank(bucket: str) -> int:
    return _RISK_BUCKET_RANK.get(bucket, 0)

APPROVAL_TYPE_LABELS: dict[ApprovalType, str] = {
    ApprovalType.TECHNICAL: "Technical",
    ApprovalType.SECURITY: "Security",
    ApprovalType.PRODUCT: "Product",
    ApprovalType.ENGINEERING_MANAGER: "Engineering Manager",
    ApprovalType.DIRECTOR: "Director",
    ApprovalType.QA: "QA",
    ApprovalType.DBA: "DBA",
    ApprovalType.RELEASE: "Release",
    ApprovalType.GENERAL: "General",
}

APPROVAL_STATUS_LABELS: dict[ApprovalStatus, str] = {
    ApprovalStatus.PENDING: "Pending",
    ApprovalStatus.APPROVED: "Approved",
    ApprovalStatus.REJECTED: "Rejected",
    ApprovalStatus.CHANGES_REQUESTED: "Changes Requested",
    ApprovalStatus.CANCELLED: "Cancelled",
}

# A responder (the tagged approver, and only the tagged approver - see
# app/models/approval.py) may only move a PENDING approval to one of these -
# never back to PENDING, and never to CANCELLED (that's a request-side
# withdrawal, not a response - see app/services/approvals.py::cancel_approval).
APPROVAL_RESPONSE_STATUSES: set[ApprovalStatus] = {
    ApprovalStatus.APPROVED,
    ApprovalStatus.REJECTED,
    ApprovalStatus.CHANGES_REQUESTED,
}

# Same "explain what went wrong" rule as REASON_REQUIRED_STATUSES above,
# applied to an approval response instead of a CR status transition.
APPROVAL_REASON_REQUIRED: set[ApprovalStatus] = {
    ApprovalStatus.REJECTED,
    ApprovalStatus.CHANGES_REQUESTED,
}

def _approval_types_from_json(raw: str) -> set[ApprovalType]:
    try:
        values = json.loads(raw) if raw else []
    except (TypeError, ValueError):
        values = []
    result = set()
    for value in values:
        try:
            result.add(ApprovalType(value))
        except ValueError:
            continue # an unrecognized value in the row - skip rather than crash a report/recommendation
    return result

def required_approval_types(
    change_request: ChangeRequest,
    analysis: Analysis,
    rules: Optional[Iterable[ApprovalRule]] = None,
) -> set[ApprovalType]:
    """AI-driven approval RECOMMENDATION only (spec section 14/29 - this
    never grants an approval, it only says which ones a human should go
    request).

    `rules` is the CURRENTLY ENABLED set of admin-configured ApprovalRule
    rows (Module 21) - the caller loads them from the database (same
    caller owns the query" convention as assignment roles for above) and
    passes them in. When `rules` is None (no caller updated yet, or a unit
    test exercising this function directly), this falls back to the exact
    hardcoded matrix Module 21 seeded those rows from, so behavior never
    silently changes for a caller that hasn't been updated.

    One thing stays hardcoded either way, deliberately not exposed as an
    editable rule: if the AI itself flagged a security risk on this
    analysis, Security approval is always added. That's not a "risk level"
    or "keyword" match - it's a structural fact about what the AI actually
    found - so it doesn't fit either rule shape section 4 asks for, and
    silently NOT requiring security sign-off on an AI-flagged security risk
    would be a real safety regression an admin could make by accident."""
    haystack = " ".join(
        filter(None, [analysis.category, change_request.target_system, change_request.title])
    ).lower()

    if rules is None:
        required = _risk_baseline(analysis.risk_score)
        for keyword, approval_type in _CATEGORY_KEYWORD_APPROVALS:
            if keyword in haystack:
                required.add(approval_type)
    else:
        required = set()
        bucket = risk_bucket(analysis.risk_score)
        for rule in rules:
            if not rule.enabled:
                continue
            if rule.rule_type == ApprovalRuleType.RISK and rule.match_value == bucket:
                required |= _approval_types_from_json(rule.approval_types)
            elif rule.rule_type == ApprovalRuleType.CATEGORY and rule.match_value in haystack:
                required |= _approval_types_from_json(rule.approval_types)

    if any((r.category is not None and r.category.value == "security") for r in analysis.risks):
        required.add(ApprovalType.SECURITY)

```

```

    return required

def format_approval_types(types: Iterable[ApprovalType]) -> str:
    """Human-readable list of approval types, e.g. "Engineering Manager and
    Security" or "None" - the one shared formatter for which approvals are
    required," used by both the analysis-to-analysis diff (app/services/analysis.delta.py) and the workflow recommendation text
    below, so the two places that describe a set of required approvals in
    prose can't drift into two different phrasings."""
    labels = sorted(APPROVAL_TYPE_LABELS.get(t, t.value) for t in types)
    if not labels:
        return "None"
    if len(labels) == 1:
        return labels[0]
    return ", ".join(labels[:-1]) + f" and {labels[-1]}"

# --- Module 13 Phase 4: workflow recommendation text + human-override ---
#
# "Workflow recommendation text" is computed, not AI-written - the same
# choice he made for Phase 2's analysis-to-analysis diff (no extra AI call
# on every read, and it's never wrong about which approvals the numbers
# actually require). It only restates what `analysis.recommendation` /
# `required_approval_types()` already say, in one plain sentence, so a
# reviewer doesn't have to piece the two together themselves.

RECOMMENDATION_LABELS: dict[ApprovalRecommendation, str] = {
    ApprovalRecommendation.APPROVE: "Approve",
    ApprovalRecommendation.APPROVE_WITH_CONDITIONS: "Approve with Conditions",
    ApprovalRecommendation.NEEDS_MORE_INFO: "Needs More Information",
    ApprovalRecommendation.REQUIRES_CLARIFICATION: "Requires Clarification",
    ApprovalRecommendation.REJECT: "Reject",
}

# What a human decision "in line with" a given AI recommendation looks
# like - used only to detect a human override for the audit trail
# (overrides ai_recommendation, below), never to block or auto-decide
# anything. A human can decide however they want; this only makes it
# visible on the record when they didn't follow the AI ("AI recommends,
# humans decide" - a decision humans make is never wrong for disagreeing).
_ALIGNED_APPROVAL_STATUS: dict[ApprovalRecommendation, ApprovalStatus] = {
    ApprovalRecommendation.APPROVE: ApprovalStatus.APPROVED,
    ApprovalRecommendation.APPROVE_WITH_CONDITIONS: ApprovalStatus.APPROVED,
    ApprovalRecommendation.NEEDS_MORE_INFO: ApprovalStatus.CHANGES_REQUESTED,
    ApprovalRecommendation.REQUIRES_CLARIFICATION: ApprovalStatus.CHANGES_REQUESTED,
    ApprovalRecommendation.REJECT: ApprovalStatus.REJECTED,
}

def recommendation_summary(
    change_request: ChangeRequest,
    analysis: Analysis,
    rules: Optional[Iterable[ApprovalRule]] = None,
) -> str:
    """One computed sentence combining the AI's own recommendation with
    what it actually takes to move this change request forward at this
    risk level - shown alongside (never in place of) the raw
    `recommendation` / `recommendation_reasoning` fields already on the
    analysis. `rules` is passed straight through to required_approval_types
    (Module 21) - see that function's own docstring."""
    rec_label = RECOMMENDATION_LABELS.get(analysis.recommendation, "No recommendation yet")
    bucket = risk_bucket(analysis.risk_score).capitalize()
    approvals = required_approval_types(change_request, analysis, rules)

    if not approvals:
        approvals_clause = "no formal approval is required at this risk level"
    else:
        approvals_clause = f"requires {format_approval_types(approvals)} approval before it can proceed"

    return f"AI recommends: {rec_label} ({bucket} risk) - {approvals_clause}."

def overrides_ai_recommendation(
    recommendation: Optional[ApprovalRecommendation], decided_status: ApprovalStatus
) -> bool:
    """True when a human's approval decision doesn't match what the AI's
    own recommendation would suggest (see _ALIGNED_APPROVAL_STATUS above) -
    purely descriptive and auditable, never enforced; a human is never
    blocked from deciding either way. No recommendation on record (e.g.
    this approval predates any analysis) means there's nothing to have
    overridden."""
    if recommendation is None:
        return False
    return _ALIGNED_APPROVAL_STATUS.get(recommendation) != decided_status

# --- Per-CR permissions -----
#
# UserRole (account-wide: admin/engineer/product_manager/reviewer) answers
# "what kind of account is this"; ChangeRequestAssignment (per-CR) answers
# "what is this person responsible for on THIS change request." Both feed
# into the helpers below - deliberately simple boolean checks, not a
# generic permissions/RBAC framework.

def assignment_roles_for(
    user: User, assignments: Iterable[ChangeRequestAssignment]
) -> set[AssignmentRole]:
    return {a.role for a in assignments if a.user_id == user.id}

def is_admin(user: User) -> bool:
    return user.role == UserRole.ADMIN

def is_active_user(user: User) -> bool:
    """Module 21: NULL (every account that existed before this column did)
    and True both mean active - only an explicit False, set by an admin
    deactivating the account, means inactive. See the column's own
    docstring on app/models/user.py:User.is_active."""
    return user.is_active is not False

def can_edit_change_request(
    user: User, change_request: ChangeRequest, assignments: Iterable[ChangeRequestAssignment]
) -> bool:
    """Requester + Owner + Technical Lead + Admin can edit (spec section 9).
    Reviewers/Approvers/QA/Security can comment and act on their own
    approvals, but not edit CR fields directly."""
    if is_admin(user):
        return True
    if change_request.created_by == user.id:
        return True
    roles = assignment_roles_for(user, assignments)
    return bool(roles & {AssignmentRole.OWNER, AssignmentRole.TECHNICAL_LEAD})

def can_manage_workflow(
    user: User, change_request: ChangeRequest, assignments: Iterable[ChangeRequestAssignment]
) -> bool:

```



```

    """Change status, assign reviewers, request approval - the CR Owner's
    job (spec section 9), plus Admin."""
    if is_admin(user):
        return True
    if change_request.created_by == user.id:
        return True
    roles = assignment_roles_for(user, assignments)
    return bool(roles & {AssignmentRole.OWNER, AssignmentRole.TECHNICAL_LEAD})

def can_assign_users(
    user: User, change_request: ChangeRequest, assignments: Iterable[ChangeRequestAssignment]
) -> bool:
    return can_manage_workflow(user, change_request, assignments)

def can_request_approval(
    user: User, change_request: ChangeRequest, assignments: Iterable[ChangeRequestAssignment]
) -> bool:
    return can_manage_workflow(user, change_request, assignments)

def can_review_analysis_findings(
    user: User, change_request: ChangeRequest, assignments: Iterable[ChangeRequestAssignment]
) -> bool:
    """Module 14 Phase 6: who may mark a Requirement Confirmed/Needs
    Clarification, or change a Security finding's Status. Broader than
    can manage workflow (the CR Owner's specific job) - reviewing AI
    findings is meant to be shared with anyone actually doing the
    technical/security review of this change, not just whoever is driving
    its workflow, so Reviewer and Security Reviewer are included here too."""
    if is_admin(user):
        return True
    if change_request.created_by == user.id:
        return True
    roles = assignment_roles_for(user, assignments)
    return bool(
        roles
        & {
            AssignmentRole.OWNER,
            AssignmentRole.TECHNICAL_LEAD,
            AssignmentRole.REVIEWER,
            AssignmentRole.SECURITY_REVIEWER,
        }
    )

# --- Editable fields / versioning snapshot -----
#
# The single source of truth for "what counts as an editable CR field" -
# used both by the PUT /change-requests/{id} handler (to know what it's
# allowed to touch and diff) and by init_db.py's one-time backfill (to
# build a Version 1 snapshot for CRs that existed before this module).
# Order here is display order for the edit form / history / diff views.

EDITABLE_FIELDS: tuple[str, ...] = (
    "title",
    "description",
    "business_objective",
    "priority",
    "requested_by",
    "target_system",
    "desired_deadline",
    "business_impact",
    "technical_impact",
    "customer_impact",
    "environment",
    "dependencies_note",
    "compliance_requirements",
    "tags",
)

FIELD_LABELS: dict[str, str] = {
    "title": "Title",
    "description": "Description",
    "business_objective": "Business Objective",
    "priority": "Priority",
    "requested_by": "Requested By",
    "target_system": "Target System",
    "desired_deadline": "Desired Deadline",
    "business_impact": "Business Impact",
    "technical_impact": "Technical Impact",
    "customer_impact": "Customer Impact",
    "environment": "Environment",
    "dependencies_note": "Dependencies",
    "compliance_requirements": "Compliance Requirements",
    "tags": "Tags",
}

def normalize_field_value(field: str, value: Any) -> Any:
    """Turns a raw field value - however it arrives, whether read off a
    ChangeRequest ORM object (an enum member, a date object, a JSON-encoded
    tags string) or a freshly-validated PUT payload (an enum member, a
    date object, a real list) - into one comparable, JSON-safe shape.
    Used for both the version snapshot and for diffing old vs new values
    on an edit, so "did this field actually change" and "what does the
    stored snapshot look like" always agree."""
    if value is None:
        return None
    if field == "priority":
        return value.value if hasattr(value, "value") else value
    if field == "desired_deadline":
        return value.isoformat() if hasattr(value, "isoformat") else value
    if field == "tags":
        if isinstance(value, str):
            try:
                return json.loads(value)
            except (TypeError, ValueError):
                return value
        return list(value) if value else None
    return value

def field_snapshot_value(change_request: ChangeRequest, field: str) -> Any:
    return normalize_field_value(field, getattr(change_request, field))

def build_snapshot(change_request: ChangeRequest) -> str:
    """A JSON object of every editable field's current value - what gets
    stored on a ChangeRequestVersion row and diffed by Compare Versions."""
    data = {field: field_snapshot_value(change_request, field) for field in EDITABLE_FIELDS}
    return json.dumps(data)

```

backend/app/utils/__init__.py

1 lines

Frontend (React)

frontend/src/App.css

17 lines

```
/* Global, app-wide baseline - per-screen styling lives with each screen
(see components/auth-layout.css, styles/home.css). */
* {
  box-sizing: border-box;
}

body {
  font-family:
    -apple-system,
    BlinkMacSystemFont,
    'Segoe UI',
    Roboto,
    Helvetica,
    Arial,
    sans-serif;
}
```

frontend/src/App.jsx

40 lines

```
import { useState } from 'react'
import './App.css'
import { AuthProvider, useAuth } from './context/AuthContext'
import AuthenticatedHome from './pages/AuthenticatedHome'
import LoginPage from './pages/LoginPage'
import RegisterPage from './pages/RegisterPage'

function AppContent() {
  const { isAuthenticated, initializing } = useAuth()
  const [authView, setAuthView] = useState('login') // 'login' | 'register'

  if (initializing) {
    return (
      <div className="app-loading">
        <p>Loading</p>
      </div>
    )
  }

  if (isAuthenticated) {
    return <AuthenticatedHome />
  }

  return authView === 'login' ? (
    <LoginPage onSwitchToRegister={() => setAuthView('register')} />
  ) : (
    <RegisterPage onSwitchToLogin={() => setAuthView('login')} />
  )
}

function App() {
  return (
    <AuthProvider>
      <AppContent />
    </AuthProvider>
  )
}

export default App
```

```
import { apiRequest } from './client'

// Module 21 (Administration & Configuration) - see backend/app/api/admin.py.
// Every one of these requires an Admin account server-side; the frontend
// also only ever shows the Admin page to an admin (see AppHeader's
// NAV_ITEMS filtering and AdminPage's own access check) - this file makes
// no access decisions of its own.

// --- Users -----
export function listUsers(token) {
  return apiRequest('/api/admin/users', { token })
}

export function createUser(token, { name, email, password, role }) {
  return apiRequest('/api/admin/users', { method: 'POST', token, body: { name, email, password, role } })
}

export function updateUser(token, userId, changes) {
  return apiRequest(`/api/admin/users/${userId}`, { method: 'PATCH', token, body: changes })
}

// --- System-level audit log -----
export function listAuditLog(token, { limit = 50, offset = 0 } = {}) {
  return apiRequest(`/api/admin/audit-log?limit=${limit}&offset=${offset}`, { token })
}

// --- Permissions matrix -----
export function listPermissions(token) {
  return apiRequest('/api/admin/permissions', { token })
}

export function updatePermissions(token, permissions) {
  return apiRequest('/api/admin/permissions', { method: 'PUT', token, body: { permissions } })
}

// --- Approval rules -----
export function listApprovalRules(token) {
  return apiRequest('/api/admin/approval-rules', { token })
}

export function createApprovalRule(token, rule) {
  return apiRequest('/api/admin/approval-rules', { method: 'POST', token, body: rule })
}

export function updateApprovalRule(token, ruleId, changes) {
  return apiRequest(`/api/admin/approval-rules/${ruleId}`, { method: 'PATCH', token, body: changes })
}

export function deleteApprovalRule(token, ruleId) {
  return apiRequest(`/api/admin/approval-rules/${ruleId}`, { method: 'DELETE', token })
}

// --- System settings -----
export function getSystemSettings(token) {
  return apiRequest('/api/admin/system-settings', { token })
}

export function updateCrCategories(token, categories) {
  return apiRequest('/api/admin/system-settings/cr-categories', { method: 'PUT', token, body: { categories } })
}
```

```

import { ApiError, API_BASE_URL, apiRequest } from './client'

// Module 6: AI Analysis Engine endpoints.

export function analyzeChangeRequest(id, token) {
  return apiRequest(`/api/change-requests/${id}/analyze`, { method: 'POST', token })
}

export function getAnalysis(id, token) {
  return apiRequest(`/api/change-requests/${id}/analysis`, { token })
}

// Module 14 Phase 6: an authorized human's verdict on one extracted
// requirement (Confirmed / Needs Clarification) - never touches the AI's
// own certainty/confidence/evidence on that row.
export function reviewRequirement(id, requirementId, payload, token) {
  return apiRequest(`/api/change-requests/${id}/requirements/${requirementId}/review`, {
    method: 'PATCH',
    body: payload,
    token,
  })
}

// Supplies the missing information the AI itself asked for, right on the
// Missing Information tab - open to anyone who can comment on this change
// request, not gated to a reviewer role. Always marks the question
// resolved; can be called again on an already-answered question to
// correct it.
export function answerClarificationQuestion(id, questionId, answer, token) {
  return apiRequest(`/api/change-requests/${id}/clarification-questions/${questionId}/answer`, {
    method: 'PATCH',
    body: { answer },
    token,
  })
}

// Module 14 Phase 6: moves a Security finding's Status (the AI itself can
// only ever set Open/Not Applicable - a human sets Acknowledged/Resolved,
// or reopens/corrects it, here)
export function updateSecurityFindingStatus(id, findingId, payload, token) {
  return apiRequest(`/api/change-requests/${id}/security-findings/${findingId}/status`, {
    method: 'PATCH',
    body: payload,
    token,
  })
}

// Module 17 Phase 6: an authorized human correcting/refining one
// AI-generated test case in place. Every field is optional - only send
// what actually changed (see TestCaseUpdate's own docstring); the AI's
// prior wording is preserved on the audit trail, never overwritten.
export function updateTestCase(id, testCaseId, payload, token) {
  return apiRequest(`/api/change-requests/${id}/test-cases/${testCaseId}`, {
    method: 'PATCH',
    body: payload,
    token,
  })
}

// Module 17 Phase 6: same as updateTestCase above, for one implementation
// task.
export function updateImplementationTask(id, taskId, payload, token) {
  return apiRequest(`/api/change-requests/${id}/implementation-tasks/${taskId}`, {
    method: 'PATCH',
    body: payload,
    token,
  })
}

// Module 13 Phase 2: every analysis ever run for this change request,
// newest first - a lightweight summary (no requirements/risks/etc.), just
// enough to list past analyses and pick one to compare against the one
// immediately before it.
export function getAnalysisHistory(id, token) {
  return apiRequest(`/api/change-requests/${id}/analyses`, { token })
}

/** Module 13 Phase 2: a computed (not AI-written) diff between two
 * analyses of the same change request. Both 'from' and 'to' are optional -
 * passing only 'to' compares it against whichever analysis ran
 * immediately before it. */
export function compareAnalyses(id, { from, to } = {}, token) {
  const params = new URLSearchParams()
  if (from != null) params.set('from', from)
  if (to != null) params.set('to', to)
  const qs = params.toString()
  return apiRequest(`/api/change-requests/${id}/analysis/compare${qs ? `? ${qs}` : ''}`, { token })
}

/** Module 20: every version this change request has ever had, newest
 * first, each flagged with whether it's the current one and whether an
 * analysis was ever run against it specifically - what the "generate a
 * report for an earlier version" picker renders from. */
export function listReportableVersions(id, token) {
  return apiRequest(`/api/change-requests/${id}/report/versions`, { token })
}

/** Module 11 (made version-aware by Module 20): fetches the PDF report and
 * triggers a browser download. Not built on apiRequest() - that helper
 * only understands JSON bodies, and this response is a binary PDF (or, on
 * failure, a JSON error body - handled the same way apiRequest reports
 * errors, for a consistent message either way).
 *
 * 'version' is optional - omit it for "the current version's report" (which
 * now 409s, rather than downloading something misleading, if the CR has
 * been edited since its last analysis - see the ApiError's '.detail' on
 * that response: {message, cr_version, last_analyzed_version,
 * can_reanalyze}); pass a specific version number to explicitly generate a
 * historical report for that version instead. */
export async function downloadReport(id, token, version) {
  const qs = version != null ? `?version=${encodeURIComponent(version)}` : ''
  let response
  try {
    response = await fetch(`${API_BASE_URL}/api/change-requests/${id}/reports${qs}`, {
      headers: token ? { Authorization: `Bearer ${token}` } : {},
    })
  } catch {
    throw new ApiError('Could not reach the backend. Is it running?', 0)
  }

  if (!response.ok) {
    let detail
    try {
      detail = (await response.json())?.detail
    }
  }

```

```

    } catch {
      detail = null
    }
    const message = Array.isArray(detail)
      ? detail.map((d) => d.msg).join(' ')
      : typeof detail === 'string'
        ? detail
        : detail?.message || `Report generation failed with status ${response.status}`
    throw new ApiError(message, response.status, detail)
  }

  const blob = await response.blob()
  const disposition = response.headers.get('content-disposition') || ''
  const match = disposition.match(/filename="?([^\"]+)"?/)
  const filename = match ? match[1] : `CR-${String(id).padStart(4, '0')}-analysis-report.pdf`

  const url = URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = filename
  document.body.appendChild(link)
  link.click()
  link.remove()
  URL.revokeObjectURL(url)
}

```



```
import { apiRequest } from './client'

// Module 19 (Engineering Change Analytics): every endpoint here shares the
// same filter shape (spec section 8) - see buildQuery() below - so calling
// all 6 with the same `filters` object always describes the same slice of
// the data, no matter which section of the dashboard is asking.

/** Turns the page's one shared filter state into a query string every
 * analytics endpoint accepts identically. Keys are omitted entirely when
 * empty/undefined rather than sent as blank strings - the backend treats a
 * missing filter as "no restriction", never as "match nothing". */
function buildQuery(filters = {}) {
  const params = new URLSearchParams()
  if (filters.dateFrom) params.set('date_from', filters.dateFrom)
  if (filters.dateTo) params.set('date_to', filters.dateTo)
  if (filters.status) params.set('status', filters.status)
  if (filters.priority) params.set('priority', filters.priority)
  if (filters.risk) params.set('risk', filters.risk)
  if (filters.category) params.set('category', filters.category)
  if (filters.ownerId) params.set('owner_id', filters.ownerId)
  const query = params.toString()
  return query ? `?${query}` : ''
}

export function getExecutiveAndWorkflowMetrics(token, filters) {
  return apiRequest(`/api/analytics/executives${buildQuery(filters)}`, { token })
}

export function getRiskAnalytics(token, filters) {
  return apiRequest(`/api/analytics/risk${buildQuery(filters)}`, { token })
}

export function getApprovalBottlenecks(token, filters) {
  return apiRequest(`/api/analytics/approval-bottlenecks${buildQuery(filters)}`, { token })
}

export function getChangeAnalytics(token, filters) {
  return apiRequest(`/api/analytics/changes${buildQuery(filters)}`, { token })
}

export function getWorkloadAnalytics(token, filters) {
  return apiRequest(`/api/analytics/workload${buildQuery(filters)}`, { token })
}

export function getAIAnalytics(token, filters) {
  return apiRequest(`/api/analytics/ai${buildQuery(filters)}`, { token })
}
```

frontend/src/api/approvals.js

30 lines

```
import { apiRequest } from './client'

// Module 12 Phase 4: approvals - tagging a real person for a specific kind
// of sign-off, and that person's own Approve/Reject/Request Changes
// response. See backend/app/api/change_requests.py's "approvals" section.

export function listApprovals(id, token) {
  return apiRequest(`/api/change-requests/${id}/approvals`, { token })
}

export function getRecommendedApprovals(id, token) {
  return apiRequest(`/api/change-requests/${id}/approvals/recommended`, { token })
}

export function createApprovalRequest(id, payload, token) {
  return apiRequest(`/api/change-requests/${id}/approvals`, { method: 'POST', body: payload, token })
}

export function respondToApproval(id, approvalId, payload, token) {
  return apiRequest(`/api/change-requests/${id}/approvals/${approvalId}/respond`, {
    method: 'POST',
    body: payload,
    token,
  })
}

export function cancelApprovalRequest(id, approvalId, token) {
  return apiRequest(`/api/change-requests/${id}/approvals/${approvalId}`, { method: 'DELETE', token })
}
```

frontend/src/api/auth.js

24 lines

```
import { apiRequest } from './client'

export function registerRequest(name, email, password, confirmPassword) {
  return apiRequest('/auth/register', {
    method: 'POST',
    body: { name, email, password, confirm_password: confirmPassword },
  })
}

export function loginRequest(email, password, rememberMe) {
  return apiRequest('/auth/login', {
    method: 'POST',
    body: { email, password, remember_me: rememberMe },
  })
}

export function getCurrentUser(token) {
  return apiRequest('/auth/me', { token })
}

export function logoutRequest(token) {
  return apiRequest('/auth/logout', { method: 'POST', token })
}
```

frontend/src/api/changeRequests.js

64 lines

```
import { apiRequest } from './client'

export function createChangeRequest(payload, token) {
  return apiRequest('/api/change-requests', { method: 'POST', body: payload, token })
}

export function listChangeRequests(token, params = {}) {
  const query = new URLSearchParams()
  if (params.search) query.set('search', params.search)
  if (params.status) query.set('status', params.status)
  if (params.priority) query.set('priority', params.priority)
  if (params.assignedToMe) query.set('assigned_to_me', 'true')
  if (params.sort) query.set('sort', params.sort)
  if (params.limit !== null) query.set('limit', params.limit)
  if (params.offset !== null) query.set('offset', params.offset)
  const qs = query.toString()
  return apiRequest(`/api/change-requests${qs ? `?${qs}` : ''}`, { token })
}

export function getChangeRequest(id, token) {
  return apiRequest(`/api/change-requests/${id}`, { token })
}

// --- Module 12 Phase 2: editing, versioning, audit history -----

export function updateChangeRequest(id, payload, token) {
  return apiRequest(`/api/change-requests/${id}`, { method: 'PUT', body: payload, token })
}

export function getChangeRequestHistory(id, token) {
  return apiRequest(`/api/change-requests/${id}/history`, { token })
}

export function getChangeRequestVersions(id, token) {
  return apiRequest(`/api/change-requests/${id}/versions`, { token })
}

export function getChangeRequestVersion(id, versionNumber, token) {
  return apiRequest(`/api/change-requests/${id}/versions/${versionNumber}`, { token })
}

export function compareChangeRequestVersions(id, fromVersion, toVersion, token) {
  const query = new URLSearchParams({ from: fromVersion, to: toVersion })
  return apiRequest(`/api/change-requests/${id}/compare?${query.toString()}`, { token })
}

// --- Module 12 Phase 3: status workflow + assignments -----

export function changeChangeRequestStatus(id, payload, token) {
  return apiRequest(`/api/change-requests/${id}/status`, { method: 'PUT', body: payload, token })
}

export function listChangeRequestAssignments(id, token) {
  return apiRequest(`/api/change-requests/${id}/assignments`, { token })
}

export function createChangeRequestAssignment(id, payload, token) {
  return apiRequest(`/api/change-requests/${id}/assignments`, { method: 'POST', body: payload, token })
}

export function deleteChangeRequestAssignment(id, assignmentId, token) {
  return apiRequest(`/api/change-requests/${id}/assignments/${assignmentId}`, { method: 'DELETE', token })
}
```

frontend/src/api/client.js

53 lines

```
// Talk to the backend directly on :8000 (override with VITE API_BASE_URL).
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000'

export class ApiError extends Error {
  // `detail` (Module 20) carries the backend's raw `detail` value whenever
  // it's more than a plain string - e.g. the version-mismatch report error,
  // which is a {message, cr_version, last_analyzed_version, can_reanalyze}
  // object - so a caller that needs those specific fields doesn't have to
  // re-parse `message` back out of a sentence. Most callers still just
  // read `.message` and never look at `.detail` at all.
  constructor(message, status, detail) {
    super(typeof message === 'string' ? message : 'Request failed.')
    this.name = 'ApiError'
    this.status = status
    this.detail = detail
  }
}

/** Thin fetch wrapper: JSON in, JSON out, attaches the bearer token when given, and
 * turns a non-2xx response into a thrown ApiError with the backend's error message. */
export async function apiRequest(path, { method = 'GET', body, token } = {}) {
  const headers = { 'Content-Type': 'application/json' }
  if (token) {
    headers.Authorization = `Bearer ${token}`
  }

  let response
  try {
    response = await fetch(`${API_BASE_URL}${path}`, {
      method,
      headers,
      body: body !== undefined ? JSON.stringify(body) : undefined,
    })
  } catch {
    throw new ApiError('Could not reach the backend. Is it running?', 0)
  }

  const isJson = response.headers.get('content-type')?.includes('application/json')
  const data = isJson ? await response.json() : null

  if (!response.ok) {
    const detail = data?.detail
    const message = Array.isArray(detail)
      ? detail.map((d) => d.msg).join(' ')
      : typeof detail === 'string'
        ? detail
        : detail?.message || `Request failed with status ${response.status}`
    throw new ApiError(message, response.status, detail)
  }

  return data
}
```

frontend/src/api/comments.js

27 lines

```
import { apiRequest } from './client'

// Module 12 Phase 5: the discussion thread on a change request - a flat
// two-level thread (top-level comments plus their direct replies, spec:
// "reply where practical"). See backend/app/api/change_requests.py's
// "comments" section.

export function listComments(id, token) {
  return apiRequest(`/api/change-requests/${id}/comments`, { token })
}

export function createComment(id, payload, token) {
  return apiRequest(`/api/change-requests/${id}/comments`, { method: 'POST', body: payload, token })
}

export function resolveComment(id, commentId, resolved, token) {
  return apiRequest(`/api/change-requests/${id}/comments/${commentId}/resolve`, {
    method: 'PUT',
    body: { resolved },
    token,
  })
}

export function deleteComment(id, commentId, token) {
  return apiRequest(`/api/change-requests/${id}/comments/${commentId}`, { method: 'DELETE', token })
}
```

frontend/src/api/dashboard.js

22 lines

```
import { apiRequest } from './client'

export function getDashboardSummary(token) {
  return apiRequest('/dashboard/summary', { token })
}

// Clicking a dashboard tile: the change requests actually counted for that
// tile (see backend/app/api/dashboard.py's own drill-down endpoint - same
// classification the tile's own number comes from, never a second count).
// `options` mirrors the backend's own query params - dateFrom/dateTo filter
// by the change request's created date, page/pageSize paginate (pageSize is
// capped server-side at 10, matching the popup's rows-per-page dropdown).
export function getDashboardDrillDown(metric, token, options = {}) {
  const { dateFrom, dateTo, page, pageSize } = options
  const params = new URLSearchParams({ metric })
  if (dateFrom) params.set('date_from', dateFrom)
  if (dateTo) params.set('date_to', dateTo)
  if (page) params.set('page', String(page))
  if (pageSize) params.set('page_size', String(pageSize))
  return apiRequest(`/dashboard/drill-down?${params.toString()}`, { token })
}
```

```

import { apiRequest, API_BASE_URL, ApiError } from './client'

// Module 16 (Project Knowledge Base & RAG): upload, list, chunk-preview,
// embed, search, and archive knowledge-base documents via
// backend/app/api/knowledge.py. Mirrors the thin-wrapper shape of every
// other endpoint module here (see repository.js) - the one exception is
// uploadKnowledgeDocument, which can't reuse apiRequest() since that
// helper always JSON-encodes its body and sets a JSON Content-Type;
// uploading a file needs a multipart/form-data body with no explicit
// Content-Type header (the browser sets its own, boundary included).

/** Every non-archived knowledge document, newest first. Pass
 * includeArchived=true to also see archived ones (e.g. for an "show
 * archived" toggle). */
export function listKnowledgeDocuments(token, { includeArchived = false } = {}) {
  const query = includeArchived ? '?includeArchived=true' : ''
  return apiRequest(`/api/knowledge/documents${query}`, { token })
}

/** One knowledge document's own metadata (status, chunk_count, etc). */
export function getKnowledgeDocument(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}`, { token })
}

/** Uploads a file (a browser File/Blob object) as a new knowledge
 * document - the backend saves it, then immediately parses+chunks it, so
 * the response already reflects "ready" or "failed" rather than always
 * coming back "uploaded". Thrown errors are ApiError, same shape as
 * apiRequest's, so callers can handle both the same way. */
export async function uploadKnowledgeDocument(file, token) {
  const formData = new FormData()
  formData.append('file', file)

  let response
  try {
    response = await fetch(`${API_BASE_URL}/api/knowledge/documents`, {
      method: 'POST',
      headers: token ? { Authorization: `Bearer ${token}` } : undefined,
      body: formData,
    })
  } catch {
    throw new ApiError('Could not reach the backend. Is it running?', 0)
  }

  const isJson = response.headers.get('content-type')?.includes('application/json')
  const data = isJson ? await response.json() : null

  if (!response.ok) {
    const detail = data?.detail
    const message = Array.isArray(detail)
      ? detail.map(d => d.msg).join(' ')
      : detail || `Upload failed with status ${response.status}`
    throw new ApiError(message, response.status)
  }

  return data
}

/** Retries parsing+chunking for a document that came back "failed" - the
 * explicit retry action, never automatic. */
export function reprocessKnowledgeDocument(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}/reprocess`, { method: 'POST', token })
}

/** Embeds (or re-embeds) every chunk of a document via the configured AI
 * provider - a separate, explicit, network-calling step from upload. */
export function embedKnowledgeDocument(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}/embed`, { method: 'POST', token })
}

/** Marks a document archived - it drops out of the default list and out
 * of search/analysis retrieval, but its file/chunks/embeddings are kept
 * intact so unarchiving is instant. */
export function archiveKnowledgeDocument(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}/archive`, { method: 'POST', token })
}

/** Reverses archiveKnowledgeDocument. */
export function unarchiveKnowledgeDocument(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}/unarchive`, { method: 'POST', token })
}

/** Every chunk belonging to one document, in original document order,
 * each reporting is embedded (never the raw vector itself) - lets the
 * Knowledge Base page preview a document's parsed content. */
export function getKnowledgeDocumentChunks(id, token) {
  return apiRequest(`/api/knowledge/documents/${id}/chunks`, { token })
}

/** Ranks every already-embedded, non-archived chunk by similarity to
 * 'query' - the same manual search a person can run themselves, distinct
 * from the automatic retrieval CR analysis does internally. */
export function searchKnowledgeBase(query, token, { topK = 5 } = {}) {
  const params = new URLSearchParams({ q: query, top_k: String(topK) })
  return apiRequest(`/api/knowledge/search?${params.toString()}`, { token })
}

```


frontend/src/api/myWork.js

41 lines

```
import { apiRequest } from './client'

// Module 18 Phase 3/4 (Notifications, My Work & Personal Engineering
// Queue): "what's on my plate right now" - see backend/app/api/my_work.py.
// Every one of these is scoped to the signed-in user automatically (the
// backend reads it off the auth token) - there's no user id parameter
// anywhere here.

export function getMyWorkSummary(token) {
  return apiRequest('/api/my-work/summary', { token })
}

export function getMyChangeRequests(token) {
  return apiRequest('/api/my-work/change-requests', { token })
}

export function getMyApprovals(token, { all = false } = {}) {
  const query = all ? '?status=all' : ''
  return apiRequest(`/api/my-work/approvals${query}`, { token })
}

export function getMyReviews(token) {
  return apiRequest('/api/my-work/reviews', { token })
}

export function getMyAssignments(token) {
  return apiRequest('/api/my-work/assignments', { token })
}

export function getChangesRequestedFromMe(token) {
  return apiRequest('/api/my-work/changes-requested', { token })
}

export function getMyMentions(token) {
  return apiRequest('/api/my-work/mentions', { token })
}

export function getOverdueItems(token) {
  return apiRequest('/api/my-work/overdue', { token })
}
```

frontend/src/api/notifications.js

25 lines

```
import { apiRequest } from './client'

// Module 12 Phase 5: a user's own in-app notification inbox. See
// backend/app/api/notifications.py.

export function listNotifications(token, { unreadOnly = false, limit } = {}) {
  const params = new URLSearchParams()
  if (unreadOnly) params.set('unread_only', 'true')
  if (limit) params.set('limit', String(limit))
  const query = params.toString()
  return apiRequest(`/api/notifications${query ? `?${query}` : ''}`, { token })
}

export function getUnreadNotificationCount(token) {
  return apiRequest(`/api/notifications/unread-count`, { token })
}

export function markNotificationRead(notificationId, token) {
  return apiRequest(`/api/notifications/${notificationId}/read`, { method: 'PUT', token })
}

export function markAllNotificationsRead(token) {
  return apiRequest(`/api/notifications/read-all`, { method: 'PUT', token })
}
```

```

import { ApiError, API_BASE_URL, apiRequest } from './client'

// Module 24: Reports section. Every "report" here is a computed view over
// data that already exists elsewhere (see backend/app/services/
// report_registry.py) - there's no separate reports database to talk to,
// and no "generate" call either: a report is recorded automatically the
// moment an analysis completes (see app/api/change_requests.py::
// analyze_change_request on the backend), so this file only ever reads.

export function listReports(token, params = {}) {
  const query = new URLSearchParams()
  if (params.search) query.set('search', params.search)
  if (params.status) query.set('status', params.status)
  if (params.risk) query.set('risk', params.risk)
  if (params.version !== null && params.version !== '') query.set('version', params.version)
  if (params.generatedBy !== null && params.generatedBy !== '') query.set('generated_by', params.generatedBy)
  if (params.dateFrom) query.set('date_from', params.dateFrom)
  if (params.dateTo) query.set('date_to', params.dateTo)
  const qs = query.toString()
  return apiRequest(`/api/reports${qs} ? `?${qs}` : ''`, { token })
}

export function getReportStats(token) {
  return apiRequest(`/api/reports/stats`, { token })
}

export function getReport(reportId, token) {
  return apiRequest(`/api/reports/${reportId}`, { token })
}

export function listReportsForChangeRequest(changeRequestId, token) {
  return apiRequest(`/api/change-requests/${changeRequestId}/reports`, { token })
}

/** Fetches the PDF for an already-generated report row and triggers a
 * browser download - mirrors app/api/analysis.js::downloadReport exactly
 * (not built on apiRequest(), since this response is binary, not JSON). */
export async function downloadReportById(reportId, token, fallbackFilename) {
  let response
  try {
    response = await fetch(`${API_BASE_URL}/api/reports/${reportId}/download`, {
      headers: token ? { Authorization: `Bearer ${token}` } : {},
    })
  } catch {
    throw new ApiError('Could not reach the backend. Is it running?', 0)
  }

  if (!response.ok) {
    let detail
    try {
      detail = (await response.json())?.detail
    } catch {
      detail = null
    }
    const message = Array.isArray(detail)
      ? detail.map((d) => d.msg).join(' ')
      : typeof detail === 'string'
        ? detail
        : detail?.message || `Report download failed with status ${response.status}`
    throw new ApiError(message, response.status, detail)
  }

  const blob = await response.blob()
  const disposition = response.headers.get('content-disposition') || ''
  const match = disposition.match(/filename="?([^\"]+)"?/)
  const filename = match ? match[1] : fallbackFilename || `report-${reportId}.pdf`

  const url = URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = filename
  document.body.appendChild(link)
  link.click()
  link.remove()
  URL.revokeObjectURL(url)
}

```

frontend/src/api/repository.js

43 lines

```
import { apiRequest } from './client'

// Module 15 (Repository Intelligence): scanning "the repository" itself
// (not scoped to any one change request - see
// backend/app/models/repository_scan.py's docstring) via
// backend/app/api/repository.py, and matching one change request's
// current analysis against whatever the most recent scan indexed via the
// repository-findings routes in backend/app/api/change_requests.py.
// Mirrors the shape of every other endpoint module here - thin
// apiRequest wrappers, no extra logic.

/** Scans the app's configured repository root and records a new
 * RepositoryScan - never touches or deletes any prior scan. */
export function triggerRepositoryScan(token) {
  return apiRequest('/api/repository/scan', { method: 'POST', token })
}

/** The most recent repository scan, with its full indexed-file list.
 * 404s (as an ApiError with status 404) if no scan has ever been run -
 * callers that treat "no scan yet" as a normal, not-yet-happened state
 * should catch that rather than surface it as a page-level error. */
export function getLatestRepositoryScan(token) {
  return apiRequest('/api/repository/latest', { token })
}

/** Matches this change request's current analysis against the most recent
 * repository scan and persists the result - "which source files may
 * actually be affected by this change request?" Re-running this replaces
 * this exact (change request, analysis, scan) triple's prior findings
 * rather than piling up duplicates. */
export function triggerRepositoryMatch(id, token) {
  return apiRequest(`/api/change-requests/${id}/repository-findings`, { method: 'POST', token })
}

/** Whatever repository findings already exist for this change request's
 * current analysis - empty (never 404) if a match hasn't been run yet.
 * Each finding reports its outdated/outdated_reasons: whether the change
 * request has since been edited, a newer analysis has run, or a newer
 * repository scan is available. */
export function getRepositoryFindings(id, token) {
  return apiRequest(`/api/change-requests/${id}/repository-findings`, { token })
}
```

frontend/src/api/users.js

7 lines

```
import { apiRequest } from './client'  
  
// Module 12 Phase 3: a simple user directory for the assignment picker.  
export function listUsers(token) {  
  return apiRequest('/api/users', { token })  
}
```

```

import { useEffect, useRef, useState } from 'react'
import {
  getUnreadNotificationCount,
  listNotifications,
  markAllNotificationsRead,
  markNotificationRead,
} from '../api/notifications'
import { useAuth } from '../context/AuthContext'
import BackendStatusBadge from '../BackendStatusBadge'
import '../styles/workflow.css'
import '../header.css'

// Module 21: 'settings' is the Administration section (AdminPage.jsx) -
// admin-only, so the render below filters it out of the list for every
// other role. Left in this shared array (rather than a separate admin-only
// list) so AuthenticatedHome's activeLabel lookup keeps working unchanged.
export const NAV_ITEMS = [
  { key: 'dashboard', label: 'Dashboard' },
  { key: 'change-requests', label: 'Change Requests' },
  { key: 'new-analysis', label: 'New Analysis' },
  { key: 'my-work', label: 'My Work' },
  // Module 24: real page (ReportsPage.jsx) - statistics, search/filter, and
  // the report list. No manual "generate" step: a report appears here
  // automatically the moment a change request is analyzed. Placed right
  // after My Work, matching the order the project owner suggested.
  { key: 'reports', label: 'Reports' },
  { key: 'analytics', label: 'Analytics' },
  { key: 'notifications', label: 'Notifications' },
  { key: 'knowledge-base', label: 'Knowledge Base' },
  { key: 'settings', label: 'Settings' },
]

// Module 23: 'Repository' and 'Reports' used to be separate nav items here,
// but neither was ever its own page - both landed on the generic "built in
// a later module" placeholder even though the real features exist
// elsewhere (Repository Intelligence is a tab inside a change request's
// own Analysis Dashboard; the report is downloaded via the "Download
// Report" button on that same Analysis Dashboard page). Removed rather
// than left as dead links a demo could click into. See DEMO.md for where
// each actually is.

// Module 18 Phase 6's own recent-notifications dropdown only ever shows a
// handful - the full, paginated inbox is still the Notifications page.
const RECENT_LIMIT = 8

function initials(name) {
  if (!name) return '?'
  const parts = name.trim().split(/\s+/)
  const first = parts[0]?.[0] ?? ''
  const last = parts.length > 1 ? parts[parts.length - 1][0] : ''
  return (first + last).toUpperCase()
}

function formatRelative(isoString) {
  if (!isoString) return ''
  const then = new Date(isoString).getTime()
  if (Number.isNaN(then)) return ''
  const diffMs = Date.now() - then
  const minutes = Math.round(diffMs / 60000)
  if (minutes < 1) return 'just now'
  if (minutes < 60) return `${minutes}m ago`
  const hours = Math.round(minutes / 60)
  if (hours < 24) return `${hours}h ago`
  const days = Math.round(hours / 24)
  if (days < 7) return `${days}d ago`
  return new Date(isoString).toLocaleDateString(undefined, { month: 'short', day: 'numeric' })
}

/** Left-hand navigation sidebar for the authenticated app. (This used to be
 * a top header bar - kept the file/component name so nothing importing it
 * had to change, only what it renders.) Brand + notification bell at top,
 * primary nav in the middle, backend status + user/logout pinned at the
 * bottom. No router - `activeView` / `onNavigate` are plain local state
 * from the app shell.
 *
 * Module 18 Phase 6: the bell button next to the logo is the spec's
 * "notification center in the application header" - unread count, recent
 * notifications, mark as read, mark all as read, link to CR - all reusing
 * the same GET /api/notifications, /unread-count, PUT /{id}/read and
 * PUT /read-all endpoints the full Notifications page already uses (no
 * second notification pathway). The "Notifications" nav item below it
 * still opens that full, paginated page - the bell is only a quick-glance
 * dropdown over the most recent few. */
export default function AppHeader({ activeView, onNavigate, onOpenChangeRequest, user, onLogout }) {
  const { token } = useAuth()
  const [unreadCount, setUnreadCount] = useState(0)
  const [notifOpen, setNotifOpen] = useState(false)
  const [recentNotifications, setRecentNotifications] = useState(null)
  const [recentError, setRecentError] = useState(null)
  const [loadingRecent, setLoadingRecent] = useState(false)
  const [markingAll, setMarkingAll] = useState(false)
  const dropdownRef = useRef(null)

  // Module 12 Phase 5: a lightweight poll for the notification badge - not
  // worth a websocket/SSE setup for a hackathon, and refreshes often enough
  // (every 30s, plus whenever the Notifications page itself is left) to
  // feel current.
  useEffect(() => {
    if (!token) return undefined
    let cancelled = false
    function poll() {
      getUnreadNotificationCount(token)
        .then((data) => {
          if (!cancelled) setUnreadCount(data.count || 0)
        })
        .catch(() => {})
    }
    poll()
    const handle = setInterval(poll, 30000)
    return () => {
      cancelled = true
      clearInterval(handle)
    }
  }, [token, activeView])

  // Close the dropdown on an outside click, same convention any menu/
  // popover in this app would use.
  useEffect(() => {
    if (!notifOpen) return undefined
    function handleClick(event) {
      if (dropdownRef.current && !dropdownRef.current.contains(event.target)) {
        setNotifOpen(false)
      }
    }
  })
}

```

```

document.addEventListener('mousedown', handleDocumentClick)
return () => document.removeEventListener('mousedown', handleDocumentClick)
}, [notifOpen])

function loadRecent() {
  setLoadingRecent(true)
  setRecentError(null)
  listNotifications(token, { limit: RECENT_LIMIT })
    .then(setRecentNotifications)
    .catch((err) => setRecentError(err.message || 'Could not load notifications.'))
    .finally(() => setLoadingRecent(false))
}

function toggleNotifOpen() {
  setNotifOpen((prev) => {
    const next = !prev
    if (next) loadRecent()
    return next
  })
}

function handleOpenNotification(notification) {
  if (!notification.is_read) {
    markNotificationRead(notification.id, token)
    .then((updated) => {
      setRecentNotifications((prev) => (prev || []).map((n) => (n.id === updated.id ? updated : n)))
      setUnreadCount((count) => Math.max(0, count - 1))
    })
    .catch(() => {})
  }
  setNotifOpen(false)
  if (notification.change_request_id != null) {
    onOpenChangeRequest?.(notification.change_request_id)
  } else {
    onNavigate('notifications')
  }
}

async function handleMarkAllRead(event) {
  event.stopPropagation()
  setMarkingAll(true)
  try {
    await markAllNotificationsRead(token)
    setUnreadCount(0)
    setRecentNotifications((prev) => (prev || []).map((n) => ({ ...n, is_read: true })))
  } catch {
    // best-effort - nothing to show beyond the list not updating
  } finally {
    setMarkingAll(false)
  }
}

function handleViewAll(event) {
  event.stopPropagation()
  setNotifOpen(false)
  onNavigate('notifications')
}

return (
  <aside className="app-sidebar">
    <div className="app-sidebar-brand">
      <span className="app-sidebar-logo-mark">AI</span>
      <span className="app-sidebar-logo-word">Change Request Analyzer</span>
    </div>
    <div className="notif-bell-wrap" ref={dropdownRef}>
      <button
        type="button"
        className="notif-bell-btn"
        onClick={toggleNotifOpen}
        aria-haspopup="true"
        aria-expanded={notifOpen}
        aria-label={unreadCount > 0 ? `Notifications, ${unreadCount} unread` : 'Notifications'}
      >
        <span aria-hidden="true">▲</span>
        {unreadCount > 0 && (
          <span className="notif-bell-badge">{unreadCount > 99 ? '99+' : unreadCount}</span>
        )}
      </button>
      {notifOpen && (
        <div className="notif-dropdown" role="menu">
          <div className="notif-dropdown-header">
            <strong>Notifications</strong>
            <button
              type="button"
              className="cr-btn cr-btn--ghost cr-btn--small"
              onClick={handleMarkAllRead}
              disabled={markingAll || unreadCount === 0}
            >
              {markingAll ? 'Marking...' : 'Mark all read'}
            </button>
          </div>
          {loadingRecent && <p className="notif-dropdown-status">Loading</p>
            {recentError && <p className="notif-dropdown-status notif-dropdown-status--error">{recentError}</p>
            {loadingRecent && !recentError && recentNotifications && recentNotifications.length === 0 && (
              <p className="notif-dropdown-status">You're all caught up - no notifications yet.</p>
            )}
            {!loadingRecent && !recentError && recentNotifications && recentNotifications.length > 0 && (
              <ul className="notif-dropdown-list">
                {recentNotifications.map((n) => (
                  <li key={n.id}>
                    <button
                      type="button"
                      className={`notif-dropdown-item${n.is_read ? '' : ' notif-dropdown-item--unread'}`}
                      onClick={() => handleOpenNotification(n)}
                    >
                      <span className="notif-dropdown-item-top">
                        <span className="notif-dropdown-item-title">{n.title}</span>
                        {n.is_read && <span className="notification-dot" aria-hidden="true" />
                      </span>
                      <span className="notif-dropdown-item-message">{n.message}</span>
                      <span className="notif-dropdown-item-time">{formatRelative(n.created_at)}</span>
                    </button>
                  </li>
                )})}
              </ul>
            )}
            <button type="button" className="notif-dropdown-footer" onClick={handleViewAll}>
              View all notifications
            </button>
          </div>
        )}
      </div>
    </div>
    <nav className="app-sidebar-nav" aria-label="Primary">
      {NAV_ITEMS.filter((item) => item.key !== 'settings' || user?.role === 'admin').map((item) => (

```

```

    <button
      key={item.key}
      type="button"
      className={`app-sidebar-link${activeView === item.key ? ' app-sidebar-link--active' : ''}`}
      onClick={() => onNavigate(item.key)}
    >
      {item.label}
      {item.key === 'notifications' && unreadCount > 0 && (
        <span className="app-sidebar-badge">unreadCount > 99 ? '99+' : unreadCount</span>
      )}
    </button>
  )}
</nav>
<div className="app-sidebar-footer">
  <BackendStatusBadge />
  <div className="app-sidebar-user">
    <span className="app-sidebar-avatar">{initials(user?.name)}</span>
    <div className="app-sidebar-user-info">
      <span className="app-sidebar-user-name">{user?.name}</span>
      <span className="app-sidebar-user-role">{user?.role}</span>
    </div>
  </div>
  <button type="button" className="app-sidebar-logout" onClick={onLogout}>
    Log out
  </button>
</div>
</aside>
)
}

```


frontend/src/components/AuthLayout.jsx

21 lines

```
import './auth-layout.css'

export default function AuthLayout({ children }) {
  return (
    <div className="auth-shell">
      <div className="auth-brand-panel">
        <div className="auth-brand-content">
          <div className="auth-logo">
            <span className="auth-logo-mark">AI</span>
            <span className="auth-logo-word">Change Request Analyzer</span>
          </div>
          <p className="auth-tagline">Turn ambiguous change requests into engineering decisions.</p>
        </div>
      </div>
      <div className="auth-form-panel">
        <div className="auth-form-card">{children}</div>
      </div>
    </div>
  )
}
```

frontend/src/components/BackendStatusBadge.jsx

26 lines

```
import { useEffect, useState } from 'react'
import { API_BASE_URL } from '../api/client'

/** Small connectivity indicator - carries forward the health-check the
 * project started with, now as a badge instead of the whole page. */
export default function BackendStatusBadge() {
  const [status, setStatus] = useState('checking') // 'checking' | 'ok' | 'error'

  useEffect(() => {
    fetch(`${API_BASE_URL}/health`)
      .then((res) => (res.ok ? res.json() : Promise.reject(new Error(`status ${res.status}`))))
      .then(() => setStatus('ok'))
      .catch(() => setStatus('error'))
  }, [])

  const label =
    status === 'ok' ? 'Backend connected' : status === 'error' ? 'Backend unreachable' : 'Checking backend...'

  return (
    <span className={`status-badge status-badge--${status}`}>
      <span className="status-dot" />
      {label}
    </span>
  )
}
```

```

import { useEffect, useRef, useState } from 'react'
import { analyzeChangeRequest } from '../api/analysis'
import {
  cancelApprovalRequest,
  createApprovalRequest,
  getRecommendedApprovals,
  listApprovals,
  respondToApproval,
} from '../api/approvals'
import {
  changeChangeRequestStatus,
  compareChangeRequestVersions,
  createChangeRequestAssignment,
  deleteChangeRequestAssignment,
  getChangeRequest,
  getChangeRequestHistory,
  getChangeRequestVersions,
  listChangeRequestAssignments,
} from '../api/changeRequests'
import { createComment, deleteComment, listComments, resolveComment } from '../api/comments'
import { listUsers } from '../api/users'
import { useAuth } from '../context/AuthContext'
// Reuses the .badge classes (dashboard.css) and the .cr-btn / breadcrumb /
// .cr-analyze-notice classes (change-request-form.css, Module 4).
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'
import '../styles/workflow.css'

const STATUS_LABELS = {
  draft: 'Draft',
  pending_analysis: 'Pending Analysis',
  analyzing: 'Analyzing',
  requires_clarification: 'Requires Clarification',
  completed: 'Completed',
  approved: 'Approved',
}

const STATUS_CLASS = {
  draft: 'badge--gray',
  pending_analysis: 'badge--indigo',
  analyzing: 'badge--blue',
  requires_clarification: 'badge--amber',
  completed: 'badge--teal',
  approved: 'badge--green',
}

const PRIORITY_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const PRIORITY_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
}

// Mirrors app/services/workflow_rules.py::ROLE_LABELS - kept in sync
// manually, same as the other label maps in this file.
const ROLE_OPTIONS = [
  { value: 'requester', label: 'Requester' },
  { value: 'owner', label: 'Owner' },
  { value: 'technical_lead', label: 'Technical Lead' },
  { value: 'reviewer', label: 'Reviewer' },
  { value: 'approver', label: 'Approver' },
  { value: 'security_reviewer', label: 'Security Reviewer' },
  { value: 'qa_owner', label: 'QA Owner' },
  { value: 'implementation_owner', label: 'Implementation Owner' },
]

// Module 12 Phase 4: mirrors app/services/workflow_rules.py:
// APPROVAL TYPE LABELS - what kind of sign-off can be requested.
const APPROVAL_TYPE_OPTIONS = [
  { value: 'technical', label: 'Technical' },
  { value: 'security', label: 'Security' },
  { value: 'product', label: 'Product' },
  { value: 'engineering_manager', label: 'Engineering Manager' },
  { value: 'director', label: 'Director' },
  { value: 'qa', label: 'QA' },
  { value: 'dba', label: 'DBA' },
  { value: 'release', label: 'Release' },
  { value: 'general', label: 'General' },
]

// Mirrors app/services/workflow_rules.py::APPROVAL_STATUS_LABELS.
const APPROVAL_STATUS_LABELS = {
  pending: 'Pending',
  approved: 'Approved',
  rejected: 'Rejected',
  changes_requested: 'Changes Requested',
  cancelled: 'Cancelled',
}

const APPROVAL_STATUS_CLASS = {
  pending: 'badge--indigo',
  approved: 'badge--green',
  rejected: 'badge--red',
  changes_requested: 'badge--amber',
  cancelled: 'badge--gray',
}

// A response can only move a pending approval to one of these three - see
// app/services/workflow_rules.py::APPROVAL_RESPONSE_STATUSES /
// APPROVAL REASON REQUIRED (the last two require a comment).
const APPROVAL_RESPONSE_OPTIONS = [
  { value: 'approved', label: 'Approve' },
  { value: 'rejected', label: 'Reject' },
  { value: 'changes_requested', label: 'Request Changes' },
]
const APPROVAL_REASON_REQUIRED = new Set(['rejected', 'changes_requested'])

const RECOMMENDATION_LABELS = {
  approve: 'Approve',
  approve_with_conditions: 'Approve with conditions',
  requires_clarification: 'Requires clarification',
  needs_more_info: 'Needs more info',
  reject: 'Reject',
}

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleDateString(undefined, { year: 'numeric', month: 'short', day: 'numeric' })
  } catch {
    return isoString
  }
}

```

```

}

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

const HISTORY_ACTION_VERBS = {
  created: 'created this change request',
  field_changed: null, // rendered specially below (shows the field/old/new)
  version_created: null,
  ai_analysis_completed: null,
  ai_analysis_invalidated: null,
  status_changed: 'changed the status',
  assigned: 'assigned a team member',
  unassigned: 'removed a team member',
  approval_requested: 'requested an approval',
  approved: 'approved this change request',
  rejected: 'rejected this change request',
  changes_requested: 'requested changes',
  approval_invalidated: 'invalidated an approval',
  approval_cancelled: 'cancelled an approval request',
  comment_added: 'added a comment',
  mentioned: 'mentioned someone',
  report_generated: 'generated a report',
  reminder_sent: 'sent a reminder',
}

/** One line of the Activity/Audit timeline (Module 12 Phase 2, spec section
 * 17) - who did what, when, and (for field-level edits) the real old/new
 * value, never a vague "updated" line. */
function HistoryLine({ event, currentUserId }) {
  const who = event.actor_label || (event.user_id === currentUserId ? 'You' : `User #${event.user_id ?? '?'}`)

  let sentence
  if (event.action === 'field_changed') {
    sentence = (
      <strong>{who}</strong> changed <strong>{event.field_name}</strong>
    )
  } else if (event.action === 'version_created') {
    sentence = (
      <strong>{who}</strong> saved a new version
    )
  } else if (event.action === 'ai_analysis_completed') {
    sentence = (
      <strong>{who}</strong> completed an AI analysis
    )
  } else if (event.action === 'ai_analysis_invalidated') {
    sentence = (
      <strong>{who}</strong> - the previous AI analysis is now outdated
    )
  } else {
    sentence = (
      <strong>{who}</strong> {HISTORY_ACTION_VERBS[event.action] || event.action}
    )
  }

  return (
    <li className="history-item">
      <span className="history-dot" aria-hidden="true" />
      <div className="history-body">
        <p className="history-line">{sentence}</p>
        {(event.old_value || event.new_value) && (
          <p className="history-value-change">
            {event.old_value && <span className="from-value">{event.old_value}</span>
              {event.old_value && event.new_value && ' → '
                {event.new_value && <span className="to-value">{event.new_value}</span>
              }
            }
          </p>
        )}
        {event.reason && <p className="history-value-change">Reason: {event.reason}</p>
        <p className="history-meta">
          {formatDateTime(event.created_at)}
          {event.version_number ? ` · v${event.version_number}` : ''}
        </p>
      </div>
    </li>
  )
}

/** A plain <textarea> that pops up a filtered list of people right below
 * it whenever the caret sits inside an "@mention" in progress - arrow
 * keys + Enter/Tab (or a click) insert "@Full Name" at that spot, no
 * need to type the exact name from memory. Mentions are still just plain
 * text either way - the backend (app/services/mentions.py) parses them
 * back out of the comment body at write time, so typing "@Full Name" by
 * hand works exactly the same as picking it from this list. */
function MentionTextarea({ id, value, onChange, users, placeholder, rows }) {
  const textareaRef = useRef(null)
  const [suggestions, setSuggestions] = useState([])
  const [mentionStart, setMentionStart] = useState(null)
  const [highlightIndex, setHighlightIndex] = useState(0)

  function refreshSuggestions(text, cursorPos) {
    const upToCursor = text.slice(0, cursorPos)
    const lastAt = upToCursor.lastIndexOf('@')
    const query = lastAt === -1 ? null : upToCursor.slice(lastAt + 1)
    // Not mid-mention if there's no "@" before the caret, or the text
    // since that "@" already moved on (a newline, or another "@").
    if (query === null || query.includes('\n') || query.includes('@')) {
      setSuggestions([])
      setMentionStart(null)
      return
    }
    const lowerQuery = query.toLowerCase()
    const matches = (users || []).filter((u) => u.name.toLowerCase().includes(lowerQuery)).slice(0, 6)
    setSuggestions(matches)
    setMentionStart(matches.length > 0 ? lastAt : null)
    setHighlightIndex(0)
  }

  function chooseSuggestion(person) {

```

```

    if (mentionStart === null || !textareaRef.current) return
    const cursorPos = textareaRef.current.selectionStart
    const before = value.slice(0, mentionStart)
    const after = value.slice(cursorPos)
    const inserted = `@${person.name}`
    onChange(before + inserted + after)
    setSuggestions([])
    setMentionStart(null)
    requestAnimationFrame(() => {
      const pos = before.length + inserted.length
      textareaRef.current?.focus()
      textareaRef.current?.setSelectionRange(pos, pos)
    })
  }

  function handleKeyDown(event) {
    if (suggestions.length === 0) return
    if (event.key === 'ArrowDown') {
      event.preventDefault()
      setHighlightIndex((i) => (i + 1) % suggestions.length)
    } else if (event.key === 'ArrowUp') {
      event.preventDefault()
      setHighlightIndex((i) => (i - 1 + suggestions.length) % suggestions.length)
    } else if (event.key === 'Enter' || event.key === 'Tab') {
      event.preventDefault()
      chooseSuggestion(suggestions[highlightIndex])
    } else if (event.key === 'Escape') {
      event.preventDefault()
      setSuggestions([])
      setMentionStart(null)
    }
  }

  return (
    <div className="mention-input-wrap">
      <textarea
        id={id}
        ref={textareaRef}
        rows={rows}
        placeholder={placeholder}
        value={value}
        onChange={(e) => {
          onChange(e.target.value)
          refreshSuggestions(e.target.value, e.target.selectionStart)
        }}
        onKeyDown={handleKeyDown}
        onSelect={(e) => refreshSuggestions(e.target.value, e.target.selectionStart)}
        onBlur={() => window.setTimeout(() => setSuggestions([], 150)}
      />
      {suggestions.length > 0 && (
        <ul className="mention-suggestions">
          {suggestions.map((person, index) => (
            <li key={person.id}>
              <button
                type="button"
                className={`mention-suggestions${index === highlightIndex ? ' mention-suggestion--active' : ''}`}
                onMouseDown={(e) => e.preventDefault()}
                onClick={() => chooseSuggestion(person)}
              >
                {person.name} <span className="version-row-meta">{person.email}</span>
              </button>
            </li>
          ))}
        </ul>
      )}
    </div>
  )
}

/** Master-detail panel shown in place of the table when a row is selected.
 * Module 5 built this as a read view of GET /api/change-requests/{id}.
 * Module 6 wired the "Analyze Change" button to the real AI Analysis Engine
 * (POST .../analyze) and added a compact summary card once an analysis
 * exists. Module 7 moved the full requirement-by-requirement breakdown out
 * to its own dedicated AnalysisDashboardPage - this panel now only shows
 * the compact summary plus a CTA into that page, instead of duplicating
 * every section inline. */
export default function ChangeRequestDetail({ id, onBack, onOpenDashboard, onEdit }) {
  const { token, user } = useAuth()
  const [detail, setDetail] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  // Module 12 Phase 6: the page grew a lot of sections (workflow status,
  // assignments, approvals, analysis, versions, history, comments) - tabs
  // group them so a first-time visitor isn't faced with one long stacked
  // page. Description/Details/Workflow Status/Assignments/Approvals/
  // Analysis stay together under Overview (they're all "about this
  // request as it stands today"); Versions + Activity History move under
  // one Versions & Activity tab (both are "what happened over time");
  // Comments gets its own tab since it's a separate, active conversation.
  const [activeTab, setActiveTab] = useState('overview')

  const [analyzing, setAnalyzing] = useState(false)
  const [analyzeError, setAnalyzeError] = useState(null)

  const [history, setHistory] = useState(null)
  const [historyError, setHistoryError] = useState(null)
  const [versions, setVersions] = useState(null)
  const [versionsError, setVersionsError] = useState(null)
  const [compareVersion, setCompareVersion] = useState(null) // version_number being compared to its predecessor
  const [compareResult, setCompareResult] = useState(null)
  const [compareLoading, setCompareLoading] = useState(false)

  // Module 12 Phase 3: status workflow + assignments
  const [statusChoice, setStatusChoice] = useState('')
  const [statusReason, setStatusReason] = useState('')
  const [statusSubmitting, setStatusSubmitting] = useState(false)
  const [statusError, setStatusError] = useState(null)

  const [assignments, setAssignments] = useState(null)
  const [assignmentsError, setAssignmentsError] = useState(null)
  const [users, setUsers] = useState(null)
  const [newAssigneeUserId, setNewAssigneeUserId] = useState('')
  const [newAssigneeRole, setNewAssigneeRole] = useState('reviewer')
  const [assignSubmitting, setAssignSubmitting] = useState(false)
  const [assignError, setAssignError] = useState(null)
  const [removingAssignmentId, setRemovingAssignmentId] = useState(null)

  // Module 12 Phase 4: approvals
  const [approvals, setApprovals] = useState(null)
  const [approvalsError, setApprovalsError] = useState(null)
  const [recommendedApprovals, setRecommendedApprovals] = useState([])
  const [newApprovalType, setNewApprovalType] = useState('')
  const [newApprovalUserId, setNewApprovalUserId] = useState('')
  const [newApprovalComment, setNewApprovalComment] = useState('')
  // Module 18 Phase 5: entirely optional, mirrors app/models/approval.py's
  // own due_date docstring - most approval requests still set nothing here.
  const [newApprovalDueDate, setNewApprovalDueDate] = useState('')
  const [requestingApproval, setRequestingApproval] = useState(false)
  const [requestApprovalError, setRequestApprovalError] = useState(null)
  const [cancellingApprovalId, setCancellingApprovalId] = useState(null)

```

```

// Which pending approval's Approve/Reject/Request-Changes form is open,
// plus that one form's own fields - only one open at a time.
const [respondingApprovalId, setRespondingApprovalId] = useState(null)
const [respondStatus, setRespondStatus] = useState('approved')
const [respondComment, setRespondComment] = useState('')
const [respondSubmitting, setRespondSubmitting] = useState(false)
const [respondError, setRespondError] = useState(null)

// Module 12 Phase 5: comments, mentions, notifications
const [comments, setComments] = useState(null)
const [commentsError, setCommentsError] = useState(null)
const [newCommentBody, setNewCommentBody] = useState('')
const [postingComment, setPostingComment] = useState(false)
const [postCommentError, setPostCommentError] = useState(null)
const [replyingToId, setReplyingToId] = useState(null)
const [replyBody, setReplyBody] = useState('')
const [postingReply, setPostingReply] = useState(false)
const [replyError, setReplyError] = useState(null)
const [commentActionError, setCommentActionError] = useState(null)
const [busyCommentId, setBusyCommentId] = useState(null)

function loadDetail() {
  setLoading(true)
  setError(null)
  return getChangeRequest(id, token)
    .then((data) => {
      setDetail(data)
      return data
    })
    .catch((err) => {
      setError(err.message || 'Could not load this change request.')
      return null
    })
    .finally(() => setLoading(false))
}

function loadHistoryAndVersions() {
  setHistoryError(null)
  getChangeRequestHistory(id, token)
    .then(setHistory)
    .catch((err) => setHistoryError(err.message || 'Could not load activity history.'))

  setVersionsError(null)
  getChangeRequestVersions(id, token)
    .then(setVersions)
    .catch((err) => setVersionsError(err.message || 'Could not load versions.'))
}

function loadAssignments() {
  setAssignmentsError(null)
  listChangeRequestAssignments(id, token)
    .then(setAssignments)
    .catch((err) => setAssignmentsError(err.message || 'Could not load assignments.'))
}

function loadApprovals() {
  setApprovalsError(null)
  listApprovals(id, token)
    .then(setApprovals)
    .catch((err) => setApprovalsError(err.message || 'Could not load approvals.'))

  // Purely informational (AI recommends, humans decide) - a failure here
  // just means the "AI recommends" hints don't show, not worth its own
  // error banner.
  getRecommendedApprovals(id, token)
    .then(setRecommendedApprovals)
    .catch(() => setRecommendedApprovals([]))
}

function loadComments() {
  setCommentsError(null)
  listComments(id, token)
    .then(setComments)
    .catch((err) => setCommentsError(err.message || 'Could not load comments.'))
}

useEffect(() => {
  setDetail(null)
  setAnalyzeError(null)
  setHistory(null)
  setVersions(null)
  setCompareVersion(null)
  setCompareResult(null)
  setStatusChoice('')
  setStatusReason('')
  setStatusError(null)
  setAssignments(null)
  setAssignError(null)
  setApprovals(null)
  setApprovalsError(null)
  setRecommendedApprovals([])
  setNewApprovalType('')
  setNewApprovalUserId('')
  setNewApprovalComment('')
  setNewApprovalDueDate('')
  setRequestApprovalError(null)
  setRespondingApprovalId(null)
  setRespondError(null)
  setComments(null)
  setCommentsError(null)
  setNewCommentBody('')
  setPostCommentError(null)
  setReplyingToId(null)
  setReplyBody('')
  setReplyError(null)
  setCommentActionError(null)
  setActiveTab('overview')
  loadDetail()
  loadHistoryAndVersions()
  loadAssignments()
  loadApprovals()
  loadComments()
  listUsers(token).then(setUsers).catch(() => setUsers([]))
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [id, token])

function handleToggleCompare(versionNumber) {
  if (compareVersion === versionNumber) {
    setCompareVersion(null)
    setCompareResult(null)
    return
  }
  setCompareVersion(versionNumber)
  setCompareResult(null)
  setCompareLoading(true)
  compareChangeRequestVersions(id, versionNumber - 1, versionNumber, token)
    .then(setCompareResult)
    .catch((err) => setCompareResult({ error: err.message || 'Could not compare these versions.' }))
    .finally(() => setCompareLoading(false))
}

```

```

async function handleAnalyze() {
  setAnalyzing(true)
  setAnalyzeError(null)
  try {
    await analyzeChangeRequest(id, token)
    await loadDetail()
    loadHistoryAndVersions()
  } catch (err) {
    setAnalyzeError(err.message || 'Analysis failed. Please try again.')
  } finally {
    setAnalyzing(false)
  }
}

const selectedTransition = detail?.available_transitions.find((t) => t.status === statusChoice)

async function handleStatusSubmit(event) {
  event.preventDefault()
  if (!statusChoice) return
  setStatusSubmitting(true)
  setStatusError(null)
  try {
    const payload = { status: statusChoice }
    if (statusReason.trim()) payload.reason = statusReason.trim()
    const updated = await changeChangeRequestStatus(id, payload, token)
    setDetail(updated)
    setStatusChoice('')
    setStatusReason('')
    loadHistoryAndVersions()
  } catch (err) {
    setStatusError(err.message || 'Could not update the status. Please try again.')
  } finally {
    setStatusSubmitting(false)
  }
}

async function handleAssignSubmit(event) {
  event.preventDefault()
  if (!newAssigneeUserId) return
  setAssignSubmitting(true)
  setAssignError(null)
  try {
    await createChangeRequestAssignment(
      id,
      { user_id: Number(newAssigneeUserId), role: newAssigneeRole },
      token
    )
    setNewAssigneeUserId('')
    loadAssignments()
    loadHistoryAndVersions()
  } catch (err) {
    setAssignError(err.message || 'Could not assign this user. Please try again.')
  } finally {
    setAssignSubmitting(false)
  }
}

async function handleRemoveAssignment(assignmentId) {
  setRemovingAssignmentId(assignmentId)
  setAssignError(null)
  try {
    await deleteChangeRequestAssignment(id, assignmentId, token)
    loadAssignments()
    loadHistoryAndVersions()
  } catch (err) {
    setAssignError(err.message || 'Could not remove this assignment. Please try again.')
  } finally {
    setRemovingAssignmentId(null)
  }
}

async function handleRequestApproval(event) {
  event.preventDefault()
  if (!newApprovalType || !newApprovalUserId) return
  setRequestingApproval(true)
  setRequestApprovalError(null)
  try {
    const payload = { approval_type: newApprovalType, approver_user_id: Number(newApprovalUserId) }
    if (newApprovalComment.trim()) payload.comment = newApprovalComment.trim()
    if (newApprovalDueDate) payload.due_date = new Date(newApprovalDueDate).toISOString()
    await createApprovalRequest(id, payload, token)
    setNewApprovalType('')
    setNewApprovalUserId('')
    setNewApprovalComment('')
    setNewApprovalDueDate('')
    loadApprovals()
    loadHistoryAndVersions()
  } catch (err) {
    setRequestApprovalError(err.message || 'Could not request this approval. Please try again.')
  } finally {
    setRequestingApproval(false)
  }
}

function openRespondForm(approval) {
  setRespondingApprovalId(approval.id)
  setRespondStatus('approved')
  setRespondComment('')
  setRespondError(null)
}

async function handleRespondSubmit(event, approvalId) {
  event.preventDefault()
  setRespondSubmitting(true)
  setRespondError(null)
  try {
    const payload = { status: respondStatus }
    if (respondComment.trim()) payload.comment = respondComment.trim()
    await respondToApproval(id, approvalId, payload, token)
    setRespondingApprovalId(null)
    setRespondComment('')
    loadApprovals()
    loadHistoryAndVersions()
  } catch (err) {
    setRespondError(err.message || 'Could not submit this response. Please try again.')
  } finally {
    setRespondSubmitting(false)
  }
}

async function handleCancelApproval(approvalId) {
  setCancellingApprovalId(approvalId)
  setApprovalsError(null)
  try {
    await cancelApprovalRequest(id, approvalId, token)
    loadApprovals()
    loadHistoryAndVersions()
  } catch (err) {
    setApprovalsError(err.message || 'Could not cancel this approval request. Please try again.')
  } finally {
    setCancellingApprovalId(null)
  }
}

```

```

}
}

async function handlePostComment(event) {
  event.preventDefault()
  if (!newCommentBody.trim()) return
  setPostingComment(true)
  setPostCommentError(null)
  try {
    await createComment(id, { body: newCommentBody.trim() }, token)
    setNewCommentBody('')
    loadComments()
    loadHistoryAndVersions()
  } catch (err) {
    setPostCommentError(err.message || 'Could not post this comment. Please try again.')
  } finally {
    setPostingComment(false)
  }
}

function openReplyForm(commentId) {
  setReplyingToId(commentId)
  setReplyBody('')
  setReplyError(null)
}

async function handlePostReply(event, parentId) {
  event.preventDefault()
  if (!replyBody.trim()) return
  setPostingReply(true)
  setReplyError(null)
  try {
    await createComment(id, { body: replyBody.trim(), parent_id: parentId }, token)
    setReplyingToId(null)
    setReplyBody('')
    loadComments()
    loadHistoryAndVersions()
  } catch (err) {
    setReplyError(err.message || 'Could not post this reply. Please try again.')
  } finally {
    setPostingReply(false)
  }
}

async function handleToggleResolved(comment) {
  setBusyCommentId(comment.id)
  setCommentActionError(null)
  try {
    await resolveComment(id, comment.id, !comment.resolved, token)
    loadComments()
  } catch (err) {
    setCommentActionError(err.message || 'Could not update this comment. Please try again.')
  } finally {
    setBusyCommentId(null)
  }
}

async function handleDeleteComment(comment) {
  setBusyCommentId(comment.id)
  setCommentActionError(null)
  try {
    await deleteComment(id, comment.id, token)
    loadComments()
    loadHistoryAndVersions()
  } catch (err) {
    setCommentActionError(err.message || 'Could not delete this comment. Please try again.')
  } finally {
    setBusyCommentId(null)
  }
}

const topLevelComments = (comments || []).filter((c) => c.parent_id == null)
function repliesTo(commentId) {
  return (comments || []).filter((c) => c.parent_id === commentId)
}
function mentionedNames(comment) {
  if (!comment.mentioned_user_ids.length || !users) return null
  return comment.mentioned_user_ids
    .map((uid) => users.find((u) => u.id === uid)?.name)
    .filter(Boolean)
    .join(', ')
}

return (
  <div className="cr-detail-shell">
    <nav className="breadcrumbs" aria-label="Breadcrumb">
      <button type="button" className="breadcrumb-link" onClick={onBack}>
        Change Requests
      </button>
      <span className="breadcrumb-separator"></span>
      <span className="breadcrumb-current">{detail ? `#${detail.id} : `#{id}`}</span>
    </nav>

    {loading && <p className="cr-list-status-text">Loading change request</p>
    {error && <p className="cr-list-status-text cr-list-status-text--error">{error}</p>

    {detail && (
      <div className="cr-detail-heading">
        <div>
          <h1>{detail.title}</h1>
          <div className="cr-detail-badges">
            <span className={badge ${STATUS_CLASS[detail.effective_status]} || 'badge--gray'}}>
              {STATUS_LABELS[detail.effective_status] || detail.effective_status}
            </span>
            <span className={badge ${PRIORITY_CLASS[detail.priority]} || 'badge--gray'}}>
              {PRIORITY_LABELS[detail.priority] || detail.priority} priority
            </span>
            <span className="badge version-badge">v{detail.current_version || 1}</span>
          </div>
        </div>
        <button type="button" className="cr-btn cr-btn--secondary" onClick={() => onEdit?.(id)}>
          Edit
        </button>
      </div>

      {detail.is_analysis_outdated && (
        <div className="outdated-banner">
          <span className="outdated-banner-text">
            <span className="outdated-banner-icon" aria-hidden="true">
              </span>
            <strong>AI analysis outdated.</strong> This request has changed since its last analysis ran -
              re-analyze to get results that reflect the current version.
            </span>
            <button
              type="button"
              className="cr-btn cr-btn--primary cr-btn--small"
              onClick={handleAnalyze}

```



```

        disabled={analyzing}
      >
        {analyzing ? 'Re-analyzing...' : 'Re-analyze'}
      </button>
    </div>
  )}

<div className="cr-detail-tabs" role="tablist" aria-label="Change request sections">
  <button
    type="button"
    role="tab"
    aria-selected={activeTab === 'overview'}
    className={cr-detail-tabs(activeTab === 'overview' ? 'cr-detail-tab--active' : '')}
    onClick={() => setActiveTab('overview')}
  >
    Overview
  </button>
  <button
    type="button"
    role="tab"
    aria-selected={activeTab === 'activity'}
    className={cr-detail-tabs(activeTab === 'activity' ? 'cr-detail-tab--active' : '')}
    onClick={() => setActiveTab('activity')}
  >
    Versions & Activity
  </button>
  <button
    type="button"
    role="tab"
    aria-selected={activeTab === 'comments'}
    className={cr-detail-tabs(activeTab === 'comments' ? 'cr-detail-tab--active' : '')}
    onClick={() => setActiveTab('comments')}
  >
    Comments {comments && comments.length > 0 ? ` (${comments.length})` : ''}
  </button>
</div>

{activeTab === 'overview' && (
  <
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Description</h2>
      <p className="cr-detail-text">{detail.description}</p>
      {detail.business_objective && (
        <
          <h2 className="cr-section-title cr-section-title--spaced">Business Objective</h2>
          <p className="cr-detail-text">{detail.business_objective}</p>
        </>
      )}
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Details</h2>
      <dl className="cr-detail-grid">
        <dt>Requested By</dt>
        <dd>{detail.requested_by || '-'}</dd>
        <dt>Target System</dt>
        <dd>{detail.target_system || '-'}</dd>
        <dt>Desired Deadline</dt>
        <dd>{formatDate(detail.desired_deadline)}</dd>
        <dt>Created</dt>
        <dd>{formatDate(detail.created_at)}</dd>
        <dt>Last Updated</dt>
        <dd>{formatDate(detail.updated_at)}</dd>
      </dl>
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Workflow Status</h2>
      <p className="cr-detail-text">
        Current status: <strong>{detail.status_label}</strong>
      </p>
      {detail.available_transitions.length > 0 ? (
        <form className="status-form" onSubmit={handleStatusSubmit}>
          {statusError && <p className="cr-form-error">{statusError}</p>}
          <div className="status-form-row">
            <select
              className="cr-filter-select"
              value={statusChoice}
              onChange={(e) => {
                setStatusChoice(e.target.value)
                setStatusReason('')
              }}
              aria-label="Move to status"
            >
              <option value="">Move to</option>
              {detail.available_transitions.map((t) => (
                <option
                  key={t.status}
                  value={t.status}
                  {t.label}
                >
                  {t.label}
                </option>
              ))}
            </select>
            <button
              type="submit"
              className="cr-btn cr-btn--secondary cr-btn--small"
              disabled={!statusChoice || statusSubmitting}
            >
              {statusSubmitting ? 'Updating...' : 'Update Status'}
            </button>
          </div>
          {selectedTransition?.requires_reason && (
            <div className="cr-field">
              <label htmlFor="status-reason">Reason</label>
              <span className="cr-optional-tag">required for this change</span>
              </div>
              <div>
                <label>
                  Reason
                </label>
                <textarea
                  id="status-reason"
                  rows={2}
                  value={statusReason}
                  onChange={(e) => setStatusReason(e.target.value)}
                >
                </div>
              </div>
            )}
          </form>
        ) : (
          <p className="no-changes-note">This is a final status - no further transitions are available.</p>
        )}
      </section>

      <section className="cr-detail-section">
        <h2 className="cr-section-title">Team & Assignments</h2>
        {assignmentsError && <p className="cr-list-status-text cr-list-status-text--error">{assignmentsError}</p>}
        {assignments && assignments.length > 0 ? (
          <ul className="assignments-list">
            {assignments.map((a) => (
              <li key={a.id} className="assignment-row">
                <span>
                  <strong>{a.role_label}</strong> - {a.user_name}
                  <span
                    className="version-row-meta"
                    {a.user_email}
                  >
                </span>
                <button
                  type="button"

```

```

        className="cr-btn cr-btn--ghost cr-btn--small"
        onClick={() => handleRemoveAssignment(a.id)}
        disabled={removingAssignmentId === a.id}
      >
        {removingAssignmentId === a.id ? 'Removing...' : 'Remove'}
      </button>
    </li>
  )}
</ul>
) : (
  assignments && <p className="cr-detail-text">No one is assigned to this request yet.</p>
)
)

<form className="assign-form" onSubmit={handleAssignSubmit}>
  {assignError && <p className="cr-form-error">{assignError}</p>}
  <div className="status-form-row">
    <select
      className="cr-filter-select"
      value={newAssigneeRole}
      onChange={(e) => setNewAssigneeRole(e.target.value)}
      aria-label="Role to assign"
    >
      {ROLE_OPTIONS.map((r) => (
        <option key={r.value} value={r.value}>
          {r.label}
        </option>
      ))}
    </select>
    <select
      className="cr-filter-select"
      value={newAssigneeUserId}
      onChange={(e) => setNewAssigneeUserId(e.target.value)}
      aria-label="Person to assign"
    >
      <option value="">Select a person</option>
      {(users || []).map((u) => (
        <option key={u.id} value={u.id}>
          {u.name} ({u.email})
        </option>
      ))}
    </select>
    <button
      type="submit"
      className="cr-btn cr-btn--secondary cr-btn--small"
      disabled={!newAssigneeUserId || assignSubmitting}
    >
      {assignSubmitting ? 'Assigning...' : 'Assign'}
    </button>
  </div>
</form>
</section>

<section className="cr-detail-section">
  <h2 className="cr-section-title">Approvals</h2>
  {recommendedApprovals.length > 0 && (
    <p className="cr-detail-text approvals-recommendation">
      AI recommends: {recommendedApprovals.map((r) => r.label).join(', ')} approval
      {recommendedApprovals.length > 1 ? 's' : ''}.{' '}</p>
      <span className="cr-optional-tag">the AI never requests these itself - you choose who to tag</span>
      <small>Module 22: recommended_approvals.is_outdated - this recommendation was computed from an analysis that no longer matches the change request's current version. </small>
      {recommendedApprovals[0]?.is_outdated && (
        <span className="badge badge--amber" title="The change request has been edited since this analysis ran.">
          May be outdated
        </span>
      )}
    </p>
  )}
  {approvalsError && <p className="cr-list-status-text cr-list-status-text--error">{approvalsError}</p>}
  {approvals && approvals.length > 0 ? (
    <ul className="approvals-list">
      {approvals.map((a) => {
        const isApprover = a.approver_id === user?.id
        const isResponding = respondingApprovalId === a.id
        return (
          <li key={a.id} className="approval-row">
            <div className="approval-row-main">
              <strong>{a.approval_type_label}</strong> - {a.approver_name}
              <span className="version-row-meta">({a.approver_email})</span>
              <span className={`badge ${APPROVAL_STATUS_CLASS[a.status]} || 'badge--gray'`} >
                {APPROVAL_STATUS_LABELS[a.status]} || a.status
              </span>
              {a.is_outdated && <span className="badge badge--amber">Outdated</span>}
              {a.override_ai_recommendation && (
                <span className="badge badge--red" title="This decision didn't match what the AI recommended">
                  Override AI
                </span>
              )}
            </div>
            <p className="version-row-meta">
              Requested by {a.requested_by name} on {formatDateTime(a.requested_at)}
              {a.responded_at ? ` · responded ${formatDateTime(a.responded_at)}` : ''}
            </p>
            <small>Module 13 Phase 4: what the AI recommended on the analysis this approval was actually requested against (by cr version) - purely informational, never a constraint on the approver's decision. </small>
            {a.ai_recommendation && (
              <p className="version-row-meta">AI recommended: {a.ai_recommendation}</p>
            )}
            {a.comment && <p className="history-value-change">Comment: {a.comment}</p>}
            {a.status === 'pending' && isApprover && !isResponding && (
              <button
                type="button"
                className="cr-btn cr-btn--secondary cr-btn--small"
                onClick={() => openRespondForm(a)}
              >
                Respond
              </button>
            )}
            {a.status === 'pending' && isApprover && isResponding && (
              <form className="respond-form" onSubmit={(e) => handleRespondSubmit(e, a.id)}>
                {respondError && <p className="cr-form-error">{respondError}</p>}
                <div className="status-form-row">
                  <select
                    className="cr-filter-select"
                    value={respondStatus}
                    onChange={(e) => setRespondStatus(e.target.value)}
                    aria-label="Your response"
                  >
                    {APPROVAL_RESPONSE_OPTIONS.map((o) => (
                      <option key={o.value} value={o.value}>
                        {o.label}
                      </option>
                    ))}
                  </select>
                  <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={respondSubmitting}>

```



```

{detail.latest_analysis.clarification_questions.length > 0 && (
  <h3 className="cr-section-title cr-section-title--spaced">Clarification Questions</h3>
  <ul className="cr-question-list">
    {detail.latest_analysis.clarification_questions.map((q) => (
      <li key={q.id} className="cr-question-item">
        <span className={badge(q.resolved ? 'badge--green' : 'badge--amber')}>
          {q.resolved ? 'Resolved' : 'Open'}
        </span>
        <div>
          <p className="cr-question-text">{q.question}</p>
          <p className="cr-question-reason">{q.reason}</p>
        </div>
      </li>
    ))}
  </ul>
)}

<div className="cr-form-actions cr-form-actions--left">
  <button
    type="button"
    className="cr-btn cr-btn--primary"
    onClick={() => onOpenDashboard?.(id)}
  >
    View Full AI Analysis Dashboard
  </button>
  <button
    type="button"
    className="cr-btn cr-btn--secondary"
    onClick={handleAnalyze}
    disabled={analyzing}
  >
    {analyzing ? 'Re-analyzing...' : 'Re-analyze'}
  </button>
</div>
{analyzeError && <p className="cr-form-error">{analyzeError}</p>}
</div>
): (
  <div className="cr-no-analysis">
    <p className="cr-detail-text">This request hasn't been analyzed yet.</p>
    <button
      type="button"
      className="cr-btn cr-btn--primary"
      onClick={handleAnalyze}
      disabled={analyzing}
    >
      {analyzing ? 'Analyzing... this can take up to a minute' : 'Analyze Change'}
    </button>
    {analyzeError && <p className="cr-form-error">{analyzeError}</p>}
  </div>
)}
</section>
</>
)}

{activeTab === 'activity' && (
  <section className="cr-detail-section">
    <h2 className="cr-section-title">Versions</h2>
    {versionsError && <p className="cr-list-status-text cr-list-status-text--error">{versionsError}</p>}
    {versions && versions.length > 0 && (
      <ul className="versions-list">
        {versions.map((version) => (
          <li key={version.version_number}>
            <div className="version-row">
              <div className="version-row-main">
                <span className="version-row-number">Version {version.version_number}</span>
                <span className="version-row-summary">{version.change_summary}</span>
                <span className="version-row-meta">{formatDateTime(version.created_at)}</span>
              </div>
              {version.version_number > 1 && (
                <button
                  type="button"
                  className="cr-btn cr-btn--secondary cr-btn--small"
                  onClick={() => handleToggleCompare(version.version_number)}
                >
                  {compareVersion === version.version_number ? 'Hide changes' : 'View changes'}
                </button>
              )}
            </div>
            {compareVersion === version.version_number && (
              <div className="cr-detail-section" style={{ marginTop: '0.5rem' }}>
                {compareLoading && <p className="cr-list-status-text">Comparing</p>}
                {compareResult?.error && (
                  <p className="cr-list-status-text cr-list-status-text--error">{compareResult.error}</p>
                )}
                {compareResult?.fields && (
                  <table className="compare-table">
                    <thead>
                      <tr>
                        <th>Field</th>
                        <th>Version {compareResult.from_version}</th>
                        <th>Version {compareResult.to_version}</th>
                      </tr>
                    </thead>
                    <tbody>
                      {compareResult.fields
                        .filter((f) => f.status !== 'unchanged')
                        .map((f) => (
                          <tr key={f.field} className={`compare-row-${f.status}`}>
                            <td>{f.label}</td>
                            <td className="compare-old">{f.old_value ?? '-'}</td>
                            <td className="compare-new">{f.new_value ?? '-'}</td>
                          </tr>
                        ))}
                      {compareResult.fields.every((f) => f.status === 'unchanged') && (
                        <tr>
                          <td colspan={3} className="compare-row--unchanged">
                            No field differences between these versions.
                          </td>
                        </tr>
                      )}
                    </tbody>
                  </table>
                </div>
              )}
            </li>
          )}
        </ul>
      )}
    </section>
  <section className="cr-detail-section">
    <h2 className="cr-section-title">Activity History</h2>
    {historyError && <p className="cr-list-status-text cr-list-status-text--error">{historyError}</p>}
    {history && history.length > 0 ? (
      <ul className="history-list">
        {history.map((event) => (

```



```

    comments && <p className="cr-detail-text">No comments yet - start the discussion below.</p>
  })
  <form className="assign-form" onSubmit={handlePostComment}>
    {postCommentError && <p className="cr-form-error">{postCommentError}</p>}
    <div className="cr-field">
      <label htmlFor="new-comment-body">Add a comment</label>
      <MentionTextarea
        id="new-comment-body"
        rows={3}
        placeholder="Write a comment... type @ to mention someone"
        value={newCommentBody}
        onChange={setNewCommentBody}
        users={users}
      />
    </div>
    <div className="status-form-row">
      <button
        type="submit"
        className="cr-btn cr-btn--primary cr-btn--small"
        disabled={!newCommentBody.trim() || postingComment}
      >
        {postingComment ? 'Posting...' : 'Post Comment'}
      </button>
    </div>
  </form>
</section>
</div>
)
}

```

frontend/src/components/DistributionChart.jsx

37 lines

```
/** Simple horizontal bar-list chart, used for both the risk overview and the
 * change-category overview. No charting library - just proportional CSS bar
 * widths - so there's no extra npm install for this module. `data` is an
 * ordered array of { label, count, color? }; bars are sized relative to the
 * largest count so differences stay readable even with few data points. */
export default function DistributionChart({ title, data, emptyLabel }) {
  const total = data.reduce((sum, row) => sum + row.count, 0)
  const max = Math.max(1, ...data.map((row) => row.count))

  return (
    <div className="chart-card">
      <h2 className="chart-card-title">{title}</h2>
      {total === 0 ? (
        <p className="chart-empty">{emptyLabel}</p>
      ) : (
        <ul className="bar-list">
          {data.map((row) => (
            <li key={row.label} className="bar-row">
              <span className="bar-row-label">{row.label}</span>
              <div className="bar-track">
                <div
                  className="bar-fill"
                  style={{
                    width: `${Math.round((row.count / max) * 100)}%`,
                    background: row.color || '#4338ca',
                  }}
                />
              </div>
              <span className="bar-row-count">{row.count}</span>
            </li>
          ))}
        </ul>
      )}
    </div>
  )
}
```

frontend/src/components/EmptyState.jsx

17 lines

```
export default function EmptyState({ onCreateClick }) {  
  return (  
    <div className="dashboard-empty">  
      <div className="dashboard-empty-icon" aria-hidden="true">  
        +  
      </div>  
      <h2 className="dashboard-empty-title">Create your first change request.</h2>  
      <p className="dashboard-empty-subtitle">  
        Once you submit a change request, its metrics, risk profile, and status will show up here.  
      </p>  
      <button type="button" className="dashboard-empty-cta" onClick={onCreateClick}>  
        + New Change Request  
      </button>  
    </div>  
  )  
}
```



```

import { useCallback, useMemo, useState } from 'react'
import { Background, Controls, Handle, Position, ReactFlow, ReactFlowProvider } from '@xyflow/react'
import '@xyflow/react/dist/style.css'
import '../styles/impact-graph.css'

/** Module 8: Impact & Dependency Graph.
 *
 * IMPORTANT data-honesty note: the AI Analysis Engine (Module 6) stores
 * affected_components and dependencies as two flat lists, each tied only to
 * the analysis itself - there is no stored "component X depends on
 * component Y" edge anywhere in the schema. So this graph does NOT draw
 * chains between individual components/dependencies (that would mean
 * inventing relationships the analysis never asserted). Instead it draws
 * exactly what the data supports: the change request branching into two
 * real categories - "Affected Components" and "Dependencies" - each
 * branching into the individual items the AI actually identified. The two
 * category nodes are a UI grouping only (clearly styled as such, dashed/
 * grey), not a claim about architecture.
 */

const IMPACT_COLORS = { critical: '#b42318', high: '#c4320a', medium: '#b54708', low: '#067647' }
const IMPACT_BADGE_CLASS = {
  critical: 'badge--red',
  high: 'badge--orange',
  medium: 'badge--amber',
  low: 'badge--green',
}

const ITEM_SPACING = 190
const HUB_Y = 140
const ITEM_Y = 300

function titleCase(value) {
  if (!value) return value
  return String(value).replace(/_/g, ' ').replace(/\b\w/g, (c) => c.toUpperCase())
}

function rowX(count, index, spacing = ITEM_SPACING) {
  if (count <= 1) return 0
  const totalWidth = (count - 1) * spacing
  return index * spacing - totalWidth / 2
}

function edge(source, target, impactLevel) {
  return {
    id: `e-${source}-${target}`,
    source,
    target,
    style: impactLevel
      ? { stroke: IMPACT_COLORS[impactLevel] || '#98a2b3', strokeWidth: 2 }
      : { stroke: '#d0d5dd', strokeWidth: 1.5 },
  }
}

/** Builds the node/edge arrays straight from the two real lists - nothing
 * here is inferred, only laid out for display. */
function layoutGraph(crCode, crTitle, components, dependencies) {
  const nodes = []
  const edges = []
  const hasComponents = components.length > 0
  const hasDependencies = dependencies.length > 0
  const compCenterX = hasComponents && hasDependencies ? -280 : 0
  const depCenterX = hasComponents && hasDependencies ? 280 : 0

  nodes.push({
    id: 'root',
    type: 'graphNode',
    position: { x: 0, y: 0 },
    data: { kind: 'root', name: crCode, subtitle: crTitle },
  })

  if (hasComponents) {
    nodes.push({
      id: 'hub-components',
      type: 'graphNode',
      position: { x: compCenterX, y: HUB_Y },
      data: { kind: 'hub', name: 'Affected Components' },
    })
    edges.push(edge('root', 'hub-components'))
  }

  components.forEach((c, i) => {
    const id = `component-${c.id}`
    nodes.push({
      id,
      type: 'graphNode',
      position: { x: compCenterX + rowX(components.length, i), y: ITEM_Y },
      data: {
        kind: 'component',
        name: c.component.name,
        subtype: c.component.type,
        impact_level: c.impact_level,
        reason: c.reason,
        confidence: c.confidence,
      },
    })
    edges.push(edge('hub-components', id, c.impact_level))
  })

  if (hasDependencies) {
    nodes.push({
      id: 'hub-dependencies',
      type: 'graphNode',
      position: { x: depCenterX, y: HUB_Y },
      data: { kind: 'hub', name: 'Dependencies' },
    })
    edges.push(edge('root', 'hub-dependencies'))
  }

  dependencies.forEach((d, i) => {
    const id = `dependency-${d.id}`
    nodes.push({
      id,
      type: 'graphNode',
      position: { x: depCenterX + rowX(dependencies.length, i), y: ITEM_Y },
      data: {
        kind: 'dependency',
        name: d.dependency.name,
        subtype: d.dependency.type,
        impact_level: d.impact_level,
        reason: d.reason,
      },
    })
    edges.push(edge('hub-dependencies', id, d.impact_level))
  })
}

```

```

}

return { nodes, edges }
}

function GraphNode({ data }) {
  if (data.kind === 'root') {
    return (
      <div className="ig-node ig-node--root">
        <span className="ig-node-title">{data.name}</span>
        {data.subtitle && <span className="ig-node-sub">{data.subtitle}</span>}
        <Handle type="source" position={Position.Bottom} />
      </div>
    )
  }
  if (data.kind === 'hub') {
    return (
      <div className="ig-node ig-node--hub">
        <Handle type="target" position={Position.Top} />
        <span className="ig-node-title">{data.name}</span>
        <Handle type="source" position={Position.Bottom} />
      </div>
    )
  }
  const color = IMPACT_COLORS[data.impact_level] || '#98a2b3'
  return (
    <div className="ig-node ig-node--item" style={{ '--ig-color': color }}>
      <Handle type="target" position={Position.Top} />
      <span className="ig-node-kind">{data.kind === 'component' ? 'Component' : 'Dependency'}</span>
      <span className="ig-node-title">{data.name}</span>
      <span className="ig-node-type">{titleCase(data.subtype)}</span>
    </div>
  )
}

const NODE_TYPES = { graphNode: GraphNode }

function NodeDetailPanel({ node, onClose }) {
  return (
    <div className="ig-detail">
      <div className="ig-detail-header">
        <span className="ig-badge ${IMPACT_BADGE_CLASS[node.impact_level]} || 'badge--gray'">
          {titleCase(node.impact_level)} impact
        </span>
        <button type="button" className="ig-detail-close" onClick={onClose} aria-label="Close details">
          ✕
        </button>
      </div>
      <h3 className="ig-detail-title">{node.name}</h3>
      <p className="ig-detail-type">
        {node.kind === 'component' ? 'Affected Component' : 'Dependency'} : {titleCase(node.subtype)}
      </p>
      <p className="ig-detail-label">Reason</p>
      <p className="ig-detail-text">{node.reason || 'Insufficient information.'}</p>

      {node.kind === 'component' && (
        <p className="ig-detail-label">Confidence</p>
        <div className="ig-detail-confidence">
          <div className="bar-track">
            <div
              className="bar-fill"
              style={{ width: `${Math.round(node.confidence)}%`, background: '#4338ca' }}
            </div>
          </div>
          <span className="ig-detail-confidence-value">{Math.round(node.confidence)}%</span>
        </div>
      )}

      {node.kind === 'dependency' && (
        <p className="ig-detail-label">Dependency Information</p>
        <p className="ig-detail-text">
          Type: {titleCase(node.subtype)} : {titleCase(node.impact_level)} impact
        </p>
      )}
    </div>
  )
}

/** The graph itself, embedded in the Analysis Dashboard's Impact tab.
 * 'components'/'dependencies' are the exact arrays from GET ../analysis -
 * no separate fetch, no invented data. */
export default function ImpactGraph({ changeRequest, crCode, components, dependencies }) {
  const { nodes, edges } = useMemo(
    () => layoutGraph(crCode, changeRequest?.title || '', components, dependencies),
    [crCode, changeRequest, components, dependencies]
  )
  const [selected, setSelected] = useState(null)

  const onNodeClick = useCallback((event, node) => {
    if (node.data.kind === 'component' || node.data.kind === 'dependency') {
      setSelected(node.data)
    } else {
      setSelected(null)
    }
  }, [])

  if (components.length === 0 && dependencies.length === 0) {
    return (
      <div className="ig-empty">
        <p className="ad-empty">
          No affected components or dependencies were identified for this change - nothing to graph yet.
        </p>
      </div>
    )
  }

  return (
    <div className="ig-shell">
      <div className="ig-canvas">
        <ReactFlowProvider>
          <ReactFlow
            nodes={nodes}
            edges={edges}
            nodeTypes={NODE_TYPES}
            onNodeClick={onNodeClick}
            onPaneClick={() => setSelected(null)}
            fitView
            fitViewOptions={{ padding: 0.3 }}
            proOptions={{ hideAttribution: true }}
            nodesDraggable={false}
            nodesConnectable={false}
            minZoom={0.3}
            maxZoom={1.5}
          >
            <Background gap={24} color="#e4e7ec" />
            <Controls showInteractive={false} />
          </ReactFlow>
        </ReactFlowProvider>
      </div>
    </div>
  )
}

```

```

    </ReactFlow>
  </ReactFlowProvider>
</div>

<div className="ig-sidebar">
  {selected ? (
    <NodeDetailPanel node={selected} onClose={() => setSelected(null)} />
  ) : (
    <div className="ig-hint">
      <p className="ad-empty">Click a node to see its reason, confidence, and dependency details.</p>
    </div>
  )}

  <div className="ig-legend">
    <span className="ig-legend-title">Impact Level</span>
    <span className="ig-legend-item">
      <i style={{ background: IMPACT_COLORS.critical }} /> Critical
    </span>
    <span className="ig-legend-item">
      <i style={{ background: IMPACT_COLORS.high }} /> High
    </span>
    <span className="ig-legend-item">
      <i style={{ background: IMPACT_COLORS.medium }} /> Medium
    </span>
    <span className="ig-legend-item">
      <i style={{ background: IMPACT_COLORS.low }} /> Low
    </span>
  </div>
</div>
)
}

```

frontend/src/components/MetricCard.jsx

29 lines

```
/** One metrics card on the dashboard. `tone` only changes the accent color
 * of the value - kept subtle on purpose (this is an internal engineering
 * tool, not a marketing page).
 *
 * `onClick` is optional: cards used purely for display (e.g. the Reports
 * page's own MetricCard-based stats, which have nothing sensible to drill
 * into) render exactly as before. Passing it turns the whole card into a
 * button that shows which change requests were actually counted for it -
 * `active` just highlights whichever card's drill-down is currently open. */
export default function MetricCard({ label, value, tone = 'default', onClick, active = false }) {
  const className = `metric-card${onClick ? ' metric-card--clickable' : ''}${active ? ' metric-card--active' : ''}`

  if (!onClick) {
    return (
      <div className={className}>
        <span className="metric-card-label">{label}</span>
        <span className={`metric-card-value metric-card-value--${tone}`}>{value}</span>
      </div>
    )
  }

  return (
    <button type="button" className={className} onClick={onClick} aria-pressed={active}>
      <span className="metric-card-label">{label}</span>
      <span className={`metric-card-value metric-card-value--${tone}`}>{value}</span>
    </button>
  )
}
```

frontend/src/components/Modal.jsx

33 lines

```
import { useEffect } from 'react'
import '../styles/modal.css'

/** A minimal, reusable popup dialog - the first one in this app, added for
 * the dashboard's clickable tiles (spec: "instead I want a popup"). Click
 * on the dimmed backdrop, the x button, or press Escape to close; clicking
 * anywhere inside the dialog itself never closes it. `actions`, if given,
 * renders in a footer strip below the body (e.g. pagination controls). */
export default function Modal({ title, onClose, children, actions }) {
  useEffect(() => {
    function handleKeyDown(event) {
      if (event.key === 'Escape') onClose()
    }
    document.addEventListener('keydown', handleKeyDown)
    return () => document.removeEventListener('keydown', handleKeyDown)
  }, [onClose])

  return (
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal-dialog" role="dialog" aria-modal="true" aria-label={title} onClick={(event) => event.stopPropagation()}>
        <div className="modal-header">
          <h2>{title}</h2>
          <button type="button" className="modal-close" onClick={onClose} aria-label="Close">
            ✕
          </button>
        </div>
        <div className="modal-body">{children}</div>
        {actions && <div className="modal-footer">{actions}</div>}
      </div>
    </div>
  )
}
```

```

const STATUS_LABELS = {
  pending_analysis: 'Pending Analysis',
  draft: 'Draft',
  submitted: 'Submitted',
  in_review: 'In Review',
  approved: 'Approved',
  rejected: 'Rejected',
  implemented: 'Implemented',
}

const STATUS_CLASS = {
  pending_analysis: 'badge--indigo',
  draft: 'badge--gray',
  submitted: 'badge--blue',
  in_review: 'badge--purple',
  approved: 'badge--green',
  rejected: 'badge--red',
  implemented: 'badge--teal',
}

const RISK_LABELS = {
  low: 'Low',
  medium: 'Medium',
  high: 'High',
  critical: 'Critical',
}

const RISK_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
}

function formatDate(isoString) {
  try {
    return new Date(isoString).toLocaleDateString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
    })
  } catch {
    return isoString
  }
}

function TableBody({ items, emptyMessage, onRowClick }) {
  if (items.length === 0) {
    return <p className="chart-empty">{emptyMessage}</p>
  }
  return (
    <div className="table-scroll">
      <table className="requests-table">
        <thead>
          <tr>
            <th>Request ID</th>
            <th>Title</th>
            <th>Category</th>
            <th>Risk</th>
            <th>Status</th>
            <th>Created</th>
          </tr>
        </thead>
        <tbody>
          {items.map((item) => (
            <tr>
              <td>
                key={item.id}
                className={onRowClick ? 'cr-list-row' : undefined}
                onClick={onRowClick ? () => onRowClick(item.id) : undefined}
              <td className="requests-table-id">#{item.id}</td>
              <td className="requests-table-title">{item.title}</td>
              <td>{item.category || <span className="table-muted">Not analyzed</span></td>
              <td>
                {item.risk ? (
                  <span className={RISK_CLASS[item.risk] || 'badge--gray'}>
                    {RISK_LABELS[item.risk] || item.risk}
                  </span>
                ) : (
                  <span className="table-muted"></span>
                )}
              </td>
              <td>
                <span className={STATUS_CLASS[item.status] || 'badge--gray'}>
                  {STATUS_LABELS[item.status] || item.status}
                </span>
              </td>
              <td className="table-muted">{formatDate(item.created_at)}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )
}

/** A simple change-request table: id/title/category/risk/status/created,
 * everything the dashboard summary's own RecentChangeRequestItem rows
 * already carry. Used both for the dashboard's "Recent Change Requests"
 * list (unchanged - no props besides `items` passed there) and, since the
 * shape is identical, inside the dashboard tile popup, rather than building
 * a second near-duplicate table component.
 *
 * `bare` skips the surrounding card and heading entirely - used inside
 * Modal, which already provides its own title/border/background, so
 * nesting a second bordered card inside it would just double the framing. */
export default function RecentRequestsTable({
  items,
  title = 'Recent Change Requests',
  emptyMessage = 'No change requests to show here.',
  onRowClick,
  actions,
  bare = false,
}) {
  if (bare) {
    return <TableBody items={items} emptyMessage={emptyMessage} onRowClick={onRowClick} />
  }

  return (
    <div className="table-card">
      <div className="table-card-header">
        <h2 className="chart-card-title">{title}</h2>
        {actions}
      </div>

```

```
    <TableBody items={items} emptyMessage={emptyMessage} onRowClick={onRowClick} />  
  </div>  
)  
}
```

frontend/src/components/RiskTrendChart.jsx

63 lines

```
/** Module 19 Phase 6 (Engineering Change Analytics): a monthly risk-mix
 * trend, hand-rolled the same way DistributionChart.jsx is - proportional
 * CSS bar widths, no charting library. Each row is one calendar month; the
 * single bar is split into 4 stacked segments (low/medium/high/critical)
 * sized relative to that month's own total, so the chart reads as "what
 * fraction of that month's analyzed change requests fell into each risk
 * bucket" rather than raw counts across very different totals. */
const SEGMENTS = [
  { key: 'low', label: 'Low', color: '#12b76a' },
  { key: 'medium', label: 'Medium', color: '#f79009' },
  { key: 'high', label: 'High', color: '#f04438' },
  { key: 'critical', label: 'Critical', color: '#b42318' },
]

export default function RiskTrendChart({ data, emptyLabel }) {
  return (
    <div className="chart-card">
      <h2 className="chart-card-title">Risk Distribution Over Time</h2>
      {!data || data.length === 0 ? (
        <p className="chart-empty">{emptyLabel}</p>
      ) : (
        <ul className="trend-list">
          {data.map((point) => {
            const total = SEGMENTS.reduce((sum, seg) => sum + (point[seg.key] || 0), 0)
            return (
              <li key={point.period} className="trend-row">
                <span className="trend-row-label">{point.period}</span>
                <div className="trend-bar-track">
                  {total === 0 ? null : (
                    SEGMENTS.map((seg) => {
                      const count = point[seg.key] || 0
                      if (count === 0) return null
                      return (
                        <div
                          key={seg.key}
                          className="trend-bar-segment"
                          style={{ width: `${(count / total) * 100}%`, background: seg.color }}
                          title={`${seg.label}: ${count}`}
                        />
                      )
                    })
                  )
                </div>
                <span className="trend-row-count">{total}</span>
              </li>
            )
          })}
        </ul>
        <div className="trend-legend">
          {SEGMENTS.map((seg) => (
            <span key={seg.key} className="trend-legend-item">
              <span className="trend-legend-swatch" style={{ background: seg.color }} />
              {seg.label}
            </span>
          ))}
        </div>
      )}
    </div>
  )
}
```



```

.auth-shell {
  min-height: 100vh;
  display: flex;
  background: #f7f7fb;
}

.auth-brand-panel {
  flex: 1 1 45%;
  min-width: 0;
  background: linear-gradient(135deg, #1e1b4b 0%, #312e81 45%, #4338ca 100%);
  color: #fff;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 3rem;
  position: relative;
  overflow: hidden;
}

.auth-brand-panel::before {
  content: '';
  position: absolute;
  inset: 0;
  background-image: radial-gradient(circle at 20% 20%, rgba(255, 255, 255, 0.08), transparent 40%),
    radial-gradient(circle at 80% 80%, rgba(255, 255, 255, 0.06), transparent 45%);
  pointer-events: none;
}

.auth-brand-content {
  max-width: 420px;
  position: relative;
}

.auth-logo {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  margin-bottom: 2rem;
}

.auth-logo-mark {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 40px;
  height: 40px;
  border-radius: 10px;
  background: rgba(255, 255, 255, 0.15);
  font-weight: 700;
  font-size: 0.95rem;
  letter-spacing: 0.02em;
  border: 1px solid rgba(255, 255, 255, 0.25);
  flex-shrink: 0;
}

.auth-logo-word {
  font-size: 1.15rem;
  font-weight: 600;
  letter-spacing: -0.01em;
}

.auth-tagline {
  font-size: 1.5rem;
  line-height: 1.4;
  font-weight: 500;
  color: rgba(255, 255, 255, 0.92);
  margin: 0;
}

.auth-form-panel {
  flex: 1 1 55%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 2rem;
}

.auth-form-card {
  width: 100%;
  max-width: 400px;
  background: #ffffff;
  border-radius: 16px;
  padding: 2.5rem 2.25rem;
  box-shadow:
    0 1px 2px rgba(16, 24, 40, 0.06),
    0 8px 24px rgba(16, 24, 40, 0.06);
  border: 1px solid #eef0f4;
}

.auth-title {
  font-size: 1.5rem;
  font-weight: 700;
  color: #101828;
  margin: 0 0 0.4rem;
}

.auth-subtitle {
  font-size: 0.9rem;
  color: #667085;
  margin: 0 0 1.75rem;
}

.auth-form {
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
}

.auth-field {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.85rem;
  font-weight: 500;
  color: #344054;
}

.auth-field input {
  font-size: 0.95rem;
  padding: 0.65rem 0.8rem;
  border-radius: 8px;
  border: 1px solid #d0d5dd;
  outline: none;
  transition:

```

```

        border-color: 0.15s ease,
        box-shadow 0.15s ease;
        font-family: inherit;
        color: #101828;
    }

    .auth-field input:focus {
        border-color: #4338ca;
        box-shadow: 0 0 0 3px rgba(67, 56, 202, 0.15);
    }

    .auth-row {
        display: flex;
        align-items: center;
        justify-content: space-between;
        font-size: 0.85rem;
    }

    .auth-checkbox {
        display: flex;
        align-items: center;
        gap: 0.5rem;
        color: #344054;
        cursor: pointer;
    }

    .auth-checkbox input {
        width: 16px;
        height: 16px;
        accent-color: #4338ca;
    }

    .auth-link-button {
        background: none;
        border: none;
        padding: 0;
        color: #4338ca;
        font-size: 0.85rem;
        font-weight: 600;
        cursor: pointer;
        font-family: inherit;
    }

    .auth-link-button:disabled {
        color: #98a2b3;
        cursor: not-allowed;
    }

    .auth-error {
        background: #fef3f2;
        border: 1px solid #fecdc4;
        color: #b42318;
        font-size: 0.85rem;
        padding: 0.65rem 0.8rem;
        border-radius: 8px;
        margin: 0;
    }

    .auth-submit {
        margin-top: 0.25rem;
        padding: 0.75rem 1rem;
        border-radius: 8px;
        border: none;
        background: #4338ca;
        color: #fff;
        font-size: 0.95rem;
        font-weight: 600;
        cursor: pointer;
        transition: background 0.15s ease;
        font-family: inherit;
    }

    .auth-submit:hover:not(:disabled) {
        background: #372ea8;
    }

    .auth-submit:disabled {
        background: #a5a3e0;
        cursor: not-allowed;
    }

    .auth-switch {
        margin: 1.75rem 0 0;
        text-align: center;
        font-size: 0.85rem;
        color: #667085;
    }

    @media (max-width: 900px) {
        .auth-brand-panel {
            display: none;
        }

        .auth-form-panel {
            padding: 1.25rem;
        }
    }

```

```

.app-sidebar {
  width: 240px;
  flex-shrink: 0;
  background: #ffffff;
  border-right: 1px solid #eaeef0;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  padding: 1.25rem 1rem;
  box-sizing: border-box;
  position: sticky;
  top: 0;
  align-self: flex-start;
}

.app-sidebar-brand {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  padding: 0 0.4rem;
  margin-bottom: 1.75rem;
  position: relative;
}

/* Module 18 Phase 6: the notification-center bell + its dropdown - see
   components/AppHeader.jsx's own doc comment. */
.notif-bell-wrap {
  position: relative;
  margin-left: auto;
}

.notif-bell-btn {
  position: relative;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 30px;
  height: 30px;
  border-radius: 8px;
  border: none;
  background: none;
  cursor: pointer;
  font-size: 1.05rem;
  line-height: 1;
  flex-shrink: 0;
}

.notif-bell-btn:hover {
  background: #f9fafb;
}

.notif-bell-badge {
  position: absolute;
  top: -2px;
  right: -2px;
  background: #d92d20;
  color: #fff;
  font-size: 0.62rem;
  font-weight: 700;
  line-height: 1;
  padding: 0.18rem 0.32rem;
  border-radius: 999px;
  min-width: 1rem;
  text-align: center;
}

.notif-dropdown {
  position: absolute;
  top: calc(100% + 0.4rem);
  left: 0;
  width: 320px;
  max-width: calc(100vw - 2rem);
  background: #ffffff;
  border: 1px solid #eaeef0;
  border-radius: 10px;
  box-shadow: 0 8px 24px rgba(16, 24, 40, 0.12);
  z-index: 50;
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.notif-dropdown-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
  padding: 0.7rem 0.85rem;
  border-bottom: 1px solid #f2f4f7;
  font-size: 0.85rem;
}

.notif-dropdown-status {
  padding: 0.9rem;
  margin: 0;
  font-size: 0.82rem;
  color: #667085;
  text-align: center;
}

.notif-dropdown-status--error {
  color: #b42318;
}

.notif-dropdown-list {
  list-style: none;
  margin: 0;
  padding: 0;
  max-height: 340px;
  overflow-y: auto;
}

.notif-dropdown-item {
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 0.2rem;
  padding: 0.65rem 0.85rem;
  border: none;
  border-bottom: 1px solid #f2f4f7;
  background: none;
  cursor: pointer;
  text-align: left;
}

```

```

    font-family: inherit;
}

.notif-dropdown-item:hover {
    background: #f9fafb;
}

.notif-dropdown-item--unread {
    background: #f8f9ff;
}

.notif-dropdown-item-top {
    display: flex;
    align-items: center;
    gap: 0.35rem;
    width: 100%;
}

.notif-dropdown-item-title {
    font-size: 0.83rem;
    font-weight: 600;
    color: #101828;
}

.notif-dropdown-item-message {
    font-size: 0.78rem;
    color: #475467;
    display: -webkit-box;
    -webkit-line-clamp: 2;
    -webkit-box-orient: vertical;
    overflow: hidden;
}

.notif-dropdown-item-time {
    font-size: 0.7rem;
    color: #98a2b3;
}

.notif-dropdown-footer {
    font-family: inherit;
    font-size: 0.8rem;
    font-weight: 600;
    color: #4338ca;
    background: #fafaff;
    border: none;
    border-top: 1px solid #f2f4f7;
    padding: 0.6rem;
    cursor: pointer;
    text-align: center;
}

.notif-dropdown-footer:hover {
    background: #eef1fd;
}

.app-sidebar-logo-mark {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: 30px;
    height: 30px;
    border-radius: 8px;
    background: #4338ca;
    color: #fff;
    font-weight: 700;
    font-size: 0.75rem;
    letter-spacing: 0.02em;
    flex-shrink: 0;
}

.app-sidebar-logo-word {
    font-size: 0.9rem;
    font-weight: 600;
    color: #101828;
    line-height: 1.25;
    letter-spacing: -0.01em;
}

.app-sidebar-nav {
    display: flex;
    flex-direction: column;
    gap: 0.2rem;
    flex: 1 1 auto;
}

.app-sidebar-link {
    font-family: inherit;
    font-size: 0.87rem;
    font-weight: 500;
    color: #475467;
    background: none;
    border: none;
    text-align: left;
    padding: 0.55rem 0.7rem;
    border-radius: 7px;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 0.5rem;
    transition:
        background 0.15s ease,
        color 0.15s ease;
}

/* Module 12 Phase 5: unread-notification count on the sidebar nav item */
.app-sidebar-badge {
    background: #d92d20;
    color: #fff;
    font-size: 0.68rem;
    font-weight: 700;
    line-height: 1;
    padding: 0.22rem 0.4rem;
    border-radius: 999px;
    min-width: 1.1rem;
    text-align: center;
}

.app-sidebar-link:hover {
    background: #f9fafb;
    color: #101828;
}

.app-sidebar-link--active {
    background: #eef1fd;
    color: #4338ca;
}

.app-sidebar-footer {
    display: flex;
    flex-direction: column;
}

```

```

gap: 0.75rem;
padding-top: 1rem;
margin-top: 0.5rem;
border-top: 1px solid #f2f4f7;
}

.app-sidebar-footer .status-badge {
  align-self: flex-start;
}

.app-sidebar-user {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  padding: 0 0.4rem;
  min-width: 0;
}

.app-sidebar-avatar {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 30px;
  height: 30px;
  border-radius: 50%;
  background: #eef1fd;
  color: #4338ca;
  font-size: 0.72rem;
  font-weight: 700;
  flex-shrink: 0;
}

.app-sidebar-user-info {
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.app-sidebar-user-name {
  font-size: 0.82rem;
  font-weight: 600;
  color: #101828;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.app-sidebar-user-role {
  font-size: 0.72rem;
  color: #98a2b3;
  text-transform: capitalize;
}

.app-sidebar-logout {
  font-family: inherit;
  font-size: 0.83rem;
  font-weight: 500;
  text-align: left;
  background: none;
  border: none;
  color: #b42318;
  padding: 0.45rem 0.4rem;
  border-radius: 6px;
  cursor: pointer;
}

.app-sidebar-logout:hover {
  background: #fef3f2;
}

@media (max-width: 900px) {
  .app-sidebar {
    width: 76px;
  }

  .app-sidebar-logo-word,
  .app-sidebar-user-info,
  .app-sidebar-logout,
  .app-sidebar-footer .status-badge {
    display: none;
  }

  .app-sidebar-brand {
    justify-content: center;
    padding: 0;
  }

  .app-sidebar-link {
    text-align: center;
    font-size: 0.62rem;
    padding: 0.55rem 0.2rem;
  }

  .app-sidebar-user {
    justify-content: center;
    padding: 0;
  }
}

```

```

import { createContext, useCallback, useContext, useEffect, useMemo, useState } from 'react'
import { getCurrentUser, loginRequest, logoutRequest, registerRequest } from '../api/auth'

const AuthContext = createContext(null)

const STORAGE_KEY = 'acra_auth' // AI Change Request Analyzer

function loadStoredToken() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY)
    return raw ? JSON.parse(raw)?.access_token ?? null : null
  } catch {
    // localStorage unavailable (private browsing, disabled, etc.) - just start logged out.
    return null
  }
}

function saveStoredToken(accessToken) {
  try {
    if (accessToken) {
      localStorage.setItem(STORAGE_KEY, JSON.stringify({ access_token: accessToken }))
    } else {
      localStorage.removeItem(STORAGE_KEY)
    }
  } catch {
    // Nothing we can do - auth just won't persist across page reloads.
  }
}

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null)
  const [token, setToken] = useState(null)
  const [initializing, setInitializing] = useState(true)

  // On first load, if a token was saved from a previous session, verify it's
  // still valid (not expired) before trusting it.
  useEffect(() => {
    const storedToken = loadStoredToken()
    if (!storedToken) {
      setInitializing(false)
      return
    }
    getCurrentUser(storedToken)
      .then((freshUser) => {
        setToken(storedToken)
        setUser(freshUser)
      })
      .catch(() => {
        saveStoredToken(null)
      })
      .finally(() => setInitializing(false))
  }, [])

  const applyAuthResponse = useCallback((data) => {
    saveStoredToken(data.access_token)
    setToken(data.access_token)
    setUser(data.user)
  }, [])

  const login = useCallback(
    async (email, password, rememberMe) => {
      const data = await loginRequest(email, password, rememberMe)
      applyAuthResponse(data)
    },
    [applyAuthResponse]
  )

  const register = useCallback(
    async (name, email, password, confirmPassword) => {
      const data = await registerRequest(name, email, password, confirmPassword)
      applyAuthResponse(data)
    },
    [applyAuthResponse]
  )

  const logout = useCallback(async () => {
    if (token) {
      try {
        await logoutRequest(token)
      } catch {
        // Best-effort - clear local state regardless of whether the backend call succeeds.
      }
    }
    saveStoredToken(null)
    setToken(null)
    setUser(null)
  }, [token])

  const value = useMemo(
    () => ({ user, token, initializing, isAuthenticated: Boolean(user), login, register, logout }),
    [user, token, initializing, login, register, logout]
  )

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
}

export function useAuth() {
  const ctx = useContext(AuthContext)
  if (!ctx) {
    throw new Error('useAuth must be used within an AuthProvider')
  }
  return ctx
}

```

frontend/src/index.css

9 lines

```
:root {  
  color-scheme: light dark;  
}  
  
body {  
  margin: 0;  
  background: #ffffff;  
}
```

frontend/src/main.jsx

11 lines

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```



```

import { useEffect, useState } from 'react'
import { useAuth } from '../context/AuthContext'
import {
  createApprovalRule,
  createUser,
  deleteApprovalRule,
  getSystemSettings,
  listApprovalRules,
  listAuditLog,
  listPermissions,
  listUsers,
  updateApprovalRule,
  updateCrCategories,
  updatePermissions,
  updateUser,
} from '../api/admin'
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/workflow.css'
import '../styles/analysis-dashboard.css'
import '../styles/admin.css'

// Module 21 (Administration & Configuration). Mirrors ROLE_LABELS/etc in
// other pages (module-local label maps rather than a shared import - see
// e.g. MyWorkPage.jsx's own ROLE_LABELS) - here matching
// backend/app/services/workflow_rules.py::USER_ROLE_LABELS exactly.
const ROLE_LABELS = {
  admin: 'Admin',
  requester: 'Requester',
  engineer: 'Engineer',
  reviewer: 'Reviewer',
  security_reviewer: 'Security Reviewer',
  approver: 'Approver',
  product_manager: 'Manager',
}
const ROLE_OPTIONS = Object.entries(ROLE_LABELS).map(([value, label]) => ({ value, label }))

const CAPABILITY_LABELS = {
  cr_create: 'Create Change Requests',
  cr_edit: 'Edit Change Requests',
  status_change: 'Change Status',
  assignment: 'Assign Users',
  approval: 'Approve',
  rejection: 'Reject / Request Changes',
  comments: 'Comment',
  reports: 'Download Reports',
  administration: 'Administration',
}
const CAPABILITY_ORDER = Object.keys(CAPABILITY_LABELS)

const APPROVAL_TYPE_LABELS = {
  technical: 'Technical',
  security: 'Security',
  product: 'Product',
  engineering_manager: 'Engineering Manager',
  director: 'Director',
  qa: 'QA',
  dba: 'DBA',
  release: 'Release',
  general: 'General',
}
const APPROVAL_TYPE_OPTIONS = Object.entries(APPROVAL_TYPE_LABELS).map(([value, label]) => ({ value, label }))

const ADMIN_ACTION_LABELS = {
  user_created: 'User created',
  user_role_changed: 'Role changed',
  user_activated: 'User activated',
  user_deactivated: 'User deactivated',
  permission_changed: 'Permission changed',
  approval_rule_created: 'Approval rule added',
  approval_rule_updated: 'Approval rule updated',
  approval_rule_deleted: 'Approval rule deleted',
  system_setting_changed: 'System setting changed',
}

const TABS = [
  { key: 'users', label: 'Users' },
  { key: 'permissions', label: 'Roles & Permissions' },
  { key: 'approval-rules', label: 'Approval Rules' },
  { key: 'system-settings', label: 'System Settings' },
  { key: 'audit-log', label: 'Audit Log' },
]

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: '2-digit',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

function errorMessage(err, fallback) {
  return err?.message || fallback
}

export default function AdminPage() {
  const { user, token } = useAuth()
  const [activeTab, setActiveTab] = useState('users')

  if (user?.role !== 'admin') {
    return (
      <div className="ad-shell">
        <p className="ad-empty">This section is only available to Admin accounts.</p>
      </div>
    )
  }

  return (
    <div className="ad-shell">
      <div className="ad-header">
        <h1>Administration</h1>
      </div>
      <div className="ad-tabs">
        {TABS.map((tab) => (
          <button
            key={tab.key}

```

```

        type="button"
        className={ad-tabs[activeTab === tab.key ? 'ad-tab--active' : '']}
        onClick={() => setActiveTab(tab.key)}
      >
        {tab.label}
      </button>
    </div>
  </div>

  <div className="ad-tab-content">
    {activeTab === 'users' && <UsersTab token={token} currentUser={user} />}
    {activeTab === 'permissions' && <PermissionsTab token={token} />}
    {activeTab === 'approval-rules' && <ApprovalRulesTab token={token} />}
    {activeTab === 'system-settings' && <SystemSettingsTab token={token} />}
    {activeTab === 'audit-log' && <AuditLogTab token={token} />}
  </div>
)
}

// --- Users -----

function UsersTab({ token, currentUser }) {
  const [users, setUsers] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [rowError, setRowError] = useState({})
  const [showCreate, setShowCreate] = useState(false)
  const [createForm, setCreateForm] = useState({ name: '', email: '', password: '', role: 'engineer' })
  const [createError, setCreateError] = useState(null)
  const [saving, setSaving] = useState(false)

  useEffect(() => {
    listUsers(token)
      .then(setUsers)
      .catch((err) => setError(errorMessage(err, 'Could not load users.')))
      .finally(() => setLoading(false))
  }, [token])

  async function handleRoleChange(targetUser, role) {
    setRowError((prev) => ({ ...prev, [targetUser.id]: null }))
    try {
      const updated = await updateUser(token, targetUser.id, { role })
      setUsers((prev) => prev.map((u) => (u.id === updated.id ? updated : u)))
    } catch (err) {
      setRowError((prev) => ({ ...prev, [targetUser.id]: errorMessage(err, "Couldn't update this user's role.") }))
    }
  }

  async function handleToggleActive(targetUser) {
    setRowError((prev) => ({ ...prev, [targetUser.id]: null }))
    try {
      const updated = await updateUser(token, targetUser.id, { is_active: !targetUser.is_active })
      setUsers((prev) => prev.map((u) => (u.id === updated.id ? updated : u)))
    } catch (err) {
      setRowError((prev) => ({ ...prev, [targetUser.id]: errorMessage(err, "Couldn't update this user's status.") }))
    }
  }

  async function handleCreate(event) {
    event.preventDefault()
    setCreateError(null)
    setSaving(true)
    try {
      const created = await createUser(token, createForm)
      setUsers((prev) => [...prev, created].sort((a, b) => a.name.localeCompare(b.name)))
      setCreateForm({ name: '', email: '', password: '', role: 'engineer' })
      setShowCreate(false)
    } catch (err) {
      setCreateError(errorMessage(err, "Couldn't create this user."))
    } finally {
      setSaving(false)
    }
  }

  if (loading) return <p className="ad-empty">Loading users</p>
  if (error) return <p className="cr-form-error">{error}</p>

  return (
    <div>
      <div className="admin-tab-toolbar">
        <p className="admin-tab-hint">Passwords are never shown or exported - only used to create the account.</p>
        <button type="button" className="cr-btn cr-btn--primary cr-btn--small" onClick={() => setShowCreate((v) => !v)}>
          {showCreate ? 'Cancel' : 'Add User'}
        </button>
      </div>

      {showCreate && (
        <form className="cr-section admin-inline-form" onSubmit={handleCreate}>
          {createError && <p className="cr-form-error">{createError}</p>}
          <div className="cr-field-row">
            <div className="cr-field">
              <label htmlFor="admin-new-name">Name</label>
              <input
                id="admin-new-name"
                type="text"
                value={createForm.name}
                onChange={(e) => setCreateForm((f) => ({ ...f, name: e.target.value })))}
                required
              />
            </div>
            <div className="cr-field">
              <label htmlFor="admin-new-email">Email</label>
              <input
                id="admin-new-email"
                type="text"
                value={createForm.email}
                onChange={(e) => setCreateForm((f) => ({ ...f, email: e.target.value })))}
                required
              />
            </div>
            <div className="cr-field-row">
              <div className="cr-field">
                <label htmlFor="admin-new-password">Password</label>
                <input
                  id="admin-new-password"
                  type="password"
                  value={createForm.password}
                  onChange={(e) => setCreateForm((f) => ({ ...f, password: e.target.value })))}
                  required
                  minLength={8}
                />
              </div>
              <div className="cr-field">
                <label htmlFor="admin-new-role">Role</label>
                <select
                  id="admin-new-role"
                  value={createForm.role}
                  onChange={(e) => setCreateForm((f) => ({ ...f, role: e.target.value })))}
                />
              </div>
            </div>
          </form>
        )
      )
    </div>
  )
}

```

```

        {ROLE_OPTIONS.map((opt) => (
          <option key={opt.value} value={opt.value}>
            {opt.label}
          </option>
        ))}
      </select>
    </div>
  </div>
  <div className="cr-form-actions">
    <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={saving}>
      {saving ? 'Creating...' : 'Create User'}
    </button>
  </div>
</form>
)}

<table className="requests-table">
  <thead>
    <tr>
      <th>Name</th>
      <th>Email</th>
      <th>Role</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    {users.map((u) => (
      <tr key={u.id}>
        <td>{u.name}</td>
        <td>{u.email}</td>
        <td>
          <select value={u.role} onChange={(e) => handleRoleChange(u, e.target.value)} disabled={u.id === currentUser.id}>
            {ROLE_OPTIONS.map((opt) => (
              <option key={opt.value} value={opt.value}>
                {opt.label}
              </option>
            ))}
          </select>
        </td>
        <td>
          <span className={`badge ${u.is_active ? 'badge--green' : 'badge--gray'}`}>
            {u.is_active ? 'Active' : 'Inactive'}
          </span>
        </td>
        <td>
          <button
            type="button"
            className="cr-btn cr-btn--ghost cr-btn--small"
            onClick={() => handleToggleActive(u)}
            disabled={u.id === currentUser.id}
          >
            {u.is_active ? 'Deactivate' : 'Activate'}
          </button>
          {rowError[u.id] && <p className="admin-row-error">{rowError[u.id]}</p>}
        </td>
      </tr>
    ))}
  </tbody>
</table>
</div>
}

```

// --- Roles & Permissions -----

```

function PermissionsTab({ token }) {
  const [permissions, setPermissions] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [pending, setPending] = useState({})
  const [saving, setSaving] = useState(false)
  const [saveError, setSaveError] = useState(null)
  const [saveMessage, setSaveMessage] = useState(null)

  useEffect(() => {
    listPermissions(token)
      .then(setPermissions)
      .catch((err) => setError(errorMessage(err, 'Could not load permissions.')))
      .finally(() => setLoading(false))
  }, [token])

  if (loading) return <p className="ad-empty">Loading permissions</p>
  if (error) return <p className="cr-form-error">{error}</p>

  const roles = ROLE_OPTIONS.map((opt) => opt.value).filter((role) => role !== 'admin')
  const key = (role, capability) => `${role}:${capability}`
  const allowedFor = (role, capability) => {
    const pendingKey = key(role, capability)
    if (pendingKey in pending) return pending[pendingKey]
    const row = permissions.find((p) => p.role === role && p.capability === capability)
    return row ? row.allowed : true
  }

  function toggle(role, capability) {
    if (capability === 'administration') return // never editable
    setPending((prev) => ({ ...prev, [key(role, capability)]: !allowedFor(role, capability) }))
    setSaveMessage(null)
  }

  async function handleSave() {
    setSaving(true)
    setSaveError(null)
    setSaveMessage(null)
    const changes = Object.entries(pending).map(([k, allowed]) => {
      const [role, capability] = k.split(':')
      return { role, capability, allowed }
    })
    try {
      const updated = await updatePermissions(token, changes)
      setPermissions(updated)
      setPending({})
      setSaveMessage('Saved.')
    } catch (err) {
      setSaveError(errorMessage(err, "Couldn't save permission changes."))
    } finally {
      setSaving(false)
    }
  }

  const hasPending = Object.keys(pending).length > 0

  return (
    <div>
      <p className="admin-tab-hint">
        Admin accounts always have full access, regardless of this table. Administration itself can never be changed here - it's always Admin-only, so an admin can never be accidentally locked out.
      </p>
      {saveError && <p className="cr-form-error">{saveError}</p>}
    </div>
  )
}

```

```

<div className="admin-matrix-scroll">
  <table className="requests-table admin-matrix">
    <thead>
      <tr>
        <th>Role</th>
        {CAPABILITY_ORDER.map((cap) => (
          <th key={cap}>{CAPABILITY_LABELS[cap]}</th>
        ))}
      </tr>
    </thead>
    <tbody>
      <tr className="admin-matrix-locked-row">
        <td>Admin</td>
        {CAPABILITY_ORDER.map((cap) => (
          <td key={cap}>
            <input type="checkbox" checked readOnly disabled />
          </td>
        ))}
      </tr>
      {roles.map((role) => (
        <tr key={role}>
          <td>{ROLE_LABELS[role]}</td>
          {CAPABILITY_ORDER.map((cap) => {
            const locked = cap === 'administration'
            return (
              <td key={cap}>
                <input
                  type="checkbox"
                  checked={locked ? false : allowedFor(role, cap)}
                  disabled={locked}
                  onChange={() => toggle(role, cap)}
                />
              </td>
            )
          })}
        </tr>
      ))}
    </tbody>
  </table>
</div>

<div className="cr-form-actions">
  {saveMessage && <span className="admin-save-message">{saveMessage}</span>}
  <button type="button" className="cr-btn cr-btn--primary cr-btn--small" onClick={handleSave} disabled={!hasPending || saving}>
    {saving ? 'Saving' : 'Save Changes'}
  </button>
</div>
)
}

// --- Approval Rules -----
function ApprovalRulesTab({ token }) {
  const [rules, setRules] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [rowError, setRowError] = useState({})
  const [showCreate, setShowCreate] = useState(false)
  const [createForm, setCreateForm] = useState({ rule_type: 'risk', match_value: '', approval_types: [] })
  const [createError, setCreateError] = useState(null)
  const [saving, setSaving] = useState(false)

  useEffect(() => {
    listApprovalRules(token)
      .then(setRules)
      .catch((err) => setError(errorMessage(err, 'Could not load approval rules.')))
      .finally(() => setLoading(false))
  }, [token])

  async function handleToggleEnabled(rule) {
    setRowError((prev) => ({ ...prev, [rule.id]: null }))
    try {
      const updated = await updateApprovalRule(token, rule.id, { enabled: !rule.enabled })
      setRules((prev) => prev.map((r) => (r.id === updated.id ? updated : r)))
    } catch (err) {
      setRowError((prev) => ({ ...prev, [rule.id]: errorMessage(err, "Couldn't update this rule.") }))
    }
  }

  async function handleToggleApprovalType(rule, type) {
    const nextTypes = rule.approval_types.includes(type)
      ? rule.approval_types.filter((t) => t !== type)
      : [...rule.approval_types, type]
    if (nextTypes.length === 0) {
      setRowError((prev) => ({ ...prev, [rule.id]: 'At least one required approval type must stay selected.' }))
      return
    }
    setRowError((prev) => ({ ...prev, [rule.id]: null }))
    try {
      const updated = await updateApprovalRule(token, rule.id, { approval_types: nextTypes })
      setRules((prev) => prev.map((r) => (r.id === updated.id ? updated : r)))
    } catch (err) {
      setRowError((prev) => ({ ...prev, [rule.id]: errorMessage(err, "Couldn't update this rule.") }))
    }
  }

  async function handleDelete(rule) {
    if (!window.confirm('Delete this approval rule (${rule.match_value})?')) return
    setRowError((prev) => ({ ...prev, [rule.id]: null }))
    try {
      await deleteApprovalRule(token, rule.id)
      setRules((prev) => prev.filter((r) => r.id !== rule.id))
    } catch (err) {
      setRowError((prev) => ({ ...prev, [rule.id]: errorMessage(err, "Couldn't delete this rule.") }))
    }
  }

  async function handleCreate(event) {
    event.preventDefault()
    setCreateError(null)
    if (createForm.approval_types.length === 0) {
      setCreateError('Select at least one required approval type.')
      return
    }
    setSaving(true)
    try {
      const created = await createApprovalRule(token, {
        ...createForm,
        match_value: createForm.match_value.trim().toLowerCase(),
      })
      setRules((prev) => [...prev, created])
      setCreateForm({ rule_type: 'risk', match_value: '', approval_types: [] })
      setShowCreate(false)
    } catch (err) {
      setCreateError(errorMessage(err, "Couldn't add this rule."))
    } finally {
      setSaving(false)
    }
  }
}

```

```

function toggleCreateType(type) {
  setCreateForm((f) => ({
    ...f,
    approval_types: f.approval_types.includes(type) ? f.approval_types.filter((t) => t !== type) : [...f.approval_types, type],
  })))
}

if (loading) return <p className="ad-empty">Loading approval rules.</p>
if (error) return <p className="cr-form-error">{error}</p>

return (
  <div>
    <p className="admin-tab-hint">
      A risk rule fires when a change request's overall risk lands in that bucket; a category rule fires when its
      keyword appears in the AI's category, target system, or title. Every enabled, matching rule's approval types
      are combined - not a rule engine, just a simple lookup.
    </p>

    <div className="admin-tab-toolbar">
      <span />
      <button type="button" className="cr-btn cr-btn--primary cr-btn--small" onClick={() => setShowCreate((v) => !v)}>
        {showCreate ? 'Cancel' : 'Add Rule'}
      </button>
    </div>

    {showCreate && (
      <form className="cr-section admin-inline-form" onSubmit={handleCreate}>
        {createError && <p className="cr-form-error">{createError}</p>}
        <div className="cr-field-row">
          <div className="cr-field">
            <label htmlFor="admin-rule-type">Rule type</label>
            <select
              id="admin-rule-type"
              value={createForm.rule_type}
              onChange={(e) => setCreateForm((f) => ({ ...f, rule_type: e.target.value, match_value: '' })))}
            >
              <option value="risk">Risk level</option>
              <option value="category">Category keyword</option>
            </select>
          </div>
          <div className="cr-field">
            <label htmlFor="admin-rule-match">
              {createForm.rule_type === 'risk' ? 'Risk level' : 'Keyword'}
            </label>
            {createForm.rule_type === 'risk' ? (
              <select
                id="admin-rule-match"
                value={createForm.match_value}
                onChange={(e) => setCreateForm((f) => ({ ...f, match_value: e.target.value })))}
              >
                <option value="">Select</option>
                <option value="low">Low</option>
                <option value="medium">Medium</option>
                <option value="high">High</option>
                <option value="critical">Critical</option>
              </select>
            ) : (
              <input
                id="admin-rule-match"
                type="text"
                placeholder="e.g. checkout"
                value={createForm.match_value}
                onChange={(e) => setCreateForm((f) => ({ ...f, match_value: e.target.value })))}
                required
              />
            )}
          </div>
        </div>
        <div className="cr-field">
          <label>Required approval types</label>
          <div className="admin-chip-row">
            {APPROVAL_TYPE_OPTIONS.map((opt) => (
              <button
                type="button"
                key={opt.value}
                className={createForm.approval_types.includes(opt.value) ? 'admin-chip--active' : ''}
                onClick={() => toggleCreateType(opt.value)}
              >
                {opt.label}
              </button>
            ))}
          </div>
        </div>
        <div className="cr-form-actions">
          <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={saving}>
            {saving ? 'Adding...' : 'Add Rule'}
          </button>
        </div>
      </form>
    )}

    <table className="requests-table">
      <thead>
        <tr>
          <th>Type</th>
          <th>Match</th>
          <th>Required Approvals</th>
          <th>Enabled</th>
        </tr>
      </thead>
      <tbody>
        {rules.length === 0 && (
          <tr>
            <td colspan={5}>
              <p className="ad-empty">No approval rules configured - add one above.</p>
            </td>
          </tr>
        )}
        {rules.map((rule) => (
          <tr key={rule.id}>
            <td>{rule.rule_type === 'risk' ? 'Risk level' : 'Category keyword'}</td>
            <td>{rule.match_value}</td>
            <div className="admin-chip-row">
              {APPROVAL_TYPE_OPTIONS.map((opt) => (
                <button
                  type="button"
                  key={opt.value}
                  className={createForm.approval_types.includes(opt.value) ? 'admin-chip--active' : ''}
                  onClick={() => handleToggleApprovalType(rule, opt.value)}
                >
                  {opt.label}
                </button>
              ))}
            </div>
            <td>
              <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={() => handleToggleEnabled(rule)}>
                {rule.enabled ? 'Enabled' : 'Disabled'}
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
)

```

```

        </button>
      </td>
    </tr>
  </tbody>
</table>
</div>
)
}

// --- System Settings -----

function SystemSettingsTab({ token }) {
  const [settings, setSettings] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [newCategory, setNewCategory] = useState('')
  const [saving, setSaving] = useState(false)
  const [saveError, setSaveError] = useState(null)

  useEffect(() => {
    getSystemSettings(token)
      .then(setSettings)
      .catch((err) => setError(errorMessage(err, 'Could not load system settings.')))
      .finally(() => setLoading(false))
  }, [token])

  async function saveCategories(categories) {
    setSaving(true)
    setSaveError(null)
    try {
      const updated = await updateCrCategories(token, categories)
      setSettings((prev) => ({ ...prev, cr_categories: updated }))
    } catch (err) {
      setError(errorMessage(err, "Couldn't update categories."))
    } finally {
      setSaving(false)
    }
  }

  function handleRemoveCategory(category) {
    saveCategories(settings.cr_categories.filter((c) => c !== category))
  }

  function handleAddCategory(event) {
    event.preventDefault()
    const trimmed = newCategory.trim()
    if (!trimmed || settings.cr_categories.some((c) => c.toLowerCase() === trimmed.toLowerCase())) {
      setNewCategory('')
      return
    }
    saveCategories([...settings.cr_categories, trimmed])
    setNewCategory('')
  }

  if (loading) return <p className="ad-empty">Loading system settings</p>
  if (error) return <p className="cr-form-error">{error}</p>

  const { ai_provider: aiProvider, repository, knowledge_base: knowledgeBase } = settings

  return (
    <div>
      <div className="cr-section">
        <h3 className="cr-section-title">Change Request Categories</h3>
        <p className="admin-tab-hint">
          The reference list of category names used across the app's grouping and filters. This list is editable
          here; the keyword matching that assigns a specific change request to a category is not (kept simple
          rather than becoming a second rule engine to configure).
        </p>
        {saveError && <p className="cr-form-error">{saveError}</p>}
        <div className="admin-chip-row">
          {settings.cr_categories.map((category) => (
            <span key={category} className="admin-chip admin-chip--removable">
              {category}
              <button type="button" onClick={() => handleRemoveCategory(category)} disabled={saving} aria-label={`Remove ${category}`}>
                x
              </button>
            </span>
          ))}
        </div>
        <form className="admin-inline-add" onSubmit={handleAddCategory}>
          <input
            type="text"
            placeholder="Add a category..."
            value={newCategory}
            onChange={(e) => setNewCategory(e.target.value)}
            disabled={saving}
          />
          <button type="submit" className="cr-btn cr-btn--secondary cr-btn--small" disabled={saving}>
            Add
          </button>
        </form>
      </div>

      <div className="cr-section">
        <h3 className="cr-section-title">AI Provider</h3>
        <p className="admin-tab-hint">Read-only - change this by editing your backend .env file directly.</p>
        <dl className="admin-info-list">
          <dt>Provider</dt>
          <dd>{aiProvider.provider}</dd>
          <dt>Model</dt>
          <dd>{aiProvider.model}</dd>
          <dt>API key configured</dt>
          <dd>
            <span className={`badge ${aiProvider.api_key_configured ? 'badge--green' : 'badge--red'}>
              {aiProvider.api_key_configured ? 'Yes' : 'No'}
            </span>
          </dd>
          <dt>Embeddings supported</dt>
          <dd>{aiProvider.embeddings_supported ? 'Yes' : 'No'}</dd>
        </dl>
      </div>

      <div className="cr-section">
        <h3 className="cr-section-title">Repository</h3>
        <p className="admin-tab-hint">Read-only - change this by editing your backend .env file directly.</p>
        <dl className="admin-info-list">
          <dt>Repository root</dt>
          <dd className="admin-info-path">{repository.repository_root}</dd>
          <dt>Last scan</dt>
          <dd>{repository.last_scan_status || 'Never scanned'}</dd>
          <dt>Last scan file count</dt>
          <dd>{repository.last_scan_file_count ?? ''}</dd>
          <dt>Last scan at</dt>
        </dl>
      </div>
    </div>
  )
}

```

```

        <dd>{formatDateTime(repository.last_scan_at)}</dd>
    </dl>
</div>

<div className="cr-section">
    <h3 className="cr-section-title">Knowledge Base</h3>
    <p className="admin-tab-hint">Read-only - change this by editing your backend .env file directly.</p>
    <dl className="admin-info-list">
        <dt>Embedding model</dt>
        <dd>{knowledgeBase.embedding_model}</dd>
        <dt>Embeddings supported</dt>
        <dd>{knowledgeBase.embeddings_supported ? 'Yes' : 'No'}</dd>
        <dt>Documents uploaded</dt>
        <dd>{knowledgeBase.document_count}</dd>
    </dl>
</div>

<p className="admin-tab-hint">
    Priority values (Low/Medium/High/Critical) and the workflow status graph aren't configurable here - both are
    used throughout risk scoring, sorting, and the status lifecycle, so changing them safely would take more
    than a settings toggle.
</p>
</div>
)
}

// --- Audit Log -----
const AUDIT_PAGE_SIZE = 25

function AuditLogTab({ token }) {
    const [data, setData] = useState(null)
    const [offset, setOffset] = useState(0)
    const [loading, setLoading] = useState(true)
    const [error, setError] = useState(null)

    useEffect(() => {
        setLoading(true)
        listAuditLog(token, { limit: AUDIT_PAGE_SIZE, offset })
        .then(setData)
        .catch((err) => setError(errorMessage(err, 'Could not load the audit log.')))
        .finally(() => setLoading(false))
    }, [token, offset])

    if (loading && !data) return <p className="ad-empty">Loading audit log</p>
    if (error) return <p className="cr-form-error">{error}</p>

    const items = data?.items || []
    const total = data?.total || 0

    return (
        <div>
            <p className="admin-tab-hint">
                Every account, role, permission, approval-rule, and system-setting change made through the admin section -
                append-only, never editable here.
            </p>

            <table className="requests-table">
                <thead>
                    <tr>
                        <th>When</th>
                        <th>Actor</th>
                        <th>Action</th>
                        <th>Detail</th>
                    </tr>
                </thead>
                <tbody>
                    {items.length === 0 && (
                        <tr>
                            <td colspan={4}>
                                <p className="ad-empty">No audit events yet.</p>
                            </td>
                        </tr>
                    )}
                    {items.map((item) => (
                        <tr key={item.id}>
                            <td>{formatDateTime(item.created_at)}</td>
                            <td>{item.actor_name || 'System'}</td>
                            <td>{ADMIN_ACTION_LABELS[item.action] || item.action}</td>
                            <td>{item.detail || '-'}</td>
                        </tr>
                    ))}
                </tbody>
            </table>

            <div className="admin-pagination">
                <button
                    type="button"
                    className="cr-btn cr-btn--ghost cr-btn--small"
                    onClick={() => setOffset((o) => Math.max(0, o - AUDIT_PAGE_SIZE))}
                    disabled={offset === 0}
                >
                    Previous
                </button>
                <span className="admin-tab-hint">
                    {total === 0 ? '0 events' : `${offset + 1} - ${Math.min(offset + AUDIT_PAGE_SIZE, total)} of ${total}`}
                </span>
                <button
                    type="button"
                    className="cr-btn cr-btn--ghost cr-btn--small"
                    onClick={() => setOffset((o) => o + AUDIT_PAGE_SIZE)}
                    disabled={offset + AUDIT_PAGE_SIZE >= total}
                >
                    Next
                </button>
            </div>
        </div>
    )
}

```

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import {
  analyzeChangeRequest,
  answerClarificationQuestion,
  compareAnalyses,
  downloadReport,
  getAnalysis,
  getAnalysisHistory,
  listReportableVersions,
  reviewRequirement,
  updateImplementationTask,
  updateSecurityFindingStatus,
  updateTestCase,
} from '../api/analysis'
// Module 15 (Repository Intelligence).
import {
  getLatestRepositoryScan,
  getRepositoryFindings,
  triggerRepositoryMatch,
  triggerRepositoryScan,
} from '../api/repository'
// Module 14 Phase 1: reuses the exact same recommended-approvals endpoint
// ChangeRequestDetail.jsx already calls - this page just also displays it,
// rather than standing up a second "what approvals does this need" system.
import { getRecommendedApprovals } from '../api/approvals'
import { getChangeRequest } from '../api/changeRequests'
import DistributionChart from '../components/DistributionChart'
import ImpactGraph from '../components/ImpactGraph'
import { useAuth } from '../context/AuthContext'
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'
import '../styles/analysis-dashboard.css'
// Module 13 Phase 3/4/5: reuses the same outdated-banner / compare-table
// styles Module 12's CR-version-compare feature already defined, rather
// than re-declaring near-identical CSS a second time.
import '../styles/workflow.css'

const TABS = [
  { key: 'overview', label: 'Overview' },
  { key: 'requirements', label: 'Requirements' },
  { key: 'impact', label: 'Impact' },
  { key: 'repository', label: 'Repository' },
  { key: 'dependencies', label: 'Dependencies' },
  { key: 'risks', label: 'Risks' },
  { key: 'security', label: 'Security' },
  { key: 'effort', label: 'Complexity & Effort' },
  { key: 'missing', label: 'Missing Information' },
  { key: 'tests', label: 'Test Cases' },
  { key: 'plan', label: 'Implementation Plan' },
  { key: 'traceability', label: 'Traceability' },
  { key: 'history', label: 'History' },
]

const PRIORITY_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const PRIORITY_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
}
const LEVEL_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
  very_high: 'badge--red',
}
const PRIORITY_ORDER = { critical: 0, high: 1, medium: 2, low: 3 }

// Module 9: the risk-analysis engine's 8 categories (analysis engine.py's
// CLASSIFICATION/RISK CATEGORIES list) in a fixed display order, each with
// its own chart color. Any risk stored under a category outside this list
// (the app's older TECHNICAL/BUSINESS values, kept for backward
// compatibility) still counts - it's just grouped into "Other" rather than
// silently dropped.
const RISK_CATEGORY_ORDER = [
  'security',
  'data',
  'performance',
  'availability',
  'integration',
  'regression',
  'compliance',
  'operational',
]
const RISK_CATEGORY_COLORS = {
  security: '#b42318',
  data: '#175cd3',
  performance: '#7839ee',
  availability: '#0e7090',
  integration: '#dc6803',
  regression: '#c11574',
  compliance: '#3538cd',
  operational: '#667085',
  other: '#98a2b3',
}

const RECOMMENDATION_META = {
  approve: { emoji: '👍', label: 'APPROVE', tone: 'success' },
  approve_with_conditions: { emoji: '👎', label: 'APPROVE WITH CONDITIONS', tone: 'warning' },
  requires_clarification: { emoji: '🔍', label: 'REQUIRES CLARIFICATION', tone: 'danger' },
  needs_more_info: { emoji: '📄', label: 'NEEDS MORE INFO', tone: 'warning' },
  reject: { emoji: '🚫', label: 'REJECT', tone: 'danger' },
}

// Module 13 Phase 1: Known/Inferred/Unknown - a separate "what kind of
// statement is this" tag from the AI, distinct from its numeric confidence
// (see backend/app/models/enums.py:Certainty and
// backend/app/schemas/ai_analysis.py's _normalize_certainty for why
// "unknown" here is a real, meaningful value rather than a placeholder).
const CERTAINTY_LABELS = { known: 'Known', inferred: 'Inferred', unknown: 'Unknown' }
const CERTAINTY_CLASS = { known: 'badge--green', inferred: 'badge--indigo', unknown: 'badge--gray' }

// Module 14 Phase 3: a coarse Low/Medium/High confidence in the
// Complexity level / Effort estimate specifically - distinct from
// CertaintyBadge above (which tags individual requirements/risks/
// components) and from the overall "AI Confidence" summary card (which is
// the classification's own confidence score). Colors run the OPPOSITE
// direction from LEVEL_CLASS on purpose: a *high* confidence is reassuring
// (green), not alarming, unlike a *high* impact/risk level.
const CONFIDENCE_LEVEL_LABELS = { low: 'Low', medium: 'Medium', high: 'High' }
const CONFIDENCE_LEVEL_CLASS = { low: 'badge--amber', medium: 'badge--blue', high: 'badge--green' }

```



```

// Module 15 (Repository Intelligence): the ONLY three labels a repository
// finding can ever carry (backend/app/models/enums.py::FileMatchLabel) -
// there is no "definitely affected" value anywhere in this app on purpose,
// computed server-side from a numeric confidence score, never trusted
// from the AI's own wording. Colors run the same direction as LEVEL_CLASS
// (more confident a file is affected = more attention-grabbing), not the
// "green is good" direction ConfidenceLevelBadge uses - a confident file
// match is something to go look at, not reassuring news.
const MATCH_LABEL_LABELS = {
  likely_affected: 'Likely Affected',
  potentially_affected: 'Potentially Affected',
  possibly_related: 'Possibly Related',
}
const MATCH_LABEL_CLASS = {
  likely_affected: 'badge--orange',
  potentially_affected: 'badge--amber',
  possibly_related: 'badge--gray',
}

// Module 14 Phase 4: the 7 fixed lenses every analysis assesses impact
// through (backend/app/models/enums.py::ImpactCategory), in a fixed
// display order - distinct from RISK_CATEGORY_ORDER above (a Risk is a
// specific thing that could go wrong; an impact assessment describes the
// change's general effect through this lens regardless of whether
// anything goes wrong).
const IMPACT_CATEGORY_ORDER = ['business', 'technical', 'customer', 'operational', 'security', 'data', 'performance']
const IMPACT_CATEGORY_LABELS = {
  business: 'Business',
  technical: 'Technical',
  customer: 'Customer',
  operational: 'Operational',
  security: 'Security',
  data: 'Data',
  performance: 'Performance',
}

function ImpactAssessmentBadges({ item }) {
  if (!item) return null
  return (
    <div className="ad-impact-card-badges">
      <span className={LEVEL_CLASS[item.impact_level] || 'badge--gray'}>
        {titleCase(item.impact_level)} impact
      </span>
      <certaintyBadge certainty={item.certainty} />
    </div>
  )
}

function ImpactAssessmentCard({ category, item }) {
  return (
    <div className="cr-detail-section ad-impact-card">
      <h3 className="ad-impact-card-title">{IMPACT_CATEGORY_LABELS[category] || titleCase(category)}</h3>
      {item ? (
        <div>
          <ImpactAssessmentBadges item={item} />
          <p className="cr-detail-text ad-mt">{item.description}</p>
          <ConfidenceBar value={item.confidence} />
          <RelatedFilesLine files={item.related_files} />
        </div>
      ) : (
        <p className="ad-empty">Insufficient information.</p>
      )}
    </div>
  )
}

// Module 14 Phase 4: the full 7-category grid - used on the Impact tab.
// The Overview tab's Business/Technical Impact sections pull the matching
// two items out of the same 'analysis.impact assessments' list directly,
// rather than duplicating this component, since they show extra context
// (business requirements, affected components) alongside the AI finding.
function ImpactAnalysisSection({ analysis }) {
  const byCategory = {}
  analysis.impact_assessments.forEach((item) => {
    if (!byCategory[item.category]) byCategory[item.category] = item
  })
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Impact Analysis</h2>
      {analysis.impact_assessments.length === 0 ? (
        <p className="ad-empty">
          No structured impact assessment is available for this analysis yet - re-analyze this change request to
          generate one.
        </p>
      ) : (
        <div className="ad-impact-grid">
          {IMPACT_CATEGORY_ORDER.map((category) => (
            <ImpactAssessmentCard key={category} category={category} item={byCategory[category]} />
          ))}
        </div>
      )}
    </section>
  )
}

// Module 14 Phase 5: the 9 fixed lenses a Security finding is classified
// under (backend/app/models/enums.py::SecurityCategory), fixed display
// order.
const SECURITY_CATEGORY_ORDER = [
  'authentication',
  'authorization',
  'data protection',
  'secrets',
  'api security',
  'rate limiting',
  'privacy',
  'audit logging',
  'compliance',
]
const SECURITY_CATEGORY_LABELS = {
  authentication: 'Authentication',
  authorization: 'Authorization',
  data_protection: 'Data Protection',
  secrets: 'Secrets',
  api_security: 'API Security',
  rate_limiting: 'Rate Limiting',
  privacy: 'Privacy',
  audit_logging: 'Audit Logging',
  compliance: 'Compliance',
}

// Only OPEN/NOT APPLICABLE ever come from the AI itself - ACKNOWLEDGED/
// RESOLVED are set by a human reviewer instead (Module 14 Phase 6, see
// SecurityFindingStatusControl below).
const SECURITY_STATUS_OPTIONS = [
  { value: 'open', label: 'Open' },
  { value: 'not applicable', label: 'Not Applicable' },
  { value: 'acknowledged', label: 'Acknowledged' },
  { value: 'resolved', label: 'Resolved' },
]
const SECURITY_STATUS_LABELS = {
  open: 'Open',

```

```

    not_applicable: 'Not Applicable',
    acknowledged: 'Acknowledged',
    resolved: 'Resolved',
  }
}
const SECURITY_STATUS_CLASS = {
  open: 'badge--orange',
  not_applicable: 'badge--gray',
  acknowledged: 'badge--blue',
  resolved: 'badge--green',
}

function SecurityStatusBadge({ status }) {
  if (!status) return null
  return (
    <span className={`badge ad-inline-badge ${SECURITY_STATUS_CLASS[status]} || 'badge--gray'}`>
      {SECURITY_STATUS_LABELS[status]} || titleCase(status)
    </span>
  )
}

/** Module 14 Phase 6: Lets an authorized human move this finding's Status
 * forward (or back, to reopen/correct one) - the AI itself can only ever
 * leave a finding Open or Not Applicable (see
 * SecurityFindingItem.normalize_status on the backend). Same "backend is
 * the real permission boundary, this control just shows the resulting
 * 403 inline" approach as RequirementReviewControl above. */
function SecurityFindingStatusControl({ finding, changeRequestId, token, onReviewed }) {
  const [editing, setEditing] = useState(false)
  const [choice, setChoice] = useState(finding.status)
  const [comment, setComment] = useState('')
  const [submitting, setSubmitting] = useState(false)
  const [error, setError] = useState(null)

  function handleSubmit(e) {
    e.preventDefault()
    setSubmitting(true)
    setError(null)
    updateSecurityFindingStatus(changeRequestId, finding.id, { status: choice, comment: comment || null }, token)
      .then(() => {
        setEditing(false)
        setComment('')
        return onReviewed()
      })
      .catch((err) => setError(err.message || 'Could not update this finding.'))
      .finally(() => setSubmitting(false))
  }

  if (!editing) {
    return (
      <button type="button" className="cr-btn cr-btn--ghost cr-btn--small ad-mt" onClick={() => setEditing(true)}>
        Update status
      </button>
    )
  }

  return (
    <form className="respond-form ad-mt" onSubmit={handleSubmit}>
      {error && <p className="cr-form-error">{error}</p>}
      <div className="status-form-row">
        <select
          className="cr-filter-select"
          value={choice}
          onChange={(e) => setChoice(e.target.value)}
          aria-label="Finding status"
        >
          {SECURITY_STATUS_OPTIONS.map((o) => (
            <option key={o.value} value={o.value}>
              {o.label}
            </option>
          ))}
        </select>
        <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={submitting}>
          {submitting ? 'Saving...' : 'Save'}
        </button>
        <button
          type="button"
          className="cr-btn cr-btn--ghost cr-btn--small"
          onClick={() => {
            setEditing(false)
            setError(null)
          }}
        >
          Cancel
        </button>
      </div>
      <div className="cr-field">
        <label htmlFor={`sec-comment-${finding.id}`}>
          Comment
          <span className="cr-optional-tag">optional</span>
        </label>
        <textarea
          id={`sec-comment-${finding.id}`}
          rows={2}
          value={comment}
          onChange={(e) => setComment(e.target.value)}
        </textarea>
      </div>
    </form>
  )
}

function SecurityFindingCard({ category, item, changeRequestId, token, onReviewed }) {
  return (
    <div className="cr-detail-section ad-impact-card">
      <h3 className="ad-impact-card-title">{SECURITY_CATEGORY_LABELS[category]} || titleCase(category)</h3>
      {item ? (
        <div className="ad-impact-card-badges">
          <span className={`badge ${LEVEL_CLASS[item.severity]} || 'badge--gray'}`>
            {titleCase(item.severity)} severity
          </span>
          <SecurityStatusBadge status={item.status} />
        </div>
        <p className="cr-detail-text ad-mt">{item.finding}</p>
        {item.evidence && <p className="cr-analysis-list-mitigation">Evidence: {item.evidence}</p>}
        {item.recommendation && <p className="cr-analysis-list-mitigation">Recommendation: {item.recommendation}</p>}
      )}
      <SecurityFindingStatusControl
        finding={item}
        changeRequestId={changeRequestId}
        token={token}
        onReviewed={onReviewed}
      />
    </div>
    <p className="ad-empty">Insufficient information.</p>
  )
}

```

```

// Module 14 Phase 5: the full 9-category grid, replacing the old flat
// concerns/summary blob as the Security tab's primary content. Kept
// separate from ImpactAnalysisSection above (different category set,
// different card fields) even though the card shell/CSS is shared.
function SecurityFindingsSection({ analysis, changeRequestId, token, onReviewed }) {
  const byCategory = {}
  analysis.security.findings.forEach((item) => {
    if (!byCategory[item.category]) byCategory[item.category] = item
  })
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Security Findings</h2>
      <div className="ad-impact-grid">
        {SECURITY_CATEGORY_ORDER.map((category) => (
          <SecurityFindingCard
            key={category}
            category={category}
            item={byCategory[category]}
            changeRequestId={changeRequestId}
            token={token}
            onReviewed={onReviewed}
          />
        ))}
      </div>
    </section>
  )
}

function ConfidenceLevelBadge({ level }) {
  if (!level) return null
  return (
    <span className={`badge ad-inline-badge ${CONFIDENCE_LEVEL_CLASS[level] || 'badge--gray'}`}>
      {CONFIDENCE_LEVEL_LABELS[level] || titleCase(level)} confidence
    </span>
  )
}

function CertaintyBadge({ certainty }) {
  if (!certainty) return null
  return (
    <span className={`badge ad-inline-badge ${CERTAINTY_CLASS[certainty] || 'badge--gray'}`}>
      {CERTAINTY_LABELS[certainty] || certainty}
    </span>
  )
}

// Same small formatter ChangeRequestDetail.jsx / NotificationsPage.jsx
// already each define locally - not worth sharing across three files for
// one date-formatting one-liner.
function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

function titleCase(value) {
  if (!value) return value
  return value.replace(/_/g, ' ').replace(/\b\w/g, (c) => c.toUpperCase())
}

function riskTone(scoreOutOf10) {
  if (scoreOutOf10 >= 7.5) return '#b42318'
  if (scoreOutOf10 >= 5) return '#c4320a'
  if (scoreOutOf10 >= 2.5) return '#b54708'
  return '#067647'
}

/** Splits the analysis engine's "Backend: X, Frontend: Y, Testing: Z,
 * Total: W" string back into its four parts for the summary card / Effort
 * tab. Falls back to nulls (rendered as "Insufficient information.") if the
 * text doesn't match - display-only, never blocks anything. */
function parseEffortEstimate(text) {
  if (!text) return null
  const labels = ['Backend', 'Frontend', 'Testing', 'Total']
  const result = {}
  labels.forEach((label, i) => {
    const rest = labels.slice(i + 1)
    const pattern = rest.length
      ? new RegExp(`${label}:\\s*(.*)?(?:,\\s*(?:${rest.join('|')})|:|)$`, 'i')
      : new RegExp(`${label}:\\s*(.*)$`, 'i')
    const match = text.match(pattern)
    result[label.toLowerCase()] = match ? match[1].replace(/\\s*$/, '').trim() : null
  })
  return result
}

function Gauge({ pct, color, children }) {
  const clamped = Math.max(0, Math.min(100, pct))
  return (
    <div className="ad-gauge" style={{ '--gauge-pct': `${clamped}%`, '--gauge-color': color }}>
      <div className="ad-gauge-inner">{children}</div>
    </div>
  )
}

function EmptyTab({ text }) {
  return (
    <section className="cr-detail-section">
      <p className="ad-empty">{text}</p>
    </section>
  )
}

// Module 15 Phase 5/6 (Repository Intelligence - Integration): the
// file path of any of this change request's own repository findings that
// share enough wording with this specific item to plausibly be "the
// file(s) behind this" - computed fresh by the backend
// (app/services/repository linkage.py), never a second, duplicate impact
// system. Renders nothing when empty - the common case for an analysis
// that predates any repository scan, or an item nothing in the repository
// overlaps with.
function RelatedFilesLine({ files }) {
  if (!files || files.length === 0) return null
  return (
    <p className="cr-analysis-list-mitigation">
      Related file{files.length > 1 ? 's' : ''}: {files.join(', ')}
    </p>
  )
}

// Module 17 Phase 4 (Test Cases & Implementation Plan 2.0 - Traceability):
// the "REQ-<id>"/"RISK-<id>" references app/services/traceability linkage.py

```

```

// computes for a test case/task - same "renders nothing when empty" rule
// as RelatedFilesLine above, for the same reason (a fresh analysis with no
// requirements/risks yet, or an item nothing overlaps with, is an honest
// "nothing found," not a gap to flag).
function ReferencesLine({ label, refs }) {
  if (!refs || refs.length === 0) return null
  return (
    <p className="cr-analysis-list-mitigation">
      {label}: {refs.join(', ')}
    </p>
  )
}

function ConfidenceBar({ value }) {
  return (
    <div className="ad-confidence-row">
      <div className="bar-track ad-confidence-track">
        <div className="bar-fill" style={{ width: `${Math.round(value)}%`, background: '#4338ca' }} />
      </div>
      <span className="ad-confidence-value">{Math.round(value)}% confidence</span>
    </div>
  )
}

function OverviewTab({ analysis, recommendedApprovals }) {
  const recMeta = analysis.recommendation ? RECOMMENDATION_META[analysis.recommendation] : null
  const affectedNames = analysis.affected_components.map(
    (c) => `${c.component_name} (${titleCase(c.component_type)})`
  )
  // Module 14 Phase 4: Business/Technical Impact now come straight from the
  // same impact assessments the Impact tab's structured cards use, instead
  // of being assembled here out of unrelated fields (business-type
  // Requirements, complexity reasoning) - one source of truth per finding,
  // per the architecture lock. 'undefined' (pre-Phase-4 analyses that
  // haven't been regenerated yet) falls through to the same "Insufficient
  // information." empty state as before.
  const businessImpact = analysis.impact_assessments.find((i) => i.category === 'business')
  const technicalImpact = analysis.impact_assessments.find((i) => i.category === 'technical')

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">AI Summary</h2>
      <p className="cr-detail-text">{analysis.summary}</p>
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Classification</h2>
      <dl className="cr-detail-grid">
        <dt>Category</dt>
        <dd>{analysis.category}</dd>
        <dt>Confidence</dt>
        <dd>{Math.round(analysis.confidence_score)}%</dd>
      </dl>
      {analysis.classification_reason && <p className="cr-detail-text ad-mt">{analysis.classification_reason}</p>}
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Business Impact</h2>
      {businessImpact ? (
        <ImpactAssessmentBadges item={businessImpact} />
        <p className="cr-detail-text ad-mt">{businessImpact.description}</p>
      ) : (
        <p className="ad-empty">Insufficient information.</p>
      )}
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Technical Impact</h2>
      {technicalImpact ? (
        <ImpactAssessmentBadges item={technicalImpact} />
        <p className="cr-detail-text ad-mt">{technicalImpact.description}</p>
        {affectedNames.length > 0 && (
          <p className="cr-detail-text ad-mt">
            <strong>Affects:</strong> {affectedNames.join(', ')}
          </p>
        )}
      ) : (
        <p className="ad-empty">Insufficient information.</p>
      )}
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Recommendation</h2>
      {recMeta ? (
        <p className="ad-recommendation-inline">
          <span aria-hidden="true">{recMeta.emoji}</span> <strong>{recMeta.label}</strong>
        </p>
      ) : (
        <p className="ad-empty">No recommendation given.</p>
      )}
      {analysis.recommendation_reasoning && <p className="cr-detail-text">{analysis.recommendation_reasoning}</p>}
      {analysis.workflow_recommendation && (
        <p className="cr-detail-text ad-mt approvals-recommendation">{analysis.workflow_recommendation}</p>
      )}
      /* Module 14 Phase 1: the same AI-derived recommended-approval-types
      list already used to build ChangeRequestDetail.jsx's Approvals
      section (GET .../approvals/recommended) - reused here as read-
      only badges. This never creates an Approval row itself; tagging
      an actual approver still only happens on the change request's
      detail page, per the architecture lock (one approval system). */
      {recommendedApprovals.length > 0 && (
        <div className="ad-mt ad-recommended-approvals">
          <span className="cr-detail-text-label">Recommended:</span>
          {recommendedApprovals.map((r) => (
            <span key={r.approval_type} className="badge badge--indigo ad-inline-badge">
              {r.label}
            </span>
          ))}
        </div>
      )}
      /* Module 22: recommended approvals is outdated - the change
      request has been edited since the analysis these
      recommendations were computed from, the same staleness
      signal the outdated-banner elsewhere on this page already
      warns about, just applied here since this section has no
      banner of its own to attach it to. */
      {recommendedApprovals[0]?.is_outdated && (
        <span className="badge badge--amber ad-inline-badge" title="The change request has been edited since this analysis ran.">
          May be outdated
        </span>
      )}
    </div>
  </section>

  <KnowledgeEvidenceSection evidence={analysis.knowledge_evidence} />
</div>
)

```

```

}

// Module 16 Phase 4/5 (Project Knowledge Base & RAG): the project
// documentation this specific analysis run actually grounded itself in -
// read straight off analysis.knowledge.evidence (already present on the
// AnalysisRead response, no separate fetch needed). Renders nothing when
// empty, which is the common case for any analysis that predates this
// module, or one where nothing in the knowledge base was genuinely
// relevant (min score filtering - see knowledge_embeddings.py). Each
// row's own is outdated (set by the API layer from the same
// is outdated computation as the analysis itself - see
// api/change_requests.py) is the ONLY staleness axis evidence has: it
// was always generated fresh as part of one analysis run, so "outdated"
// here means only "the change request has been edited since," never a
// second, independent staleness check.
function KnowledgeEvidenceSection({ evidence }) {
  if (!evidence || evidence.length === 0) return null
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Evidence Used</h2>
      <p className="cr-detail-text ig-intro">
        Excerpts from your team's own knowledge base that this analysis was grounded in - cited exactly as
        given to the AI, never asserted with more certainty than the similarity score below supports.
      </p>
      <ul className="cr-analysis-list ad-mt">
        {evidence.map((item) => (
          <li key={item.id} className="cr-analysis-list-item">
            {item.is_outdated && <span className="badge badge--amber">Outdated</span>}
            <div>
              <p className="cr-analysis-list-tag">
                {item.document_title}
                {item.section_label ? ` · ${item.section_label}` : ''}
              </p>
              <ConfidenceBar value={item.similarity_score * 100} />
              <p className="cr-analysis-list-text ad-mt">{item.content_snippet}</p>
              {item.is_outdated && (
                <p className="cr-analysis-list-mitigation">
                  This analysis (and the evidence it used) predates a later edit to the change request - re-analyze
                  for up-to-date grounding.
                </p>
              )}
            </div>
          </li>
        )}
      </ul>
    </section>
  )
}

// Module 14 Phase 6: a human reviewer's verdict on a Requirement -
// entirely separate from CertaintyBadge above (the AI's own Known/
// Inferred/Unknown label, never touched by reviewing).
const REVIEW_STATUS_LABELS = { confirmed: 'Confirmed', needs_clarification: 'Needs Clarification' }
const REVIEW_STATUS_CLASS = { confirmed: 'badge--green', needs_clarification: 'badge--amber' }

function ReviewStatusBadge({ status }) {
  if (!status) return null
  return (
    <span className={`badge ad-inline-badge ${REVIEW_STATUS_CLASS[status]} || 'badge--gray'}`>
      {REVIEW_STATUS_LABELS[status]}
    </span>
  )
}

/** Module 14 Phase 6: lets an authorized human mark this requirement
 * Confirmed or Needs Clarification. The real permission boundary is the
 * backend (workflow rules.can_review_analysis_findings) - this control is
 * shown to every viewer, and an unauthorized attempt simply surfaces the
 * backend's 403 message inline below, rather than duplicating the CR's
 * assignment-role logic on this page too. */
function RequirementReviewControl({ requirement, changeRequestId, token, onReviewed }) {
  const [editing, setEditing] = useState(false)
  const [choice, setChoice] = useState(requirement.review_status || 'confirmed')
  const [comment, setComment] = useState('')
  const [submitting, setSubmitting] = useState(false)
  const [error, setError] = useState(null)

  function handleSubmit(e) {
    e.preventDefault()
    setSubmitting(true)
    setError(null)
    reviewRequirement(changeRequestId, requirement.id, { review_status: choice, comment: comment || null }, token)
    .then(() => {
      setEditing(false)
      setComment('')
      return onReviewed()
    })
    .catch((err) => setError(err.message || 'Could not save this review.'))
    .finally(() => setSubmitting(false))
  }

  return (
    <div className="ad-mt">
      <ReviewStatusBadge status={requirement.review_status} />
      {editing ? (
        <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={() => setEditing(true)}>
          {requirement.review_status ? 'Change review' : 'Review'}
        </button>
      ) : (
        <form className="respond-form" onSubmit={handleSubmit}>
          {error && <p className="cr-form-error">{error}</p>}
          <div className="status-form-row">
            <select
              className="cr-filter-select"
              value={choice}
              onChange={(e) => setChoice(e.target.value)}
              aria-label="Review verdict"
            >
              <option value="confirmed">Confirmed</option>
              <option value="needs_clarification">Needs Clarification</option>
            </select>
            <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={submitting}>
              {submitting ? 'Saving...' : 'Save'}
            </button>
            <button
              type="button"
              className="cr-btn cr-btn--ghost cr-btn--small"
              onClick={() => {
                setEditing(false)
                setError(null)
              }}
            >
              Cancel
            </button>
          </div>
          {choice === 'needs_clarification' && (
            <div className="cr-field">
              <label htmlFor={`review-comment-${requirement.id}`}>
                Comment
              <span className="cr-optional-tag">required for this response</span>
              </label>
              <textarea

```

```

        id={`review-comment-${requirement.id}`}
        rows={2}
        value={comment}
        onChange={(e) => setComment(e.target.value)}
      </div>
    )}
  </form>
</div>
)
}

function RequirementsTab({ analysis, changeRequestId, token, onReviewed }) {
  const items = [...analysis.requirements].sort(
    (a, b) => (PRIORITY_ORDER[a.priority] ?? 9) - (PRIORITY_ORDER[b.priority] ?? 9)
  )
  if (items.length === 0) return <EmptyTab text="No requirements were extracted for this change." />
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Extracted Requirements</h2>
      <ul className="cr-analysis-list">
        {items.map((r) => (
          <li key={r.id} className="cr-analysis-list-item">
            <span className={`badge ${PRIORITY_CLASS[r.priority] || 'badge--gray'}`}>
              {PRIORITY_LABELS[r.priority] || r.priority}
            </span>
            <div>
              <p className="cr-analysis-list-tag">
                {titleCase(r.requirement_type)} <CertaintyBadge certainty={r.certainty} />
              </p>
              <p className="cr-analysis-list-text">{r.description}</p>
              {typeof r.confidence === 'number' && <ConfidenceBar value={r.confidence} />}
              {r.evidence && <p className="cr-analysis-list-mitigation">Evidence: {r.evidence}</p>}}
              <RequirementReviewControl
                requirement={r}
                changeRequestId={changeRequestId}
                token={token}
                onReviewed={onReviewed}
              />
            </div>
          </li>
        ))}
      </ul>
    </section>
  )
}

function ImpactTab({ analysis, changeRequest, crCode }) {
  const items = analysis.affected_components
  return (
    <ImpactAnalysisSection analysis={analysis} />
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Impact & Dependency Graph</h2>
      <p className="cr-detail-text ig-intro">
        Every affected component and dependency the AI identified for this change, branching from the change request itself. The analysis doesn't specify how these items relate to each other, so - rather than guessing at connections that were never asserted - each one is shown as directly tied to the change, grouped by category. Click any node for its full detail.
      </p>
      <ImpactGraph
        changeRequest={changeRequest}
        crCode={crCode}
        components={analysis.affected_components}
        dependencies={analysis.dependencies}
      />
    </section>
    {items.length === 0 ? (
      <EmptyTab text="No affected components were identified." />
    ) : (
      <section className="cr-detail-section">
        <h2 className="cr-section-title">Affected Components</h2>
        <ul className="cr-analysis-list">
          {items.map((c) => (
            <li key={c.id} className="cr-analysis-list-item">
              <span className={`badge ${LEVEL_CLASS[c.impact_level] || 'badge--gray'}`}>
                {titleCase(c.impact_level)} impact
              </span>
              <CertaintyBadge certainty={c.certainty} />
              <div>
                <p className="cr-analysis-list-tag">
                  {c.component_name} {titleCase(c.component_type)}
                </p>
                <p className="cr-analysis-list-text">{c.reason}</p>
                <ConfidenceBar value={c.confidence} />
                {/" Module 14 Phase 2 "/}
                {c.evidence && <p className="cr-analysis-list-mitigation">Evidence: {c.evidence}</p>}}
                <RelatedFilesLine files={c.related_files} />
              </div>
            </li>
          ))}
        </ul>
      </section>
    )}
  )
}

// Module 14 Phase 2: how directly this change actually relies on a
// dependency - distinct from its impact-level badge (how much *this
// change* would be affected). "Potential" is deliberately the AI's way of
// flagging "plausible but not confidently established" rather than
// asserting a dependency it isn't sure about - see the "do not invent
// dependencies" prompt rule in analysis_engine.py.
const RELATIONSHIP_LABELS = { direct: 'Direct', indirect: 'Indirect', potential: 'Potential' }
const RELATIONSHIP_CLASS = { direct: 'badge--blue', indirect: 'badge--purple', potential: 'badge--gray' }

function DependenciesTab({ analysis }) {
  const items = analysis.dependencies
  if (items.length === 0) return <EmptyTab text="No dependencies were identified." />
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Dependencies</h2>
      <ul className="cr-analysis-list">
        {items.map((d) => (
          <li key={d.id} className="cr-analysis-list-item">
            <span className={`badge ${LEVEL_CLASS[d.impact_level] || 'badge--gray'}`}>
              {titleCase(d.impact_level)} impact
            </span>
            {d.relationship_type && (
              <span className={`badge ${RELATIONSHIP_CLASS[d.relationship_type] || 'badge--gray'}`}>
                {RELATIONSHIP_LABELS[d.relationship_type] || titleCase(d.relationship_type)}
              </span>
            )}
            {d.risk_severity && (
              <span className={`badge ${LEVEL_CLASS[d.risk_severity] || 'badge--gray'}`}>
                {titleCase(d.risk_severity)} risk
              </span>
            )}
          </li>
        ))}
      </ul>
    </section>
  )
}

```

```

    </span>
  })
  <div>
    <p className="cr-analysis-list-tag">
      {d.dependency_name} {titleCase(d.dependency_type)}
    </p>
    <p className="cr-analysis-list-text">{d.reason}</p>
    <RelatedFilesLine files={d.related_files} />
  </div>
</li>
</ul>
</div>
</section>
}

/** Module 9: the 0-10 overall risk score, shown again (bigger, with
 * context) inside the Risks tab itself rather than only in the top summary
 * card - reuses the same Gauge/riskTone the summary card uses so the
 * number is always consistent. */
function OverallRiskCard({ analysis, items }) {
  const scoreOutOf10 = analysis.risk.score / 10
  const severityCounts = { critical: 0, high: 0, medium: 0, low: 0 }
  items.forEach((r) => {
    if (severityCounts[r.severity] !== undefined) severityCounts[r.severity] += 1
  })
  const highest = ['critical', 'high', 'medium', 'low'].find((level) => severityCounts[level] > 0)

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Overall Risk Score</h2>
      <div className="rk-overall">
        <Gauge pct={scoreOutOf10 * 10} color={riskTone(scoreOutOf10)}>
          <span className="ad-gauge-value">{scoreOutOf10.toFixed(1)}</span>
        </Gauge>
        <div className="rk-overall-text">
          <p className="ad-summary-text">{scoreOutOf10.toFixed(1)} out of 10</p>
          <p className="cr-detail-text">
            {items.length === 0
              ? 'No individual risks were identified for this change.'
              : `${items.length} risks${items.length === 1 ? '' : 's'} identified${
                  highest ? 'highest severity: ' + titleCase(highest) : ''
                }`}
          </p>
        </div>
      </div>
    </section>
  )
}

/** One risk, laid out as a small card - severity/category header, then
 * probability + score as mini bars, then the two narrative fields the spec
 * asks for explicitly: Explanation and Mitigation. "Explanation" here is
 * the AI's own description/explanation text (analysis_engine.py already
 * combines the two before storage) - nothing added beyond what it wrote. */
function RiskCard({ risk }) {
  return (
    <div className="rk-card">
      <div className="rk-card-header">
        <span className={`badge ${LEVEL_CLASS[risk.severity]} ${'badge--gray'}}>{titleCase(risk.severity)}</span>
        <span className="rk-card-category">{titleCase(risk.category)}</span>
        <CertaintyBadge certainty={risk.certainty} />
      </div>
      <div className="rk-card-metrics">
        <div className="rk-metric">
          <span className="rk-metric-label">Probability</span>
          <div className="rk-metric-row">
            <div className="bar-track">
              <div
                className="bar-fill"
                style={{ width: `${Math.round(risk.probability * 100)}%`, background: '#4338ca' }}
              </div>
            <span className="rk-metric-value">{Math.round(risk.probability * 100)}%</span>
          </div>
        </div>
        <div className="rk-metric">
          <span className="rk-metric-label">Score</span>
          <div className="rk-metric-row">
            <div className="bar-track">
              <div
                className="bar-fill"
                style={{ width: `${Math.round(risk.score)}%`, background: riskTone(risk.score / 10) }}
              </div>
            <span className="rk-metric-value">{Math.round(risk.score)} / 100</span>
          </div>
        </div>
      </div>
      <p className="rk-card-label">Explanation</p>
      <p className="cr-detail-text">{risk.description || 'Insufficient information.'}</p>
      <p className="rk-card-label">Mitigation</p>
      <p className="cr-detail-text">{risk.mitigation || 'Insufficient information.'}</p>
      {typeof risk.confidence === 'number' && (
        <p className="rk-card-label">AI Confidence in This Assessment</p>
        <p className="cr-detail-text">{Math.round(risk.confidence)}%</p>
      )}
      <RelatedFilesLine files={risk.related_files} />
    </div>
  )
}

function RisksTab({ analysis }) {
  const items = analysis.risks

  if (items.length === 0) {
    return (
      <OverallRiskCard analysis={analysis} items={items} />
      <EmptyTab text="No risks were identified for this change." />
    )
  }

  const severityCounts = { critical: 0, high: 0, medium: 0, low: 0 }
  items.forEach((r) => {
    if (severityCounts[r.severity] !== undefined) severityCounts[r.severity] += 1
  })
  const severityChartData = [
    { label: 'Critical', count: severityCounts.critical, color: '#b42318' },
    { label: 'High', count: severityCounts.high, color: '#c4320a' },
    { label: 'Medium', count: severityCounts.medium, color: '#b54708' },
    { label: 'Low', count: severityCounts.low, color: '#067647' },
  ]
}

```



```

const categoryCounts = {}
let otherCount = 0
items.forEach((r) => {
  if (RISK_CATEGORY_ORDER.includes(r.category)) {
    categoryCounts[r.category] = (categoryCounts[r.category] || 0) + 1
  } else {
    otherCount += 1
  }
})
const categoryChartData = RISK_CATEGORY_ORDER.map((cat) => ({
  label: titleCase(cat),
  count: categoryCounts[cat] || 0,
  color: RISK_CATEGORY_COLORS[cat],
}))
if (otherCount > 0) {
  categoryChartData.push({ label: 'Other', count: otherCount, color: RISK_CATEGORY_COLORS.other })
}

const sorted = [...items].sort((a, b) => (PRIORITY_ORDER[a.severity] ?? 9) - (PRIORITY_ORDER[b.severity] ?? 9))

return (
  <OverallRiskCard analysis={analysis} items={items} />
  <DistributionChart title="Risk Severity Breakdown" data={severityChartData} emptyLabel="No risks." />
  <DistributionChart title="Risk Category Breakdown" data={categoryChartData} emptyLabel="No risks." />
  <section className="cr-detail-section">
    <h2 className="cr-section-title">Risk Details</h2>
    <div className="rk-list">
      {sorted.map((r) => (
        <RiskCard key={r.id} risk={r} />
      ))}
    </div>
  </section>
)

/** Module 9's security checklist - built only from the concerns and
 * mitigations already shown above (each tagged so it's clear which is
 * which), turned into checkable review items. Purely a local, in-page
 * interaction (nothing persisted) - it doesn't add any fact that wasn't
 * already displayed in the sections above it. */
function SecurityChecklist({ concerns, mitigations }) {
  const items = useMemo(
    () => [
      ...concerns.map((text, i) => ({ id: `concern-${i}`, tag: 'Concern', text })),
      ...mitigations.map((text, i) => ({ id: `mitigation-${i}`, tag: 'Mitigation', text })),
    ],
    [concerns, mitigations]
  )
  const [checked, setChecked] = useState({})

  if (items.length === 0) return null

  const doneCount = items.filter((item) => checked[item.id]).length

  function toggle(id) {
    setChecked(prev => ({ ...prev, [id]: !prev[id] }))
  }

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Security Checklist</h2>
      <p className="cr-detail-text ig-intro">
        Built from the concerns and mitigations above - check items off as you review them for this change.
      </p>
      <div className="sec-checklist-progress">
        <div className="bar-track">
          <div
            className="bar-fill"
            style={{ width: `${items.length ? Math.round((doneCount / items.length) * 100) : 0}%`, background: '#067647' }}
          />
        </div>
        <span className="sec-checklist-progress-text">
          {doneCount}/{items.length} reviewed
        </span>
      </div>
      <ul className="sec-checklist">
        {items.map((item) => (
          <li key={item.id} className="sec-checklist-item">
            <label>
              <input type="checkbox" checked={Boolean(checked[item.id])} onChange={() => toggle(item.id)} />
              <span className={`sec-checklist-tag sec-checklist-tag-${item.tag.toLowerCase()}`}>{item.tag}</span>
              <span className={`sec-checklist-text${checked[item.id] ? ' sec-checklist-text-done' : ''}`}>
                {item.text}
              </span>
            </label>
          </li>
        ))}
      </ul>
    </section>
  )
}

function SecurityTab({ analysis, changeRequestId, token, onReviewed }) {
  let security = null
  try {
    security = analysis.security_analysis ? JSON.parse(analysis.security_analysis) : null
  } catch {
    security = null
  }
  const concerns = security?.concerns || []
  const summary = security?.summary
  const securityRisks = analysis.risks.filter((r) => r.category === 'security')
  const mitigations = securityRisks.map((r) => r.mitigation).filter(Boolean)
  // Module 14 Phase 5: once an analysis has real structured findings, they
  // fully replace the old flat concerns/mitigations/checklist rendering
  // below - showing both would be two different views of "what's wrong,
  // security-wise" built at different times or by different logic, which
  // is exactly the duplicate-system the architecture lock rules out. Only
  // an analysis that predates Phase 5 (no security findings rows at all)
  // still gets the old rendering.
  const hasStructuredFindings = analysis.security_findings.length > 0

  const hasAnything = hasStructuredFindings || Boolean(summary) || concerns.length > 0 || securityRisks.length > 0
  if (!hasAnything) {
    return <EmptyTab text="No security concerns were identified." />
  }

  // Security Risk Level is derived only from security-category risk items
  // (real AI-assigned severity/score) - never estimated from the concern
  // count, which would mean inventing a rating the AI never gave.
  const worstSeverity =
    securityRisks.length > 0
      ? securityRisks.reduce(
          (worst, r) => ((PRIORITY_ORDER[r.severity] ?? 9) < (PRIORITY_ORDER[worst] ?? 9) ? r.severity : worst),
          securityRisks[0].severity
        )
      : null
  const worstScore = securityRisks.length > 0 ? Math.max(...securityRisks.map((r) => r.score)) : null

```



```

return (
  <section className="cr-detail-section">
    <h2 className="cr-section-title">Security Risk Level</h2>
    {worstSeverity ? (
      <div className="sec-level-row">
        <span className={`badge sec-level-badge ${LEVEL_CLASS[worstSeverity]} || 'badge--gray'}`>
          {titleCase(worstSeverity)}
        </span>
        <span className="cr-detail-text">
          Based on {securityRisks.length} security-related risk{securityRisks.length === 1 ? '' : 's'} identified
          {worstScore !== null ? 'highest score ' + Math.round(worstScore)/100 : ''}.
        </span>
      </div>
    ) : (
      <p className="ad-empty">
        The AI didn't score a specific security risk level for this change
        {concerns.length > 0 ? ' - see the concerns below.' : ''}
      </p>
    )}
  </section>

  {summary && (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Summary</h2>
      <p className="cr-detail-text">{summary}</p>
    </section>
  )}

  {hasStructuredFindings ? (
    <SecurityFindingsSection
      analysis={analysis}
      changeRequestId={changeRequestId}
      token={token}
      onReviewed={onReviewed}
    />
  ) : (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Potential Concerns</h2>
      {concerns.length > 0 ? (
        <ul className="cr-plain-list">
          {concerns.map((concern, index) => (
            <li key={index}>{concern}</li>
          ))}
        </ul>
      ) : (
        <p className="ad-empty ad-mt">No specific concerns flagged.</p>
      )}
    </section>

    <section className="cr-detail-section">
      <h2 className="cr-section-title">Recommended Mitigations</h2>
      {mitigations.length > 0 ? (
        <ul className="cr-plain-list">
          {mitigations.map((mitigation, index) => (
            <li key={index}>{mitigation}</li>
          ))}
        </ul>
      ) : (
        <p className="ad-empty ad-mt">No specific mitigations beyond the concerns above were provided.</p>
      )}
    </section>

    <SecurityChecklist concerns={concerns} mitigations={mitigations} />
  )}
)

}

/** Module 14 Phase 3: Complexity and Effort share one tab (and one section
 * here) because the spec pairs them - both are estimates, both get a
 * Low/Medium/High confidence instead of a fake-precise number, and both
 * are explained in plain language rather than left as a bare badge. */
function ComplexityCard({ analysis }) {
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Complexity</h2>
      <div className="ad-complexity-row">
        <span className={`badge ad-summary-badge ${LEVEL_CLASS[analysis.complexity]} || 'badge--gray'}`>
          {titleCase(analysis.complexity)}
        </span>
        {analysis.complexity_confidence && <ConfidenceLevelBadge level={analysis.complexity_confidence} />}
      </div>
      <p className="cr-detail-text ad-mt">{analysis.complexity_reasoning || 'Insufficient information.'}</p>
    </section>
  )
}

function EffortTab({ analysis, effort }) {
  return (
    <ComplexityCard analysis={analysis} />
    {!analysis.effort_estimate ? (
      <EmptyTab text="No effort estimate available." />
    ) : (
      <section className="cr-detail-section">
        <h2 className="cr-section-title">
          Estimated Effort {analysis.effort_confidence && <ConfidenceLevelBadge level={analysis.effort_confidence} />}
        </h2>
        <p className="ad-effort-note">
          Ranges, not fake precision - the AI's honest estimate from the request text alone.
        </p>
        <div className="ad-effort-grid">
          {
            [
              { label: 'Backend', value: effort?.backend },
              { label: 'Frontend', value: effort?.frontend },
              { label: 'Testing', value: effort?.testing },
              { label: 'Total', value: effort?.total },
            ].map((row) => (
              <div key={row.label} className="ad-effort-row">
                <span className="ad-effort-label">{row.label}</span>
                <span className="ad-effort-value">{row.value || 'Insufficient information.'}</span>
              </div>
            ))}
        </div>
      </section>
    )}
  )
}

/** Answers one clarification question in place - the missing information
 * the AI itself asked for, typed in by whoever actually has it (often the
 * requester, not necessarily whoever's reviewing the analysis - so this is
 * open to anyone who can comment, same as ClarificationAnswerControl's own
 * backend gate). Submitting always marks the question Resolved; it can be
 * opened again afterward to correct the answer, which is why an already-
 * answered question still shows this control rather than locking it away. */

```

```

function ClarificationAnswerControl({ question, changeRequestId, token, onAnswered }) {
  const [editing, setEditing] = useState(false)
  const [answer, setAnswer] = useState(question.answer_text || '')
  const [submitting, setSubmitting] = useState(false)
  const [error, setError] = useState(null)

  function handleSubmit(e) {
    e.preventDefault()
    setSubmitting(true)
    setError(null)
    answerClarificationQuestion(changeRequestId, question.id, answer, token)
      .then(() => {
        setEditing(false)
        return onAnswered()
      })
      .catch((err) => setError(err.message || 'Could not save this answer.'))
      .finally(() => setSubmitting(false))
  }

  if (!editing) {
    return (
      <div className="ad-mt">
        {question.resolved && question.answer_text && (
          <p className="cr-question-answer">
            <span className="cr-question-answer-label">Answer:</span> {question.answer_text}
          </p>
        )}
        <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={() => setEditing(true)}>
          {question.resolved ? 'Edit answer' : 'Provide this information'}
        </button>
      </div>
    )
  }

  return (
    <form className="respond-form ad-mt" onSubmit={handleSubmit}>
      {error && <p className="cr-form-error">{error}</p>}
      <div className="cr-field">
        <label htmlFor={`clarification-answer-${question.id}`}>Your answer</label>
        <textarea
          id={`clarification-answer-${question.id}`}
          rows={3}
          value={answer}
          onChange={(e) => setAnswer(e.target.value)}
          placeholder="Type the missing information here..."
        />
      </div>
      <div className="status-form-row">
        <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={submitting || !answer.trim()}>
          {submitting ? 'Saving' : 'Save answer'}
        </button>
        <button
          type="button"
          className="cr-btn cr-btn--ghost cr-btn--small"
          onClick={() => {
            setEditing(false)
            setAnswer(question.answer_text || '')
            setError(null)
          }}
        > Cancel
        </button>
      </div>
    </form>
  )
}

function MissingInfoTab({ analysis, changeRequestId, token, onAnswered }) {
  const items = analysis.clarification_questions
  const unresolved = items.filter(q => !q.resolved)
  const sorted = [...items].sort((a, b) => (PRIORITY_ORDER[a.priority] ?? 9) - (PRIORITY_ORDER[b.priority] ?? 9))

  if (items.length === 0) {
    return (
      <div className="ad-missing-banner ad-missing-banner--clear">
        <span className="ad-missing-count">0</span>
        <span>No clarification items - nothing blocking this change.</span>
      </div>
    )
  }

  return (
    <div className={`ad-missing-banner ${unresolved.length > 0 ? 'ad-missing-banner--alert' : 'ad-missing-banner--clear'}`>
      <span className="ad-missing-count">{unresolved.length}</span>
      <span>{unresolved.length === 1 ? 'clarification item required.' : 'clarification items required.'}</span>
    </div>
    <section className="cr-detail-section">
      <ul className="cr-question-list">
        {sorted.map((q) => (
          <li key={q.id} className="cr-question-item">
            <span className={`badge ${q.resolved ? 'badge--green' : 'badge--amber'}`}>
              {q.resolved ? 'Resolved' : 'Open'}
            </span>
            <div>
              <p className="cr-question-text">
                <span className={`badge ad-inline-badge ${PRIORITY_CLASS[q.priority] || 'badge--gray'}`}>
                  {PRIORITY_LABELS[q.priority] || q.priority}
                </span>
                {q.question}
              </p>
              <p className="cr-question-reason">{q.reason}</p>
              <ClarificationAnswerControl
                question={q}
                changeRequestId={changeRequestId}
                token={token}
                onAnswered={onAnswered}
              />
            </div>
          </li>
        ))}
      </ul>
    </section>
  )
}

// Module 10: the AI Analysis Engine's real TestType vocabulary
// (analysis_engine.py's TEST_TYPES / app.models.enums.TestType) - note this
// doesn't literally match every label in the Module 10 spec's example list
// ("Functional", "Negative"); those aren't values the engine can produce.
// Rather than inventing new categories the AI was never asked to classify
// against, the filter below is built from whatever types are actually
// present in this analysis's real test cases.
const TEST_TYPE_LABELS = {
  unit: 'Unit',
  integration: 'Integration',
  e2e: 'End-to-End',
  regression: 'Regression',
  performance: 'Performance',
  security: 'Security',

```

```

manual: 'Manual',
// Module 17 Phase 1/2 (Test Cases & Implementation Plan 2.0)
negative: 'Negative',
boundary: 'Boundary',
api: 'API',
ui: 'UI',
data_validation: 'Data Validation',
}

// Module 17 Phase 3 - Module 12's own per-CR role vocabulary (see
// AssignmentRole in the backend), reused here purely to label a
// *suggestion* (see ImplementationPlanTab's own "Suggested Owner" field) -
// never an actual assignment control.
const ASSIGNMENT_ROLE_LABELS = {
  requester: 'Requester',
  owner: 'Owner',
  technical_lead: 'Technical Lead',
  reviewer: 'Reviewer',
  approver: 'Approver',
  security_reviewer: 'Security Reviewer',
  qa_owner: 'QA Owner',
  implementation_owner: 'Implementation Owner',
}

/** Module 17 Phase 6 (human editing): lets an authorized human correct one
 * AI-generated test case in place. Same pattern as RequirementReviewControl
 * above - shown to every viewer, the real permission boundary is the
 * backend (workflow rules.can review analysis findings), so an
 * unauthorized attempt just surfaces the backend's 403 message inline
 * rather than duplicating that logic here. Never touches this row's own
 * requirement/risk/related-file references - those stay computed fresh by
 * the backend on the next load, from whatever the edited wording now is. */
function TestCaseEditControl({ testCase, changeRequestId, token, onSave }) {
  const [editing, setEditing] = useState(false)
  const [title, setTitle] = useState(testCase.title)
  const [description, setDescription] = useState(testCase.description)
  const [testType, setTestType] = useState(testCase.test_type)
  const [priority, setPriority] = useState(testCase.priority)
  const [preconditions, setPreconditions] = useState(testCase.preconditions || '')
  const [steps, setSteps] = useState((testCase.steps || []).join('\n'))
  const [expectedResult, setExpectedResult] = useState(testCase.expected_result)
  const [reason, setReason] = useState('')
  const [submitting, setSubmitting] = useState(false)
  const [error, setError] = useState(null)

  function startEditing() {
    setTitle(testCase.title)
    setDescription(testCase.description)
    setTestType(testCase.test_type)
    setPriority(testCase.priority)
    setPreconditions(testCase.preconditions || '')
    setSteps((testCase.steps || []).join('\n'))
    setExpectedResult(testCase.expected_result)
    setReason('')
    setError(null)
    setEditing(true)
  }

  function handleSubmit(e) {
    e.preventDefault()
    setSubmitting(true)
    setError(null)
    updateTestCase(
      changeRequestId,
      testCase.id,
      {
        title,
        description,
        test_type: testType,
        priority,
        preconditions: preconditions || null,
        steps: steps
          .split('\n')
          .map(s => s.trim())
          .filter(Boolean),
        expected_result: expectedResult,
        reason: reason || null,
      },
      token
    )
    .then(() => {
      setEditing(false)
      return onSave()
    })
    .catch(err => setError(err.message || 'Could not save this test case.'))
    .finally(() => setSubmitting(false))
  }

  if (!editing) {
    return (
      <div className="ad-mt">
        {testCase.edited by && <span className="badge ad-inline-badge badge--blue">Edited</span>}
        <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={startEditing}>
          Edit
        </button>
      </div>
    )
  }

  return (
    <form className="respond-form ad-mt" onSubmit={handleSubmit}>
      <error && <p className="cr-form-error">{error}</p>
      <div className="cr-field">
        <label htmlFor={`tc-title-${testCase.id}`}>Title</label>
        <input
          id={`tc-title-${testCase.id}`}
          type="text"
          value={title}
          onChange={(e) => setTitle(e.target.value)}
        </input>
      </div>
      <div className="cr-field">
        <label htmlFor={`tc-description-${testCase.id}`}>Description</label>
        <textarea
          id={`tc-description-${testCase.id}`}
          rows={2}
          value={description}
          onChange={(e) => setDescription(e.target.value)}
        </textarea>
      </div>
      <div className="status-form-row">
        <select
          className="cr-filter-select"
          value={testType}
          onChange={(e) => setTestType(e.target.value)}
          aria-label="Test type"
        >
          {Object.entries(TEST_TYPE_LABELS).map(([value, label]) => (
            <option key={value} value={value}>
              {label}
            </option>
          ))}
        </select>
      </div>
    </form>
  )
}

```

```

    </option>
  )}
</select>
<select
  className="cr-filter-select"
  value={priority}
  onChange={(e) => setPriority(e.target.value)}
  aria-label="Priority"
>
  {Object.entries(PRIORITY_LABELS).map(([value, label]) => (
    <option key={value} value={value}>
      {label}
    </option>
  ))}
</select>
</div>
<div className="cr-field">
  <label htmlFor={`tc-preconditions-${testCase.id}`}>
    Preconditions <span className="cr-optional-tag">optional</span>
  </label>
  <textarea
    id={`tc-preconditions-${testCase.id}`}
    rows={2}
    value={preconditions}
    onChange={(e) => setPreconditions(e.target.value)}
  />
</div>
<div className="cr-field">
  <label htmlFor={`tc-steps-${testCase.id}`}>
    Steps <span className="cr-optional-tag">one per line</span>
  </label>
  <textarea id={`tc-steps-${testCase.id}`} rows={3} value={steps} onChange={(e) => setSteps(e.target.value)} />
</div>
<div className="cr-field">
  <label htmlFor={`tc-expected-${testCase.id}`}>Expected Result</label>
  <textarea
    id={`tc-expected-${testCase.id}`}
    rows={2}
    value={expectedResult}
    onChange={(e) => setExpectedResult(e.target.value)}
  />
</div>
<div className="cr-field">
  <label htmlFor={`tc-reason-${testCase.id}`}>
    Reason for this edit <span className="cr-optional-tag">optional</span>
  </label>
  <textarea id={`tc-reason-${testCase.id}`} rows={2} value={reason} onChange={(e) => setReason(e.target.value)} />
</div>
<div className="status-form-row">
  <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={submitting}>
    {submitting ? 'Saving...' : 'Save'}
  </button>
  <button
    type="button"
    className="cr-btn cr-btn--ghost cr-btn--small"
    onClick={() => {
      setEditing(false)
      setError(null)
    }}
  />
  <button type="button" className="cr-btn cr-btn--ghost cr-btn--small">Cancel</button>
</div>
</form>
)
}

function TestCasesTab({ analysis, changeRequestId, token, onRegenerate, regenerating, onSave }) {
  const items = analysis.testCases
  const [typeFilter, setTypeFilter] = useState('')
  const [priorityFilter, setPriorityFilter] = useState('')

  if (items.length === 0) return <EmptyTab text="No test cases were suggested." />

  const availableTypes = [...new Set(items.map((t) => t.testType))].sort()
  const availablePriorities = [...new Set(items.map((t) => t.priority))].sort()
  (a, b) => (PRIORITY_ORDER[a] ?? 9) - (PRIORITY_ORDER[b] ?? 9)
  const filtered = items.filter(
    (t) => (!typeFilter || t.testType === typeFilter) && (!priorityFilter || t.priority === priorityFilter)
  )

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Suggested Test Cases</h2>

      { /* Module 17 Phase 5 - every test case shares its parent analysis's
         own isOutdated exactly (see TestCaseRead's own docstring), so
         this reuses that same flag rather than computing a second,
         independent staleness check - mirrors the Repository tab's own
         outdated-banner pattern, just scoped to this one tab. Reuses the
         existing full re-analysis action (no separate "regenerate just
         the tests" AI capability - see AnalysisDashboardPage's own
         handleRegenerate). */ }
      {analysis.isOutdated && (
        <div className="outdated-banner">
          <span className="outdated-banner-text">
            <strong>These test cases may be outdated.</strong> The change request has been edited since this
            analysis ran - regenerate to get test cases based on the current version.
          </span>
          <button
            type="button"
            className="cr-btn cr-btn--primary cr-btn--small"
            onClick={onRegenerate}
            disabled={regenerating}
          >
            {regenerating ? 'Regenerating...' : 'Regenerate Test Cases'}
          </button>
        </div>
      )}

      <div className="tc-filters">
        <select
          className="cr-filter-select"
          value={typeFilter}
          onChange={(e) => setTypeFilter(e.target.value)}
          aria-label="Filter by type"
        >
          <option value="">All types</option>
          {availableTypes.map((type) => (
            <option key={type} value={type}>
              {TEST_TYPE_LABELS[type] || titleCase(type)}
            </option>
          ))}
        </select>
      </div>
    </section>
  )
}

```

```

<select
  className="cr-filter-select"
  value={priorityFilter}
  onChange={(e) => setPriorityFilter(e.target.value)}
  aria-label="Filter by priority"
>
  <option value="">All priorities</option>
  {availablePriorities.map((p) => (
    <option key={p} value={p}>
      {PRIORITY_LABELS[p] || p}
    </option>
  ))}
</select>
<span className="tc-filter-count">
  {filtered.length} of {items.length} shown
</span>
</div>

{filtered.length === 0 ? (
  <p className="ad-empty">No test cases match these filters.</p>
) : (
  <ul className="cr-analysis-list">
    {filtered.map((t) => (
      <li key={t.id} className="cr-analysis-list-item">
        <span className={`badge ${PRIORITY_CLASS[t.priority]} || 'badge--gray'`}>
          {PRIORITY_LABELS[t.priority] || t.priority}
        </span>
        <div>
          <p className="cr-analysis-list-tag">
            {t.test_id} {t.title} {TEST_TYPE_LABELS[t.test_type] || titleCase(t.test_type)}
          </p>
          <p className="cr-analysis-list-text">{t.description}</p>
          <p className="cr-analysis-list-mitigation">Expected: {t.expected_result}</p>
          {t.preconditions && <p className="cr-analysis-list-mitigation">Preconditions: {t.preconditions}</p>
          {t.steps && t.steps.length > 0 && (
            <ol className="tc-steps">
              {t.steps.map((step, i) => (
                <li key={i}>{step}</li>
              ))}
            </ol>
          )}
          <ReferencesLine label="Traces to requirement(s)" refs={t.requirement_references} />
          <ReferencesLine label="Traces to risk(s)" refs={t.risk_references} />
          <RelatedFilesLine files={t.related_files} />
          <TestCaseEditControl testCase={t} changeRequestId={changeRequestId} token={token} onSave={onSaved} />
        </div>
      </li>
    ))}
  </ul>
</div>
)
}

/** Module 17 Phase 6 (human editing): same pattern as TestCaseEditControl
 * above, applied to one ImplementationTask instead - see that control's
 * own docstring. */
function ImplementationTaskEditControl({ task, changeRequestId, token, onSave }) {
  const [editing, setEditing] = useState(false)
  const [taskName, setTaskName] = useState(task.task)
  const [description, setDescription] = useState(task.description)
  const [component, setComponent] = useState(task.component)
  const [priority, setPriority] = useState(task.priority)
  const [estimatedEffort, setEstimatedEffort] = useState(task.estimated_effort)
  const [dependencies, setDependencies] = useState(task.dependencies || '')
  const [ownerSuggestion, setOwnerSuggestion] = useState(task.owner_suggestion || '')
  const [reason, setReason] = useState('')
  const [submitting, setSubmitting] = useState(false)
  const [error, setError] = useState(null)

  function startEditing() {
    setTaskName(task.task)
    setDescription(task.description)
    setComponent(task.component)
    setPriority(task.priority)
    setEstimatedEffort(task.estimated_effort)
    setDependencies(task.dependencies || '')
    setOwnerSuggestion(task.owner_suggestion || '')
    setReason('')
    setError(null)
    setEditing(true)
  }

  function handleSubmit(e) {
    e.preventDefault()
    setSubmitting(true)
    setError(null)
    updateImplementationTask(
      changeRequestId,
      task.id,
      {
        task: taskName,
        description,
        component,
        priority,
        estimated_effort: estimatedEffort,
        dependencies: dependencies || null,
        owner_suggestion: ownerSuggestion || null,
        reason: reason || null,
      },
      token
    )
    .then(() => {
      setEditing(false)
      return onSave()
    })
    .catch((err) => setError(err.message || 'Could not save this task.'))
    .finally(() => setSubmitting(false))
  }

  if (!editing) {
    return (
      <div className="ad-mt">
        {task.edited by && <span className="badge ad-inline-badge badge--blue">Edited</span>
        <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={startEditing}>
          Edit
        </button>
      </div>
    )
  }

  return (
    <form className="respond-form ad-mt" onSubmit={handleSubmit}>
      {error && <p className="cr-form-error">{error}</p>
      <div className="cr-field">
        <label htmlFor={`ip-task-${task.id}`}>Task</label>
        <input
          id={`ip-task-${task.id}`}
          type="text"
          value={taskName}
          onChange={(e) => setTaskName(e.target.value)}

```

```

    />
  </div>
  <div className="cr-field">
    <label htmlFor={`ip-description-${task.id}`}>Description</label>
    <textarea
      id={`ip-description-${task.id}`}
      rows={2}
      value={description}
      onChange={(e) => setDescription(e.target.value)}
    />
  </div>
  <div className="status-form-row">
    <input
      type="text"
      className="cr-filter-select"
      value={component}
      onChange={(e) => setComponent(e.target.value)}
      aria-label="Component"
    />
    <select
      className="cr-filter-select"
      value={priority}
      onChange={(e) => setPriority(e.target.value)}
      aria-label="Priority"
    >
      {Object.entries(PRIORITY_LABELS).map(([value, label]) => (
        <option key={value} value={value}>
          {label}
        </option>
      ))}
    </select>
  </div>
  <div className="status-form-row">
    <input
      type="text"
      className="cr-filter-select"
      value={estimatedEffort}
      onChange={(e) => setEstimatedEffort(e.target.value)}
      aria-label="Estimated effort"
    />
    <select
      className="cr-filter-select"
      value={ownerSuggestion}
      onChange={(e) => setOwnerSuggestion(e.target.value)}
      aria-label="Suggested owner"
    >
      <option value="">No suggestion</option>
      {Object.entries(ASSIGNMENT_ROLE_LABELS).map(([value, label]) => (
        <option key={value} value={value}>
          {label}
        </option>
      ))}
    </select>
  </div>
  <div className="cr-field">
    <label htmlFor={`ip-dependencies-${task.id}`}>
      Depends on <span className="cr-optional-tag">optional</span>
    </label>
    <input
      id={`ip-dependencies-${task.id}`}
      type="text"
      value={dependencies}
      onChange={(e) => setDependencies(e.target.value)}
    />
  </div>
  <div className="cr-field">
    <label htmlFor={`ip-reason-${task.id}`}>
      Reason for this edit <span className="cr-optional-tag">optional</span>
    </label>
    <textarea id={`ip-reason-${task.id}`} rows={2} value={reason} onChange={(e) => setReason(e.target.value)} />
  </div>
  <div className="status-form-row">
    <button type="submit" className="cr-btn cr-btn--primary cr-btn--small" disabled={submitting}>
      {submitting ? 'Saving...' : 'Save'}
    </button>
    <button
      type="button"
      className="cr-btn cr-btn--ghost cr-btn--small"
      onClick={() => {
        setEditing(false)
        setError(null)
      }}
    >
      Cancel
    </button>
  </div>
</form>
)
}

/** Module 10: implementation plan shown as an ordered, numbered timeline -
 * the order is simply the order the AI returned implementation plan in
 * (already the order it persisted in), not a computed schedule. */
function ImplementationPlanTab({ analysis, changeRequestId, token, onRegenerate, regenerating, onSave }) {
  const items = analysis.implementationTasks
  if (items.length === 0) return <EmptyTab text="No implementation plan was suggested." />
  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Implementation Plan</h2>
      <p className="cr-detail-text ig-intro">Shown in the order the AI suggested.</p>

      {/* Module 17 Phase 5 - see TestCasesTab's own identical banner for
       why this reuses analysis.is outdated and the existing full
       re-analysis action rather than a separate computation/capability. */}
      {analysis.is outdated && (
        <div className="outdated-banner">
          <span className="outdated-banner-text">
            <span className="outdated-banner-icon" aria-hidden="true">
              </span>
            </span>
            <strong>This implementation plan may be outdated.</strong> The change request has been edited since
            this analysis ran - regenerate to get a plan based on the current version.
          </span>
          <button
            type="button"
            className="cr-btn cr-btn--primary cr-btn--small"
            onClick={onRegenerate}
            disabled={regenerating}
          >
            {regenerating ? 'Regenerating...' : 'Regenerate Implementation Plan'}
          </button>
        </div>
      )}

      <ol className="ip-timeline">
        {items.map((t, index) => (
          <li key={t.id} className="ip-step">
            <div className="ip-step-marker">

```

```

    <span className="ip-step-number">{index + 1}</span>
    {index < items.length - 1 && <span className="ip-step-line" aria-hidden="true" />}
  </div>
  <div className="ip-step-card">
    <div className="ip-step-header">
      <span className={`badge ${PRIORITY_CLASS[t.priority]} || 'badge--gray'`}>
        {PRIORITY_LABELS[t.priority]} || t.priority
      </span>
      <span className="ip-step-task">{t.task}</span>
    </div>
    <p className="cr-detail-text">{t.description}</p>
    <dl className="ip-step-meta">
      <dt>Component</dt>
      <dd>{t.component}</dd>
      <dt>Estimated Effort</dt>
      <dd>{t.estimated_effort}</dd>
      <dt>Suggested Owner</dt>
      <dd>{ASSIGNMENT_ROLE_LABELS[t.owner_suggestion] || titleCase(t.owner_suggestion)}</dd>
    </dl>
    {t.dependencies && <p className="cr-analysis-list-mitigation">Depends on: {t.dependencies}</p>
    <ReferencesLine label="Traces to requirement(s)" refs={t.requirement_references} />
    <RelatedFilesLine files={t.related_files} />
    <ImplementationTaskEditControl task={t} changeRequestId={changeRequestId} token={token} onSave={onSaved} />
  </div>
</li>
</ul>
</section>
)
}

/** Module 17 Phase 4 (Test Cases & Implementation Plan 2.0 -
 * Traceability): Requirement -> Implementation Task -> Test Case, grouped
 * by requirement. The links themselves are computed server-side
 * (app/services/traceability_linkage.py) by keyword overlap - never
 * AI-generated, so a requirement with nothing listed under it just means
 * no task/test case's wording overlapped enough to link automatically,
 * not a real gap in coverage. Anything that didn't link to any
 * requirement is listed separately at the end, so nothing the AI produced
 * silently disappears from this view. */
function TraceabilityTab({ analysis }) {
  const requirements = analysis.requirements
  const risks = analysis.risks
  const testCases = analysis.test_cases
  const tasks = analysis.implementation_tasks

  if (requirements.length === 0 && risks.length === 0) {
    return (
      <EmptyTab text="No requirements or risks were extracted for this change, so there's nothing yet to trace implementation tasks or test cases back to." />
    )
  }

  const rows = requirements.map((r) => {
    const ref = `REQ-${r.id}`
    return {
      ref,
      requirement: r,
      tasks: tasks.filter((t) => t.requirement_references.includes(ref)),
      testCases: testCases.filter((t) => t.requirement_references.includes(ref)),
    }
  })

  const untracedTasks = tasks.filter((t) => t.requirement_references.length === 0)
  const untracedTestCases = testCases.filter((t) => t.requirement_references.length === 0)

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Traceability</h2>
      <p className="cr-detail-text ig-intro">
        Requirement &arr; Implementation Task &arr; Test Case, computed by matching each task&apos;s/test
        case&apos;s own wording against this analysis&apos;s own requirements - never AI-generated, so an ID here
        never points at something that doesn&apos;t actually exist.
      </p>
      {rows.length === 0 ? (
        <p className="ad-empty">No requirements were extracted for this change.</p>
      ) : (
        <ul className="tl-list">
          {rows.map(({ ref, requirement, tasks: linkedTasks, testCases: linkedTestCases }) => (
            <li key={ref} className="tl-row">
              <div className="tl-requirement">
                <span className={`badge ${PRIORITY_CLASS[requirement.priority]} || 'badge--gray'`}>{ref}</span>
                <p className="cr-analysis-list-text">{requirement.description}</p>
              </div>
              <div className="tl-linked">
                <div className="tl-column">
                  <h4 className="tl-column-title">Implementation Tasks</h4>
                  {linkedTasks.length === 0 ? (
                    <p className="ad-empty">None linked.</p>
                  ) : (
                    <ul className="tl-column-list">
                      {linkedTasks.map((t) => (
                        <li key={t.id}>{t.task}</li>
                      ))}
                    </ul>
                  )}
                </div>
                <div className="tl-column">
                  <h4 className="tl-column-title">Test Cases</h4>
                  {linkedTestCases.length === 0 ? (
                    <p className="ad-empty">None linked.</p>
                  ) : (
                    <ul className="tl-column-list">
                      {linkedTestCases.map((t) => (
                        <li key={t.id}>
                          {t.test_id} &mdash; {t.title}
                        </li>
                      ))}
                    </ul>
                  )}
                </div>
              </div>
            </li>
          ))}
        </ul>
      )}

      {(untracedTasks.length > 0 || untracedTestCases.length > 0) && (
        <div className="tl-untraced">
          <h3 className="tl-untraced-title">Not linked to a specific requirement</h3>
          <p className="cr-detail-text ig-intro">
            Still part of the plan - their own wording just didn&apos;t overlap enough with any single requirement
            above to link automatically.
          </p>
          {untracedTasks.length > 0 && (
            <p className="cr-analysis-list-mitigation">
              Implementation tasks: {untracedTasks.map((t) => t.task).join(', ')}
            </p>
          )}
        </div>
      )}
    </section>
  )
}

```

```

    </p>
  })
  {untracedTestCases.length > 0 && (
    <p className="cr-analysis-list-mitigation">
      Test cases: {untracedTestCases.map((t) => `${t.test_id} (${t.title})`).join(', ')}
    </p>
  )}
</div>
</section>
)
}

/** Module 13 Phase 2/5: every analysis ever run for this change request,
 * newest first, with a "View changes vs. previous" action per row that
 * fetches a computed (not AI-written) diff against whichever analysis ran
 * immediately before it - mirrors the CR-version "Compare Versions" UI
 * Module 12 already built (same compare-table styling), just for
 * analysis-to-analysis instead of version-to-version.
 *
 * Deliberately tracks TWO different numbers per row, never conflating
 * them: "Analysis #N" is just this row's position in this list (oldest
 * run is #1, and a plain re-analyze with no content edit always adds a
 * new #N here) - it has nothing to do with the CR's own version number,
 * which only Module 12's edit-triggered versioning ever changes. "CR vX"
 * is that real content version this particular run analyzed. Two runs
 * can and often will share the same CR version (re-analyzing without
 * editing in between) - that's correct, not a bug, and is exactly why
 * these can't be the same number. */
function AnalysisHistoryTab({ history, currentAnalysisId, compareToId, compareResult, compareLoading, onToggleCompare }) {
  if (!history || history.length === 0) {
    return <EmptyTab text="No analysis history available." />
  }

  return (
    <section className="cr-detail-section">
      <h2 className="cr-section-title">Analysis History</h2>
      <p className="cr-detail-text ig-intro">
        Every AI analysis run for this change request, newest first - re-analyzing always adds a new entry here, even
        without editing the change request itself. "View changes" computes a diff against the analysis run
        immediately before it - no extra AI call, and it&apos;s never wrong about the numbers.
      </p>
      <ul className="versions-list">
        {history.map((a, index) => {
          const isOldest = index === history.length - 1
          const runNumber = history.length - index
          const recMeta = a.recommendation ? RECOMMENDATION_META[a.recommendation] : null
          const isOpen = compareToId === a.id

          return (
            <li key={a.id}>
              <div className="version-row">
                <div className="version-row-main">
                  <span className="version-row-number">
                    Analysis #{runNumber} <span className="version-row-cr-version">(CR v{a.change_request_version ?? '-'})</span>
                    {a.id === currentAnalysisId ? ' (current)' : ''}
                  </span>
                  <span className="version-row-summary">
                    {titleCase(a.category)} {titleCase(a.complexity)} complexity {a.risk} risk{' '}
                    {Math.round(a.risk_score)/100}
                    {recMeta ? ` · ${recMeta.label}` : ''}
                  </span>
                  <span className="version-row-meta">{formatDate(a.created_at)}</span>
                </div>
                {!isOldest && (
                  <button
                    type="button"
                    className="cr-btn cr-btn--secondary cr-btn--small"
                    onClick={() => onToggleCompare(a.id)}
                  >
                    {isOpen ? 'Hide changes' : 'View changes vs. previous'}
                  </button>
                )}
              </div>
              {isOpen && (
                <div className="cr-detail-section" style={{ marginTop: '0.5rem' }}>
                  {compareLoading && <p className="cr-list-status-text">Comparing...</p>
                  {compareResult?.error && (
                    <p className="cr-list-status-text cr-list-status-text--error">{compareResult.error}</p>
                  )}
                  {compareResult?.fields && (
                    <div>
                      {compareResult.is_significant_change && (
                        <div className="ad-missing-banner ad-missing-banner--alert">
                          <span className="ad-missing-count">!</span>
                          <span>
                            Significant change detected: {compareResult.significant_change_reasons.join(' ')}
                          </span>
                        </div>
                      )}
                    </div>
                    <table className="compare-table">
                      <thead>
                        <tr>
                          <th>Field</th>
                          <th>Previous</th>
                          <th>This Version</th>
                        </tr>
                      </thead>
                      <tbody>
                        {compareResult.fields
                          .filter((f) => f.status !== 'unchanged')
                          .map((f) => (
                            <tr key={f.field} className={`compare-row-${f.status}`}>
                              <td>{f.label}</td>
                              <td className="compare-old">{f.old_value ?? '-'}</td>
                              <td className="compare-new">{f.new_value ?? '-'}</td>
                            </tr>
                          ))}
                        {compareResult.fields.every((f) => f.status === 'unchanged') && (
                          <tr>
                            <td colspan={3} className="compare-row--unchanged">
                              No field differences between these two analyses.
                            </td>
                          </tr>
                        )}
                      </tbody>
                    </table>
                    {(compareResult.requirements_added.length > 0 ||
                     compareResult.requirements_removed.length > 0) && (
                      <div className="ad-mt">
                        <p className="rk-card-label">Requirements</p>
                        {compareResult.requirements_added.map((r) => (
                          <p key={`req-add-${r}`} className="cr-analysis-list-text diff-added">
                            + {r}
                          </p>
                        ))}
                        {compareResult.requirements_removed.map((r) => (
                          <p key={`req-rem-${r}`} className="cr-analysis-list-text diff-removed">

```



```

    {r}
  }
</div>
))
}

{compareResult.affected_components_added.length > 0 ||
compareResult.affected_components_removed.length > 0} && (
  <div className="ad-mt">
    <p className="rk-card-label">Affected Components</p>
    {compareResult.affected_components_added.map((c) => {
      <p key={`comp-add-${c}`} className="cr-analysis-list-text diff-added">
        {c}
      </p>
    })}
    {compareResult.affected_components_removed.map((c) => {
      <p key={`comp-rem-${c}`} className="cr-analysis-list-text diff-removed">
        {c}
      </p>
    })}
  </div>
</div>
))
</li>
))
</ul>
</section>
)
}

/** Module 15 (Repository Intelligence): this module's own headline
 * question - "which source files may actually be affected by this change
 * request?" - answered as its own tab rather than folded into the Impact
 * tab, since a repository finding is a different KIND of thing (evidence
 * about the actual codebase) from an AI-inferred affected component. Every
 * row here already passed the backend's hallucination guard (a file the
 * AI named that wasn't in the matcher's own candidate shortlist is never
 * persisted at all - app/services/repository_matcher.py) and its
 * match label is always one of exactly three hedged values, computed
 * server-side from a numeric confidence score - never trusted from the
 * AI's own wording (see MATCH_LABEL_LABELS above). Version-aware like
 * everything else this module touches: 'is outdated'/'outdated reasons'
 * are computed fresh by the backend every time this loads (Module 15
 * Phase 4), never stored - the same rule the analysis's own outdated
 * banner already follows. */
function RepositoryTab({ findings, scan, onRescan, rescanning, rescanError }) {
  const anyOutdated = findings.some(f => f.is_outdated)
  const sorted = [...findings].sort((a, b) => b.confidence - a.confidence)

  return (
    <section className="cr-detail-section">
      <div className="ad-header--row">
        <div>
          <h2 className="cr-section-title">Repository / Affected Files</h2>
          <p className="cr-detail-text ig-intro">
            Source files from the project&aposs own repository that may actually be affected by this change,
            matched against this analysis - never asserted with more certainty than the evidence supports (see
            each row&aposs label and evidence below).
          </p>
          {scan && (
            <p className="cr-detail-text ad-mt">
              Last scan: {scan.file_count} file{scan.file_count === 1 ? '' : 's'} indexed
              {scan.skipped_count > 0 ? ', ${scan.skipped_count} skipped' : ''} on {formatDate(scan.started_at)}.
            </p>
          )}
        </div>
        <button
          type="button"
          className="cr-btn cr-btn--secondary cr-btn--small"
          onClick={onRescan}
          disabled={!rescanning}
        >
          {rescanning ? 'Re-scanning...' : 'Re-scan Repository'}
        </button>
      </div>
      {rescanError && <p className="cr-form-error ad-header-error">{rescanError}</p>}
    </section>

    {anyOutdated && (
      <div className="outdated-banner">
        <span className="outdated-banner-text">
          <span className="outdated-banner-icon" aria-hidden="true">
            </span>
          <strong>Some repository findings below may be outdated.</strong> The change request has been edited,
          a newer AI analysis has run, or a newer repository scan is available since these were generated - see
          each row for which.
        </span>
        <button
          type="button"
          className="cr-btn cr-btn--primary cr-btn--small"
          onClick={onRescan}
          disabled={!rescanning}
        >
          {rescanning ? 'Re-scanning...' : 'Re-scan Repository'}
        </button>
      </div>
    ))

    {sorted.length === 0 ? (
      <EmptyTab
        text={
          scan
          ? 'No files in the scanned repository were found to plausibly relate to this change request yet - ' +
            'click "Re-scan Repository" above to try again after the code has changed, or after re-analyzing.'
          : 'The repository hasn't been scanned yet - click "Re-scan Repository" above to index it and ' +
            'match it against this change request.'
        }
      />
    ) : (
      <section className="cr-detail-section">
        <ul className="cr-analysis-list">
          {sorted.map((f) => (
            <li key={f.id} className="cr-analysis-list-item">
              <span className={`badge ${MATCH_LABEL_CLASS[f.match_label]} || 'badge--gray'`} >
                {MATCH_LABEL_LABELS[f.match_label] || titleCase(f.match_label)}
              </span>
              <span className={`badge ${LEVEL_CLASS[f.impact_level]} || 'badge--gray'`} >
                {titleCase(f.impact_level)} impact
              </span>
              {f.is_outdated && <span className="badge badge--amber">Outdated</span>}
            </li>
          ))}
        </ul>
      </section>
    )}
  )
}

```

```

        {f.file_path}
        {f.language ? ` · ${titleCase(f.language)}` : ''}
      </p>
      <p className="cr-analysis-list-text">{f.reason}</p>
      <ConfidenceBar value={f.confidence} />
      <p className="cr-analysis-list-mitigation">Evidence: {f.evidence}</p>
      {f.is_outdated && f.outdated_reasons.length > 0 && (
        <p className="cr-analysis-list-mitigation">Why outdated: {f.outdated_reasons.join(' ')}</p>
      )}
    </div>
  </li>
</ul>
</section>
</>
}
)

/** Module 7: the dedicated AI Analysis Dashboard - header, five summary
 * cards, and a 10-tab breakdown of the latest analysis for one change
 * request. Pure read view of GET /api/change-requests/{id} (title) and
 * GET /api/change-requests/{id}/analysis (everything else, Module 6) - no
 * AI calls happen here, and every number/label on the page traces back to
 * a real stored field (nothing invented for the demo). */
export default function AnalysisDashboardPage({ id, onBackToDetail, onBackToList }) {
  const { token } = useAuth()
  const [changeRequest, setChangeRequest] = useState(null)
  const [analysis, setAnalysis] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [activeTab, setActiveTab] = useState('overview')
  // Spec: clicking the "CR v1"/"Analysis v1" badges at the top of the page
  // should jump straight to the version/analysis history below, rather
  // than making the visitor go hunt for the History tab themselves.
  const tabContentRef = useRef(null)
  const goToHistoryTab = useCallback(() => {
    setActiveTab('history')
    tabContentRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' })
  }, [])
  const [regenerating, setRegenerating] = useState(false)
  const [regenerateError, setRegenerateError] = useState(null)
  const [downloading, setDownloading] = useState(false)
  const [downloadError, setDownloadError] = useState(null)
  // Module 20 (Version-Aware Reports): the structured 409 the report
  // endpoint now returns when the current version hasn't been analyzed
  // (rather than silently building a misleading report) - null whenever
  // the last download attempt didn't hit that specific case. The full
  // version list backs the "generate a report for an earlier version"
  // picker.
  const [reportMismatch, setReportMismatch] = useState(null)
  const [reportableVersions, setReportableVersions] = useState([])
  const [selectedReportVersion, setSelectedReportVersion] = useState('')
  // Module 13 Phase 2/5: analysis history + compare.
  const [analysisHistory, setAnalysisHistory] = useState([])
  const [compareToId, setCompareToId] = useState(null)
  const [compareResult, setCompareResult] = useState(null)
  const [compareLoading, setCompareLoading] = useState(false)
  // Module 14 Phase 1: the same AI-derived recommended-approval-types list
  // ChangeRequestDetail.jsx's Approvals section already fetches - shown
  // here too so "what does this need to move forward" is visible without
  // leaving the Analysis Dashboard.
  const [recommendedApprovals, setRecommendedApprovals] = useState([])
  // Module 15 (Repository Intelligence).
  const [repositoryFindings, setRepositoryFindings] = useState([])
  const [repositoryScan, setRepositoryScan] = useState(null)
  const [rescanning, setRescanning] = useState(false)
  const [rescanError, setRescanError] = useState(null)

  const loadAll = useCallback(() => {
    return Promise.all([
      getChangeRequest(id, token),
      getAnalysis(id, token),
      getAnalysisHistory(id, token),
      getRecommendedApprovals(id, token),
      getRepositoryFindings(id, token),
      // 404s (as an ApiError) if no repository scan has ever been run at
      // all - a normal, not-yet-happened state for a fresh install, not a
      // page-level error, so it's swallowed to null right here rather than
      // failing the whole Promise.all.
      getLatestRepositoryScan(token).catch(() => null),
      // Module 20: powers the "generate a report for an earlier version"
      // picker below - never fails the whole page load if it errors (a
      // report picker that's temporarily unavailable shouldn't block
      // seeing the analysis itself).
      listReportableVersions(id, token).catch(() => []),
    ]).then([cr, an, hist, recommended, findings, scan, versions]) => {
      setChangeRequest(cr)
      setAnalysis(an)
      setAnalysisHistory(hist)
      setRecommendedApprovals(recommended)
      setReportableVersions(versions)
      setRepositoryFindings(findings)
      setRepositoryScan(scan)
    }
  }, [id, token])

  useEffect(() => {
    let cancelled = false
    setLoading(true)
    setError(null)
    setActiveTab('overview')
    setCompareToId(null)
    setCompareResult(null)

    loadAll()
      .catch((err) => {
        if (!cancelled) setError(err.message || 'Could not load this analysis.')
      })
      .finally(() => {
        if (!cancelled) setLoading(false)
      })

    return () => {
      cancelled = true
    }
  }, [id, token])

  /** Module 10: "Regenerate Analysis" - the architecture already supports
   * re-running the AI on this change request (Module 6's POST .../analyze,
   * already used by the "Re-analyze" button on the change request's detail
   * page). This just exposes the same action from the dashboard itself, so
   * a fresh run's results (new test cases, new implementation plan, etc.)
   * show up here without navigating away first. */
  function handleRegenerate() {
    setRegenerating(true)
    setRegenerateError(null)
    setCompareToId(null)
    setCompareResult(null)
    // A fresh analysis run means whatever "current version has not been

```

```

// analyzed" warning was showing (Module 20) is about to be stale
// either way - cleared here rather than left displayed against an
// analysis that no longer reflects the CR's real state.
setReportMismatch(null)
setDownloadError(null)
analyzeChangeRequest(id, token)
  .then(() => loadAll())
  .catch((err) => setRegenerateError(err.message || 'Regenerating the analysis failed. Please try again.))
  .finally(() => setRegenerating(false))
}

/** Module 15 (Repository Intelligence): "Re-scan Repository" - scans the
 * configured repository fresh, then re-runs the CR-file match against
 * this analysis, then reloads everything so the Repository tab (and any
 * related files line elsewhere on this page) reflects the new scan.
 * Two backend calls in sequence rather than one combined endpoint - the
 * backend deliberately keeps "scan the repo" and "match this CR against
 * the latest scan" as two separate, independently-triggerable actions
 * (Phase 1 and Phase 3), and this button is just the one place the
 * frontend chains them together for convenience. */
function handleRescanRepository() {
  setRescanning(true)
  setRescanError(null)
  triggerRepositoryScan(token)
    .then(() => triggerRepositoryMatch(id, token))
    .then(() => loadAll())
    .catch((err) => setRescanError(err.message || 'Re-scanning the repository failed. Please try again.))
    .finally(() => setRescanning(false))
}

/** Module 13 Phase 2/5: fetch the computed diff between analysis `id`
 * (the "to" side) and whichever analysis ran immediately before it - the
 * History tab's "View changes vs. previous" button. */
function handleToggleAnalysisCompare(analysisId) {
  if (compareToId === analysisId) {
    setCompareToId(null)
    setCompareResult(null)
    return
  }
  setCompareToId(analysisId)
  setCompareResult(null)
  setCompareLoading(true)
  compareAnalyses(id, { to: analysisId }, token)
    .then(setCompareResult)
    .catch((err) => setCompareResult({ error: err.message || 'Could not compare these analyses.' }))
    .finally(() => setCompareLoading(false))
}

/** Module 11 (version-aware since Module 20): "Download Report" - builds
 * the PDF entirely from already-persisted data (no re-running the AI).
 * With no `version`, this is "the current report" - which now comes
 * back as a structured 409 (spec section 3's "current CR version has
 * not been analyzed" case) instead of a misleading current-looking
 * report, whenever the CR has been edited since its last analysis;
 * passing an explicit `version` (from the mismatch banner's "view that
 * version instead" button, or from the version picker below) always
 * generates a clearly-labeled historical report for that version. */
function handleDownloadReport(version) {
  setDownloading(true)
  setDownloadError(null)
  setReportMismatch(null)
  downloadReport(id, token, version)
    .then(() => {
      // Refresh the version picker's has analysis flags - best-effort,
      // never treated as the download itself having failed.
      listReportableVersions(id, token)
        .then(setReportableVersions)
        .catch(() => {})
    })
    .catch((err) => {
      if (err.status === 409 && err.detail && typeof err.detail === 'object') {
        setReportMismatch(err.detail)
      } else {
        setDownloadError(err.message || 'Generating the report failed. Please try again.')
      }
    })
    .finally(() => setDownloading(false))
}

const crCode = `CR-${String(id).padStart(4, '0')}`

const breadcrumb = (current) => (
  <nav className="breadcrumbs" aria-label="Breadcrumb">
    <button type="button" className="breadcrumb-link" onClick={onBackToList}>
      Change Requests
    </button>
    <span className="breadcrumb-separator">/</span>
    <button type="button" className="breadcrumb-link" onClick={onBackToDetail}>
      {crCode}
    </button>
    <span className="breadcrumb-separator">/</span>
    <span className="breadcrumb-current">{current}</span>
  </nav>
)

if (loading) {
  return (
    <div className="ad-shell">
      <p className="cr-list-status-text">Loading analysis</p>
    </div>
  )
}

if (error) {
  return (
    <div className="ad-shell">
      {breadcrumb('AI Analysis')}
      <p className="cr-list-status-text cr-list-status-text--error">{error}</p>
    </div>
  )
}

const riskOutOf10 = analysis.risk.score / 10
const recMeta = analysis.recommendation ? RECOMMENDATION_META[analysis.recommendation] : null
const effort = parseEffortEstimate(analysis.effort.estimate)
const unresolvedQuestions = analysis.clarification_questions.filter((q) => !q.resolved)

return (
  <div className="ad-shell">
    {breadcrumb('AI Analysis')}

    <div className="ad-header ad-header--row">
      <div>
        <span className="ad-header-code">{crCode}</span>
        <h1>{changeRequest.title}</h1>
        { /* Module 14 Phase 1: makes version-awareness visible, not just
         structurally true - the CR's current version and the version
         this specific analysis ran against, side by side, plus the
         CR's real workflow status. is outdated below already covers
         *whether* these two numbers disagree; this is just always
         showing the numbers themselves. */ }

```

```

<div className="ad-header-meta">
  <button
    type="button"
    className="badge version-badge version-badge--clickable"
    onClick={goToHistoryTab}
    title="View every analysis run for this change request"
  >
    CR v{changeRequest.current_version || 1}
  </button>
  <button
    type="button"
    className="badge version-badge version-badge--clickable"
    onClick={goToHistoryTab}
    title="View every analysis run for this change request"
  >
    Analysis v{analysis.change_request_version ?? '-'}
  </button>
  <span className="badge badge--gray">{changeRequest.status_label || titleCase(changeRequest.status)}</span>
</div>
<div className="ad-header-actions">
  <div className="ad-header-buttons">
    <button
      type="button"
      className="cr-btn cr-btn--secondary"
      onClick={handleRegenerate}
      disabled={regenerating}
    >
      {regenerating ? 'Regenerating... this can take up to a minute' : 'Regenerate Analysis'}
    </button>
    <button
      type="button"
      className="cr-btn cr-btn--primary"
      onClick={() => handleDownloadReport()}
      disabled={downloading}
    >
      {downloading ? 'Generating report...' : 'Download Report'}
    </button>
  </div>
  {regenerateError && <p className="cr-form-error ad-header-error">{regenerateError}</p>}
  {downloadError && <p className="cr-form-error ad-header-error">{downloadError}</p>}

  {/* Module 20 spec section 3: never show a misleading "current"
  report - if the current version hasn't been analyzed, show
  the warning plus both ways forward (re-analyze, or fall back
  to the last version that WAS analyzed) instead of a PDF. */}
  {reportMismatch && (
    <div className="outdated-banner ad-report-mismatch">
      <span className="outdated-banner-text">
        <span className="outdated-banner-icon" aria-hidden="true">
          <img alt="Outdated banner icon" data-bbox="165 395 185 415"/>
        </span>
        <strong>Current CR version has not been analyzed.</strong> Version{' '}
        {reportMismatch.cr_version} of this change request has no analysis of its own yet, so a
        "current" report can't be generated without mixing data from two different versions.
      </span>
      <div className="ad-report-mismatch-actions">
        <button
          type="button"
          className="cr-btn cr-btn--primary cr-btn--small"
          onClick={handleRegenerate}
          disabled={regenerating}
        >
          {regenerating ? 'Regenerating' : 'Re-analyze'}
        </button>
        {reportMismatch.last_analyzed_version != null && (
          <button
            type="button"
            className="cr-btn cr-btn--secondary cr-btn--small"
            onClick={() => handleDownloadReport(reportMismatch.last_analyzed_version)}
            disabled={downloading}
          >
            View Version {reportMismatch.last_analyzed_version} Report Instead
          </button>
        )}
      </div>
    </div>
  )}

  {/* Module 20 spec section 7: an authorized user (anyone who can
  already view this change request) can pull a report for any
  specific past version on purpose, regardless of whether the
  current version is up to date. Only worth showing once
  there's more than one version to choose from. */}
  {reportableVersions.length > 1 && (
    <div className="ad-report-version-picker">
      <label htmlFor="report-version-select">Report for a specific version</label>
      <select
        id="report-version-select"
        value={selectedReportVersion}
        onChange={(event) => setSelectedReportVersion(event.target.value)}
      >
        <option value="">Current (Version {changeRequest.current_version || 1})</option>
        {reportableVersions.map((v) => (
          <option key={v.version_number} value={v.version_number}>
            Version {v.version_number}
            {v.is_current ? '(current)' : ''} {v.has_analysis ? 'analyzed' : 'not analyzed'}
          </option>
        ))}
      </select>
      <button
        type="button"
        className="cr-btn cr-btn--secondary cr-btn--small"
        onClick={() =>
          handleDownloadReport(selectedReportVersion ? Number(selectedReportVersion) : undefined)
        }
        disabled={downloading}
      >
        {downloading ? 'Generating...' : 'Generate Report'}
      </button>
    </div>
  )}

  {analysis.is outdated && (
    <div className="outdated-banner">
      <span className="outdated-banner-text">
        <span className="outdated-banner-icon" aria-hidden="true">
          <img alt="Outdated banner icon" data-bbox="145 875 165 895"/>
        </span>
        <strong>This analysis is outdated.</strong> The change request has been edited since this analysis
        ran - regenerate it to see results that reflect the current version.
      </span>
      <button
        type="button"
        className="cr-btn cr-btn--primary cr-btn--small"

```

```

        onClick={handleRegenerate}
        disabled={regenerating}
      >
        {regenerating ? 'Regenerating...' : 'Regenerate Analysis'}
      </button>
    </div>
  )}

<div className="ad-summary-grid">
  <div className="ad-summary-card">
    <span className="ad-summary-label">Risk Score</span>
    <div className="ad-summary-body">
      <Gauge pct={riskOutOf10 * 10} color={riskTone(riskOutOf10)}>
        <span className="ad-gauge-value">{riskOutOf10.toFixed(1)}</span>
      </Gauge>
      <span className="ad-summary-sub">out of 10</span>
    </div>
  </div>

  <div className="ad-summary-card">
    <span className="ad-summary-label">Complexity</span>
    <div className="ad-summary-body ad-summary-body--center">
      <div className="ad-summary-stack">
        <span className={badge ad-summary-badge ${LEVEL_CLASS[analysis.complexity]} || 'badge--gray'}}>
          {titleCase(analysis.complexity)}
        </span>
        {analysis.complexity.confidence && (
          <ConfidenceLevelBadge level={analysis.complexity.confidence} />
        )}
      </div>
    </div>
  </div>

  <div className="ad-summary-card">
    <span className="ad-summary-label">Estimated Effort</span>
    <div className="ad-summary-body ad-summary-body--center">
      <div className="ad-summary-stack">
        <span className="ad-summary-text">{effort?.total || 'Insufficient information.'}</span>
        {analysis.effort.confidence && <ConfidenceLevelBadge level={analysis.effort.confidence} />}
      </div>
    </div>
  </div>

  <div className="ad-summary-card">
    <span className="ad-summary-label">AI Confidence</span>
    <div className="ad-summary-body">
      <Gauge pct={analysis.confidence.score} color="#4338ca">
        <span className="ad-gauge-value">{Math.round(analysis.confidence.score)}%</span>
      </Gauge>
    </div>
  </div>

  <div className={ad-summary-card ad-recommendation-card ad-recommendation-card--${recMeta?.tone || 'neutral'}}>
    <span className="ad-summary-label">Recommendation</span>
    <div className="ad-summary-body ad-summary-body--center">
      {recMeta ? (
        <span className="ad-recommendation-value">
          <span aria-hidden="true">{recMeta.emoji}</span> {recMeta.label}
        </span>
      ) : (
        <span className="ad-summary-text">⚠️</span>
      )}
    </div>
  </div>

</div>

<div className="ad-tabs" role="tablist">
  {TABS.map((tab) => (
    <button
      key={tab.key}
      type="button"
      role="tab"
      aria-selected={activeTab === tab.key}
      className={ad-tabs{activeTab === tab.key ? 'ad-tab--active' : ''}}
      onClick={() => setActiveTab(tab.key)}
    >
      {tab.label}
      {tab.key === 'missing' && unresolvedQuestions.length > 0 && (
        <span className="ad-tab-count">{unresolvedQuestions.length}</span>
      )}
    </button>
  ))}
</div>

<div className="ad-tab-content" ref={tabContentRef}>
  {activeTab === 'overview' && (
    <OverviewTab analysis={analysis} recommendedApprovals={recommendedApprovals} />
  )}
  {activeTab === 'requirements' && (
    <RequirementsTab analysis={analysis} changeRequestId={id} token={token} onReviewed={loadAll} />
  )}
  {activeTab === 'impact' && (
    <ImpactTab analysis={analysis} changeRequest={changeRequest} crCode={crCode} />
  )}
  {activeTab === 'repository' && (
    <RepositoryTab
      findings={repositoryFindings}
      scan={repositoryScan}
      onRescan={handleRescanRepository}
      rescanning={rescanning}
      rescanError={rescanError}
    />
  )}
  {activeTab === 'dependencies' && <DependenciesTab analysis={analysis} />}
  {activeTab === 'risks' && <RisksTab analysis={analysis} />}
  {activeTab === 'security' && (
    <SecurityTab analysis={analysis} changeRequestId={id} token={token} onReviewed={loadAll} />
  )}
  {activeTab === 'effort' && <EffortTab analysis={analysis} effort={effort} />}
  {activeTab === 'missing' && (
    <MissingInfoTab analysis={analysis} changeRequestId={id} token={token} onAnswered={loadAll} />
  )}
  {activeTab === 'tests' && (
    <TestCasesTab
      analysis={analysis}
      changeRequestId={id}
      token={token}
      onRegenerate={handleRegenerate}
      regenerating={regenerating}
      onSaved={loadAll}
    />
  )}
  {activeTab === 'plan' && (
    <ImplementationPlanTab
      analysis={analysis}
      changeRequestId={id}
      token={token}
      onRegenerate={handleRegenerate}
      regenerating={regenerating}
      onSaved={loadAll}
    />
  )}

```

```
    })
    {activeTab === 'traceability' && <TraceabilityTab analysis={analysis} />}
    {activeTab === 'history' && (
      <AnalysisHistoryTab
        history={analysisHistory}
        currentAnalysisId={analysis.id}
        compareToId={compareToId}
        compareResult={compareResult}
        compareLoading={compareLoading}
        onToggleCompare={handleToggleAnalysisCompare}
      />
    )
  }
</div>
}
```

```

import { useEffect, useMemo, useState } from 'react'
import {
  getAIAnalytics,
  getApprovalBottlenecks,
  getChangeAnalytics,
  getExecutiveAndWorkflowMetrics,
  getRiskAnalytics,
  getWorkloadAnalytics,
} from '../api/analytics'
import { listUsers } from '../api/users'
import DistributionChart from '../components/DistributionChart'
import MetricCard from '../components/MetricCard'
import RiskTrendChart from '../components/RiskTrendChart'
import { useAuth } from '../context/AuthContext'
import '../styles/dashboard.css'
import '../styles/analytics.css'

/** Module 19 (Engineering Change Analytics), Phase 6: the frontend for all
 * 5 backend endpoints Phases 2-5 built (executive+workflow, risk, approval
 * bottlenecks, change analytics, workload, AI). Every section here reads
 * ONLY what those endpoints return - no client-side math invents a number
 * the backend didn't already compute, per the spec's own "every metric
 * should be traceable to real database records" rule.
 *
 * One shared filter bar (spec section 8) drives all 6 API calls together -
 * changing a filter re-fetches every section at once, so every card on the
 * page always describes the exact same slice of the data. No charting
 * library is used anywhere on this page (see DistributionChart.jsx's own
 * docstring and this page's new RiskTrendChart) - proportional CSS bars
 * only, matching this project's existing dashboard.
 */

const STATUS_OPTIONS = [
  { value: '', label: 'Any status' },
  { value: 'draft', label: 'Draft' },
  { value: 'submitted', label: 'Submitted' },
  { value: 'pending_analysis', label: 'Pending Analysis' },
  { value: 'analyzed', label: 'Analyzed' },
  { value: 'in_review', label: 'Under Review' },
  { value: 'changes_requested', label: 'Changes Requested' },
  { value: 'approval_required', label: 'Approval Required' },
  { value: 'approved', label: 'Approved' },
  { value: 'rejected', label: 'Rejected' },
  { value: 'implementation_planned', label: 'Implementation Planned' },
  { value: 'in_progress', label: 'In Progress' },
  { value: 'implemented', label: 'Implemented' },
  { value: 'validated', label: 'Validated' },
  { value: 'closed', label: 'Closed' },
  { value: 'cancelled', label: 'Cancelled' },
]

const PRIORITY_OPTIONS = [
  { value: '', label: 'Any priority' },
  { value: 'low', label: 'Low' },
  { value: 'medium', label: 'Medium' },
  { value: 'high', label: 'High' },
  { value: 'critical', label: 'Critical' },
]

const RISK_OPTIONS = [
  { value: '', label: 'Any risk' },
  { value: 'low', label: 'Low' },
  { value: 'medium', label: 'Medium' },
  { value: 'high', label: 'High' },
  { value: 'critical', label: 'Critical' },
]

const CATEGORY_OPTIONS = [
  { value: '', label: 'Any category' },
  { value: 'Feature', label: 'Feature' },
  { value: 'Bug Fix', label: 'Bug Fix' },
  { value: 'Security', label: 'Security' },
  { value: 'Database', label: 'Database' },
  { value: 'API', label: 'API' },
  { value: 'Infrastructure', label: 'Infrastructure' },
  { value: 'Integration', label: 'Integration' },
  { value: 'Other', label: 'Other' },
]

const RISK_ROW_ORDER = [
  { key: 'low', label: 'Low', color: '#12b76a' },
  { key: 'medium', label: 'Medium', color: '#f79009' },
  { key: 'high', label: 'High', color: '#f04438' },
  { key: 'critical', label: 'Critical', color: '#b42318' },
  { key: 'not_analyzed', label: 'Not analyzed', color: '#98a2b3' },
]

const DEFAULT_FILTERS = {
  dateFrom: '',
  dateTo: '',
  status: '',
  priority: '',
  risk: '',
  category: '',
  ownerId: '',
}

function formatHours(value) {
  if (value === null || value === undefined) return 'No data yet'
  if (value < 1) return `${Math.round(value * 60)}m`
  return `${value.toFixed(1)}h`
}

function formatPercent(value) {
  if (value === null || value === undefined) return '—'
  return `${Math.round(value * 100)}%`
}

function formatConfidence(value) {
  if (value === null || value === undefined) return '—'
  return `${Math.round(value)}%`
}

function formatCount(value) {
  return value === null || value === undefined ? '—' : value
}

export default function AnalyticsPage() {
  const { token } = useAuth()
  const [filters, setFilters] = useState(DEFAULT_FILTERS)
  const [users, setUsers] = useState([])
  const [executive, setExecutive] = useState(null)
  const [risk, setRisk] = useState(null)
  const [bottlenecks, setBottlenecks] = useState(null)

```

```

const [changeAnalytics, setChangeAnalytics] = useState(null)
const [workload, setWorkload] = useState(null)
const [aiAnalytics, setAiAnalytics] = useState(null)
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)

useEffect(() => {
  listUsers(token)
    .then((rows) => setUsers(rows || []))
    .catch(() => setUsers([]))
}, [token])

const queryFilters = useMemo(
  () => ({
    // date to is inclusive of the whole day the user picked - anchoring
    // it to end-of-day (not midnight) avoids silently excluding change
    // requests created later that same day.
    dateFrom: filters.dateFrom ? new Date(`${filters.dateFrom}T00:00:00`).toISOString() : '',
    dateTo: filters.dateTo ? new Date(`${filters.dateTo}T23:59:59.999`).toISOString() : '',
    status: filters.status,
    priority: filters.priority,
    risk: filters.risk,
    category: filters.category,
    ownerId: filters.ownerId,
  }),
  [filters]
)

useEffect(() => {
  let cancelled = false
  setLoading(true)
  setError(null)

  Promise.all([
    getExecutiveAndWorkflowMetrics(token, queryFilters),
    getRiskAnalytics(token, queryFilters),
    getApprovalBottlenecks(token, queryFilters),
    getChangeAnalytics(token, queryFilters),
    getWorkloadAnalytics(token, queryFilters),
    getAiAnalytics(token, queryFilters),
  ])
    .then([execData, riskData, bottleneckData, changeData, workloadData, aiData]) => {
      if (cancelled) return
      setExecutive(execData)
      setRisk(riskData)
      setBottlenecks(bottleneckData)
      setChangeAnalytics(changeData)
      setWorkload(workloadData)
      setAiAnalytics(aiData)
    }
    .catch((err) => {
      if (!cancelled) setError(err.message || 'Could not load analytics.')
    })
    .finally(() => {
      if (!cancelled) setLoading(false)
    })

    return () => {
      cancelled = true
    }
  }, [token, queryFilters])

  function updateFilter(key, value) {
    setFilters((prev) => ({ ...prev, [key]: value }))
  }

  function clearFilters() {
    setFilters(DEFAULT_FILTERS)
  }

  const hasActiveFilters = Object.values(filters).some((value) => value !== '')

  const riskData = risk
    ? RISK_ROW_ORDER.map((row) => ({
        label: row.label,
        count: risk.current_distribution[row.key] || 0,
        color: row.color,
      }))
    : []

  return (
    <div className="dashboard-shell analytics-shell">
      <div className="dashboard-heading">
        <h1>Engineering Change Analytics</h1>
        <p>Real-time metrics computed directly from the database - nothing here is estimated or invented.</p>
      </div>

      <div className="analytics-filter-bar">
        <div className="analytics-filter">
          <label htmlFor="analytics-date-from">From</label>
          <input
            id="analytics-date-from"
            type="date"
            value={filters.dateFrom}
            onChange={(e) => updateFilter('dateFrom', e.target.value)}
          />
        </div>
        <div className="analytics-filter">
          <label htmlFor="analytics-date-to">To</label>
          <input
            id="analytics-date-to"
            type="date"
            value={filters.dateTo}
            onChange={(e) => updateFilter('dateTo', e.target.value)}
          />
        </div>
        <div className="analytics-filter">
          <label htmlFor="analytics-status">Status</label>
          <select id="analytics-status" value={filters.status} onChange={(e) => updateFilter('status', e.target.value)}>
            {STATUS_OPTIONS.map((opt) => (
              <option key={opt.value} value={opt.value}>
                {opt.label}
              </option>
            ))}
          </select>
        </div>
        <div className="analytics-filter">
          <label htmlFor="analytics-priority">Priority</label>
          <select
            id="analytics-priority"
            value={filters.priority}
            onChange={(e) => updateFilter('priority', e.target.value)}
          >
            {PRIORITY_OPTIONS.map((opt) => (
              <option key={opt.value} value={opt.value}>
                {opt.label}
              </option>
            ))}
          </select>
        </div>
      </div>
    </div>
  )

```



```

<label htmlFor="analytics-risk">Risk</label>
<select id="analytics-risk" value={filters.risk} onChange={(e) => updateFilter('risk', e.target.value)}>
  {RISK_OPTIONS.map((opt) => (
    <option key={opt.value} value={opt.value}>
      {opt.label}
    </option>
  ))}
</select>
</div>
<div className="analytics-filter">
  <label htmlFor="analytics-category">Category</label>
  <select
    id="analytics-category"
    value={filters.category}
    onChange={(e) => updateFilter('category', e.target.value)}
  >
    {CATEGORY_OPTIONS.map((opt) => (
      <option key={opt.value} value={opt.value}>
        {opt.label}
      </option>
    ))}
  </select>
</div>
<div className="analytics-filter">
  <label htmlFor="analytics-owner">Owner</label>
  <select id="analytics-owner" value={filters.ownerId} onChange={(e) => updateFilter('ownerId', e.target.value)}>
    <option value="">Any owner</option>
    {users.map((u) => (
      <option key={u.id} value={u.id}>
        {u.name}
      </option>
    ))}
  </select>
</div>
{hasActiveFilters && (
  <button type="button" className="analytics-clear-filters" onClick={clearFilters}>
    Clear filters
  </button>
)}
</div>

{error && <p className="dashboard-status-text dashboard-status-text--error">{error}</p>
!loading && !executive && <p className="dashboard-status-text">Loading analytics</p>

{executive && (
  <
    <h2 className="analytics-section-title">Executive Metrics</h2>
    <div className="metric-grid analytics-metric-grid-7">
      <MetricCard label="Total CRs" value={executive.executive.total} />
      <MetricCard label="Open" value={executive.executive.open} tone="neutral" />
      <MetricCard label="Pending Approval" value={executive.executive.pending_approval} tone="warning" />
      <MetricCard label="Approved" value={executive.executive.approved} tone="success" />
      <MetricCard label="Rejected" value={executive.executive.rejected} tone="danger" />
      <MetricCard label="In Progress" value={executive.executive.in_progress} tone="neutral" />
      <MetricCard label="Closed" value={executive.executive.closed} tone="success" />
    </div>

    <h2 className="analytics-section-title">Workflow Metrics</h2>
    <div className="metric-grid analytics-metric-grid-6">
      <MetricCard label="Avg. Time to Analysis" value={formatHours(executive.workflow.avg_time_to_analysis_hours)} />
      <MetricCard label="Avg. Approval Time" value={formatHours(executive.workflow.avg_approval_time_hours)} />
      <MetricCard
        label="Avg. Time to Implementation"
        value={formatHours(executive.workflow.avg_time_to_implementation_hours)}
      />
      <MetricCard label="Avg. Time to Closure" value={formatHours(executive.workflow.avg_time_to_closure_hours)} />
      <MetricCard label="Waiting for Approval" value={executive.workflow.crs_waiting_for_approval} tone="warning" />
      <MetricCard label="Changes Requested" value={executive.workflow.crs_with_requested_changes} tone="warning" />
    </div>
  </>
)}

{risk && (
  <
    <h2 className="analytics-section-title">Risk Analytics</h2>
    <div className="chart-grid">
      <DistributionChart title="Current Risk Distribution" data={riskData} emptyLabel="No change requests yet." />
      <RiskTrendChart data={risk.over_time} emptyLabel="No analyzed change requests yet." />
    </div>
  </>
)}

{bottlenecks && (
  <
    <h2 className="analytics-section-title">Approval Bottlenecks</h2>
    <div className="chart-grid">
      <div className="chart-card">
        <h2 className="chart-card-title">Pending Approvals by Type</h2>
        {Object.keys(bottlenecks.pending_by_type).length === 0 ? (
          <p className="chart-empty">No pending approvals.</p>
        ) : (
          <ul className="analytics-simple-list">
            {Object.entries(bottlenecks.pending_by_type).map(([label, count]) => (
              <li key={label}>
                <span>{label}</span>
                <span>{count}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
      <div className="chart-card">
        <h2 className="chart-card-title">Pending Approvals by Person</h2>
        {bottlenecks.pending_by_person.length === 0 ? (
          <p className="chart-empty">No pending approvals.</p>
        ) : (
          <ul className="analytics-simple-list">
            {bottlenecks.pending_by_person.map((entry) => (
              <li key={entry.approver_name}>
                <span>{entry.approver_name}</span>
                <span>{entry.count}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
      <div className="chart-card">
        <h2 className="chart-card-title">Avg. Approval Duration by Type</h2>
        {Object.keys(bottlenecks.avg_duration_hours_by_type).length === 0 ? (
          <p className="chart-empty">No resolved approvals yet.</p>
        ) : (
          <ul className="analytics-simple-list">
            {Object.entries(bottlenecks.avg_duration_hours_by_type).map(([label, hours]) => (
              <li key={label}>
                <span>{label}</span>
                <span>{formatHours(hours)}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  </>
)}

```

```

    </div>
    <div className="chart-card">
      <h2 className="chart-card-title">Most Common Approval Blockers</h2>
      {bottlenecks.mostCommonBlockers.length === 0 ? (
        <p className="chart-empty">No rejections or changes-requested responses yet.</p>
      ) : (
        <ul className="analytics-simple-list">
          {bottlenecks.mostCommonBlockers.map((entry) => (
            <li key={entry.approvalTypeLabel}>
              <span>{entry.approvalTypeLabel}</span>
              <span>{entry.count}</span>
            </li>
          ))}
        </ul>
      )}
    </div>
  </div>
</div>
)}

{changeAnalytics && (
  <div>
    <h2 className="analytics-section-title">Change Analytics</h2>
    <div className="metric-grid analytics-metric-grid-3">
      <MetricCard label="CRs with Multiple Revisions" value={changeAnalytics.crsWithMultipleRevisions} />
      <MetricCard
        label="Avg. Versions per CR"
        value={changeAnalytics.avgVersionsPerCr === null ? '-' : changeAnalytics.avgVersionsPerCr.toFixed(1)}
      />
      <MetricCard
        label="Returned for Clarification"
        value={changeAnalytics.crsReturnedForClarification}
        tone="warning"
      />
    </div>
    <div className="chart-grid">
      <div className="chart-card">
        <h2 className="chart-card-title">Most Frequently Changed Fields</h2>
        {changeAnalytics.mostChangedFields.length === 0 ? (
          <p className="chart-empty">No field edits recorded yet.</p>
        ) : (
          <ul className="analytics-simple-list">
            {changeAnalytics.mostChangedFields.map((entry) => (
              <li key={entry.fieldLabel}>
                <span>{entry.fieldLabel}</span>
                <span>{entry.count}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  </div>
)}

{workload && (
  <div>
    <h2 className="analytics-section-title">Workload</h2>
    <div className="metric-grid analytics-metric-grid-3">
      <MetricCard label="Pending Reviews" value={workload.pendingReviews} />
      <MetricCard label="Pending Approvals" value={workload.pendingApprovals} tone="warning" />
      <MetricCard label="Overdue Tasks" value={workload.overdueTasks} tone="danger" />
    </div>
    <div className="chart-grid">
      <div className="chart-card">
        <h2 className="chart-card-title">Change Requests per Owner</h2>
        <p className="analytics-note">Excludes this app's own bulk load-test data.</p>
        {workload.crsPerOwner.length === 0 ? (
          <p className="chart-empty">No change requests match these filters.</p>
        ) : (
          <ul className="analytics-simple-list">
            {workload.crsPerOwner.map((entry) => (
              <li key={entry.ownerName}>
                <span>{entry.ownerName}</span>
                <span>{entry.count}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  </div>
)}

{aiAnalytics && (
  <div>
    <h2 className="analytics-section-title">AI Recommendation Statistics</h2>
    <p className="analytics-note">
      These describe what the AI recommended and did, not whether it was right - this app doesn't record
      ground-truth outcomes, so no "accuracy" figure is shown here.
    </p>
    <div className="metric-grid analytics-metric-grid-6">
      <MetricCard label="Number of Analyses" value={formatCount(aiAnalytics.totalAnalyses)} />
      <MetricCard label="Re-analysis Rate" value={formatPercent(aiAnalytics.reAnalysisRate)} />
      <MetricCard label="Avg. Confidence" value={formatConfidence(aiAnalytics.averageConfidence)} />
      <MetricCard label="Risk Changes After Edits" value={formatCount(aiAnalytics.riskChangesAfterEdits)} />
      <MetricCard label="AI-Recommended Approvals" value={formatCount(aiAnalytics.aiRecommendedApprovals)} />
      <MetricCard label="AI Analysis Failures" value={formatCount(aiAnalytics.aiAnalysisFailures)} tone="danger" />
    </div>
  </div>
)}
)

```

frontend/src/pages/AuthenticatedHome.jsx

87 lines

```
import { useState } from 'react'
import AppHeader, { NAV_ITEMS } from '../components/AppHeader'
import { useAuth } from '../context/AuthContext'
import '../styles/home.css'
import AdminPage from './AdminPage'
import AnalyticsPage from './AnalyticsPage'
import ChangeRequestsPage from './ChangeRequestsPage'
import DashboardPage from './DashboardPage'
import KnowledgeBasePage from './KnowledgeBasePage'
import MyWorkPage from './MyWorkPage'
import NewChangeRequestPage from './NewChangeRequestPage'
import NotificationsPage from './NotificationsPage'
import ReportsPage from './ReportsPage'
import ComingSoonPage from './ComingSoonPage'

/** Shell for everything behind login: the header/nav lives here once, and
 * `activeView` (plain local state - no router in this project) decides
 * which page renders below it. Dashboard (Module 3), New Change Request
 * (Module 4), Change Requests (Module 5), Notifications (Module 12
 * Phase 5), My Work (Module 18 Phase 4), and Analytics (Module 19
 * Phase 6) are the real pages built so far; every other nav item still
 * shows a "coming later" placeholder.
 */
* `jumpToCrId` lets Notifications/My Work/the header's own notification
* bell (Module 18 Phase 6) hand off "open change request #N" to Change
* Requests without a router - clicking a notification or a My Work row
* sets it and switches the view; ChangeRequestsPage consumes it via its
* `jumpToId` prop and reports back through `onJumpHandled` so it's only
* used once. */
export default function AuthenticatedHome() {
  const { user, logout } = useAuth()
  const [activeView, setActiveView] = useState('dashboard')
  const [jumpToCrId, setJumpToCrId] = useState(null)

  const activeLabel = NAV_ITEMS.find((item) => item.key === activeView)?.label || activeView

  function openChangeRequest(id) {
    setJumpToCrId(id)
    setActiveView('change-requests')
  }

  let page
  if (activeView === 'dashboard') {
    page = <DashboardPage onNavigate={setActiveView} onOpenChangeRequest={openChangeRequest} />
  } else if (activeView === 'new-analysis') {
    page = <NewChangeRequestPage onNavigate={setActiveView} />
  } else if (activeView === 'change-requests') {
    page = (
      <ChangeRequestsPage
        onNavigate={setActiveView}
        jumpToId={jumpToCrId}
        onJumpHandled={() => setJumpToCrId(null)}
      />
    )
  } else if (activeView === 'my-work') {
    page = <MyWorkPage onOpenChangeRequest={openChangeRequest} />
  } else if (activeView === 'analytics') {
    page = <AnalyticsPage />
  } else if (activeView === 'notifications') {
    page = <NotificationsPage onOpenChangeRequest={openChangeRequest} />
  } else if (activeView === 'reports') {
    page = <ReportsPage onOpenChangeRequest={openChangeRequest} onNavigate={setActiveView} />
  } else if (activeView === 'knowledge-base') {
    page = <KnowledgeBasePage />
  } else if (activeView === 'settings') {
    // Module 21: AdminPage does its own admin-only gate (and AppHeader
    // already hides this nav item for non-admins), so nothing extra is
    // needed here.
    page = <AdminPage />
  } else {
    page = <ComingSoonPage label={activeLabel} />
  }

  return (
    <div className="app-shell">
      <AppHeader
        activeView={activeView}
        onNavigate={setActiveView}
        onOpenChangeRequest={openChangeRequest}
        user={user}
        onLogout={logout}
      />
      <main className="app-main">{page}</main>
    </div>
  )
}
```

```

import { useEffect, useState } from 'react'
import { listChangeRequests } from '../api/changeRequests'
import ChangeRequestDetail from '../components/ChangeRequestDetail'
import AnalysisDashboardPage from '../AnalysisDashboardPage'
import EditChangeRequestPage from '../EditChangeRequestPage'
import { useAuth } from '../context/AuthContext'
// Reuses the .badge / .table-card / .requests-table classes (dashboard.css)
// and the .cr-btn / breadcrumb classes (change-request-form.css, Module 4)
// rather than redefining them.
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'

const PAGE_SIZE = 25

const STATUS_OPTIONS = [
  { value: '', label: 'All statuses' },
  { value: 'draft', label: 'Draft' },
  { value: 'pending_analysis', label: 'Pending Analysis' },
  { value: 'analyzing', label: 'Analyzing' },
  { value: 'requires_clarification', label: 'Requires Clarification' },
  { value: 'completed', label: 'Completed' },
  { value: 'approved', label: 'Approved' },
]

const PRIORITY_OPTIONS = [
  { value: '', label: 'All priorities' },
  { value: 'low', label: 'Low' },
  { value: 'medium', label: 'Medium' },
  { value: 'high', label: 'High' },
  { value: 'critical', label: 'Critical' },
]

const STATUS_LABELS = {
  draft: 'Draft',
  pending_analysis: 'Pending Analysis',
  analyzing: 'Analyzing',
  requires_clarification: 'Requires Clarification',
  completed: 'Completed',
  approved: 'Approved',
}

const STATUS_CLASS = {
  draft: 'badge--gray',
  pending_analysis: 'badge--indigo',
  analyzing: 'badge--blue',
  requires_clarification: 'badge--amber',
  completed: 'badge--teal',
  approved: 'badge--green',
}

const PRIORITY_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const PRIORITY_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
}

const RISK_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const RISK_CLASS = {
  low: 'badge--green',
  medium: 'badge--amber',
  high: 'badge--orange',
  critical: 'badge--red',
}

const COMPLEXITY_LABELS = { low: 'Low', medium: 'Medium', high: 'High', very_high: 'Very High' }

function formatDate(isoString) {
  try {
    return new Date(isoString).toLocaleDateString(undefined, { year: 'numeric', month: 'short', day: 'numeric' })
  } catch {
    return isoString
  }
}

const emptyFilters = { search: '', status: '', priority: '', assignedToMe: false, sort: 'newest' }

/** Module 5: the full Change Requests list - search, filter, sort,
 * pagination, and a master-detail panel (click a row to open it). Reads
 * only - no AI logic lives here. */
export default function ChangeRequestsPage({ onNavigate, jumpToId, onJumpHandled }) {
  const { token } = useAuth()
  const [filters, setFilters] = useState(emptyFilters)
  const [searchInput, setSearchInput] = useState('')
  const [offset, setOffset] = useState(0)
  const [data, setData] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [selectedId, setSelectedId] = useState(null)
  const [dashboardId, setDashboardId] = useState(null)
  const [editId, setEditId] = useState(null)
  // Bumped every time we come back to this list from a change request's
  // detail/edit/dashboard view, so the list re-fetches instead of showing
  // whatever it last loaded (e.g. a status that changed while we were away
  // analyzing or editing that change request).
  const [refreshToken, setRefreshToken] = useState(0)

  // Debounce the free-text search box so we're not firing a request on
  // every keystroke - everything else applies immediately.
  useEffect(() => {
    const handle = setTimeout(() => {
      setFilters((prev) => ({ ...prev, search: searchInput.trim() }))
      setOffset(0)
    }, 350)
    return () => clearTimeout(handle)
  }, [searchInput])

  useEffect(() => {
    let cancelled = false
    setLoading(true)
    setError(null)

    listChangeRequests(token, { ...filters, limit: PAGE_SIZE, offset })
      .then((response) => {
        if (!cancelled) setData(response)
      })
      .catch((err) => {
        if (!cancelled) setError(err.message || 'Could not load change requests.')
      })
      .finally(() => {
        if (!cancelled) setLoading(false)
      })
  })
}

```

```

    return () => {
      cancelled = true
    }, [token, filters, offset, refreshToken])

// A notification (or anything else outside this page) can ask to jump
// straight to one change request's detail view - consume it once and
// report back so the caller doesn't keep re-triggering it.
useEffect(() => {
  if (jumpToId !== null) {
    setDashboardId(null)
    setEditId(null)
    setSelectedId(jumpToId)
    onJumpHandled?.()
  }
}, [jumpToId])

function updateFilter(key, value) {
  setFilters((prev) => ({ ...prev, [key]: value }))
  setOffset(0)
}

function clearFilters() {
  setSearchInput('')
  setFilters(emptyFilters)
  setOffset(0)
}

const filtersActive = Boolean(filters.search || filters.status || filters.priority || filters.assignedToMe)

if (dashboardId !== null) {
  return (
    <AnalysisDashboardPage
      id={dashboardId}
      onBackToDetail={() => setDashboardId(null)}
      onBackToList={() => {
        setDashboardId(null)
        setSelectedId(null)
        setRefreshToken((k) => k + 1)
      }}
    />
  )
}

if (editId !== null) {
  return (
    <EditChangeRequestPage
      id={editId}
      onCancel={() => setEditId(null)}
      onSave={() => {
        setEditId(null)
        setRefreshToken((k) => k + 1)
      }}
    />
  )
}

if (selectedId !== null) {
  return (
    <ChangeRequestDetail
      id={selectedId}
      onBack={() => {
        setSelectedId(null)
        setRefreshToken((k) => k + 1)
      }}
      onOpenDashboard={(id) => setDashboardId(id)}
      onEdit={(id) => setEditId(id)}
    />
  )
}

const total = data?.total ?? 0
const items = data?.items ?? []
const rangeStart = total === 0 ? 0 : offset + 1
const rangeEnd = Math.min(offset + PAGE_SIZE, total)

return (
  <div className="cr-list-shell">
    <div className="cr-list-heading">
      <div>
        <h1>Change Requests</h1>
        <p>Every change request tracked in the system.</p>
      </div>
      <button type="button" className="cr-btn cr-btn--primary" onClick={() => onNavigate('new-analysis')}>
        + New Change Request
      </button>
    </div>

    <div className="cr-toolbar">
      <input
        type="text"
        className="cr-search-input"
        placeholder="Search by ticket number, title, description, requester, or system..."
        value={searchInput}
        onChange={(e) => setSearchInput(e.target.value)}
      />

      <select
        className="cr-filter-select"
        value={filters.status}
        onChange={(e) => updateFilter('status', e.target.value)}
        aria-label="Filter by status"
      >
        {STATUS_OPTIONS.map((option) => (
          <option key={option.value || 'all'} value={option.value}>
            {option.label}
          </option>
        ))}
      </select>

      <select
        className="cr-filter-select"
        value={filters.priority}
        onChange={(e) => updateFilter('priority', e.target.value)}
        aria-label="Filter by priority"
      >
        {PRIORITY_OPTIONS.map((option) => (
          <option key={option.value || 'all'} value={option.value}>
            {option.label}
          </option>
        ))}
      </select>

      <select
        className="cr-filter-select"
        value={filters.sort}
        onChange={(e) => updateFilter('sort', e.target.value)}
        aria-label="Sort by date"
      >
        <option value="newest">Newest first</option>
        <option value="oldest">Oldest first</option>
      </select>
    </div>
  </div>
)

```

```

</select>
<label className="cr-toolbar-checkbox">
  <input
    type="checkbox"
    checked={filters.assignedToMe}
    onChange={(e) => updateFilter('assignedToMe', e.target.checked)}
  />
  Assigned to me
</label>
</div>
{loading && <p className="cr-list-status-text">Loading change requests</p>}
{error && <p className="cr-list-status-text cr-list-status-text--error">{error}</p>}}

{!loading && !error && total === 0 && (
  <div className="cr-list-empty">
    {filtersActive ? (
      <p className="cr-list-empty-title">No change requests match your filters.</p>
      <button type="button" className="cr-btn cr-btn--secondary" onClick={clearFilters}>
        Clear filters
      </button>
    ) : (
      <p className="cr-list-empty-title">No change requests yet.</p>
      <button
        type="button"
        className="cr-btn cr-btn--primary"
        onClick={() => onNavigate('new-analysis')}
      >
        + New Change Request
      </button>
    )}
  </div>
)}

{!loading && !error && total > 0 && (
  <div className="table-card cr-table-card">
    <div className="table-scroll">
      <table className="requests-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Title</th>
            <th>Category</th>
            <th>Priority</th>
            <th>Status</th>
            <th>Risk</th>
            <th>Complexity</th>
            <th>Created</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {items.map(item => (
            <tr key={item.id} className="cr-list-row" onClick={() => setSelectedId(item.id)}>
              <td className="requests-table-id">#{item.id}</td>
              <td className="requests-table-title">{item.title}</td>
              <td>{item.category} || <span className="table-muted">Not analyzed</span></td>
              <td>
                <span className={`badge ${PRIORITY_CLASS[item.priority]} || 'badge--gray'}>
                  {PRIORITY_LABELS[item.priority]} || item.priority
                </span>
              </td>
              <td>
                <span className={`badge ${STATUS_CLASS[item.effective_status]} || 'badge--gray'}>
                  {STATUS_LABELS[item.effective_status]} || item.effective_status
                </span>
              </td>
              <td>
                {item.risk ? (
                  <span className={`badge ${RISK_CLASS[item.risk]} || 'badge--gray'}>
                    {RISK_LABELS[item.risk]} || item.risk
                  </span>
                ) : (
                  <span className="table-muted"></span>
                )}
              </td>
              <td className="table-muted">
                {item.complexity ? COMPLEXITY_LABELS[item.complexity] || item.complexity : '-'}
              </td>
              <td className="table-muted">{formatDate(item.created_at)}</td>
              <td>
                <button type="button" className="cr-open-link">
                  Open
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
)

<div className="cr-pagination">
  <span className="cr-pagination-summary">
    Showing {rangeStart} {rangeEnd} of {total}
  </span>
  <div className="cr-pagination-buttons">
    <button
      type="button"
      className="cr-btn cr-btn--secondary"
      disabled={offset === 0}
      onClick={() => setOffset(Math.max(0, offset - PAGE_SIZE))}
    >
      Previous
    </button>
    <button
      type="button"
      className="cr-btn cr-btn--secondary"
      disabled={rangeEnd >= total}
      onClick={() => setOffset(offset + PAGE_SIZE)}
    >
      Next
    </button>
  </div>
</div>
)
}

```

frontend/src/pages/ComingSoonPage.jsx

17 lines

```
import '../styles/dashboard.css'

/** Placeholder for nav items outside this module's scope (Change Requests,
 * New Analysis, Knowledge Base, Repository, Reports, Settings). Keeps the
 * full navigation feeling real instead of dead links, without building any
 * of those pages out yet - they're later modules. */
export default function ComingSoonPage({ label }) {
  return (
    <div className="dashboard-shell">
      <div className="dashboard-empty">
        <h2 className="dashboard-empty-title">{label}</h2>
        <p className="dashboard-empty-subtitle">This part of the app is built in a later module.</p>
      </div>
    </div>
  )
}
```

```

import { useEffect, useState } from 'react'
import { getDashboardDrillDown, getDashboardSummary } from '../api/dashboard'
import DistributionChart from '../components/DistributionChart'
import EmptyState from '../components/EmptyState'
import MetricCard from '../components/MetricCard'
import Modal from '../components/Modal'
import RecentRequestsTable from '../components/RecentRequestsTable'
import { useAuth } from '../context/AuthContext'
import '../styles/dashboard.css'
import '../styles/change-requests-list.css'
import '../styles/change-request-form.css'

const RISK_ORDER = [
  { key: 'Low', label: 'Low', color: '#12b76a' },
  { key: 'medium', label: 'Medium', color: '#f79609' },
  { key: 'high', label: 'High', color: '#04438' },
  { key: 'critical', label: 'Critical', color: '#b42318' },
  { key: 'not_analyzed', label: 'Not analyzed', color: '#98a2b3' },
]

const CATEGORY_ORDER = ['Feature', 'Bug Fix', 'Security', 'Database', 'API', 'Infrastructure', 'Integration', 'Other']

// The 5 clickable tiles: each maps to a `metric` value the backend's
// /dashboard/drill-down endpoint understands (see backend/app/api/
// dashboard.py) and the heading its table should show once clicked.
const DRILLDOWN_TITLES = {
  total: 'All Change Requests',
  pending_analysis: 'Pending Analysis',
  high_risk: 'High Risk Changes',
  approved: 'Approved Changes',
  requires_clarification: 'Requires Clarification',
}

// The popup's rows-per-page dropdown - capped at 10 to match the backend's
// own page size ceiling (spec: "page dropdown till 10").
const PAGE_SIZE_OPTIONS = [5, 10]

export default function DashboardPage({ onNavigate, onOpenChangeRequest }) {
  const { token } = useAuth()
  const [summary, setSummary] = useState(null)
  const [error, setError] = useState(null)
  const [loading, setLoading] = useState(true)

  // Module: clickable dashboard tiles. Selecting a tile a second time
  // closes its table again - `selectedMetric` is the single source of
  // truth for both "which tile is highlighted" and "which table shows."
  const [selectedMetric, setSelectedMetric] = useState(null)
  const [drillDown, setDrillDown] = useState(null)
  const [drillDownLoading, setDrillDownLoading] = useState(false)
  const [drillDownError, setDrillDownError] = useState(null)

  // The popup's own filter/pagination controls (spec: "give option of the
  // filter to select the date range, page dropdown till 10"). All reset
  // together whenever the popup closes, so reopening a different tile never
  // carries over a stale date range or page number.
  const [dateFrom, setDateFrom] = useState('')
  const [dateTo, setDateTo] = useState('')
  const [page, setPage] = useState(1)
  const [pageSize, setPageSize] = useState(10)

  useEffect(() => {
    let cancelled = false
    setLoading(true)
    setError(null)

    getDashboardSummary(token)
      .then((data) => {
        if (!cancelled) setSummary(data)
      })
      .catch((err) => {
        if (!cancelled) setError(err.message || 'Could not load the dashboard.')
      })
      .finally(() => {
        if (!cancelled) setLoading(false)
      })

    return () => {
      cancelled = true
    }
  }, [token])

  useEffect(() => {
    if (!selectedMetric) {
      setDrillDown(null)
      setDrillDownError(null)
      return undefined
    }
    let cancelled = false
    setDrillDownLoading(true)
    setDrillDownError(null)
    getDashboardDrillDown(selectedMetric, token, { dateFrom, dateTo, page, pageSize })
      .then((data) => {
        if (!cancelled) setDrillDown(data)
      })
      .catch((err) => {
        if (!cancelled) setDrillDownError(err.message || 'Could not load that list.')
      })
      .finally(() => {
        if (!cancelled) setDrillDownLoading(false)
      })

    return () => {
      cancelled = true
    }
  }, [selectedMetric, token, dateFrom, dateTo, page, pageSize])

  // Changing the tile, the date range, or rows-per-page always jumps back
  // to page 1 - otherwise "page 3" from a longer list could land out of
  // range for a shorter, newly-filtered one.
  useEffect(() => {
    setPage(1)
  }, [selectedMetric, dateFrom, dateTo, pageSize])

  function toggleTile(metric) {
    if (selectedMetric === metric) {
      closeDrillDown()
    } else {
      setSelectedMetric(metric)
    }
  }

  function closeDrillDown() {
    setSelectedMetric(null)
    setDateFrom('')
    setDateTo('')
  }

```



```

    setPage(1)
    setPageSize(10)
  }

  if (loading) {
    return (
      <div className="dashboard-shell">
        <p className="dashboard-status-text">Loading dashboard</p>
      </div>
    )
  }

  if (error) {
    return (
      <div className="dashboard-shell">
        <p className="dashboard-status-text dashboard-status-text--error"><error></p>
      </div>
    )
  }

  const { metrics, risk_distribution: riskDistribution, category_breakdown: categoryBreakdown, recent_change_requests: recent } =
    summary

  if (metrics.total_change_requests === 0) {
    return (
      <div className="dashboard-shell">
        <EmptyState onCreateClick={() => onNavigate('new-analysis')} />
      </div>
    )
  }

  const riskData = RISK_ORDER.map((row) => ({
    label: row.label,
    count: riskDistribution[row.key] || 0,
    color: row.color,
  }))

  const categoryData = CATEGORY_ORDER.map((label) => ({
    label,
    count: categoryBreakdown[label] || 0,
  }))

  // Pagination summary for the popup's footer - "Showing 1-10 of 23" plus
  // whether there's another page to move to. Guarded against `drillDown`
  // being null (still loading, or errored) with sensible zero defaults.
  const drillDownTotal = drillDown ? drillDown.total : 0
  const rangeStart = drillDownTotal === 0 ? 0 : (page - 1) * pageSize + 1
  const rangeEnd = drillDown ? rangeStart + drillDown.items.length - 1 : 0
  const hasNextPage = drillDown ? rangeEnd < drillDownTotal : false

  return (
    <div className="dashboard-shell">
      <div className="dashboard-heading">
        <h1>Dashboard</h1>
        <p>An overview of every change request tracked in the system.</p>
      </div>

      <div className="metric-grid">
        <MetricCard
          label="Total Change Requests"
          value={metrics.total_change_requests}
          onClick={() => toggleTile('total')}
          active={selectedMetric === 'total'}
        />
        <MetricCard
          label="Pending Analysis"
          value={metrics.pending_analysis}
          tone="neutral"
          onClick={() => toggleTile('pending_analysis')}
          active={selectedMetric === 'pending_analysis'}
        />
        <MetricCard
          label="High Risk Changes"
          value={metrics.high_risk_changes}
          tone="danger"
          onClick={() => toggleTile('high_risk')}
          active={selectedMetric === 'high_risk'}
        />
        <MetricCard
          label="Approved Changes"
          value={metrics.approved_changes}
          tone="success"
          onClick={() => toggleTile('approved')}
          active={selectedMetric === 'approved'}
        />
        <MetricCard
          label="Requires Clarification"
          value={metrics.requires_clarification}
          tone="warning"
          onClick={() => toggleTile('requires_clarification')}
          active={selectedMetric === 'requires_clarification'}
        />
      </div>

      {selectedMetric && (
        <Modal
          title={` ${DRILLDOWN_TITLES[selectedMetric]} ${drillDown ? ` (${drillDown.total})` : ''} `}
          onClose={closeDrillDown}
          actions={
            <div>
              <span className="drilldown-pagination-info">
                {drillDown ? `Showing ${rangeStart}–${rangeEnd} of ${drillDownTotal}` : ''}
              </span>
              <div className="drilldown-pagination-controls">
                <button
                  type="button"
                  className="cr-btn cr-btn--ghost cr-btn--small"
                  onClick={() => setPage((current) => Math.max(1, current - 1))}
                  disabled={page <= 1}
                />
                <button
                  type="button"
                  className="cr-btn cr-btn--ghost cr-btn--small"
                  onClick={() => setPage((current) => current + 1)}
                  disabled={!hasNextPage}
                />
                <button
                  type="button"
                  className="cr-btn cr-btn--ghost cr-btn--small"
                  onClick={closeDrillDown}
                />
              </div>
            </div>
          )
        />
      )}

      <div className="drilldown-filters">
        <label className="drilldown-filter-field">
          <span>From</span>
          <input

```

```

        type="date"
        value={dateFrom}
        max={dateTo || undefined}
        onChange={(event) => setDateFrom(event.target.value)}
      />
    </label>
    <label className="drilldown-filter-field">
      <span>To</span>
      <input
        type="date"
        value={dateTo}
        min={dateFrom || undefined}
        onChange={(event) => setDateTo(event.target.value)}
      />
    </label>
    <label className="drilldown-filter-field">
      <span>Rows per page</span>
      <select value={pageSize} onChange={(event) => setPageSize(Number(event.target.value))}>
        {PAGE_SIZE_OPTIONS.map((size) => (
          <option key={size} value={size}>
            {size}
          </option>
        ))}
      </select>
    </label>
    {(dateFrom || dateTo) && (
      <button
        type="button"
        className="cr-btn cr-btn--ghost cr-btn--small"
        onClick={() => {
          setDateFrom('')
          setDateTo('')
        }}
      >
        Clear dates
      </button>
    )}
  </div>

  {drillDownLoading && <p className="dashboard-status-text">Loading...</p>
  {drillDownError && <p className="dashboard-status-text dashboard-status-text--error">{drillDownError}</p>
  {drillDownLoading && !drillDownError && drillDown && (
    <RecentRequestsTable
      items={drillDown.items}
      emptyMessage="No change requests match this filter."
      onRowClick={onOpenChangeRequest}
      bare
    />
  )}
</Modal>
))

<div className="chart-grid">
  <DistributionChart
    title="Risk Overview"
    data={riskData}
    emptyLabel="No analyzed change requests yet."
  />
  <DistributionChart
    title="Change Category Overview"
    data={categoryData}
    emptyLabel="No categorized change requests yet."
  />
</div>

<RecentRequestsTable items={recent} />
</div>
)
}

```

```

import { useEffect, useState } from 'react'
import { getChangeRequest, updateChangeRequest } from '../api/changeRequests'
import { useAuth } from '../context/AuthContext'
import '../styles/change-request-form.css'
import '../styles/workflow.css'

const PRIORITY_OPTIONS = [
  { value: 'low', label: 'Low', color: '#12b76a' },
  { value: 'medium', label: 'Medium', color: '#f79009' },
  { value: 'high', label: 'High', color: '#f04438' },
  { value: 'critical', label: 'Critical', color: '#b42318' },
]

const TARGET_SYSTEM_SUGGESTIONS = [
  'POS Backend',
  'POS Frontend',
  'Mobile App',
  'Admin Panel',
  'Payment Gateway',
  'Inventory System',
  'Reporting Dashboard',
  'API Gateway',
  'Database',
  'Infrastructure',
]

const DESCRIPTION_MIN = 20
const DESCRIPTION_MAX = 4000
const TITLE_MAX = 255

function todayIso() {
  return new Date().toISOString().slice(0, 10)
}

function formFromDetail(detail) {
  return {
    title: detail.title || '',
    description: detail.description || '',
    businessObjective: detail.business_objective || '',
    priority: detail.priority || 'medium',
    requestedBy: detail.requested_by || '',
    targetSystem: detail.target_system || '',
    desiredDeadline: detail.desired_deadline || '',
    businessImpact: detail.business_impact || '',
    technicalImpact: detail.technical_impact || '',
    customerImpact: detail.customer_impact || '',
    environment: detail.environment || '',
    dependenciesNote: detail.dependencies_note || '',
    complianceRequirements: detail.compliance_requirements || '',
    tags: (detail.tags || []).join(', '),
  }
}

function validate(form) {
  const errors = {}
  const title = form.title.trim()
  const description = form.description.trim()
  const requestedBy = form.requestedBy.trim()
  const targetSystem = form.targetSystem.trim()

  if (!title) errors.title = 'Title is required.'
  else if (title.length < 5) errors.title = 'Title must be at least 5 characters.'
  else if (title.length > TITLE_MAX) errors.title = `Title must be ${TITLE_MAX} characters or fewer.`

  if (!description) errors.description = 'Description is required.'
  else if (description.length < DESCRIPTION_MIN)
    errors.description = `Add a bit more detail - at least ${DESCRIPTION_MIN} characters.`
  else if (description.length > DESCRIPTION_MAX)
    errors.description = `Description must be ${DESCRIPTION_MAX} characters or fewer.`

  if (!requestedBy) errors.requestedBy = 'Requested By is required.'
  if (!targetSystem) errors.targetSystem = 'Target System is required.'

  if (form.desiredDeadline && form.desiredDeadline < todayIso()) {
    errors.desiredDeadline = 'Deadline can\'t be in the past.'
  }

  return errors
}

function parseTags(raw) {
  const values = raw
    .split(',')
    .map((v) => v.trim())
    .filter(Boolean)
  return values.length > 0 ? values : null
}

/** Module 12 Phase 2: the Edit form. Prefills from GET .../{id}, submits a
 * full PUT (the backend only ever records fields that actually changed -
 * see app/services/versioning.py - so sending back untouched values is a
 * harmless no-op), then shows the field-by-field "here's what changed"
 * confirmation the spec asks for before returning to the detail view. */
export default function EditChangeRequestPage({ id, onCancel, onSave }) {
  const { token } = useAuth()
  const [loading, setLoading] = useState(true)
  const [loadError, setLoadError] = useState(null)
  const [form, setForm] = useState(null)
  const [fieldErrors, setFieldErrors] = useState({})
  const [submitError, setSubmitError] = useState(null)
  const [submitting, setSubmitting] = useState(false)
  const [result, setResult] = useState(null) // the PUT response, once saved

  useEffect(() => {
    let cancelled = false
    setLoading(true)
    setLoadError(null)
    getChangeRequest(id, token)
      .then((detail) => {
        if (!cancelled) setForm(formFromDetail(detail))
      })
      .catch((err) => {
        if (!cancelled) setLoadError(err.message || 'Could not load this change request.')
      })
      .finally(() => {
        if (!cancelled) setLoading(false)
      })
    return () => {
      cancelled = true
    }
  }, [id, token])

  function updateField(field, value) {

```



```

{submitError} && <p className="cr-form-error">{submitError}</p>
<section className="cr-section">
  <h2 className="cr-section-title">Request Details</h2>

  <div className="cr-field">
    <label htmlFor="cr-title">Title</label>
    <input
      id="cr-title"
      type="text"
      value={form.title}
      maxLength={TITLE_MAX}
      onChange={(e) => updateField('title', e.target.value)}
      aria-invalid={Boolean(fieldErrors.title)}
    />
    <div className="cr-field-footer">
      {fieldErrors.title ? (
        <span className="cr-field-error">{fieldErrors.title}</span>
      ) : (
        <span className="cr-field-hint">A short, specific summary.</span>
      )}
      <span className="cr-char-count">
        {form.title.length}/{TITLE_MAX}
      </span>
    </div>
  </div>

  <div className="cr-field">
    <label htmlFor="cr-description">Description</label>
    <textarea
      id="cr-description"
      rows={5}
      value={form.description}
      maxLength={DESCRIPTION_MAX}
      onChange={(e) => updateField('description', e.target.value)}
      aria-invalid={Boolean(fieldErrors.description)}
    />
    <div className="cr-field-footer">
      {fieldErrors.description ? (
        <span className="cr-field-error">{fieldErrors.description}</span>
      ) : (
        <span className="cr-field-hint">Minimum {DESCRIPTION_MIN} characters.</span>
      )}
      <span className="cr-char-count">
        {form.description.length}/{DESCRIPTION_MAX}
      </span>
    </div>
  </div>

  <div className="cr-field">
    <label htmlFor="cr-objective">Business Objective <span className="cr-optional-tag">optional</span>
  </label>
  <textarea
    id="cr-objective"
    rows={2}
    value={form.businessObjective}
    onChange={(e) => updateField('businessObjective', e.target.value)}
  />
</div>
</section>

<section className="cr-section">
  <h2 className="cr-section-title">Classification & Ownership</h2>

  <div className="cr-field">
    <label>Priority</label>
    <div className="cr-priority-group" role="radiogroup" aria-label="Priority">
      {PRIORITY_OPTIONS.map((option) => (
        <button
          key={option.value}
          type="button"
          role="radio"
          aria-checked={form.priority === option.value}
          className={`cr-priority-btn${
            form.priority === option.value ? ' cr-priority-btn--active' : ''
          }`}
          style={form.priority === option.value ? { '--priority-color': option.color } : undefined}
          onClick={() => updateField('priority', option.value)}
        >
          {option.label}
        </button>
      ))}
    </div>
  </div>

  <div className="cr-field-row">
    <div className="cr-field">
      <label htmlFor="cr-requested-by">Requested By</label>
      <input
        id="cr-requested-by"
        type="text"
        value={form.requestedBy}
        maxLength={255}
        onChange={(e) => updateField('requestedBy', e.target.value)}
        aria-invalid={Boolean(fieldErrors.requestedBy)}
      />
      {fieldErrors.requestedBy} && <span className="cr-field-error">{fieldErrors.requestedBy}</span>
    </div>

    <div className="cr-field">
      <label htmlFor="cr-target-system">Target System</label>
      <input
        id="cr-target-system"
        type="text"
        list="cr-target-system-options"
        value={form.targetSystem}
        maxLength={255}
        onChange={(e) => updateField('targetSystem', e.target.value)}
        aria-invalid={Boolean(fieldErrors.targetSystem)}
      />
      <datalist id="cr-target-system-options">
        {TARGET_SYSTEM_SUGGESTIONS.map((suggestion) => (
          <option key={suggestion} value={suggestion} />
        ))}
      </datalist>
      {fieldErrors.targetSystem} && (
        <span className="cr-field-error">{fieldErrors.targetSystem}</span>
      )
    </div>
  </div>

  <div className="cr-field">
    <label htmlFor="cr-deadline">Desired Deadline <span className="cr-optional-tag">optional</span>
  </label>
  <input
    id="cr-deadline"
    type="date"
    value={form.desiredDeadline}
  />

```

```

        onChange={(e) => updateField('desiredDeadline', e.target.value)}
        aria-invalid={Boolean(fieldErrors.desiredDeadline)}
      />
      {fieldErrors.desiredDeadline && (
        <span className="cr-field-error">{fieldErrors.desiredDeadline}</span>
      )}
    </div>
  </section>

  <section className="cr-section">
    <h2 className="cr-section-title">Additional Details</h2>

    <div className="cr-field-row">
      <div className="cr-field">
        <label htmlFor="cr-business-impact">
          Business Impact <span className="cr-optional-tag">optional</span>
        </label>
        <textarea
          id="cr-business-impact"
          rows={2}
          value={form.businessImpact}
          onChange={(e) => updateField('businessImpact', e.target.value)}
        />
      </div>
      <div className="cr-field">
        <label htmlFor="cr-technical-impact">
          Technical Impact <span className="cr-optional-tag">optional</span>
        </label>
        <textarea
          id="cr-technical-impact"
          rows={2}
          value={form.technicalImpact}
          onChange={(e) => updateField('technicalImpact', e.target.value)}
        />
      </div>
    </div>

    <div className="cr-field-row">
      <div className="cr-field">
        <label htmlFor="cr-customer-impact">
          Customer Impact <span className="cr-optional-tag">optional</span>
        </label>
        <textarea
          id="cr-customer-impact"
          rows={2}
          value={form.customerImpact}
          onChange={(e) => updateField('customerImpact', e.target.value)}
        />
      </div>
      <div className="cr-field">
        <label htmlFor="cr-environment">
          Environment <span className="cr-optional-tag">optional</span>
        </label>
        <input
          id="cr-environment"
          type="text"
          value={form.environment}
          placeholder="e.g. Production"
          onChange={(e) => updateField('environment', e.target.value)}
        />
      </div>
    </div>

    <div className="cr-field">
      <label htmlFor="cr-dependencies">
        Dependencies <span className="cr-optional-tag">optional</span>
      </label>
      <textarea
        id="cr-dependencies"
        rows={2}
        value={form.dependenciesNote}
        onChange={(e) => updateField('dependenciesNote', e.target.value)}
      />
    </div>

    <div className="cr-field">
      <label htmlFor="cr-compliance">
        Compliance Requirements <span className="cr-optional-tag">optional</span>
      </label>
      <textarea
        id="cr-compliance"
        rows={2}
        value={form.complianceRequirements}
        onChange={(e) => updateField('complianceRequirements', e.target.value)}
      />
    </div>

    <div className="cr-field">
      <label htmlFor="cr-tags">
        Tags <span className="cr-optional-tag">optional, comma-separated</span>
      </label>
      <input
        id="cr-tags"
        type="text"
        value={form.tags}
        placeholder="e.g. mobile, auth, otp"
        onChange={(e) => updateField('tags', e.target.value)}
      />
    </div>
  </section>

  <div className="cr-form-actions">
    <button
      type="button"
      className="cr-btn cr-btn--ghost"
      onClick={onCancel}
      disabled={submitting}
    >
      Cancel
    </button>
    <button type="submit" className="cr-btn cr-btn--primary" disabled={submitting}>
      {submitting ? 'Saving...' : 'Save Changes'}
    </button>
  </div>
</div>
)
}

```

```

import { Fragment, useEffect, useState } from 'react'
import {
  archiveKnowledgeDocument,
  embedKnowledgeDocument,
  getKnowledgeDocumentChunks,
  listKnowledgeDocuments,
  reprocessKnowledgeDocument,
  searchKnowledgeBase,
  unarchiveKnowledgeDocument,
  uploadKnowledgeDocument,
} from '../api/knowledge'
import { useAuth } from '../context/AuthContext'
// Reuses the exact same .cr-list-shell / .cr-toolbar / .table-card /
// .requests-table / .badge / .cr-btn / .cr-analysis-list classes every
// other list page here already defines, rather than inventing a second
// set - only knowledge-base.css adds the handful of rules genuinely new
// to this page (the upload control, the search-results panel).
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'
import '../styles/workflow.css'
import '../styles/knowledge-base.css'

const STATUS_LABELS = { uploaded: 'Uploaded', processing: 'Processing', ready: 'Ready', failed: 'Failed' }
const STATUS_CLASS = {
  uploaded: 'badge--gray',
  processing: 'badge--blue',
  ready: 'badge--green',
  failed: 'badge--red',
}

const ALLOWED_EXTENSIONS = ['.txt', '.md', '.pdf']

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleDateString(undefined, { year: 'numeric', month: 'short', day: 'numeric' })
  } catch {
    return isoString
  }
}

/** One document's inline, expandable chunk preview - fetched lazily the
 * first time a row is expanded, then cached in the parent's chunksById so
 * re-expanding doesn't re-fetch. Shows each chunk's section label and
 * whether it's been embedded yet (never the raw vector itself - see
 * knowledge chunk.py's own schema docstring for why). */
function ChunkPreview({ chunks, loading, error }) {
  if (loading) return <p className="cr-list-status-text">Loading chunks</p>
  if (error) return <p className="cr-list-status-text cr-list-status-text--error">{error}</p>
  if (!chunks || chunks.length === 0) {
    return <p className="ad-empty">This document has no chunks yet.</p>
  }
  return (
    <ul className="cr-analysis-list kb-chunk-list">
      {chunks.map((chunk) => (
        <li key={chunk.id} className="cr-analysis-list-item">
          <span className={`badge ${chunk.is_embedded ? 'badge--green' : 'badge--gray'}`}>
            {chunk.is_embedded ? 'Embedded' : 'Not embedded'}
          </span>
          <div>
            <p className="cr-analysis-list-tag">
              {chunk.section_label || `Chunk ${chunk.chunk_index + 1}`}
            </p>
            <p className="cr-analysis-list-text">{chunk.content}</p>
          </div>
        </li>
      ))}
    </ul>
  )
}

/** Module 16 Frontend phase: the Knowledge Base page - upload documents
 * (.txt/.md/.pdf), see their parse/embed status, search the knowledge
 * base directly, preview a document's parsed chunks, and archive/
 * unarchive documents. Everything shown here is read straight from the
 * backend (Module 16 Phases 1-3) - no client-side chunking, embedding, or
 * scoring logic lives in this file. */
export default function KnowledgeBasePage() {
  const { token } = useAuth()

  const [documents, setDocuments] = useState([])
  const [loading, setLoading] = useState(true)
  const [loadError, setLoadError] = useState(null)
  const [refreshToken, setRefreshToken] = useState(0)
  const [showArchived, setShowArchived] = useState(false)

  const [uploading, setUploading] = useState(false)
  const [uploadError, setUploadError] = useState(null)

  const [actionBusyId, setActionBusyId] = useState(null)
  const [actionError, setActionError] = useState(null)

  const [expandedId, setExpandedId] = useState(null)
  const [chunksById, setChunksById] = useState({})
  const [chunksLoadingId, setChunksLoadingId] = useState(null)
  const [chunksError, setChunksError] = useState(null)

  const [searchInput, setSearchInput] = useState('')
  const [searching, setSearching] = useState(false)
  const [searchError, setSearchError] = useState(null)
  const [searchResults, setSearchResults] = useState(null)

  useEffect(() => {
    let cancelled = false
    setLoading(true)
    setLoadError(null)

    listKnowledgeDocuments(token, { includeArchived: showArchived })
      .then((response) => {
        if (!cancelled) setDocuments(response)
      })
      .catch((err) => {
        if (!cancelled) setLoadError(err.message || 'Could not load the knowledge base.')
      })
      .finally(() => {
        if (!cancelled) setLoading(false)
      })

    return () => {
      cancelled = true
    }
  }, [token, showArchived, refreshToken])
}

```

```

function refresh() {
  setRefreshToken((k) => k + 1)
}

function handleFileChosen(e) {
  const file = e.target.files?.[0]
  e.target.value = ''
  if (!file) return

  const extension = `${file.name.split('.').pop()?.toLowerCase()}`
  if (!ALLOWED_EXTENSIONS.includes(extension)) {
    setUploadError(`Unsupported file type "${extension}" - only .txt, .md, and .pdf are accepted.`)
    return
  }

  setUploading(true)
  setUploadError(null)
  uploadKnowledgeDocument(file, token)
    .then(() => refresh())
    .catch((err) => setUploadError(err.message || 'Upload failed.))
    .finally(() => setUploading(false))
}

function runAction(id, action) {
  setActionBusyId(id)
  setActionError(null)
  action()
    .then(() => refresh())
    .catch((err) => setActionError(err.message || 'That action failed.))
    .finally(() => setActionBusyId(null))
}

function toggleExpanded(doc) {
  if (expandedId === doc.id) {
    setExpandedId(null)
    return
  }
  setExpandedId(doc.id)
  if (chunksById[doc.id]) return

  setChunksLoadingId(doc.id)
  setChunksError(null)
  getKnowledgeDocumentChunks(doc.id, token)
    .then((chunks) => setChunksById((prev) => ({ ...prev, [doc.id]: chunks })))
    .catch((err) => setChunksError(err.message || 'Could not load this document's chunks.))
    .finally(() => setChunksLoadingId(null))
}

function handleSearchSubmit(e) {
  e.preventDefault()
  const query = searchInput.trim()
  if (!query) return

  setSearching(true)
  setSearchError(null)
  searchKnowledgeBase(query, token)
    .then((results) => setSearchResults(results))
    .catch((err) => setSearchError(err.message || 'Search failed.))
    .finally(() => setSearching(false))
}

function clearSearch() {
  setSearchInput('')
  setSearchResults(null)
  setSearchError(null)
}

return (
  <div className="cr-list-shell">
    <div className="cr-list-heading">
      <div>
        <h1>Knowledge Base</h1>
        <p>Upload project documentation so CR analysis can ground itself in what your team already knows.</p>
      </div>
      <div className="kb-upload-control">
        <input
          type="file"
          accept=".txt,.md,.pdf"
          onChange={handleFileChosen}
          className="kb-file-input"
          id="kb-file-input"
        />
        <label htmlFor="kb-file-input" className="cr-btn cr-btn--primary">
          {uploading ? 'Uploading...' : 'Upload Document'}
        </label>
      </div>
      <div>
        {uploadError && <p className="cr-form-error">{uploadError}</p>}
      </div>
      <form className="cr-toolbar" onSubmit={handleSearchSubmit}>
        <input
          type="text"
          className="cr-search-input"
          placeholder="Search the knowledge base (e.g. "OTP rate limiting")..."
          value={searchInput}
          onChange={(e) => setSearchInput(e.target.value)}
        />
        <button type="submit" className="cr-btn cr-btn--secondary" disabled={searching || !searchInput.trim()}>
          {searching ? 'Searching...' : 'Search'}
        </button>
        {searchResults != null && (
          <button type="button" className="cr-btn cr-btn--ghost" onClick={clearSearch}>
            Clear
          </button>
        )}
        <label className="cr-toolbar-checkbox">
          <input type="checkbox" checked={showArchived} onChange={(e) => setShowArchived(e.target.checked)} />
          Show archived
        </label>
      </form>
      {searchError && <p className="cr-form-error">{searchError}</p>}
      {searchResults != null && (
        <div className="cr-detail-section kb-search-results">
          <h2 className="cr-section-title">Search Results</h2>
          {searchResults.length === 0 ? (
            <p className="ad-empty">Nothing in the knowledge base is relevant to that search.</p>
          ) : (
            <ul className="cr-analysis-list">
              {searchResults.map((result) => (
                <li key={result.chunk_id} className="cr-analysis-list-item">
                  <div>
                    <p className="cr-analysis-list-tag">
                      {result.document title}
                      {result.section_label ? ` · ${result.section_label}` : ''}
                    </p>
                    <p className="cr-analysis-list-text">{result.content}</p>
                  </div>
                </li>
              )}
            </ul>
          )}
        </div>
      )}
    </div>
  )

```



```

    </li>
  </ul>
</div>
))
{loading && <p className="cr-list-status-text">Loading knowledge base...</p>}
{loadError && <p className="cr-list-status-text cr-list-status-text-error">{loadError}</p>}
{actionError && <p className="cr-form-error">{actionError}</p>}

{!loading && !loadError && documents.length === 0 && (
  <div className="cr-list-empty">
    <p className="cr-list-empty-title">
      {showArchived ? 'No documents at all yet.' : 'No documents yet.'}
    </p>
    <p className="cr-list-status-text">
      Upload a .txt, .md, or .pdf file above to start building your team's knowledge base.
    </p>
  </div>
)}

{!loading && !loadError && documents.length > 0 && (
  <div className="table-card cr-table-card">
    <div className="table-scroll">
      <table className="requests-table">
        <thead>
          <tr>
            <th>Title</th>
            <th>Status</th>
            <th>Chunks</th>
            <th>Uploaded</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {documents.map((doc) => {
            const busy = actionBusyId === doc.id
            return (
              <Fragment key={doc.id}>
                <tr className="cr-list-row" onClick={() => toggleExpanded(doc)}>
                  <td className="requests-table-title">
                    {doc.title}
                  </td>
                  <td>
                    {doc.archived && <span className="badge badge--gray kb-inline-badge">Archived</span>}
                  </td>
                  <td>
                    <span className={`badge ${STATUS_CLASS[doc.status] || 'badge--gray'}`}>
                      {STATUS_LABELS[doc.status] || doc.status}
                    </span>
                    {doc.status === 'failed' && doc.error_message && (
                      <p className="cr-analysis-list-mitigation">{doc.error_message}</p>
                    )}
                  </td>
                  <td className="table-muted">{doc.chunk_count}</td>
                  <td className="table-muted">{formatDate(doc.uploaded_at)}</td>
                  <td className="kb-actions" onClick={(e) => e.stopPropagation()}>
                    {doc.status === 'failed' && (
                      <button
                        type="button"
                        className="cr-btn cr-btn--secondary cr-btn--small"
                        disabled={busy}
                        onClick={() => runAction(doc.id, () => reprocessKnowledgeDocument(doc.id, token))}
                      >
                        Reprocess
                      </button>
                    )}
                    {doc.status === 'ready' && (
                      <button
                        type="button"
                        className="cr-btn cr-btn--secondary cr-btn--small"
                        disabled={busy}
                        onClick={() => runAction(doc.id, () => embedKnowledgeDocument(doc.id, token))}
                      >
                        {doc.chunk_count > 0 ? 'Embed / Re-embed' : 'Embed'}
                      </button>
                    )}
                    {doc.archived ? (
                      <button
                        type="button"
                        className="cr-btn cr-btn--secondary cr-btn--small"
                        disabled={busy}
                        onClick={() => runAction(doc.id, () => unarchiveKnowledgeDocument(doc.id, token))}
                      >
                        Unarchive
                      </button>
                    ) : (
                      <button
                        type="button"
                        className="cr-btn cr-btn--secondary cr-btn--small"
                        disabled={busy}
                        onClick={() => runAction(doc.id, () => archiveKnowledgeDocument(doc.id, token))}
                      >
                        Archive
                      </button>
                    )}
                  </td>
                </tr>
              </Fragment>
            )
          )}
          {expandedId === doc.id && (
            <tr className="kb-preview-row">
              <td colspan={5}>
                <ChunkPreview
                  chunks={chunksById[doc.id]}
                  loading={chunksLoadingId === doc.id}
                  error={chunksLoadingId === doc.id ? null : chunksError}
                />
              </td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  </div>
)}
</div>
)
}

```

```

import { useState } from 'react'
import AuthLayout from '../components/AuthLayout'
import { ApiError } from '../api/client'
import { useAuth } from '../context/AuthContext'

export default function LoginPage({ onSwitchToRegister }) {
  const { login } = useAuth()
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [rememberMe, setRememberMe] = useState(false)
  const [error, setError] = useState(null)
  const [submitting, setSubmitting] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    setError(null)
    setSubmitting(true)
    try {
      await login(email, password, rememberMe)
    } catch (err) {
      setError(err instanceof ApiError ? err.message : 'Something went wrong. Please try again.')
    } finally {
      setSubmitting(false)
    }
  }

  return (
    <AuthLayout>
      <h1 className="auth-title">Welcome back</h1>
      <p className="auth-subtitle">Sign in to continue to your workspace.</p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label className="auth-field">
          <span>Email</span>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            autoComplete="email"
            placeholder="you@company.com"
          />
        </label>

        <label className="auth-field">
          <span>Password</span>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            autoComplete="current-password"
            placeholder="....."
          />
        </label>

        <div className="auth-row">
          <label className="auth-checkbox">
            <input type="checkbox" checked={rememberMe} onChange={(e) => setRememberMe(e.target.checked)} />
            <span>Remember me</span>
          </label>
          /* Module 23: removed a disabled "Forgot password? (Coming soon)"
             control that lived here - a dead, always-disabled link isn't
             worth showing on the very first screen of a demo. */
        </div>

        {error && <p className="auth-error">{error}</p>}

        <button type="submit" className="auth-submit" disabled={submitting}>
          {submitting ? 'Signing in...' : 'Sign in'}
        </button>
      </form>

      <p className="auth-switch">
        Don't have an account?{' '}
        <button type="button" className="auth-link-button" onClick={onSwitchToRegister}>
          Create one
        </button>
      </p>
    </AuthLayout>
  )
}

```

```

import { useEffect, useState } from 'react'
import {
  getChangesRequestedFromMe,
  getMyApprovals,
  getMyAssignments,
  getMyChangeRequests,
  getMyMentions,
  getMyReviews,
  getMyWorkSummary,
  getOverdueItems,
} from '../api/myWork'
import { respondToApproval } from '../api/approvals'
import { useAuth } from '../context/AuthContext'
import MetricCard from '../components/MetricCard'
import '../styles/dashboard.css'
import '../styles/change-requests-list.css'
import '../styles/analysis-dashboard.css'
import '../styles/workflow.css'
import '../styles/my-work.css'

// Mirrors ChangeRequestsPage.jsx's own label/class maps (module-local by
// the same convention, rather than a shared import - see that file).
const STATUS_LABELS = {
  draft: 'Draft',
  pending_analysis: 'Pending Analysis',
  requires_clarification: 'Requires Clarification',
  completed: 'Completed',
  approved: 'Approved',
}
const STATUS_CLASS = {
  draft: 'badge--gray',
  pending_analysis: 'badge--indigo',
  requires_clarification: 'badge--amber',
  completed: 'badge--teal',
  approved: 'badge--green',
}
const PRIORITY_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const PRIORITY_CLASS = { low: 'badge--green', medium: 'badge--amber', high: 'badge--orange', critical: 'badge--red' }
const RISK_LABELS = { low: 'Low', medium: 'Medium', high: 'High', critical: 'Critical' }
const RISK_CLASS = { low: 'badge--green', medium: 'badge--amber', high: 'badge--orange', critical: 'badge--red' }

// Module 18 Phase 1's own two due status values.
const DUE_LABELS = { overdue: 'Overdue', due_soon: 'Due soon' }
const DUE_CLASS = { overdue: 'badge--red', due_soon: 'badge--amber' }

const ROLE_LABELS = {
  requester: 'Requester',
  owner: 'Owner',
  technical_lead: 'Technical Lead',
  reviewer: 'Reviewer',
  approver: 'Approver',
  security_reviewer: 'Security Reviewer',
  qa_owner: 'QA Owner',
  implementation_owner: 'Implementation Owner',
}

const TABS = [
  { key: 'change-requests', label: 'My Change Requests', summaryKey: 'my_change_requests' },
  { key: 'approvals', label: 'My Approvals', summaryKey: 'my_approvals_pending' },
  { key: 'reviews', label: 'My Reviews', summaryKey: 'my_reviews' },
  { key: 'assignments', label: 'My Assignments', summaryKey: 'my_assignments' },
  { key: 'changes-requested', label: 'Changes Requested From Me', summaryKey: 'changes_requested_from_me' },
  { key: 'mentions', label: 'Mentions', summaryKey: 'unread_mentions' },
  { key: 'overdue', label: 'Overdue Items', summaryKey: 'overdue_approvals' },
]

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleDateString(undefined, { year: 'numeric', month: 'short', day: 'numeric' })
  } catch {
    return isoString
  }
}

function formatDateTime(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

function RiskBadge({ risk }) {
  if (!risk) return <span className="table-muted"></span>
  return <span className={ `badge ${RISK_CLASS[risk]} || 'badge--gray'` }>{RISK_LABELS[risk] || risk}</span>
}

function DueBadge({ dueStatus, dueDate }) {
  if (!dueDate) return <span className="table-muted"></span>
  if (!dueStatus) return <span className="table-muted">{formatDate(dueDate)}</span>
  return (
    <span className={ `badge ${DUE_CLASS[dueStatus]} || 'badge--gray'` }>
      {DUE_LABELS[dueStatus] || dueStatus} {formatDate(dueDate)}
    </span>
  )
}

function RolesCell({ roles }) {
  if (!roles || roles.length === 0) return <span className="table-muted"></span>
  return <{roles.map((r) => ROLE_LABELS[r] || r).join(', ')}></>
}

/** Module 18 Phase 5: the CR/Approval Type/Risk/Requested By/Due Date/
 * Status table (spec section 4) gets its Actions column from this shared
 * cell - Approve (one click, no comment), Reject/Request Changes (open an
 * inline comment box, comment required - same rule
 * app/services/workflow.rules.py:APPROVAL_REASON_REQUIRED already
 * enforces server-side), and View CR. Used by both the My Approvals and
 * Overdue Items tables since both list the same underlying approval rows. */
function ApprovalActionsCell({
  item,
  isResponding,
  respondStatus,
  respondComment,
  respondSubmitting,
  respondError,

```

```

onOpenRespondForm,
onCloseRespondForm,
onSubmitResponse,
onSetRespondComment,
onOpenCr,
}) {
  if (isResponding) {
    return (
      <div className="respond-form" onClick={(e) => e.stopPropagation()}>
        {respondError && <p className="cr-form-error">{respondError}</p>}
        <p className="version-row-meta">
          {respondStatus === 'rejected' ? 'Rejecting' : 'Requesting changes'} - a comment is required.
        </p>
        <textarea
          rows={2}
          value={respondComment}
          onChange={(e) => onSetRespondComment(e.target.value)}
          aria-label="Comment"
        </textarea>
        <div className="status-form-row">
          <button
            type="button"
            className="cr-btn cr-btn--primary cr-btn--small"
            disabled={respondSubmitting || !respondComment.trim()}
            onClick={() => onSubmitResponse(item, respondStatus, respondComment)}
          >
            {respondSubmitting ? 'Submitting...' : 'Submit'}
          </button>
          <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={onCloseRespondForm}>
            Cancel
          </button>
        </div>
      </div>
    )
  }
  return (
    <div className="approval-actions-cell" onClick={(e) => e.stopPropagation()}>
      {item.status === 'pending' && (
        <button
          type="button"
          className="cr-btn cr-btn--primary cr-btn--small"
          disabled={respondSubmitting}
          onClick={() => onSubmitResponse(item, 'approved', null)}
        >
          Approve
        </button>
        <button
          type="button"
          className="cr-btn cr-btn--secondary cr-btn--small"
          disabled={respondSubmitting}
          onClick={() => onOpenRespondForm(item, 'rejected')}
        >
          Reject
        </button>
        <button
          type="button"
          className="cr-btn cr-btn--secondary cr-btn--small"
          disabled={respondSubmitting}
          onClick={() => onOpenRespondForm(item, 'changes_requested')}
        >
          Request Changes
        </button>
      )}
      <button type="button" className="cr-btn cr-btn--ghost cr-btn--small" onClick={() => onOpenCr(item.change_request_id)}>
        View CR
      </button>
    </div>
  )
}

```

```

/** Module 18 Phase 4 (Notifications, My Work & Personal Engineering
 * Queue): the personal "what's on my plate" dashboard - My Change
 * Requests, My Approvals, My Reviews, My Assignments, Changes Requested
 * From Me, Mentions, and Overdue Items, each fetched from its own
 * dedicated, efficiently-filtered backend endpoint (see
 * backend/app/api/my_work.py) and lazily loaded only once its tab is
 * actually opened - never all seven sections fetched up front.
 *
 * Approve/Reject/Request Changes actions on My Approvals (and Overdue
 * Items, the same underlying approval rows) are wired up as of Phase 5,
 * reusing the same POST .../approvals/{id}/respond endpoint and
 * respondToApproval() helper Module 12's own ChangeRequestDetail.jsx page
 * already uses - no second response pathway. The priority highlighting
 * spec section 6 asks for (Overdue/Due Soon/High Risk/Critical Risk/
 * Waiting for Me) is computed already by the backend and just rendered
 * here as badges. */
export default function MyWorkPage({ onOpenChangeRequest }) {
  const { token } = useAuth()
  const [activeTab, setActiveTab] = useState('approvals')
  const [summary, setSummary] = useState(null)
  const [dataByTab, setDataByTab] = useState({})
  const [loadingTab, setLoadingTab] = useState(null)
  const [error, setError] = useState(null)
  const [approvalsShowAll, setApprovalsShowAll] = useState(false)

  // Module 18 Phase 5: which pending approval's Approve/Reject/Request-
  // Changes form is open, plus that one form's own fields - only one open
  // at a time, mirrors ChangeRequestDetail.jsx's own respond-form state.
  const [respondingApprovalId, setRespondingApprovalId] = useState(null)
  const [respondStatus, setRespondStatus] = useState('approved')
  const [respondComment, setRespondComment] = useState('')
  const [respondSubmitting, setRespondSubmitting] = useState(false)
  const [respondError, setRespondError] = useState(null)

  function loadSummary() {
    getMyWorkSummary(token)
      .then(setSummary)
      .catch(() => {})
  }

  function loadTab(tab, { force = false } = {}) {
    if (!force && dataByTab[tab] !== undefined) return
    setLoadingTab(tab)
    setError(null)

    const fetcher = {
      'change-requests': () => getMyChangeRequests(token),
      approvals: () => getMyApprovals(token, { all: approvalsShowAll }),
      reviews: () => getMyReviews(token),
      assignments: () => getMyAssignments(token),
      'changes-requested': () => getChangesRequestedFromMe(token),
      mentions: () => getMyMentions(token),
      overdue: () => getOverdueItems(token),
    }[tab]

    fetcher()
      .then((data) => setDataByTab((prev) => ({ ...prev, [tab]: data })))
      .catch((err) => setError(err.message || 'Could not load this list.'))
  }

```



```

    />
    Show every approval (including already-decided ones)
  </label>
)}

{!isLoading && <p className="dashboard-status-text">Loading...</p>}
{error && <p className="dashboard-status-text dashboard-status-text--error">{error}</p>}}

{!isLoading && !error && rows && rows.length === 0 && (
  <div className="dashboard-empty">
    <h2 className="dashboard-empty-title">Nothing here</h2>
    <p className="dashboard-empty-subtitle">
      {activeTab === 'approvals' && !approvalsShowAll
        ? "You're all caught up - no approvals waiting on you."
        : "Nothing to show in this section right now."}
    </p>
  </div>
)}

{!isLoading && !error && rows && rows.length > 0 && activeTab === 'change-requests' && (
  <TableCard>
    <thead>
      <tr>
        <th>ID</th>
        <th>Title</th>
        <th>Priority</th>
        <th>Status</th>
        <th>Risk</th>
        <th>Created</th>
      </tr>
    </thead>
    <tbody>
      {rows.map((item) => (
        <tr key={item.id} className="cr-list-row" onClick={() => openCr(item.id)}>
          <td className="requests-table-id">#{item.id}</td>
          <td className="requests-table-title">{item.title}</td>
          <td>
            <span className={`badge ${PRIORITY_CLASS[item.priority]} || 'badge--gray'}`>
              {PRIORITY_LABELS[item.priority]} || item.priority
            </span>
          </td>
          <td>
            <span className={`badge ${STATUS_CLASS[item.status]} || 'badge--gray'}`>
              {STATUS_LABELS[item.status]} || item.status
            </span>
          </td>
          <td>
            <RiskBadge risk={item.risk} />
          </td>
          <td className="table-muted">{formatDate(item.created_at)}</td>
        </tr>
      ))}
    </tbody>
  </TableCard>
)}

{!isLoading && !error && rows && rows.length > 0 && activeTab === 'approvals' && (
  <TableCard>
    <thead>
      <tr>
        <th>CR</th>
        <th>Approval Type</th>
        <th>Risk</th>
        <th>Requested By</th>
        <th>Due Date</th>
        <th>Status</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {rows.map((item) => (
        <tr key={item.id} className="cr-list-row" onClick={() => openCr(item.change_request_id)}>
          <td className="requests-table-title">
            #{item.change_request_id} · {item.change_request_title}
          </td>
          <td>{item.approval_type_label}</td>
          <td>
            <RiskBadge risk={item.risk} />
          </td>
          <td>{item.requested_by_name}</td>
          <td>
            <DueBadge dueStatus={item.due_status} dueDate={item.due_date} />
          </td>
          <td>
            <span className="badge badge--gray">{item.status_label}</span>
            {item.status === 'pending' && <span className="badge badge--indigo waiting-badge">Waiting for you</span>}
          </td>
          <td>
            <ApprovalActionsCell
              item={item}
              isResponding={respondingApprovalId === item.id}
              respondStatus={respondStatus}
              respondComment={respondComment}
              respondSubmitting={respondSubmitting}
              respondError={respondError}
              onOpenRespondForm={openRespondForm}
              onCloseRespondForm={closeRespondForm}
              onSubmitResponse={submitResponse}
              onSetRespondComment={setRespondComment}
              onOpenCr={openCr}
            />
          </td>
        </tr>
      ))}
    </tbody>
  </TableCard>
)}

{!isLoading && !error && rows && rows.length > 0 && (activeTab === 'reviews' || activeTab === 'assignments') && (
  <TableCard>
    <thead>
      <tr>
        <th>CR</th>
        <th>Your Role(s)</th>
        <th>Priority</th>
        <th>Status</th>
        <th>Risk</th>
      </tr>
    </thead>
    <tbody>
      {rows.map((item) => (
        <tr key={item.change_request_id} className="cr-list-row" onClick={() => openCr(item.change_request_id)}>
          <td className="requests-table-title">
            #{item.change_request_id} · {item.title}
          </td>
          <td>
            <RolesCell roles={item.roles} />
          </td>
          <td>
            <span className={`badge ${PRIORITY_CLASS[item.priority]} || 'badge--gray'}`>
              {PRIORITY_LABELS[item.priority]} || item.priority
            </span>
          </td>
          <td>
            <span className={`badge ${STATUS_CLASS[item.status]} || 'badge--gray'}`>
              {STATUS_LABELS[item.status]} || item.status
            </span>
          </td>
          <td>
            <RiskBadge risk={item.risk} />
          </td>
        </tr>
      ))}
    </tbody>
  </TableCard>
)}

```



```

import { useState } from 'react'
import { analyzeChangeRequest } from '../api/analysis'
import { createChangeRequest } from '../api/changeRequests'
import { useAuth } from '../context/AuthContext'
import '../styles/change-request-form.css'

const PRIORITY_OPTIONS = [
  { value: 'low', label: 'Low', color: '#12b76a' },
  { value: 'medium', label: 'Medium', color: '#f79009' },
  { value: 'high', label: 'High', color: '#f04438' },
  { value: 'critical', label: 'Critical', color: '#b42318' },
]

const TARGET_SYSTEM_SUGGESTIONS = [
  'POS Backend',
  'POS Frontend',
  'Mobile App',
  'Admin Panel',
  'Payment Gateway',
  'Inventory System',
  'Reporting Dashboard',
  'API Gateway',
  'Database',
  'Infrastructure',
]

const DESCRIPTION_MIN = 20
const DESCRIPTION_MAX = 4000
const TITLE_MAX = 255

function todayIso() {
  return new Date().toISOString().slice(0, 10)
}

function emptyForm(defaultRequestedBy) {
  return {
    title: '',
    description: '',
    businessObjective: '',
    priority: 'medium',
    requestedBy: defaultRequestedBy || '',
    targetSystem: '',
    desiredDeadline: '',
  }
}

function validate(form) {
  const errors = {}
  const title = form.title.trim()
  const description = form.description.trim()
  const requestedBy = form.requestedBy.trim()
  const targetSystem = form.targetSystem.trim()

  if (!title) errors.title = 'Title is required.'
  else if (title.length < 5) errors.title = 'Title must be at least 5 characters.'
  else if (title.length > TITLE_MAX) errors.title = `Title must be ${TITLE_MAX} characters or fewer.`

  if (!description) errors.description = 'Description is required.'
  else if (description.length < DESCRIPTION_MIN)
    errors.description = `Add a bit more detail - at least ${DESCRIPTION_MIN} characters.`
  else if (description.length > DESCRIPTION_MAX)
    errors.description = `Description must be ${DESCRIPTION_MAX} characters or fewer.`

  if (!requestedBy) errors.requestedBy = 'Requested By is required.'
  if (!targetSystem) errors.targetSystem = 'Target System is required.'

  if (form.desiredDeadline && form.desiredDeadline < todayIso()) {
    errors.desiredDeadline = "Deadline can't be in the past."
  }

  return errors
}

/** The Module 4 workflow: a form that creates a ChangeRequest (status
 * "pending analysis") and, on success, a confirmation screen with an
 * "Analyze Change" button. Clicking it just explains AI analysis isn't
 * built yet (later module) - it deliberately does nothing else here. */
export default function NewChangeRequestPage({ onNavigate }) {
  const { user, token } = useAuth()
  const [form, setForm] = useState(() => emptyForm(user?.name))
  const [fieldErrors, setFieldErrors] = useState({})
  const [submitError, setSubmitError] = useState(null)
  const [submitting, setSubmitting] = useState(false)
  const [created, setCreated] = useState(null)
  const [analyzing, setAnalyzing] = useState(false)
  const [analyzeError, setAnalyzeError] = useState(null)
  const [analyzeDone, setAnalyzeDone] = useState(false)

  function updateField(field, value) {
    setForm((prev) => ({ ...prev, [field]: value }))
    if (fieldErrors[field]) {
      setFieldErrors((prev) => {
        const next = { ...prev }
        delete next[field]
        return next
      })
    }
  }

  async function handleSubmit(event) {
    event.preventDefault()
    const errors = validate(form)
    setFieldErrors(errors)
    if (Object.keys(errors).length > 0) {
      return
    }

    setSubmitting(true)
    setSubmitError(null)
    try {
      const payload = {
        title: form.title.trim(),
        description: form.description.trim(),
        businessObjective: form.businessObjective.trim() || null,
        priority: form.priority,
        requestedBy: form.requestedBy.trim(),
        targetSystem: form.targetSystem.trim(),
        desiredDeadline: form.desiredDeadline || null,
      }
      const response = await createChangeRequest(payload, token)
      setCreated(response)
    } catch (err) {
      setSubmitError(err.message || 'Something went wrong. Please try again.')
    } finally {

```



```

import { useEffect, useState } from 'react'
import {
  listNotifications,
  markAllNotificationsRead,
  markNotificationRead,
} from '../api/notifications'
import { useAuth } from '../context/AuthContext'
import '../styles/dashboard.css'
import '../styles/workflow.css'

// Mirrors app/models/enums.py::NotificationType.
const TYPE_LABELS = {
  assigned: 'Assigned',
  approval_requested: 'Approval Requested',
  approved: 'Approved',
  rejected: 'Rejected',
  changes_requested: 'Changes Requested',
  mentioned: 'Mentioned',
  status_changed: 'Status Changed',
  analysis_outdated: 'Analysis Outdated',
  reapproval_required: 'Re-approval Needed',
  reminder: 'Reminder',
  comment_added: 'New Comment',
  // Module 18 Phase 1/2 (Notifications, My Work & Personal Engineering Queue).
  risk_escalated: 'Risk Escalated',
  analysis_completed: 'Analysis Completed',
  deadline_approaching: 'Deadline Approaching',
}

const TYPE_CLASS = {
  assigned: 'badge--indigo',
  approval_requested: 'badge--indigo',
  approved: 'badge--green',
  rejected: 'badge--red',
  changes_requested: 'badge--amber',
  mentioned: 'badge--blue',
  status_changed: 'badge--gray',
  analysis_outdated: 'badge--amber',
  reapproval_required: 'badge--amber',
  reminder: 'badge--blue',
  comment_added: 'badge--gray',
  // Module 18 Phase 1/2.
  risk_escalated: 'badge--red',
  analysis_completed: 'badge--green',
  deadline_approaching: 'badge--amber',
}

function formatDateTime(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

/** Module 12 Phase 5: a user's own in-app notification inbox - assigned to
 * a CR, tagged for (or reminded about) an approval, an approval was
 * responded to, a CR's status changed, @mentioned or commented on, an AI
 * analysis went outdated, or a re-approval is needed. Clicking a
 * notification that references a change request marks it read and jumps
 * straight to that change request. */
export default function NotificationsPage({ onOpenChangeRequest }) {
  const { token } = useAuth()
  const [notifications, setNotifications] = useState(null)
  const [error, setError] = useState(null)
  const [loading, setLoading] = useState(true)
  const [unreadOnly, setUnreadOnly] = useState(false)
  const [markingAll, setMarkingAll] = useState(false)

  function load() {
    setLoading(true)
    setError(null)
    listNotifications(token, { unreadOnly, limit: 100 })
      .then(setNotifications)
      .catch((err) => setError(err.message || 'Could not load notifications.'))
      .finally(() => setLoading(false))
  }

  useEffect(() => {
    load()
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [token, unreadOnly])

  function handleOpen(notification) {
    if (!notification.is_read) {
      markNotificationRead(notification.id, token)
        .then((updated) => {
          setNotifications((prev) => (prev || []).map((n) => (n.id === updated.id ? updated : n)))
        })
        .catch(() => {})
    }
    if (notification.change_request_id !== null) {
      onOpenChangeRequest?.(notification.change_request_id)
    }
  }

  function handleMarkRead(event, notification) {
    event.stopPropagation()
    markNotificationRead(notification.id, token)
      .then((updated) => {
        setNotifications((prev) => (prev || []).map((n) => (n.id === updated.id ? updated : n)))
      })
      .catch(() => {})
  }

  async function handleMarkAllRead() {
    setMarkingAll(true)
    try {
      await markAllNotificationsRead(token)
      load()
    } catch {
      // best-effort - nothing to show the user beyond the list not updating
    } finally {
      setMarkingAll(false)
    }
  }
}

```

```

const unreadCount = (notifications || []).filter((n) => !n.is_read).length

return (
  <div className="dashboard-shell">
    <div className="dashboard-heading" style={{ display: 'flex', alignItems: 'flex-start', justifyContent: 'space-between', flexWrap: 'wrap', gap: '0.75rem' }}>
      <div>
        <h1>Notifications</h1>
        <p>Assignments, approvals, mentions, status changes, and outdated analyses - all in one place.</p>
      </div>
      <div style={{ display: 'flex', gap: '0.6rem', alignItems: 'center' }}>
        <label style={{ display: 'flex', alignItems: 'center', gap: '0.4rem', fontSize: '0.85rem', color: '#667085' }}>
          <input type="checkbox" checked={unreadOnly} onChange={(e) => setUnreadOnly(e.target.checked)} />
          Unread only
        </label>
        <button
          type="button"
          className="cr-btn cr-btn--secondary cr-btn--small"
          onClick={handleMarkAllRead}
          disabled={markingAll || unreadCount === 0}
        >
          {markingAll ? 'Marking...' : 'Mark all read'}
        </button>
      </div>
    </div>
    {loading && <p className="dashboard-status-text">Loading notifications.</p>}
    {error && <p className="dashboard-status-text dashboard-status-text--error">{error}</p>}}

    {notifications && notifications.length === 0 && (
      <div className="dashboard-empty">
        <h2 className="dashboard-empty-title">
          {unreadOnly ? 'No unread notifications' : 'No notifications yet'}
        </h2>
        <p className="dashboard-empty-subtitle">
          You'll see updates here when you're assigned, tagged for approval, mentioned, or a change request you're involved in changes.
        </p>
      </div>
    )}

    {notifications && notifications.length > 0 && (
      <ul className="notifications-list">
        {notifications.map((n) => (
          <li
            key={n.id}
            className={`notification-row-${n.is_read ? '' : 'notification-row--unread'}`}
            onClick={() => handleOpen(n)}
            role={n.change_request_id != null ? 'button' : undefined}
            tabIndex={n.change_request_id != null ? 0 : undefined}
          >
            <div className="notification-row-main">
              <span className={`badge ${TYPE_CLASS[n.type] || 'badge--gray'}`}>
                {TYPE_LABELS[n.type] || n.type}
              </span>
              <strong>{n.title}</strong>
              {!n.is_read && <span className="notification-dot" aria-hidden="true" />}}
            </div>
            <p className="cr-detail-text">{n.message}</p>
            <div className="notification-row-footer">
              <span className="version-row-meta">{formatDateTime(n.created_at)}</span>
              {!n.is_read && (
                <button
                  type="button"
                  className="cr-btn cr-btn--ghost cr-btn--small"
                  onClick={(e) => handleMarkRead(e, n)}
                >
                  Mark read
                </button>
              )}
            </div>
          </li>
        )}
      </ul>
    )}
  </div>
)

```

```

import { useState } from 'react'
import AuthLayout from '../components/AuthLayout'
import { ApiError } from '../api/client'
import { useAuth } from '../context/AuthContext'

export default function RegisterPage({ onSwitchToLogin }) {
  const { register } = useAuth()
  const [name, setName] = useState('')
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [confirmPassword, setConfirmPassword] = useState('')
  const [error, setError] = useState(null)
  const [submitting, setSubmitting] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    setError(null)

    if (password !== confirmPassword) {
      setError('Passwords do not match.')
      return
    }
    if (password.length < 8) {
      setError('Password must be at least 8 characters long.')
      return
    }

    setSubmitting(true)
    try {
      await register(name, email, password, confirmPassword)
    } catch (err) {
      setError(err instanceof ApiError ? err.message : 'Something went wrong. Please try again.')
    } finally {
      setSubmitting(false)
    }
  }

  return (
    <AuthLayout>
      <h1 className="auth-title">Create your account</h1>
      <p className="auth-subtitle">Start turning change Requests into engineering decisions.</p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label className="auth-field">
          <span>Name</span>
          <input
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)}
            required
            autoComplete="name"
            placeholder="Ada Lovelace"
          />
        </label>

        <label className="auth-field">
          <span>Email</span>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            autoComplete="email"
            placeholder="you@company.com"
          />
        </label>

        <label className="auth-field">
          <span>Password</span>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            autoComplete="new-password"
            placeholder="At least 8 characters"
            minLength={8}
          />
        </label>

        <label className="auth-field">
          <span>Confirm password</span>
          <input
            type="password"
            value={confirmPassword}
            onChange={(e) => setConfirmPassword(e.target.value)}
            required
            autoComplete="new-password"
            placeholder="Re-enter your password"
            minLength={8}
          />
        </label>

        {error && <p className="auth-error">{error}</p>}

        <button type="submit" className="auth-submit" disabled={submitting}>
          {submitting ? 'Creating account...' : 'Create account'}
        </button>
      </form>

      <p className="auth-switch">
        Already have an account?{' '}
        <button type="button" className="auth-link-button" onClick={onSwitchToLogin}>
          Sign in
        </button>
      </p>
    </AuthLayout>
  )
}

```

```

import { useEffect, useState } from 'react'
import { downloadReportById, getReport } from '../api/reports'
import { useAuth } from '../context/AuthContext'
import { API_BASE_URL } from '../api/client'
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'
import '../styles/workflow.css'

const RISK_CLASS = { low: 'badge--green', medium: 'badge--amber', high: 'badge--orange', critical: 'badge--red' }

function formatDate(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

/** Module 24: a report's view page - a metadata card (spec section 12: id,
 * CR, title, version, analysis version, status, risk, generated by/at) plus
 * the actual PDF embedded right below it, fetched once as a blob (an
 * <iframe src="..."> can't carry the Authorization header this endpoint
 * requires) and shown in the browser's own PDF viewer - the same PDF
 * "Download" produces, never a second, separately-built rendering of the
 * same report content (confirmed with the project owner before building
 * this page). */
export default function ReportDetailPage({ reportId, onBack, onOpenChangeRequest }) {
  const { token } = useAuth()
  const [report, setReport] = useState(null)
  const [error, setError] = useState(null)
  const [pdfUrl, setPdfUrl] = useState(null)
  const [pdfError, setPdfError] = useState(null)
  const [downloading, setDownloading] = useState(false)

  useEffect(() => {
    let cancelled = false
    setError(null)
    getReport(reportId, token)
      .then((data) => {
        if (!cancelled) setReport(data)
      })
      .catch((err) => {
        if (!cancelled) setError(err.message || 'Could not load this report.')
      })
    return () => {
      cancelled = true
    }
  }, [reportId, token])

  useEffect(() => {
    let cancelled = false
    let objectUrl = null
    setPdfError(null)
    setPdfUrl(null)

    fetch(`${API_BASE_URL}/api/reports/${reportId}/download`, {
      headers: token ? { Authorization: `Bearer ${token}` } : {},
    })
      .then(async (response) => {
        if (!response.ok) {
          let detail
          try {
            detail = (await response.json())?.detail
          } catch {
            detail = null
          }
          const message =
            typeof detail === 'string' ? detail : detail?.message || 'Could not load the PDF preview.'
          throw new Error(message)
        }
        return response.blob()
      })
      .then((blob) => {
        if (cancelled) return
        objectUrl = URL.createObjectURL(blob)
        setPdfUrl(objectUrl)
      })
      .catch((err) => {
        if (!cancelled) setPdfError(err.message || 'Could not load the PDF preview.')
      })

    return () => {
      cancelled = true
      if (objectUrl) URL.revokeObjectURL(objectUrl)
    }
  }, [reportId, token])

  async function handleDownload() {
    setDownloading(true)
    setPdfError(null)
    try {
      await downloadReportById(
        reportId,
        token,
        report ? `${report.change_request_code}-v${report.version}-report.pdf` : undefined
      )
    } catch (err) {
      setPdfError(err.message || 'Could not download this report.')
    } finally {
      setDownloading(false)
    }
  }

  return (
    <div className="cr-detail-shell">
      <nav className="breadcrumbs" aria-label="Breadcrumb">
        <button type="button" className="breadcrumb-link" onClick={onBack}>
          Reports
        </button>
        <span className="breadcrumb-separator">/</span>
        <span className="breadcrumb-current">{report ? `${report.change_request_code} · v${report.version}` : `Report #${reportId}`}</span>
      </nav>

      {error && <p className="cr-list-status-text cr-list-status-text--error">{error}</p>}

      {report && (

```

```

<div className="cr-detail-heading">
  <div>
    <h1>
      {report.change_request_code} {report.version}
    </h1>
    <div className="cr-detail-badges">
      <span className="badge badge--gray">{report.cr_status_label}</span>
      {report.risk_bucket ? (
        <span className="badge ${RISK_CLASS[report.risk_bucket.toLowerCase()] || 'badge--gray'}">
          {report.risk_bucket} risk
        </span>
      ) : (
        <span className="badge badge--gray">Not analyzed</span>
      )}
      {report.is_historical_pull && <span className="badge badge--indigo">Historical</span>
      {!report.is_current_version && (
        <span className="badge badge--amber" title="A newer version of this change request now exists.">
          Outdated
        </span>
      )}
    </div>
  </div>
  <div>
    <div className="approval-actions-cell">
      <button
        type="button"
        className="cr-btn cr-btn--secondary"
        onClick={() => onOpenChangeRequest?.(report.change_request_id)}
      >
        Open Change Request
      </button>
      <button type="button" className="cr-btn cr-btn--primary" disabled={downloading} onClick={handleDownload}>
        {downloading ? 'Working...' : 'Download PDF'}
      </button>
    </div>
  </div>

  <section className="cr-detail-section">
    <h2 className="cr-section-title">Report Details</h2>
    <dl className="cr-detail-grid">
      <dt>Report ID</dt>
      <dd>{report.report_id}</dd>
      <dt>Change Request</dt>
      <dd>
        {report.change_request_code} {report.change_request_title}
      </dd>
      <dt>Change Request Version</dt>
      <dd>{report.version}</dd>
      <dt>Analysis Version</dt>
      <dd>{report.has_analysis ? report.analysis_version : 'Analysis unavailable for this version.'}</dd>
      <dt>Current Status</dt>
      <dd>{report.cr_status_label}</dd>
      <dt>Risk</dt>
      <dd>{report.risk_bucket || 'Not analyzed'}</dd>
      <dt>Generated By</dt>
      <dd>{report.generated_by_name}</dd>
      <dt>Generated At</dt>
      <dd>{formatDateTime(report.generated_at)}</dd>
      <dd>{report.pull_count > 1 && (
        <dt>Times Downloaded</dt>
        <dd>{report.pull_count}</dd>
      )}
    </dd>
  </dl>
</section>

  <section className="cr-detail-section">
    <h2 className="cr-section-title">Report Content</h2>
    {pdfError && <p className="cr-list-status-text cr-list-status-text--error">{pdfError}</p>}
    {pdfUrl && <p className="cr-list-status-text">Loading report</p>}
    {pdfUrl && (
      <iframe
        title={`Report ${report.report_id}`}
        src={pdfUrl}
        style={{ width: '100%', height: '80vh', border: '1px solid #d0d5dd', borderRadius: '8px' }}
      />
    )}
  </section>
</div>
)
}

```



```

import { useEffect, useState } from 'react'
import { downloadReportById, getReportStats, listReports } from '../api/reports'
import { listUsers } from '../api/users'
import { useAuth } from '../context/AuthContext'
import MetricCard from '../components/MetricCard'
import ReportDetailPage from '../ReportDetailPage'
import '../styles/dashboard.css'
import '../styles/change-request-form.css'
import '../styles/change-requests-list.css'
import '../styles/workflow.css'
import '../styles/my-work.css'

const STATUS_OPTIONS = [
  { value: '', label: 'All statuses' },
  { value: 'draft', label: 'Draft' },
  { value: 'submitted', label: 'Submitted' },
  { value: 'pending analysis', label: 'Pending Analysis' },
  { value: 'analyzed', label: 'Analyzed' },
  { value: 'in review', label: 'Under Review' },
  { value: 'changes requested', label: 'Changes Requested' },
  { value: 'approval required', label: 'Approval Required' },
  { value: 'approved', label: 'Approved' },
  { value: 'rejected', label: 'Rejected' },
  { value: 'implementation planned', label: 'Implementation Planned' },
  { value: 'in progress', label: 'In Progress' },
  { value: 'implemented', label: 'Implemented' },
  { value: 'validated', label: 'Validated' },
  { value: 'closed', label: 'Closed' },
  { value: 'cancelled', label: 'Cancelled' },
]

const RISK_OPTIONS = [
  { value: '', label: 'All risk levels' },
  { value: 'low', label: 'Low' },
  { value: 'medium', label: 'Medium' },
  { value: 'high', label: 'High' },
  { value: 'critical', label: 'Critical' },
]

const RISK_CLASS = { low: 'badge--green', medium: 'badge--amber', high: 'badge--orange', critical: 'badge--red' }

// Mirrors app/services/workflow rules.py::STATUS_LABELS' own status set -
// kept in sync manually, same convention as every other label/class map
// already in this codebase (e.g. components/ChangeRequestDetail.jsx's own
// STATUS_CLASS).
const STATUS_CLASS = {
  draft: 'badge--gray',
  submitted: 'badge--indigo',
  pending analysis: 'badge--indigo',
  analyzed: 'badge--blue',
  in review: 'badge--indigo',
  changes requested: 'badge--amber',
  approval required: 'badge--amber',
  approved: 'badge--green',
  rejected: 'badge--red',
  implementation planned: 'badge--teal',
  in progress: 'badge--teal',
  implemented: 'badge--teal',
  validated: 'badge--green',
  closed: 'badge--gray',
  cancelled: 'badge--gray',
}

function formatDateTime(isoString) {
  if (!isoString) return '-'
  try {
    return new Date(isoString).toLocaleString(undefined, {
      year: 'numeric',
      month: 'short',
      day: 'numeric',
      hour: 'numeric',
      minute: '2-digit',
    })
  } catch {
    return isoString
  }
}

const emptyFilters = { search: '', status: '', risk: '', version: '', generatedBy: '', dateFrom: '', dateTo: '' }

/** Module 24: the Reports section - statistics, search/filter, and the
 * report list, all reading from GET /api/reports* (see backend/app/
 * services/report_registry.py - reports are a computed view over the
 * existing audit trail, not their own table).
 *
 * There is deliberately no "Generate Report" action anywhere on this page:
 * a report is never a separate, manual step - the moment an analysis
 * completes for a change request (Analyze or Re-analyze, on the Analysis
 * Dashboard), it automatically shows up here (confirmed with the project
 * owner). "View" opens the report's PDF via ReportDetailPage's metadata
 * card + embedded PDF; "Download" saves it. */
export default function ReportsPage({ onOpenChangeRequest, onNavigate }) {
  const { token } = useAuth()

  const [filters, setFilters] = useState(emptyFilters)
  const [searchInput, setSearchInput] = useState('')
  const [stats, setStats] = useState(null)
  const [data, setData] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [refreshToken, setRefreshToken] = useState(0)
  const [users, setUsers] = useState([])
  const [selectedReportId, setSelectedReportId] = useState(null)

  const [rowBusyId, setRowBusyId] = useState(null)
  const [rowError, setRowError] = useState(null)

  useEffect(() => {
    if (!token) return
    listUsers(token)
      .then(setUsers)
      .catch(() => {})
  }, [token])

  useEffect(() => {
    const handle = setTimeout(() => {
      setFilters((prev) => ({ ...prev, search: searchInput.trim() }))
    }, 350)
    return () => clearTimeout(handle)
  }, [searchInput])

  useEffect(() => {
    if (!token) return
    getReportStats(token)

```

```

        .then(setStats)
        .catch(() => {})
    }, [token, refreshToken])

    useEffect(() => {
        if (!token) return
        let cancelled = false
        setLoading(true)
        setError(null)
        listReports(token, filters)
        .then((response) => {
            if (!cancelled) setData(response)
        })
        .catch((err) => {
            if (!cancelled) setError(err.message || 'Could not load reports.')
        })
        .finally(() => {
            if (!cancelled) setLoading(false)
        })
        return () => {
            cancelled = true
        }
    }, [token, filters, refreshToken])

    function updateFilter(key, value) {
        setFilters((prev) => ({ ...prev, [key]: value }))
    }

    function clearFilters() {
        setSearchInput('')
        setFilters(emptyFilters)
    }

    const filtersActive = Boolean(
        filters.search || filters.status || filters.risk || filters.version || filters.generatedBy || filters.dateFrom || filters.dateTo
    )

    async function handleDownload(row) {
        setRowBusyId(row.report_id)
        setRowError(null)
        try {
            await downloadReportById(
                row.report_id,
                token,
                `${row.change_request_code}-${row.version}-report.pdf`
            )
        } catch (err) {
            setRowError(err.message || 'Could not download this report.')
        } finally {
            setRowBusyId(null)
        }
    }

    if (selectedReportId !== null) {
        return (
            <ReportDetailPage
                reportId={selectedReportId}
                onBack={() => {
                    setSelectedReportId(null)
                    setRefreshToken((k) => k + 1)
                }}
                onOpenChangeRequest={onOpenChangeRequest}
            />
        )
    }

    const total = data?.total ?? 0
    const items = data?.items ?? []

    return (
        <div className="cr-list-shell">
            <div className="cr-list-heading">
                <div>
                    <h1>Reports</h1>
                    <p>Every report generated from a Change Request analysis, in one place.</p>
                </div>
            </div>
            {stats && (
                <div className="metric-grid">
                    <MetricCard label="Total Reports" value={stats.total_reports} />
                    <MetricCard label="CRs With Reports" value={stats.change_requests_with_reports} />
                    <MetricCard label="Generated Last 7 Days" value={stats.recent_reports} />
                    <MetricCard
                        label="Outdated (Newer CR Version Exists)"
                        value={stats.outdated_reports}
                        tone={stats.outdated_reports > 0 ? 'danger' : 'default'}
                    />
                </div>
            )}
        </div>

        <div className="cr-toolbar">
            <input
                type="text"
                className="cr-search-input"
                placeholder="Search by CR id, CR code, or title..."
                value={searchInput}
                onChange={(e) => setSearchInput(e.target.value)}
            />
            <select
                className="cr-filter-select"
                value={filters.status}
                onChange={(e) => updateFilter('status', e.target.value)}
                aria-label="Filter by status"
            >
                {STATUS_OPTIONS.map((option) => (
                    <option key={option.value} value={option.value}>
                        {option.label}
                    </option>
                ))}
            </select>
            <select
                className="cr-filter-select"
                value={filters.risk}
                onChange={(e) => updateFilter('risk', e.target.value)}
                aria-label="Filter by risk"
            >
                {RISK_OPTIONS.map((option) => (
                    <option key={option.value} value={option.value}>
                        {option.label}
                    </option>
                ))}
            </select>
            <input
                type="number"
                min="1"
                className="cr-filter-select"
                style={{ width: '7rem' }}
                placeholder="Version"
                value={filters.version}
                onChange={(e) => updateFilter('version', e.target.value)}
            />
        </div>
    )

```



```
} ) </div>
```

```

/* Module 21: Administration & Configuration page (AdminPage.jsx). This page
already reuses .ad-shell/.ad-tabs/.ad-tab/.ad-tab-content (analysis-
dashboard.css), .cr-section/.cr-field/.cr-field-row/.cr-form-error/
.cr-form-actions (change-request-form.css), .cr-btn* (workflow.css /
change-request-form.css), .requests-table and .badge* (dashboard.css) -
everything below is specific to the admin tabs themselves, styled to
match the same palette those files already use (grays #101828 / #344054
/ #475467 / #667085 / #98a2b3 / #eaeef0 / #f2f4f7 / #f9fafb, indigo
accent #4338ca, green #ecfdf3 / #067647, red #fef3f2 / #b42318). */

/* Users tab: "Add User" row above the table */

.admin-tab-toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 0.85rem;
}

.admin-tab-hint {
  margin: 0 0 0.75rem;
  font-size: 0.82rem;
  color: #667085;
  line-height: 1.5;
}

.admin-tab-toolbar .admin-tab-hint {
  margin: 0;
}

/* Inline create/add forms (new user, new approval rule) - a .cr-section
that sits above the table it feeds, slightly tighter than a full-page
form section. */

.admin-inline-form {
  margin-bottom: 1rem;
  background: #f9fafb;
}

.admin-row-error {
  margin: 0.35rem 0 0;
  font-size: 0.78rem;
  color: #b42318;
}

/* Permissions matrix */

.admin-matrix-scroll {
  overflow-x: auto;
  margin-bottom: 0.5rem;
}

.admin-matrix {
  min-width: 720px;
}

.admin-matrix th,
.admin-matrix td {
  text-align: center;
  white-space: nowrap;
}

.admin-matrix th:first-child,
.admin-matrix td:first-child {
  text-align: left;
  white-space: nowrap;
}

.admin-matrix input[type='checkbox'] {
  width: 16px;
  height: 16px;
  cursor: pointer;
}

.admin-matrix-locked-row {
  background: #f9fafb;
}

.admin-matrix-locked-row td {
  color: #98a2b3;
}

.admin-save-message {
  font-size: 0.82rem;
  font-weight: 600;
  color: #067647;
  margin-right: auto;
}

/* Chips - used for approval-rule required-types selection and the
editable cr_categories list */

.admin-chip-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
  margin: 0.3rem 0 0.6rem;
}

.admin-chip {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  font-family: inherit;
  font-size: 0.78rem;
  font-weight: 600;
  color: #475467;
  background: #f2f4f7;
  border: 1px solid #eaeef0;
  border-radius: 999px;
  padding: 0.3rem 0.75rem;
  cursor: pointer;
  transition:
    background 0.15s ease,
    color 0.15s ease,
    border-color 0.15s ease;
}

.admin-chip:hover {
  background: #eaeef0;
}

.admin-chip--active {

```

```

background: #eef1fd;
border-color: #c7d2fe;
color: #4338ca;
}

.admin-chip--small {
font-size: 0.72rem;
padding: 0.2rem 0.6rem;
}

.admin-chip--removable {
cursor: default;
background: #fff;
border-color: #d8d5dd;
color: #344054;
}

.admin-chip--removable button {
font-family: inherit;
font-size: 0.95rem;
line-height: 1;
color: #98a2b3;
background: none;
border: none;
cursor: pointer;
padding: 0;
}

.admin-chip--removable button:hover {
color: #b42318;
}

.admin-chip--removable button:disabled {
cursor: not-allowed;
opacity: 0.5;
}

.admin-inline-add {
display: flex;
align-items: center;
gap: 0.5rem;
}

.admin-inline-add input[type='text'] {
font-family: inherit;
font-size: 0.85rem;
color: #101828;
border: 1px solid #d8d5dd;
border-radius: 8px;
padding: 0.5rem 0.65rem;
min-width: 220px;
}

.admin-inline-add input[type='text']:focus {
outline: none;
border-color: #4338ca;
}

/* Read-only system-settings info panels (AI provider / repository /
knowledge base) */

.admin-info-list {
display: grid;
grid-template-columns: max-content 1fr;
gap: 0.4rem 1rem;
margin: 0.5rem 0 0;
}

.admin-info-list dt {
font-size: 0.75rem;
font-weight: 600;
color: #98a2b3;
text-transform: uppercase;
letter-spacing: 0.03em;
align-self: center;
}

.admin-info-list dd {
margin: 0;
font-size: 0.85rem;
color: #344054;
align-self: center;
}

.admin-info-path {
font-family: 'SFMono-Regular', Consolas, 'Liberation Mono', Menlo, monospace;
font-size: 0.78rem;
color: #475467;
word-break: break-all;
}

/* Audit log pagination */

.admin-pagination {
display: flex;
align-items: center;
justify-content: flex-end;
gap: 0.6rem;
margin-top: 0.85rem;
}

@media (max-width: 640px) {
.admin-tab-toolbar {
flex-direction: column;
align-items: flex-start;
}

.admin-info-list {
grid-template-columns: 1fr;
}

.admin-info-list dt {
margin-top: 0.4rem;
}
}

```

```

/* Module 7: AI Analysis Dashboard - the dedicated results page reached from
a change request's "View Full AI Analysis" button. Reuses .cr-detail-
section / .cr-analysis-list / .badge / .chart-card / .bar-* classes from
the existing stylesheets rather than redefining them; everything below
is specific to this page (header, summary cards, gauges, tabs). */

.ad-shell {
  max-width: 1100px;
  margin: 0 auto;
  padding: 2rem 1.5rem 4rem;
  box-sizing: border-box;
}

/* Header */

.ad-header {
  margin-bottom: 1.5rem;
}

.ad-header-code {
  display: block;
  font-size: 0.8rem;
  font-weight: 700;
  letter-spacing: 0.04em;
  color: #4338ca;
  margin-bottom: 0.25rem;
}

.ad-header h1 {
  margin: 0;
  font-size: 1.5rem;
  font-weight: 700;
  color: #101828;
}

/* Module 14 Phase 1: CR version / analysis version / status, shown as a
small badge row under the title. */
.ad-header-meta {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  flex-wrap: wrap;
  margin-top: 0.5rem;
}

/* Module 14 Phase 1: the recommended-approval-types badge row on the
Overview tab's Recommendation section. */
.ad-recommended-approvals {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  flex-wrap: wrap;
}

.cr-detail-text-label {
  font-size: 0.85rem;
  font-weight: 600;
  color: #475467;
}

.ad-header--row {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
}

.ad-header-actions {
  display: flex;
  flex-direction: column;
  align-items: flex-end;
  gap: 0.4rem;
  flex: none;
}

.ad-header-buttons {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  flex-wrap: wrap;
  justify-content: flex-end;
}

.ad-header-error {
  margin: 0;
  text-align: right;
  max-width: 260px;
}

/* Module 20 (Version-Aware Reports): the "current version has not been
analyzed" warning and the "report for a specific version" picker both
live in the same narrow, right-aligned .ad-header-actions column as the
buttons above them - .outdated-banner's own side-by-side layout (built
for a full-width banner elsewhere on this page) is overridden here to
stack instead, and given room to actually read left-aligned. */

.ad-report-mismatch {
  flex-direction: column;
  align-items: flex-start;
  width: 100%;
  max-width: 380px;
  text-align: left;
  margin: 0;
}

.ad-report-mismatch-actions {
  display: flex;
  gap: 0.5rem;
  flex-wrap: wrap;
}

.ad-report-version-picker {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  flex-wrap: wrap;
  justify-content: flex-end;
  font-size: 0.82rem;
  color: #475467;
}

.ad-report-version-picker label {
  font-weight: 600;
}

```

```

}

.ad-report-version-picker select {
  font-family: inherit;
  font-size: 0.82rem;
  color: #101828;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  padding: 0.35rem 0.5rem;
  background: #fff;
}

/* Summary cards */

.ad-summary-grid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 1rem;
  margin-bottom: 1.5rem;
}

.ad-summary-card {
  background: #fff;
  border: 1px solid #eaecef;
  border-radius: 12px;
  padding: 1.1rem 1.1rem;
  display: flex;
  flex-direction: column;
  gap: 0.6rem;
  min-width: 0;
}

.ad-summary-label {
  font-size: 0.75rem;
  font-weight: 600;
  color: #667085;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.ad-summary-body {
  display: flex;
  align-items: center;
  gap: 0.7rem;
  flex: 1;
}

.ad-summary-body--center {
  align-items: center;
}

.ad-summary-sub {
  font-size: 0.78rem;
  color: #98a2b3;
}

.ad-summary-text {
  font-size: 0.9rem;
  font-weight: 600;
  color: #101828;
  line-height: 1.4;
}

.ad-summary-badge {
  font-size: 0.85rem;
  padding: 0.35rem 0.75rem;
}

/* Module 14 Phase 3: stacks a summary card's main value above its
   confidence badge, rather than fighting .ad-summary-body's row layout
   at narrow card widths. */
.ad-summary-stack {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.4rem;
}

.ad-complexity-row {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  flex-wrap: wrap;
}

/* Module 14 Phase 4: the 7-category Impact Analysis card grid on the
   Impact tab. */
.ad-impact-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.75rem;
}

.ad-impact-card {
  background: #f9fafb;
  border-radius: 8px;
  margin: 0;
  padding: 1rem 1.1rem;
}

.ad-impact-card-title {
  margin: 0 0 0.5rem;
  font-size: 0.85rem;
  font-weight: 700;
  color: #101828;
  text-transform: uppercase;
  letter-spacing: 0.02em;
}

.ad-impact-card-badges {
  display: flex;
  align-items: center;
  gap: 0.4rem;
  flex-wrap: wrap;
}

@media (max-width: 640px) {
  .ad-impact-grid {
    grid-template-columns: 1fr;
  }
}

/* Gauges - pure CSS donut via conic-gradient, no charting library */

.ad-gauge {
  --gauge-pct: 0%;
  --gauge-color: #4338ca;
  width: 56px;
  height: 56px;
  border-radius: 50%;
  flex: none;
}

```



```

background: conic-gradient(var(--gauge-color) var(--gauge-pct), #f2f4f7 0);
display: flex;
align-items: center;
justify-content: center;
position: relative;
}

.ad-gauge-inner {
position: absolute;
inset: 7px;
border-radius: 50%;
background: #fff;
display: flex;
align-items: center;
justify-content: center;
}

.ad-gauge-value {
position: relative;
font-size: 0.78rem;
font-weight: 700;
color: #101828;
}

/* Recommendation card */

.ad-recommendation-card {
border-width: 1px;
border-style: solid;
}

.ad-recommendation-card--success {
background: #ecfdf3;
border-color: #abefc6;
}

.ad-recommendation-card--warning {
background: #fff4e6;
border-color: #fedf89;
}

.ad-recommendation-card--danger {
background: #ffe4e6;
border-color: #fecdc6;
}

.ad-recommendation-card--neutral {
background: #f9fafb;
}

.ad-recommendation-value {
font-size: 0.82rem;
font-weight: 700;
color: #101828;
line-height: 1.4;
}

.ad-recommendation-inline {
font-size: 0.95rem;
margin: 0 0 0.5rem;
}

/* Tabs */

.ad-tabs {
display: flex;
gap: 0.35rem;
overflow-x: auto;
padding-bottom: 0.5rem;
margin-bottom: 1rem;
border-bottom: 1px solid #eaeaf0;
}

.ad-tab {
font-family: inherit;
font-size: 0.85rem;
font-weight: 600;
color: #667085;
background: none;
border: none;
border-radius: 999px;
padding: 0.5rem 0.95rem;
cursor: pointer;
white-space: nowrap;
display: inline-flex;
align-items: center;
gap: 0.4rem;
transition:
background 0.15s ease,
color 0.15s ease;
}

.ad-tab:hover {
background: #f9fafb;
color: #344054;
}

.ad-tab--active {
background: #eef1fd;
color: #4338ca;
}

.ad-tab-count {
display: inline-flex;
align-items: center;
justify-content: center;
min-width: 18px;
height: 18px;
padding: 0 0.3rem;
border-radius: 999px;
background: #b42318;
color: #fff;
font-size: 0.7rem;
font-weight: 700;
}

.ad-tab-content {
display: flex;
flex-direction: column;
gap: 1rem;
}

/* Shared small helpers */

.ad-empty {
margin: 0;
font-size: 0.88rem;
color: #98a2b3;
padding: 0.5rem 0;
}

```

```

.ad-mt {
  margin-top: 0.75rem;
}

.ad-inline-badge {
  margin-right: 0.4rem;
  vertical-align: middle;
}

/* Module 13 Phase 2/5: added/removed requirement & affected-component
 * lines in the analysis History/Compare tab - same red/green language as
 * .compare-row--added/.compare-row--removed (workflow.css), just applied
 * to plain text lines rather than table cells. */
.diff-added {
  color: #067647;
}

.diff-removed {
  color: #b42318;
  text-decoration: line-through;
}

/* Confidence bar (Impact tab) */

.ad-confidence-row {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  margin-top: 0.5rem;
}

.ad-confidence-track {
  flex: 1;
  max-width: 220px;
}

.ad-confidence-value {
  font-size: 0.78rem;
  color: #667085;
  white-space: nowrap;
}

/* Missing Information banner */

.ad-missing-banner {
  display: flex;
  align-items: center;
  gap: 0.9rem;
  padding: 1.1rem 1.4rem;
  border-radius: 12px;
  border: 1px solid;
  font-size: 0.95rem;
  font-weight: 600;
}

.ad-missing-banner--alert {
  background: #fef3f2;
  border-color: #fecfca;
  color: #b42318;
}

.ad-missing-banner--clear {
  background: #ecfdf3;
  border-color: #abefc6;
  color: #067647;
}

.ad-missing-count {
  font-size: 1.8rem;
  font-weight: 800;
  line-height: 1;
}

/* Module 9: Risks tab */

.rk-overall {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.rk-overall-text {
  display: flex;
  flex-direction: column;
  gap: 0.2rem;
}

.rk-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.rk-card {
  border: 1px solid #eaecef;
  border-radius: 10px;
  padding: 0.9rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  background: #fff;
}

.rk-card-header {
  display: flex;
  align-items: center;
  gap: 0.6rem;
}

.rk-card-category {
  font-size: 0.8rem;
  font-weight: 600;
  color: #475467;
}

.rk-card-metrics {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.75rem;
  margin: 0.2rem 0;
}

.rk-metric {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.rk-metric-label {
  font-size: 0.7rem;
  font-weight: 600;
}

```

```

    color: #98a2b3;
    text-transform: uppercase;
    letter-spacing: 0.03em;
}

.rk-metric-row {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.rk-metric-value {
    font-size: 0.78rem;
    color: #667085;
    white-space: nowrap;
}

.rk-card-label {
    margin: 0.2rem 0 0;
    font-size: 0.7rem;
    font-weight: 700;
    color: #98a2b3;
    text-transform: uppercase;
    letter-spacing: 0.03em;
}

@media (max-width: 640px) {
    .rk-card-metrics {
        grid-template-columns: 1fr;
    }
}

/* Module 9: Security tab */

.sec-level-row {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    flex-wrap: wrap;
}

.sec-level-badge {
    font-size: 0.85rem;
    padding: 0.35rem 0.75rem;
}

.sec-checklist-progress {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    margin: 0.5rem 0 0 0.85rem;
}

.sec-checklist-progress-text {
    font-size: 0.78rem;
    color: #667085;
    white-space: nowrap;
}

.sec-checklist {
    list-style: none;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
}

.sec-checklist-item label {
    display: flex;
    align-items: flex-start;
    gap: 0.6rem;
    cursor: pointer;
}

.sec-checklist-item input[type='checkbox'] {
    margin-top: 0.2rem;
    flex: none;
    width: 15px;
    height: 15px;
    cursor: pointer;
}

.sec-checklist-tag {
    flex: none;
    font-size: 0.65rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.03em;
    padding: 0.1rem 0.45rem;
    border-radius: 999px;
    margin-top: 0.05rem;
}

.sec-checklist-tag--concern {
    background: #fef3f2;
    color: #b42318;
}

.sec-checklist-tag--mitigation {
    background: #ecfdf3;
    color: #067647;
}

.sec-checklist-text {
    font-size: 0.85rem;
    color: #344054;
    line-height: 1.4;
}

.sec-checklist-text--done {
    color: #98a2b3;
    text-decoration: line-through;
}

/* Effort tab */

.ad-effort-note {
    margin: 0 0 1rem;
    font-size: 0.82rem;
    color: #98a2b3;
    font-style: italic;
}

.ad-effort-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 0.75rem;
}

.ad-effort-row {

```

```

background: #f9fafb;
border-radius: 8px;
padding: 0.85rem 1rem;
display: flex;
flex-direction: column;
gap: 0.3rem;
}

.ad-effort-label {
font-size: 0.75rem;
font-weight: 600;
color: #667085;
text-transform: uppercase;
letter-spacing: 0.03em;
}

.ad-effort-value {
font-size: 0.92rem;
font-weight: 600;
color: #101828;
}

/* Module 10: Test Cases tab filters */

.tc-filters {
display: flex;
align-items: center;
gap: 0.6rem;
flex-wrap: wrap;
margin-bottom: 1rem;
}

.tc-filter-count {
font-size: 0.8rem;
color: #98a2b3;
margin-left: auto;
}

/* Module 10: Implementation Plan timeline */

.ip-timeline {
list-style: none;
margin: 0;
padding: 0;
display: flex;
flex-direction: column;
}

.ip-step {
display: flex;
gap: 1rem;
}

.ip-step-marker {
flex: none;
display: flex;
flex-direction: column;
align-items: center;
}

.ip-step-number {
width: 28px;
height: 28px;
border-radius: 50%;
background: #4338ca;
color: #fff;
font-size: 0.8rem;
font-weight: 700;
display: flex;
align-items: center;
justify-content: center;
flex: none;
}

.ip-step-line {
flex: 1;
width: 2px;
min-height: 24px;
background: #e4e7ec;
margin: 0.25rem 0;
}

.ip-step-card {
flex: 1;
min-width: 0;
border: 1px solid #eaecef;
border-radius: 10px;
padding: 0.85rem 1rem;
margin-bottom: 1rem;
display: flex;
flex-direction: column;
gap: 0.4rem;
background: #fff;
}

.ip-step-header {
display: flex;
align-items: center;
gap: 0.6rem;
flex-wrap: wrap;
}

.ip-step-task {
font-size: 0.92rem;
font-weight: 700;
color: #101828;
}

.ip-step-meta {
display: grid;
grid-template-columns: auto 1fr;
gap: 0.15rem 0.6rem;
margin: 0.2rem 0 0;
}

.ip-step-meta dt {
font-size: 0.72rem;
font-weight: 600;
color: #98a2b3;
text-transform: uppercase;
letter-spacing: 0.03em;
}

.ip-step-meta dd {
margin: 0;
font-size: 0.85rem;
color: #344054;
}

@media (max-width: 1100px) {
.ad-summary-grid {

```

```

        grid-template-columns: repeat(3, minmax(0, 1fr));
    }
}

@media (max-width: 640px) {
    .ad-shell {
        padding: 1.25rem 1rem 3rem;
    }

    .ad-summary-grid {
        grid-template-columns: repeat(2, minmax(0, 1fr));
    }

    .ad-effort-grid {
        grid-template-columns: 1fr;
    }

    .tc-filter-count {
        margin-left: 0;
        width: 100%;
    }

    .ip-step-meta {
        grid-template-columns: 1fr;
    }

    .tl-linked {
        grid-template-columns: 1fr;
    }
}

/* Module 17 Phase 2: Test Cases tab - ordered steps list */

.tc-steps {
    margin: 0.2rem 0 0;
    padding-left: 1.1rem;
    font-size: 0.85rem;
    color: #344054;
    display: flex;
    flex-direction: column;
    gap: 0.15rem;
}

/* Module 17 Phase 4: Traceability tab */

.tl-list {
    list-style: none;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    gap: 0.9rem;
}

.tl-row {
    border: 1px solid #eaecef;
    border-radius: 10px;
    padding: 0.85rem 1rem;
    background: #fff;
}

.tl-requirement {
    display: flex;
    align-items: baseline;
    gap: 0.6rem;
    margin-bottom: 0.6rem;
}

.tl-linked {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1rem;
    padding-top: 0.6rem;
    border-top: 1px dashed #eaecef;
}

.tl-column-title {
    margin: 0 0 0.3rem;
    font-size: 0.72rem;
    font-weight: 600;
    color: #98a2b3;
    text-transform: uppercase;
    letter-spacing: 0.03em;
}

.tl-column-list {
    margin: 0;
    padding-left: 1.1rem;
    font-size: 0.85rem;
    color: #344054;
    display: flex;
    flex-direction: column;
    gap: 0.2rem;
}

.tl-untraced {
    margin-top: 1.25rem;
    border-top: 1px solid #eaecef;
    padding-top: 1rem;
}

.tl-untraced-title {
    margin: 0 0 0.2rem;
    font-size: 0.92rem;
    font-weight: 700;
    color: #101828;
}

```

```

/* Module 19 (Engineering Change Analytics) - styles specific to the new
 * Analytics page. Metric cards, chart cards and bar lists reuse
 * dashboard.css's own classes (imported alongside this file) rather than
 * redefining them here. */

.analytics-shell {
  display: flex;
  flex-direction: column;
}

.analytics-section-title {
  margin: 1.75rem 0 0.9rem;
  font-size: 1.05rem;
  font-weight: 700;
  color: #101828;
}

.analytics-section-title:first-of-type {
  margin-top: 0.5rem;
}

.analytics-note {
  margin: -0.4rem 0 0.9rem;
  font-size: 0.82rem;
  color: #667085;
}

/* Filter bar */

.analytics-filter-bar {
  display: flex;
  flex-wrap: wrap;
  align-items: flex-end;
  gap: 0.9rem;
  background: #fff;
  border: 1px solid #eaecef;
  border-radius: 12px;
  padding: 1rem 1.2rem;
  margin-bottom: 1.5rem;
}

.analytics-filter {
  display: flex;
  flex-direction: column;
  gap: 0.3rem;
  min-width: 130px;
}

.analytics-filter label {
  font-size: 0.72rem;
  font-weight: 600;
  color: #667085;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.analytics-filter select,
.analytics-filter input[type='date'] {
  font-family: inherit;
  font-size: 0.85rem;
  color: #101828;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  padding: 0.45rem 0.6rem;
  background: #fff;
}

.analytics-filter select:focus,
.analytics-filter input[type='date']:focus {
  outline: none;
  border-color: #4338ca;
  box-shadow: 0 0 3px rgba(67, 56, 202, 0.12);
}

.analytics-clear-filters {
  font-family: inherit;
  font-size: 0.82rem;
  font-weight: 600;
  color: #475467;
  background: #f9fafb;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  padding: 0.5rem 0.85rem;
  cursor: pointer;
  height: fit-content;
}

.analytics-clear-filters:hover {
  background: #f2f4f7;
}

/* Metric grid variants - dashboard.css's own .metric-grid defaults to 5
 * columns; these override the column count for sections with a different
 * number of cards, without touching the shared card styling itself. */

.analytics-metric-grid-7 {
  grid-template-columns: repeat(7, minmax(0, 1fr));
}

.analytics-metric-grid-6 {
  grid-template-columns: repeat(6, minmax(0, 1fr));
}

.analytics-metric-grid-3 {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

/* Simple label/count lists (approval bottlenecks, most-changed fields,
 * workload per owner) - same row shape as dashboard.css's .bar-row, minus
 * the proportional bar, for sections where a plain ranked list reads more
 * clearly than a bar chart. */

.analytics-simple-list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.55rem;
}

.analytics-simple-list li {
  display: flex;
  justify-content: space-between;
}

```

```

align-items: center;
font-size: 0.85rem;
color: #344054;
padding-bottom: 0.55rem;
border-bottom: 1px solid #f2f4f7;
}

.analytics-simple-list li:last-child {
border-bottom: none;
padding-bottom: 0;
}

.analytics-simple-list li span:last-child {
font-weight: 600;
color: #101828;
}

/* Risk trend chart (RiskTrendChart.jsx) */

.trend-list {
list-style: none;
margin: 0 0 0.9rem;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

.trend-row {
display: grid;
grid-template-columns: 70px 1fr 32px;
align-items: center;
gap: 0.75rem;
}

.trend-row-label {
font-size: 0.78rem;
color: #475467;
white-space: nowrap;
}

.trend-bar-track {
height: 10px;
border-radius: 999px;
background: #f2f4f7;
overflow: hidden;
display: flex;
}

.trend-bar-segment {
height: 100%;
}

.trend-row-count {
font-size: 0.78rem;
font-weight: 600;
color: #101828;
text-align: right;
}

.trend-legend {
display: flex;
flex-wrap: wrap;
gap: 0.9rem;
}

.trend-legend-item {
display: inline-flex;
align-items: center;
gap: 0.35rem;
font-size: 0.75rem;
color: #667085;
}

.trend-legend-swatch {
width: 8px;
height: 8px;
border-radius: 2px;
display: inline-block;
}

@media (max-width: 1300px) {
.analytics-metric-grid-7 {
grid-template-columns: repeat(4, minmax(0, 1fr));
}

.analytics-metric-grid-6 {
grid-template-columns: repeat(3, minmax(0, 1fr));
}
}

@media (max-width: 900px) {
.analytics-metric-grid-7,
.analytics-metric-grid-6,
.analytics-metric-grid-3 {
grid-template-columns: repeat(2, minmax(0, 1fr));
}
}

@media (max-width: 640px) {
.analytics-metric-grid-7,
.analytics-metric-grid-6,
.analytics-metric-grid-3 {
grid-template-columns: 1fr;
}
}

```

```
.cr-form-shell {
  max-width: 760px;
  margin: 0 auto;
  padding: 2rem 1.5rem 4rem;
  box-sizing: border-box;
}

/* Breadcrumbs */

.breadcrumbs {
  display: flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.82rem;
  margin-bottom: 1rem;
}

.breadcrumb-link {
  font-family: inherit;
  font-size: inherit;
  background: none;
  border: none;
  padding: 0;
  color: #667085;
  cursor: pointer;
}

.breadcrumb-link:hover {
  color: #4338ca;
  text-decoration: underline;
}

.breadcrumb-separator {
  color: #d0d5dd;
}

.breadcrumb-current {
  color: #101828;
  font-weight: 600;
}

/* Heading */

.cr-form-heading {
  margin-bottom: 1.5rem;
}

.cr-form-heading h1 {
  margin: 0 0 0.3rem;
  font-size: 1.4rem;
  font-weight: 700;
  color: #101828;
}

.cr-form-heading p {
  margin: 0;
  font-size: 0.9rem;
  color: #667085;
}

/* Form + sections */

.cr-form {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.cr-form-error {
  background: #fef3f2;
  border: 1px solid #fecdc4;
  color: #b42318;
  font-size: 0.85rem;
  padding: 0.65rem 0.8rem;
  border-radius: 8px;
  margin: 0;
}

.cr-section {
  background: #fff;
  border: 1px solid #eaecef;
  border-radius: 12px;
  padding: 1.5rem 1.5rem 0.5rem;
}

.cr-section-title {
  margin: 0 0 1.1rem;
  font-size: 0.95rem;
  font-weight: 600;
  color: #101828;
}

/* Fields */

.cr-field {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  margin-bottom: 1.1rem;
}

.cr-field label {
  font-size: 0.85rem;
  font-weight: 600;
  color: #344054;
}

.cr-optional-tag {
  font-weight: 400;
  color: #96a2b3;
  font-size: 0.78rem;
}

.cr-field-example {
  margin: -0.1rem 0 0.1rem;
  font-size: 0.82rem;
  font-style: italic;
  color: #667085;
}

.cr-field input[type='text'],
.cr-field input[type='date'],
.cr-field textarea {
  font-family: inherit;
```



```

font-size: 0.95rem;
padding: 0.65rem 0.8rem;
border-radius: 8px;
border: 1px solid #d0d5dd;
outline: none;
color: #101828;
resize: vertical;
transition:
  border-color 0.15s ease,
  box-shadow 0.15s ease;
}

.cr-field input:focus,
.cr-field textarea:focus {
border-color: #4338ca;
box-shadow: 0 0 3px rgba(67, 56, 202, 0.15);
}

.cr-field input[aria-invalid='true'],
.cr-field textarea[aria-invalid='true'] {
border-color: #fda29b;
}

.cr-field-footer {
display: flex;
align-items: baseline;
justify-content: space-between;
gap: 0.75rem;
}

.cr-field-hint {
font-size: 0.78rem;
color: #667085;
}

.cr-field-error {
font-size: 0.78rem;
color: #b42318;
font-weight: 500;
}

.cr-char-count {
font-size: 0.75rem;
color: #98a2b3;
font-variant-numeric: tabular-nums;
white-space: nowrap;
}

.cr-field-row {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 1rem;
}

/* Priority selector */

.cr-priority-group {
display: flex;
gap: 0.6rem;
flex-wrap: wrap;
}

.cr-priority-btn {
font-family: inherit;
font-size: 0.85rem;
font-weight: 600;
color: #475467;
background: #fff;
border: 1px solid #d0d5dd;
border-radius: 8px;
padding: 0.5rem 1.1rem;
cursor: pointer;
transition:
  border-color 0.15s ease,
  color 0.15s ease,
  background 0.15s ease;
}

.cr-priority-btn:hover {
border-color: #98a2b3;
}

.cr-priority-btn--active {
border-color: var(--priority-color, #4338ca);
color: var(--priority-color, #4338ca);
background: color-mix(in srgb, var(--priority-color, #4338ca) 12%, white);
}

/* Actions */

.cr-form-actions {
display: flex;
justify-content: flex-end;
gap: 0.75rem;
padding-top: 0.25rem;
}

.cr-btn {
font-family: inherit;
font-size: 0.9rem;
font-weight: 600;
border-radius: 8px;
padding: 0.65rem 1.15rem;
cursor: pointer;
border: none;
transition: background 0.15s ease;
}

.cr-btn--primary {
background: #4338ca;
color: #fff;
}

.cr-btn--primary:hover:not(:disabled) {
background: #372ea8;
}

.cr-btn--primary:disabled {
background: #a5a3e0;
cursor: not-allowed;
}

.cr-btn--secondary {
background: #fff;
color: #344054;
border: 1px solid #d0d5dd;
}

.cr-btn--secondary:hover {
background: #f9fafb;
}

```

```

.cr-btn--ghost {
  background: none;
  color: #667085;
}

.cr-btn--ghost:hover:not(:disabled) {
  color: #344054;
}

.cr-btn--ghost:disabled {
  color: #d0d5dd;
  cursor: not-allowed;
}

/* Success screen */

.cr-success {
  max-width: 480px;
  margin: 3.5rem auto;
  text-align: center;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.6rem;
}

.cr-success-icon {
  width: 52px;
  height: 52px;
  border-radius: 50%;
  background: #ecfdf3;
  color: #067647;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.5rem;
  font-weight: 700;
  margin-bottom: 0.5rem;
}

.cr-success-title {
  margin: 0;
  font-size: 1.2rem;
  font-weight: 700;
  color: #101828;
}

.cr-success-subtitle {
  margin: 0 0 0.75rem;
  font-size: 0.9rem;
  color: #475467;
  line-height: 1.5;
}

.cr-success-actions {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 0.6rem;
}

.cr-analyze-notice {
  margin: 1.25rem 0 0;
  background: #f9fafb;
  border: 1px solid #eaecef;
  color: #475467;
  font-size: 0.85rem;
  line-height: 1.5;
  padding: 0.8rem 1rem;
  border-radius: 8px;
}

@media (max-width: 640px) {
  .cr-form-shell {
    padding: 1.25rem 1rem 3rem;
  }

  .cr-field-row {
    grid-template-columns: 1fr;
  }

  .cr-form-actions {
    flex-direction: column-reverse;
  }

  .cr-form-actions .cr-btn {
    width: 100%;
  }
}

```

```
/* Shared shell width for both the list and the detail view */
```

```
.cr-list-shell,
.cr-detail-shell {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem 1.5rem 3rem;
  box-sizing: border-box;
}
```

```
.cr-detail-shell {
  max-width: 840px;
}
```

```
.cr-list-status-text {
  color: #667085;
  font-size: 0.95rem;
  padding: 3rem 0;
  text-align: center;
}
```

```
.cr-list-status-text--error {
  color: #b42318;
}
```

```
/* Heading + toolbar */
```

```
.cr-list-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 1.5rem;
}
```

```
.cr-list-heading h1 {
  margin: 0 0 0.3rem;
  font-size: 1.5rem;
  font-weight: 700;
  color: #101828;
}
```

```
.cr-list-heading p {
  margin: 0;
  font-size: 0.9rem;
  color: #667085;
}
```

```
.cr-toolbar {
  display: flex;
  flex-wrap: wrap;
  gap: 0.7rem;
  margin-bottom: 1.25rem;
}
```

```
.cr-search-input {
  flex: 1 1 260px;
  font-family: inherit;
  font-size: 0.9rem;
  padding: 0.6rem 0.8rem;
  border-radius: 8px;
  border: 1px solid #d0d5dd;
  outline: none;
  color: #101828;
}
```

```
.cr-search-input:focus {
  border-color: #4338ca;
  box-shadow: 0 0 3px rgba(67, 56, 202, 0.15);
}
```

```
.cr-filter-select {
  font-family: inherit;
  font-size: 0.85rem;
  padding: 0.6rem 0.7rem;
  border-radius: 8px;
  border: 1px solid #d0d5dd;
  color: #344054;
  background: #fff;
  outline: none;
  min-width: 150px;
}
```

```
.cr-filter-select:focus {
  border-color: #4338ca;
}
```

```
/* Module 12 Phase 6: "Assigned to me" list toggle */
```

```
.cr-toolbar-checkbox {
  display: flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.85rem;
  color: #344054;
  padding: 0.6rem 0.4rem;
  white-space: nowrap;
}
```

```
/* Empty state */
```

```
.cr-list-empty {
  max-width: 380px;
  margin: 3.5rem auto;
  text-align: center;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.9rem;
}
```

```
.cr-list-empty-title {
  margin: 0;
  font-size: 1rem;
  font-weight: 600;
  color: #101828;
}
```

```
/* Table */
```

```
.cr-table-card {
  padding: 0.5rem 1.4rem 0.75rem;
  margin-bottom: 1rem;
}
```

```

.cr-list-row {
  cursor: pointer;
  transition: background 0.1s ease;
}

.cr-list-row:hover {
  background: #f9fafb;
}

.cr-open-link {
  font-family: inherit;
  font-size: 0.8rem;
  font-weight: 600;
  color: #4338ca;
  background: none;
  border: none;
  padding: 0;
  cursor: pointer;
  white-space: nowrap;
}

.cr-list-row:hover .cr-open-link {
  text-decoration: underline;
}

/* Pagination */

.cr-pagination {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
}

.cr-pagination-summary {
  font-size: 0.82rem;
  color: #667085;
}

.cr-pagination-buttons {
  display: flex;
  gap: 0.5rem;
}

/* Detail view */

.cr-detail-heading {
  margin-bottom: 1.5rem;
}

.cr-detail-heading h1 {
  margin: 0 0 0.6rem;
  font-size: 1.35rem;
  font-weight: 700;
  color: #101828;
}

.cr-detail-badges {
  display: flex;
  gap: 0.5rem;
  flex-wrap: wrap;
}

.cr-detail-section {
  background: #fff;
  border: 1px solid #eaecef;
  border-radius: 12px;
  padding: 1.4rem 1.5rem;
  margin-bottom: 1rem;
}

.cr-section-title--spaced {
  margin-top: 1.4rem;
}

.cr-detail-text {
  margin: 0;
  font-size: 0.9rem;
  color: #344054;
  line-height: 1.6;
  white-space: pre-wrap;
}

.cr-detail-grid {
  display: grid;
  grid-template-columns: auto 1fr;
  gap: 0.5rem 1.5rem;
  margin: 0.75rem 0 0;
}

.cr-detail-grid dt {
  font-size: 0.8rem;
  font-weight: 600;
  color: #667085;
}

.cr-detail-grid dd {
  margin: 0;
  font-size: 0.88rem;
  color: #101828;
}

.cr-capitalize {
  text-transform: capitalize;
}

.cr-no-analysis {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 0.75rem;
}

.cr-question-list {
  list-style: none;
  margin: 0.75rem 0 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.cr-question-item {
  display: flex;
  gap: 0.75rem;
  align-items: flex-start;
  padding: 0.75rem;
  background: #f9fafb;
  border-radius: 8px;
}

```

```

.cr-question-text {
  margin: 0 0 0.2rem;
  font-size: 0.87rem;
  font-weight: 600;
  color: #101828;
}

.cr-question-reason {
  margin: 0;
  font-size: 0.82rem;
  color: #667085;
}

/* A human's answer to one clarification question, once one has been
 * provided - see ClarificationAnswerControl in AnalysisDashboardPage.jsx. */
.cr-question-answer {
  margin: 0.5rem 0 0;
  padding: 0.5rem 0.65rem;
  background: #ecfdf3;
  border: 1px solid #d1f2e1;
  border-radius: 6px;
  font-size: 0.85rem;
  color: #101828;
}

.cr-question-answer-label {
  font-weight: 600;
  color: #067647;
}

/* Module 6: AI analysis detail sections (Requirements, Affected
 * Components, Dependencies, Risks, Test Cases, Implementation Plan) - reuse
 * the same .cr-detail-section shell and .badge classes as the rest of the
 * page, just a list layout instead of a definition grid. */

.cr-analysis-list {
  list-style: none;
  margin: 0.75rem 0 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.cr-analysis-list-item {
  display: flex;
  gap: 0.75rem;
  align-items: flex-start;
  padding: 0.75rem;
  background: #f9fafb;
  border-radius: 8px;
}

.cr-analysis-list-tag {
  margin: 0 0 0.2rem;
  font-size: 0.8rem;
  font-weight: 600;
  color: #101828;
}

.cr-analysis-list-text {
  margin: 0;
  font-size: 0.87rem;
  color: #344054;
  line-height: 1.5;
}

.cr-analysis-list-mitigation {
  margin: 0.35rem 0 0;
  font-size: 0.82rem;
  color: #667085;
  line-height: 1.5;
}

.cr-plain-list {
  margin: 0.5rem 0 0;
  padding-left: 1.2rem;
  font-size: 0.87rem;
  color: #344054;
  line-height: 1.6;
}

.cr-form-actions--left {
  justify-content: flex-start;
  padding-top: 0.75rem;
}

@media (max-width: 900px) {
  .cr-toolbar {
    flex-direction: column;
  }

  .cr-search-input,
  .cr-filter-select {
    width: 100%;
  }

  .cr-list-heading {
    flex-direction: column;
  }
}

@media (max-width: 640px) {
  .cr-list-shell,
  .cr-detail-shell {
    padding: 1.25rem 1rem 2rem;
  }

  .cr-detail-grid {
    grid-template-columns: 1fr;
  }

  .cr-detail-grid dt {
    margin-top: 0.5rem;
  }
}

```

```

.dashboard-shell {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem 1.5rem 3rem;
  box-sizing: border-box;
}

.dashboard-status-text {
  color: #667085;
  font-size: 0.95rem;
  padding: 3rem 0;
  text-align: center;
}

.dashboard-status-text--error {
  color: #b42318;
}

.dashboard-heading {
  margin-bottom: 1.5rem;
}

.dashboard-heading h1 {
  margin: 0 0 0.3rem;
  font-size: 1.5rem;
  font-weight: 700;
  color: #101828;
}

.dashboard-heading p {
  margin: 0;
  font-size: 0.9rem;
  color: #667085;
}

/* Metric cards */

.metric-grid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 1rem;
  margin-bottom: 1.5rem;
}

.metric-card {
  background: #fff;
  border: 1px solid #eaeef0;
  border-radius: 12px;
  padding: 1.1rem 1.2rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  /* Reset the browser's default <button> styling for cards rendered as a
   button (see MetricCard.jsx's own docstring) - a plain display-only
   card is still a <div> and never picks these up. */
  width: 100%;
  font: inherit;
  text-align: left;
  cursor: default;
}

.metric-card--clickable {
  cursor: pointer;
  transition: border-color 0.12s ease, box-shadow 0.12s ease;
}

.metric-card--clickable:hover {
  border-color: #b0b8ff;
  box-shadow: 0 1px 4px rgba(16, 24, 40, 0.06);
}

.metric-card--active {
  border-color: #4338ca;
  box-shadow: 0 0 0 1px #4338ca inset;
}

/* The tile drill-down popup's filter toolbar (date range + rows-per-page),
and its footer's pagination info/controls - see components/Modal.jsx,
which supplies the popup shell itself. */

.drilldown-filters {
  display: flex;
  flex-wrap: wrap;
  align-items: flex-end;
  gap: 1rem;
  margin-bottom: 1.1rem;
  padding-bottom: 1.1rem;
  border-bottom: 1px solid #eaeef0;
}

.drilldown-filter-field {
  display: flex;
  flex-direction: column;
  gap: 0.3rem;
  font-size: 0.78rem;
  font-weight: 500;
  color: #667085;
}

.drilldown-filter-field input,
.drilldown-filter-field select {
  font: inherit;
  font-size: 0.85rem;
  color: #101828;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  padding: 0.4rem 0.55rem;
  background: #fff;
}

.drilldown-pagination-info {
  font-size: 0.82rem;
  color: #667085;
  white-space: nowrap;
}

.drilldown-pagination-controls {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.metric-card-label {
  font-size: 0.78rem;
}

```

```

    font-weight: 500;
    color: #667085;
}

.metric-card-value {
    font-size: 1.7rem;
    font-weight: 700;
    color: #101828;
}

.metric-card-value--danger {
    color: #b42318;
}

.metric-card-value--success {
    color: #067647;
}

.metric-card-value--warning {
    color: #b54708;
}

.metric-card-value--neutral {
    color: #344054;
}

/* Charts */

.chart-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1rem;
    margin-bottom: 1.5rem;
}

.chart-card,
.table-card {
    background: #fff;
    border: 1px solid #eaecef;
    border-radius: 12px;
    padding: 1.25rem 1.4rem;
}

.chart-card-title {
    margin: 0 0 1rem;
    font-size: 0.95rem;
    font-weight: 600;
    color: #101828;
}

.table-card-header {
    display: flex;
    align-items: baseline;
    justify-content: space-between;
    gap: 0.75rem;
}

.table-card-header .chart-card-title {
    margin-bottom: 1rem;
}

.chart-empty {
    margin: 0;
    font-size: 0.85rem;
    color: #98a2b3;
    padding: 1rem 0;
}

.bar-list {
    list-style: none;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    gap: 0.7rem;
}

.bar-row {
    display: grid;
    grid-template-columns: 90px 1fr 28px;
    align-items: center;
    gap: 0.75rem;
}

.bar-row-label {
    font-size: 0.8rem;
    color: #475467;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}

.bar-track {
    height: 8px;
    border-radius: 999px;
    background: #f2f4f7;
    overflow: hidden;
}

.bar-fill {
    height: 100%;
    border-radius: 999px;
    transition: width 0.2s ease;
}

.bar-row-count {
    font-size: 0.8rem;
    font-weight: 600;
    color: #101828;
    text-align: right;
}

/* Recent requests table */

.table-scroll {
    overflow-x: auto;
}

.requests-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.85rem;
}

.requests-table th {
    text-align: left;
    font-size: 0.75rem;
    font-weight: 600;
    color: #667085;
    text-transform: uppercase;
    letter-spacing: 0.03em;
}

```

```

padding: 0 0.6rem 0.6rem;
border-bottom: 1px solid #eaeef0;
white-space: nowrap;
}

.requests-table td {
padding: 0.75rem 0.6rem;
border-bottom: 1px solid #f2f4f7;
color: #344054;
vertical-align: middle;
}

.requests-table tr:last-child td {
border-bottom: none;
}

.requests-table-id {
color: #98a2b3;
font-variant-numeric: tabular-nums;
}

.requests-table-title {
max-width: 320px;
overflow: hidden;
text-overflow: ellipsis;
white-space: nowrap;
color: #101828;
font-weight: 500;
}

.table-muted {
color: #98a2b3;
}

/* Badges (status / risk) */

.badge {
display: inline-flex;
align-items: center;
padding: 0.2rem 0.55rem;
border-radius: 999px;
font-size: 0.75rem;
font-weight: 600;
white-space: nowrap;
}

.badge--gray {
background: #f2f4f7;
color: #475467;
}

.badge--blue {
background: #eff8ff;
color: #175cd3;
}

.badge--purple {
background: #f4f3ff;
color: #5925dc;
}

.badge--green {
background: #ecfdf3;
color: #067647;
}

.badge--red {
background: #fef3f2;
color: #b42318;
}

.badge--amber {
background: #ffffae;
color: #b54708;
}

.badge--orange {
background: #fff4ed;
color: #c4320a;
}

.badge--teal {
background: #f0fdf9;
color: #107569;
}

.badge--indigo {
background: #eef1fd;
color: #4338ca;
}

/* Empty state */

.dashboard-empty {
max-width: 420px;
margin: 4rem auto;
text-align: center;
display: flex;
flex-direction: column;
align-items: center;
gap: 0.6rem;
}

.dashboard-empty-icon {
width: 48px;
height: 48px;
border-radius: 12px;
background: #eef1fd;
color: #4338ca;
display: flex;
align-items: center;
justify-content: center;
font-size: 1.4rem;
font-weight: 700;
margin-bottom: 0.5rem;
}

.dashboard-empty-title {
margin: 0;
font-size: 1.15rem;
font-weight: 700;
color: #101828;
}

.dashboard-empty-subtitle {
margin: 0 0 0.75rem;
font-size: 0.88rem;
color: #667085;
line-height: 1.5;
}

```



```
.dashboard-empty-cta {
  font-family: inherit;
  font-size: 0.9rem;
  font-weight: 600;
  color: #fff;
  background: #4338ca;
  border: none;
  border-radius: 8px;
  padding: 0.65rem 1.1rem;
  cursor: pointer;
  transition: background 0.15s ease;
}

.dashboard-empty-cta:hover {
  background: #372ea8;
}

@media (max-width: 1100px) {
  .metric-grid {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }

  .chart-grid {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 640px) {
  .metric-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .dashboard-shell {
    padding: 1.25rem 1rem 2rem;
  }
}
```

frontend/src/styles/home.css

25 lines

```
/* Shared shell styles for the authenticated app - the dashboard-specific
styling itself lives in dashboard.css, and the header in
components/header.css. */

.app-shell {
  min-height: 100vh;
  background: #f9fafb;
  display: flex;
  align-items: stretch;
}

.app-main {
  flex: 1 1 auto;
  min-width: 0;
}

.app-loading {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  color: #667085;
  font-size: 0.95rem;
}
```

```

/* Module 8: Impact & Dependency Graph - the interactive React Flow diagram
embedded in the Impact tab of the AI Analysis Dashboard. Scoped under
.ig- prefixed classes; reuses .badge / .bar-track / .bar-fill / .ad-empty
from the existing stylesheets rather than redefining them. */

.ig-shell {
  display: grid;
  grid-template-columns: 1fr 280px;
  gap: 1rem;
  align-items: stretch;
}

.ig-canvas {
  position: relative;
  height: 460px;
  border: 1px solid #eaecef;
  border-radius: 12px;
  background: #fbfbfd;
  overflow: hidden;
}

.ig-sidebar {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  min-width: 0;
}

.ig-hint {
  border: 1px dashed #d0d5dd;
  border-radius: 12px;
  padding: 1rem;
}

.ig-empty {
  border: 1px solid #eaecef;
  border-radius: 12px;
  padding: 1.5rem;
}

/* Nodes */

.ig-node {
  min-width: 140px;
  max-width: 190px;
  padding: 0.6rem 0.75rem;
  border-radius: 10px;
  background: #fff;
  border: 2px solid var(--ig-color, #d0d5dd);
  box-shadow: 0 1px 2px rgba(16, 24, 40, 0.06);
  display: flex;
  flex-direction: column;
  gap: 0.15rem;
  cursor: pointer;
}

.ig-node--root {
  background: #4338ca;
  border-color: #4338ca;
  color: #fff;
  align-items: center;
  text-align: center;
  cursor: default;
}

.ig-node--root .ig-node-title {
  font-size: 0.85rem;
  font-weight: 700;
  color: #fff;
}

.ig-node--root .ig-node-sub {
  font-size: 0.72rem;
  opacity: 0.85;
  max-width: 160px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.ig-node--hub {
  background: #f2f4f7;
  border: 1px dashed #98a2b3;
  align-items: center;
  text-align: center;
  cursor: default;
}

.ig-node--hub .ig-node-title {
  font-size: 0.75rem;
  font-weight: 700;
  color: #475467;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.ig-node--item:hover {
  box-shadow: 0 2px 6px rgba(16, 24, 40, 0.12);
}

.ig-node-kind {
  font-size: 0.65rem;
  font-weight: 700;
  color: var(--ig-color, #667085);
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.ig-node-title {
  font-size: 0.85rem;
  font-weight: 700;
  color: #101828;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.ig-node-type {
  font-size: 0.72rem;
  color: #667085;
}

/* React Flow overrides, scoped to this canvas only */

```

```

.ig-canvas .react-flow__handle {
  width: 6px;
  height: 6px;
  background: #98a2b3;
  border: none;
}

.ig-canvas .react-flow__attribution {
  font-size: 0.6rem;
}

/* Detail panel */

.ig-detail {
  border: 1px solid #eaeef0;
  border-radius: 12px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.ig-detail-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.ig-detail-close {
  border: none;
  background: none;
  font-size: 1.1rem;
  line-height: 1;
  color: #98a2b3;
  cursor: pointer;
  padding: 0.15rem;
}

.ig-detail-close:hover {
  color: #475467;
}

.ig-detail-title {
  margin: 0;
  font-size: 1rem;
  font-weight: 700;
  color: #101828;
}

.ig-detail-type {
  margin: 0;
  font-size: 0.8rem;
  color: #667085;
}

.ig-detail-label {
  margin: 0.25rem 0 0;
  font-size: 0.7rem;
  font-weight: 700;
  color: #98a2b3;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.ig-detail-text {
  margin: 0;
  font-size: 0.85rem;
  color: #344054;
  line-height: 1.45;
}

.ig-detail-confidence {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.ig-detail-confidence-value {
  font-size: 0.78rem;
  color: #667085;
  white-space: nowrap;
}

/* Legend */

.ig-legend {
  border: 1px solid #eaeef0;
  border-radius: 12px;
  padding: 0.85rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
}

.ig-legend-title {
  font-size: 0.7rem;
  font-weight: 700;
  color: #98a2b3;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

.ig-legend-item {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.8rem;
  color: #344054;
}

.ig-legend-item i {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  display: inline-block;
}

.ig-intro {
  font-style: italic;
  color: #667085;
  font-size: 0.85rem;
}

@media (max-width: 900px) {
  .ig-shell {
    grid-template-columns: 1fr;
  }

  .ig-canvas {
    height: 380px;
  }
}

```


57 lines

```
/* Module 16 Frontend phase: the handful of rules genuinely new to the
Knowledge Base page - everything else (list shell, toolbar, table,
badges, buttons, analysis-list rows) reuses classes already defined in
dashboard.css / change-request-form.css / change-requests-list.css /
workflow.css. */

.kb-upload-control {
  display: flex;
  align-items: center;
}

/* The real <input type="file"> is visually hidden - its own <label>
(styled as a normal .cr-btn) is what the person actually clicks, so an
upload control looks like every other button on this page instead of
the browser's default file-picker chrome. */
.kb-file-input {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

.kb-file-input:focus-visible + label,
.kb-file-input:focus + label {
  outline: 2px solid #4338ca;
  outline-offset: 2px;
}

.kb-search-results {
  margin-bottom: 1.25rem;
}

.kb-actions {
  display: flex;
  gap: 0.5rem;
  flex-wrap: wrap;
  white-space: nowrap;
}

.kb-inline-badge {
  margin-left: 0.5rem;
}

.kb-preview-row td {
  background: #f9fafb;
  padding: 1rem 1.4rem 1.25rem;
}

.kb-chunk-list {
  margin-top: 0;
}
```

```
.modal-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(16, 24, 40, 0.45);
  display: flex;
  align-items: flex-start;
  justify-content: center;
  padding: 4vh 1rem;
  z-index: 1000;
  overflow-y: auto;
}

.modal-dialog {
  background: #fff;
  border-radius: 12px;
  width: 100%;
  max-width: 920px;
  max-height: 90vh;
  display: flex;
  flex-direction: column;
  box-shadow: 0 20px 60px rgba(16, 24, 40, 0.25);
}

.modal-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.75rem;
  padding: 1.1rem 1.4rem;
  border-bottom: 1px solid #eaecef;
  flex-shrink: 0;
}

.modal-header h2 {
  margin: 0;
  font-size: 1.05rem;
  font-weight: 600;
  color: #101828;
}

.modal-close {
  background: none;
  border: none;
  font-size: 1.5rem;
  line-height: 1;
  color: #667085;
  cursor: pointer;
  padding: 0.15rem 0.5rem;
  border-radius: 6px;
}

.modal-close:hover {
  background: #f2f4f7;
  color: #101828;
}

.modal-body {
  padding: 1.2rem 1.4rem;
  overflow-y: auto;
}

.modal-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.75rem;
  padding: 0.85rem 1.4rem;
  border-top: 1px solid #eaecef;
  flex-shrink: 0;
}
```

frontend/src/styles/my-work.css

36 lines

```
/* Module 18 Phase 4 (Notifications, My Work & Personal Engineering Queue) -
small additions on top of dashboard.css/change-requests-list.css/
analysis-dashboard.css, which supply everything else this page uses
(.dashboard-shell, .metric-card, .ad-tabs, .requests-table, .badge, ...). */

.metric-card-clickable {
  cursor: pointer;
  border-radius: 12px;
  transition: transform 0.1s ease, box-shadow 0.15s ease;
}

.metric-card-clickable:hover {
  transform: translateY(-1px);
  box-shadow: 0 2px 10px rgba(16, 24, 40, 0.06);
}

.metric-card-clickable:focus-visible {
  outline: 2px solid #4338ca;
  outline-offset: 2px;
}

.waiting-badge {
  margin-left: 0.4rem;
}

/* Module 18 Phase 5 - Approve/Reject/Request Changes/View CR buttons in
the My Approvals and Overdue Items tables' Actions column. .respond-form
itself (the inline comment box that opens for Reject/Request Changes)
is already styled by workflow.css, reused as-is. */
.approval-actions-cell {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
  align-items: center;
}
```



```

/* Module 12 Phase 2: version badge, "AI analysis outdated" banner, and the
 * simple History / Versions sections + version-compare view on the Change
 * Request detail page and the Edit page's "here's what changed" screen. */

.version-badge {
  background: #f2f4f7;
  color: #475467;
  font-variant-numeric: tabular-nums;
}

/* The "CR v1"/"Analysis v1" badges at the top of the Analysis Dashboard
 * (see AnalysisDashboardPage.jsx) are real <button> elements, not <span>,
 * so they need the browser's own button chrome reset before .badge's own
 * pill styling shows through cleanly. Clicking either jumps to the
 * History tab below. */
.version-badge--clickable {
  border: none;
  font: inherit;
  cursor: pointer;
  transition: background 0.12s ease, color 0.12s ease;
}

.version-badge--clickable:hover {
  background: #e4e7ec;
  color: #101828;
}

/* Analysis History rows (AnalysisHistoryTab): "Analysis #N" is this run's
 * own position in the list; "(CR vX)" is a secondary, de-emphasized
 * detail showing which content version it ran against - the two numbers
 * are deliberately not the same thing (see that component's own comment). */
.version-row-cr-version {
  font-weight: 500;
  color: #667085;
}

/* Module 12 Phase 2 added an Edit button to the detail heading - lay the
 * title/badges out on the left and the button on the right rather than
 * letting it stack under the badges (change-requests-list.css's
 * .cr-detail-heading predates this and is just a plain block). */
.cr-detail-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
}

.cr-detail-heading > div:first-child {
  flex: 1;
  min-width: 0;
}

/* Outdated-analysis banner */
.outdated-banner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
  background: #ffffaeb;
  border: 1px solid #fedf89;
  color: #b54708;
  padding: 0.85rem 1.1rem;
  border-radius: 10px;
  margin: 0 0 1.25rem;
}

.outdated-banner-text {
  display: flex;
  align-items: flex-start;
  gap: 0.6rem;
  font-size: 0.87rem;
  line-height: 1.45;
}

.outdated-banner-icon {
  font-size: 1rem;
  line-height: 1.4;
}

.outdated-banner-strong {
  font-weight: 700;
}

/* Module 12 Phase 6: detail-page tabs (Overview / Versions & Activity /
 * Comments) - the page had grown into one long stack of sections. */
.cr-detail-tabs {
  display: flex;
  gap: 0.3rem;
  border-bottom: 1px solid #eaeaf0;
  margin: 0 0 1.25rem;
}

.cr-detail-tab {
  font-family: inherit;
  font-size: 0.88rem;
  font-weight: 600;
  color: #667085;
  background: none;
  border: none;
  border-bottom: 2px solid transparent;
  padding: 0.65rem 0.9rem;
  margin-bottom: -1px;
  cursor: pointer;
  transition:
    color 0.15s ease,
    border-color 0.15s ease;
}

.cr-detail-tab:hover {
  color: #101828;
}

.cr-detail-tab--active {
  color: #4338ca;
  border-bottom-color: #4338ca;
}

/* History timeline */
.history-list {

```

```

list-style: none;
margin: 0;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.7rem;
}

.history-item {
display: flex;
gap: 0.75rem;
padding-bottom: 0.7rem;
border-bottom: 1px solid #f2f4f7;
}

.history-item:last-child {
border-bottom: none;
padding-bottom: 0;
}

.history-dot {
width: 8px;
height: 8px;
border-radius: 50%;
background: #4338ca;
margin-top: 0.4rem;
flex-shrink: 0;
}

.history-body {
flex: 1;
min-width: 0;
}

.history-line {
font-size: 0.87rem;
color: #101828;
margin: 0;
line-height: 1.5;
}

.history-line strong {
font-weight: 600;
}

.history-meta {
font-size: 0.78rem;
color: #98a2b3;
margin: 0.15rem 0 0;
}

.history-value-change {
font-size: 0.82rem;
color: #475467;
margin: 0.2rem 0 0;
}

.history-value-change .from-value {
color: #b42318;
text-decoration: line-through;
}

.history-value-change .to-value {
color: #067647;
font-weight: 600;
}

/* Versions list */

.versions-list {
list-style: none;
margin: 0;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

.version-row {
display: flex;
align-items: center;
justify-content: space-between;
gap: 0.75rem;
padding: 0.7rem 0.85rem;
background: #f9fafb;
border: 1px solid #eaecef;
border-radius: 8px;
flex-wrap: wrap;
}

.version-row-main {
display: flex;
align-items: baseline;
gap: 0.6rem;
flex-wrap: wrap;
}

.version-row-number {
font-weight: 700;
color: #101828;
font-size: 0.88rem;
}

.version-row-summary {
font-size: 0.85rem;
color: #475467;
}

.version-row-meta {
font-size: 0.78rem;
color: #98a2b3;
}

.cr-btn--small {
padding: 0.35rem 0.75rem;
font-size: 0.78rem;
}

/* Version compare */

.compare-table {
width: 100%;
border-collapse: collapse;
font-size: 0.85rem;
margin-top: 0.75rem;
}

.compare-table th,
.compare-table td {
text-align: left;
padding: 0.55rem 0.7rem;
}

```

```

border-bottom: 1px solid #f2f4f7;
vertical-align: top;
}

.compare-table th {
  color: #667085;
  font-weight: 600;
  font-size: 0.78rem;
  text-transform: uppercase;
  letter-spacing: 0.02em;
}

.compare-row--unchanged {
  color: #98a2b3;
}

.compare-row--changed .compare-old {
  color: #b42318;
  text-decoration: line-through;
}

.compare-row--changed .compare-new {
  color: #067647;
  font-weight: 600;
}

.compare-row--added .compare-new {
  color: #067647;
  font-weight: 600;
}

.compare-row--removed .compare-old {
  color: #b42318;
  text-decoration: line-through;
}

/* Edit page: "here's what changed" confirmation */

.edit-confirm-list {
  list-style: none;
  margin: 1rem 0 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.6rem;
  text-align: left;
}

.edit-confirm-item {
  background: #f9fafb;
  border: 1px solid #eaecef;
  border-radius: 8px;
  padding: 0.65rem 0.85rem;
  font-size: 0.85rem;
}

.edit-confirm-field {
  font-weight: 600;
  color: #101828;
  display: block;
  margin-bottom: 0.2rem;
}

.edit-confirm-values .from-value {
  color: #b42318;
  text-decoration: line-through;
}

.edit-confirm-values .to-value {
  color: #067647;
  font-weight: 600;
}

.no-changes-note {
  margin: 1rem 0 0;
  font-size: 0.87rem;
  color: #667085;
}

/* Module 12 Phase 3: status workflow + assignments */

.status-form,
.assign-form,
.respond-form {
  margin-top: 0.75rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.status-form-row {
  display: flex;
  gap: 0.6rem;
  flex-wrap: wrap;
  align-items: center;
}

.assignments-list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.assignment-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.75rem;
  padding: 0.55rem 0.8rem;
  background: #f9fafb;
  border: 1px solid #eaecef;
  border-radius: 8px;
  font-size: 0.85rem;
  flex-wrap: wrap;
}

/* Module 12 Phase 4: approvals */

.approvals-recommendation {
  background: #f2f4f7;
  border: 1px solid #d1d9ff;
  color: #363f9c;
  padding: 0.6rem 0.85rem;
  border-radius: 8px;
  font-size: 0.85rem;
}

.approvals-list {

```

```

list-style: none;
margin: 0 0 0.75rem;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

.approval-row {
padding: 0.65rem 0.85rem;
background: #f9fafb;
border: 1px solid #eaecef;
border-radius: 8px;
font-size: 0.85rem;
display: flex;
flex-direction: column;
gap: 0.35rem;
}

.approval-row-main {
display: flex;
align-items: center;
gap: 0.5rem;
flex-wrap: wrap;
}

/* Module 12 Phase 5: comments + mentions */

.comments-list {
list-style: none;
margin: 0 0 0.9rem;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

.comment-row {
padding: 0.65rem 0.85rem;
background: #f9fafb;
border: 1px solid #eaecef;
border-radius: 8px;
font-size: 0.85rem;
display: flex;
flex-direction: column;
gap: 0.3rem;
}

.comment-row--reply {
background: #fff;
}

.comment-row-header {
display: flex;
align-items: center;
gap: 0.5rem;
flex-wrap: wrap;
}

.comment-body {
white-space: pre-wrap;
}

/* Module 12 Phase 5: inline @mention autocomplete (MentionTextarea) */

.mention-input-wrap {
position: relative;
}

.mention-suggestions {
position: absolute;
z-index: 20;
top: 100%;
left: 0;
right: 0;
margin: 0.25rem 0 0;
padding: 0.25rem;
list-style: none;
background: #fff;
border: 1px solid #d0d5dd;
border-radius: 8px;
box-shadow: 0 8px 20px rgba(16, 24, 40, 0.12);
max-height: 220px;
overflow-y: auto;
}

.mention-suggestion {
display: block;
width: 100%;
text-align: left;
background: none;
border: none;
font-family: inherit;
font-size: 0.85rem;
color: #101828;
padding: 0.45rem 0.6rem;
border-radius: 6px;
cursor: pointer;
}

.mention-suggestion:hover,
.mention-suggestion--active {
background: #f2f4ff;
}

.comment-row-actions {
display: flex;
gap: 0.4rem;
flex-wrap: wrap;
}

.comment-reply-form {
margin-top: 0.4rem;
}

.comment-reply-form textarea,
.assign-form textarea {
width: 100%;
font-family: inherit;
font-size: 0.85rem;
padding: 0.5rem 0.65rem;
border: 1px solid #d0d5dd;
border-radius: 8px;
resize: vertical;
box-sizing: border-box;
}

.comment-replies-list {
list-style: none;
margin: 0.5rem 0 0 1.25rem;
padding: 0.5rem 0 0 0.75rem;
}

```

```
border-left: 2px solid #eaeef0;
display: flex;
flex-direction: column;
gap: 0.5rem;
}

/* Module 12 Phase 5: notifications inbox (NotificationsPage) */

.notifications-list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 0.55rem;
}

.notification-row {
  padding: 0.75rem 1rem;
  background: #fff;
  border: 1px solid #eaeef0;
  border-radius: 10px;
  display: flex;
  flex-direction: column;
  gap: 0.3rem;
}

.notification-row[role='button'] {
  cursor: pointer;
}

.notification-row[role='button']:hover {
  border-color: #b8c0ff;
}

.notification-row--unread {
  background: #f7f8ff;
  border-color: #d1d9ff;
}

.notification-row-main {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  flex-wrap: wrap;
}

.notification-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #4046ca;
  display: inline-block;
}

.notification-row-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
}
```

Tests

tests/__init__.py

1 lines

tests/conftest.py

37 lines

```
"""Pytest bootstrap - runs before any test module in this directory is
imported, and is why this file has no tests of its own.

Every test file in this suite talks to the app the same simple way: import
app.database.init_db / app.main directly and drive them with FastAPI's
TestClient (hackathon-simple, no separate test-database wiring). Without
this file, that means every `pytest ../tests` run reads and writes the
exact same backend/data/app.db the developer is using in the browser at
the same time - so every user a test registers (a fresh one per test, via
each file's _register()/ _auth() helper, uuid'd email but a fixed name)
permanently piles up in the real database. That's mostly just clutter in
the real app's "assign a team member" / "@mention" pickers, but as of
Module 12 Phase 5 it can cause a genuine test failure: run the suite
twice, and there are now two different users both named e.g. "Priya
Shah" - and @mention matching (app/services/mentions.py), which can only
go on the plain text of the comment, correctly matches both of them,
breaking a test written assuming there's exactly one.

Redirecting DATABASE_URL to its own throwaway sqlite file - before
app.core.config.get_settings() is ever called anywhere - keeps every
pytest run isolated from the real app's data and from every previous
pytest run. Deleting that file first means a clean slate every run, so no
test result ever depends on what a previous run left behind. This only
changes the environment of this pytest process - the already-running
`uvicorn app.main:app --reload` in your other terminal is a separate
program and is completely unaffected.
"""

import os
from pathlib import Path

_TEST_DB_PATH = Path(__file__).resolve().parent.parent / "backend" / "data" / "test_app.db"
_TEST_DB_PATH.parent.mkdir(parents=True, exist_ok=True)
if _TEST_DB_PATH.exists():
    _TEST_DB_PATH.unlink()

os.environ["DATABASE_URL"] = f"sqlite:///({_TEST_DB_PATH.as_posix()})"
```


tests/test_ai_analysis_phase3.py

384 lines

```
"""Verifies Module 13 Phase 3 (AI Analysis 2.0 - outdated awareness,
re-analysis notifications, AI-failure safety):

* AnalysisRead.is_outdated - computed fresh by GET .../analysis and
  POST .../analyze, using the same workflow rules.is_analysis_outdated
  ChangeRequestDetail.is_analysis_outdated already relies on - just
  exposed on the Analysis object itself so a caller doesn't have to also
  fetch the CR detail to know whether what they're looking at is stale.
* re-analyzing into a materially different result (bigger risk bucket,
  changed recommendation, ...) notifies the CR's creator + assignees
  with a new ANALYSIS SIGNIFICANTLY CHANGED notification - but never the
  person who triggered the re-analysis - reusing Phase 2's
  compare_analyses()/is_significant_change rather than a second
  "did this change materially" check.
* re-analyzing into something NOT materially different sends no such
  notification.
* a failed (re-)analysis never touches whatever analysis already existed
  - the API layer's own error message says so.

The real AI provider is never called - same monkeypatched FakeProvider
pattern as test_analysis_engine.py / test_change_request_editing.py /
test_analysis_delta.py.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"phase3-{uuid.uuid4().hex[:10]}@example.com"

def _register(name: str) -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _auth() -> tuple[dict, int]:
    return _register("Phase3 Tester")

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested_by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _notifications(headers: dict):
    return client.get("/api/notifications", headers=headers).json()

class _FakeProvider:
    def __init__(self, response_text=None, configured=True, raise_exc=None):
        self._text = response_text
        self._configured = configured
        self._raise = raise_exc

    def is_configured(self) -> bool:
        return self._configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _analyze(headers, cr_id, response_dict, monkeypatch):
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_dict)))
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return response.json()

# Same low-risk -> high-risk pair test_analysis_delta.py already validated
# as "a significant change" (risk moved medium -> high, complexity medium
# -> high, recommendation approve_with_conditions -> requires_clarification).
FIRST_PASS = {
    "Summary": "Adds OTP-based authentication at login.",
    "classification": {"category": "Security", "confidence": 0.8, "reason": "New authentication factor."},
    "requirements": [
        {"category": "functional", "description": "OTP is sent to the customer's registered mobile number."},
    ],
    "affected_components": [
        {
            "name": "Authentication Service",
            "type": "backend",
            "impact_level": "medium",
            "reason": "Adds OTP verification.",
            "confidence": 70,
        }
    ]
}
```

```

    },
    "dependencies": [],
    "risks": [
        {
            "category": "security",
            "description": "OTP interception risk.",
            "severity": "medium",
            "probability": 0.3,
            "score": 40,
            "explanation": "Standard OTP risk.",
            "mitigation": "Use a short expiry window.",
        }
    ],
    "security_analysis": {"concerns": [], "summary": ""},
    "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
    "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs OTP expiry defined."},
}

SECOND_PASS = {
    "summary": "Adds OTP-based authentication at login, now including a payment confirmation step.",
    "classification": {"category": "Security", "confidence": 0.8, "reason": "New authentication factor."},
    "requirements": [
        {"category": "functional", "description": "OTP is sent to the customer's registered mobile number."},
        {"category": "functional", "description": "OTP must also confirm high-value payment transactions."},
    ],
    "affected_components": [
        {
            "name": "Authentication Service",
            "type": "backend",
            "impact_level": "high",
            "reason": "Adds OTP verification.",
            "confidence": 75,
        },
        {
            "name": "Payment Gateway",
            "type": "third_party",
            "impact_level": "high",
            "reason": "OTP now confirms payments.",
            "confidence": 65,
        },
    ],
    "dependencies": [],
    "risks": [
        {
            "category": "security",
            "description": "OTP interception risk.",
            "severity": "high",
            "probability": 0.4,
            "score": 70,
            "explanation": "Now guards a financial transaction, not just login.",
            "mitigation": "Use a short expiry window and rate limiting.",
        },
        {
            "category": "compliance",
            "description": "Payment confirmation via OTP may need compliance review.",
            "severity": "high",
            "probability": 0.5,
            "score": 60,
            "explanation": "Financial transactions were added to scope.",
            "mitigation": "Confirm with compliance before implementation.",
        },
    ],
    "security_analysis": {"concerns": ["Financial transactions now in scope"], "summary": "Treat as high-risk."},
    "complexity": {"level": "high", "reasoning": "Now touches payments as well as auth."},
    "effort": {"backend": "4 days", "frontend": "2 days", "testing": "2 days", "total": "8 developer-days"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "requires_clarification", "reasoning": "Compliance review needed first."},
}

```

--- is_outdated on the Analysis itself -----

```

def test_is_outdated_false_immediately_after_analysis(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    analysis = _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    assert analysis["is_outdated"] is False

    fetched = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    assert fetched["is_outdated"] is False

```

```

def test_is_outdated_true_after_edit_until_reanalyzed(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )
    assert edit.status_code == 200

    fetched = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    assert fetched["is_outdated"] is True
    assert fetched["change_request_version"] == 1 # still the version it actually ran against

    reanalyzed = _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    assert reanalyzed["is_outdated"] is False
    assert reanalyzed["change_request_version"] == 2

```

--- Significant-change notification on re-analysis -----

```

def test_significant_reanalysis_notifies_creator_and_assignees_not_the_reanalyzer(monkeypatch):
    creator_headers, _ = _auth()
    created = _create_change_request(creator_headers)

    assignee_headers, assignee_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "technical_lead"},
        headers=creator_headers,
    )

    reanalyzer_headers, _ = _auth() # a different, unassigned authenticated user
    _analyze(creator_headers, created["id"], FIRST_PASS, monkeypatch)
    _analyze(reanalyzer_headers, created["id"], SECOND_PASS, monkeypatch)

    creator_notifs = _notifications(creator_headers)

```

```

assignee_notifs = notifications(assignee_headers)
reanalyzer_notifs = _notifications(reanalyzer_headers)

assert any(n["type"] == "analysis_significantly_changed" for n in creator_notifs)
assert any(n["type"] == "analysis_significantly_changed" for n in assignee_notifs)
# The person who triggered the re-analysis never gets notified about
# their own action.
assert not any(n["type"] == "analysis_significantly_changed" for n in reanalyzer_notifs)

def test_non_significant_reanalysis_sends_no_notification(monkeypatch):
    creator_headers, _ = _auth()
    created = _create_change_request(creator_headers)

    assignee_headers, assignee_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )

    analyze(creator_headers, created["id"], FIRST_PASS, monkeypatch)
    # Re-analyzing with the exact same canned response - nothing material
    # moved, so no ANALYSIS SIGNIFICANTLY CHANGED notification should fire.
    _analyze(creator_headers, created["id"], FIRST_PASS, monkeypatch)

    assignee_notifs = notifications(assignee_headers)
    assert not any(n["type"] == "analysis_significantly_changed" for n in assignee_notifs)

def test_first_ever_analysis_sends_no_significant_change_notification(monkeypatch):
    """There's nothing to compare against yet - a first analysis is never
    itself a "change", significant or otherwise."""
    creator_headers, _ = _auth()
    created = _create_change_request(creator_headers)

    assignee_headers, assignee_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )

    _analyze(creator_headers, created["id"], SECOND_PASS, monkeypatch) # even the "riskier" shape

    assignee_notifs = notifications(assignee_headers)
    assert not any(n["type"] == "analysis_significantly_changed" for n in assignee_notifs)

# --- AI-failure safety -----

def test_failed_reanalysis_leaves_prior_analysis_untouched_and_says_so(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    prior = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()

    _patch_provider(monkeypatch, _FakeProvider(response_text="not valid json at all"))
    failed = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert failed.status_code == 422
    detail = failed.json()["detail"]
    assert "still available" in detail
    assert "Version 1" in detail

    still_there = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    assert still_there["id"] == prior["id"]
    assert still_there["summary"] == prior["summary"]

def test_failed_first_analysis_error_has_no_prior_analysis_mention(monkeypatch):
    """No prior analysis exists yet, so the error shouldn't claim one is
    "still available" - that would be misleading, not reassuring."""
    headers, _ = _auth()
    created = _create_change_request(headers)

    _patch_provider(monkeypatch, _FakeProvider(configured=False))
    failed = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert failed.status_code == 503
    assert "still available" not in failed.json()["detail"]

# --- Report generation and an outdated analysis -----
#
# Module 20 (Version-Aware Reports) deliberately changed this behavior: this
# test used to assert that downloading the CURRENT report still succeeded
# even when the analysis was outdated (just with a warning banner) - but
# that meant the report silently mixed the CR's live (post-edit) fields with
# the stale, pre-edit analysis, exactly what that module's own spec says
# never to do ("do not generate a misleading current report"). The current-
# report request now correctly refuses (409) instead; the change request's
# data is still fully reportable, just explicitly as a labeled historical
# report for the version that was actually analyzed. Full coverage of the
# new version-mismatch/historical-report behavior lives in
# tests/test_module20_phase1.py - this one test is kept here (updated) since
# it's the one place in this file that already had the exact "outdated
# analysis" setup handy.

def test_current_report_now_refuses_when_analysis_is_outdated_but_historical_report_still_works(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)

    client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )

    current = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert current.status_code == 409
    assert current.json()["detail"]["last_analyzed_version"] == 1

    historical = client.get(f"/api/change-requests/{created['id']}/report?version=1", headers=headers)
    assert historical.status_code == 200
    assert historical.headers["content-type"] == "application/pdf"
    assert len(historical.content) > 0

def test_report_downloads_successfully_when_analysis_is_current(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    assert len(response.content) > 0

```

tests/test_ai_analysis_phase4.py

289 lines

```
"""Verifies Module 13 Phase 4 (AI Analysis 2.0 - workflow recommendation
text + human-override recording):

* AnalysisRead.workflow_recommendation - one computed (not AI-written)
sentence combining the AI's own recommendation with which approvals it
actually requires (app/services/workflow_rules.py::recommendation_summary)
* ApprovalRead.ai_recommendation / override_ai_recommendation - what the
AI recommended on the analysis THIS approval was requested against (by
approval.cr_version, not necessarily the CR's *current* analysis), and
whether the approver's decision matched it
* responding in a way that doesn't match the AI's recommendation records
an additional AI RECOMMENDATION_OVERRIDDEN history event alongside the
normal APPROVED/REJECTED/CHANGES_REQUESTED one - never instead of it,
and never blocking the decision itself ("AI recommends, humans decide")

The real AI provider is never called - same monkeypatched _FakeProvider
pattern as the other test_ai_analysis_*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def unique_email() -> str:
    return f"phase4-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> tuple[dict, int]:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Phase4 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _request_approval(cr_id: int, headers: dict, approver_id: int, approval_type: str = "engineering_manager"):
    return client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": approval_type, "approver_user_id": approver_id},
        headers=headers,
    )

class _FakeProvider:
    def __init__(self, response_text=None, configured=True):
        self._text = response_text
        self._configured = configured

    def is_configured(self) -> bool:
        return self._configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _analyze(headers, cr_id, response_dict, monkeypatch):
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_dict)))
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return response.json()

def _base_response(decision: str, risks=None, category: str = "Security"):
    return {
        "summary": "Adds OTP-based authentication at login.",
        "classification": {"category": category, "confidence": 0.8, "reason": "New authentication factor."},
        "requirements": [],
        "affected components": [],
        "dependencies": [],
        "risks": risks or [],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
        "missing information": [],
        "test cases": [],
        "implementation plan": [],
        "recommendation": {"decision": decision, "reasoning": "See analysis."},
    }

# "Feature Enhancement" (not "Security") deliberately avoids the
# category-keyword approval add-ons below, so this exercises the plain
# low-risk baseline (Technical) on its own.
APPROVE_RESPONSE = _base_response("approve", category="Feature Enhancement")

# NOTE: the AI Analysis Engine's own response schema (app/schemas/ai_analysis.py
```

```

# ::RecommendationResult.decision allowed) only ever lets the AI recommend
# "approve" / "approve with conditions" / "requires clarification" - REJECT
# is a real ApprovalRecommendation value (kept for old seeded rows / a
# human's own decision) but the AI itself can never produce it via
# /analyze. So these tests use a high-risk "requires clarification"
# response for the override scenarios instead of "reject".
HIGH_RISK_CLARIFICATION_RESPONSE = _base_response(
    "requires_clarification",
    risks=[
        {
            "category": "security",
            "description": "Severe vulnerability.",
            "severity": "critical",
            "probability": 0.8,
            "score": 90,
            "explanation": "Critical exposure if shipped as-is.",
            "mitigation": "Redesign before proceeding.",
        }
    ],
)
CLARIFICATION_RESPONSE = _base_response("requires_clarification")

# --- workflow_recommendation on the analysis -----

def test_workflow_recommendation_mentions_ai_decision_and_required_approvals(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    analysis = _analyze(headers, created["id"], HIGH_RISK_CLARIFICATION_RESPONSE, monkeypatch)

    assert "Requires Clarification" in analysis["workflow_recommendation"]
    # Risk score 90 -> critical baseline (Engineering Manager, Security, Director).
    assert "Engineering Manager" in analysis["workflow_recommendation"]
    assert "Security" in analysis["workflow_recommendation"]

    fetched = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    assert fetched["workflow_recommendation"] == analysis["workflow_recommendation"]

def test_workflow_recommendation_names_the_low_risk_baseline_approval(monkeypatch):
    """No risks -> risk score floors at 5.0 -> the low-risk baseline
    (Technical) - required approval types() always returns at least that
    baseline, so "no approval needed" never actually happens once a change
    request has been analyzed at all (see
    app/services/workflow_rules.py::risk_baseline's fallback)."""
    headers, _ = _auth()
    created = _create_change_request(headers)
    analysis = _analyze(headers, created["id"], APPROVE_RESPONSE, monkeypatch)

    assert "Approve" in analysis["workflow_recommendation"]
    assert "requires Technical approval" in analysis["workflow_recommendation"]

# --- ai_recommendation / override_ai_recommendation on the approval -----

def test_approving_when_ai_only_asked_for_clarification_is_flagged_as_an_override(monkeypatch):
    """The AI recommended "requires clarification" (which aligns with a
    human responding "changes requested" - see
    test_requesting_changes_when_ai_only_asked_for_clarification_is_not_flagged
    below); approving it directly instead, skipping that clarification, is
    a real divergence from what the AI suggested."""
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], HIGH_RISK_CLARIFICATION_RESPONSE, monkeypatch)

    approver_headers, approver_id = _auth()
    requested = _request_approval(created["id"], headers, approver_id, "engineering_manager").json()
    assert requested["ai_recommendation"] == "Requires Clarification"
    assert requested["override_ai_recommendation"] is False # still pending - nothing decided yet

    responded = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    ).json()
    assert responded["ai_recommendation"] == "Requires Clarification"
    assert responded["override_ai_recommendation"] is True

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    override_events = [h for h in history if h["action"] == "ai_recommendation_overridden"]
    assert len(override_events) == 1
    assert "Requires Clarification" in override_events[0]["old_value"]
    # The normal APPROVED event is still recorded too - this is additional,
    # never a replacement.
    assert any(h["action"] == "approved" for h in history)

def test_approving_in_line_with_an_ai_approve_recommendation_is_not_flagged(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], APPROVE_RESPONSE, monkeypatch)

    approver_headers, approver_id = _auth()
    requested = _request_approval(created["id"], headers, approver_id, "technical").json()

    responded = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    ).json()
    assert responded["override_ai_recommendation"] is False

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert not any(h["action"] == "ai_recommendation_overridden" for h in history)

def test_requesting_changes_when_ai_only_asked_for_clarification_is_not_flagged(monkeypatch):
    """NEEDS MORE INFO/REQUIRES CLARIFICATION both align with a human
    responding "changes requested" - that's not a disagreement, it's the
    same call."""
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], CLARIFICATION_RESPONSE, monkeypatch)

    approver_headers, approver_id = _auth()
    requested = _request_approval(created["id"], headers, approver_id, "technical").json()
    assert requested["ai_recommendation"] == "Requires Clarification"

    responded = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "changes requested", "comment": "Needs the OTP expiry window specified."},
        headers=approver_headers,
    ).json()
    assert responded["override_ai_recommendation"] is False

def test_override_uses_the_analysis_the_approval_was_actually_requested_against(monkeypatch):
    """Version-correctness: if the CR gets re-analyzed with a DIFFERENT
    recommendation after an approval was already requested, the override

```

```

check must still use the analysis/version the approval was requested
against - never whatever the CR's *current* analysis says now."""
headers, _ = auth()
created = create_change_request(headers)
# version 1: AI says requires clarification
_analyze(headers, created["id"], HIGH_RISK_CLARIFICATION_RESPONSE, monkeypatch)

approver_headers, approver_id = _auth()
requested = _request_approval(created["id"], headers, approver_id, "engineering_manager").json()
assert requested["cr_version"] == 1
assert requested["ai_recommendation"] == "Requires Clarification"

# Edit the CR (bumps to version 2) and re-analyze with a completely
# different recommendation. The pending approval above stays tied to
# version 1's analysis (approval.cr_version doesn't move just because
# the CR did - see app/services/approvals.py::is_approval_outdated).
client.put(
    f"/api/change-requests/{created['id']}",
    json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
    headers=headers,
)
_analyze(headers, created["id"], APPROVE_RESPONSE, monkeypatch) # version 2: AI now says approve

responded = client.post(
    f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
    json={"status": "approved"},
    headers=approver_headers,
).json()
# Still compared against version 1's "Requires Clarification" (what
# this approval was actually requested against), not version 2's
# "Approve".
assert responded["ai_recommendation"] == "Requires Clarification"
assert responded["overrode_ai_recommendation"] is True

```

tests/test_ai_analysis_v2.py

293 lines

```
"""Verifies Module 13 Phase 1 (AI Analysis 2.0 - epistemic tagging):
Requirement/AffectedComponent/Risk each carry a Known/Inferred/Unknown
`certainty` label, and Requirement/Risk also carry a per-finding
`confidence` (0-100) and `evidence` string.

Three things matter most here, since they're easy to get subtly wrong:
* "unknown" must persist as a real Certainty.UNKNOWN value - not be
  swallowed by the "no answer provided" placeholder handling that every
  other enum field in this codebase uses (where the string "unknown"
  means "the AI didn't answer", not "the AI said unknown").
* A canned response shaped like pre-Module-13 output (no certainty/
  confidence/evidence keys at all) must still validate and persist -
  existing behavior must not break.
* Risk.evidence is populated from the AI's `explanation` field.

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as test_analysis_engine.py.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"ai2-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "AI2 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    """The smallest valid AIAnalysisResult shape (mirrors the required
    fields only) - callers layer in requirements/affected_components/risks
    to exercise Module 13's new fields."""
    base = {
        "summary": "Adds OTP-based authentication at login.",
        "classification": {"category": "Security", "confidence": 0.9, "reason": "New authentication factor."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
        "effort": {
            "backend": "2-3 days",
            "frontend": "1 day",
            "testing": "1 day",
            "total": "4-5 developer-days",
        },
        "missing_information": [],
        "test_cases": [],
        "implementation_plan": [],
        "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs OTP expiry defined."},
    }
    base.update(overrides)
    return base

def test_known_inferred_unknown_round_trip_via_api(monkeypatch):
    """The exact three-way example from the spec: a known fact, an
    inferred judgment call, and a genuinely unknown detail - each must
    persist and come back through the API as the distinct value it was,
    not collapsed into one default."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        requirements=[
            {
                "category": "functional",
                "description": "OTP is sent to the customer's registered mobile number.",
                "priority": "high",
            }
        ]
    )
```

```

        "certainty": "known",
        "confidence": 95,
        "evidence": "Explicitly stated in the change request description.",
    },
    {
        "category": "non functional",
        "description": "Rate limiting is probably required on OTP requests.",
        "priority": "medium",
        "certainty": "inferred",
        "confidence": 60,
        "evidence": "Common practice for OTP flows, not stated in the request.",
    },
    {
        "category": "functional",
        "description": "OTP expiration window.",
        "priority": "high",
        "certainty": "unknown",
        "confidence": 20,
        "evidence": "Not mentioned anywhere in the change request.",
    },
],
)
import json as _json

_patch_provider(monkeypatch, _FakeProvider(_json.dumps(response_json)))

analyze_response = client.post(
    f"/api/change-requests/{created['id']}/analyze", headers=headers
)
assert analyze_response.status_code == 201

detail = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
reqs = {r["description"]: r for r in detail["requirements"]}

known = reqs["OTP is sent to the customer's registered mobile number."]
assert known["certainty"] == "known"
assert known["confidence"] == 95
assert "Explicitly stated" in known["evidence"]

inferred = reqs["Rate limiting is probably required on OTP requests."]
assert inferred["certainty"] == "inferred"
assert inferred["confidence"] == 60

unknown = reqs["OTP expiration window."]
# This is the critical assertion: "unknown" must survive as a real
# value, not be swallowed by the generic placeholder-detection used
# for every other enum field in this codebase.
assert unknown["certainty"] == "unknown"
assert unknown["confidence"] == 20

def test_affected_component_certainty_round_trips(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        affected_components=[
            {
                "name": "Authentication Service",
                "type": "backend",
                "impact_level": "high",
                "reason": "Central place OTP verification would be added.",
                "confidence": 78,
                "certainty": "inferred",
            }
        ],
    )
    import json as _json

    _patch_provider(monkeypatch, _FakeProvider(_json.dumps(response_json)))
    assert client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    component = detail["affected components"][0]
    assert component["certainty"] == "inferred"
    assert component["confidence"] == 78

def test_risk_confidence_and_evidence_round_trip(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        risks=[
            {
                "category": "security",
                "description": "OTP brute-forcing without a retry limit.",
                "severity": "high",
                "probability": 0.6,
                "score": 75,
                "explanation": "No retry limit was mentioned in the request.",
                "mitigation": "Add a maximum attempt count with lockout.",
                "certainty": "known",
                "confidence": 85,
            }
        ],
    )
    import json as _json

    _patch_provider(monkeypatch, _FakeProvider(_json.dumps(response_json)))
    assert client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    risk = detail["risks"][0]
    assert risk["certainty"] == "known"
    assert risk["confidence"] == 85
    assert risk["evidence"] == "No retry limit was mentioned in the request."

def test_pre_module_13_shaped_response_still_validates(monkeypatch):
    """A response with no certainty/confidence/evidence keys at all (the
    exact shape Module 6 originally produced) must still validate and
    persist - Module 13's additions must not be a breaking change."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        requirements=[{"category": "functional", "description": "Legacy-shaped requirement.", "priority": "medium"}],
        risks=[
            {
                "category": "operational",
                "description": "Legacy-shaped risk.",
                "severity": "low",
                "probability": 0.2,
                "score": 20,
                "explanation": "",
                "mitigation": "None needed.",
            }
        ],
    )
    import json as _json

```



```

_patch_provider(monkeypatch, _FakeProvider(_json.dumps(response_json)))
analyze_response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
assert analyze_response.status_code == 201

detail = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
# Defaults kick in rather than a validation failure.
assert detail["requirements"][0]["certainty"] == "inferred"
assert detail["requirements"][0]["confidence"] == 70.0
assert detail["risks"][0]["certainty"] == "inferred"
assert detail["risks"][0]["confidence"] == 70.0

def test_garbled_certainty_value_falls_back_to_inferred(monkeypatch):
    """A weaker model that writes something other than known/inferred/
    unknown shouldn't fail validation - it should fall back to the safe
    "inferred" default, same pattern as every other enum field here."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        requirements=[
            {
                "category": "functional",
                "description": "Garbled certainty value.",
                "priority": "medium",
                "certainty": "maybe kind of sure",
            }
        ],
    )
    import json as _json

    _patch_provider(monkeypatch, _FakeProvider(_json.dumps(response_json)))
    assert client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers).json()
    assert detail["requirements"][0]["certainty"] == "inferred"

```

tests/test_analysis_delta.py

302 lines

```
"""Verifies Module 13 Phase 2 (AI Analysis 2.0 - analysis history +
version-to-version comparison):

* GET /{id}/analyses - lightweight history list, newest first
* GET /{id}/analysis/compare - a computed diff between two analyses,
including the "significant change detected" flag/reasons and the
best-effort added/removed requirement & affected-component lists

The real AI provider is never called - same monkeypatched _FakeProvider
pattern as test_analysis_engine.py and test_ai_analysis_v2.py.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"delta-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Delta Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested_by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _analyze(headers, cr_id, response_dict, monkeypatch):
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_dict)))
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return response.json()

# --- Two canned responses: a low-risk first pass, then a materially
# riskier second pass after the request "grew" - mirrors the spec's own
# example (Version 3 -> Version 4, "Financial transactions were added to
# scope", Risk: Medium -> High). -----
FIRST_PASS = {
    "summary": "Adds OTP-based authentication at login.",
    "classification": {"category": "Security", "confidence": 0.8, "reason": "New authentication factor."},
    "requirements": [
        {"category": "functional", "description": "OTP is sent to the customer's registered mobile number."},
    ],
    "affected_components": [
        {
            "name": "Authentication Service",
            "type": "backend",
            "impact_level": "medium",
            "reason": "Adds OTP verification.",
            "confidence": 70,
        }
    ],
    "dependencies": [],
    "risks": [
        {
            "category": "security",
            "description": "OTP interception risk.",
            "severity": "medium",
            "probability": 0.3,
            "score": 40,
            "explanation": "Standard OTP risk.",
            "mitigation": "Use a short expiry window.",
        }
    ],
    "security_analysis": {"concerns": [], "summary": ""},
    "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
    "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs OTP expiry defined."},
}
```

```

# Second pass: scope grew to include a financial/payment step, materially
# raising risk - same change request, re-analyzed after an edit.
SECOND_PASS = {
    "summary": "Adds OTP-based authentication at login, now including a payment confirmation step.",
    "classification": {"category": "Security", "confidence": 0.8, "reason": "New authentication factor."},
    "requirements": [
        {"category": "functional", "description": "OTP is sent to the customer's registered mobile number."},
        {"category": "functional", "description": "OTP must also confirm high-value payment transactions."},
    ],
    "affected_components": [
        {
            "name": "Authentication Service",
            "type": "backend",
            "impact_level": "high",
            "reason": "Adds OTP verification.",
            "confidence": 75,
        },
        {
            "name": "Payment Gateway",
            "type": "third_party",
            "impact_level": "high",
            "reason": "OTP now confirms payments.",
            "confidence": 65,
        },
    ],
    "dependencies": [],
    "risks": [
        {
            "category": "security",
            "description": "OTP interception risk.",
            "severity": "high",
            "probability": 0.4,
            "score": 70,
            "explanation": "Now guards a financial transaction, not just login.",
            "mitigation": "Use a short expiry window and rate limiting.",
        },
        {
            "category": "compliance",
            "description": "Payment confirmation via OTP may need compliance review.",
            "severity": "high",
            "probability": 0.5,
            "score": 60,
            "explanation": "Financial transactions were added to scope.",
            "mitigation": "Confirm with compliance before implementation.",
        },
    ],
    "security_analysis": {"concerns": ["Financial transactions now in scope"], "summary": "Treat as high-risk."},
    "complexity": {"level": "high", "reasoning": "Now touches payments as well as auth."},
    "effort": {"backend": "4 days", "frontend": "2 days", "testing": "2 days", "total": "8 developer-days"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "requires_clarification", "reasoning": "Compliance review needed first."},
}

def test_list_analyses_returns_newest_first(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    second = _analyze(headers, created["id"], SECOND_PASS, monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/analyses", headers=headers)
    assert response.status_code == 200
    items = response.json()
    assert len(items) == 2
    assert items[0]["id"] == second["id"] # newest first

def test_compare_with_no_params_diffs_two_most_recent(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    _analyze(headers, created["id"], SECOND_PASS, monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/analysis/compare", headers=headers)
    assert response.status_code == 200
    body = response.json()

    fields = {f["field"]: f for f in body["fields"]}
    assert fields["risk_bucket"]["old_value"] == "Medium"
    assert fields["risk_bucket"]["new_value"] == "High"
    assert fields["risk_bucket"]["status"] == "changed"
    assert fields["complexity"]["old_value"] == "Medium"
    assert fields["complexity"]["new_value"] == "High"
    assert fields["recommendation"]["old_value"] == "approve with conditions"
    assert fields["recommendation"]["new_value"] == "requires_clarification"
    assert fields["affected_component_count"]["old_value"] == "1"
    assert fields["affected_component_count"]["new_value"] == "2"
    assert fields["requirement_count"]["old_value"] == "1"
    assert fields["requirement_count"]["new_value"] == "2"

    assert body["is_significant_change"] is True
    assert any("Risk moved from medium to high" in r for r in body["significant_change_reasons"])

    assert "OTP must also confirm high-value payment transactions." in body["requirements_added"]
    assert body["requirements_removed"] == []
    assert "Payment Gateway" in body["affected_components_added"]
    assert body["affected_components_removed"] == []

def test_compare_explicit_ids_regardless_of_order(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    first = _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    second = _analyze(headers, created["id"], SECOND_PASS, monkeypatch)

    # Pass them backwards - the endpoint must still compare chronologically
    # (old -> new), not literally "from" -> "to" as given.
    response = client.get(
        f"/api/change-requests/{created['id']}/analysis/compare",
        params={"from": second["id"], "to": first["id"]},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["from_analysis_id"] == first["id"]
    assert body["to_analysis_id"] == second["id"]
    fields = {f["field"]: f for f in body["fields"]}
    assert fields["risk_bucket"]["old_value"] == "Medium"
    assert fields["risk_bucket"]["new_value"] == "High"

def test_compare_returns_409_with_only_one_analysis(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/analysis/compare", headers=headers)
    assert response.status_code == 409

```

```

def test_compare_returns_404_for_analysis_from_a_different_change_request(monkeypatch):
    headers = _auth()
    created_a = _create_change_request(headers, title="Change request Alpha")
    created_b = _create_change_request(headers, title="Change request Beta")
    _analyze(headers, created_a["id"], FIRST_PASS, monkeypatch)
    _analyze(headers, created_a["id"], SECOND_PASS, monkeypatch)
    other = _analyze(headers, created_b["id"], FIRST_PASS, monkeypatch)

    response = client.get(
        f"/api/change-requests/{created_a['id']}/analysis/compare",
        params={"to": other["id"]},
        headers=headers,
    )
    assert response.status_code == 404

def test_compare_same_id_twice_returns_400(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    second = _analyze(headers, created["id"], SECOND_PASS, monkeypatch)

    response = client.get(
        f"/api/change-requests/{created['id']}/analysis/compare",
        params={"from": second["id"], "to": second["id"]},
        headers=headers,
    )
    assert response.status_code == 400

def test_no_significant_change_when_nothing_material_moved(monkeypatch):
    """Re-analyzing with an identical-shaped result (same risk bucket,
    complexity, recommendation, and recommended approvals) should not be
    flagged as a significant change, even though it's technically a new
    analysis row."""
    headers = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)
    _analyze(headers, created["id"], FIRST_PASS, monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/analysis/compare", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert body["is_significant_change"] is False
    assert body["significant_change_reasons"] == []

```

tests/test_analysis_engine.py

457 lines

```
"""Verifies Module 6 (AI Analysis Engine): POST /analyze and GET /analysis,
prompt->JSON->Pydantic validation, persistence across every child table,
and graceful handling of a not-configured provider, invalid JSON, a
schema-validation failure, and a timeout.

The real Anthropic API is never called here - `get_ai_provider()` is
monkeypatched with a fake provider that returns canned text, so these tests
run offline and don't cost API credits. Three distinct canned change
requests (a simple config tweak, a security-sensitive change, and one that
comes back needing clarification) exercise different parts of the schema,
satisfying the "test with at least 3 different change requests" requirement
without spending real API calls on every test run - the user separately
verifies the same endpoint live against the real provider.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.database.session import SessionLocal
from app.main import app
from app.models import Analysis

client = TestClient(app)

def _unique_email() -> str:
    return f"ai-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "AI Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add loyalty points to checkout",
        "description": "Award loyalty points automatically when a customer completes checkout on the POS.",
        "business_objective": "Increase repeat visits by rewarding customers automatically.",
        "priority": "medium",
        "requested_by": "Morgan Lee",
        "target_system": "POS Backend",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeProvider:
    """Stands in for AnthropicProvider in tests - returns canned text (or
    raises a canned exception) instead of calling the real API."""
    def __init__(self, response_text=None, configured=True, raise_exc=None):
        self._text = response_text
        self._configured = configured
        self._raise = raise_exc

    def is_configured(self) -> bool:
        return self._configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

# --- Three distinct canned AI responses -----
"""Change request #1: a small, low-risk configuration change.
CONFIG CHANGE RESPONSE = {
    "summary": "Adds a configurable loyalty-points multiplier to the POS checkout flow.",
    "classification": {"category": "Configuration", "confidence": 0.86, "reason": "A tunable value, no new flow."},
    "requirements": [
        {"category": "business objective", "description": "Increase repeat visits.", "priority": "medium"},
        {"category": "functional", "description": "Award points on checkout completion.", "priority": "high"},
    ],
    "affected_components": [
        {
            "name": "Checkout Service",
            "type": "backend",
            "impact_level": "low",
            "reason": "Reads a new config value at checkout time.",
            "confidence": 70,
        }
    ],
    "dependencies": [],
    "risks": [
        {
            "category": "operational",
            "description": "Misconfigured multiplier could award incorrect points.",
            "severity": "low",
            "probability": 0.2,
            "score": 15,
            "explanation": "Low blast radius, easy to correct.",
            "mitigation": "Validate the multiplier value before saving it.",
        }
    ],
    "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "low", "reasoning": "Single config value, existing code path."},
    "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
    "missing_information": [],
    "test_cases": [

```

```

    {
      "id": "TC-001",
      "title": "Checkout awards multiplied points",
      "type": "integration",
      "priority": "high",
      "description": "Complete a checkout and verify points awarded match the configured multiplier.",
      "expected_result": "Points awarded equal base points * multiplier.",
    }
  ],
  "implementation_plan": [
    {
      "task": "Add multiplier config field",
      "description": "Add a validated numeric config field for the multiplier.",
      "component": "Checkout Service",
      "priority": "high",
      "estimated_effort": "4h",
      "dependencies": "None",
    }
  ],
  "recommendation": {"decision": "approve", "reasoning": "Small, well-scoped, low risk."},
}

# Change request #2: a security-sensitive change, high risk.
SECURITY_CHANGE_RESPONSE = {
  "summary": "Adds an admin API endpoint to refund a customer's order without manager approval.",
  "classification": {"category": "Security", "confidence": 0.74, "reason": "Bypasses an existing approval control."},
  "requirements": [
    {"category": "constraint", "description": "Must not remove the existing audit log entry.", "priority": "critical"},
  ],
  "affected_components": [
    {
      "name": "Admin API",
      "type": "api",
      "impact_level": "high",
      "reason": "New endpoint with elevated privileges.",
      "confidence": 65,
    },
    {
      "name": "Orders Database",
      "type": "database",
      "impact_level": "medium",
      "reason": "Refund writes bypass the approval-gated path.",
      "confidence": 60,
    },
  ],
  "dependencies": [
    {"name": "Auth Service", "type": "service", "impact_level": "medium", "reason": "Needs a new permission scope."}
  ],
  "risks": [
    {
      "category": "security",
      "description": "Removing manager approval increases fraud/misuse risk.",
      "severity": "critical",
      "probability": 0.5,
      "score": 88,
      "explanation": "No compensating control described.",
      "mitigation": "Require a secondary approval or hard refund cap instead of removing the control.",
    },
    {
      "category": "compliance",
      "description": "May violate change-control policy for financial transactions.",
      "severity": "high",
      "probability": 0.4,
      "score": 60,
      "explanation": "Insufficient information about applicable policy.",
      "mitigation": "Confirm with compliance before implementation.",
    },
  ],
  "security_analysis": {
    "concerns": ["Privilege escalation risk", "Missing audit trail requirement is unclear"],
    "summary": "This request removes an existing control; treat as high-risk until reviewed.",
  },
  "complexity": {"level": "high", "reasoning": "Touches auth, API, and financial data."},
  "effort": {"backend": "3-5 days", "frontend": "1 day", "testing": "2-3 days", "total": "6-9 developer-days"},
  "missing_information": [
    {
      "question": "Why is manager approval being removed rather than streamlined?",
      "priority": "critical",
      "reason": "Directly affects the security assessment.",
    },
    {
      "question": "Is there a refund amount cap in mind?",
      "priority": "important",
      "reason": "Affects mitigation design.",
    },
  ],
  "test_cases": [
    {
      "id": "TC-001",
      "title": "Refund without approval is logged",
      "type": "security",
      "priority": "critical",
      "description": "Issue a refund via the new endpoint.",
      "expected_result": "An audit log entry is created with the acting admin's identity.",
    }
  ],
  "implementation_plan": [
    {
      "task": "Add refund endpoint",
      "description": "Add the new admin refund endpoint behind a new permission scope.",
      "component": "Admin API",
      "priority": "critical",
      "estimated_effort": "2d",
      "dependencies": "Auth Service permission scope",
    }
  ],
  "recommendation": {
    "decision": "requires_clarification",
    "reasoning": "Removing an approval control needs a documented reason and compensating mitigation first.",
  },
}

# Change request #3: too vague to say much - tests the "insufficient
# information" / low-confidence path.
VAGUE_CHANGE_RESPONSE = {
  "summary": "Requests an unspecified improvement to reporting; too vague to scope further.",
  "classification": {"category": "Performance", "confidence": 0.3, "reason": "No concrete detail provided."},
  "requirements": [
    {"category": "assumption", "description": "Assuming this refers to the existing Reporting Dashboard.", "priority": "low"}
  ],
  "affected_components": [],
  "dependencies": [],
  "risks": [],
  "security_analysis": {"concerns": [], "summary": "Insufficient information."},
  "complexity": {"level": "medium", "reasoning": "Insufficient information."},
  "effort": {
    "backend": "Insufficient information.",
    "frontend": "Insufficient information.",
    "testing": "Insufficient information.",
    "total": "Insufficient information.",
  },
}

```

```

"missing_information": [
    {
        "question": "Which report(s) and what specific improvement is requested?",
        "priority": "critical",
        "reason": "Cannot scope work without this.",
    },
    {
        "question": "Is there a performance target (e.g. load time)?",
        "priority": "nice to have",
        "reason": "Would help size the effort.",
    },
],
"test_cases": [],
"implementation_plan": [],
"recommendation": {
    "decision": "requires clarification",
    "reasoning": "Not enough detail to assess or plan.",
},
},
}

def test_analyze_requires_authentication():
    response = client.post("/api/change-requests/1/analyze")
    assert response.status_code == 401

def test_get_analysis_requires_authentication():
    response = client.get("/api/change-requests/1/analysis")
    assert response.status_code == 401

def test_analyze_returns_404_for_unknown_change_request(monkeypatch):
    headers = _auth()
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(CONFIG_CHANGE_RESPONSE)))
    response = client.post(f"/api/change-requests/999999999/analyze", headers=headers)
    assert response.status_code == 404

def test_get_analysis_before_any_analysis_returns_404():
    headers = _auth()
    created = _create_change_request(headers)
    response = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert response.status_code == 404

def test_analyze_returns_503_when_provider_not_configured(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FakeProvider(configured=False))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 503

def test_analyze_returns_422_on_invalid_json(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FakeProvider("this is not json at all"))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 422

def test_analyze_strips_markdown_fences_before_parsing(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    fenced = "```\njson\n```\n" + json.dumps(CONFIG_CHANGE_RESPONSE) + "\n```\n"
    _patch_provider(monkeypatch, _FakeProvider(fenced))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

def test_analyze_returns_422_on_schema_validation_failure(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    broken = dict(CONFIG_CHANGE_RESPONSE)
    broken["classification"] = {"category": "Configuration", "confidence": 2.5, "reason": "out of range"}
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(broken)))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 422

def test_analyze_returns_504_on_timeout(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    class _Timeout(Exception):
        pass

    _Timeout.__name__ = "APITimeoutError"
    _patch_provider(monkeypatch, _FakeProvider(raise_exc=_Timeout("took too long")))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 504

def test_analyze_config_change_persists_low_risk_result(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers, title="Loyalty multiplier config change")
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(CONFIG_CHANGE_RESPONSE)))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    assert body["category"] == "Configuration"
    assert body["complexity"] == "low"
    assert body["recommendation"] == "approve"
    assert 0 < body["risk_score"] <= 20
    assert len(body["requirements"]) == 2
    assert len(body["affected_components"]) == 1
    assert len(body["risks"]) == 1
    assert len(body["test_cases"]) == 1
    assert len(body["implementation_tasks"]) == 1
    assert body["clarification_questions"] == []
    assert body["effort_estimate"].startswith("Backend: 1 day")

def test_analyze_security_change_persists_high_risk_and_clarification(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers, title="Refund endpoint without manager approval")
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(SEcurity_CHANGE_RESPONSE)))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    assert body["category"] == "Security"
    assert body["complexity"] == "high"

```

```

assert body["recommendation"] == "requires clarification"
assert body["risk_score"] == 88 # max() of the two risk scores (88, 60)
assert len(body["risks"]) == 2
assert len(body["dependencies"]) == 1
assert len(body["affected_components"]) == 2

priorities = {q["priority"] for q in body["clarification_questions"]}
assert priorities == {"critical", "high"} # critical->critical, important->high

security = json.loads(body["security_analysis"])
assert len(security["concerns"]) == 2

def test_analyze_vague_change_low_confidence_and_missing_info(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers, title="Improve reporting somehow")
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(VAGUE_CHANGE_RESPONSE)))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    assert body["confidence_score"] == 30 # 0.3 * 100
    assert body["risk_score"] == 5.0 # no risks returned -> low default
    assert len(body["clarification_questions"]) == 2
    priorities = {q["priority"] for q in body["clarification_questions"]}
    assert priorities == {"critical", "low"} # critical->critical, nice_to_have->low

def test_reanalyze_creates_new_row_and_get_analysis_returns_latest(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers, title="Re-analyzed request")

    _patch_provider(monkeypatch, _FakeProvider(json.dumps(CONFIG_CHANGE_RESPONSE)))
    first = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert first.status_code == 201

    _patch_provider(monkeypatch, _FakeProvider(json.dumps(SEcurity_CHANGE_RESPONSE)))
    second = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert second.status_code == 201
    assert second.json()["id"] != first.json()["id"]

    db = SessionLocal()
    try:
        count = db.query(Analysis).filter(Analysis.change_request_id == created["id"]).count()
        assert count == 2
    finally:
        db.close()

    latest = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert latest.status_code == 200
    assert latest.json()["id"] == second.json()["id"]
    assert latest.json()["category"] == "Security"

def test_change_request_detail_reflects_new_analysis(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers, title="Detail view picks up new analysis")
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(SEcurity_CHANGE_RESPONSE)))

    analyze_response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze_response.status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers)
    assert detail.status_code == 200
    body = detail.json()
    assert body["effective_status"] == "requires clarification"
    assert body["latest_analysis"]["category"] == "Security"
    assert body["latest_analysis"]["recommendation"] == "requires clarification"
    assert len(body["latest_analysis"]["clarification_questions"]) == 2

```


tests/test_auth.py

103 lines

```
"""Verifies the authentication endpoints against the real running app (same
pattern as test_health.py). Each test uses a fresh, randomly-generated
email so re-running the suite never collides with a previous run's data.

Note: this writes real (if randomly-emailed) user rows into your actual
backend/data/app.db each time you run it - harmless, and consistent with
how test_health.py already works, but it means that file will accumulate
test users over time. Delete it any time to reset; tables are recreated
automatically on next run.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db() # make sure the users table exists even if init_db was never run manually

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"test-{uuid.uuid4().hex[:10]}@example.com"

def test_register_login_and_me_flow():
    email = _unique_email()
    password = "supersecret123"

    register_response = client.post(
        "/auth/register",
        json={"name": "Test User", "email": email, "password": password, "confirm_password": password},
    )
    assert register_response.status_code == 201
    register_body = register_response.json()
    assert register_body["user"]["email"] == email
    assert "access_token" in register_body

    login_response = client.post(
        "/auth/login", json={"email": email, "password": password, "remember_me": False}
    )
    assert login_response.status_code == 200
    token = login_response.json()["access_token"]

    me_response = client.get("/auth/me", headers={"Authorization": f"Bearer {token}"})
    assert me_response.status_code == 200
    assert me_response.json()["email"] == email

    logout_response = client.post("/auth/logout", headers={"Authorization": f"Bearer {token}"})
    assert logout_response.status_code == 200

def test_register_rejects_duplicate_email():
    email = _unique_email()
    password = "supersecret123"
    body = {"name": "Test User", "email": email, "password": password, "confirm_password": password}

    first = client.post("/auth/register", json=body)
    assert first.status_code == 201

    second = client.post("/auth/register", json=body)
    assert second.status_code == 409

def test_register_rejects_mismatched_passwords():
    response = client.post(
        "/auth/register",
        json={
            "name": "Test User",
            "email": _unique_email(),
            "password": "supersecret123",
            "confirm_password": "different123",
        },
    )
    assert response.status_code == 422

def test_login_rejects_wrong_password():
    email = _unique_email()
    password = "supersecret123"
    client.post(
        "/auth/register",
        json={"name": "Test User", "email": email, "password": password, "confirm_password": password},
    )

    response = client.post(
        "/auth/login", json={"email": email, "password": "wrongpassword", "remember_me": False}
    )
    assert response.status_code == 401

def test_me_requires_authentication():
    response = client.get("/auth/me")
    assert response.status_code == 401
```

tests/test_change_request_editing.py

379 lines

```
"""Verifies Module 12 Phase 2 (Enterprise Workflow - Editing, Versioning,
Audit Trail, AI-outdated detection):

* PUT /api/change-requests/{id} - partial-update editing, permission
  check, no-op edits create no version/history noise
* field-level history (app/services/history.py + versioning.py) records
  the real old/new value per changed field, not one vague "updated" line
* GET .../history, GET .../versions, GET .../versions/{n}, GET .../compare
* "is the AI analysis outdated" - becomes true after an edit, becomes
  false again after re-analyzing (uses the same FakeProvider/monkeypatch
  pattern as tests/test_analysis_engine.py so no real API key/call is
  needed)
* the first successful analysis auto-transitions status from
  Pending Analysis -> Analyzed
* tags round-trip through create and edit

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.database.session import SessionLocal
from app.main import app
from app.models.change_request import ChangeRequest

client = TestClient(app)

def unique_email() -> str:
    return f"edit-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Editing Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP login to mobile app",
        "description": "Allow customers to log in using a one-time passcode sent to their registered number.",
        "priority": "medium",
        "requested by": "Morgan Lee",
        "target_system": "Mobile App",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeProvider:
    """Same stand-in used in tests/test_analysis_engine.py - returns canned
    text instead of calling the real AI provider."""

    def __init__(self, response_text=None, configured=True, raise_exc=None):
        self.text = response_text
        self.configured = configured
        self.raise = raise_exc

    def is_configured(self) -> bool:
        return self.configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self.raise is not None:
            raise self.raise
        return self.text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

# A minimal, valid AIAnalysisResult - only the editing/versioning behavior
# is under test here, not the analysis content itself (that's Module 6's
# own test file), so this stays as small as the schema allows.
_MINIMAL_ANALYSIS_RESPONSE = {
    "summary": "Adds OTP-based login to the mobile app.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.8, "reason": "New auth flow."},
    "requirements": [],
    "affected components": [],
    "dependencies": [],
    "risks": [],
    "security analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "medium", "reasoning": "New auth flow, moderate scope."},
    "effort": {"backend": "2 days", "frontend": "2 days", "testing": "1 day", "total": "5 developer-days"},
    "missing information": [],
    "test cases": [],
    "implementation plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Well-scoped."},
}

# --- Basic auth / not-found guards -----

def test_edit_requires_authentication():
    response = client.put("/api/change-requests/1", json={"title": "New title here"})
    assert response.status_code == 401

def test_edit_returns_404_for_unknown_change_request():
    headers = _auth()
    response = client.put(
        "/api/change-requests/999999999", json={"title": "New title here"}, headers=headers
    )
    assert response.status_code == 404
```

```
# --- Editing / versioning -----

def test_creator_can_edit_and_version_increments():
    headers = _auth()
    created = _create_change_request(headers)
    assert created["current_version"] == 1

    response = client.put(
        f"/api/change-requests/{created['id']}",
        json={"title": "Add OTP login to mobile checkout"},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()

    assert body["new_version"] == 2
    assert body["change_request"]["current_version"] == 2
    assert body["change_request"]["title"] == "Add OTP login to mobile checkout"
    assert len(body["changes"]) == 1
    assert body["changes"][0]["field"] == "title"
    assert body["changes"][0]["old_value"] == "Add OTP login to mobile app"
    assert body["changes"][0]["new_value"] == "Add OTP login to mobile checkout"

def test_edit_with_no_real_changes_creates_no_new_version():
    headers = _auth()
    created = _create_change_request(headers)

    # Re-submitting the exact same values that are already stored is a
    # no-op edit - shouldn't create version 2 or any history noise.
    response = client.put(
        f"/api/change-requests/{created['id']}",
        json={
            "title": created["title"],
            "priority": created["priority"],
            "requested_by": created["requested_by"],
            "target_system": created["target_system"],
        },
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()

    assert body["changes"] == []
    assert body["new_version"] is None
    assert body["change_request"]["current_version"] == 1

def test_edit_multiple_fields_records_one_history_event_per_field():
    headers = _auth()
    created = _create_change_request(headers)

    response = client.put(
        f"/api/change-requests/{created['id']}",
        json={"priority": "high", "environment": "Production"},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert {c["field"] for c in body["changes"]} == {"priority", "environment"}

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    field_changed_events = [h for h in history if h["action"] == "field changed"]
    assert {e["field_name"] for e in field_changed_events} == {"priority", "environment"}

    priority_event = next(e for e in field_changed_events if e["field_name"] == "priority")
    assert priority_event["old_value"] == "Medium"
    assert priority_event["new_value"] == "High"

    env_event = next(e for e in field_changed_events if e["field_name"] == "environment")
    assert env_event["old_value"] is None
    assert env_event["new_value"] == "Production"

    # A VERSION CREATED event is also recorded alongside the field changes.
    assert any(h["action"] == "version_created" for h in history)

def test_unrelated_user_cannot_edit():
    headers = _auth()
    created = _create_change_request(headers)
    other_headers = _auth()

    response = client.put(
        f"/api/change-requests/{created['id']}",
        json={"title": "Hijacked title change"},
        headers=other_headers,
    )
    assert response.status_code == 403

# --- History / versions / compare endpoints -----

def test_history_returns_events_newest_first():
    headers = _auth()
    created = _create_change_request(headers)
    client.put(f"/api/change-requests/{created['id']}", json={"priority": "high"}, headers=headers)
    client.put(f"/api/change-requests/{created['id']}", json={"priority": "critical"}, headers=headers)

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert len(history) >= 3 # created + 2 edits (each edit adds a version_created + field_changed pair)
    timestamps = [h["created_at"] for h in history]
    assert timestamps == sorted(timestamps, reverse=True)
    assert history[-1]["action"] == "created" # oldest event is always creation

def test_versions_list_and_get_single_version():
    headers = _auth()
    created = _create_change_request(headers)
    client.put(f"/api/change-requests/{created['id']}", json={"priority": "high"}, headers=headers)

    versions = client.get(f"/api/change-requests/{created['id']}/versions", headers=headers).json()
    assert [v["version_number"] for v in versions] == [2, 1] # newest first

    v1 = client.get(f"/api/change-requests/{created['id']}/versions/1", headers=headers)
    assert v1.status_code == 200
    assert v1.json()["snapshot"]["priority"] == "medium"

    v2 = client.get(f"/api/change-requests/{created['id']}/versions/2", headers=headers)
    assert v2.status_code == 200
    assert v2.json()["snapshot"]["priority"] == "high"

    missing = client.get(f"/api/change-requests/{created['id']}/versions/99", headers=headers)
    assert missing.status_code == 404

def test_compare_versions_endpoint():
    headers = _auth()
```

```

created = _create_change_request(headers)
client.put(
    f"/api/change-requests/{created['id']}",
    json={"priority": "high", "business_impact": "Improves conversion"},
    headers=headers,
)

response = client.get(
    f"/api/change-requests/{created['id']}/compare",
    params={"from": 1, "to": 2},
    headers=headers,
)
assert response.status_code == 200
body = response.json()
assert body["from_version"] == 1
assert body["to_version"] == 2

fields = {f["field"]: f for f in body["fields"]}
assert fields["priority"]["status"] == "changed"
assert fields["priority"]["old_value"] == "Medium"
assert fields["priority"]["new_value"] == "High"
assert fields["business_impact"]["status"] == "added"
assert fields["title"]["status"] == "unchanged"

# --- AI-outdated detection + re-analyze -----

def test_first_analysis_transitions_status_to_analyzed(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    assert created["status"] == "pending_analysis"
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_MINIMAL_ANALYSIS_RESPONSE)))

    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze.status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["status"] == "analyzed"
    assert detail["is_analysis_outdated"] is False

def test_editing_after_analysis_marks_it_outdated_then_reanalyze_clears_it(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_MINIMAL_ANALYSIS_RESPONSE)))

    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze.status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["is_analysis_outdated"] is False
    assert detail["latest_analysis"]["change_request_version"] == 1

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["is_analysis_outdated"] is True

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["is_analysis_outdated"] is True
    assert detail["current_version"] == 2

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert any(h["action"] == "ai_analysis_invalidated" for h in history)

    # Re-analyzing brings it back up to date, tagged with the new version.
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_MINIMAL_ANALYSIS_RESPONSE)))
    reanalyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert reanalyze.status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["is_analysis_outdated"] is False
    assert detail["latest_analysis"]["change_request_version"] == 2

def test_reanalyzing_a_cr_already_past_analyzed_does_not_move_status_backwards(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_MINIMAL_ANALYSIS_RESPONSE)))
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)

    # Manually move the CR further along the lifecycle than Analyzed.
    with SessionLocal() as db:
        from app.models.enums import ChangeRequestStatus

        cr = db.get(ChangeRequest, created["id"])
        cr.status = ChangeRequestStatus.IN_REVIEW
        db.commit()

    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_MINIMAL_ANALYSIS_RESPONSE)))
    reanalyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert reanalyze.status_code == 201

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["status"] == "in_review" # unchanged - not forced back to "analyzed"

# --- Tags round-trip -----

def test_tags_round_trip_through_create_and_edit():
    headers = _auth()
    created = _create_change_request(headers, tags=["mobile", "auth"])
    assert created["tags"] == ["mobile", "auth"]

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["tags"] == ["mobile", "auth"]

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"tags": ["mobile", "auth", "otp"]},
        headers=headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["tags"] == ["mobile", "auth", "otp"]
    assert edit.json()["changes"][0]["field"] == "tags"

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    assert detail["tags"] == ["mobile", "auth", "otp"]

```

tests/test_change_request_management.py

286 lines

```
"""Verifies Module 5 (Change Request Management): the list endpoint's
search/filter/sort/pagination, the computed "effective status" values, and
the enriched detail endpoint (latest analysis + clarification questions).

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.database.session import SessionLocal
from app.main import app
from app.models import Analysis, ChangeRequest, ClarificationQuestion
from app.models.enums import ChangeRequestStatus, ComplexityLevel, Priority

client = TestClient(app)

def _unique_email() -> str:
    return f"crm-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "CRM Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return ("Authorization": f"Bearer {body['access_token']}", body["user"]["id"])

def _create_via_api(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add loyalty points to checkout",
        "description": "Award loyalty points automatically when a customer checks out.",
        "priority": "medium",
        "requested by": "Morgan Lee",
        "target_system": "POS Backend",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def test_list_response_has_items_total_limit_offset():
    headers, _ = _auth()
    response = client.get("/api/change-requests", headers=headers)
    assert response.status_code == 200
    body = response.json()
    for key in ("items", "total", "limit", "offset"):
        assert key in body

def test_search_matches_title():
    headers, _ = _auth()
    unique_word = f"Zephyr/{uuid.uuid4().hex[:6]}"
    _create_via_api(headers, title=f"{unique_word} sync improvements")

    response = client.get("/api/change-requests", params={"search": unique_word}, headers=headers)
    assert response.status_code == 200
    items = response.json()["items"]
    assert len(items) >= 1
    assert all(unique_word.lower() in item["title"].lower() for item in items)

def test_search_matches_ticket_number():
    headers, _ = _auth()
    created = _create_via_api(headers, title=f"Ticket search target {uuid.uuid4().hex[:6]}")
    ticket_id = created["id"]

    # Searching with the "#" prefix the UI displays should find it by id,
    # even though the id doesn't appear in the title/description/etc.
    response = client.get("/api/change-requests", params={"search": f"#{ticket_id}"}, headers=headers)
    assert response.status_code == 200
    items = response.json()["items"]
    assert any(item["id"] == ticket_id for item in items)

    # Also works without the "#"
    response = client.get("/api/change-requests", params={"search": str(ticket_id)}, headers=headers)
    assert response.status_code == 200
    items = response.json()["items"]
    assert any(item["id"] == ticket_id for item in items)

def test_filter_by_priority():
    headers, _ = _auth()
    _create_via_api(headers, title="Critical priority filter check", priority="critical")

    response = client.get("/api/change-requests", params={"priority": "critical"}, headers=headers)
    assert response.status_code == 200
    items = response.json()["items"]
    assert len(items) >= 1
    assert all(item["priority"] == "critical" for item in items)

def test_filter_by_assigned_to_me():
    """Module 12 Phase 6: GET /api/change-requests?assigned_to_me=true -
    only change requests the CURRENT user is assigned to, in any role."""
    creator_headers, _ = _auth()
    assignee_headers, assignee_id = _auth()
    bystander_headers, _ = _auth()

    created = _create_via_api(creator_headers, title=f"Assigned-to-me filter check {uuid.uuid4().hex[:6]}")
    response = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )
    assert response.status_code == 201

    # The assignee sees it when filtering to their own assignments.
    response = client.get("/api/change-requests", params={"assigned_to_me": "true"}, headers=assignee_headers)
```

```

assert response.status_code == 200
assert any(item["id"] == created["id"] for item in response.json()["items"])

# Someone never assigned to anything does not see it.
response = client.get("/api/change-requests", params={"assigned_to_me": "true"}, headers=bystander_headers)
assert response.status_code == 200
assert all(item["id"] != created["id"] for item in response.json()["items"])

# Without the filter, everyone can still see it (it's a real, visible
# change request - assigned to me only narrows, it never hides).
response = client.get("/api/change-requests", headers=bystander_headers)
assert response.status_code == 200
assert any(item["id"] == created["id"] for item in response.json()["items"])

def test_new_request_has_pending_analysis_effective_status():
    headers, _ = auth()
    created = _create_via_api(headers, title="Freshly created, not analyzed yet")

    response = client.get("/api/change-requests", params={"status": "pending_analysis"}, headers=headers)
    assert response.status_code == 200
    ids = {item["id"] for item in response.json()["items"]}
    assert created["id"] in ids

def test_invalid_status_filter_returns_422():
    headers, _ = auth()
    response = client.get("/api/change-requests", params={"status": "not_a_real_status"}, headers=headers)
    assert response.status_code == 422

def test_effective_status_transitions_with_analysis_and_clarifications():
    headers, user_id = auth()
    created = _create_via_api(headers, title="Needs clarification before approval")

    db = SessionLocal()
    try:
        analysis = Analysis(
            change_request_id=created["id"],
            summary="Touches customer data - needs a rollback plan confirmed.",
            category="security",
            complexity=ComplexityLevel.HIGH,
            risk_score=70.0,
            confidence_score=65.0,
        )
        db.add(analysis)
        db.flush()
        db.add(
            ClarificationQuestion(
                analysis_id=analysis.id,
                question="What's the rollback plan?",
                priority=Priority.HIGH,
                reason="Needed before approval.",
                resolved=False,
            )
        )
        db.commit()
    finally:
        db.close()

    response = client.get("/api/change-requests", params={"status": "requires_clarification"}, headers=headers)
    assert response.status_code == 200
    ids = {item["id"] for item in response.json()["items"]}
    assert created["id"] in ids

    # Resolve the question - it should move to "completed".
    db = SessionLocal()
    try:
        analysis_row = db.query(Analysis).filter(Analysis.change_request_id == created["id"]).first()
        question = (
            db.query(ClarificationQuestion)
            .filter(ClarificationQuestion.analysis_id == analysis_row.id)
            .first()
        )
        question.resolved = True
        db.commit()
    finally:
        db.close()

    response = client.get("/api/change-requests", params={"status": "completed"}, headers=headers)
    assert response.status_code == 200
    ids = {item["id"] for item in response.json()["items"]}
    assert created["id"] in ids

def test_approved_status_overrides_analysis_state():
    headers, user_id = auth()
    created = _create_via_api(headers, title="Already approved by a reviewer")

    db = SessionLocal()
    try:
        cr = db.get(ChangeRequest, created["id"])
        cr.status = ChangeRequestStatus.APPROVED
        db.commit()
    finally:
        db.close()

    response = client.get("/api/change-requests", params={"status": "approved"}, headers=headers)
    assert response.status_code == 200
    ids = {item["id"] for item in response.json()["items"]}
    assert created["id"] in ids

def test_sort_newest_orders_most_recent_first():
    headers, _ = auth()
    _create_via_api(headers, title="Sort check A")
    second = _create_via_api(headers, title="Sort check B")

    response = client.get("/api/change-requests", params={"sort": "newest", "limit": 1}, headers=headers)
    assert response.status_code == 200
    items = response.json()["items"]
    assert items[0]["id"] == second["id"]

def test_detail_endpoint_includes_latest_analysis_and_questions():
    headers, _ = auth()
    created = _create_via_api(headers, title="Detail view with an analysis")

    db = SessionLocal()
    try:
        analysis = Analysis(
            change_request_id=created["id"],
            summary="Small, contained change.",
            category="Bug Fix",
            complexity=ComplexityLevel.LOW,
            risk_score=15.0,
            confidence_score=90.0,
        )
        db.add(analysis)
        db.flush()
        db.add(

```

```

        ClarificationQuestion(
            analysis_id=analysis.id,
            question="Any edge cases with refunds?",
            priority=Priority.MEDIUM,
            reason="Worth confirming before shipping.",
            resolved=True,
        )
    )
    db.commit()
finally:
    db.close()

response = client.get(f"/api/change-requests/{created['id']}", headers=headers)
assert response.status_code == 200
body = response.json()
assert body["effective_status"] == "completed"
assert body["latest_analysis"] is not None
assert body["latest_analysis"]["category"] == "Bug Fix"
assert len(body["latest_analysis"]["clarification_questions"]) == 1

def test_detail_endpoint_without_analysis_has_null_latest_analysis():
    headers, _ = _auth()
    created = _create_via_api(headers, title="Detail view without an analysis yet")

    response = client.get(f"/api/change-requests/{created['id']}", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert body["effective_status"] == "pending_analysis"
    assert body["latest_analysis"] is None

```

tests/test_change_requests.py

124 lines

```
"""Verifies the change-request creation workflow (Module 4): auth is
required, valid submissions are persisted with status "pending_analysis",
validation rejects bad input, and the list/get-one endpoints return what was
just created.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from datetime import date, timedelta
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db() # make sure every table/column exists even if init_db was never run manually

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"cr-{uuid.uuid4().hex[:10]}@example.com"

def _auth_headers() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "CR Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    token = response.json()["access_token"]
    return {"Authorization": f"Bearer {token}"}

def _valid_payload(**overrides) -> dict:
    payload = {
        "title": "Allow OTP login for customers",
        "description": "Allow customers to log in using OTP sent to their registered mobile number.",
        "business_objective": "Reduce password-reset support tickets.",
        "priority": "high",
        "requested_by": "Jamie Rivera",
        "target_system": "Mobile App",
        "desired_deadline": (date.today() + timedelta(days=14)).isoformat(),
    }
    payload.update(overrides)
    return payload

def test_create_requires_authentication():
    response = client.post("/api/change-requests", json=_valid_payload())
    assert response.status_code == 401

def test_create_change_request_succeeds_with_valid_data():
    headers = _auth_headers()
    response = client.post("/api/change-requests", json=_valid_payload(), headers=headers)
    assert response.status_code == 201
    body = response.json()
    assert body["title"] == "Allow OTP login for customers"
    assert body["status"] == "pending_analysis"
    assert body["requested_by"] == "Jamie Rivera"
    assert body["target_system"] == "Mobile App"
    assert "id" in body

def test_create_rejects_short_title():
    headers = _auth_headers()
    response = client.post("/api/change-requests", json=_valid_payload(title="Fix"), headers=headers)
    assert response.status_code == 422

def test_create_rejects_short_description():
    headers = _auth_headers()
    response = client.post(
        "/api/change-requests", json=_valid_payload(description="Too short."), headers=headers
    )
    assert response.status_code == 422

def test_create_rejects_missing_requested_by():
    headers = _auth_headers()
    response = client.post("/api/change-requests", json=_valid_payload(requested_by=" "), headers=headers)
    assert response.status_code == 422

def test_create_rejects_past_deadline():
    headers = _auth_headers()
    past_date = (date.today() - timedelta(days=1)).isoformat()
    response = client.post(
        "/api/change-requests", json=_valid_payload(desired_deadline=past_date), headers=headers
    )
    assert response.status_code == 422

def test_created_request_appears_in_list_and_get_one():
    headers = _auth_headers()
    create_response = client.post(
        "/api/change-requests", json=_valid_payload(title="Add barcode rescan option"), headers=headers
    )
    assert create_response.status_code == 201
    created_id = create_response.json()["id"]

    list_response = client.get("/api/change-requests", headers=headers)
    assert list_response.status_code == 200
    ids_in_list = {item["id"] for item in list_response.json()["items"]}
    assert created_id in ids_in_list

    get_response = client.get(f"/api/change-requests/{created_id}", headers=headers)
    assert get_response.status_code == 200
    assert get_response.json()["title"] == "Add barcode rescan option"

def test_get_one_returns_404_for_unknown_id():
    headers = _auth_headers()
    response = client.get("/api/change-requests/999999999", headers=headers)
    assert response.status_code == 404
```


tests/test_clarification_answer.py

247 lines

```
"""Verifies the clarification-question answer feature (a human supplying the
missing information the AI itself asked for, right on the Missing
Information tab):

* PATCH ../clarification-questions/{id}/answer - sets answer_text/
answered_by/answered_at and always marks the question resolved=True.
Open to anyone who can comment on the change request (Capability.
COMMENTS) - not gated to a reviewer role, since supplying a known fact
isn't the same as passing judgment on the AI's findings (contrast with
review requirement/update_security_finding_status in
test_module14_phase6.py, which ARE gated that way). Records
HistoryAction.CLARIFICATION_ANSWERED on the change request's audit
history. A blank/whitespace-only answer is rejected with 422. Can be
called again on an already-resolved question to correct the answer.

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as the other test_module14_*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def unique_email() -> str:
    return f"clarify-{uuid.uuid4().hex[:10]}@example.com"

def auth() -> tuple:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Clarification Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add bulk export to admin dashboard",
        "description": "Let Alight.com admins export the full customer list as a CSV from the dashboard.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Admin Dashboard",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds a CSV export button to the admin dashboard.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New export capability."},
        "requirements": [
            {
                "category": "functional",
                "description": "Admins can export the full customer list as a CSV file.",
                "priority": "medium",
                "certainty": "known",
                "confidence": 90.0,
                "evidence": "The request explicitly asks for a CSV export.",
            }
        ],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_findings": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "low", "reasoning": "A straightforward export feature."},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
        # The one clarification question every test in this file exercises.
        "missing_information": [
            {
                "question": "What specific UI changes are required for the password setup process?",
                "priority": "important",
                "reason": "Directly impacts frontend implementation.",
            }
        ],
        "test_cases": [],
        "implementation_plan": [],
        "recommendation": {"decision": "requires_clarification", "reasoning": "Needs one open question answered."},
    }
    base.update(overrides)
    return base

def analyze(headers, cr_id):
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
```

```

return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def _setup_analysis(monkeypatch, headers, cr_id):
    _patch_provider(monkeypatch, FakeProvider(json.dumps(_minimal_response())))
    return _analyze(headers, cr_id)

def test_answer_clarification_question_round_trip(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    question = detail["clarification_questions"][0]
    assert question["resolved"] is False
    assert question["answer_text"] is None

    response = client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "The password setup screen gains a 'Set password' button and a confirm-password field."},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["resolved"] is True
    assert body["answer_text"] == "The password setup screen gains a 'Set password' button and a confirm-password field."
    assert body["answered_by"] is not None
    assert body["answered_at"] is not None
    # The AI's own question/priority/reason are untouched by answering it.
    assert body["question"] == question["question"]
    assert body["priority"] == question["priority"]
    assert body["reason"] == question["reason"]

def test_answer_clarification_question_rejects_blank_answer(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    question = detail["clarification_questions"][0]

    response = client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": ""},
        headers=headers,
    )
    assert response.status_code == 422

def test_answer_clarification_question_wrong_change_request_404s(monkeypatch):
    headers, _ = _auth()
    created_a = _create_change_request(headers, title="CR A")
    created_b = _create_change_request(headers, title="CR B")
    detail_a = _setup_analysis(monkeypatch, headers, created_a["id"])
    question = detail_a["clarification_questions"][0]

    response = client.patch(
        f"/api/change-requests/{created_b['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "This does not belong to CR B."},
        headers=headers,
    )
    assert response.status_code == 404

def test_answer_clarification_question_open_to_any_commenter_not_just_reviewers(monkeypatch):
    """Deliberately NOT gated like review requirement/update security finding status
    (see test_module14_phase6.py) - any authenticated user who can comment on
    the change request can supply the missing information, since it's often
    the requester (not a designated reviewer) who actually knows the answer."""
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    question = detail["clarification_questions"][0]

    other_headers, _ = _auth()
    response = client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "Answered by someone other than the CR's own creator."},
        headers=other_headers,
    )
    assert response.status_code == 200
    assert response.json()["resolved"] is True

def test_answer_clarification_question_can_be_corrected(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    question = detail["clarification_questions"][0]

    client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "First draft of the answer."},
        headers=headers,
    ).raise_for_status()

    response = client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "Corrected, more complete answer."},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["answer_text"] == "Corrected, more complete answer."
    assert body["resolved"] is True

def test_answer_clarification_question_records_history(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    question = detail["clarification_questions"][0]

    client.patch(
        f"/api/change-requests/{created['id']}/clarification-questions/{question['id']}/answer",
        json={"answer": "The answer, for the history record."},
        headers=headers,
    ).raise_for_status()

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    matching = [h for h in history if h["action"] == "clarification_answered"]
    assert len(matching) == 1
    assert matching[0]["new_value"] == "The answer, for the history record."
    assert matching[0]["field_name"] == question["question"]

```

tests/test_dashboard.py

323 lines

```
"""Verifies the dashboard summary endpoint: it requires authentication, and
the metrics/risk distribution/category breakdown/recent list it returns are
computed from real rows written straight into the database - never
hardcoded or faked.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from datetime import date, timedelta
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db() # make sure every table exists even if init_db was never run manually

from fastapi.testclient import TestClient

from app.database.session import SessionLocal
from app.main import app
from app.models import Analysis, ChangeRequest, ClarificationQuestion
from app.models.enums import ApprovalRecommendation, ChangeRequestStatus, ComplexityLevel, Priority

client = TestClient(app)

def _unique_email() -> str:
    return f"dash-{uuid.uuid4().hex[:10]}@example.com"

def _register_user() -> tuple[str, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Dash Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return body["access_token"], body["user"]["id"]

def test_dashboard_requires_authentication():
    response = client.get("/dashboard/summary")
    assert response.status_code == 401

def test_dashboard_returns_well_formed_summary():
    token, _ = _register_user()
    response = client.get("/dashboard/summary", headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 200
    body = response.json()
    assert "metrics" in body
    assert "risk distribution" in body
    assert "category breakdown" in body
    assert "recent change requests" in body

def test_dashboard_reflects_real_change_request_data():
    token, user_id = _register_user()

    db = SessionLocal()
    try:
        cr_no_analysis = ChangeRequest(
            title="Pending analysis example",
            description="Not analyzed yet.",
            priority=Priority.MEDIUM,
            status=ChangeRequestStatus.DRAFT,
            created_by=user_id,
        )
        cr_high_risk = ChangeRequest(
            title="High risk example",
            description="Touches payments.",
            priority=Priority.HIGH,
            status=ChangeRequestStatus.SUBMITTED,
            created_by=user_id,
        )
        cr_approved = ChangeRequest(
            title="Approved example",
            description="Small safe fix.",
            priority=Priority.LOW,
            status=ChangeRequestStatus.APPROVED,
            created_by=user_id,
        )
        db.add_all([cr_no_analysis, cr_high_risk, cr_approved])
        db.flush()

        high_risk_analysis = Analysis(
            change_request_id=cr_high_risk.id,
            summary="Risky payments change.",
            category="security",
            complexity=ComplexityLevel.HIGH,
            risk_score=90.0,
            confidence_score=70.0,
            recommendation=ApprovalRecommendation.NEEDS_MORE_INFO,
        )
        approved_analysis = Analysis(
            change_request_id=cr_approved.id,
            summary="Safe small fix.",
            category="bug fix",
            complexity=ComplexityLevel.LOW,
            risk_score=10.0,
            confidence_score=95.0,
            recommendation=ApprovalRecommendation.APPROVE,
        )
        db.add_all([high_risk_analysis, approved_analysis])
        db.flush()

        db.add(
            ClarificationQuestion(
                analysis_id=high_risk_analysis.id,
                question="What's the rollback plan?",
                priority=Priority.HIGH,
                reason="Needed before approval.",
                resolved=False,
            )
        )
    finally:
        db.commit()

    cr_ids = {
        "no_analysis": cr_no_analysis.id,
        "high_risk": cr_high_risk.id,
```

```

        "approved": cr_approved.id,
    }
    finally:
        db.close()

response = client.get("/dashboard/summary", headers={"Authorization": f"Bearer {token}"})
assert response.status_code == 200
body = response.json()

# These 3 rows were just inserted, so they're the most recent - always
# within the top-10 recent list regardless of how much other data exists.
by_id = {item["id"]: item for item in body["recent_change_requests"]}
assert cr_ids["high_risk"] in by_id
assert cr_ids["approved"] in by_id
assert cr_ids["no_analysis"] in by_id

assert by_id[cr_ids["high_risk"]]["risk"] == "critical"
assert by_id[cr_ids["high_risk"]]["category"] == "Security"
assert by_id[cr_ids["approved"]]["risk"] == "low"
assert by_id[cr_ids["approved"]]["category"] == "Bug Fix"
assert by_id[cr_ids["no_analysis"]]["risk"] is None
assert by_id[cr_ids["no_analysis"]]["category"] is None

metrics = body["metrics"]
assert metrics["total_change_requests"] >= 3
assert metrics["approved_changes"] >= 1
assert metrics["high_risk_changes"] >= 1
assert metrics["pending_analysis"] >= 1
assert metrics["requires_clarification"] >= 1

# --- Clicking a dashboard tile (spec: "make the tiles clickable, show
# the underlying change requests in table format") must show exactly
# the change requests that were counted for it - never a different
# number than the tile itself, since both come from the same
# classification (see app/api/dashboard.py::classify_change_request).
headers = {"Authorization": f"Bearer {token}"}

def _drill_down(metric):
    response = client.get("/dashboard/drill-down", params={"metric": metric}, headers=headers)
    assert response.status_code == 200
    return response.json()

total_drill = _drill_down("total")
assert total_drill["total"] == metrics["total_change_requests"]
# The popup never shows more than one page (capped at 10 rows) at a
# time - see the pagination-specific tests below for paging through
# the rest.
assert total_drill["page"] == 1
assert total_drill["page_size"] == 10
assert len(total_drill["items"]) == min(metrics["total_change_requests"], 10)

pending_drill = _drill_down("pending analysis")
assert pending_drill["total"] == metrics["pending_analysis"]
assert cr_ids["no_analysis"] in {item["id"] for item in pending_drill["items"]}
assert cr_ids["approved"] not in {item["id"] for item in pending_drill["items"]}

high_risk_drill = _drill_down("high risk")
assert high_risk_drill["total"] == metrics["high_risk_changes"]
assert cr_ids["high_risk"] in {item["id"] for item in high_risk_drill["items"]}
assert cr_ids["approved"] not in {item["id"] for item in high_risk_drill["items"]}

approved_drill = _drill_down("approved")
assert approved_drill["total"] == metrics["approved_changes"]
assert cr_ids["approved"] in {item["id"] for item in approved_drill["items"]}
assert cr_ids["no_analysis"] not in {item["id"] for item in approved_drill["items"]}

clarification_drill = _drill_down("requires clarification")
assert clarification_drill["total"] == metrics["requires_clarification"]
assert cr_ids["high_risk"] in {item["id"] for item in clarification_drill["items"]}
assert cr_ids["approved"] not in {item["id"] for item in clarification_drill["items"]}

def test_drill_down_date_range_filters_by_created_date():
    token, user_id = _register_user()
    headers = {"Authorization": f"Bearer {token}"}

    db = SessionLocal()
    try:
        cr = ChangeRequest(
            title="Date range filter probe CR",
            description="Only used to test date_from/date_to.",
            priority=Priority.MEDIUM,
            status=ChangeRequestStatus.DRAFT,
            created_by=user_id,
        )
        db.add(cr)
        db.commit()
        cr_id = cr.id
    finally:
        db.close()

    today = date.today().isoformat()
    tomorrow = (date.today() + timedelta(days=1)).isoformat()
    yesterday = (date.today() - timedelta(days=1)).isoformat()

    # A range that includes today finds it...
    in_range = client.get(
        "/dashboard/drill-down",
        params={"metric": "total", "date_from": today, "date_to": tomorrow},
        headers=headers,
    ).json()
    assert cr_id in {item["id"] for item in in_range["items"]}

    # ...a range entirely before today does not, and the filtered total
    # reflects that - never a mismatch between what's counted and what's
    # shown.
    out_of_range = client.get(
        "/dashboard/drill-down",
        params={"metric": "total", "date_from": yesterday, "date_to": yesterday},
        headers=headers,
    ).json()
    assert cr_id not in {item["id"] for item in out_of_range["items"]}
    assert out_of_range["total"] == len(out_of_range["items"])

def test_drill_down_pagination_pages_through_without_duplicates_or_gaps():
    token, user_id = _register_user()
    headers = {"Authorization": f"Bearer {token}"}

    db = SessionLocal()
    try:
        created_ids = []
        for i in range(12):
            cr = ChangeRequest(
                title=f"Pagination probe CR {i}",
                description="Only used to test drill-down pagination.",
                priority=Priority.MEDIUM,
                status=ChangeRequestStatus.DRAFT,
                created_by=user_id,
            )
            db.add(cr)

```

```

        db.flush()
        created_ids.append(cr.id)
    db.commit()
finally:
    db.close()

# The dashboard shows every change request in the system, not just this
# user's own (matches this app's existing "no per-CR visibility
# restriction" rule - see report_generator.py's own docstring). By the
# time this test runs, dozens of other test files have already added
# their own rows to this shared database, so 'total' is only ever a
# meaningless ">= 12" on its own, and page 2 can easily be filled out
# entirely once our own rows run out - it is NOT safe to assume page 2
# holds only our leftover 2 rows and nothing else.
#
# What's actually provable, and what this test verifies, is pagination
# itself: ids are assigned in strict insertion order, and the endpoint
# orders newest-first by (created at desc, id desc) - see
# app/api/dashboard.py:: load classified change requests - so these 12
# freshly-inserted rows are provably the highest ids in the whole table
# and must occupy exactly the first 12 positions of the "total" list,
# most-recently-inserted (highest id) first.
def _page(page_size, page):
    response = client.get(
        "/dashboard/drill-down",
        params={"metric": "total", "page": page, "page_size": page_size},
        headers=headers,
    )
    assert response.status_code == 200
    return response.json()

newest_first = list(reversed(created_ids))

first_page = _page(10, 1)
assert first_page["total"] >= 12
assert [item["id"] for item in first_page["items"]] == newest_first[:10]

second_page = _page(10, 2)
# Positions 11 and 12 overall are our own last two rows, still in the
# same id-descending order. Whatever else fills out the rest of this
# page comes from older, pre-existing rows - and must never repeat
# anything already shown on page 1.
assert len(second_page["items"]) >= 2
assert [item["id"] for item in second_page["items"][:2]] == newest_first[10:12]
first_ids = {item["id"] for item in first_page["items"]}
second_ids = {item["id"] for item in second_page["items"]}
assert first_ids.isdisjoint(second_ids)

# page_size above 10 is rejected outright - the popup never shows more
# than 10 rows at once.
assert client.get(
    "/dashboard/drill-down", params={"metric": "total", "page_size": 11}, headers=headers
).status_code == 422

def test_drill_down_requires_authentication():
    assert client.get("/dashboard/drill-down", params={"metric": "total"}).status_code == 401

def test_drill_down_rejects_unknown_metric():
    token, _ = _register_user()
    response = client.get(
        "/dashboard/drill-down",
        params={"metric": "not_a_real_metric"},
        headers={"Authorization": f"Bearer {token}"},
    )
    assert response.status_code == 422

```

tests/test_database.py

115 lines

```
"""Verifies the database layer: every table creates successfully, and
relationships/cascades behave correctly - using a throwaway in-memory
SQLite database (never touches the real backend/data/app.db file).

Run with (from backend/): pytest ../tests
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

from app import models  # noqa: F401 - registers all models on Base.metadata
from app.database.session import Base
from app.models.enums import ChangeRequestStatus, ComplexityLevel, Priority, RequirementType, UserRole

@pytest.fixture()
def db_session():
    engine = create_engine("sqlite:///memory:", connect_args={"check_same_thread": False})
    Base.metadata.create_all(bind=engine)
    session = Session(bind=engine)
    try:
        yield session
    finally:
        session.close()
        engine.dispose()

def test_all_tables_created(db_session):
    table_names = set(Base.metadata.tables.keys())
    expected = {
        "users",
        "change_requests",
        "analyses",
        "requirements",
        "affected_components",
        "dependencies",
        "risks",
        "clarification_questions",
        "test_cases",
        "implementation_tasks",
    }
    assert expected.issubset(table_names)

def test_change_request_belongs_to_its_creator(db_session):
    user = models.User(
        name="Ada Lovelace",
        email="ada@example.com",
        password_hash="hashed",
        role=UserRole.ENGINEER,
    )
    db_session.add(user)
    db_session.flush()

    change_request = models.ChangeRequest(
        title="Add dark mode",
        description="Add a dark theme toggle to settings.",
        priority=Priority.MEDIUM,
        status=ChangeRequestStatus.DRAFT,
        created_by=user.id,
    )
    db_session.add(change_request)
    db_session.commit()

    assert change_request.creator.email == "ada@example.com"
    assert user.change_requests == [change_request]

def test_analysis_cascades_delete_to_its_children(db_session):
    user = models.User(name="Ada", email="ada2@example.com", password_hash="x", role=UserRole.ENGINEER)
    db_session.add(user)
    db_session.flush()

    cr = models.ChangeRequest(
        title="Add SSO login",
        description="Support SAML-based SSO.",
        priority=Priority.HIGH,
        status=ChangeRequestStatus.SUBMITTED,
        created_by=user.id,
    )
    db_session.add(cr)
    db_session.flush()

    analysis = models.Analysis(
        change_request_id=cr.id,
        summary="Adds SSO login support via SAML.",
        category="feature",
        complexity=ComplexityLevel.HIGH,
        risk_score=65.0,
        confidence_score=80.0,
    )
    analysis.requirements.append(
        models.Requirement(
            requirement_type=RequirementType.FUNCTIONAL,
            description="Users can log in via SAML SSO.",
            priority=Priority.HIGH,
        )
    )
    db_session.add(analysis)
    db_session.commit()

    assert len(cr.analyses) == 1
    assert cr.analyses[0].requirements[0].description.startswith("Users can log in")

    # Deleting the analysis should cascade-delete its requirement row too.
    db_session.delete(analysis)
    db_session.commit()

    assert db_session.query(models.Requirement).count() == 0
```

tests/test_health.py

30 lines

```
"""Smoke test for the health-check endpoint.

Run with (from backend/): pytest ../tests
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)


def test_health_check_returns_ok():
    response = client.get("/health")
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "ok"
    assert "database" in body
    assert "ai_provider_configured" in body


def test_root_returns_metadata():
    response = client.get("/")
    assert response.status_code == 200
    assert "message" in response.json()
```

tests/test_module14_phase2.py

284 lines

```
"""Verifies Module 14 Phase 2 (Analysis & Impact Intelligence -
Affected Components + Dependencies filled out):

* AffectedComponent now carries an `evidence` string, the same role it
  already plays on Requirement/Risk (Module 13 Phase 1).
* Dependency now carries `relationship_type` (direct/indirect/potential -
  how directly this change actually relies on it) and `risk_severity` (a
  severity rating of relying on it), both distinct from the pre-existing
  `impact_level`.
* A response shaped like pre-Module-14 output (no evidence/relationship/
  risk keys at all) must still validate and persist - these additions
  must not be a breaking change, same rule Module 13 Phase 1 followed.
* The "do not invent dependencies" placeholder-name filtering
  (AIAnalysisResult.drop_placeholder_list_items) still works unchanged
  alongside the new fields.

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as test_ai_analysis_v2.py.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m14p2-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module14 Phase2 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds OTP-based authentication at login.",
        "classification": {"category": "Security", "confidence": 0.9, "reason": "New authentication factor."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
        "effort": {"backend": "2-3 days", "frontend": "1 day", "testing": "1 day", "total": "4-5 developer-days"},
        "missing_information": [],
        "test_cases": [],
        "implementation_plan": [],
        "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs OTP expiry defined."},
    }
    base.update(overrides)
    return base

def _analyze(headers, cr_id, response_dict):
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def test_affected_component_evidence_round_trips(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        affected_components=[
            {
                "name": "Authentication Service",
                "type": "backend",
                "impact_level": "high",
                "reason": "Central place OTP verification would be added.",
                "confidence": 78,
                "certainty": "inferred",
                "evidence": "The description names login as the affected flow.",
            }
        ]
    )
```



```

    },
    ],
)
_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"], response_json)

component = detail["affected_components"][0]
assert component["evidence"] == "The description names login as the affected flow."

def test_dependency_relationship_and_risk_round_trip(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        dependencies=[
            {
                "name": "SMS Gateway",
                "type": "external",
                "impact level": "high",
                "reason": "OTP codes must be delivered via SMS.",
                "relationship": "direct",
                "risk": "high",
            },
            {
                "name": "User Session Store",
                "type": "internal",
                "impact level": "medium",
                "reason": "Session state may need a new OTP-pending flag.",
                "relationship": "indirect",
                "risk": "low",
            },
            {
                "name": "Fraud Detection Service",
                "type": "service",
                "impact level": "low",
                "reason": "May eventually want to flag suspicious OTP requests.",
                "relationship": "potential",
                "risk": "medium",
            },
        ],
    ),
    ],
)
_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"], response_json)

deps = {d["dependency_name"]: d for d in detail["dependencies"]}
assert deps["SMS Gateway"]["relationship_type"] == "direct"
assert deps["SMS Gateway"]["risk_severity"] == "high"
assert deps["User Session Store"]["relationship_type"] == "indirect"
assert deps["User Session Store"]["risk_severity"] == "low"
assert deps["Fraud Detection Service"]["relationship_type"] == "potential"
assert deps["Fraud Detection Service"]["risk_severity"] == "medium"

def test_pre_module_14_shaped_dependency_and_component_still_validate(monkeypatch):
    """No evidence/relationship/risk keys at all (the exact shape Module 6/
    13 produced before this module) must still validate and persist, with
    safe defaults rather than a validation failure."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        affected_components=[
            {
                "name": "Legacy-shaped Component",
                "type": "backend",
                "impact level": "medium",
                "reason": "No evidence field at all.",
                "confidence": 50,
            },
        ],
        dependencies=[
            {
                "name": "Legacy-shaped Dependency",
                "type": "service",
                "impact level": "low",
                "reason": "No relationship/risk fields at all.",
            },
        ],
    ),
    ],
)
_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"], response_json)

component = detail["affected_components"][0]
assert component["evidence"] is None

dependency = detail["dependencies"][0]
assert dependency["relationship_type"] == "direct"
assert dependency["risk_severity"] == "low"

def test_placeholder_relationship_and_risk_fall_back_to_safe_defaults(monkeypatch):
    """A weaker model's "no answer" placeholder text (e.g. "not specified",
    "n/a") should fall back to a safe default rather than fail validation -
    same normalize() "no answer" handling every other enum field in this
    file already relies on (see ai.analysis.py's module docstring). Note
    this is specifically about a *placeholder* string, not arbitrary
    gibberish: normalize() only rescues values in its known "no answer"
    set - a genuinely garbled value that ISN'T one of those (e.g. "sort of
    maybe") still fails validation today, and that's true for every other
    enum field here (type, impact level, category, ...), not something
    Module 14 introduced or is expected to change."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        dependencies=[
            {
                "name": "Placeholder-value Dependency",
                "type": "service",
                "impact level": "medium",
                "reason": "Placeholder relationship/risk values.",
                "relationship": "not specified",
                "risk": "n/a",
            },
        ],
    ),
    ],
)
_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"], response_json)

dependency = detail["dependencies"][0]
assert dependency["relationship_type"] == "direct"
assert dependency["risk_severity"] == "low"

def test_placeholder_named_dependency_still_dropped(monkeypatch):
    """Module 6's existing "don't persist a fake dependency literally named
    'None specified'" filtering (drop placeholder list items) must keep
    working unchanged alongside the new relationship/risk fields - Module
    14 doesn't touch that logic, this just confirms it wasn't disturbed."""
    headers = _auth()

```

```

created = _create_change_request(headers)

response_json = _minimal_response(
    dependencies=[
        {
            "name": "None specified",
            "type": "internal",
            "impact_level": "low",
            "reason": "Placeholder row a weaker model sometimes emits.",
            "relationship": "direct",
            "risk": "low",
        },
        {
            "name": "Real Payment Gateway",
            "type": "external",
            "impact_level": "high",
            "reason": "Payments are processed through it.",
            "relationship": "direct",
            "risk": "high",
        },
    ],
)

_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"], response_json)

names = [d["dependency_name"] for d in detail["dependencies"]]
assert "None specified" not in names
assert "Real Payment Gateway" in names

```

tests/test_module14_phase3.py

164 lines

```
"""Verifies Module 14 Phase 3 (Analysis & Impact Intelligence - Complexity
& Effort confidence):

* Analysis now carries `complexity_confidence` and `effort_confidence`,
each a coarse Low/Medium/High rating - deliberately not a finer-grained
numeric confidence, since Complexity/Effort are already plain-language
estimates on purpose ("a range, not fake precision").
* Both are distinct from `confidence_score` (the overall classification
confidence) - a change request can have high overall confidence but a
low confidence specifically in its effort estimate, and vice versa.
* A response shaped like pre-Module-14 output (no confidence key on
complexity/effort at all) must still validate and persist, falling
back to "medium" - same rule every other Module 13/14 addition follows.

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as the other test_module14_*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m14p3-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module14 Phase3 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication to login",
        "description": "Send a one-time password to the customer's registered mobile number at login.",
        "priority": "high",
        "requested by": "Shivani",
        "target_system": "Auth Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds OTP-based authentication at login.",
        "classification": {"category": "Security", "confidence": 0.9, "reason": "New authentication factor."},
        "requirements": [],
        "affected components": [],
        "dependencies": [],
        "risks": [],
        "security analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches auth flow."},
        "effort": {"backend": "2-3 days", "frontend": "1 day", "testing": "1 day", "total": "4-5 developer-days"},
        "missing information": [],
        "test cases": [],
        "implementation plan": [],
        "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs OTP expiry defined."},
    }
    base.update(overrides)
    return base

def _analyze(headers, cr_id, response_dict):
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def test_complexity_and_effort_confidence_round_trip(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        complexity={"level": "high", "reasoning": "Touches auth and session handling.", "confidence": "low"},
        effort={
            "backend": "5-8 days",
            "frontend": "2-3 days",
            "testing": "2 days",
            "total": "9-13 developer-days",
            "confidence": "high",
        },
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"], response_json)
```

```

assert detail["complexity_confidence"] == "low"
assert detail["effort_confidence"] == "high"
# Distinct from the overall classification confidence.
assert detail["confidence_score"] == 90.0

def test_pre_module_14_shaped_response_defaults_to_medium(monkeypatch):
    """No confidence key at all on complexity/effort (the exact shape
    Module 6 originally produced) must still validate and persist, falling
    back to "medium" rather than a validation failure."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = minimal_response(
        complexity={"level": "medium", "reasoning": "Legacy-shaped complexity, no confidence key."},
        effort={"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"], response_json)

    assert detail["complexity_confidence"] == "medium"
    assert detail["effort_confidence"] == "medium"

def test_garbled_confidence_value_falls_back_to_medium(monkeypatch):
    """Same "no answer" placeholder handling every other enum field in this
    codebase relies on - a recognized placeholder string falls back safely
    rather than failing validation."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = minimal_response(
        complexity={"level": "medium", "reasoning": "Placeholder confidence value.", "confidence": "not specified"},
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"], response_json)

    assert detail["complexity_confidence"] == "medium"

```

tests/test_module14_phase4.py

230 lines

```
"""Verifies Module 14 Phase 4 (Analysis & Impact Intelligence - Impact
Analysis):

* Analysis now carries a full "impact assessments" list - one structured
finding per fixed lens (Business/Technical/Customer/Operational/
Security/Data/Performance - see app.models.enums.ImpactCategory), each
with its own impact_level/description/certainty/confidence.
* The AI is expected to return exactly one entry per category, but if it
leaves one out (or all of them, for a pre-Module-14-shaped response),
AIAnalysisResult.fill_missing_impact_categories fills the gap with an
honest "no assessment given" placeholder (medium impact, unknown
certainty, 0 confidence) rather than silently rendering fewer than 7
cards - the result is always exactly 7 entries, one per category.
* A placeholder/garbled category or impact_level value falls back to a
safe default ("technical" / "medium" respectively) via the same
_normalize() rules every other enum field in this codebase relies on.

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as the other test_module14_*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

ALL_CATEGORIES = ["business", "technical", "customer", "operational", "security", "data", "performance"]

def _unique_email() -> str:
    return f"m14p4-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module14 Phase4 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add multi-factor authentication to checkout",
        "description": "Require a second verification step before a customer can complete checkout on Alight.com.",
        "priority": "high",
        "requested_by": "Shivani",
        "target_system": "Checkout Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds an MFA step to checkout.",
        "classification": {"category": "Security", "confidence": 0.9, "reason": "New verification step."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches checkout flow."},
        "effort": {"backend": "3-4 days", "frontend": "2 days", "testing": "1 day", "total": "6-7 developer-days"},
        "missing_information": [],
        "test_cases": [],
        "implementation_plan": [],
        "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs UX review of the MFA prompt."},
    }
    base.update(overrides)
    return base

def _analyze(headers, cr_id):
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def test_impact_assessments_round_trip(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        impact_assessments=[
            {
                "category": category,
                "impact_level": "high" if category == "security" else "medium",
                "description": f"{category.title()} impact description.",
                "certainty": "known",
            }
        ]
    )
```

```

        "confidence": 85.0,
    }
    for category in ALL_CATEGORIES
]
)
_patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
detail = _analyze(headers, created["id"])

assert len(detail["impact_assessments"]) == 7
by_category = {item["category"]: item for item in detail["impact_assessments"]}
assert set(by_category.keys()) == set(ALL_CATEGORIES)
assert by_category["security"]["impact_level"] == "high"
assert by_category["security"]["certainty"] == "known"
assert by_category["security"]["confidence"] == 85.0
assert by_category["business"]["description"] == "Business impact description."

def test_pre_module_14_shaped_response_fills_all_categories(monkeypatch):
    """No "impact_assessments" key at all (the exact shape every prior
    module produced) must still validate and persist, filling in all 7
    categories with an honest placeholder rather than a validation
    failure or a shorter-than-7 list."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response()
    assert "impact_assessments" not in response_json
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    assert len(detail["impact_assessments"]) == 7
    assert {item["category"] for item in detail["impact_assessments"]} == set(ALL_CATEGORIES)
    for item in detail["impact_assessments"]:
        assert item["certainty"] == "unknown"
        assert item["confidence"] == 0.0
        assert item["description"] == "Insufficient information."

def test_partial_impact_assessments_fills_only_missing_categories(monkeypatch):
    """Only 2 of the 7 categories provided - the other 5 get filled in with
    the placeholder, but the 2 real ones are kept exactly as given."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        impact_assessments=[
            {
                "category": "customer",
                "impact_level": "high",
                "description": "Customers will see a new MFA prompt at checkout.",
                "certainty": "known",
                "confidence": 90.0,
            },
            {
                "category": "data",
                "impact_level": "low",
                "description": "No new data is stored beyond the MFA code itself.",
                "certainty": "inferred",
                "confidence": 60.0,
            },
        ],
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    assert len(detail["impact_assessments"]) == 7
    by_category = {item["category"]: item for item in detail["impact_assessments"]}
    assert by_category["customer"]["description"] == "Customers will see a new MFA prompt at checkout."
    assert by_category["customer"]["confidence"] == 90.0
    assert by_category["data"]["impact_level"] == "low"
    for category in ["business", "technical", "operational", "security", "performance"]:
        assert by_category[category]["description"] == "Insufficient information."
        assert by_category[category]["certainty"] == "unknown"

def test_placeholder_category_and_impact_level_fall_back_to_safe_defaults(monkeypatch):
    """Same "no answer" placeholder handling every other enum field in this
    codebase relies on - a recognized placeholder string (never invented
    gibberish, which normalize() does NOT rescue) falls back safely
    rather than failing validation."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        impact_assessments=[
            {
                "category": "not specified",
                "impact_level": "n/a",
                "description": "Placeholder category and impact level.",
                "certainty": "known",
                "confidence": 55.0,
            },
        ],
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    by_category = {item["category"]: item for item in detail["impact_assessments"]}
    # Falls back to "technical" (category default) / "medium" (impact level
    # default) - and since "technical" is one of the 7 required categories,
    # the fill-missing-categories step does NOT add a second placeholder
    # entry for it.
    assert by_category["technical"]["impact_level"] == "medium"
    assert by_category["technical"]["description"] == "Placeholder category and impact level."
    assert len(detail["impact_assessments"]) == 7

```

tests/test_module14_phase5.py

240 lines

```
"""Verifies Module 14 Phase 5 (Analysis & Impact Intelligence - Security):

* Analysis now carries a full "security findings" list - one structured
  finding per fixed lens (Authentication/Authorization/Data Protection/
  Secrets/API Security/Rate Limiting/Privacy/Audit Logging/Compliance -
  see app.models.enums.SecurityCategory), each with severity/evidence/
  recommendation/status.
* The AI is expected to return exactly one entry per category; if it
  leaves one out (or all of them, for a pre-Module-14-Phase-5-shaped
  response), AIAnalysisResult.fill_missing_security_categories fills the
  gap with an honest "nothing flagged" placeholder (low severity,
  not applicable status) - the result is always exactly 9 entries.
* The AI can only ever set status to "open" or "not applicable" -
  "acknowledged"/"resolved" are reserved for a human reviewer (Module 14
  Phase 6, not yet built) - an AI response that tries to claim either of
  those falls back to "open" rather than being trusted.
* A placeholder/garbled category or severity value falls back to a safe
  default ("compliance" / "medium" respectively).
* The old flat 'security analysis' blob is still persisted unchanged
  alongside the new structured findings - Phase 5 adds to it, it doesn't
  replace it at the database level (only the frontend switches which one
  it shows as primary).

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as the other test_module14_*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

ALL_CATEGORIES = [
    "authentication",
    "authorization",
    "data protection",
    "secrets",
    "api security",
    "rate limiting",
    "privacy",
    "audit logging",
    "compliance",
]

def _unique_email() -> str:
    return f"ml14p5-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module14 Phase5 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add API key rotation for partner integrations",
        "description": "Let Alight.com admins rotate partner API keys without downtime.",
        "priority": "high",
        "requested by": "Shivani",
        "target_system": "Partner API Gateway",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds a key-rotation flow for partner API keys.",
        "classification": {"category": "Security", "confidence": 0.9, "reason": "Touches credential lifecycle."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security analysis": {"concerns": ["Old keys must be invalidated immediately on rotation."], "summary": ""},
        "complexity": {"level": "medium", "reasoning": "Touches auth/credential flow."},
        "effort": {"backend": "3-4 days", "frontend": "1 day", "testing": "1 day", "total": "5-6 developer-days"},
        "missing information": [],
        "test cases": [],
        "implementation plan": [],
        "recommendation": {"decision": "approve_with_conditions", "reasoning": "Needs a key-invalidation audit."},
    }
    base.update(overrides)
    return base

def _analyze(headers, cr_id):
```

```

response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
assert response.status_code == 201
return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def test_security_findings_round_trip(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        security_findings=[
            {
                "category": category,
                "finding": f"{category.title()} finding.",
                "severity": "critical" if category == "secrets" else "medium",
                "evidence": f"{category} evidence.",
                "recommendation": f"{category} recommendation.",
                "status": "open",
            }
            for category in ALL_CATEGORIES
        ]
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    assert len(detail["security_findings"]) == 9
    by_category = {item["category"]: item for item in detail["security_findings"]}
    assert set(by_category.keys()) == set(ALL_CATEGORIES)
    assert by_category["secrets"]["severity"] == "critical"
    assert by_category["secrets"]["status"] == "open"
    assert by_category["compliance"]["evidence"] == "compliance evidence."
    # The old flat blob is still persisted alongside the new list.
    assert detail["security_analysis"] is not None
    assert "invalidated" in detail["security_analysis"]

def test_pre_module_14_phase5_shaped_response_fills_all_categories(monkeypatch):
    """No "security_findings" key at all (the exact shape every prior
    module produced) must still validate and persist, filling in all 9
    categories with an honest placeholder rather than a validation failure
    or a shorter-than-9 list."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response()
    assert "security_findings" not in response_json
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    assert len(detail["security_findings"]) == 9
    assert {item["category"] for item in detail["security_findings"]} == set(ALL_CATEGORIES)
    for item in detail["security_findings"]:
        assert item["status"] == "not applicable"
        assert item["severity"] == "low"

def test_ai_cannot_claim_acknowledged_or_resolved_status(monkeypatch):
    """Status is only ever AI-settable as open/not_applicable - Module 14
    Phase 6 reserves acknowledged/resolved for a human reviewer. An AI
    response that tries to claim either of those falls back to "open"
    rather than being trusted at face value."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        security_findings=[
            {
                "category": "authentication",
                "finding": "MFA is not required for admin key rotation.",
                "severity": "high",
                "evidence": "The request doesn't mention any additional verification step.",
                "recommendation": "Require MFA before allowing key rotation.",
                "status": "resolved",
            }
        ]
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    by_category = {item["category"]: item for item in detail["security_findings"]}
    assert by_category["authentication"]["status"] == "open"
    assert len(detail["security_findings"]) == 9

def test_placeholder_category_and_severity_fall_back_to_safe_defaults(monkeypatch):
    """Same "no answer" placeholder handling every other enum field in this
    codebase relies on - a recognized placeholder string (never invented
    gibberish, which _normalize() does NOT rescue) falls back safely
    rather than failing validation."""
    headers = _auth()
    created = _create_change_request(headers)

    response_json = _minimal_response(
        security_findings=[
            {
                "category": "not specified",
                "severity": "n/a",
                "finding": "Placeholder category and severity.",
                "evidence": "Evidence text.",
                "recommendation": "Recommendation text.",
                "status": "open",
            }
        ]
    )
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(response_json)))
    detail = _analyze(headers, created["id"])

    by_category = {item["category"]: item for item in detail["security_findings"]}
    # Falls back to "compliance" (category default) / "medium" (severity
    # default) - and since "compliance" is one of the 9 required
    # categories, the fill-missing-categories step does NOT add a second
    # placeholder entry for it.
    assert by_category["compliance"]["severity"] == "medium"
    assert by_category["compliance"]["finding"] == "Placeholder category and severity."
    assert len(detail["security_findings"]) == 9

```


tests/test_module14_phase6.py

285 lines

```
"""Verifies Module 14 Phase 6 (Analysis & Impact Intelligence - Human
Review):

* PATCH ../requirements/{id}/review - an authorized human marks a
Requirement Confirmed or Needs Clarification, WITHOUT touching that
row's AI-generated certainty/confidence/evidence. A comment is
required for Needs Clarification, optional for Confirmed. Permission-
gated (workflow_rules.can_review_analysis_findings) and recorded on
the change request's audit history (HistoryAction.REQUIREMENT_REVIEWED).
* PATCH ../security-findings/{id}/status - an authorized human moves a
Security finding's Status (typically Open -> Acknowledged ->
Resolved, but a human may also reopen one) - the AI itself can only
ever leave a finding Open/Not Applicable (see test_module14_phase5.py's
test ai cannot claim acknowledged or resolved status), so Acknowledged/
Resolved only ever appear via this endpoint. Same permission-gating
and audit-history recording (HistoryAction.SECURITY_FINDING_STATUS_CHANGED).

The real AI provider is never called here - same monkeypatched
_FakeProvider pattern as the other test_module14*.py files.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def unique_email() -> str:
    return f"m14p6-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> tuple:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module14 Phase6 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add bulk export to admin dashboard",
        "description": "Let Alight.com admins export the full customer list as a CSV from the dashboard.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Admin Dashboard",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _minimal_response(**overrides) -> dict:
    base = {
        "summary": "Adds a CSV export button to the admin dashboard.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New export capability."},
        "requirements": [
            {
                "category": "functional",
                "description": "Admins can export the full customer list as a CSV file.",
                "priority": "medium",
                "certainty": "known",
                "confidence": 90.0,
                "evidence": "The request explicitly asks for a CSV export.",
            }
        ],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_findings": [
            {
                "category": "data_protection",
                "finding": "Bulk customer data export needs access controls.",
                "severity": "high",
                "evidence": "The export includes the full customer list.",
                "recommendation": "Restrict export to admin roles and log every export.",
                "status": "open",
            }
        ],
        "security_analysis": {"concerns": [], "summary": ""},
        "complexity": {"level": "low", "reasoning": "A straightforward export feature."},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
        "missing_information": [],
        "test_cases": [],
        "implementation_plan": [],
        "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
    }
    base.update(overrides)
    return base
```

```

def _analyze(headers, cr_id):
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return client.get(f"/api/change-requests/{cr_id}/analysis", headers=headers).json()

def _setup_analysis(monkeypatch, headers, cr_id):
    _patch_provider(monkeypatch, _FakeProvider(json.dumps(_minimal_response()))
    return _analyze(headers, cr_id)

def test_review_requirement_round_trip_never_touchees_ai_columns(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    requirement = detail["requirements"][0]
    assert requirement["review_status"] is None # unreviewed until a human acts

    response = client.patch(
        f"/api/change-requests/{created['id']}/requirements/{requirement['id']}/review",
        json={"review_status": "confirmed"},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["review_status"] == "confirmed"
    assert body["reviewed_by"] is not None
    assert body["reviewed_at"] is not None
    # The AI's own findings are untouched by the review.
    assert body["certainty"] == requirement["certainty"]
    assert body["confidence"] == requirement["confidence"]
    assert body["evidence"] == requirement["evidence"]

    # A human can change their mind - moving to Needs Clarification with a
    # comment overwrites review_status/comment but still leaves certainty/
    # confidence/evidence alone.
    response = client.patch(
        f"/api/change-requests/{created['id']}/requirements/{requirement['id']}/review",
        json={"review_status": "needs_clarification", "comment": "Which admin roles should have export access?"},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["review_status"] == "needs clarification"
    assert body["review_comment"] == "Which admin roles should have export access?"
    assert body["certainty"] == requirement["certainty"]

def test_review_requirement_needs_clarification_requires_comment(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    requirement = detail["requirements"][0]

    response = client.patch(
        f"/api/change-requests/{created['id']}/requirements/{requirement['id']}/review",
        json={"review_status": "needs_clarification"},
        headers=headers,
    )
    assert response.status_code == 422

def test_review_requirement_forbidden_for_unrelated_user(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    requirement = detail["requirements"][0]

    outsider_headers, _ = _auth()
    response = client.patch(
        f"/api/change-requests/{created['id']}/requirements/{requirement['id']}/review",
        json={"review_status": "confirmed"},
        headers=outsider_headers,
    )
    assert response.status_code == 403

def test_review_requirement_records_history(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    requirement = detail["requirements"][0]

    client.patch(
        f"/api/change-requests/{created['id']}/requirements/{requirement['id']}/review",
        json={"review_status": "confirmed"},
        headers=headers,
    ).raise_for_status()

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    matching = [h for h in history if h["action"] == "requirement_reviewed"]
    assert len(matching) == 1
    assert matching[0]["new_value"] == "confirmed"
    assert matching[0]["old_value"] == "unreviewed"

def test_update_security_finding_status_round_trip(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    finding = next(f for f in detail["security_findings"] if f["category"] == "data_protection")
    assert finding["status"] == "open"

    response = client.patch(
        f"/api/change-requests/{created['id']}/security-findings/{finding['id']}/status",
        json={"status": "acknowledged", "comment": "Tracked in the export-access ticket."},
        headers=headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "acknowledged"
    assert body["review_comment"] == "Tracked in the export-access ticket."
    assert body["reviewed_by"] is not None
    assert body["reviewed_at"] is not None
    # The AI's own finding/severity/evidence/recommendation are untouched.
    assert body["finding"] == finding["finding"]
    assert body["severity"] == finding["severity"]

    response = client.patch(
        f"/api/change-requests/{created['id']}/security-findings/{finding['id']}/status",
        json={"status": "resolved"},
        headers=headers,
    )
    assert response.status_code == 200
    assert response.json()["status"] == "resolved"

def test_update_security_finding_status_forbidden_for_unrelated_user(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)

```

```

detail = _setup_analysis(monkeypatch, headers, created["id"])
finding = next(f for f in detail["security_findings"] if f["category"] == "data_protection")

outsider_headers, _ = auth()
response = client.patch(
    f"/api/change-requests/{created['id']}/security-findings/{finding['id']}/status",
    json={"status": "resolved"},
    headers=outsider_headers,
)
assert response.status_code == 403

def test_update_security_finding_status_records_history(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers)
    detail = _setup_analysis(monkeypatch, headers, created["id"])
    finding = next(f for f in detail["security_findings"] if f["category"] == "data_protection")

    client.patch(
        f"/api/change-requests/{created['id']}/security-findings/{finding['id']}/status",
        json={"status": "acknowledged"},
        headers=headers,
    ).raise_for_status()

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    matching = [h for h in history if h["action"] == "security_finding_status_changed"]
    assert len(matching) == 1
    assert matching[0]["old_value"] == "open"
    assert matching[0]["new_value"] == "acknowledged"

```

tests/test_module15_phase1.py

181 lines

```
"""Verifies Module 15 Phase 1 (Repository Intelligence - Foundation):

* scan_repository() walks a given folder and records exactly the files
  it should - source-like extensions only - while silently skipping
  node_modules/.git/build/_pypcache_/binaries/oversized files/.env and
  other secret-shaped files/anything matched by the repo's own
  .gitignore. A hard-excluded file's contents are never read at all -
  it's filtered out by name/extension before any read is attempted.
* scan_number increments per repeated scan of the same root path.
* Scanning a nonexistent path raises RepositoryScanError *before* any
  RepositoryScan row is created (no junk "attempted" rows left behind).
* Settings.repository_root_path defaults to this project's own root
  folder when REPOSITORY_ROOT isn't set.
* The API layer (POST/GET /api/repository/...) requires auth, and its
  list/latest/detail/404 responses behave as expected.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.core.config import get_settings
from app.database.session import SessionLocal
from app.main import app
from app.models.enums import RepositoryScanStatus
from app.models.repository_scan import RepositoryScan
from app.services.repository_scanner import RepositoryScanError, scan_repository

client = TestClient(app)

def _unique_email() -> str:
    return f"m15pl-{uuid.uuid4()}.hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module15 Phase1 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}

def _make_sample_repo(tmp_path: Path) -> Path:
    root = tmp_path / "sample_repo"
    (root / "src").mkdir(parents=True)
    (root / "node_modules" / "pkg").mkdir(parents=True)
    (root / ".git").mkdir(parents=True)
    (root / "build").mkdir(parents=True)
    (root / "ignored_dir").mkdir(parents=True)

    (root / "src" / "otp_service.py").write_text(
        "import hashlib\n\ndef generate_otp():\n    return 123456\n"
    )
    (root / "src" / "app.jsx").write_text("export function App() { return null }\n")
    (root / "src" / "widget.generated.js").write_text("// generated, gitignored\n")
    (root / "node_modules" / "pkg" / "index.js").write_text("module.exports = {}\n")
    (root / ".git" / "HEAD").write_text("ref: refs/heads/main\n")
    (root / "build" / "bundle.js").write_text("/* generated */\n")
    (root / "ignored_dir" / "should skip.py").write_text("print('should not be indexed')\n")
    (root / ".env").write_text("SECRET_KEY=super-secret-value\nDATABASE_PASSWORD=hunter2\n")
    (root / "logo.png").write_bytes(b"\x89PNG\r\n\x1a\n" + b"\x00" * 20)
    (root / "big.json").write_text("x" * (600 * 1024)) # over the 512KB size cap
    (root / ".gitignore").write_text("ignored_dir/\n*.generated.js\n")

    return root

def test_scan_indexes_source_and_skips_everything_it_should(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root, triggered_by=None)
        assert scan.status == RepositoryScanStatus.COMPLETED
        indexed_paths = {f.file_path for f in scan.files}

        assert "src/otp_service.py" in indexed_paths
        assert "src/app.jsx" in indexed_paths

        # node_modules/.git/build are never descended into at all.
        assert not any(p.startswith("node_modules/") for p in indexed_paths)
        assert not any(p.startswith(".git/") for p in indexed_paths)
        assert not any(p.startswith("build/") for p in indexed_paths)
        # secrets, oversized files, and .gitignore-matched files/dirs are skipped.
        assert ".env" not in indexed_paths
        assert "logo.png" not in indexed_paths
        assert "big.json" not in indexed_paths
        assert "ignored_dir/should skip.py" not in indexed_paths
        assert "src/widget.generated.js" not in indexed_paths

        otp_file = next(f for f in scan.files if f.file_path == "src/otp_service.py")
        assert otp_file.language == "python"
        assert otp_file.line_count == 4
        assert scan.file_count == len(indexed_paths)
        assert scan.skipped_count > 0
    finally:
        db.close()

def test_scan_number_increments_per_root_path(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        first = scan_repository(db, root_path=root)
        second = scan_repository(db, root_path=root)
        assert second.scan_number == first.scan_number + 1
    finally:
        db.close()

def test_scan_nonexistent_path_raises_before_creating_a_row(tmp_path):
```

```

missing = tmp_path / "does_not_exist"
db = SessionLocal()
try:
    count_before = db.query(RepositoryScan).count()
    raised = False
    try:
        scan_repository(db, root_path=missing)
    except RepositoryScanError:
        raised = True
    assert raised
    assert db.query(RepositoryScan).count() == count_before
finally:
    db.close()

def test_default_repository_root_is_this_projects_own_folder():
    settings = get_settings()
    if settings.repository_root.strip():
        return # explicitly overridden in this environment - nothing to assert
    root = settings.repository_root_path
    assert (root / "backend").is_dir()
    assert (root / "frontend").is_dir()

def test_api_requires_auth():
    assert client.post("/api/repository/scan").status_code == 401
    assert client.get("/api/repository").status_code == 401
    assert client.get("/api/repository/latest").status_code == 401

def test_api_scan_list_and_detail_round_trip():
    headers = _auth()
    response = client.post("/api/repository/scan", headers=headers)
    assert response.status_code == 201
    created = response.json()
    assert created["status"] == "completed"
    assert created["file_count"] >= 0

    listing = client.get("/api/repository", headers=headers)
    assert listing.status_code == 200
    assert any(s["id"] == created["id"] for s in listing.json())

    latest = client.get("/api/repository/latest", headers=headers)
    assert latest.status_code == 200
    assert latest.json()["id"] == created["id"]
    assert "files" in latest.json()

    detail = client.get(f"/api/repository/{created['id']}", headers=headers)
    assert detail.status_code == 200
    assert detail.json()["id"] == created["id"]

def test_api_unknown_scan_id_404s():
    headers = _auth()
    response = client.get("/api/repository/999999999", headers=headers)
    assert response.status_code == 404

```

tests/test_module15_phase2.py

232 lines

```
"""Verifies Module 15 Phase 2 (Repository Intelligence - Code
Understanding):

* A Python file gets exact imports/functions/classes/API-route/database-
reference/config-reference extraction via the stdlib `ast` module -
including this project's own real patterns: an APIRouter prefix
combined with a route decorator, a SQLAlchemy `tablename`, a
`Mapped["User"]`-style relationship forward-reference, `os.getenv`/
`os.environ[...]`, and a `settings.xxx` attribute access.
* A JavaScript file gets a best-effort regex extraction covering this
project's own conventions: ES imports, function/arrow-function
declarations, a class, an `apiRequest(...)`/`fetch(...)` call's route,
and `import.meta.env`/`process.env.*` references.
* A JSON config file's top-level keys become config references.
* A Python file with a genuine syntax error still gets indexed (path/
language/size/line-count are unaffected) - it just gets empty
understanding lists instead of failing the whole scan.
* A file type Phase 2 doesn't attempt (markdown) gets empty lists too -
never invented, never a reason to skip indexing the file itself.
* The API layer returns these six fields as real JSON lists (never a
raw JSON-encoded string, never null).

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.database.session import SessionLocal
from app.main import app
from app.services.code_understanding import understand_file
from app.services.repository_scanner import scan_repository

client = TestClient(app)

def _unique_email() -> str:
    return f"m15p2-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module15 Phase2 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

_PY_SAMPLE = '''\
import os
from fastapi import APIRouter, Depends
from app.models.user import User

router = APIRouter(prefix="/api/widgets", tags=["widgets"])

class Widget(Base):
    __tablename__ = "widgets"
    owner: Mapped["User"] = relationship(back_populates="widgets")

@router.post("/create")
def create_widget(current_user: User = Depends(get_current_user)):
    api_key = os.getenv("WIDGET_API_KEY")
    db_password = os.getenv("WIDGET_DB_PASSWORD")
    result = db.query(Widget)
    return settings.repository_root_path
'''

_JS_SAMPLE = """\
import { apiRequest } from './client'

export function createWidget(payload, token) {
    return apiRequest('/api/widgets/create', { method: 'POST', body: payload, token })
}

export const getWidget = (id, token) => {
    return apiRequest(`/api/widgets/${id}`, { token })
}

class WidgetForm {}

const key = import.meta.env.VITE_WIDGET_KEY
"""

_JSON_SAMPLE = '{"name": "widgets", "version": "1.0", "scripts": {"dev": "vite"}}'

def _make_sample_repo(tmp_path: Path) -> Path:
    root = tmp_path / "phase2_repo"
    root.mkdir()
    (root / "widgets.py").write_text(_PY_SAMPLE)
    (root / "widgets.js").write_text(_JS_SAMPLE)
    (root / "package.json").write_text(_JSON_SAMPLE)
    (root / "broken.py").write_text("def broken():\n    pass\n")
    (root / "README.md").write_text("# Widgets\n\nSome notes about widgets.\n")
    return root

def _file_by_path(scan, path: str):
    return next(f for f in scan.files if f.file_path == path)

def test_python_file_extraction(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
        widget = _file_by_path(scan, "widgets.py")

        assert set(json.loads(widget.imports)) >= {"os", "fastapi.APIRouter", "fastapi.Depends", "app.models.user.User"}
        assert json.loads(widget.functions) == ["create_widget"]
    
```

```

    assert json.loads(widget.classes) == ["Widget"]
    assert json.loads(widget.api_routes) == ["POST /api/widgets/create"]
    db_refs = set(json.loads(widget.database_references))
    assert "table:widgets" in db_refs
    assert "relationship:User" in db_refs
    assert "query:Widget" in db_refs
    config_refs = set(json.loads(widget.config_references))
    assert "env:WIDGET_API_KEY" in config_refs
    assert "env:WIDGET_DB_PASSWORD" in config_refs
    assert "settings.repository_root_path" in config_refs
finally:
    db.close()

def test_js_file_extraction(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
        widget_js = _file_by_path(scan, "widgets.js")

        assert json.loads(widget_js.imports) == [".client"]
        assert set(json.loads(widget_js.functions)) == {"createWidget", "getWidget"}
        assert json.loads(widget_js.classes) == ["WidgetForm"]
        assert set(json.loads(widget_js.api_routes)) == {"/api/widgets/create", "/api/widgets/${id}"}
        assert "env:VITE_WIDGET_KEY" in json.loads(widget_js.config_references)
    finally:
        db.close()

def test_json_config_extraction(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
        pkg = _file_by_path(scan, "package.json")
        config_refs = set(json.loads(pkg.config_references))
        assert {"key:name", "key:version", "key:scripts"} <= config_refs
    finally:
        db.close()

def test_syntax_error_file_still_indexed_with_empty_understanding(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
        broken = _file_by_path(scan, "broken.py")
        assert broken.line_count == 2
        assert broken.imports is None
        assert broken.functions is None
        assert broken.classes is None
    finally:
        db.close()

def test_unattempted_file_type_has_no_understanding_fields(tmp_path):
    root = _make_sample_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
        readme = _file_by_path(scan, "README.md")
        assert readme.imports is None
        assert readme.functions is None
        assert readme.database_references is None
    finally:
        db.close()

def test_understand_file_never_raises_on_garbage_input():
    # Defensive: whatever language string comes through, or unparseable
    # content, understand_file() always returns something, never raises -
    # a scan should never fail because one file's contents were odd.
    result = understand_file("\x00\x01 not real code {{{", "python")
    assert result.imports == []
    result = understand_file("", "unknown_language")
    assert result.functions == []

def test_api_returns_real_lists_not_json_strings():
    headers = _auth()
    response = client.post("/api/repository/scan", headers=headers)
    assert response.status_code == 201
    scan_id = response.json()["id"]

    detail = client.get(f"/api/repository/{scan_id}", headers=headers)
    assert detail.status_code == 200
    files = detail.json()["files"]
    assert len(files) > 0

    # This project's own backend/app/api/repository.py is always part of a
    # scan of its own repository - a real file, not a synthetic fixture -
    # confirming the whole pipeline (walk -> understand -> persist ->
    # serialize) works end to end on real code, not just a crafted sample.
    repository_router_file = next(
        (f for f in files if f["file_path"].endswith("backend/app/api/repository.py")), None
    )
    if repository_router_file is not None:
        assert isinstance(repository_router_file["imports"], list)
        assert isinstance(repository_router_file["api_routes"], list)
        assert any("scan" in route for route in repository_router_file["api_routes"])

    for f in files:
        for field_name in (
            "imports", "functions", "classes", "api_routes", "database_references", "config_references",
        ):
            assert isinstance(f[field_name], list)

```

```

"""Verifies Module 15 Phase 3 (Repository Intelligence - CR -> File
Matching): "which source files may actually be affected by this change
request?"

* A change request with no keyword overlap against anything a scan
indexed yields zero findings and never even calls the AI provider -
an honest "nothing found" is a valid outcome, not an error, and not
worth an AI call.
* A change request that DOES overlap with an indexed file's own
extracted imports/functions/etc gets a persisted RepositoryFinding
whose 'match_label' is computed purely from the numeric 'confidence'
the AI returned - never trusted from the AI's own wording - and whose
'change_request_version'/'analysis_id'/'repository_scan_id' are
exactly the ones the match actually ran against.
* A match naming a file that was never in the candidate shortlist (a
hallucinated path) is silently dropped, never persisted.
* Re-running a match for the exact same (change request, analysis,
scan) triple replaces that triple's own findings - it does not pile
up duplicates, and it does not touch a finding tied to a DIFFERENT
analysis or scan.
* The API 404s cleanly for a change request with no analysis yet.

The real AI provider is never called here - same monkeypatched
FakeProvider pattern as the other test module14 *.py / test module15 *.py
files, applied to both analysis engine.get_ai_provider (for /analyze) and
repository_matcher.get_ai_provider (for the match itself).

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.repository_matcher as repository_matcher
from app.database.session import SessionLocal
from app.main import app
from app.models.enums import FileMatchLabel
from app.models.repository_finding import RepositoryFinding
from app.services.repository_scanner import scan_repository

client = TestClient(app)

def _unique_email() -> str:
    return f"m15p3-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module15 Phase3 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify their identity with a one-time password (OTP) "
        "sent by SMS before completing a purchase.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FakeAnalysisProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "summary": "Adds OTP-based verification to the customer checkout flow.",
                "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
                "requirements": [
                    {
                        "category": "functional",
                        "description": "Customers receive and enter a one-time password (OTP) before checkout.",
                        "priority": "medium",
                        "certainty": "known",
                        "confidence": 90.0,
                        "evidence": "Explicitly requested.",
                    }
                ],
                "affected_components": [],
                "dependencies": [],
                "risks": [],
                "security_findings": [],
                "security_analysis": {"concerns": [], "summary": ""},
                "complexity": {"level": "low", "reasoning": "A contained auth feature."},
                "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
                "missing_information": [],
                "test_cases": [],
                "implementation_plan": [],
                "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
            }
        )

class _FakeMatchProvider:
    def __init__(self, response_obj):
        self._response_obj = response_obj

    def is_configured(self) -> bool:
        return True

```



```

def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
    return json.dumps(self._response_obj)

class _NeverCallProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        raise AssertionError("the AI provider should never be called when there are no candidate files")

def _setup_cr_with_analysis(monkeypatch, **cr_overrides):
    headers = {}
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FakeAnalysisProvider())
    created = _create_change_request(headers, **cr_overrides)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    analysis = response.json()
    return headers, created, analysis

def _make_otp_repo(tmp_path: Path) -> Path:
    root = tmp_path / "otp_repo"
    root.mkdir()
    (root / "otp_service.py").write_text(
        "import hashlib\n\ndef generate_otp():\n    return 123456\n\ndef send_otp_sms(phone):\n    pass\n"
    )
    (root / "unrelated_widget.py").write_text(
        "def render_widget():\n    return '<div>widget</div>'\n"
    )
    return root

def test_no_candidates_never_calls_ai(monkeypatch, tmp_path):
    headers, created, analysis = _setup_cr_with_analysis(monkeypatch, title="Improve warehouse label printing")
    root = tmp_path / "empty_repo"
    root.mkdir()
    (root / "totally_unrelated.py").write_text("def render_widget():\n    return 1\n")

    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(repository_matcher, "get_ai_provider", lambda: _NeverCallProvider())
    response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert response.status_code == 201
    assert response.json() == []

def test_match_creates_finding_with_computed_label(monkeypatch, tmp_path):
    headers, created, analysis = _setup_cr_with_analysis(monkeypatch)
    root = _make_otp_repo(tmp_path)

    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(
        repository_matcher,
        "get_ai_provider",
        lambda: _FakeMatchProvider(
            {
                "matches": [
                    {
                        "file_path": "otp_service.py",
                        "impact_level": "high",
                        "confidence": 88,
                        "reason": "Implements OTP generation and delivery, exactly what this change extends.",
                        "evidence": "Functions generate_otp, send_otp_sms",
                    }
                ]
            }
        ),
    )
    response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert response.status_code == 201
    findings = response.json()
    assert len(findings) == 1
    finding = findings[0]
    assert finding["file_path"] == "otp_service.py"
    assert finding["impact_level"] == "high"
    assert finding["confidence"] == 88
    assert finding["match_label"] == "likely_affected" # >= 75, see FileMatchLabel.from_confidence
    assert finding["change_request_id"] == created["id"]
    assert finding["analysis_id"] == analysis["id"]
    assert "unrelated_widget.py" not in finding["file_path"]

    listing = client.get(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert listing.status_code == 200
    assert len(listing.json()) == 1

def test_low_confidence_gets_possibly_related_label(monkeypatch, tmp_path):
    headers, created, analysis = _setup_cr_with_analysis(monkeypatch)
    root = _make_otp_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(
        repository_matcher,
        "get_ai_provider",
        lambda: _FakeMatchProvider(
            {
                "matches": [
                    {
                        "file_path": "otp_service.py",
                        "impact_level": "low",
                        "confidence": 30,
                        "reason": "Loosely related by naming only.",
                        "evidence": "file name relevance",
                    }
                ]
            }
        ),
    )
    response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert response.status_code == 201
    assert response.json()[0]["match_label"] == "possibly_related" # < 45

def test_hallucinated_file_path_is_dropped(monkeypatch, tmp_path):
    headers, created, analysis = _setup_cr_with_analysis(monkeypatch)
    root = _make_otp_repo(tmp_path)

```

```

db = SessionLocal()
try:
    scan = scan_repository(db, root_path=root)
finally:
    db.close()

monkeypatch.setattr(
    repository_matcher,
    "get_ai_provider",
    lambda: _FakeMatchProvider(
        {
            "matches": [
                {
                    "file_path": "otp_service.py",
                    "impact level": "high",
                    "confidence": 80,
                    "reason": "Real candidate.",
                    "evidence": "generate_otp",
                },
                {
                    "file_path": "this/file/was/never/indexed.py",
                    "impact level": "high",
                    "confidence": 95,
                    "reason": "Hallucinated - should be dropped.",
                    "evidence": "made up",
                },
            ],
        }
    ),
)
response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
assert response.status_code == 201
findings = response.json()
assert len(findings) == 1
assert findings[0]["file_path"] == "otp_service.py"

def test_rerun_replaces_findings_for_same_analysis_and_scan(monkeypatch, tmp_path):
    headers, created, analysis = _setup_cr_with_analysis(monkeypatch)
    root = _make_otp_repo(tmp_path)
    db = SessionLocal()
    try:
        scan = scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(
        repository_matcher,
        "get_ai_provider",
        lambda: _FakeMatchProvider({"matches": [{"file_path": "otp_service.py", "confidence": 80}]}),
    )
    first = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert len(first.json()) == 1

    monkeypatch.setattr(
        repository_matcher,
        "get_ai_provider",
        lambda: _FakeMatchProvider({"matches": [{"file_path": "otp_service.py", "confidence": 40}]}),
    )
    second = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert len(second.json()) == 1
    assert second.json()[0]["match_label"] == "possibly_related"

    db = SessionLocal()
    try:
        remaining = (
            db.query(RepositoryFinding)
            .filter(RepositoryFinding.change_request_id == created["id"])
            .all()
        )
        assert len(remaining) == 1 # the first run's row was replaced, not duplicated
    finally:
        db.close()

def test_no_analysis_yet_404s(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert response.status_code == 404

```

tests/test_module15_phase4.py

242 lines

```
"""Verifies Module 15 Phase 4 (Repository Intelligence - Version
Awareness + Re-scan):

* A repository finding, read back right after it's generated, is never
  outdated - it IS "the latest" of everything the moment it's written.
* Editing the change request afterward (bumping its version) makes
  every one of its repository findings report is_outdated=True with a
  reason naming the edit - via GET, without anything about the finding
  rows themselves changing (workflow_rules.repository_finding_
  outdated_reasons is a computed-fresh check, same rule
  is analysis outdated already follows for AI analyses).
* Running a NEW repository scan afterward (without touching the change
  request at all) makes existing findings outdated too, with a reason
  naming the newer scan - proving the three staleness triggers (CR
  edit / newer analysis / newer scan) are genuinely independent of each
  other, not all folded into "the CR changed."
* The underlying rule function (workflow_rules.
  repository_finding_outdated_reasons) is also exercised directly for
  its third branch (a newer AI analysis) - not reachable through the
  two repository-findings endpoints today, since both of them only ever
  operate on a change request's current latest analysis by
  construction, but the rule itself is written generally enough to
  apply if a future endpoint ever needs it (e.g. viewing an older
  analysis's own repository findings on a history view).

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.repository_matcher as repository_matcher
from app.database.session import SessionLocal
from app.main import app
from app.models.change_request import ChangeRequest
from app.services import workflow_rules
from app.services.repository_scanner import scan_repository

client = TestClient(app)

def _unique_email() -> str:
    return f"ml5p4-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module15 Phase4 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify their identity with a one-time password (OTP) "
        "sent by SMS before completing a purchase.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeAnalysisProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "summary": "Adds OTP-based verification to the customer checkout flow.",
                "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
                "requirements": [
                    {
                        "category": "functional",
                        "description": "Customers receive and enter a one-time password (OTP) before checkout.",
                        "priority": "medium",
                        "certainty": "known",
                        "confidence": 90.0,
                        "evidence": "Explicitly requested.",
                    }
                ],
                "affected_components": [],
                "dependencies": [],
                "risks": [],
                "security_findings": [],
                "security_analysis": {"concerns": [], "summary": ""},
                "complexity": {"level": "low", "reasoning": "A contained auth feature."},
                "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
                "missing_information": [],
                "test_cases": [],
                "implementation_plan": [],
                "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
            }
        )

class FakeMatchProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "matches": [
                    {

```

```

        "file_path": "otp_service.py",
        "impact_level": "high",
        "confidence": 85,
        "reason": "Implements OTP generation, exactly what this change extends.",
        "evidence": "Function generate_otp",
    }
}
)
}

def _make_otp_repo(tmp_path: Path) -> Path:
    root = tmp_path / "otp_repo"
    root.mkdir()
    (root / "otp_service.py").write_text("def generate_otp():\n    return 123456\n")
    return root

def _setup_cr_with_findings(monkeypatch, tmp_path):
    headers = _auth()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FakeAnalysisProvider())
    created = _create_change_request(headers)
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).raise_for_status()

    root = _make_otp_repo(tmp_path)
    db = SessionLocal()
    try:
        scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(repository_matcher, "get_ai_provider", lambda: FakeMatchProvider())
    response = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert response.status_code == 201
    assert len(response.json()) == 1
    return headers, created

def test_fresh_findings_are_not_outdated(monkeypatch, tmp_path):
    headers, created = _setup_cr_with_findings(monkeypatch, tmp_path)
    listing = client.get(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert listing.status_code == 200
    finding = listing.json()[0]
    assert finding["is_outdated"] is False
    assert finding["outdated_reasons"] == []

def test_editing_change_request_marks_findings_outdated(monkeypatch, tmp_path):
    headers, created = _setup_cr_with_findings(monkeypatch, tmp_path)

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"title": "Add OTP authentication for customers at checkout"},
        headers=headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["current_version"] == 2

    listing = client.get(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    finding = listing.json()[0]
    assert finding["is_outdated"] is True
    assert any("edited" in reason for reason in finding["outdated_reasons"])

def test_new_scan_marks_findings_outdated(monkeypatch, tmp_path):
    headers, created = _setup_cr_with_findings(monkeypatch, tmp_path)

    # A second scan of a DIFFERENT folder - the finding's own scan is no
    # longer "the latest" one, even though nothing about the change
    # request itself changed.
    other_root = tmp_path / "other_repo"
    other_root.mkdir()
    (other_root / "misc.py").write_text("def noop():\n    pass\n")
    db = SessionLocal()
    try:
        scan_repository(db, root_path=other_root)
    finally:
        db.close()

    listing = client.get(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    finding = listing.json()[0]
    assert finding["is_outdated"] is True
    assert any("newer repository scan" in reason for reason in finding["outdated_reasons"])

def test_repository_finding_outdated_reasons_unit():
    change_request = ChangeRequest(current_version=3)

    # Nothing has moved - no reasons.
    assert workflow_rules.repository_finding_outdated_reasons(
        change_request, 3, 10, 20, latest_analysis_id=10, latest_repository_scan_id=20
    ) == []

    # CR edited since (version 3 -> finding still says 2).
    reasons = workflow_rules.repository_finding_outdated_reasons(
        change_request, 2, 10, 20, latest_analysis_id=10, latest_repository_scan_id=20
    )
    assert len(reasons) == 1 and "edited" in reasons[0]

    # A newer analysis exists (not reachable via the current endpoints,
    # but the rule itself must still recognize it).
    reasons = workflow_rules.repository_finding_outdated_reasons(
        change_request, 3, 10, 20, latest_analysis_id=11, latest_repository_scan_id=20
    )
    assert len(reasons) == 1 and "newer AI analysis" in reasons[0]

    # A newer scan exists.
    reasons = workflow_rules.repository_finding_outdated_reasons(
        change_request, 3, 10, 20, latest_analysis_id=10, latest_repository_scan_id=21
    )
    assert len(reasons) == 1 and "newer repository scan" in reasons[0]

    # All three at once.
    reasons = workflow_rules.repository_finding_outdated_reasons(
        change_request, 1, 5, 5, latest_analysis_id=11, latest_repository_scan_id=21
    )
    assert len(reasons) == 3

```

tests/test_module15_phase5.py

322 lines

```
"""Verifies Module 15 Phase 5 (Repository Intelligence - Integration):
repository findings feed the existing Impact Analysis / Dependencies /
Risk / Implementation Plan / Test Cases sections, without creating a
second, parallel impact system (per the module's own spec and the
POST-MODULE-12 architecture lock).

* Once a repository match has run for a change request's current
analysis, GET .../analysis attaches `related_files` to every affected
component / impact assessment / dependency / risk / test case /
implementation task whose own text plausibly overlaps with one of the
CR's repository findings - and leaves it empty for one that doesn't,
proving this is real overlap-based linking, not "attach everything to
everything."
* The exact same happens on POST .../analyze's own response for an
analysis that already has repository findings tied to it.
* Before any repository match has ever been run, `related_files` is
simply [] everywhere - no crash, no special-casing needed by the
caller.
* Nothing about this phase invents a new persisted table or duplicates
RepositoryFinding itself - `related_files` is computed fresh on every
response (app/services/repository/linkage.py), exactly like
is_outdated/workflow_recommendation before it.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.repository_matcher as repository_matcher
from app.database.session import SessionLocal
from app.main import app
from app.services.repository_scanner import scan_repository

client = TestClient(app)

def unique_email() -> str:
    return f"m15p5-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module15 Phase5 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify their identity with a one-time password (OTP) "
        "sent by SMS before completing a purchase.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class FakeAnalysisProvider:
    """Populates one item in each of the 6 sections Phase 5 links against -
    one item's own words deliberately echo the repository finding's own
    reason/evidence text (about otp_service.py's OTP generation), the
    other-flavored fields (dependency/risk/test/task) are worded the same
    way so every section gets a genuine, checkable overlap - and nothing
    here is worded to overlap with the unrelated widget file."""

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "summary": "Adds OTP-based verification to the customer checkout flow.",
                "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
                "requirements": [],
                "affected_components": [
                    {
                        "name": "OTP Service",
                        "type": "backend",
                        "impact_level": "high",
                        "reason": "Generates the one-time password (OTP) used during checkout.",
                        "confidence": 90,
                        "certainty": "known",
                        "evidence": "New OTP generation logic.",
                    }
                ],
                "dependencies": [
                    {
                        "name": "SMS Gateway",
                        "type": "external",
                        "impact_level": "medium",
                        "reason": "Delivers the OTP generation output by text message.",
                    }
                ],
                "risks": [
                    {
                        "category": "security",
                        "description": "OTP generation could be brute-forced if not rate-limited.",
                        "severity": "medium",
                        "probability": 0.3,
                        "score": 40.0,
                        "mitigation": "Rate-limit OTP generation attempts per phone number.",
                    }
                ],
                "security_findings": [],
            }
        )
```

```

        "security_analysis": {"concerns": [], "summary": ""},
        "impact_assessments": [
            {
                "category": "security",
                "impact_level": "medium",
                "description": "Introduces new OTP generation and delivery into the checkout flow.",
            },
        ],
        "complexity": {"level": "low", "reasoning": "A contained auth feature."},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 developer-days"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-1",
                "title": "OTP generation succeeds",
                "type": "manual",
                "priority": "medium",
                "description": "Verify OTP generation returns a valid 6-digit code.",
                "expected_result": "A 6-digit OTP is generated.",
            },
        ],
        "implementation_plan": [
            {
                "task": "Build OTP generation endpoint",
                "description": "Implement the OTP generation function and wire it into checkout.",
                "component": "Backend",
                "priority": "medium",
                "estimated_effort": "2 days",
            },
        ],
        "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
    }
)

class FakeMatchProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "matches": [
                    {
                        "file_path": "otp_service.py",
                        "impact_level": "high",
                        "confidence": 88,
                        "reason": "Implements OTP generation, exactly what this change extends.",
                        "evidence": "Function generate_otp",
                    },
                ],
            },
        )

def _make_otp_repo(tmp_path: Path) -> Path:
    root = tmp_path / "otp_repo"
    root.mkdir()
    (root / "otp_service.py").write_text(
        "def generate_otp():\n    return 123456\n\ndef send_otp_sms(phone):\n    pass\n"
    )
    (root / "unrelated_widget.py").write_text("def render_widget():\n    return '<div>widget</div>'\n")
    return root

def _setup_cr_with_analysis_and_findings(monkeypatch, tmp_path):
    headers = auth()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FakeAnalysisProvider())
    created = create_change_request(headers)
    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze.status_code == 201

    root = _make_otp_repo(tmp_path)
    db = SessionLocal()
    try:
        scan_repository(db, root_path=root)
    finally:
        db.close()

    monkeypatch.setattr(repository_matcher, "get_ai_provider", lambda: FakeMatchProvider())
    match = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
    assert match.status_code == 201
    assert len(match.json()) == 1
    return headers, created

def test_related_files_empty_before_any_repository_match(monkeypatch):
    headers = auth()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FakeAnalysisProvider())
    created = create_change_request(headers)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    analysis = response.json()

    assert analysis["affected_components"][0]["related_files"] == []
    assert analysis["impact_assessments"][0]["related_files"] == []
    assert analysis["dependencies"][0]["related_files"] == []
    assert analysis["risks"][0]["related_files"] == []
    assert analysis["test_cases"][0]["related_files"] == []
    assert analysis["implementation_tasks"][0]["related_files"] == []

def test_analyze_response_links_repository_findings_when_they_already_exist(monkeypatch, tmp_path):
    # Run analysis + a repository match FIRST, then re-analyze - the second
    # /analyze response is for a brand-new Analysis row with no findings of
    # its own yet (matching is a separate, later step), so this proves
    # annotate_analysis_response() is a safe no-op there, not that it's
    # broken.
    headers, created = _setup_cr_with_analysis_and_findings(monkeypatch, tmp_path)
    reanalyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert reanalyze.status_code == 201
    assert reanalyze.json()["affected_components"][0]["related_files"] == []

def test_get_analysis_links_related_files_across_all_sections(monkeypatch, tmp_path):
    headers, created = _setup_cr_with_analysis_and_findings(monkeypatch, tmp_path)

    response = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert response.status_code == 200
    analysis = response.json()

    assert analysis["affected_components"][0]["related_files"] == ["otp_service.py"]
    assert analysis["impact_assessments"][0]["related_files"] == ["otp_service.py"]
    assert analysis["dependencies"][0]["related_files"] == ["otp_service.py"]
    assert analysis["risks"][0]["related_files"] == ["otp_service.py"]
    assert analysis["test_cases"][0]["related_files"] == ["otp_service.py"]
    assert analysis["implementation_tasks"][0]["related_files"] == ["otp_service.py"]

def test_no_overlap_item_gets_empty_related_files(monkeypatch, tmp_path):
    """An item whose own words share nothing meaningful with a finding's

```

own file_path/reason/evidence never gets that file attached, even though a real, persisted finding exists for this same analysis - proving this is genuine per-item overlap, not "attach every finding to every item."

The matched file below (checkout_widget.py) has to be a genuine candidate - it shares "checkout" with the change request's own title/description/analysis summary, which is what repository_matcher.score_candidates uses to shortlist candidates *before* the AI is ever called (a file with zero overlap there is dropped before reaching the AI at all, per Phase 3's own test_no_candidates_never_calls_ai - it would never make it into result.matches's candidate set in the first place). Its reason/evidence text is deliberately generic so it shares only that one word ("checkout") with the OTP-specific affected_component/risk wording below - one shared word is below repository_linkage's own 2-word minimum, so related_files stays empty for both.

```
headers = auth()
monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FakeAnalysisProvider())
created = create_change_request(headers)
client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).raise_for_status()

root = make_otp_repo(tmp_path)
(root / "checkout_widget.py").write_text(
    "def render_checkout_widget():\n    return '<div>checkout</div>'\n"
)
db = SessionLocal()
try:
    scan_repository(db, root_path=root)
finally:
    db.close()

class _GenericMatchProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(
            {
                "matches": [
                    {
                        "file_path": "checkout_widget.py",
                        "impact level": "low",
                        "confidence": 35,
                        "reason": "Only a loose, generic naming similarity to the checkout flow.",
                        "evidence": "file name relevance only",
                    }
                ]
            }
        )

monkeypatch.setattr(repository_matcher, "get_ai_provider", lambda: _GenericMatchProvider())
match = client.post(f"/api/change-requests/{created['id']}/repository-findings", headers=headers)
assert match.status_code == 201
assert len(match.json()) == 1

response = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
assert response.status_code == 200
analysis = response.json()
# A real finding exists for this analysis, but its own reason/evidence
# text ("loose, generic naming similarity" / "file name relevance")
# shares nothing meaningful with the OTP-generation-worded analysis
# items above - so every related_files list stays empty.
assert analysis["affected_components"][0]["related_files"] == []
assert analysis["risks"][0]["related_files"] == []
```

tests/test_module15_phase7_security.py

215 lines

```
"""Module 15 Phase 7 (Repository Intelligence - Security pass): a
dedicated test for the module's own spec rule 9 - "never expose API
keys/passwords/tokens/secrets/.env contents; don't index sensitive files
unnecessarily." Phases 1/2 already cover parts of this incidentally
(test_module15_phase1.py's "secrets are skipped" test,
test_module15_phase2.py's env-var-NAME extraction test) - this file is the
dedicated pass asked for by the spec, closing the gaps those didn't cover:

* A HARCODED secret literal - a bare module-level constant, a class
attribute, or a getenv/envIRON.get *fallback default* - is never
captured by the code-understanding extractor at all, under any field
name. This is the one gap Phase 2's own test didn't cover: it proved
the extractor captures the env var's NAME ("env:SECRET_KEY"), but never
proved a literal secret VALUE sitting elsewhere in the same file stays
out of every field.
* Every category of "secret-shaped" file the scanner is supposed to
refuse to open at all (.env and its .env.* variants, credentials.json,
a private key file) is skipped before scanning - re-verified together
here as one dedicated security checklist, rather than trusting each
exclusion is still in place only because Phase 1's own test happens to
check a couple of them.
* The exact shape of what gets sent to the AI for repository matching
(app/services/repository_matcher.py:: candidate digest) is locked down
to the known-safe field set - never raw file content, never anything
resembling a "value" or "content" key - so a future change can't
accidentally widen what leaves this app for an AI call.
* The two public API response schemas this module added
(IndexedFileRead, RepositoryFindingRead) are locked down to their known
field sets too, for the same "a future change can't accidentally add a
raw-content field" reason.

Run with (from backend/): pytest ../tests
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from app.database.session import SessionLocal
from app.schemas.repository_finding import RepositoryFindingRead
from app.schemas.repository_scan import IndexedFileRead
from app.services import repository_matcher
from app.services.code_understanding import understand_file
from app.services.repository_scanner import scan_repository

# Secret-shaped VALUES that must never appear anywhere in extracted
# structure, however they're written into source - only their key/variable
# NAMES are ever legitimate to extract.
_SECRET_VALUES = [
    "sk-hardcoded-real-secret-abc123xyz",
    "ghp_realtokenvalue1234567890",
    "hunter2-actual-password",
    "sk_live_51ABCDEFGHIJKLMNOP",
]

_PYTHON_SOURCE_WITH_HARCODED_SECRETS = '''
import os

SECRET_KEY = "sk-hardcoded-real-secret-abc123xyz"
API_TOKEN = "ghp_realtokenvalue1234567890"

def get_secret():
    fallback = os.getenv("SECRET_KEY", "sk-hardcoded-real-secret-abc123xyz")
    other = os.environ.get("DB_PASSWORD", "hunter2-actual-password")
    return fallback, other

class Config:
    STRIPE_KEY = "sk_live_51ABCDEFGHIJKLMNOP"
'''

def _all_extracted_strings(understanding) -> list[str]:
    return (
        understanding.imports
        + understanding.functions
        + understanding.classes
        + understanding.api_routes
        + understanding.database_references
        + understanding.config_references
    )

def test_hardcoded_secret_literals_never_leak_into_extracted_structure():
    understanding = understand_file(_PYTHON_SOURCE_WITH_HARCODED_SECRETS, "python")

    all_extracted = _all_extracted_strings(understanding)
    for secret_value in _SECRET_VALUES:
        assert not any(secret_value in extracted for extracted in all_extracted), (
            f"Secret value {secret_value!r} leaked into extracted structure: {all_extracted}"
        )

    # The env var NAMES themselves are still legitimately captured - this
    # isn't "extract nothing about config," just "never the value."
    assert "env:SECRET_KEY" in understanding.config_references
    assert "env:DB_PASSWORD" in understanding.config_references
    # The class constant and the bare module-level secrets aren't captured
    # under ANY field - the extractor only ever looks at getenv/envIRON/
    # settings access patterns for config references, never arbitrary
    # module-level or class-level assignments.
    assert understanding.config_references == ["env:SECRET_KEY", "env:DB_PASSWORD"]

def test_hardcoded_secret_in_javascript_never_leaks():
    js_source = """
const STRIPE_KEY = "sk_live_51ABCDEFGHIJKLMNOP"
const apiKey = process.env.WIDGET_API_KEY
function chargeCard() {
    return fetch('/api/charge')
}
"""
    understanding = understand_file(js_source, "javascript")
    all_extracted = _all_extracted_strings(understanding)
    assert not any("sk_live_51ABCDEFGHIJKLMNOP" in extracted for extracted in all_extracted)
    assert understanding.config_references == ["env:WIDGET_API_KEY"]

def test_secret_shaped_files_are_never_indexed(tmp_path):
    """One combined checklist for every "never open this at all" category
```



```

the module spec calls out, scanned together so a future refactor that
accidentally narrows the exclusion list fails one obvious test rather
than silently regressing one file type at a time."""
root = tmp_path / "secrets_repo"
root.mkdir()

(root / ".env").write_text("SECRET_KEY=super-secret-value\nDATABASE_PASSWORD=hunter2\n")
(root / ".env.production").write_text("SECRET_KEY=another-secret\n")
(root / "credentials.json").write_text('{"private key": "-----BEGIN PRIVATE KEY-----"}')
(root / "service-account.json").write_text('{"private key": "-----BEGIN PRIVATE KEY-----"}')
(root / "id_rsa.pem").write_text("-----BEGIN RSA PRIVATE KEY-----\nmIIeow...\n-----END RSA PRIVATE KEY-----\n")
(root / "server.key").write_text("-----BEGIN PRIVATE KEY-----\nmIIeow...\n-----END PRIVATE KEY-----\n")
(root / ".npmrc").write_text("//registry.npmjs.org/:_authToken=npm_realtoken123\n")
# A legitimate source file, so the scan itself succeeds and this test
# can tell "nothing indexed because everything was correctly skipped"
# apart from "nothing indexed because the scan itself failed."
(root / "app.py").write_text("def handler():\n    return 'ok'\n")

db = SessionLocal()
try:
    scan = scan_repository(db, root_path=root)
    # Asserted here, before the session closes below - scan.files is a
    # lazy relationship (same reason test module15 phase1.py's own scan
    # test reads it inside this same try block rather than after).
    assert scan.status.value == "completed"
    indexed_paths = {f.file_path for f in scan.files}
    assert indexed_paths == {"app.py"}
    assert scan.skipped_count >= 7
finally:
    db.close()

def test_ai_candidate_digest_never_includes_raw_content():
    """Locks down exactly what app/services/repository_matcher.py sends the
    AI provider for one candidate file - the known-safe field set only,
    never a "content"/"source"/"text" key that could carry raw file bytes
    (and therefore any secret literal sitting in them, whatever the
    scanner's own file-level exclusions miss)."""

    class FakeIndexedFile:
        file_path = "app/config.py"
        language = "python"
        imports = None
        functions = None
        classes = None
        api_routes = None
        database_references = None
        config_references = ["env:SECRET_KEY"]

    digest = repository_matcher._candidate_digest(_FakeIndexedFile())
    assert set(digest.keys()) == {
        "file_path",
        "language",
        "imports",
        "functions",
        "classes",
        "api_routes",
        "database_references",
        "config_references",
    }
    for forbidden_key in ("content", "source", "text", "body", "raw"):
        assert forbidden_key not in digest

def test_response_schemas_never_expose_raw_file_content():
    """IndexedFileRead and RepositoryFindingRead - the two schemas this
    module returns from the API - are locked to their known field sets, so
    a future field added to either model doesn't silently start returning
    raw file content to the frontend without anyone deciding that on
    purpose."""
    indexed_file_fields = set(IndexedFileRead.model_fields.keys())
    assert indexed_file_fields == {
        "id",
        "file_path",
        "language",
        "size_bytes",
        "line_count",
        "imports",
        "functions",
        "classes",
        "api_routes",
        "database_references",
        "config_references",
    }
    for forbidden_key in ("content", "source", "text", "body", "raw"):
        assert forbidden_key not in indexed_file_fields

    finding_fields = set(RepositoryFindingRead.model_fields.keys())
    for forbidden_key in ("content", "source", "text", "body", "raw", "file_content"):
        assert forbidden_key not in finding_fields

```

tests/test_module16_phase1.py

192 lines

```
"""Verifies Module 16 Phase 1 (Project Knowledge Base & RAG - Foundation):

* A valid .txt/.md upload is accepted: saved to disk under backend/data/
  knowledge_documents/, and a KnowledgeDocument row is created with the
  right title/extension/size (status/chunk_count now reflect Module 16
  Phase 2's synchronous parsing+chunking, added after this file was
  first written - see test_module16_phase2.py for the dedicated parsing/
  chunking tests).
* An unsupported extension (.docx), an empty file, and an oversized file
  are each rejected with a 400 and a plain-language reason - and nothing
  is written to disk or the database for a rejected upload.
* GET /api/knowledge/documents lists uploaded documents newest first,
  and never includes an archived document.
* GET /api/knowledge/documents/{id} returns one document, 404s for a
  nonexistent id.
* Every endpoint requires auth (401 with no token).
* The response schema never exposes raw file content - only the known
  metadata field set - mirroring Module 15 Phase 7's own schema lock-down
  test for IndexedFileRead/RepositoryFindingRead.

Run with (from backend/): pytest ../tests
"""
import io
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app
from app.schemas.knowledge_document import KnowledgeDocumentRead
from app.services import Knowledge_documents

client = TestClient(app)

def _unique_email() -> str:
    return f"m16p1-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Phase1 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

def test_valid_upload_creates_uploaded_document():
    headers = _auth()
    response = _upload(headers, "Security Guidelines.md", b"# Security Guidelines\n\nUse OTP for login.\n")
    assert response.status_code == 201
    body = response.json()
    assert body["title"] == "Security Guidelines"
    assert body["original_filename"] == "Security Guidelines.md"
    assert body["extension"] == ".md"
    # As of Module 16 Phase 2, the upload endpoint parses+chunks a
    # document synchronously right after saving it, so a valid upload
    # comes back already "ready" (see test_module16_phase2.py for the
    # dedicated parsing/chunking tests) rather than sitting at "uploaded"
    # forever - "uploaded" now only describes the brief in-flight moment
    # before that happens, not a resting state a caller can rely on
    # observing.
    assert body["status"] == "ready"
    assert body["chunk_count"] == 1
    assert body["archived"] is False
    assert body["size_bytes"] == len(b"# Security Guidelines\n\nUse OTP for login.\n")

    # The file actually landed on disk, under the expected storage root.
    from app.database.session import SessionLocal
    from app.models.knowledge_document import KnowledgeDocument

    db = SessionLocal()
    try:
        document = db.get(KnowledgeDocument, body["id"])
        assert document is not None
        path = knowledge_documents.document_file_path(document)
        assert path.exists()
        assert path.read_bytes() == b"# Security Guidelines\n\nUse OTP for login.\n"
    finally:
        db.close()

def test_unsupported_extension_is_rejected():
    headers = _auth()
    response = _upload(headers, "spec.docx", b"not really a docx but irrelevant here")
    assert response.status_code == 400
    assert "Unsupported file type" in response.json()["detail"]

def test_empty_file_is_rejected():
    headers = _auth()
    response = _upload(headers, "empty.txt", b"")
    assert response.status_code == 400
    assert "empty" in response.json()["detail"].lower()

def test_oversized_file_is_rejected():
    headers = _auth()
    oversized = b"x" * (knowledge_documents.MAX_DOCUMENT_BYTES + 1)
    response = _upload(headers, "huge.txt", oversized)
    assert response.status_code == 400
    assert "too large" in response.json()["detail"].lower()

def test_list_documents_newest_first_and_excludes_archived():
    headers = _auth()
```

```

first = _upload(headers, "Architecture Overview.txt", b"System overview text.")
second = _upload(headers, "API Reference.txt", b"API reference text.")
assert first.status_code == 201 and second.status_code == 201

from app.database.session import SessionLocal
from app.models.knowledge_document import KnowledgeDocument

db = SessionLocal()
try:
    archived_doc = db.get(KnowledgeDocument, first.json()["id"])
    archived_doc.archived = True
    db.commit()
finally:
    db.close()

response = client.get("/api/knowledge/documents", headers=headers)
assert response.status_code == 200
titles = [item["title"] for item in response.json()]
assert "API Reference" in titles
assert "Architecture Overview" not in titles
# Newest first: the un-archived "API Reference" document should appear,
# and appear before any document uploaded earlier by this same test run
# that also isn't archived (there's only one such document here).
assert titles[0] == "API Reference"

def test_get_document_by_id_and_404_for_missing():
    headers = _auth()
    uploaded = _upload(headers, "Deployment Guide.md", b"# Deployment\n\nSteps here.")
    assert uploaded.status_code == 201
    document_id = uploaded.json()["id"]

    response = client.get(f"/api/knowledge/documents/{document_id}", headers=headers)
    assert response.status_code == 200
    assert response.json()["title"] == "Deployment Guide"

    missing = client.get("/api/knowledge/documents/999999999", headers=headers)
    assert missing.status_code == 404

def test_endpoints_require_authentication():
    assert _upload({}, "doc.txt", b"content").status_code == 401
    assert client.get("/api/knowledge/documents").status_code == 401
    assert client.get("/api/knowledge/documents/1").status_code == 401

def test_response_schema_never_exposes_raw_file_content():
    """Locks KnowledgeDocumentRead to its known metadata field set - same
    "a future field can't silently start leaking raw content" reasoning as
    Module 15 Phase 7's schema lock-down test."""
    fields = set(KnowledgeDocumentRead.model_fields.keys())
    assert fields == {
        "id",
        "title",
        "original_filename",
        "extension",
        "size_bytes",
        "status",
        "error_message",
        "chunk_count",
        "archived",
        "uploaded_by",
        "uploaded_at",
        "processed_at",
    }
    for forbidden_key in ("content", "source", "text", "body", "raw", "file_content"):
        assert forbidden_key not in fields

```

tests/test_module16_phase2.py

211 lines

```
"""Verifies Module 16 Phase 2 (Project Knowledge Base & RAG - Parsing +
Chunking):

* A plain .txt upload becomes exactly one chunk (no natural section
structure to split on) and the document ends up "ready".
* A .md upload with headings splits into one chunk per heading, in
order, each carrying that heading's own text as section_label.
* A single paragraph longer than MAX_CHUNK_CHARS is hard-split into
several chunks whose combined length exactly reconstructs the
original text - nothing dropped, nothing duplicated.
* A .pdf upload is parsed page-by-page (via pypdf) - each page becomes
one chunk labeled "Page N" containing that page's own text.
* A .pdf that isn't actually a valid PDF is marked "failed" with a
plain-language error message - proving a parsing failure is a
recorded state, not a 500 or a silently-dropped upload (the module's
own spec asks to test "Failed documents").
* GET ../chunks returns a document's chunks in order and requires
auth; POST ../reprocess re-runs chunking and requires auth; both
404 for a document id that doesn't exist.
* KnowledgeChunkRead never exposes the internal `embedding` field.

Run with (from backend/): pytest ../tests
"""

import io
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app
from app.schemas.knowledge_chunk import KnowledgeChunkRead
from app.services.knowledge_chunker import MAX_CHUNK_CHARS

client = TestClient(app)

def _unique_email() -> str:
    return f"m16p2-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Phase2 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

def _make_pdf_bytes(text: str) -> bytes:
    """Builds a real one-page PDF containing `text`, using this project's
    own already-approved PDF library (fpdf2 - see
    app/services/report_generator.py, which builds the Module 6/13 PDF
    reports the same way) so this test needs no new/unapproved
    dependency beyond the pypdf *reader* Phase 2 itself adds."""
    from fpdf import FPDF

    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Helvetica", size=12)
    pdf.multi_cell(0, 10, text)
    raw = pdf.output()
    return bytes(raw) if isinstance(raw, (bytes, bytearray)) else raw.encode("latin-1")

def test_txt_upload_creates_single_ready_chunk():
    headers = _auth()
    response = _upload(headers, "Deployment Notes.txt", b"Deploy the backend, then the frontend. Done.")
    assert response.status_code == 201
    body = response.json()
    assert body["status"] == "ready"
    assert body["chunk_count"] == 1
    assert body["error_message"] is None
    assert body["processed_at"] is not None

    chunks = client.get(f"/api/knowledge/documents/{body['id']}/chunks", headers=headers).json()
    assert len(chunks) == 1
    assert chunks[0]["section_label"] is None
    assert chunks[0]["content"] == "Deploy the backend, then the frontend. Done."
    assert chunks[0]["chunk_index"] == 0

def test_markdown_upload_splits_by_heading_into_ordered_chunks():
    headers = _auth()
    markdown = (
        "# Introduction\n"
        "This document describes the OTP authentication rollout.\n\n"
        "## OTP Authentication\n"
        "Use a 6-digit OTP delivered by SMS. Rate-limit attempts per phone number.\n\n"
        "## Deployment\n"
        "Ship the backend first, then enable the feature flag.\n"
    )
    response = _upload(headers, "Security Guidelines.md", markdown.encode("utf-8"))
    assert response.status_code == 201
    body = response.json()
    assert body["status"] == "ready"
    assert body["chunk_count"] == 3

    chunks = client.get(f"/api/knowledge/documents/{body['id']}/chunks", headers=headers).json()
    assert [c["section_label"] for c in chunks] == ["Introduction", "OTP Authentication", "Deployment"]
    assert [c["chunk_index"] for c in chunks] == [0, 1, 2]
    assert "OTP" in chunks[1]["content"]
    assert "Ship the backend" in chunks[2]["content"]

def test_long_single_paragraph_hard_splits_across_multiple_chunks():
    headers = _auth()
```

```

# One paragraph (no blank lines) at exactly 2x the cap plus a partial
# third piece - deterministic chunk boundaries: MAX_CHUNK_CHARS,
# MAX_CHUNK_CHARS, then the 100-character remainder.
long_text = "A" * (2 * MAX_CHUNK_CHARS + 100)
response = upload(headers, "Big Spec.txt", long_text.encode("utf-8"))
assert response.status_code == 201
body = response.json()
assert body["status"] == "ready"
assert body["chunk_count"] == 3

chunks = client.get(f"/api/knowledge/documents/{body['id']}/chunks", headers=headers).json()
char_counts = [c["char_count"] for c in chunks]
assert char_counts == [MAX_CHUNK_CHARS, MAX_CHUNK_CHARS, 100]
# Nothing dropped, nothing duplicated - the pieces reconstruct the
# original text exactly.
assert "".join(c["content"] for c in chunks) == long_text

def test_pdf_upload_extracts_text_and_marks_ready():
    headers = _auth()
    pdf_bytes = make_pdf_bytes("This deployment guide covers OTP authentication rollout steps.")
    response = upload(headers, "Deployment Guide.pdf", pdf_bytes)
    assert response.status_code == 201
    body = response.json()
    assert body["extension"] == ".pdf"
    assert body["status"] == "ready"
    assert body["chunk_count"] >= 1

    chunks = client.get(f"/api/knowledge/documents/{body['id']}/chunks", headers=headers).json()
    assert chunks[0]["section_label"] == "Page 1"
    combined = " ".join(c["content"] for c in chunks)
    assert "OTP" in combined

def test_invalid_pdf_content_is_marked_failed_with_error_message():
    headers = _auth()
    # A .pdf extension around content that isn't a real PDF at all - the
    # upload itself is still accepted (extension/size/emptiness are the
    # only upload-time checks), but parsing fails and is recorded as such.
    response = upload(headers, "Not Actually A PDF.pdf", b"this is plain text, not a pdf file")
    assert response.status_code == 201
    body = response.json()
    assert body["status"] == "failed"
    assert body["chunk_count"] == 0
    assert body["error_message"]

def test_chunks_and_reprocess_endpoints_require_auth_and_404_for_missing_document():
    headers = _auth()
    uploaded = upload(headers, "Notes.txt", b"Some notes.")
    document_id = uploaded.json()["id"]

    assert client.get(f"/api/knowledge/documents/{document_id}/chunks").status_code == 401
    assert client.post(f"/api/knowledge/documents/{document_id}/reprocess").status_code == 401
    assert client.get("/api/knowledge/documents/999999999/chunks", headers=headers).status_code == 404
    assert client.post("/api/knowledge/documents/999999999/reprocess", headers=headers).status_code == 404

def test_reprocess_recomputes_chunks_for_an_already_ready_document():
    headers = _auth()
    uploaded = upload(headers, "Notes Again.txt", b"Some notes that will be reprocessed.")
    document_id = uploaded.json()["id"]

    response = client.post(f"/api/knowledge/documents/{document_id}/reprocess", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "ready"
    assert body["chunk_count"] == 1

    chunks = client.get(f"/api/knowledge/documents/{document_id}/chunks", headers=headers).json()
    assert len(chunks) == 1
    assert chunks[0]["content"] == "Some notes that will be reprocessed."

def test_chunk_schema_never_exposes_internal_embedding_field():
    fields = set(KnowledgeChunkRead.model_fields.keys())
    assert fields == {
        "id",
        "document_id",
        "chunk_index",
        "section_label",
        "content",
        "char_count",
        "is_embedded",
    }
    assert "embedding" not in fields

```

tests/test_module16_phase3.py

241 lines

```
"""Verifies Module 16 Phase 3 (Project Knowledge Base & RAG - Embedding +
Vector Search):

* POST .../embed calls the configured AI provider's embed() for every
chunk of a document and stores each vector - never exposed in any API
response (KnowledgeChunkRead/KnowledgeDocumentRead both omit it).
* Embedding a document with no chunks (e.g. one whose parsing failed -
see test_module16_phase2.py's own "failed document" test) is a clean
400, not a crash.
* A provider that isn't configured for embeddings (or at all) fails the
endpoint cleanly with a 400 and a plain-language reason, rather than a
bare 500 - the module's own spec asks for this to be testable.
* GET /api/knowledge/search embeds the query and ranks chunks by cosine
similarity to it - only chunks that are both embedded AND belong to a
non-archived document are ever candidates; 'top_k' limits how many
come back.
* Every test here uses a fake, deterministic embedding provider
(monkeypatched the same way tests/test_module15_phase4.py and
test_module15_phase5.py monkeypatch repository matcher's
get_ai_provider) - no real network call to OpenRouter, so this suite
stays fast, free, and reliable regardless of whatever's in the
developer's own backend/.env.

Run with (from backend/): pytest ../tests
"""
import io
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.knowledge_embeddings as knowledge_embeddings
from app.main import app
from app.schemas.knowledge_search import KnowledgeSearchResult

client = TestClient(app)

def _unique_email() -> str:
    return f"m16p3-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Phase3 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

def keyword_vector(text: str) -> list:
    """A tiny, fully deterministic 3-dimensional "embedding" - one
dimension per keyword - so a test can assert exact cosine-similarity
outcomes (1.0 for an exact keyword match, 0.0 for no overlap at all)
without depending on any real embedding model's actual output."""
    lowered = text.lower()
    return [
        1.0 if "otp" in lowered else 0.0,
        1.0 if "deployment" in lowered else 0.0,
        1.0 if "database" in lowered else 0.0,
    ]

class _FakeEmbeddingProvider:
    def is_configured(self) -> bool:
        return True

    def embed(self, texts, *, timeout=None):
        return [_keyword_vector(t) for t in texts]

class _UnconfiguredEmbeddingProvider:
    def is_configured(self) -> bool:
        return False

    def embed(self, texts, *, timeout=None):
        raise RuntimeError("OpenRouterProvider is not configured - set OPENROUTER_API_KEY in backend/.env")

def test_embed_populates_chunk_embeddings_and_hides_them_from_responses(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()
    uploaded = _upload(headers, "OTP Guide.txt", b"Use OTP for login security.")
    document_id = uploaded.json()["id"]

    response = client.post(f"/api/knowledge/documents/{document_id}/embed", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert "embedding" not in body
    assert body["status"] == "ready"

    from app.database.session import SessionLocal
    from app.models.knowledge_document import KnowledgeDocument

    db = SessionLocal()
    try:
        document = db.get(KnowledgeDocument, document_id)
        assert document.chunks[0].embedding is not None
        import json
        assert json.loads(document.chunks[0].embedding) == [1.0, 0.0, 0.0]
    finally:
        db.close()

    chunk_response = client.get(f"/api/knowledge/documents/{document_id}/chunks", headers=headers)
    assert "embedding" not in chunk_response.json()[0]
```

```

def test_embed_requires_auth_and_404_for_missing_document(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()
    uploaded = _upload(headers, "Notes.txt", b"Some notes.")
    document_id = uploaded.json()["id"]

    assert client.post(f"/api/knowledge/documents/{document_id}/embed").status_code == 401
    assert client.post("/api/knowledge/documents/999999999/embed", headers=headers).status_code == 404

def test_embed_fails_cleanly_when_provider_unconfigured(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _UnconfiguredEmbeddingProvider())
    headers = _auth()
    uploaded = _upload(headers, "Notes.txt", b"Some notes.")
    document_id = uploaded.json()["id"]

    response = client.post(f"/api/knowledge/documents/{document_id}/embed", headers=headers)
    assert response.status_code == 400
    assert "not configured" in response.json()["detail"].lower()

def test_embed_document_with_no_chunks_returns_400(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()
    # A .pdf extension around content that isn't a real PDF fails parsing
    # (see test_module16_phase2.py) and ends up with zero chunks.
    uploaded = _upload(headers, "Broken.pdf", b"this is not a real pdf")
    document_id = uploaded.json()["id"]
    assert uploaded.json()["status"] == "failed"
    assert uploaded.json()["chunk_count"] == 0

    response = client.post(f"/api/knowledge/documents/{document_id}/embed", headers=headers)
    assert response.status_code == 400
    assert "no chunks" in response.json()["detail"].lower()

def test_search_ranks_by_similarity_and_excludes_unembedded_and_archived_chunks(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    otp_doc = _upload(headers, "OTP Security.txt", b"Use OTP for every login attempt.").json()
    client.post(f"/api/knowledge/documents/{otp_doc['id']}/embed", headers=headers)

    deploy_doc = _upload(headers, "Deployment Steps.txt", b"Deployment steps for the backend service.").json()
    client.post(f"/api/knowledge/documents/{deploy_doc['id']}/embed", headers=headers)

    unembedded_doc = _upload(headers, "Database Notes.txt", b"Database schema notes.").json()
    # Deliberately never embedded - its chunk must never appear in search results.

    archived_doc = _upload(headers, "Old OTP Notes.txt", b"Archived OTP rollout notes.").json()
    client.post(f"/api/knowledge/documents/{archived_doc['id']}/embed", headers=headers)
    from app.database.session import SessionLocal
    from app.models.knowledge_document import KnowledgeDocument

    db = SessionLocal()
    try:
        doc = db.get(KnowledgeDocument, archived_doc["id"])
        doc.archived = True
        db.commit()
    finally:
        db.close()

    response = client.get("/api/knowledge/search", params={"q": "otp"}, headers=headers)
    assert response.status_code == 200
    results = response.json()
    document_ids = {r["document_id"] for r in results}

    assert unembedded_doc["id"] not in document_ids
    assert archived_doc["id"] not in document_ids
    assert otp_doc["id"] in document_ids
    assert deploy_doc["id"] in document_ids

    # Look up each of this test's own documents by id rather than assuming
    # a fixed position - other tests in this same shared database may have
    # already embedded their own "OTP"-flavored documents too, which
    # legitimately tie this test's otp_doc at a perfect 1.0 score (that's
    # correct search behavior, not a bug, so this test must tolerate it
    # rather than assuming otp_doc is the only 1.0 in the whole table).
    results_by_document = {r["document_id"]: r for r in results}
    assert results_by_document[otp_doc["id"]]["score"] == 1.0
    assert results_by_document[deploy_doc["id"]]["score"] == 0.0

    # Ranking itself still has to actually work - every returned score
    # sits in descending order, so a perfect match is never ranked behind
    # an unrelated one, whoever else's documents happen to also be in the
    # table.
    scores = [r["score"] for r in results]
    assert scores == sorted(scores, reverse=True)

def test_search_top_k_limits_results(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()
    for name, text in [
        ("otp1.txt", b"otp otp otp"),
        ("otp2.txt", b"otp again"),
        ("deploy1.txt", b"deployment notes"),
    ]:
        doc = _upload(headers, name, text).json()
        client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)

    response = client.get("/api/knowledge/search", params={"q": "otp", "top_k": 1}, headers=headers)
    assert response.status_code == 200
    assert len(response.json()) == 1

def test_search_requires_authentication():
    assert client.get("/api/knowledge/search", params={"q": "otp"}).status_code == 401

def test_search_result_schema_field_set():
    fields = set(KnowledgeSearchResult.model_fields.keys())
    assert fields == {"score", "chunk_id", "document_id", "document_title", "section_label", "content"}
    assert "embedding" not in fields

```

tests/test_module16_phase4.py

275 lines

```
"""Verifies Module 16 Phase 4/5 (Project Knowledge Base & RAG - CR
Analysis Integration + Source Attribution & Version Awareness):

* When the knowledge base has a document genuinely relevant to a change
request, POST .../analyze retrieves it, injects a "Relevant
Documentation" section (with Source/Section attribution) into the AI
prompt, and persists one KnowledgeEvidence row per retrieved chunk -
returned on the analysis response as 'knowledge_evidence'.
* When nothing in the knowledge base is actually relevant, no
"Relevant Documentation" section is injected at all, and
"knowledge_evidence" is simply [] - proving this is genuine relevance
filtering (min score), not "always attach the closest thing on file."
* A knowledge-base problem (the embedding provider itself unavailable)
never fails the analysis itself - retrieval degrades to [] and the
core analysis still succeeds - per the module's own "optional
grounding, not a hard dependency" design.
* Evidence carries a real source-attribution fields (document, section,
content, similarity score) and becomes "outdated" exactly when its
parent analysis does (the change request was edited since) - the
module's own version-awareness requirement.
* KnowledgeEvidenceRead never exposes anything beyond its documented
field set.

Every test here uses fake, monkeypatched AI providers for both the
analysis completion call (analysis_engine.get ai provider) and the
embedding call (knowledge_embeddings.get ai provider) - no real network
call, matching this app's established testing pattern.

Run with (from backend/): pytest ../tests
"""
import io
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.knowledge_embeddings as knowledge_embeddings
from app.main import app
from app.schemas.knowledge_evidence import KnowledgeEvidenceRead

client = TestClient(app)

def _unique_email() -> str:
    return f"m16p4-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Phase4 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def keyword_vector(text: str) -> list:
    """A tiny, self-contained "embedding" scoped to this test
    file only (a different dimensionality than any other test file's own
    fake scheme, so cross-test leftovers can never accidentally score a
    false match - see knowledge_embeddings.py::_cosine_similarity's own
    dimension-mismatch guard)."""
    lowered = text.lower()
    return [1.0 if "otp" in lowered else 0.0]

class FakeEmbeddingProvider:
    def is_configured(self) -> bool:
        return True

    def embed(self, texts, *, timeout=None):
        return [_keyword_vector(t) for t in texts]

class UnavailableEmbeddingProvider:
    def is_configured(self) -> bool:
        return False

    def embed(self, texts, *, timeout=None):
        raise RuntimeError("OpenRouterProvider is not configured - set OPENROUTER_API_KEY in backend/.env")

_VALID_ANALYSIS_JSON = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_findings": [],
    "security_analysis": {"concerns": [], "summary": ""},
}
```



```

"impact_assessments": [],
"complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
"effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
"missing_information": [],
"test_cases": [],
"implementation_plan": [],
"recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

```

```

class PromptCheckingAnalysisProvider:
    """Returns a fixed valid analysis, but first records whether the
    prompt it was given actually contained the "Relevant Documentation"
    section - lets a test assert on prompt content without needing a real
    AI call."""

    def __init__(self):
        self.last_prompt: str | None = None

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt: str, *, system=None, max_tokens=1024, timeout=None):
        self.last_prompt = prompt
        return json.dumps(_VALID_ANALYSIS_JSON)

def test_analyze_injects_documentation_and_persists_knowledge_evidence(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(
        headers,
        "OTP Security Guidelines.md",
        b"# OTP Authentication\n\nRate-limit OTP attempts per phone number and expire codes after 5 minutes.\n",
    ).json()
    assert doc["status"] == "ready"
    embed_response = client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)
    assert embed_response.status_code == 200

    fake_provider = PromptCheckingAnalysisProvider()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: fake_provider)

    created = _create_change_request(headers)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    assert "Relevant Documentation" in fake_provider.last_prompt
    assert "OTP Security Guidelines" in fake_provider.last_prompt
    assert "Source:" in fake_provider.last_prompt
    assert "Section:" in fake_provider.last_prompt

    evidence = body["knowledge_evidence"]
    assert len(evidence) == 1
    assert evidence[0]["document_id"] == doc["id"]
    assert evidence[0]["document_title"] == "OTP Security Guidelines"
    assert evidence[0]["section_label"] == "OTP Authentication"
    assert evidence[0]["similarity_score"] == 1.0
    assert evidence[0]["change_request_version"] == 1
    assert evidence[0]["is_outdated"] is False
    assert "Rate-limit OTP attempts" in evidence[0]["content_snippet"]

def test_analyze_with_no_relevant_documentation_has_empty_evidence(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    # Even with an embedded document in the system (from another test, or
    # created here), a change request about something entirely unrelated
    # must retrieve nothing - min_score filtering, not "always attach the
    # closest thing on file."
    unrelated_doc = _upload(headers, "OTP Guidelines.md", b"OTP rate limiting guidance.").json()
    client.post(f"/api/knowledge/documents/{unrelated_doc['id']}/embed", headers=headers)

    fake_provider = PromptCheckingAnalysisProvider()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: fake_provider)

    created = _create_change_request(
        headers,
        title="Repaint the office lobby",
        description="Repaint the lobby walls a lighter shade of grey before the open house.",
    )
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    assert body["knowledge_evidence"] == []
    assert "Relevant Documentation" not in fake_provider.last_prompt

def test_analyze_succeeds_even_when_embedding_provider_is_unavailable(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _UnavailableEmbeddingProvider())
    fake_provider = PromptCheckingAnalysisProvider()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: fake_provider)

    headers = _auth()
    created = _create_change_request(headers)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    assert response.json()["knowledge_evidence"] == []
    assert "Relevant Documentation" not in fake_provider.last_prompt

def test_knowledge_evidence_becomes_outdated_after_change_request_is_edited(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(headers, "OTP Policy.md", b"OTP codes must expire after 5 minutes.").json()
    client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)

    fake_provider = PromptCheckingAnalysisProvider()
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: fake_provider)

    created = _create_change_request(headers)
    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze.status_code == 201
    assert analyze.json()["knowledge_evidence"][0]["is_outdated"] is False

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"title": "Add OTP authentication for customers at checkout"},
        headers=headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["current_version"] == 2

    latest = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert latest.status_code == 200
    body = latest.json()
    assert body["is_outdated"] is True

```

```
assert body["knowledge_evidence"][0]["is_outdated"] is True
# The evidence row itself never changes what it recorded - it still
# says which version it was actually retrieved against.
assert body["knowledge_evidence"][0]["change_request_version"] == 1

def test_knowledge_evidence_schema_field_set():
    fields = set(KnowledgeEvidenceRead.model_fields.keys())
    assert fields == {
        "id",
        "document_id",
        "document_title",
        "section_label",
        "content_snippet",
        "similarity_score",
        "change_request_version",
        "retrieved_at",
        "is_outdated",
    }
```

tests/test_module16_phase6.py

154 lines

```
"""Verifies the Module 16 Frontend phase's backend support pieces (the
parts of this phase that aren't purely visual and can be tested here):

* GET /api/knowledge/documents/{id}/chunks reports `is embedded` per
  chunk correctly - False right after upload/chunk (Phase 2), True for
  every chunk once the document has been embedded (Phase 3) - without
  ever exposing the raw embedding vector itself (already covered by
  test_module16_phase2.py's own schema field-set test, re-affirmed here
  since this test also asserts "embedding" is absent from the response
  JSON).
* POST .../documents/{id}/archive marks a document archived - it drops
  out of the default GET /api/knowledge/documents list, reappears with
  `?include_archived=true`, and its chunks stay fully intact (never
  deleted).
* POST .../documents/{id}/unarchive reverses that - the document comes
  back in the default list.
* An archived document's embedded chunks are excluded from
  GET /api/knowledge/search results (this was already true for
  search_chunks() itself per test_module16_phase3.py; this test
  confirms it holds through the new archive endpoint specifically,
  since that's the only new way a document actually becomes archived).

Every test here uses a fake, deterministic embedding provider, the same
pattern as test_module16_phase3.py and test_module16_phase4.py - no real
network call.

Run with (from backend/): python -m pytest ../tests
"""
import io
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.knowledge_embeddings as knowledge_embeddings
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m16p6-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Frontend-Phase Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

class _FakeEmbeddingProvider:
    def is_configured(self) -> bool:
        return True

    def embed(self, texts, *, timeout=None):
        return [[1.0 if "otp" in t.lower() else 0.0] for t in texts]

def test_chunks_report_is_embedded_false_then_true_without_exposing_vector(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(headers, "OTP Notes.txt", b"OTP codes expire after five minutes.").json()
    assert doc["status"] == "ready"

    before = client.get(f"/api/knowledge/documents/{doc['id']}/chunks", headers=headers)
    assert before.status_code == 200
    before_chunks = before.json()
    assert len(before_chunks) >= 1
    assert all(chunk["is_embedded"] is False for chunk in before_chunks)
    assert all("embedding" not in chunk for chunk in before_chunks)

    embed_response = client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)
    assert embed_response.status_code == 200

    after = client.get(f"/api/knowledge/documents/{doc['id']}/chunks", headers=headers)
    assert after.status_code == 200
    after_chunks = after.json()
    assert len(after_chunks) == len(before_chunks)
    assert all(chunk["is_embedded"] is True for chunk in after_chunks)
    assert all("embedding" not in chunk for chunk in after_chunks)

def test_archive_hides_document_from_default_list_but_keeps_chunks(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(headers, "Archivable Guide.txt", b"OTP guidance to be archived.").json()

    archive_response = client.post(f"/api/knowledge/documents/{doc['id']}/archive", headers=headers)
    assert archive_response.status_code == 200
    assert archive_response.json()["archived"] is True

    default_list = client.get("/api/knowledge/documents", headers=headers).json()
    assert all(d["id"] != doc["id"] for d in default_list)

    with archived = client.get("/api/knowledge/documents?include_archived=true", headers=headers).json()
    assert any(d["id"] == doc["id"] and d["archived"] is True for d in with_archived)

    chunks = client.get(f"/api/knowledge/documents/{doc['id']}/chunks", headers=headers)
    assert chunks.status_code == 200
    assert len(chunks.json()) >= 1
```

```

def test_unarchive_restores_document_to_default_list(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(headers, "Restorable Guide.txt", b"OTP guidance to be restored.").json()
    client.post(f"/api/knowledge/documents/{doc['id']}/archive", headers=headers)

    unarchive_response = client.post(f"/api/knowledge/documents/{doc['id']}/unarchive", headers=headers)
    assert unarchive_response.status_code == 200
    assert unarchive_response.json()["archived"] is False

    default_list = client.get("/api/knowledge/documents", headers=headers).json()
    assert any(d["id"] == doc["id"] for d in default_list)

def test_archived_documents_embedded_chunks_excluded_from_search(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = _upload(headers, "Searchable OTP Guide.txt", b"OTP rate limiting details.").json()
    client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)

    before_archive = client.get("/api/knowledge/search?q=otp", headers=headers).json()
    assert any(r["document_id"] == doc["id"] for r in before_archive)

    client.post(f"/api/knowledge/documents/{doc['id']}/archive", headers=headers)

    after_archive = client.get("/api/knowledge/search?q=otp", headers=headers).json()
    assert all(r["document_id"] != doc["id"] for r in after_archive)

```

tests/test_module16_phase7_security.py

296 lines

```
"""Module 16 Phase 7 - dedicated security pass for Project Knowledge Base
& RAG, mirroring test_module15_phase7_security.py's own approach (a
dedicated audit, not just incidental coverage from earlier phases):

* Raw embedding vectors - the one genuinely sensitive-shaped internal
value this module ever computes - never leak through ANY API response
(document read, chunk list, search results, or CR-analysis evidence),
even once a document has actually been embedded.
* Every knowledge-base endpoint requires authentication - no anonymous
read or write access to uploaded documents, search, or the knowledge
base's own management actions (embed/reprocess/archive/unarchive).
* A filename crafted to escape the storage directory (path traversal)
can never write outside backend/data/knowledge_documents/ - upload
always resolves to a flat, sanitized filename under that one folder,
regardless of what the browser sent as the original filename.
* An archived document's chunks - even ones that would otherwise be a
perfect keyword/embedding match - are never retrieved into a real
CR analysis's "Relevant Documentation" grounding context. Archiving
is a genuine access boundary for retrieval, not just a list-filter.
* Every response schema this module defines (KnowledgeDocumentRead,
KnowledgeChunkRead, KnowledgeSearchResult, KnowledgeEvidenceRead) is
locked to its documented field set - a future change to any of them
can't silently start returning something new (like the raw embedding,
or a server filesystem path) without a test having to be updated to
notice it.

Role-based restriction (who may upload/delete vs who may only view) is
NOT added here - per this module's own spec section 9 and api/knowledge.py's
own docstring, "any authenticated user" remains the deliberate starting
point for this hackathon, the same scope Module 15's own repository
endpoints shipped with (see PROJECT_REPORT.md's Known Limitations for
both modules). This pass verifies the boundaries that DO exist -
authentication, embedding-vector secrecy, path safety, and archive-as-
retrieval-boundary - are real and enforced in code, not just assumed.

Run with (from backend/): python -m pytest ../tests
"""
import io
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.knowledge_embeddings as knowledge_embeddings
from app.main import app
from app.schemas.knowledge_chunk import KnowledgeChunkRead
from app.schemas.knowledge_document import KnowledgeDocumentRead
from app.schemas.knowledge_evidence import KnowledgeEvidenceRead
from app.schemas.knowledge_search import KnowledgeSearchResult
from app.services.knowledge_documents import _storage_root

client = TestClient(app)

def unique_email() -> str:
    return f"m16p7-{uuid.uuid4().hex[:10]}@example.com"

def auth() -> dict:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module16 Phase7 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def upload(headers, filename, content: bytes):
    return client.post(
        "/api/knowledge/documents",
        headers=headers,
        files={"file": (filename, io.BytesIO(content), "application/octet-stream")},
    )

class FakeEmbeddingProvider:
    def is_configured(self) -> bool:
        return True

    def embed(self, texts, *, timeout=None):
        return [[1.0 if "otp" in t.lower() else 0.0] for t in texts]

_VALID_ANALYSIS_JSON = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_findings": [],
    "security_analysis": {"concerns": [], "summary": ""},
    "impact_assessments": [],
    "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
    "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

class FixedAnalysisProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(_VALID_ANALYSIS_JSON)

def create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
    }
```

```

        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def test_embedding_vector_never_leaks_through_any_response(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    doc = upload(headers, "OTP Guide.md", b"# OTP\n\nOTP codes expire after five minutes.\n").json()
    embed_response = client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)
    assert embed_response.status_code == 200
    assert "embedding" not in embed_response.text

    get_response = client.get(f"/api/knowledge/documents/{doc['id']}", headers=headers)
    assert "embedding" not in get_response.text

    chunks_response = client.get(f"/api/knowledge/documents/{doc['id']}/chunks", headers=headers)
    assert "embedding" not in chunks_response.text

    search_response = client.get("/api/knowledge/search?q=otp", headers=headers)
    assert search_response.status_code == 200
    assert "embedding" not in search_response.text

def test_every_knowledge_endpoint_requires_authentication(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()
    doc = _upload(headers, "Auth Boundary Guide.txt", b"OTP guidance for the auth-boundary test.").json()

    # No Authorization header on any of them - each must be a clean 401/403,
    # never a 200 or an unhandled error.
    unauthenticated_calls = [
        ("GET", "/api/knowledge/documents"),
        ("GET", f"/api/knowledge/documents/{doc['id']}"),
        ("GET", f"/api/knowledge/documents/{doc['id']}/chunks"),
        ("GET", "/api/knowledge/search?q=otp"),
        ("POST", f"/api/knowledge/documents/{doc['id']}/embed"),
        ("POST", f"/api/knowledge/documents/{doc['id']}/reprocess"),
        ("POST", f"/api/knowledge/documents/{doc['id']}/archive"),
        ("POST", f"/api/knowledge/documents/{doc['id']}/unarchive"),
    ]
    for method, path in unauthenticated_calls:
        response = client.request(method, path)
        assert response.status_code in (401, 403), f"{method} {path} should require auth, got {response.status_code}"

    # Uploading with no auth header must also be rejected, not silently accepted.
    upload_response = client.post(
        "/api/knowledge/documents",
        files={"file": ("no-auth.txt", io.BytesIO(b"should be rejected")), "text/plain": ""},
    )
    assert upload_response.status_code in (401, 403)

def test_path_traversal_filename_cannot_escape_storage_directory(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    headers = _auth()

    malicious_names = [
        ".././../etc/passwd.txt",
        "..\\..\\windows\\win.ini.txt",
        "/etc/cron.d/evil.txt",
    ]
    storage_root = _storage_root().resolve()

    for name in malicious_names:
        response = upload(headers, name, b"content from a path-traversal attempt")
        assert response.status_code == 201, f"upload with filename {name!r} should still succeed safely"
        # KnowledgeDocumentRead never exposes the server-side stored filename
        # or path (see test_response_schemas locked to their documented
        # field sets below) - so the check here is on the storage directory
        # itself: every file it now holds must resolve inside storage_root,
        # flat, no subdirectories created by a traversal attempt.
        entries = list(storage_root.rglob("*"))
        for entry in entries:
            resolved = entry.resolve()
            assert storage_root in resolved.parents or resolved == storage_root, (
                f"a file for upload {name!r} escaped the storage root: {resolved}"
            )
        # Never a subdirectory - every stored file sits flat in storage_root.
        if entry.is_file():
            assert entry.parent == storage_root

def test_archived_documents_excluded_from_cr_analysis_retrieval(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedAnalysisProvider())
    headers = _auth()

    doc = upload(headers, "OTP Archived Guide.md", b"# OTP\n\nOTP rate limiting details for Alight.com.\n").json()
    client.post(f"/api/knowledge/documents/{doc['id']}/embed", headers=headers)
    archive_response = client.post(f"/api/knowledge/documents/{doc['id']}/archive", headers=headers)
    assert archive_response.status_code == 200

    created = create_change_request(headers)
    analyze_response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze_response.status_code == 201
    body = analyze_response.json()

    # A perfect keyword/embedding match, but the document is archived - it
    # must never surface as evidence, regardless of how relevant its text is.
    # (Asserting the evidence list is entirely empty would be too strong a
    # claim here: this suite's tests all share one SQLite database for the
    # whole run - test_module16_phase4.py and test_module16_phase6.py both
    # embed their own "OTP"-related documents using this same simple 1-
    # dimensional keyword scheme, and this test reuses create_change
    # request's default "Add OTP authentication..." title, so a *different*,
    # non-archived document from an earlier test file can legitimately also
    # match and appear here. What must be true - the only thing this test is
    # actually about - is that THIS document, once archived, is never among
    # them.)
    assert not any(e["document_id"] == doc["id"] for e in body["knowledge_evidence"])

def test_response_schemas_locked_to_their_documented_field_sets():
    assert set(KnowledgeDocumentRead.model_fields.keys()) == {
        "id",
        "title",
        "original_filename",
        "extension",
        "size_bytes",
        "status",
        "error_message",
        "chunk_count",
        "archived",
    }

```

```

        "uploaded_by",
        "uploaded_at",
        "processed_at",
    }
    assert set(KnowledgeChunkRead.model_fields.keys()) == {
        "id",
        "document_id",
        "chunk_index",
        "section_label",
        "content",
        "char_count",
        "is_embedded",
    }
    assert set(KnowledgeSearchResult.model_fields.keys()) == {
        "score",
        "chunk_id",
        "document_id",
        "document_title",
        "section_label",
        "content",
    }
    assert set(KnowledgeEvidenceRead.model_fields.keys()) == {
        "id",
        "document_id",
        "document_title",
        "section_label",
        "content_snippet",
        "similarity_score",
        "change_request_version",
        "retrieved_at",
        "is_outdated",
    }
    # None of the four schemas ever names the raw vector or a server
    # filesystem path - the two internal shapes this module must never
    # expose - under any field name.
    for schema in (KnowledgeDocumentRead, KnowledgeChunkRead, KnowledgeSearchResult, KnowledgeEvidenceRead):
        fields = set(schema.model_fields.keys())
        assert "embedding" not in fields
        assert "stored_filename" not in fields
        assert not any("path" in f for f in fields)

```

tests/test_module17_phase1.py

253 lines

```
"""Verifies Module 17 Phase 1 (Test Cases & Implementation Plan 2.0 -
Foundation):

* The new TestType values (negative/boundary/api/ui/data validation) and
HistoryAction values (test_case_edited/implementation_task_edited)
exist with the exact string values the rest of the app will rely on.
* TestCaseRead/ImplementationTaskRead expose every new Phase 1+ field
(preconditions, steps, owner suggestion, requirement/risk references,
analysis_version, is_outdated, edited_by/edited_at/edit_reason) with
safe defaults for an analysis that predates this module - nothing
from before Module 17 breaks.
* 'steps' round-trips correctly through its JSON-encoded storage column,
the same pattern ChangeRequest.tags already uses.
* analysis_version and is_outdated are populated fresh by the API layer
on every test case/task, mirroring KnowledgeEvidenceRead.is_outdated -
both start false/current right after a fresh analysis, and is_outdated
flips to true for every test case/task once the change request is
edited, exactly like the analysis (and its knowledge evidence)
already do.

Every test here uses a fake, monkeypatched AI provider (analysis engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app
from app.models.enums import HistoryAction, TestType
from app.schemas.implementation_task import ImplementationTaskRead
from app.schemas.test_case import TestCaseRead

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p1-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase1 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

_VALID_ANALYSIS_JSON = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_findings": [],
    "security_analysis": {"concerns": [], "summary": ""},
    "impact_assessments": [
        {"complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-001",
                "title": "OTP code expires after 5 minutes",
                "type": "security",
                "priority": "high",
                "description": "Verify an OTP code can't be used once it has expired.",
                "expected_result": "The expired code is rejected with a clear error.",
            }
        ]
    ],
    "implementation_plan": [
        {
            "task": "Add OTP generation endpoint",
            "description": "Backend endpoint that generates and stores a one-time password.",
            "component": "Backend",
            "priority": "high",
            "estimated_effort": "2 days",
            "dependencies": None,
        }
    ],
    "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

class FixedAnalysisProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(_VALID_ANALYSIS_JSON)

def test_new_enum_values_exist_with_expected_string_values():
    assert TestType.NEGATIVE.value == "negative"
```



```

assert TestType.BOUNDARY.value == "boundary"
assert TestType.API.value == "api"
assert TestType.UI.value == "ui"
assert TestType.DATA_VALIDATION.value == "data_validation"
assert HistoryAction.TEST_CASE_EDITED.value == "test_case_edited"
assert HistoryAction.IMPLEMENTATION_TASK_EDITED.value == "implementation_task_edited"

def test_schemas_expose_every_new_field_with_safe_defaults():
    test_case_fields = set(TestCaseRead.model_fields.keys())
    assert {
        "preconditions",
        "steps",
        "requirement_references",
        "risk_references",
        "analysis_version",
        "is_outdated",
        "edited_by",
        "edited_at",
        "edit_reason",
    }.issubset(test_case_fields)

    task_fields = set(ImplementationTaskRead.model_fields.keys())
    assert {
        "owner_suggestion",
        "requirement_references",
        "analysis_version",
        "is_outdated",
        "edited_by",
        "edited_at",
        "edit_reason",
    }.issubset(task_fields)

    # An analysis from before this module ran has none of this recorded -
    # the schema must still validate cleanly with every new field at its
    # safe default (empty list / None / False), never a validation error.
    pre_module17_test_case = TestCaseRead(
        id=1,
        analysis_id=1,
        test_id="TC-001",
        title="Legacy test",
        test_type="manual",
        priority="medium",
        description="d",
        expected_result="e",
    )
    assert pre_module17_test_case.preconditions is None
    assert pre_module17_test_case.steps == []
    assert pre_module17_test_case.requirement_references == []
    assert pre_module17_test_case.is_outdated is False
    assert pre_module17_test_case.edited_by is None

    pre_module17_task = ImplementationTaskRead(
        id=1,
        analysis_id=1,
        task="Legacy task",
        description="d",
        component="Backend",
        priority="medium",
        estimated_effort="1 day",
    )
    assert pre_module17_task.owner_suggestion is None
    assert pre_module17_task.is_outdated is False

def test_steps_round_trips_through_json_storage():
    parsed = TestCaseRead(
        id=1,
        analysis_id=1,
        test_id="TC-002",
        title="Steps round-trip",
        test_type="unit",
        priority="medium",
        description="d",
        expected_result="e",
        steps=json.dumps(["Open checkout", "Enter an expired OTP code", "Submit"]),
    )
    assert parsed.steps == ["Open checkout", "Enter an expired OTP code", "Submit"]

    # Malformed/garbage stored JSON degrades to an empty list, never a
    # validation error or a crash.
    garbage = TestCaseRead(
        id=1,
        analysis_id=1,
        test_id="TC-003",
        title="Garbage steps",
        test_type="unit",
        priority="medium",
        description="d",
        expected_result="e",
        steps="not valid json",
    )
    assert garbage.steps == []

def test_analysis_version_and_outdated_populated_fresh_and_flip_on_edit(monkeypatch):
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedAnalysisProvider())
    headers = _auth()
    created = _create_change_request(headers)

    analyze_response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze_response.status_code == 201
    body = analyze_response.json()

    assert len(body["test_cases"]) == 1
    assert len(body["implementation_tasks"]) == 1
    assert body["test_cases"][0]["analysis_version"] == 1
    assert body["test_cases"][0]["is_outdated"] is False
    assert body["implementation_tasks"][0]["analysis_version"] == 1
    assert body["implementation_tasks"][0]["is_outdated"] is False

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"title": "Add OTP authentication for customers at checkout"},
        headers=headers,
    )
    assert edit.status_code == 200

    latest = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert latest.status_code == 200
    latest_body = latest.json()
    assert latest_body["is_outdated"] is True
    assert latest_body["test_cases"][0]["is_outdated"] is True
    assert latest_body["implementation_tasks"][0]["is_outdated"] is True
    # The test case/task itself never silently changes what version it
    # actually belongs to - only whether that version is now stale.
    assert latest_body["test_cases"][0]["analysis_version"] == 1
    assert latest_body["implementation_tasks"][0]["analysis_version"] == 1

```

tests/test_module17_phase2.py

282 lines

```
"""Verifies Module 17 Phase 2 (Test Cases & Implementation Plan 2.0 -
richer test-case generation):

* TestCaseItem (the AI's own output shape) accepts preconditions/steps
and TestCase persists them correctly end-to-end through a real
POST /analyze call - preconditions as free text, steps as an ordered
list (round-tripped through the app's established JSON-text-column
pattern, same as Module 17 Phase 1 already proved for the schema
layer alone).
* The five test types Module 17 Phase 1 added to the TestType enum
(negative/boundary/api/ui/data_validation) are genuine, usable
choices end-to-end - not just enum members nothing ever emits - by
persisting a test case of one of them through the same real
POST /analyze call.
* A test case naming NO preconditions/steps at all (the shape every
test in test analysis engine.py already uses) still persists cleanly
with safe empty defaults - Phase 2 never made these fields required.
* `coerce_steps` (the AI-output-shape robustness helper) turns a
weaker/local model's numbered-list-in-a-single-string habit into a
real ordered list instead of one giant blob, the same "handle
validation errors gracefully" spirit as every other normalizer in
app/schemas/ai_analysis.py.
* The widened test-type menu is actually present in the system prompt
sent to the AI (a silent prompt/enum drift bug elsewhere in this
module would otherwise never be caught by any test).

Every test here uses a fake, monkeypatched AI provider (analysis engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app
from app.schemas.ai_analysis import TestCaseItem, _coerce_steps

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p2-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase2 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _base_analysis_json(test_cases):
    return {
        "summary": "Adds OTP-based verification to checkout.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security findings": [],
        "security analysis": {"concerns": [], "summary": ""},
        "impact assessments": [],
        "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
        "missing information": [],
        "test_cases": test_cases,
        "implementation_plan": [
            {
                "task": "Add OTP generation endpoint",
                "description": "Backend endpoint that generates and stores a one-time password.",
                "component": "Backend",
                "priority": "high",
                "estimated_effort": "2 days",
                "dependencies": None,
            }
        ],
        "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
    }

class FixedProvider:
    def __init__(self, analysis_json):
        self._analysis_json = analysis_json

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(self._analysis_json)

def test_preconditions_and_steps_persist_end_to_end(monkeypatch):
    analysis_json = _base_analysis_json([

```

```

        {
            "id": "TC-001",
            "title": "OTP code expires after 5 minutes",
            "type": "security",
            "priority": "high",
            "description": "Verify an OTP code can't be used once it has expired.",
            "preconditions": "Customer has requested an OTP code and 5 minutes have elapsed.",
            "steps": ["Open checkout", "Enter the expired OTP code", "Submit the form"],
            "expected_result": "The expired code is rejected with a clear error.",
        }
    ]
)
monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
headers = _auth()
created = _create_change_request(headers)

response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
assert response.status_code == 201
test_case = response.json()["test_cases"][0]

assert test_case["preconditions"] == "Customer has requested an OTP code and 5 minutes have elapsed."
assert test_case["steps"] == ["Open checkout", "Enter the expired OTP code", "Submit the form"]

def test_new_test_types_are_usable_end_to_end(monkeypatch):
    analysis_json = _base_analysis_json(
        [
            {
                "id": "TC-001",
                "title": "OTP field rejects a code one character short",
                "type": "boundary",
                "priority": "medium",
                "description": "Submit a 5-digit code when 6 digits are required.",
                "expected_result": "The form shows a validation error and does not submit.",
            },
            {
                "id": "TC-002",
                "title": "OTP endpoint rejects a malformed request body",
                "type": "api",
                "priority": "medium",
                "description": "POST to the OTP verification endpoint with a missing field.",
                "expected_result": "A 422 response is returned; no OTP is marked as used.",
            }
        ]
    )
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    test_cases = {tc["test_id"]: tc for tc in response.json()["test_cases"]}

    assert test_cases["TC-001"]["test_type"] == "boundary"
    assert test_cases["TC-002"]["test_type"] == "api"

def test_test_case_with_no_preconditions_or_steps_still_persists_cleanly(monkeypatch):
    # The exact shape test analysis engine.py's own fixtures already use -
    # Phase 2 must never make preconditions/steps required.
    analysis_json = _base_analysis_json(
        [
            {
                "id": "TC-001",
                "title": "Checkout awards multiplied points",
                "type": "integration",
                "priority": "high",
                "description": "Complete a checkout and verify points awarded match the configured multiplier.",
                "expected_result": "Points awarded equal base points * multiplier.",
            }
        ]
    )
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    test_case = response.json()["test_cases"][0]

    assert test_case["preconditions"] is None
    assert test_case["steps"] == []

def test_coerce_steps_splits_a_numbered_single_string_into_real_steps():
    assert _coerce_steps("1. Open checkout\n2) Enter code\n3. Submit") == [
        "Open checkout",
        "Enter code",
        "Submit",
    ]
    assert _coerce_steps(None) == []
    assert _coerce_steps([]) == []
    assert _coerce_steps(["Already a list", ""]) == ["Already a list"]

def test_test_case_item_normalizes_placeholder_preconditions_to_none():
    item = TestCaseItem(
        id="TC-001",
        title="t",
        type="unit",
        description="d",
        expected_result="e",
        preconditions="e.g. user is logged in",
    )
    assert item.preconditions == "user is logged in"

def test_test_case_item_normalizes_validation_shorthand_to_data_validation():
    # Regression test: a real analysis run failed validation entirely
    # ("1 validation error for AIAnalysisResult test cases.1.type") because
    # the AI shortened DATA_VALIDATION to just "validation" - a plain
    # near-miss, not a genuinely wrong answer, so it should normalize
    # cleanly rather than fail the whole analysis.
    item = TestCaseItem(
        id="TC-001",
        title="t",
        type="validation",
        description="d",
        expected_result="e",
    )
    assert item.type.value == "data_validation"

def test_test_case_item_normalizes_functional_to_manual():
    # Regression test: a second real analysis run failed the same way, this
    # time with the AI writing "functional" - a type this enum deliberately
    # doesn't have (see TestType's own docstring: MANUAL/UNIT/INTEGRATION/
    # E2E already cover it). Same fix family as the "validation" case above.
    item = TestCaseItem(
        id="TC-001",
        title="t",
    )

```

```

        type="functional",
        description="d",
        expected_result="e",
    )
    assert item.type.value == "manual"

def test_test_case_item_strips_trailing_test_suffix():
    # "unit test", "negative test", etc. are a very common way to phrase
    # these that this enum's own values never include.
    item = TestCaseItem(
        id="TC-001",
        title="t",
        type="negative_test",
        description="d",
        expected_result="e",
    )
    assert item.type.value == "negative"

def test_widened_test_type_menu_is_present_in_the_system_prompt():
    # Quoted so a short value like "ui" can't false-positive-match inside
    # an unrelated word (e.g. "guidance") elsewhere in the prompt text.
    for test_type in ("negative", "boundary", "api", "ui", "data_validation"):
        assert test_type in analysis_engine.TEST_TYPES
        assert f'"{test_type}"' in analysis_engine._SYSTEM_PROMPT

```

tests/test_module17_phase3.py

214 lines

```
"""Verifies Module 17 Phase 3 (Test Cases & Implementation Plan 2.0 -
richer implementation-plan generation):

* ImplementationTaskItem (the AI's own output shape) accepts
owner_suggestion and ImplementationTask persists it correctly
end-to-end through a real POST /analyze call, using Module 12's own
AssignmentRole vocabulary (see Module 17 Phase 1's own docstrings for
why this reuses that enum rather than inventing a second one).
* An owner_suggestion value that isn't one of Module 12's real
AssignmentRole options (a placeholder like "N/A", or a value the AI
simply made up) normalizes to null rather than a fabricated role -
the same "never invent, say so instead" rule every other AI-facing
normalizer in app/schemas/ai_analysis.py already follows.
* A task naming no owner_suggestion at all (the exact shape every
fixture in test_analysis_engine.py already uses) still persists
cleanly - Phase 3 never made this field required.
* The full owner-suggestion menu (Module 12's 8 AssignmentRole values)
is actually present in the system prompt sent to the AI - a silent
prompt/enum drift bug elsewhere in this module would otherwise never
be caught by any test.

Every test here uses a fake, monkeypatched AI provider (analysis_engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app
from app.models.enums import AssignmentRole
from app.schemas.ai_analysis import ImplementationTaskItem

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p3-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase3 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _base_analysis_json(implementation_plan):
    return {
        "summary": "Adds OTP-based verification to checkout.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
        "requirements": [],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_findings": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "impact_assessments": [{"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"}],
        "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
        "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-001",
                "title": "OTP code expires after 5 minutes",
                "type": "security",
                "priority": "high",
                "description": "Verify an OTP code can't be used once it has expired.",
                "expected_result": "The expired code is rejected with a clear error.",
            }
        ],
        "implementation_plan": implementation_plan,
        "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
    }

class _FixedProvider:
    def __init__(self, analysis_json):
        self._analysis_json = analysis_json

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(self._analysis_json)

def test_owner_suggestion_persists_end_to_end(monkeypatch):
    analysis_json = _base_analysis_json(
        [
            {
                "task": "Add rate limiting to the OTP verification endpoint",
                "description": "Cap OTP verification attempts to stop brute-forcing.",
                "component": "Backend",
            }
        ]
    )
```

```

        "priority": "high",
        "estimated effort": "1 day",
        "dependencies": None,
        "owner_suggestion": "security_reviewer",
    }
]
)
monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
headers = _auth()
created = _create_change_request(headers)

response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
assert response.status_code == 201
task = response.json()["implementation_tasks"][0]
assert task["owner_suggestion"] == "security_reviewer"

def test_invalid_owner_suggestion_normalizes_to_none_end_to_end(monkeypatch):
    analysis_json = _base_analysis_json(
        [
            {
                "task": "Write release notes",
                "description": "Summarize the change for the changelog.",
                "component": "Docs",
                "priority": "low",
                "estimated effort": "1 hour",
                "dependencies": None,
                "owner_suggestion": "N/A",
            }
        ]
    )
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    task = response.json()["implementation_tasks"][0]
    assert task["owner_suggestion"] is None

def test_task_with_no_owner_suggestion_still_persists_cleanly(monkeypatch):
    # The exact shape test_analysis_engine.py's own fixtures already use -
    # Phase 3 must never make owner suggestion required.
    analysis_json = _base_analysis_json(
        [
            {
                "task": "Add OTP generation endpoint",
                "description": "Backend endpoint that generates and stores a one-time password.",
                "component": "Backend",
                "priority": "high",
                "estimated effort": "2 days",
                "dependencies": None,
            }
        ]
    )
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(analysis_json))
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    task = response.json()["implementation_tasks"][0]
    assert task["owner_suggestion"] is None

def test_implementation_task_item_rejects_a_fabricated_role_in_favor_of_none():
    item = ImplementationTaskItem(
        task="t",
        description="d",
        owner_suggestion="chief wizard",
    )
    assert item.owner_suggestion is None

    item2 = ImplementationTaskItem(
        task="t",
        description="d",
        owner_suggestion="Technical Lead",
    )
    assert item2.owner_suggestion == AssignmentRole.TECHNICAL_LEAD

def test_owner_suggestion_menu_is_present_in_the_system_prompt():
    for role in AssignmentRole:
        assert role.value in analysis_engine.OWNER_SUGGESTIONS
        assert f"{role.value}" in analysis_engine._SYSTEM_PROMPT

```

tests/test_module17_phase4.py

269 lines

```
"""Verifies Module 17 Phase 4 (Test Cases & Implementation Plan 2.0 -
Traceability):

* app/services/traceability_linkage.py's keyword-overlap matching itself
(requirement references/risk references), the same kind of unit-level
coverage repository_linkage's own test file gives its own tokenizer/
overlap functions.
* A real POST /analyze call ends up with requirement_references/
risk references correctly populated on test cases, and
requirement references on implementation tasks - proving the wiring in
app/api/change_requests.py (both the fresh-analysis and
get_latest_analysis code paths) actually calls this module, not just
that the module works in isolation.
* The references are genuinely computed, not AI-generated: a made-up
"REQ-999"/"RISK-999" in the AI's own raw text never appears in a
response's real reference lists, since nothing here ever trusts an
AI-supplied ID (the fixed provider used below doesn't even have the
opportunity to supply one - TestCaseItem/ImplementationTaskItem have
no such field - but the point is proven positively: only real,
persisted Requirement/Risk ids from *this* analysis ever appear).
* An analysis with no requirements and no risks at all leaves every
reference list at its empty-list default rather than erroring.
* Both API code paths (a fresh POST /analyze, and a later GET
/analysis for the same change request) return the identical
reference lists - the computation is deterministic and re-derived
the same way every time, not accidentally order-dependent.

Every test here uses a fake, monkeypatched AI provider (analysis_engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app
from app.schemas.requirement import RequirementRead
from app.schemas.risk import RiskRead
from app.services.traceability_linkage import requirement_references, risk_references

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p4-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase4 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

_ANALYSIS_JSON_WITH_REQUIREMENTS_AND_RISKS = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [
        {
            "category": "functional",
            "description": "Customers must receive a one-time password by SMS to verify their identity during checkout.",
            "priority": "high",
        },
        {
            "category": "non functional",
            "description": "Refund processing must remain unaffected by this change.",
            "priority": "low",
        },
    ],
    "affected components": [],
    "dependencies": [],
    "risks": [
        {
            "category": "security",
            "description": "An attacker could brute-force the one-time password within its validity window.",
            "severity": "high",
            "probability": 0.4,
            "score": 60,
            "mitigation": "Rate-limit OTP verification attempts.",
        }
    ],
    "security findings": [],
    "security analysis": {"concerns": [], "summary": ""},
    "impact assessments": [
        {
            "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
            "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
            "missing information": [],
            "test cases": [
                {
                    "id": "TC-001",
                    "title": "OTP brute-force attempts are rate-limited",
                    "type": "security",
                    "priority": "high",
                    "description": "Attempt to brute-force the one-time password sent by SMS during checkout verification.",
                }
            ]
        }
    ]
}
```

```

        "expected_result": "Further attempts are blocked after repeated failures.",
    },
    {
        "id": "TC-002",
        "title": "Existing refund flow remains unaffected",
        "type": "regression",
        "priority": "medium",
        "description": "Process a refund and confirm nothing about refund processing changed.",
        "expected_result": "The refund transaction proceeds exactly as it did previously.",
    },
],
"implementation_plan": [
    {
        "task": "Rate-limit the OTP verification endpoint",
        "description": "Add a rate limit to the one-time password SMS verification endpoint.",
        "component": "Backend",
        "priority": "high",
        "estimated_effort": "1 day",
        "dependencies": None,
    }
],
"recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

_ANALYSIS_JSON_NO_REQUIREMENTS_OR_RISKS = {
    **_ANALYSIS_JSON_WITH_REQUIREMENTS_AND_RISKS,
    "requirements": [],
    "risks": [],
}

class FixedProvider:
    def __init__(self, analysis_json):
        self._analysis_json = analysis_json

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(self._analysis_json)

def test_requirement_and_risk_references_populate_end_to_end(monkeypatch):
    monkeypatch.setattr(
        analysis_engine, "get_ai_provider", lambda: FixedProvider(_ANALYSIS_JSON_WITH_REQUIREMENTS_AND_RISKS)
    )
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    otp_requirement_id = next(
        r["id"] for r in body["requirements"] if "one-time password by SMS" in r["description"]
    )
    refund_requirement_id = next(r["id"] for r in body["requirements"] if "Refund processing" in r["description"])
    brute_force_risk_id = body["risks"][0]["id"]

    test_cases = {tc["test_id"]: tc for tc in body["test_cases"]}
    assert test_cases["TC-001"]["requirement_references"] == [f"REQ-{otp_requirement_id}"]
    assert test_cases["TC-001"]["risk_references"] == [f"RISK-{brute_force_risk_id}"]
    assert test_cases["TC-002"]["requirement_references"] == [f"REQ-{refund_requirement_id}"]
    assert test_cases["TC-002"]["risk_references"] == []

    task = body["implementation_tasks"][0]
    assert task["requirement_references"] == [f"REQ-{otp_requirement_id}"]

def test_no_requirements_or_risks_leaves_references_empty(monkeypatch):
    monkeypatch.setattr(
        analysis_engine, "get_ai_provider", lambda: FixedProvider(_ANALYSIS_JSON_NO_REQUIREMENTS_OR_RISKS)
    )
    headers = _auth()
    created = _create_change_request(headers)

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    body = response.json()

    for test_case in body["test_cases"]:
        assert test_case["requirement_references"] == []
        assert test_case["risk_references"] == []
    for task in body["implementation_tasks"]:
        assert task["requirement_references"] == []

def test_get_latest_analysis_returns_identical_references(monkeypatch):
    monkeypatch.setattr(
        analysis_engine, "get_ai_provider", lambda: FixedProvider(_ANALYSIS_JSON_WITH_REQUIREMENTS_AND_RISKS)
    )
    headers = _auth()
    created = _create_change_request(headers)

    analyze_response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze_response.status_code == 201
    fresh_refs = [
        (tc["test_id"], tc["requirement_references"], tc["risk_references"])
        for tc in analyze_response.json()["test_cases"]
    ]

    latest_response = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert latest_response.status_code == 200
    latest_refs = [
        (tc["test_id"], tc["requirement_references"], tc["risk_references"])
        for tc in latest_response.json()["test_cases"]
    ]

    assert fresh_refs == latest_refs

def test_traceability_linkage_functions_never_cite_a_fabricated_id():
    requirements = [
        RequirementRead(
            id=42,
            analysis_id=1,
            requirement_type="functional",
            description="Customers must verify their identity with a one-time password.",
            priority="high",
        )
    ]
    risks = [
        RiskRead(
            id=7,
            analysis_id=1,
            category="security",
            description="One-time password brute-forcing risk.",
            severity="high",
            probability=0.4,
            score=60,
            mitigation="Rate-limit attempts.",
        )
    ]

```



```
] )

matched_reqs = requirement_references("Verify a one-time password during checkout", requirements)
assert matched_reqs == ["REQ-42"]

matched_risks = risk_references("Brute-forcing a one-time password should be blocked", risks)
assert matched_risks == ["RISK-7"]

# Nothing shares enough real overlap with an unrelated topic - an
# honest empty list, never a guess.
assert requirement_references("Completely unrelated billing invoice export feature", requirements) == []
assert risk_references("Completely unrelated billing invoice export feature", risks) == []
```

tests/test_module17_phase6.py

284 lines

```
"""Verifies Module 17 Phase 6 (Test Cases & Implementation Plan 2.0 -
Human Editing):

* PATCH ../test-cases/{id} and PATCH ../implementation-tasks/{id} let
  an authorized human correct one AI-generated row in place - only the
  fields actually sent are changed, edited_by/edited_at/edit_reason are
  set, and a TEST_CASE_EDITED/IMPLEMENTATION_TASK_EDITED history event is
  recorded (same permission gate - workflow rules.can_review_analysis_
  findings - Module 14 Phase 6 already established for Requirements/
  Security findings; same 403 for an unrelated user).
* The AI's own original wording is never lost: it's preserved as the
  old value on that edit's own history row, never overwritten or
  duplicated onto a second column on the row itself.
* A no-op submission (nothing actually different from what's already
  stored) never records a history event and never sets edited_by/
  edited_at - editing is a real, audited action, not a no-op timestamp
  bump.
* The response after an edit still carries every computed field
  (analysis version, is outdated, requirement references/
  risk references, related files) fully populated - not the schema's
  bare defaults an edit-only code path could otherwise leave them at
  (see app/api/change_requests.py::full_analysis read).
* steps round-trips correctly through an edit (sent as a real list,
  read back as the same real list).
* owner suggestion can be explicitly cleared back to null by a human,
  unlike the AI which can only ever suggest one or leave it null.
* A test case/task id that doesn't belong to the given change request
  404s rather than ever operating on the wrong CR's row.

Every test here uses a fake, monkeypatched AI provider (analysis_engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p6-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase6 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

_ANALYSIS_JSON = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [
        {
            "category": "functional",
            "description": "Customers must receive a one-time password by SMS to verify their identity during checkout.",
            "priority": "high",
        }
    ],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_findings": [],
    "security_analysis": {"concerns": [], "summary": ""},
    "impact_assessments": [],
    "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
    "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
    "missing_information": [],
    "test_cases": [
        {
            "id": "TC-001",
            "title": "OTP code expires after 5 minutes",
            "type": "security",
            "priority": "high",
            "description": "Verify one-time password sent by SMS during checkout expires correctly.",
            "preconditions": "A one-time password has been sent by SMS during checkout.",
            "steps": ["Wait 5 minutes", "Submit the one-time password"],
            "expected_result": "The expired code is rejected with a clear error.",
        }
    ],
    "implementation_plan": [
        {
            "task": "Add OTP generation endpoint",
            "description": "Backend endpoint that generates and stores a one-time password.",
            "component": "Backend",
            "priority": "high",
            "estimated_effort": "2 days",
            "dependencies": "None",
            "owner_suggestion": "technical_lead",
        }
    ]
}
```

```

    ],
    "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

class FixedProvider:
    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(_ANALYSIS_JSON)

def analyze(monkeypatch, headers) -> dict:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider())
    created = _create_change_request(headers)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201
    return created, response.json()

def test_update_test_case_edits_fields_and_records_history(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    test_case = body["test_cases"][0]

    response = client.patch(
        f"/api/change-requests/{created['id']}/test-cases/{test_case['id']}",
        json={
            "title": "OTP code is rejected once expired",
            "steps": ["Wait 5 minutes", "Submit the same one-time password again"],
            "reason": "Clarified the exact steps.",
        },
        headers=headers,
    )
    assert response.status_code == 200
    updated = response.json()
    assert updated["title"] == "OTP code is rejected once expired"
    assert updated["steps"] == ["Wait 5 minutes", "Submit the same one-time password again"]
    # Untouched fields keep their original AI-written values.
    assert updated["description"] == test_case["description"]
    assert updated["edited_by"] is not None
    assert updated["edited_at"] is not None
    assert updated["edit_reason"] == "Clarified the exact steps."

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers)
    assert history.status_code == 200
    edited_events = [h for h in history.json() if h["action"] == "test_case_edited"]
    assert len(edited_events) == 1
    # The AI's original title is preserved as this event's old_value -
    # never lost, never silently overwritten.
    assert test_case["title"] in edited_events[0]["old_value"]
    assert "OTP code is rejected once expired" in edited_events[0]["new_value"]

def test_update_test_case_response_includes_computed_fields(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    test_case = body["test_cases"][0]
    requirement_id = body["requirements"][0]["id"]

    response = client.patch(
        f"/api/change-requests/{created['id']}/test-cases/{test_case['id']}",
        json={"priority": "critical"},
        headers=headers,
    )
    assert response.status_code == 200
    updated = response.json()
    assert updated["priority"] == "critical"
    # Still fully computed, not left at bare schema defaults - same
    # requirement this test case traced to before the edit.
    assert updated["analysis_version"] == 1
    assert updated["is_outdated"] is False
    assert updated["requirement_references"] == [f"REQ-{requirement_id}"]

def test_update_test_case_no_op_edit_does_not_record_history_or_set_edited_by(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    test_case = body["test_cases"][0]

    response = client.patch(
        f"/api/change-requests/{created['id']}/test-cases/{test_case['id']}",
        json={"title": test_case["title"]},
        headers=headers,
    )
    assert response.status_code == 200
    updated = response.json()
    assert updated["edited_by"] is None
    assert updated["edited_at"] is None

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers)
    edited_events = [h for h in history.json() if h["action"] == "test_case_edited"]
    assert len(edited_events) == 0

def test_update_test_case_forbidden_for_unrelated_user(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    test_case = body["test_cases"][0]

    outsider_headers = _auth()
    response = client.patch(
        f"/api/change-requests/{created['id']}/test-cases/{test_case['id']}",
        json={"title": "Hijacked title"},
        headers=outsider_headers,
    )
    assert response.status_code == 403

def test_update_test_case_404_for_wrong_change_request(monkeypatch):
    headers = _auth()
    created_a, body_a = _analyze(monkeypatch, headers)
    created_b, body_b = _analyze(monkeypatch, headers)
    test_case_a = body_a["test_cases"][0]

    response = client.patch(
        f"/api/change-requests/{created_b['id']}/test-cases/{test_case_a['id']}",
        json={"title": "Should not apply"},
        headers=headers,
    )
    assert response.status_code == 404

def test_update_implementation_task_edits_fields_and_can_clear_owner_suggestion(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    task = body["implementation_tasks"][0]
    assert task["owner_suggestion"] == "technical_lead"

    response = client.patch(

```

```

        f"/api/change-requests/{created['id']}/implementation-tasks/{task['id']}",
        json={"estimated_effort": "3 days", "owner_suggestion": None, "reason": "Re-estimated after review."},
        headers=headers,
    )
    assert response.status_code == 200
    updated = response.json()
    assert updated["estimated_effort"] == "3 days"
    assert updated["owner_suggestion"] is None
    assert updated["edited_by"] is not None
    assert updated["edit_reason"] == "Re-estimated after review."

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers)
    edited_events = [h for h in history.json() if h["action"] == "implementation_task_edited"]
    assert len(edited_events) == 1

def test_update_implementation_task_forbidden_for_unrelated_user(monkeypatch):
    headers = _auth()
    created, body = _analyze(monkeypatch, headers)
    task = body["implementation_tasks"][0]

    outsider_headers = _auth()
    response = client.patch(
        f"/api/change-requests/{created['id']}/implementation-tasks/{task['id']}",
        json={"task": "Hijacked task"},
        headers=outsider_headers,
    )
    assert response.status_code == 403

```

tests/test_module17_phase7_sync.py

300 lines

```
"""Module 17 Phase 7 - dedicated end-to-end version-sync pass for Test
Cases & Implementation Plan 2.0 (the module's final phase, per your own
7-section spec: "a dedicated sync test verifying CR Version -> Analysis
Version -> Requirements -> Test Cases -> Implementation Plan all remain
synchronized"). This is not incidental coverage from earlier phases - it
walks the FULL lifecycle in one continuous story and checks every layer
agrees with every other layer at each step, mirroring
test_module15_phase7_security.py / test_module16_phase7_security.py's own
"a dedicated audit, not just leftover assertions" approach for this
module's own closing phase:

* Right after the first /analyze: CR v1, Analysis v1, every test case/
task reports analysis version == 1 and is outdated == False, and their
requirement_references correctly point at THIS analysis's own
requirement rows.
* Editing the change request (PUT) bumps only the CR's own version - the
analysis row itself is untouched - but GET .../analysis immediately
starts reporting is outdated == True, and that staleness propagates to
every test case and task's own is outdated exactly the same way it
already does for knowledge evidence (Module 16) and repository
findings (Module 15). analysis_version does NOT change just because
the CR did - it still correctly names the analysis's own version, not
the CR's current one.
* A human can still edit a test case/task while its parent analysis is
outdated (editing is never blocked by staleness), and the edit's own
audit-trail row is stamped with the CR's version AT THE TIME OF THE
EDIT (2, since the CR was edited first) - proving the audit trail's
version number and the analysis's own change_request version are two
independent, correctly-tracked numbers, never conflated.
* Re-analyzing (POST /analyze again) creates a brand-new Analysis tied
to the CR's now-current version (2) with its own fresh requirements -
every test case/task on THIS new analysis reports analysis_version
== 2, is outdated == False again, and requirement_references naming
only this new analysis's own requirement id(s) - never a stale
reference back to the previous analysis's now-superseded requirement
row, even though both analyses happen to describe similar content.
* The previous (now-superseded) analysis's own edited test case is
never resurrected onto the new analysis - GET .../analysis (latest)
never returns a test case whose edited-by was set on the old
analysis; the edit lives only in that old analysis's own child rows
and in the permanent history trail, exactly the "never presented as
current once the CR has moved to a newer version" rule from the
Module 12 architecture lock.
* The full history trail, read once at the end, contains both the
earlier CR field edit and the test-case edit, each correctly stamped
with its own version number, in the order they actually happened -
a real end-to-end audit story, not just isolated per-action checks.

Every test here uses a fake, monkeypatched AI provider (analysis engine.
get ai provider) - no real network call, matching this app's established
testing pattern. The provider returns a slightly different requirement
wording on its second call, so the two analyses genuinely have distinct
requirement rows (never the same row reused across versions).

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m17p7-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module17 Phase7 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Customer Portal",
    }
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _analysis_json(requirement_text: str, test_case_title: str, task_title: str) -> dict:
    """One shared shape, but with the requirement/test-case/task wording
    parameterized - the second analyze() call passes genuinely different
    text so the two analyses' own child rows are never accidentally
    identical, which would make "does analysis 2 reference analysis 2's
    own requirement, not analysis 1's" impossible to tell apart."""
    return {
        "summary": "Adds OTP-based verification to checkout.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
        "requirements": [
            {
                "category": "functional",
                "description": requirement_text,
                "priority": "high",
            }
        ],
        "affected_components": [],
        "dependencies": [],
        "risks": [],
        "security_findings": [],
        "security_analysis": {"concerns": [], "summary": ""},
    }
```

```

"impact_assessments": [],
"complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
"effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
"missing_information": [],
"test_cases": [
    {
        "id": "TC-001",
        "title": test_case_title,
        "type": "security",
        "priority": "high",
        "description": f"Verify {requirement_text.lower()}",
        "preconditions": "A one-time password has been sent by SMS during checkout.",
        "steps": ["Wait 5 minutes", "Submit the one-time password"],
        "expected_result": "The expired code is rejected with a clear error.",
    }
],
"implementation_plan": [
    {
        "task": task_title,
        "description": f"Implements: {requirement_text}",
        "component": "Backend",
        "priority": "high",
        "estimated_effort": "2 days",
        "dependencies": None,
        "owner_suggestion": "technical_lead",
    }
],
"recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}

class ScriptedProvider:
    """Returns RESPONSES[call_index] in order, one per call to
    complete() - lets a single test drive two distinct /analyze runs
    without a second monkeypatch."""

    def __init__(self, responses):
        self._responses = list(responses)
        self._call_count = 0

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        response = self._responses[min(self._call_count, len(self._responses) - 1)]
        self._call_count += 1
        return json.dumps(response)

def test_full_lifecycle_stays_synchronized_across_edit_and_reanalysis(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)

    provider = _ScriptedProvider(
        [
            _analysis_json(
                "Customers must receive a one-time password by SMS to verify their identity during checkout.",
                "OTP code expires after 5 minutes",
                "Add OTP generation endpoint",
            ),
            _analysis_json(
                "Customers must be able to request a fresh one-time password if the first one expired.",
                "OTP code can be resent after expiry",
                "Add OTP resend endpoint",
            ),
        ]
    )
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

    # --- Step 1: first analysis. CR v1, Analysis v1, everything fresh. ---
    first = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert first.status_code == 201
    first_body = first.json()
    first_requirement_id = first_body["requirements"][0]["id"]
    first_test_case = first_body["test_cases"][0]
    first_task = first_body["implementation_tasks"][0]

    assert first_body["is_outdated"] is False
    assert first_test_case["analysis_version"] == 1
    assert first_test_case["is_outdated"] is False
    assert first_test_case["requirement_references"] == [f"REQ-{first_requirement_id}"]
    assert first_task["analysis_version"] == 1
    assert first_task["is_outdated"] is False
    assert first_task["requirement_references"] == [f"REQ-{first_requirement_id}"]

    # --- Step 2: edit the CR. Only the CR's own version moves. ---
    edit_response = client.put(
        f"/api/change-requests/{created['id']}",
        json={"priority": "high"},
        headers=headers,
    )
    assert edit_response.status_code == 200
    assert edit_response.json()["new_version"] == 2

    # The analysis itself didn't change, but is now stale relative to the
    # CR's new version - and that staleness must be visible on every test
    # case/task, not just on the analysis header.
    stale = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
    assert stale.status_code == 200
    stale_body = stale.json()
    assert stale_body["is_outdated"] is True
    stale_test_case = stale_body["test_cases"][0]
    stale_task = stale_body["implementation_tasks"][0]
    assert stale_test_case["is_outdated"] is True
    assert stale_task["is_outdated"] is True
    # analysis_version still correctly names the ANALYSIS's own version -
    # it must never drift just because the CR moved on.
    assert stale_test_case["analysis_version"] == 1
    assert stale_task["analysis_version"] == 1

    # --- Step 3: a human can still edit a test case while it's outdated. ---
    edit_test_case = client.patch(
        f"/api/change-requests/{created['id']}/test-cases/{first_test_case['id']}",
        json={"title": "OTP code is definitively rejected once expired", "reason": "Clarified wording."},
        headers=headers,
    )
    assert edit_test_case.status_code == 200
    edited_body = edit_test_case.json()
    assert edited_body["edited_by"] is not None
    # Editing doesn't secretly "fix" staleness - it's still the same old
    # (now-superseded) analysis underneath.
    assert edited_body["is_outdated"] is True

    # --- Step 4: re-analyze. Brand-new Analysis tied to CR v2. ---
    second = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert second.status_code == 201
    second_body = second.json()
    second_requirement_id = second_body["requirements"][0]["id"]
    second_test_case = second_body["test_cases"][0]
    second_task = second_body["implementation_tasks"][0]

```

```

assert second_requirement_id != first_requirement_id
assert second_body["is outdated"] is False
assert second_test_case["analysis version"] == 2
assert second_test_case["is outdated"] is False
# References only this new analysis's own requirement - never the old,
# now-superseded one, even though both requirements are about OTP.
assert second_test_case["requirement_references"] == [f"REQ-{second_requirement_id}"]
assert second_task["requirement_references"] == [f"REQ-{second_requirement_id}"]

# --- Step 5: the old analysis's edit is never resurrected as current. ---
latest = client.get(f"/api/change-requests/{created['id']}/analysis", headers=headers)
assert latest.status_code == 200
latest_test_case_ids = {tc["id"] for tc in latest.json()["test_cases"]}
assert first_test_case["id"] not in latest_test_case_ids
assert second_test_case["id"] in latest_test_case_ids
# The new analysis's own test case was never touched by a human -
# it's a fresh AI-written row, not the edited one from before.
assert latest.json()["test_cases"][0]["edited_by"] is None

# --- Step 6: the full history trail agrees with both events, each
# correctly stamped with the CR version in effect at that moment. ---
history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers)
assert history.status_code == 200
events = history.json()

priority_edits = [h for h in events if h["field_name"] == "priority"]
assert len(priority_edits) == 1
assert priority_edits[0]["version_number"] == 2

test_case_edits = [h for h in events if h["action"] == "test_case_edited"]
assert len(test_case_edits) == 1
# The edit happened after the CR's own version-2 bump, so the audit
# row correctly reflects version 2 - not version 1 (when the analysis
# it belongs to was created) and not some future version.
assert test_case_edits[0]["version_number"] == 2
assert "OTP code expires after 5 minutes" in test_case_edits[0]["old_value"]
assert "OTP code is definitively rejected once expired" in test_case_edits[0]["new_value"]

# The field edit happened before the test-case edit in real time -
# confirm the trail's own ordering (newest first, matching every other
# history endpoint in this app) agrees with that.
priority_index = events.index(priority_edits[0])
test_case_index = events.index(test_case_edits[0])
assert test_case_index < priority_index

```

tests/test_module18_phase1.py

216 lines

```
"""Verifies Module 18 Phase 1 (Notifications, My Work & Personal
Engineering Queue - Data Foundation):

* POST ../approvals accepts an optional due_date, persisted on the
Approval row and returned on ApprovalRead - entirely optional, never
defaulted or required (existing approval-request behavior with no
due_date is completely unchanged).
* due_status (app/services/approvals.py::approval_due_status) is
computed fresh at read time, never stored: None for a due_date far in
the future, "due_soon" inside the 2-day window, "overdue" once past
due_date - and always None again the moment the approval is actually
responded to (Approved/Rejected/Changes Requested), no matter how far
in the past its due_date now sits.
* The lazy deadline check (app/services/deadlines.py) fires a
DEADLINE_APPROACHING notification, tied to that exact approval via
Notification.approval_id, the first time its own approver reads their
notification inbox (GET /api/notifications or /unread-count) - and
never fires a second one for the same approval on a later read.
* Scoping: checking your own notifications never creates or surfaces a
deadline notification for someone ELSE's pending approval, even one
that's badly overdue - the check only ever looks at the reading
user's own approvals.

Every test here uses a fake, monkeypatched AI provider where analysis is
needed - no real network call, matching this app's established testing
pattern. No test data ever uses grubbrr.com - alight.com throughout, per
this project's own standing rule.

Run with (from backend/): python -m pytest ../tests
"""
import sys
import uuid
from datetime import datetime, timedelta
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m18p1-{uuid.uuid4().hex[:10]}@example.com"

def _auth(name: str = "Module18 Phase1 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _request_approval(cr_id: int, headers: dict, approver_id: int, due_date: str = None, approval_type: str = "security"):
    body = {"approval_type": approval_type, "approver_user_id": approver_id}
    if due_date is not None:
        body["due_date"] = due_date
    response = client.post(f"/api/change-requests/{cr_id}/approvals", json=body, headers=headers)
    assert response.status_code == 201
    return response.json()

def _iso(dt: datetime) -> str:
    return dt.isoformat()

def test_due_date_is_optional_and_round_trips(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver With No Due Date")
    created = _create_change_request(owner_headers)

    approval = _request_approval(created["id"], owner_headers, approver_id)
    assert approval["due_date"] is None
    assert approval["due_status"] is None

def test_due_date_far_in_future_is_not_due_soon(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver Far Future")
    created = _create_change_request(owner_headers)

    far_future = _iso(datetime.utcnow() + timedelta(days=30))
    approval = _request_approval(created["id"], owner_headers, approver_id, due_date=far_future)
    assert approval["due_date"] is not None
    assert approval["due_status"] is None

def test_due_date_inside_window_is_due_soon_and_past_is_overdue(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver Due Soon And Overdue")
    created = _create_change_request(owner_headers)

    due_soon_date = _iso(datetime.utcnow() + timedelta(days=1))
    overdue_date = _iso(datetime.utcnow() - timedelta(days=1))

    due_soon_approval = _request_approval(
        created["id"], owner_headers, approver_id, due_date=due_soon_date, approval_type="security"
    )
    overdue_approval = _request_approval(
        created["id"], owner_headers, approver_id, due_date=overdue_date, approval_type="technical"
```



```

)

listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=owner_headers)
assert listing.status_code == 200
by_id = {a["id"]: a for a in listing.json()}
assert by_id[due_soon_approval["id"]]["due_status"] == "due soon"
assert by_id[overdue_approval["id"]]["due_status"] == "overdue"

def test_due_status_clears_once_approval_is_responded_to(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver Who Responds")
    created = _create_change_request(owner_headers)

    overdue_date = _iso(datetime.utcnow() - timedelta(days=3))
    approval = _request_approval(created["id"], owner_headers, approver_id, due_date=overdue_date)

    respond = client.post(
        f"/api/change-requests/{created['id']}/approvals/{approval['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )
    assert respond.status_code == 200
    responded = respond.json()
    # Still badly overdue by the clock, but a decision has already been
    # made - due status is never meaningful for a resolved approval.
    assert responded["due_status"] is None
    assert responded["due_date"] is not None

def test_deadline_approaching_notification_fires_once_via_notification_list(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver Getting Notified")
    created = _create_change_request(owner_headers)

    overdue_date = _iso(datetime.utcnow() - timedelta(hours=6))
    approval = _request_approval(created["id"], owner_headers, approver_id, due_date=overdue_date)

    first_read = client.get("/api/notifications", headers=approver_headers)
    assert first_read.status_code == 200
    deadline_notifications = [n for n in first_read.json() if n["type"] == "deadline_approaching"]
    assert len(deadline_notifications) == 1
    assert deadline_notifications[0]["approval_id"] == approval["id"]
    assert deadline_notifications[0]["change_request_id"] == created["id"]

    # Reading again must never create a second one for the same approval.
    second_read = client.get("/api/notifications", headers=approver_headers)
    assert second_read.status_code == 200
    deadline_notifications_again = [n for n in second_read.json() if n["type"] == "deadline_approaching"]
    assert len(deadline_notifications_again) == 1
    assert deadline_notifications_again[0]["id"] == deadline_notifications[0]["id"]

def test_deadline_approaching_notification_fires_via_unread_count_too(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver Checking Count")
    created = _create_change_request(owner_headers)

    due_soon_date = _iso(datetime.utcnow() + timedelta(hours=6))
    _request_approval(created["id"], owner_headers, approver_id, due_date=due_soon_date)

    count_response = client.get("/api/notifications/unread-count", headers=approver_headers)
    assert count_response.status_code == 200
    assert count_response.json()["count"] >= 1

    listing = client.get("/api/notifications", headers=approver_headers)
    deadline_notifications = [n for n in listing.json() if n["type"] == "deadline_approaching"]
    assert len(deadline_notifications) == 1

def test_deadline_notification_never_leaks_to_a_different_user(monkeypatch):
    owner_headers, owner_id = _auth()
    approver_headers, approver_id = _auth("Approver With Overdue Item")
    bystander_headers, bystander_id = _auth("Unrelated Bystander")
    created = _create_change_request(owner_headers)

    overdue_date = _iso(datetime.utcnow() - timedelta(days=2))
    _request_approval(created["id"], owner_headers, approver_id, due_date=overdue_date)

    # An unrelated user (not the tagged approver) reading their own
    # notifications must never see - or trigger - a deadline notification
    # for someone else's pending approval.
    bystander_read = client.get("/api/notifications", headers=bystander_headers)
    assert bystander_read.status_code == 200
    assert all(n["type"] != "deadline_approaching" for n in bystander_read.json())

    # The CR owner (who requested the approval, but isn't the approver)
    # must not get it either.
    owner_read = client.get("/api/notifications", headers=owner_headers)
    assert owner_read.status_code == 200
    assert all(n["type"] != "deadline_approaching" for n in owner_read.json())

```

tests/test_module18_phase2.py

248 lines

```
"""Verifies Module 18 Phase 2 (Notifications, My Work & Personal
Engineering Queue - Risk Escalation + Analysis Completion notifications):

* ANALYSIS COMPLETED fires for the CR's creator/assignees (never the
person who triggered it) the very first time a change request is
analyzed - and never again on a later re-analysis of the same CR,
where ANALYSIS SIGNIFICANTLY CHANGED/RISK ESCALATED take over instead
(see test ai_analysis_phase3.py for the former, already established).
* RISK ESCALATED fires when a re-analysis lands the change request in a
strictly HIGHER risk bucket than it was in before (e.g. low -> high) -
reusing app/services/analysis_delta.py::compare_analyses's own
risk bucket before/risk bucket after, never a second risk computation.
* RISK ESCALATED never fires when risk stays the same bucket, or actually
improves (moves to a LOWER bucket) - only a genuine escalation counts.
* RISK ESCALATED can fire alongside ANALYSIS SIGNIFICANTLY CHANGED for
the same re-analysis (a risk-bucket move is itself one of that
notification's own significant-change reasons) - both are expected,
not a duplicate of one event.
* Both new notification types are scoped exactly like the existing
ANALYSIS SIGNIFICANTLY CHANGED notification - the CR's creator and
assignees, never the user who actually ran the analysis.

Every test here uses a fake, monkeypatched AI provider (analysis engine.
get_ai_provider) - no real network call, matching this app's established
testing pattern. No test data ever uses grubrr.com - alight.com
throughout, per this project's own standing rule.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m18p2-{uuid.uuid4().hex[:10]}@example.com"

def _auth(name: str = "Module18 Phase2 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict) -> dict:
    payload = {
        "title": "Add OTP authentication for customers",
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Customer Portal",
    }
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _analysis_json(risk_score: float) -> dict:
    """One shared shape, with only the one Risk's own `score` varying -
that's the single number app/services/analysis engine.py::
_overall_risk_score() actually uses to set Analysis.risk score (the
worst individual risk, not an average - see that function's own
docstring), so this is the one thing that needs to move between calls
to move the resulting risk bucket."""
    return {
        "summary": "Adds OTP-based verification to checkout.",
        "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
        "requirements": [
            {
                "category": "functional",
                "description": "Customers must receive a one-time password by SMS to verify their identity during checkout.",
                "priority": "high",
            }
        ],
        "affected_components": [],
        "dependencies": [],
        "risks": [
            {
                "category": "security",
                "description": "OTP codes could be intercepted or brute-forced.",
                "severity": "high",
                "probability": 0.3,
                "score": risk_score,
                "explanation": "Rate limiting mitigates most of this.",
                "mitigation": "Rate-limit OTP attempts per phone number.",
            }
        ],
        "security_findings": [],
        "security_analysis": {"concerns": [], "summary": ""},
        "impact_assessments": [
            {
                "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
                "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
                "missing_information": [],
                "test_cases": [],
                "implementation_plan": [],
                "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
            }
        ]
    }

class ScriptedProvider:
    def __init__(self, responses):
        self._responses = list(responses)
        self._call_count = 0
```

```

def is_configured(self) -> bool:
    return True

def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
    response = self._responses[min(self._call_count, len(self._responses) - 1)]
    self._call_count += 1
    return json.dumps(response)

def _notifications_for(headers: dict, notification_type: str) -> list[dict]:
    response = client.get("/api/notifications", headers=headers)
    assert response.status_code == 200
    return [n for n in response.json() if n["type"] == notification_type]

def test_analysis_completed_fires_for_creator_on_first_analysis_only(monkeypatch):
    creator_headers, creator_id = _auth("CR Creator")
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _ScriptedProvider([_analysis_json(10.0), _analysis_json(10.0)]))
    created = _create_change_request(creator_headers)

    first = client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    assert first.status_code == 201

    completed = _notifications_for(creator_headers, "analysis_completed")
    # The creator is the one who ran it - never notified about their own action.
    assert len(completed) == 0

def test_analysis_completed_notifies_assignee_but_not_the_analyzer(monkeypatch):
    creator_headers, creator_id = _auth("CR Creator Two")
    assignee_headers, assignee_id = _auth("Assigned Engineer")
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _ScriptedProvider([_analysis_json(10.0)]))
    created = _create_change_request(creator_headers)

    assign = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )
    assert assign.status_code == 201

    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    assert analyze.status_code == 201

    assignee_completed = _notifications_for(assignee_headers, "analysis_completed")
    assert len(assignee_completed) == 1
    assert assignee_completed[0]["change_request_id"] == created["id"]

    analyzer_completed = _notifications_for(creator_headers, "analysis_completed")
    assert len(analyzer_completed) == 0

def test_risk_escalated_fires_when_bucket_moves_from_low_to_critical(monkeypatch):
    creator_headers, creator_id = _auth("Escalation Creator")
    assignee_headers, assignee_id = _auth("Escalation Assignee")
    provider = _ScriptedProvider([_analysis_json(10.0), _analysis_json(90.0)])
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)
    created = _create_change_request(creator_headers)

    assign = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "technical_lead"},
        headers=creator_headers,
    )
    assert assign.status_code == 201

    first = client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    assert first.status_code == 201
    assert first.json()["risk_score"] == 10.0

    second = client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    assert second.status_code == 201
    assert second.json()["risk_score"] == 90.0

    escalated = _notifications_for(assignee_headers, "risk_escalated")
    assert len(escalated) == 1
    assert "critical" in escalated[0]["title"].lower()
    assert escalated[0]["change_request_id"] == created["id"]

    # The same re-analysis is ALSO a materially significant change (risk
    # bucket is one of compare_analyses' own significant-change reasons) -
    # both notifications are expected together, not a contradiction.
    significantly_changed = _notifications_for(assignee_headers, "analysis_significantly_changed")
    assert len(significantly_changed) == 1

    # Never notifies the person who actually ran the re-analysis.
    assert len(_notifications_for(creator_headers, "risk_escalated")) == 0

def test_risk_escalated_never_fires_when_risk_bucket_is_unchanged(monkeypatch):
    creator_headers, creator_id = _auth("Stable Risk Creator")
    assignee_headers, assignee_id = _auth("Stable Risk Assignee")
    provider = _ScriptedProvider([_analysis_json(10.0), _analysis_json(12.0)])
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)
    created = _create_change_request(creator_headers)

    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)

    assert len(_notifications_for(assignee_headers, "risk_escalated")) == 0

def test_risk_escalated_never_fires_when_risk_actually_improves(monkeypatch):
    creator_headers, creator_id = _auth("Improving Risk Creator")
    assignee_headers, assignee_id = _auth("Improving Risk Assignee")
    provider = _ScriptedProvider([_analysis_json(90.0), _analysis_json(10.0)])
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)
    created = _create_change_request(creator_headers)

    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)

    assert len(_notifications_for(assignee_headers, "risk_escalated")) == 0

```

tests/test_module18_phase3.py

289 lines

```
"""Verifies Module 18 Phase 3 (Notifications, My Work & Personal
Engineering Queue - "My Work" backend):

* GET /api/my-work/change-requests - only change requests the CURRENT
  user personally created, never anyone else's.
* GET /api/my-work/approvals - only approvals tagged to the current
  user; defaults to PENDING only (the real queue), ?status=all returns
  every status including already-decided ones.
* GET /api/my-work/reviews - only change requests where the current user
  holds one of Reviewer/Technical Lead/Security Reviewer/Owner - a
  QA Owner or Implementation Owner assignment (real roles that exist,
  just not "review" roles) must NOT show up here...
* GET /api/my-work/assignments - ...but MUST show up here, since this
  endpoint is "any assignment role at all", broader than /reviews.
* GET /api/my-work/changes-requested - only change requests the current
  user OWNS where an approver responded Changes Requested - confirmed
  interpretation: this is "someone is waiting on ME", not "I asked
  someone else to make changes" (the approver who requested changes
  must never see it on their OWN changes-requested list).
* GET /api/my-work/mentions - reuses the existing MENTIONED notification
  row, never a second mentions table.
* GET /api/my-work/overdue - only PENDING approvals whose due date has
  already passed - a due-soon-but-not-yet-overdue approval must NOT
  appear here.
* GET /api/my-work/summary - counts agree with what each section
  endpoint actually returns.
* Every one of the above is scoped to `current_user` - a second,
  unrelated user's own My Work never shows the first user's items.

Every test here uses a fake, monkeypatched AI provider where analysis is
needed - no real network call, matching this app's established testing
pattern. No test data ever uses grubrr.com - alight.com throughout.

Run with (from backend/): python -m pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m18p3-{uuid.uuid4().hex[:10]}@example.com"

def _auth(name: str = "Module18 Phase3 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, title: str = "Add OTP authentication for customers") -> dict:
    payload = {
        "title": title,
        "description": "Let Alight.com customers verify identity with a one-time password before checkout.",
        "priority": "medium",
        "requested by": "Shivani",
        "target_system": "Customer Portal",
    }
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _assign(cr_id: int, headers: dict, user_id: int, role: str) -> dict:
    response = client.post(
        f"/api/change-requests/{cr_id}/assignments", json={"user_id": user_id, "role": role}, headers=headers
    )
    assert response.status_code == 201
    return response.json()

def _request_approval(cr_id: int, headers: dict, approver_id: int, due date: str = None, approval_type: str = "security"):
    body = {"approval_type": approval_type, "approver_user_id": approver_id}
    if due date is not None:
        body["due date"] = due date
    response = client.post(f"/api/change-requests/{cr_id}/approvals", json=body, headers=headers)
    assert response.status_code == 201
    return response.json()

def _respond(cr_id: int, approval_id: int, headers: dict, status_value: str, comment: str = None):
    body = {"status": status_value}
    if comment is not None:
        body["comment"] = comment
    response = client.post(
        f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond", json=body, headers=headers
    )
    assert response.status_code == 200
    return response.json()

def test_my_change_requests_only_shows_my_own(monkeypatch):
    mine_headers, _mine_id = _auth("Owner Of My CRs")
    other_headers, _other_id = _auth("Someone Else Entirely")
    mine = _create_change_request(mine_headers, title="Add OTP authentication for customers")
    _create_change_request(other_headers, title="Add SSO login for Alight.com admins")

    response = client.get("/api/my-work/change-requests", headers=mine_headers)
    assert response.status_code == 200
    ids = {item["id"] for item in response.json()}
    assert mine["id"] in ids
    assert len(ids) == 1

def test_my_approvals_defaults_to_pending_and_all_shows_everything(monkeypatch):
    owner_headers, _owner_id = _auth("Approvals Owner")
```

```

approver_headers, approver_id = _auth("Approvals Approver")
cr_a = _create_change_request(owner_headers, title="Add OTP authentication for customers")
cr_b = _create_change_request(owner_headers, title="Add loyalty points to Alight.com checkout")

pending_approval = _request_approval(cr_a["id"], owner_headers, approver_id, approval_type="security")
resolved_approval = _request_approval(cr_b["id"], owner_headers, approver_id, approval_type="technical")
_respond(cr_b["id"], resolved_approval["id"], approver_headers, "approved")

pending_only = client.get("/api/my-work/approvals", headers=approver_headers)
assert pending_only.status_code == 200
pending_ids = {a["id"] for a in pending_only.json()}
assert pending_ids == {pending_approval["id"]}

everything = client.get("/api/my-work/approvals?status=all", headers=approver_headers)
assert everything.status_code == 200
all_ids = {a["id"] for a in everything.json()}
assert all_ids == {pending_approval["id"], resolved_approval["id"]}

def test_my_reviews_excludes_non_review_roles_but_my_assignments_includes_them(monkeypatch):
    owner_headers, _owner_id = _auth("Roles Owner")
    person_headers, person_id = _auth("QA Owner Person")
    cr = _create_change_request(owner_headers)

    # qa owner is a real AssignmentRole, but not one of the four "review"
    # roles (Reviewer/Technical Lead/Security Reviewer/Owner).
    _assign(cr["id"], owner_headers, person_id, "qa_owner")

    reviews = client.get("/api/my-work/reviews", headers=person_headers)
    assert reviews.status_code == 200
    assert cr["id"] not in {item["change_request_id"] for item in reviews.json()}

    assignments = client.get("/api/my-work/assignments", headers=person_headers)
    assert assignments.status_code == 200
    assignment_ids = {item["change_request_id"] for item in assignments.json()}
    assert cr["id"] in assignment_ids
    matching = next(item for item in assignments.json() if item["change_request_id"] == cr["id"])
    assert matching["roles"] == ["qa_owner"]

def test_my_reviews_includes_the_four_review_roles(monkeypatch):
    owner_headers, _owner_id = _auth("Review Roles Owner")
    reviewer_headers, reviewer_id = _auth("Security Reviewer Person")
    cr = _create_change_request(owner_headers)
    _assign(cr["id"], owner_headers, reviewer_id, "security_reviewer")

    reviews = client.get("/api/my-work/reviews", headers=reviewer_headers)
    assert reviews.status_code == 200
    ids = {item["change_request_id"] for item in reviews.json()}
    assert cr["id"] in ids

def test_changes_requested_from_me_shows_for_owner_not_for_the_approver(monkeypatch):
    owner_headers, _owner_id = _auth("Changes Requested Owner")
    approver_headers, approver_id = _auth("Changes Requesting Approver")
    cr = _create_change_request(owner_headers)
    approval = _request_approval(cr["id"], owner_headers, approver_id, approval_type="security")
    _respond(cr["id"], approval["id"], approver_headers, "changes_requested", comment="Please add rate limiting.")

    owner_view = client.get("/api/my-work/changes-requested", headers=owner_headers)
    assert owner_view.status_code == 200
    owner_ids = {item["change_request_id"] for item in owner_view.json()}
    assert cr["id"] in owner_ids
    matching = next(item for item in owner_view.json() if item["change_request_id"] == cr["id"])
    assert matching["comment"] == "Please add rate limiting."

    # The approver who personally requested the changes is NOT the CR's
    # owner - this must never show up on THEIR own changes-requested list.
    approver_view = client.get("/api/my-work/changes-requested", headers=approver_headers)
    assert approver_view.status_code == 200
    assert cr["id"] not in {item["change_request_id"] for item in approver_view.json()}

def test_mentions_reuses_the_mentioned_notification(monkeypatch):
    author_headers, author_id = _auth("Comment Author")
    mentioned_headers, mentioned_id = _auth("Mentioned Person")
    cr = _create_change_request(author_headers)

    # Register the mentioned user's exact display name into the comment.
    mentioned_name = "Mentioned Person"
    comment = client.post(
        f"/api/change-requests/{cr['id']}/comments",
        json={"body": f"@{mentioned_name} can you take a look at this OTP flow?"},
        headers=author_headers,
    )
    assert comment.status_code == 201

    mentions = client.get("/api/my-work/mentions", headers=mentioned_headers)
    assert mentions.status_code == 200
    assert len(mentions.json()) == 1
    assert mentions.json()[0]["type"] == "mentioned"
    assert mentions.json()[0]["change_request_id"] == cr["id"]

    # The comment's own author never mentions themselves.
    author_mentions = client.get("/api/my-work/mentions", headers=author_headers)
    assert author_mentions.status_code == 200
    assert len(author_mentions.json()) == 0

def test_test_overdue_items_excludes_due_soon_and_resolved(monkeypatch):
    from datetime import datetime, timedelta

    owner_headers, _owner_id = _auth("Overdue Owner")
    approver_headers, approver_id = _auth("Overdue Approver")
    cr = _create_change_request(owner_headers)

    overdue = _request_approval(
        cr["id"], owner_headers, approver_id,
        due_date=(datetime.utcnow() - timedelta(days=2)).isoformat(),
        approval_type="security",
    )
    due_soon = _request_approval(
        cr["id"], owner_headers, approver_id,
        due_date=(datetime.utcnow() + timedelta(hours=6)).isoformat(),
        approval_type="technical",
    )
    resolved_but_overdue = _request_approval(
        cr["id"], owner_headers, approver_id,
        due_date=(datetime.utcnow() - timedelta(days=5)).isoformat(),
        approval_type="product",
    )
    _respond(cr["id"], resolved_but_overdue["id"], approver_headers, "approved")

    response = client.get("/api/my-work/overdue", headers=approver_headers)
    assert response.status_code == 200
    ids = {a["id"] for a in response.json()}
    assert ids == {overdue["id"]}
    assert due_soon["id"] not in ids
    assert resolved_but_overdue["id"] not in ids

```

```

def test_summary_counts_match_section_endpoints(monkeypatch):
    owner_headers, _owner_id = _auth("Summary Owner")
    person_headers, person_id = _auth("Summary Person")
    cr = _create_change_request(owner_headers)
    _assign(cr["id"], owner_headers, person_id, "reviewer")
    approval = _request_approval(cr["id"], owner_headers, person_id, approval_type="security")

    summary = client.get("/api/my-work/summary", headers=person_headers)
    assert summary.status_code == 200
    body = summary.json()

    reviews = client.get("/api/my-work/reviews", headers=person_headers).json()
    assignments = client.get("/api/my-work/assignments", headers=person_headers).json()
    approvals_pending = client.get("/api/my-work/approvals", headers=person_headers).json()

    assert body["my_reviews"] == len(reviews)
    assert body["my_assignments"] == len(assignments)
    assert body["my_approvals_pending"] == len(approvals_pending)
    assert approval["id"] in {a["id"] for a in approvals_pending}

def test_scoping_never_leaks_between_users(monkeypatch):
    a_headers, a_id = _auth("Scoping User A")
    b_headers, b_id = _auth("Scoping User B")
    cr_a = _create_change_request(a_headers, title="Add OTP authentication for customers")

    # User B has nothing at all on cr_a - none of B's My Work endpoints
    # should show any trace of it.
    for path in ("/api/my-work/change-requests", "/api/my-work/reviews", "/api/my-work/assignments"):
        response = client.get(path, headers=b_headers)
        assert response.status_code == 200
        ids = {item.get("id") or item.get("change_request_id") for item in response.json()}
        assert cr_a["id"] not in ids

```

tests/test_module18_phase7_notifications.py

425 lines

```
"""Verifies Module 18 Phase 7 (Notifications, My Work & Personal
Engineering Queue - dedicated notification coverage + report):

Spec section 8 asks for dedicated tests confirming notifications for:
Approval, Assignment, Mention, Changes Requested, Re-analysis, Re-approval
- and that users only ever see notifications belonging to them. Several of
these already have thorough dedicated coverage elsewhere in this test
suite (this file's docstrings on each test say exactly where); this file
fills the gaps that were still Notification-row-less - real
POST/PUT-triggered assertions that a Notification row was actually
created for the right person, and never for the wrong one - and adds one
cross-cutting scoping test specific to Module 18's own newer notification
types (risk escalated, analysis completed, reapproval required,
analysis outdated) alongside the older ones, closing Spec section 8's
"ensure users only see notifications belonging to them" for this module.

* Approval - APPROVAL REQUESTED notifies the tagged approver only (never
the requester); a response (Approved/Rejected/Changes Requested)
notifies the requester only (never the responder) - see
test_workflow_approvals.py for the history-event/permission side of
this same flow, not previously asserted against a Notification row.
* Assignment - ASSIGNED notifies the newly-assigned person only (never
whoever assigned them) - see test_workflow_status_assignments.py for
the history-event/permission side, not previously asserted against a
Notification row.
* Mention - already extensively covered by
test_workflow_comments_notifications.py (dedup against COMMENT_ADDED,
scoping, etc.) - one light test here for this module's own
self-contained record that the spec item is satisfied.
* Changes Requested - same Approval-response flow above, called out on
its own since spec section 8 lists it separately (it's also what
drives My Work's own "Changes Requested From Me" tab - see
test_module18_phase3.py).
* Re-analysis - ANALYSIS SIGNIFICANTLY CHANGED/RISK_ESCALATED/
ANALYSIS_COMPLETED are already extensively covered by
test_ai_analysis_phase3.py and test_module18_phase2.py; this file adds
the one still-missing piece - ANALYSIS_OUTDATED (a CR edited after it
was analyzed, telling its stakeholders a re-analysis is worth doing)
had never been asserted against a real Notification row, only the
is analysis outdated computed flag (test_change_request_editing.py).
* Re-approval - REAPPROVAL_REQUIRED (a CR edited while an approval
against an earlier version is still pending) had never been asserted
against a real Notification row either - only its own history event
(app/services/versioning.py).

Every test here uses a fake, monkeypatched AI provider (analysis engine.
get_ai_provider) where analysis is involved - no real network call,
matching this app's established testing pattern. No test data ever uses
grubrrr.com - alight.com throughout, per this project's own standing rule.

Run with (from backend/): python -m pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m18p7-{uuid.uuid4().hex[:10]}@example.com"

def _auth(name: str = "Module18 Phase7 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP login to Alight.com checkout",
        "description": "Let customers verify identity with a one-time passcode before checkout.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Alight.com Checkout",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _notifications_for(headers: dict, notification_type: str) -> list[dict]:
    response = client.get("/api/notifications", headers=headers)
    assert response.status_code == 200
    return [n for n in response.json() if n["type"] == notification_type]

_ANALYSIS_RESPONSE = {
    "summary": "Adds OTP-based verification to checkout.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.9, "reason": "New auth step."},
    "requirements": [],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_findings": [],
    "security_analysis": {"concerns": [], "summary": ""},
    "impact_assessments": [],
    "complexity": {"level": "low", "reasoning": "A contained auth feature.", "confidence": "medium"},
    "effort": {"backend": "2 days", "frontend": "1 day", "testing": "1 day", "total": "4 days", "confidence": "medium"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Low risk, clear requirement."},
}
```

```

class FixedProvider:
    """Same stand-in used across this project's other analysis tests -
    always returns the same, minimal, well-formed analysis JSON."""

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return json.dumps(_ANALYSIS_RESPONSE)

# --- Approval (request + response) -----

def test_approval_requested_notifies_approver_not_requester():
    requester_headers, _ = auth("Approval Requester")
    approver_headers, approver_id = auth("Approval Approver")
    created = _create_change_request(requester_headers)

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval type": "general", "approver_user_id": approver_id},
        headers=requester_headers,
    )
    assert response.status_code == 201

    approver_notifs = _notifications_for(approver_headers, "approval_requested")
    assert len(approver_notifs) == 1
    assert approver_notifs[0]["change_request_id"] == created["id"]
    assert approver_notifs[0]["is_read"] is False

    # The requester never gets a notification about their own request.
    assert _notifications_for(requester_headers, "approval_requested") == []

def test_approval_approved_notifies_requester_not_approver():
    requester_headers, _ = auth("Approve Requester")
    approver_headers, approver_id = auth("Approve Approver")
    created = _create_change_request(requester_headers)
    approval = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval type": "general", "approver_user_id": approver_id},
        headers=requester_headers,
    ).json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{approval['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )
    assert response.status_code == 200

    requester_notifs = _notifications_for(requester_headers, "approved")
    assert len(requester_notifs) == 1
    assert requester_notifs[0]["change_request_id"] == created["id"]

    # The approver never gets notified about their own decision.
    assert _notifications_for(approver_headers, "approved") == []

def test_approval_rejected_notifies_requester_not_approver():
    requester_headers, _ = auth("Reject Requester")
    approver_headers, approver_id = auth("Reject Approver")
    created = _create_change_request(requester_headers)
    approval = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval type": "general", "approver_user_id": approver_id},
        headers=requester_headers,
    ).json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{approval['id']}/respond",
        json={"status": "rejected", "comment": "Missing rate-limit design."},
        headers=approver_headers,
    )
    assert response.status_code == 200

    requester_notifs = _notifications_for(requester_headers, "rejected")
    assert len(requester_notifs) == 1
    assert requester_notifs[0]["change_request_id"] == created["id"]
    assert _notifications_for(approver_headers, "rejected") == []

# --- Changes Requested (spec section 8's own separate line item) -----

def test_changes_requested_notifies_requester_not_approver():
    requester_headers, _ = auth("ChangesReq Requester")
    approver_headers, approver_id = auth("ChangesReq Approver")
    created = _create_change_request(requester_headers)
    approval = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval type": "general", "approver_user_id": approver_id},
        headers=requester_headers,
    ).json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{approval['id']}/respond",
        json={"status": "changes requested", "comment": "Please add OTP rate limiting first."},
        headers=approver_headers,
    )
    assert response.status_code == 200

    requester_notifs = _notifications_for(requester_headers, "changes_requested")
    assert len(requester_notifs) == 1
    assert requester_notifs[0]["change_request_id"] == created["id"]
    assert _notifications_for(approver_headers, "changes_requested") == []

    # This is also exactly what should now show up on the requester's own
    # My Work "Changes Requested From Me" tab (see test_module18_phase3.py
    # for that endpoint's own dedicated coverage) - a quick cross-check
    # here that the two features agree on the same underlying event.
    changes_requested = client.get("/api/my-work/changes-requested", headers=requester_headers).json()
    assert any(item["change_request_id"] == created["id"] for item in changes_requested)

# --- Assignment -----

def test_assignment_notifies_assignee_not_assigner():
    assigner_headers, _ = auth("Assigner")
    assignee_headers, assignee_id = auth("Assignee")
    created = _create_change_request(assigner_headers)

    response = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=assigner_headers,
    )

```



```

assert response.status_code == 201

assignee_notifs = notifications_for(assignee_headers, "assigned")
assert len(assignee_notifs) == 1
assert assignee_notifs[0]["change_request_id"] == created["id"]

# Nobody assigns themselves a notification for their own action.
assert _notifications_for(assigner_headers, "assigned") == []

# --- Mention (already thoroughly covered elsewhere - light check here) ----

def test_mention_notification_fires():
    # Mention matching (app/services/mentions.py) checks every registered
    # user's name as a substring, system-wide - not scoped to this test -
    # so this name has to be unique across the whole shared test database,
    # not just this file (a plain "Mentioned Person" collided with
    # test module18 phase3.py's own fixture of the same name and matched
    # both users). A uuid suffix guarantees no other test can ever collide
    # with it, mirroring this project's own established fix for this exact
    # class of problem (see test module18 phase2.py's own docstring on
    # "fixture wording carefully designed to avoid accidental keyword-
    # overlap collisions between unrelated rows").
    unique_name = f"Mentioned Phase7 Person {uuid.uuid4().hex[:8]}"
    author_headers, _ = auth("Mention Author Phase7")
    mentioned_headers, mentioned_id = auth(unique_name)
    created = _create_change_request(author_headers)

    response = client.post(
        f"/api/change-requests/{created['id']}/comments",
        json={"body": f"@{unique_name} can you weigh in on this?"},
        headers=author_headers,
    )
    assert response.status_code == 201
    assert response.json()["mentioned_user_ids"] == [mentioned_id]

    mentioned_notifs = _notifications_for(mentioned_headers, "mentioned")
    assert len(mentioned_notifs) == 1
    assert mentioned_notifs[0]["change_request_id"] == created["id"]

# --- Re-analysis: ANALYSIS_OUTDATED (editing after analysis) -----

def test_analysis_outdated_notifies_creator_and_assignees_on_edit(monkeypatch):
    creator_headers, creator_id = auth("Outdated Creator")
    assignee_headers, assignee_id = auth("Outdated Assignee")
    monkeypatch.setattr(analysis_engine, "get ai provider", lambda: _FixedProvider())
    created = _create_change_request(creator_headers)

    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=creator_headers,
    )
    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
    assert analyze.status_code == 201

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Let customers verify identity with a one-time passcode sent by SMS before checkout."},
        headers=creator_headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["is_analysis_outdated"] is True

    assignee_notifs = _notifications_for(assignee_headers, "analysis_outdated")
    assert len(assignee_notifs) == 1
    assert assignee_notifs[0]["change_request_id"] == created["id"]

    # The person who made the edit never gets notified about their own edit.
    assert _notifications_for(creator_headers, "analysis_outdated") == []

# --- Re-approval: REAPPROVAL_REQUIRED (editing while an approval pends) ----

def test_reapproval_required_notifies_approver_when_cr_edited_while_pending():
    creator_headers, _ = auth("Reapproval Creator")
    approver_headers, approver_id = auth("Reapproval Approver")
    created = _create_change_request(creator_headers)

    approval = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "general", "approver_user_id": approver_id},
        headers=creator_headers,
    ).json()
    assert approval["cr_version"] == 1

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"priority": "high"},
        headers=creator_headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["current_version"] == 2

    approver_notifs = _notifications_for(approver_headers, "reapproval_required")
    assert len(approver_notifs) == 1
    assert approver_notifs[0]["change_request_id"] == created["id"]

    # The person who made the edit (the requester/creator here) never gets
    # a reapproval notification about their own edit.
    assert _notifications_for(creator_headers, "reapproval_required") == []

def test_reapproval_required_never_fires_without_a_pending_approval():
    creator_headers, _ = auth("No Pending Approval Creator")
    created = _create_change_request(creator_headers)

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"priority": "critical"},
        headers=creator_headers,
    )
    assert edit.status_code == 200

    assert _notifications_for(creator_headers, "reapproval_required") == []

# --- Cross-cutting: every user only ever sees their own notifications ----

def test_module18_notification_types_are_scoped_to_the_current_user(monkeypatch):
    """One consolidated check that spec section 8's "ensure users only see
    notifications belonging to them" holds for Module 18's own newer
    notification types (risk escalated, analysis completed,
    reapproval required, analysis outdated) - deadline approaching already
    has its own dedicated scoping test in test module18 phase1.py, and the
    older types (assigned/approval requested/approved/rejected)

```

```

changes_requested/mentioned/comment added) already have theirs in
test workflow_comments notifications.py."""
creator_headers, _ = auth("Scoping Creator")
assignee_headers, assignee_id = _auth("Scoping Assignee")
approver_headers, approver_id = _auth("Scoping Approver")
outsider_headers, _ = _auth("Scoping Outsider") # has no stake in this CR at all

monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider())
created = _create_change_request(creator_headers)
client.post(
    f"/api/change-requests/{created['id']}/assignments",
    json={"user_id": assignee_id, "role": "reviewer"},
    headers=creator_headers,
)
client.post(f"/api/change-requests/{created['id']}/analyze", headers=creator_headers)
client.post(
    f"/api/change-requests/{created['id']}/approvals",
    json={"approval_type": "general", "approver_user_id": approver_id},
    headers=creator_headers,
)
client.put(
    f"/api/change-requests/{created['id']}",
    json={"priority": "high"},
    headers=creator_headers,
)

outsider_ids = {n["id"] for n in client.get("/api/notifications", headers=outsider_headers).json()}
assert outsider_ids == set()

# Sanity check the fixture actually produced notifications for someone,
# so an empty outsider inbox above is a real scoping result and not
# just because nothing fired at all.
assignee_ids = {n["id"] for n in client.get("/api/notifications", headers=assignee_headers).json()}
approver_ids = {n["id"] for n in client.get("/api/notifications", headers=approver_headers).json()}
assert len(assignee_ids) > 0
assert len(approver_ids) > 0
assert outsider_ids.isdisjoint(assignee_ids)
assert outsider_ids.isdisjoint(approver_ids)

```

tests/test_module19_phase1.py

203 lines

```
"""Verifies Module 19 Phase 1 (Engineering Change Analytics - foundation):
the new 'AI ANALYSIS FAILED' history event that gets logged whenever a
POST /analyze attempt fails, and the shared 'app/services/analytics.py'
filtering helpers those failure counts (and every later Module 19 metric)
will build on.

Before this phase, a failed analysis attempt (bad JSON, schema validation
failure, timeout, etc.) resulted in nothing at all being persisted to the
database - just an HTTPException returned to the caller. That was a real
gap: Module 19's "AI analysis failures" metric (spec section 7) would have
had zero real data to report. This is the one dedicated test file for that
new tracking hook.

Following this project's established HTTP-only testing convention, the new
history event is verified the same way every other history event in this
suite is - through the real /analyze endpoint (monkeypatched with a fake
AI provider that fails in different ways) and the real
GET /{id}/history endpoint, never by touching the database directly.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m19-{uuid.uuid4().hex[:10]}@alight.com"

def _auth() -> dict:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Module19 Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return {"Authorization": f"Bearer {response.json()['access_token']}"}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add order-status webhook to POS",
        "description": "Emit a webhook event whenever an order's status changes on the POS.",
        "business_objective": "Let downstream systems react to order status changes in real time.",
        "priority": "medium",
        "requested_by": "Casey Alight",
        "target_system": "POS Backend",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

class _FailingProvider:
    """Same shape as test analysis_engine.py's own `FakeProvider` - stands
    in for AnthropicProvider, but always fails in one of the ways the real
    provider already can (invalid JSON, schema violation, or a raised
    exception standing in for a timeout)."""
    def __init__(self, response_text=None, raise_exc=None):
        self._text = response_text
        self._raise = raise_exc
    def is_configured(self) -> bool:
        return True
    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

def _get_history(headers: dict, change_request_id: int) -> list:
    response = client.get(f"/api/change-requests/{change_request_id}/history", headers=headers)
    assert response.status_code == 200
    return response.json()

def test_invalid_json_failure_logs_ai_analysis_failed_history_event(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FailingProvider("this is not json at all"))
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 422
    history = _get_history(headers, created["id"])
    failed_events = [event for event in history if event["action"] == "ai_analysis_failed"]
    assert len(failed_events) == 1
    assert failed_events[0]["actor_label"] == "AI Analyzer"
    assert failed_events[0]["user_id"] is None
    assert failed_events[0]["new_value"]

def test_timeout_failure_logs_ai_analysis_failed_history_event(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    class _Timeout(Exception):
        pass
    _Timeout.__name__ = "APITimeoutError"
    _patch_provider(monkeypatch, _FailingProvider(raise_exc=_Timeout("took too long")))
```

```

response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
assert response.status_code == 504

history = _get_history(headers, created["id"])
failed_events = [event for event in history if event["action"] == "ai_analysis_failed"]
assert len(failed_events) == 1
assert "took too long" in failed_events[0]["new_value"]

def test_successful_analysis_does_not_log_a_failure_event(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    ok_response = {
        "summary": "Adds a webhook emitted on order status change.",
        "classification": {"category": "Integration", "confidence": 0.8, "reason": "New outbound event."},
        "requirements": [
            {"category": "functional", "description": "Emit webhook on status change.", "priority": "high"},
        ],
        "affected_components": [
            {
                "name": "Order Service",
                "type": "backend",
                "impact_level": "low",
                "reason": "Adds an outbound call on an existing state transition.",
                "confidence": 70,
            }
        ],
        "dependencies": [],
        "risks": [],
        "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
        "complexity": {"level": "low", "reasoning": "Single new outbound call."},
        "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-001",
                "title": "Webhook fires on status change",
                "type": "integration",
                "priority": "high",
                "description": "Change an order's status and verify the webhook fires.",
                "expected_result": "Webhook payload matches the new status.",
            }
        ],
        "implementation_plan": [
            {
                "task": "Add webhook emitter",
                "description": "Emit the webhook after the status transition commits.",
                "component": "Order Service",
                "priority": "high",
                "estimated_effort": "4h",
            }
        ],
        "recommendation": {"decision": "approve", "reasoning": "Small, well-scoped, low risk."},
    }
    _patch_provider(monkeypatch, _FailingProvider(json.dumps(ok_response)))

    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

    history = _get_history(headers, created["id"])
    failed_events = [event for event in history if event["action"] == "ai_analysis_failed"]
    assert failed_events == []

def test_repeated_failures_log_one_event_each(monkeypatch):
    headers = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, _FailingProvider("still not json"))

    first = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert first.status_code == 422
    second = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert second.status_code == 422

    history = _get_history(headers, created["id"])
    failed_events = [event for event in history if event["action"] == "ai_analysis_failed"]
    assert len(failed_events) == 2

```

tests/test_module19_phase2.py

305 lines

```
"""Verifies Module 19 Phase 2 (Engineering Change Analytics): the new
GET /api/analytics/executive endpoint - spec section 1 (Executive Metrics)
and section 2 (Workflow Metrics), computed together over real database
rows this test file creates itself (never invented numbers - every count
asserted below is checked against change requests this file can see the
full lifecycle of).

Following this project's established HTTP-only testing convention, every
check here goes through real endpoints: change requests are created and
walked through the real status-transition endpoint
(PUT /{id}/status), approvals through the real approval request/respond
endpoints, analyses through the real /analyze endpoint (monkeypatched
provider) - never a direct database write.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from datetime import datetime, timedelta, timezone
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

CONFIG_CHANGE_RESPONSE = {
    "summary": "Adds a configurable timeout to the payment gateway client.",
    "classification": {"category": "Configuration", "confidence": 0.8, "reason": "Tunable value only."},
    "requirements": [
        {"category": "functional", "description": "Add a timeout config value.", "priority": "medium"},
    ],
    "affected_components": [
        {
            "name": "Payment Client",
            "type": "backend",
            "impact_level": "low",
            "reason": "Reads a new timeout value.",
            "confidence": 70,
        }
    ],
    "dependencies": [],
    "risks": [],
    "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "low", "reasoning": "Single config value."},
    "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
    "missing_information": [],
    "test_cases": [
        {
            "id": "TC-001",
            "title": "Timeout applies to slow gateway calls",
            "type": "integration",
            "priority": "medium",
            "description": "Verify the configured timeout is honored.",
            "expected_result": "Call aborts at the configured timeout.",
        }
    ],
    "implementation_plan": [
        {
            "task": "Add timeout config field",
            "description": "Add a validated numeric timeout config field.",
            "component": "Payment Client",
            "priority": "medium",
            "estimated_effort": "4h",
        }
    ],
    "recommendation": {"decision": "approve", "reasoning": "Small, well-scoped, low risk."},
}

class FixedProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def patch_provider(monkeypatch) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: FixedProvider(json.dumps(CONFIG_CHANGE_RESPONSE)))

def unique_email() -> str:
    return f"m19p2-{uuid.uuid4().hex[:10]}@alight.com"

def auth(name: str = "Module19 Phase2 Tester") -> tuple[dict, int]:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add payment gateway timeout config",
        "description": "Make the payment gateway client timeout configurable instead of hardcoded.",
        "business_objective": "Avoid hung requests during gateway outages.",
        "priority": "medium",
        "requested_by": "Jamie Alight",
        "target_system": "Payment Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()
```

```

def _set_status(headers: dict, cr_id: int, target: str, reason: str | None = None) -> None:
    payload = {"status": target}
    if reason is not None:
        payload["reason"] = reason
    response = client.put(f"/api/change-requests/{cr_id}/status", json=payload, headers=headers)
    assert response.status_code == 200, response.text

def _get_executive(headers: dict, **params) -> dict:
    response = client.get("/api/analytics/executive", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def test_executive_metrics_bucket_a_pending_analysis_cr_as_open():
    headers, owner_id = _auth()
    created = _create_change_request(headers)

    data = _get_executive(headers, owner_id=owner_id)
    assert data["executive"]["total"] == 1
    assert data["executive"]["open"] == 1
    assert data["executive"]["pending_approval"] == 0
    assert data["executive"]["approved"] == 0
    assert data["executive"]["closed"] == 0

def test_executive_metrics_approval_required_bucket():
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _set_status(headers, created["id"], "analyzed")
    _set_status(headers, created["id"], "in_review")
    _set_status(headers, created["id"], "approval_required")

    data = _get_executive(headers, owner_id=owner_id)
    assert data["executive"]["pending_approval"] == 1
    assert data["executive"]["open"] == 0

def test_executive_metrics_full_lifecycle_to_closed():
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    cr_id = created["id"]
    for target in (
        "analyzed",
        "in_review",
        "approval_required",
        "approved",
        "implementation_planned",
        "in_progress",
        "implemented",
        "validated",
        "closed",
    ):
        _set_status(headers, cr_id, target)

    data = _get_executive(headers, owner_id=owner_id)
    assert data["executive"]["closed"] == 1
    assert data["executive"]["total"] == 1
    # Every other named bucket is 0 - a CR only ever occupies exactly one.
    assert data["executive"]["open"] == 0
    assert data["executive"]["pending_approval"] == 0
    assert data["executive"]["approved"] == 0
    assert data["executive"]["rejected"] == 0
    assert data["executive"]["in_progress"] == 0

    # Workflow metrics: this CR really did reach In Progress and Closed, so
    # both averages should now be real, non-negative numbers, not null.
    assert data["workflow"]["avg_time_to_implementation_hours"] is not None
    assert data["workflow"]["avg_time_to_implementation_hours"] >= 0
    assert data["workflow"]["avg_time_to_closure_hours"] is not None
    assert data["workflow"]["avg_time_to_closure_hours"] >= 0

def test_executive_metrics_cancelled_counts_in_total_but_no_named_bucket():
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _set_status(headers, created["id"], "cancelled", reason="No longer needed.")

    data = _get_executive(headers, owner_id=owner_id)
    assert data["executive"]["total"] == 1
    assert data["executive"]["open"] == 0
    assert data["executive"]["pending_approval"] == 0
    assert data["executive"]["approved"] == 0
    assert data["executive"]["rejected"] == 0
    assert data["executive"]["in_progress"] == 0
    assert data["executive"]["closed"] == 0

def test_workflow_avg_time_to_analysis_and_crs_with_requested_changes(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

    data = _get_executive(headers, owner_id=owner_id)
    assert data["workflow"]["avg_time_to_analysis_hours"] is not None
    assert data["workflow"]["avg_time_to_analysis_hours"] >= 0

    # No CR here has ever been moved to Changes Requested.
    assert data["workflow"]["crs_with_requested_changes"] == 0

def test_workflow_crs_with_requested_changes_and_no_data_defaults():
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _set_status(headers, created["id"], "analyzed")
    _set_status(headers, created["id"], "in_review")
    _set_status(headers, created["id"], "changes_requested", reason="Needs more detail.")

    data = _get_executive(headers, owner_id=owner_id)
    assert data["executive"]["open"] == 1 # CHANGES REQUESTED is one of the "open" statuses
    assert data["workflow"]["crs_with_requested_changes"] == 1
    # This CR was never analyzed, so there's nothing to average - null, not 0.
    assert data["workflow"]["avg_time_to_analysis_hours"] is None

def test_workflow_approval_duration_and_waiting_for_approval(monkeypatch):
    requester_headers, requester_id = _auth("Approval Requester Phase2")
    approver_headers, approver_id = _auth("Approver Phase2")
    created = _create_change_request(requester_headers)
    cr_id = created["id"]
    _set_status(requester_headers, cr_id, "analyzed")
    _set_status(requester_headers, cr_id, "in_review")

    request_response = client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": "technical", "approver_user_id": approver_id},
        headers=requester_headers,
    )

```

```

assert request_response.status_code == 201
approval_id = request_response.json()["id"]

# Still pending: counts toward crs waiting for approval.
mid_data = _get_executive(requester_headers, owner_id=requester_id)
assert mid_data["workflow"]["crs_waiting_for_approval"] == 1
assert mid_data["workflow"]["avg_approval_time_hours"] is None

respond_response = client.post(
    f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond",
    json={"status": "approved"},
    headers=approver_headers,
)
assert respond_response.status_code == 200

final_data = _get_executive(requester_headers, owner_id=requester_id)
assert final_data["workflow"]["crs_waiting_for_approval"] == 0
assert final_data["workflow"]["avg_approval_time_hours"] is not None
assert final_data["workflow"]["avg_approval_time_hours"] >= 0

def test_owner_filter_scopes_to_only_that_owners_change_requests():
    headers_a, owner_a = _auth("Owner A Phase2")
    headers_b, owner_b = _auth("Owner B Phase2")
    _create_change_request(headers_a)
    _create_change_request(headers_b)
    _create_change_request(headers_b)

    data_a = _get_executive(headers_a, owner_id=owner_a)
    data_b = _get_executive(headers_a, owner_id=owner_b)
    assert data_a["executive"]["total"] == 1
    assert data_b["executive"]["total"] == 2

def test_invalid_risk_filter_returns_422():
    headers, _ = _auth()
    response = client.get("/api/analytics/executive", params={"risk": "extreme"}, headers=headers)
    assert response.status_code == 422

def test_invalid_category_filter_returns_422():
    headers, _ = _auth()
    response = client.get("/api/analytics/executive", params={"category": "Not A Real Category"}, headers=headers)
    assert response.status_code == 422

def test_executive_endpoint_requires_authentication():
    response = client.get("/api/analytics/executive")
    assert response.status_code == 401

```

tests/test_module19_phase3.py

267 lines

```
"""Verifies Module 19 Phase 3 (Engineering Change Analytics): the new
GET /api/analytics/risk and GET /api/analytics/approval-bottlenecks
endpoints - spec sections 3 (Risk Analytics) and 4 (Approval Bottlenecks) -
against real database rows this test file creates itself.

Following this project's established HTTP-only testing convention, every
check here goes through real endpoints: analyses via the real /analyze
endpoint (monkeypatched provider, controlling the resulting risk_score via
the AI response's own `risks` list - see
app/services/analysis_engine.py: overall_risk_score, which this file
mirrors rather than reimplements), approvals through the real approval
request/respond endpoints, never a direct database write.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from datetime import datetime
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _analysis_response(risk_score: float | None) -> dict:
    """A minimal, schema-valid AI response. risk_score is controlled via
    the `risks` list exactly the way overall_risk_score computes it; no
    risks -> 5.0 (low); one risk with the given score -> that score."""
    risks = []
    if risk_score is not None:
        risks = [
            {
                "category": "operational",
                "description": "A risk used only to pin the overall risk score for this test.",
                "severity": "high" if risk_score >= 50 else "low",
                "probability": 0.5,
                "score": risk_score,
                "explanation": "Deterministic test fixture risk.",
                "mitigation": "N/A - test fixture.",
            }
        ]
    return {
        "summary": "A test change used only to exercise Module 19 Phase 3's analytics endpoints.",
        "classification": {"category": "Configuration", "confidence": 0.8, "reason": "Test fixture."},
        "requirements": [
            {"category": "functional", "description": "Test fixture requirement.", "priority": "medium"}
        ],
        "affected_components": [
            {
                "name": "Test Component",
                "type": "backend",
                "impact_level": "low",
                "reason": "Test fixture.",
                "confidence": 70,
            }
        ],
        "dependencies": [],
        "risks": risks,
        "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
        "complexity": {"level": "low", "reasoning": "Test fixture."},
        "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-001",
                "title": "Test fixture case",
                "type": "integration",
                "priority": "medium",
                "description": "Test fixture.",
                "expected_result": "Test fixture.",
            }
        ],
        "implementation_plan": [
            {
                "task": "Test fixture task",
                "description": "Test fixture.",
                "component": "Test Component",
                "priority": "medium",
                "estimated_effort": "4h",
            }
        ],
        "recommendation": {"decision": "approve", "reasoning": "Test fixture - deterministic risk score only."},
    }

class FixedProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, risk_score: float | None) -> None:
    monkeypatch.setattr(
        analysis_engine, "get_ai_provider", lambda: _FixedProvider(json.dumps(_analysis_response(risk_score)))
    )

def _unique_email() -> str:
    return f"m19p3-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module19 Phase3 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
```



```

assert response.status_code == 201
body = response.json()
return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Module 19 Phase 3 fixture change request",
        "description": "A change request used only to exercise the risk/approval-bottleneck analytics endpoints.",
        "business_objective": "N/A - test fixture.",
        "priority": "medium",
        "requested_by": "Test Fixture",
        "target_system": "Test System",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _analyze(headers: dict, cr_id: int, monkeypatch, risk_score: float | None) -> None:
    _patch_provider(monkeypatch, risk_score)
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201, response.text

def _set_status(headers: dict, cr_id: int, target: str, reason: str | None = None) -> None:
    payload = {"status": target}
    if reason is not None:
        payload["reason"] = reason
    response = client.put(f"/api/change-requests/{cr_id}/status", json=payload, headers=headers)
    assert response.status_code == 200, response.text

def _get_risk(headers: dict, **params) -> dict:
    response = client.get("/api/analytics/risk", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def _get_bottlenecks(headers: dict, **params) -> dict:
    response = client.get("/api/analytics/approval-bottlenecks", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def test_risk_not_analyzed_bucket():
    headers, owner_id = _auth()
    _create_change_request(headers)

    data = _get_risk(headers, owner_id=owner_id)
    assert data["current_distribution"]["not_analyzed"] == 1
    assert data["current_distribution"]["low"] == 0

def test_risk_low_bucket_when_no_risks_returned(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch, risk_score=None)

    data = _get_risk(headers, owner_id=owner_id)
    assert data["current_distribution"]["low"] == 1
    assert data["current_distribution"]["not_analyzed"] == 0

def test_risk_critical_bucket_and_over_time_entry(monkeypatch):
    headers, owner_id = _auth()
    created = create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch, risk_score=90)

    data = _get_risk(headers, owner_id=owner_id)
    assert data["current_distribution"]["critical"] == 1

    this_month = datetime.utcnow().strftime("%Y-%m")
    matching_periods = [point for point in data["over_time"] if point["period"] == this_month]
    assert len(matching_periods) == 1
    assert matching_periods[0]["critical"] == 1
    assert matching_periods[0]["low"] == 0

def test_bottlenecks_pending_by_type_and_person():
    requester_headers, requester_id = _auth("Bottleneck Requester Phase3")
    approver_headers, approver_id = _auth("Bottleneck Approver Phase3")
    created = _create_change_request(requester_headers)
    cr_id = created["id"]
    _set_status(requester_headers, cr_id, "analyzed")
    _set_status(requester_headers, cr_id, "in_review")

    response = client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": "security", "approver_user_id": approver_id},
        headers=requester_headers,
    )
    assert response.status_code == 201

    data = _get_bottlenecks(requester_headers, owner_id=requester_id)
    assert data["pending_by_type"].get("Security") == 1
    assert len(data["pending_by_person"]) == 1
    entry = data["pending_by_person"][0]
    # Only a name and a count - no email or other account details leak here.
    assert set(entry.keys()) == {"approver_name", "count"}
    assert entry["count"] == 1

def test_bottlenecks_avg_duration_and_blockers_after_rejection():
    requester_headers, requester_id = _auth("Bottleneck Requester2 Phase3")
    approver_headers, approver_id = _auth("Bottleneck Approver2 Phase3")
    created = _create_change_request(requester_headers)
    cr_id = created["id"]
    _set_status(requester_headers, cr_id, "analyzed")
    _set_status(requester_headers, cr_id, "in_review")

    response = client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": "technical", "approver_user_id": approver_id},
        headers=requester_headers,
    )
    approval_id = response.json()["id"]

    respond = client.post(
        f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond",
        json={"status": "rejected", "comment": "Not ready yet."},
        headers=approver_headers,
    )
    assert respond.status_code == 200

    data = _get_bottlenecks(requester_headers, owner_id=requester_id)
    assert data["pending_by_type"] == {}
    assert data["avg_duration_hours_by_type"].get("Technical") is not None
    assert data["avg_duration_hours_by_type"]["Technical"] >= 0
    blocker_labels = {entry["approval_type_label"]: entry["count"] for entry in data["most_common_blockers"]}

```

```
assert blocker_labels.get("Technical") == 1

def test_risk_endpoint_requires_authentication():
    response = client.get("/api/analytics/risk")
    assert response.status_code == 401

def test_bottlenecks_endpoint_requires_authentication():
    response = client.get("/api/analytics/approval-bottlenecks")
    assert response.status_code == 401
```

tests/test_module19_phase4.py

271 lines

```
"""Verifies Module 19 Phase 4 (Engineering Change Analytics): the new
GET /api/analytics/change and GET /api/analytics/workload endpoints - spec
sections 5 (Change Analytics) and 6 (Workload) - against real database rows
this test file creates itself.

Following this project's established HTTP-only testing convention, every
check here goes through real endpoints: edits via the real PUT
/{id} endpoint (version bump + FIELD_CHANGED history), analyses via the
real /analyze endpoint (monkeypatched provider, controlling
missing information to produce a real, unresolved ClarificationQuestion),
assignments and approvals via their own real endpoints - never a direct
database write.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from datetime import datetime, timedelta
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _analysis_response(*, with_missing_information: bool) -> dict:
    missing_information = []
    if with_missing_information:
        missing_information = [
            {
                "question": "Which environments does this rollout target?",
                "priority": "important",
                "reason": "The change request doesn't say whether this covers staging, production, or both.",
            }
        ]
    return {
        "summary": "A test change used only to exercise Module 19 Phase 4's analytics endpoints.",
        "classification": {"category": "Configuration", "confidence": 0.8, "reason": "Test fixture."},
        "requirements": [
            {"category": "functional", "description": "Test fixture requirement.", "priority": "medium"},
        ],
        "affected_components": [
            {
                "name": "Test Component",
                "type": "backend",
                "impact_level": "low",
                "reason": "Test fixture.",
                "confidence": 70,
            }
        ],
        "dependencies": [],
        "risks": [],
        "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
        "complexity": {"level": "low", "reasoning": "Test fixture."},
        "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
        "missing_information": missing_information,
        "test_cases": [
            {
                "id": "TC-001",
                "title": "Test fixture case",
                "type": "integration",
                "priority": "medium",
                "description": "Test fixture.",
                "expected_result": "Test fixture.",
            }
        ],
        "implementation_plan": [
            {
                "task": "Test fixture task",
                "description": "Test fixture.",
                "component": "Test Component",
                "priority": "medium",
                "estimated_effort": "4h",
            }
        ],
        "recommendation": {
            "decision": "requires_clarification" if with_missing_information else "approve",
            "reasoning": "Test fixture.",
        },
    }

class FixedProvider:
    def __init__(self, response_text):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _patch_provider(monkeypatch, *, with_missing_information: bool) -> None:
    monkeypatch.setattr(
        analysis_engine,
        "get_ai_provider",
        lambda: FixedProvider(json.dumps(_analysis_response(with_missing_information=with_missing_information))),
    )

def _unique_email() -> str:
    return f"m19p4-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module19 Phase4 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
```

```

    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Module 19 Phase 4 fixture change request",
        "description": "A change request used only to exercise the change/workload analytics endpoints.",
        "business_objective": "N/A - test fixture.",
        "priority": "medium",
        "requested_by": "Test Fixture",
        "target_system": "Test System",
    }
    payload.update(overrides)
    response = client.post(f"/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _get_change(headers: dict, **params) -> dict:
    response = client.get(f"/api/analytics/change", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def _get_workload(headers: dict, **params) -> dict:
    response = client.get(f"/api/analytics/workload", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def test_no_multiple_revisions_when_never_edited():
    headers, owner_id = _auth()
    _create_change_request(headers)

    data = _get_change(headers, owner_id=owner_id)
    assert data["crs_with_multiple_revisions"] == 0
    assert data["avg_versions_per_cr"] == 1.0
    assert data["most_changed_fields"] == []

def test_editing_bumps_multiple_revisions_and_most_changed_fields():
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    response = client.put(
        f"/api/change-requests/{created['id']}", json={"priority": "high"}, headers=headers
    )
    assert response.status_code == 200

    data = _get_change(headers, owner_id=owner_id)
    assert data["crs_with_multiple_revisions"] == 1
    assert data["avg_versions_per_cr"] == 2.0
    field_labels = {entry["field_label"]: entry["count"] for entry in data["most_changed_fields"]}
    assert field_labels.get("Priority") == 1

def test_crs_returned_for_clarification(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, with_missing_information=True)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

    data = _get_change(headers, owner_id=owner_id)
    assert data["crs_returned_for_clarification"] == 1

def test_no_clarification_needed_when_nothing_missing(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, with_missing_information=False)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

    data = _get_change(headers, owner_id=owner_id)
    assert data["crs_returned_for_clarification"] == 0

def test_workload_crs_per_owner():
    headers_a, owner_a = _auth("Workload Owner A Phase4")
    headers_b, owner_b = _auth("Workload Owner B Phase4")
    _create_change_request(headers_a)
    _create_change_request(headers_b)
    _create_change_request(headers_b)

    data = _get_workload(headers_a, owner_id=owner_a)
    owners = {entry["owner_name"]: entry["count"] for entry in data["crs_per_owner"]}
    assert owners == {"Workload Owner A Phase4": 1}

    data_b = _get_workload(headers_b, owner_id=owner_b)
    owners_b = {entry["owner_name"]: entry["count"] for entry in data_b["crs_per_owner"]}
    assert owners_b == {"Workload Owner B Phase4": 2}

def test_workload_pending_reviews_pending_approvals_and_overdue():
    owner_headers, owner_id = _auth("Workload Requester Phase4")
    reviewer_headers, reviewer_id = _auth("Workload Reviewer Phase4")
    approver_headers, approver_id = _auth("Workload Approver Phase4")
    created = _create_change_request(owner_headers)
    cr_id = created["id"]

    assign_response = client.post(
        f"/api/change-requests/{cr_id}/assignments",
        json={"user_id": reviewer_id, "role": "reviewer"},
        headers=owner_headers,
    )
    assert assign_response.status_code == 201

    status_response = client.put(
        f"/api/change-requests/{cr_id}/status", json={"status": "analyzed"}, headers=owner_headers
    )
    assert status_response.status_code == 200
    status_response = client.put(
        f"/api/change-requests/{cr_id}/status", json={"status": "in_review"}, headers=owner_headers
    )
    assert status_response.status_code == 200

    past_due = (datetime.utcnow() - timedelta(days=3)).isoformat()
    approval_response = client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": "technical", "approver_user_id": approver_id, "due_date": past_due},
        headers=owner_headers,
    )
    assert approval_response.status_code == 201

    data = _get_workload(owner_headers, owner_id=owner_id)
    assert data["pending_reviews"] == 1
    assert data["pending_approvals"] == 1
    assert data["overdue_tasks"] == 1

def test_workload_excludes_bulk_seed_bot_accounts_from_crs_per_owner():

```

```
# loadtest1@alight.com is one of the app's own bulk-seed accounts (see
# backend/app/database/seed_bulk.py) - never created through this test
# file, so this only asserts it's absent from the breakdown when it
# exists, not that it was ever present.
headers, owner_id = _auth("Workload Real User Phase4")
_create_change_request(headers)

data = _get_workload(headers, owner_id=owner_id)
owner_names = {entry["owner_name"] for entry in data["crs_per_owner"]}
assert not any("loadtest" in name.lower() for name in owner_names)

def test_change_and_workload_endpoints_require_authentication():
    assert client.get("/api/analytics/change").status_code == 401
    assert client.get("/api/analytics/workload").status_code == 401
```

tests/test_module19_phase5.py

235 lines

```
"""Verifies Module 19 Phase 5 (Engineering Change Analytics): the new
GET /api/analytics/ai endpoint - spec section 7, deliberately called "AI
recommendation statistics" (never "AI accuracy" - see
app/services/analytics.py::AIAnalyticsMetrics for why) - against real
database rows this test file creates itself.

Following this project's established HTTP-only testing convention, every
check here goes through the real /analyze endpoint (monkeypatched
provider, controlling confidence/risk score/recommendation via the AI
response's own fields) - never a direct database write. The AI-failure
count reuses Module 19 Phase 1's own AI ANALYSIS_FAILED history hook,
verified end-to-end here through this new endpoint rather than only
through the /history endpoint Phase 1's own test file already covers.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _analysis_response(*, confidence: float, risk_score: float, decision: str) -> dict:
    risks = []
    if risk_score:
        risks = [
            {
                "category": "operational",
                "description": "A risk used only to pin the overall risk score for this test.",
                "severity": "high" if risk_score >= 50 else "low",
                "probability": 0.5,
                "score": risk_score,
                "explanation": "Deterministic test fixture risk.",
                "mitigation": "N/A - test fixture.",
            }
        ]
    return {
        "summary": "A test change used only to exercise Module 19 Phase 5's AI analytics endpoint.",
        "classification": {"category": "Configuration", "confidence": confidence, "reason": "Test fixture."},
        "requirements": [
            {"category": "functional", "description": "Test fixture requirement.", "priority": "medium"},
        ],
        "affected_components": [
            {
                "name": "Test Component",
                "type": "backend",
                "impact_level": "low",
                "reason": "Test fixture.",
                "confidence": 70,
            }
        ],
        "dependencies": [],
        "risks": risks,
        "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
        "complexity": {"level": "low", "reasoning": "Test fixture."},
        "effort": {"backend": "1 day", "frontend": "Insufficient information.", "testing": "0.5 day", "total": "1-2 developer-days"},
        "missing_information": [],
        "test_cases": [
            {
                "id": "TC-001",
                "title": "Test fixture case",
                "type": "integration",
                "priority": "medium",
                "description": "Test fixture.",
                "expected_result": "Test fixture.",
            }
        ],
        "implementation_plan": [
            {
                "task": "Test fixture task",
                "description": "Test fixture.",
                "component": "Test Component",
                "priority": "medium",
                "estimated_effort": "4h",
            }
        ],
        "recommendation": {"decision": decision, "reasoning": "Test fixture."},
    }

class FixedProvider:
    def __init__(self, response_text=None, raise_exc=None):
        self._text = response_text
        self._raise = raise_exc

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, *, confidence=0.8, risk_score=10.0, decision="approve") -> None:
    response_text = json.dumps(_analysis_response(confidence=confidence, risk_score=risk_score, decision=decision))
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider(response_text))

def _patch_failure(monkeypatch) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: _FixedProvider("this is not json at all"))

def _unique_email() -> str:
    return f"m19p5-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module19 Phase5 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
```

```

        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Module 19 Phase 5 fixture change request",
        "description": "A change request used only to exercise the AI analytics endpoint.",
        "business_objective": "N/A - test fixture.",
        "priority": "medium",
        "requested by": "Test Fixture",
        "target_system": "Test System",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _get_ai(headers: dict, **params) -> dict:
    response = client.get("/api/analytics/ai", params=params, headers=headers)
    assert response.status_code == 200, response.text
    return response.json()

def test_total_analyses_and_average_confidence(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _patch_provider(monkeypatch, confidence=0.8)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 201

    data = _get_ai(headers, owner_id=owner_id)
    assert data["total_analyses"] == 1
    assert abs(data["average_confidence"] - 80.0) < 0.01
    assert data["re_analysis_rate"] == 0.0

def test_re_analysis_rate_none_with_nothing_analyzed():
    headers, owner_id = _auth()
    _create_change_request(headers)

    data = _get_ai(headers, owner_id=owner_id)
    assert data["total_analyses"] == 0
    assert data["re_analysis_rate"] is None
    assert data["average_confidence"] is None

def test_re_analysis_rate_combines_analyzed_and_reanalyzed_cr(monkeypatch):
    headers, owner_id = _auth()
    once_analyzed = _create_change_request(headers, title="Analyzed once")
    twice_analyzed = _create_change_request(headers, title="Analyzed twice")

    _patch_provider(monkeypatch)
    assert client.post(f"/api/change-requests/{once_analyzed['id']}/analyze", headers=headers).status_code == 201
    assert client.post(f"/api/change-requests/{twice_analyzed['id']}/analyze", headers=headers).status_code == 201
    assert client.post(f"/api/change-requests/{twice_analyzed['id']}/analyze", headers=headers).status_code == 201

    data = _get_ai(headers, owner_id=owner_id)
    assert data["total_analyses"] == 3
    # 2 analyzed CRs total, 1 of them (twice analyzed) re-analyzed - 0.5.
    assert abs(data["re_analysis_rate"] - 0.5) < 0.001

def test_risk_changes_after_edits_counts_bucket_transitions(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)

    _patch_provider(monkeypatch, risk_score=10.0) # low
    assert client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).status_code == 201
    _patch_provider(monkeypatch, risk_score=90.0) # critical
    assert client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers).status_code == 201

    data = _get_ai(headers, owner_id=owner_id)
    assert data["risk_changes_after_edits"] == 1

def test_ai_recommended_approvals_counts_approve_decisions_only(monkeypatch):
    headers, owner_id = _auth()
    approved_cr = _create_change_request(headers, title="AI recommends approve")
    clarification_cr = _create_change_request(headers, title="AI requires clarification")

    _patch_provider(monkeypatch, decision="approve")
    assert client.post(f"/api/change-requests/{approved_cr['id']}/analyze", headers=headers).status_code == 201
    _patch_provider(monkeypatch, decision="requires clarification")
    assert client.post(f"/api/change-requests/{clarification_cr['id']}/analyze", headers=headers).status_code == 201

    data = _get_ai(headers, owner_id=owner_id)
    assert data["ai_recommended_approvals"] == 1

def test_ai_analysis_failures_counted(monkeypatch):
    headers, owner_id = _auth()
    created = _create_change_request(headers)
    _patch_failure(monkeypatch)
    response = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert response.status_code == 422

    data = _get_ai(headers, owner_id=owner_id)
    assert data["ai_analysis_failures"] == 1
    assert data["total_analyses"] == 0

def test_ai_endpoint_requires_authentication():
    response = client.get("/api/analytics/ai")
    assert response.status_code == 401

```

tests/test_module19_phase7.py

185 lines

```
"""Verifies Module 19 Phase 7 (Engineering Change Analytics) - the final
testing pass the spec's own section 10 asks for: calculations checked
against real database records, empty data, a larger dataset, and filters,
across all 6 analytics endpoints together. Phases 1-5 already covered each
endpoint's own logic in detail (see those files); this file's job is the
four things spec section 10 asks for explicitly, end-to-end, rather than
one more slice of any single endpoint's business rules.

This project's bulk-synthetic dataset (`app/database/seed_bulk.py`, ~3000
rows) is a standalone CLI script never run as part of `init_db()` or this
test suite - the shared pytest database never contains it. "Test large
data" here instead means a real, moderately large set of change requests
(60) created through this test file itself, independently counted in
Python and compared against what the endpoints report - large enough to
be a genuine multi-row aggregation check, not just "does it happen to work
for one row."

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

ANALYTICS_ENDPOINTS = [
    "/api/analytics/executive",
    "/api/analytics/risk",
    "/api/analytics/approval-bottlenecks",
    "/api/analytics/change",
    "/api/analytics/workload",
    "/api/analytics/ai",
]

def _unique_email() -> str:
    return f"m19p7-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module19 Phase7 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Module 19 Phase 7 fixture change request",
        "description": "A change request used only to exercise the final analytics testing pass.",
        "business objective": "N/A - test fixture.",
        "priority": "medium",
        "requested by": "Test Fixture",
        "target_system": "Test System",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def test_every_endpoint_handles_a_brand_new_user_with_zero_data():
    """Empty data (spec 10.2): a freshly-registered user who has never
    created a change request. Every endpoint must respond 200 with a real
    "nothing yet" shape - zero counts, null averages, empty lists - never
    an error, and never a stray row belonging to some other test."""
    headers, owner_id = _auth("Phase7 Empty Data User")

    for path in ANALYTICS_ENDPOINTS:
        response = client.get(path, params={"owner_id": owner_id}, headers=headers)
        assert response.status_code == 200, f"{path} failed on empty data: {response.text}"

    executive = client.get("/api/analytics/executive", params={"owner_id": owner_id}, headers=headers).json()
    assert executive["executive"]["total"] == 0
    assert executive["workflow"]["avg_time_to_analysis_hours"] is None

    risk = client.get("/api/analytics/risk", params={"owner_id": owner_id}, headers=headers).json()
    assert risk["current distribution"] == {"low": 0, "medium": 0, "high": 0, "critical": 0, "not_analyzed": 0}
    assert risk["over_time"] == []

    bottlenecks = client.get(
        "/api/analytics/approval-bottlenecks", params={"owner_id": owner_id}, headers=headers
    ).json()
    assert bottlenecks["pending_by_type"] == {}
    assert bottlenecks["pending_by_person"] == []
    assert bottlenecks["most_common_blockers"] == []

    change = client.get("/api/analytics/change", params={"owner_id": owner_id}, headers=headers).json()
    assert change["crs_with_multiple_revisions"] == 0
    assert change["avg_versions_per_cr"] is None
    assert change["most_changed_fields"] == []

    workload = client.get("/api/analytics/workload", params={"owner_id": owner_id}, headers=headers).json()
    assert workload["crs_per_owner"] == []
    assert workload["pending_reviews"] == 0

    ai = client.get("/api/analytics/ai", params={"owner_id": owner_id}, headers=headers).json()
    assert ai["total analyses"] == 0
    assert ai["re_analysis_rate"] is None
    assert ai["average_confidence"] is None

def test_large_dataset_totals_and_filters_match_an_independent_python_count():
    """Large data + filters (spec 10.3/10.4), verified against real
    database records (spec 10.1): create 60 change requests under one
    owner with a deliberately uneven priority mix, independently count
    them in Python, and confirm both the unfiltered total and a
    priority-filtered count match the endpoint exactly - not an
    approximation."""
```



```

headers, owner_id = _auth("Phase7 Large Dataset Owner")

priorities = (["high"] * 15) + (["medium"] * 25) + (["low"] * 20)
assert len(priorities) == 60
for i, priority in enumerate(priorities):
    _create_change_request(headers, title=f"Phase7 bulk fixture #{i}", priority=priority)

expected_high_count = sum(1 for p in priorities if p == "high")

unfiltered = client.get("/api/analytics/executive", params={"owner_id": owner_id}, headers=headers)
assert unfiltered.status_code == 200
assert unfiltered.json()["executive"]["total"] == len(priorities)

filtered = client.get(
    "/api/analytics/executive", params={"owner_id": owner_id, "priority": "high"}, headers=headers
)
assert filtered.status_code == 200
assert filtered.json()["executive"]["total"] == expected_high_count

# Every one of these 60 is freshly created (PENDING ANALYSIS) - all
# should land in the "Open" bucket, none anywhere else.
executive = unfiltered.json()["executive"]
assert executive["open"] == len(priorities)
assert executive["closed"] == 0
assert executive["approved"] == 0

# Workload's own per-owner breakdown must agree with the same total -
# cross-endpoint consistency, not just internal to one endpoint.
workload = client.get("/api/analytics/workload", params={"owner_id": owner_id}, headers=headers).json()
owner_counts = {entry["owner_name"]: entry["count"] for entry in workload["crs_per_owner"]}
assert owner_counts.get("Phase7 Large Dataset Owner") == len(priorities)

def test_totals_agree_with_the_plain_change_requests_list_endpoint():
    """Cross-checks analytics against the plain change-requests list
    endpoint (not just against Python's own count) - the most direct
    "verify against real database records" check available: two
    independently-implemented endpoints reading the same rows must agree.
    'ChangeRequestListItem' has no owner field to filter list by, so this
    scopes both sides the same way instead - a unique title fragment only
    this test's own fixtures use, via the list endpoint's own 'search'
    parameter. """
    headers, owner_id = _auth("Phase7 Cross-Check Owner")
    unique_marker = f"phase7-cross-check-{uuid.uuid4().hex[:8]}"
    for i in range(5):
        _create_change_request(headers, title=f"Phase7 cross-check fixture {unique_marker} #{i}")

    list_response = client.get("/api/change-requests", params={"search": unique_marker}, headers=headers)
    assert list_response.status_code == 200
    assert list_response.json()["total"] == 5

    executive = client.get("/api/analytics/executive", params={"owner_id": owner_id}, headers=headers).json()
    assert executive["executive"]["total"] == 5

def test_invalid_owner_id_returns_empty_rather_than_error():
    headers, _ = _auth("Phase7 Invalid Owner Requester")
    response = client.get("/api/analytics/executive", params={"owner_id": "999999999"}, headers=headers)
    assert response.status_code == 200
    assert response.json()["executive"]["total"] == 0

```

tests/test_module20_phase1.py

399 lines

```
"""Verifies Module 20 Phase 1 (Version-Aware Reports - engine + endpoint):

* GET /{id}/report (no `?version=`) now REFUSES to build a report when
the change request has been edited since its last analysis - spec
section 3 ("do not generate a misleading current report") - instead of
the old behavior of silently mixing today's CR fields with yesterday's
analysis. It returns 409 with the last-analyzed version named, so a
caller can offer that as an explicit historical report instead of a
dead end.

* GET /{id}/report?version=N builds a report for that specific past
version - CR fields reconstructed from that version's own snapshot,
analysis (if any) that ran against exactly that version, never mixed
with data from any other version - clearly labeled "HISTORICAL REPORT"
in the PDF itself.

* GET /{id}/report/versions Lists every version with an is_current /
has_analysis flag each, what the frontend's version picker reads from.

* The new Approval Status section reflects real Approval rows (approved,
rejected, pending), never invented data.

Following this project's established HTTP-only testing convention, all of
this is verified through the real endpoints - never by calling
report_generator.py's functions directly. Verifying the PDF's actual
CONTENT (not just that bytes came back) uses pypdf, already a project
dependency (see app/services/knowledge_chunker.py's own PDF parsing).

Run with (from backend/): pytest ../tests
"""
import io
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient
from pypdf import PdfReader

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m20p1-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module20 Phase1 Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add order-status webhook to POS",
        "description": "Emit a webhook event whenever an order's status changes on the POS.",
        "business_objective": "Let downstream systems react to order status changes in real time.",
        "priority": "medium",
        "requested_by": "Casey Alight",
        "target_system": "POS Backend",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

_OK_AI_RESPONSE = {
    "summary": "Adds a webhook emitted on order status change.",
    "classification": {"category": "Integration", "confidence": 0.8, "reason": "New outbound event."},
    "requirements": [
        {"category": "functional", "description": "Emit webhook on status change.", "priority": "high"},
    ],
    "affected_components": [
        {
            "name": "Order Service",
            "type": "backend",
            "impact_level": "low",
            "reason": "Adds an outbound call on an existing state transition.",
            "confidence": 70,
        }
    ],
    "dependencies": [],
    "risks": [],
    "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "low", "reasoning": "Single new outbound call."},
    "effort": {
        "backend": "1 day",
        "frontend": "Insufficient information.",
        "testing": "0.5 day",
        "total": "1-2 developer-days",
    },
    "missing_information": [],
    "test_cases": [
        {
            "id": "TC-001",
            "title": "Webhook fires on status change",
            "type": "integration",
            "priority": "high",
            "description": "Change an order's status and verify the webhook fires.",
            "expected_result": "Webhook payload matches the new status.",
        }
    ],
    "implementation_plan": [
        {
            "task": "Add webhook emitter",
            "description": "Emit the webhook after the status transition commits.",
            "component": "Order Service",
            "priority": "high",
            "estimated_effort": "4h",
        }
    ],
}
```

```

    "recommendation": {"decision": "approve", "reasoning": "Small, well-scoped, low risk."},
}

class _FakeProvider:
    def __init__(self, response_text=None):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _analyze(headers: dict, cr_id: int, monkeypatch, response_dict=None) -> dict:
    monkeypatch.setattr(
        analysis_engine, "get_ai_provider", lambda: _FakeProvider(json.dumps(response_dict or _OK_AI_RESPONSE))
    )
    response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
    assert response.status_code == 201
    return response.json()

def _edit(headers: dict, cr_id: int, **fields) -> dict:
    response = client.put(f"/api/change-requests/{cr_id}", json=fields, headers=headers)
    assert response.status_code == 200
    return response.json()

def _request_approval(cr_id: int, headers: dict, approver_id: int, approval_type: str = "security") -> dict:
    response = client.post(
        f"/api/change-requests/{cr_id}/approvals",
        json={"approval_type": approval_type, "approver_user_id": approver_id},
        headers=headers,
    )
    assert response.status_code == 201
    return response.json()

def _respond_to_approval(cr_id: int, approval_id: int, headers: dict, status_value: str, comment=None) -> dict:
    body = {"status": status_value}
    if comment is not None:
        body["comment"] = comment
    response = client.post(
        f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond", json=body, headers=headers
    )
    assert response.status_code == 200
    return response.json()

def _pdf_text(content: bytes) -> str:
    reader = PdfReader(io.BytesIO(content))
    raw = "\n".join(page.extract_text() or "" for page in reader.pages)
    # A narrow table column (e.g. the Approval Status table's Comment
    # column) word-wraps a longer phrase onto more than one visual line -
    # correct, professional PDF rendering - but pypdf's extraction turns
    # that in-cell wrap into a hard newline rather than the space it
    # visually reads as. Collapsing all whitespace runs to single spaces
    # means an assertion checking a multi-word phrase verifies the real
    # wording, not incidental line breaks from the layout.
    return " ".join(raw.split())

# --- Current version, up to date -----

def test_current_version_report_succeeds_and_contains_real_data(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Order Status Webhook Report Check")
    _analyze(headers, created["id"], monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    assert response.headers["content-type"] == "application/pdf"
    assert "v1-analysis-report.pdf" in response.headers["content-disposition"]

    text = _pdf_text(response.content)
    assert f"CR-{created['id']:04d}" in text
    assert "Order Status Webhook Report Check" in text
    assert "Version 1" in text
    assert "HISTORICAL REPORT" not in text
    assert "Adds a webhook emitted on order status change." in text

def test_report_requires_authentication():
    headers, _ = _auth()
    created = _create_change_request(headers)
    response = client.get(f"/api/change-requests/{created['id']}/report")
    assert response.status_code == 401

# --- Version mismatch: the core Module 20 bugfix -----

def test_editing_after_analysis_returns_409_instead_of_a_misleading_report(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)

    _edit(headers, created["id"], description="A materially different description than what was analyzed.")

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 409
    detail = response.json()["detail"]
    assert detail["cr_version"] == 2
    assert detail["last_analyzed_version"] == 1
    assert detail["can_reanalyze"] is True

def test_never_analyzed_current_version_returns_409(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 409
    detail = response.json()["detail"]
    assert detail["last_analyzed_version"] is None

# --- Explicit historical reports -----

def test_historical_report_for_the_last_analyzed_version_succeeds(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Historical Report Target")
    _analyze(headers, created["id"], monkeypatch)
    _edit(headers, created["id"], description="Edited after analysis - CR is now Version 2.")

    response = client.get(f"/api/change-requests/{created['id']}/report?version=1", headers=headers)
    assert response.status_code == 200

```

```

assert "v1-historical-report.pdf" in response.headers["content-disposition"]

text = _pdf_text(response.content)
assert "HISTORICAL REPORT - Version 1" in text
assert "Historical Report Target" in text
# The analysis that ran against Version 1 is still shown...
assert "Adds a webhook emitted on order status change." in text
# ...and the description shown is Version 1's original text...
assert "Emit a webhook event whenever an order's status changes on the POS." in text
# ...never the live Version 2 edit.
assert "Edited after analysis" not in text

def test_historical_report_for_a_never_analyzed_version_is_cr_only(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Never Analyzed Version One")
    _edit(headers, created["id"], description="Version 2 description.")
    _analyze(headers, created["id"], monkeypatch) # analyzes Version 2, not Version 1

    response = client.get(f"/api/change-requests/{created['id']}/report?version=1", headers=headers)
    assert response.status_code == 200

    text = _pdf_text(response.content)
    assert "HISTORICAL REPORT - Version 1" in text
    assert "No analysis was performed against Version 1" in text
    # Version 2's edited description must never leak into the Version 1 report.
    assert "Version 2 description." not in text

def test_requesting_a_version_that_does_not_exist_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)
    response = client.get(f"/api/change-requests/{created['id']}/report?version=99", headers=headers)
    assert response.status_code == 404

# --- Reportable-versions listing -----

def test_list_reportable_versions(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)
    _edit(headers, created["id"], description="Bumps this CR to Version 2, unanalyzed.")

    response = client.get(f"/api/change-requests/{created['id']}/report/versions", headers=headers)
    assert response.status_code == 200
    versions = {row["version_number"]: row for row in response.json()}

    assert versions[1]["has_analysis"] is True
    assert versions[1]["is_current"] is False
    assert versions[2]["has_analysis"] is False
    assert versions[2]["is_current"] is True

# --- Approval Status section -----

def test_report_shows_approved_approval(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)
    approver_headers, approver_id = _auth("Approver One")

    requested = _request_approval(created["id"], headers, approver_id, approval_type="technical")
    _respond_to_approval(created["id"], requested["id"], approver_headers, "approved")

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    text = _pdf_text(response.content)
    assert "Approval Status" in text
    assert "Approver One" in text
    assert "Approved" in text

def test_report_shows_rejected_approval_with_comment(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)
    approver_headers, approver_id = _auth("Approver Two")

    requested = _request_approval(created["id"], headers, approver_id, approval_type="security")
    _respond_to_approval(
        created["id"], requested["id"], approver_headers, "rejected", comment="Needs a rate-limit design."
    )

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    text = _pdf_text(response.content)
    assert "Rejected" in text
    assert "Needs a rate-limit design." in text

def test_report_shows_pending_approval_count(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)
    _, approver_id = _auth("Approver Three")

    _request_approval(created["id"], headers, approver_id, approval_type="qa")

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    text = _pdf_text(response.content)
    assert "Pending" in text
    assert "Approver Three" in text

def test_report_with_no_approvals_says_so(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert response.status_code == 200
    text = _pdf_text(response.content)
    assert "No approvals have been requested" in text

# --- REPORT_GENERATED audit event -----

def test_downloading_a_report_logs_a_report_generated_history_event(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers)
    _analyze(headers, created["id"], monkeypatch)

    # Module 24: analyzing a change request already logs one
    # REPORT_GENERATED event on its own (see app/api/change_requests.py::
    # analyze_change_request - a report is never a separate, manual step),
    # so this counts before/after the download rather than asserting an

```

```
# absolute total, to isolate what THIS test is actually about: that
# downloading logs its own, independent event too.
before = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
before_count = len([h for h in before if h["action"] == "report_generated"])

response = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
assert response.status_code == 200

history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
report_events = [h for h in history if h["action"] == "report_generated"]
assert len(report_events) == before_count + 1
assert all(event["new_value"] == "Version 1" for event in report_events)
```

tests/test_module21_admin.py

483 lines

```
"""Verifies Module 21 (Administration & Configuration).

Following this project's established HTTP-only testing convention, every
admin behavior is verified the same way every other endpoint in this suite
is - through the real API (registering real users via /auth/register, then
promoting/demoting/deactivating them directly through SessionLocal the
same way test_workflow_foundation.py's own test_admin_can_always_edit does)
and real HTTP calls, never by calling internal functions where an endpoint
exists to exercise instead. The one exception is required_approval_types()
itself (spec section 4's "a custom rule actually changes recommended
approvals") - that pure function is already unit-tested directly by
test_workflow_foundation.py, so this file follows the same convention to
prove a rule ADDED THROUGH THE REAL ADMIN API changes its result, without
needing a full AI-analysis pipeline just to get an Analysis row.

Covers spec section 8's test list:
- Role permissions actually blocking disallowed roles
- Admin access (every /api/admin/* endpoint requires an admin)
- Unauthorized API calls (a non-admin gets 403 from every admin endpoint)
- Approval rules (an admin-created rule changes recommended approvals)
- User deactivation (login refused, existing token stops working)
- Existing CR access (a deactivated user's own change requests/history
  stay fully visible to everyone else, unaffected)

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.database.session import SessionLocal
from app.main import app
from app.models.analysis import Analysis
from app.models.approval_rule import ApprovalRule
from app.models.enums import ApprovalType, ComplexityLevel, UserRole
from app.models.role_permission import RolePermission
from app.models.user import User
from app.services import workflow_rules

client = TestClient(app)

def _unique_email(tag: str = "m21") -> str:
    return f"{tag}-{uuid.uuid4().hex[:10]}@alight.com"

def _register(role: str = None, name: str = "Module21 Tester") -> tuple[dict, int, str]:
    """Registers a real account through the real endpoint (self-service
    signup always makes an Engineer, same as every other test file in this
    suite), then - only if a non-default role was asked for - promotes it
    directly through the database, the same way
    test_workflow_foundation.py::test_admin_can_always_edit does. Returns
    (auth headers, user id, email). The already-issued JWT stays valid
    across the role change because get_current_user looks the user up
    fresh from the database on every request - no new token needed."""
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    user_id = body["user"]["id"]
    if role is not None:
        with SessionLocal() as db:
            user = db.get(User, user_id)
            user.role = UserRole(role)
            db.commit()
    return {"Authorization": f"Bearer {body['access_token']}"}, user_id, email

def _admin() -> tuple[dict, int]:
    headers, user_id, _ = _register(role="admin")
    return headers, user_id

def _create_cr(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add loyalty points sync job",
        "description": "Nightly job to reconcile loyalty points between the POS and the loyalty service.",
        "business_objective": "Keep loyalty balances accurate across systems.",
        "priority": "medium",
        "requested by": "Casey Alight",
        "target_system": "Loyalty Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

# --- Admin access / unauthorized API calls (spec section 8) -----

def test_non_admin_gets_403_from_every_admin_endpoint():
    headers, _, _ = _register() # plain Engineer

    calls = [
        ("GET", "/api/admin/users", None),
        ("POST", "/api/admin/users", {"name": "X", "email": _unique_email(), "password": "supersecret123"}),
        ("PATCH", "/api/admin/users/1", {"is_active": False}),
        ("GET", "/api/admin/audit-log", None),
        ("GET", "/api/admin/permissions", None),
        ("PUT", "/api/admin/permissions", {"permissions": []}),
        ("GET", "/api/admin/approval-rules", None),
        (
            "POST",
            "/api/admin/approval-rules",
            {"rule_type": "risk", "match_value": "low", "approval_types": ["qa"]},
        ),
        ("PATCH", "/api/admin/approval-rules/1", {"enabled": False}),
        ("DELETE", "/api/admin/approval-rules/1", None),
        ("GET", "/api/admin/system-settings", None),
        ("PUT", "/api/admin/system-settings/cr-categories", {"categories": ["Feature"]}),
    ]
    ]
```

```

for method, path, body in calls:
    response = client.request(method, path, json=body, headers=headers)
    assert response.status_code == 403, f"{method} {path} should be 403 for a non-admin, got {response.status_code}"

def test_unauthenticated_call_gets_401_not_403():
    # No Authorization header at all - should fail at authentication, before
    # the admin-role check even runs.
    response = client.get("/api/admin/users")
    assert response.status_code == 401

def test_admin_can_reach_every_admin_endpoint():
    headers, _ = _admin()

    assert client.get("/api/admin/users", headers=headers).status_code == 200
    assert client.get("/api/admin/audit-log", headers=headers).status_code == 200
    assert client.get("/api/admin/permissions", headers=headers).status_code == 200
    assert client.get("/api/admin/approval-rules", headers=headers).status_code == 200
    assert client.get("/api/admin/system-settings", headers=headers).status_code == 200

    created = client.post(
        "/api/admin/users",
        json={"name": "New Hire", "email": _unique_email(), "password": "supersecret123", "role": "engineer"},
        headers=headers,
    )
    assert created.status_code == 201
    assert "password" not in created.json() and "password_hash" not in created.json()

def test_administration_capability_can_never_be_edited_through_permissions_endpoint():
    headers, _ = _admin()
    response = client.put(
        "/api/admin/permissions",
        json={"permissions": [{"role": "engineer", "capability": "administration", "allowed": True}]},
        headers=headers,
    )
    assert response.status_code == 400

# --- Role permissions actually blocking disallowed roles -----

def test_permissions_matrix_actually_blocks_and_unblocks_cr_creation():
    admin_headers, _ = _admin()
    requester_headers, _, _ = _register(role="requester")

    # Default permissions (seeded True for every non-Administration
    # capability) - a Requester can create a CR out of the box.
    ok = client.post(
        "/api/change-requests",
        json={
            "title": "Requester-created CR",
            "description": "A CR created while cr_create is still allowed for Requester.",
            "priority": "low",
            "requested by": "Casey Aight",
            "target_system": "POS Backend",
        },
        headers=requester_headers,
    )
    assert ok.status_code == 201

    # Admin turns cr_create OFF for Requester.
    updated = client.put(
        "/api/admin/permissions",
        json={"permissions": [{"role": "requester", "capability": "cr_create", "allowed": False}]},
        headers=admin_headers,
    )
    assert updated.status_code == 200
    row = next(p for p in updated.json() if p["role"] == "requester" and p["capability"] == "cr_create")
    assert row["allowed"] is False

    blocked = client.post(
        "/api/change-requests",
        json={
            "title": "Should be blocked",
            "description": "A CR create attempt after cr_create was turned off for Requester.",
            "priority": "low",
            "requested by": "Casey Aight",
            "target_system": "POS Backend",
        },
        headers=requester_headers,
    )
    assert blocked.status_code == 403

    # An Engineer (untouched by the change above) is unaffected.
    engineer_headers, _, _ = _register(role="engineer")
    still_ok = client.post(
        "/api/change-requests",
        json={
            "title": "Engineer-created CR",
            "description": "Engineer's cr_create permission was never touched.",
            "priority": "low",
            "requested by": "Casey Aight",
            "target_system": "POS Backend",
        },
        headers=engineer_headers,
    )
    assert still_ok.status_code == 201

    # Turn it back on - Requester can create again.
    restored = client.put(
        "/api/admin/permissions",
        json={"permissions": [{"role": "requester", "capability": "cr_create", "allowed": True}]},
        headers=admin_headers,
    )
    assert restored.status_code == 200
    restored_again = client.post(
        "/api/change-requests",
        json={
            "title": "Requester-created CR again",
            "description": "A CR created after cr_create was turned back on for Requester.",
            "priority": "low",
            "requested by": "Casey Aight",
            "target_system": "POS Backend",
        },
        headers=requester_headers,
    )
    assert restored_again.status_code == 201

def test_admin_is_never_blocked_by_the_permissions_matrix():
    admin_headers, _ = _admin()
    # Turn cr_create off for Admin too - has_permission() still bypasses
    # the matrix entirely for an admin (same superuser convention as
    # is admin() everywhere else in this app).
    client.put(
        "/api/admin/permissions",
        json={"permissions": [{"role": "admin", "capability": "cr_create", "allowed": False}]},
        headers=admin_headers,
    )

```

```

response = client.post(
    "/api/change-requests",
    json={
        "title": "Admin-created CR",
        "description": "Admins bypass the permissions matrix entirely.",
        "priority": "low",
        "requested_by": "Casey Alight",
        "target_system": "POS Backend",
    },
    headers=admin_headers,
)
assert response.status_code == 201

# --- Approval rules (spec section 4) -----

def fake_low_risk_analysis() -> Analysis:
    analysis = Analysis(
        change_request_id=0,
        summary="s",
        category="Feature",
        complexity=ComplexityLevel.LOW,
        risk_score=10, # risk_bucket(10) == "low"
        confidence_score=90,
    )
    analysis.risks = []
    return analysis

def test_admin_created_approval_rule_actually_changes_recommended_approvals():
    admin_headers, _ = _admin()
    from app.models.change_request import ChangeRequest

    cr = ChangeRequest(title="t", description="d", target_system="Reporting Dashboard")

    # Before the custom rule exists, a low-risk CR with no admin-configured
    # rules for "low" requires nothing (rules=[]) - no fallback to the old
    # hardcoded matrix once rules are being passed at all).
    before = workflow_rules.required_approval_types(cr, _fake_low_risk_analysis(), rules=[])
    assert before == set()

    created = client.post(
        "/api/admin/approval-rules",
        json={"rule_type": "risk", "match_value": "low", "approval_types": ["qa"], "enabled": True},
        headers=admin_headers,
    )
    assert created.status_code == 201
    rule_id = created.json()["id"]

    with SessionLocal() as db:
        rules = db.query(ApprovalRule).filter(ApprovalRule.id == rule_id).all()
        after = workflow_rules.required_approval_types(cr, _fake_low_risk_analysis(), rules=rules)
    assert after == {ApprovalType.QA}

    # Disabling the rule through the real PATCH endpoint takes it back out.
    disabled = client.patch(
        f"/api/admin/approval-rules/{rule_id}", json={"enabled": False}, headers=admin_headers
    )
    assert disabled.status_code == 200

    with SessionLocal() as db:
        rules = db.query(ApprovalRule).filter(ApprovalRule.id == rule_id).all()
        after_disabled = workflow_rules.required_approval_types(cr, _fake_low_risk_analysis(), rules=rules)
    assert after_disabled == set()

    # Clean up so this rule doesn't leak into other tests in this file that
    # exercise a fresh low-risk CR through the real recommended-approvals
    # endpoint.
    client.delete(f"/api/admin/approval-rules/{rule_id}", headers=admin_headers)

def test_approval_rule_create_rejects_unknown_risk_bucket_and_unknown_approval_type():
    admin_headers, _ = _admin()

    bad_bucket = client.post(
        "/api/admin/approval-rules",
        json={"rule_type": "risk", "match_value": "extreme", "approval_types": ["qa"]},
        headers=admin_headers,
    )
    assert bad_bucket.status_code == 422

    bad_type = client.post(
        "/api/admin/approval-rules",
        json={"rule_type": "category", "match_value": "checkout", "approval_types": ["not_a_real_type"]},
        headers=admin_headers,
    )
    assert bad_type.status_code == 422

# --- User deactivation (spec section 1/8) -----

def test_deactivated_user_cannot_log_in_and_loses_existing_token_access():
    admin_headers, _ = _admin()
    headers, user_id, email = register()
    password = "supersecret123"

    # Token works before deactivation.
    assert client.get("/auth/me", headers=headers).status_code == 200

    deactivate = client.patch(f"/api/admin/users/{user_id}", json={"is_active": False}, headers=admin_headers)
    assert deactivate.status_code == 200
    assert deactivate.json()["is_active"] is False

    # The already-issued token now fails on its very next request.
    still_using_old_token = client.get("/auth/me", headers=headers)
    assert still_using_old_token.status_code == 401

    # A fresh login attempt is refused with 403 (account exists, wrong
    # state) rather than a generic 401 (which would look like a bad
    # password).
    login_attempt = client.post("/auth/login", json={"email": email, "password": password})
    assert login_attempt.status_code == 403

    # Re-activating restores both.
    reactivate = client.patch(f"/api/admin/users/{user_id}", json={"is_active": True}, headers=admin_headers)
    assert reactivate.status_code == 200
    assert reactivate.json()["is_active"] is True
    login_again = client.post("/auth/login", json={"email": email, "password": password})
    assert login_again.status_code == 200

def test_admin_cannot_deactivate_or_demote_their_own_account():
    admin_headers, admin_id = _admin()

    self_deactivate = client.patch(f"/api/admin/users/{admin_id}", json={"is_active": False}, headers=admin_headers)
    assert self_deactivate.status_code == 400

    self_demote = client.patch(f"/api/admin/users/{admin_id}", json={"role": "engineer"}, headers=admin_headers)
    assert self_demote.status_code == 400

```



```

def test_wrong_password_still_gives_generic_401_not_a_deactivated_hint():
    # Guards against leaking "this account exists but is deactivated" to a
    # wrong-password guess - a deactivated account with the WRONG password
    # still gets the same generic 401 an active account would.
    admin_headers, _ = _admin()
    headers, user_id, email = _register()
    client.patch(f"/api/admin/users/{user_id}", json={"is_active": False}, headers=admin_headers)

    response = client.post("/auth/login", json={"email": email, "password": "totally-wrong-password"})
    assert response.status_code == 401

# --- Existing CR access (spec section 8) -----

def test_deactivated_users_existing_change_requests_and_history_stay_visible_to_others():
    admin_headers, _ = _admin()
    creator_headers, creator_id, _ = _register(name="Soon Deactivated")
    created = create_cr(creator_headers)
    cr_id = created["id"]

    deactivate = client.patch(f"/api/admin/users/{creator_id}", json={"is_active": False}, headers=admin_headers)
    assert deactivate.status_code == 200

    other_headers, _, _ = _register(name="Unrelated Viewer")

    detail = client.get(f"/api/change-requests/{cr_id}", headers=other_headers)
    assert detail.status_code == 200
    assert detail.json()["id"] == cr_id

    history = client.get(f"/api/change-requests/{cr_id}/history", headers=other_headers)
    assert history.status_code == 200

    listing = client.get("/api/change-requests", headers=other_headers)
    assert listing.status_code == 200
    assert any(item["id"] == cr_id for item in listing.json()["items"])

    # Admin can see it too.
    admin_view = client.get(f"/api/change-requests/{cr_id}", headers=admin_headers)
    assert admin_view.status_code == 200

# --- Roles reused / labels (spec section 2) -----

def test_manager_role_reuses_product_manager_wire_value():
    # Module 21's "Manager" role is a display label over the existing
    # product manager wire value, not a new enum member - a user created
    # with role=product manager round-trips correctly through the admin
    # API and is a real, valid role for permission checks.
    admin_headers, _ = _admin()
    created = client.post(
        "/api/admin/users",
        json={
            "name": "A Manager",
            "email": unique_email(),
            "password": "supersecret123",
            "role": "product_manager",
        },
        headers=admin_headers,
    )
    assert created.status_code == 201
    assert created.json()["role"] == "product_manager"

    with SessionLocal() as db:
        row = (
            db.query(RolePermission)
            .filter(RolePermission.role == UserRole.PRODUCT_MANAGER)
            .first()
        )
        assert row is not None # init_db seeded a permissions row for this role too

# --- System settings (spec section 5) -----

def test_cr_categories_are_editable_but_secrets_are_never_exposed():
    admin_headers, _ = _admin()

    settings = client.get("/api/admin/system-settings", headers=admin_headers).json()
    assert "cr_categories" in settings
    assert "api_key_configured" in settings["ai_provider"]
    # Never a raw key anywhere in the response.
    assert "api_key" not in settings["ai_provider"]
    assert "secret" not in str(settings).lower()

    new_categories = settings["cr_categories"] + ["Integration Test Category"]
    updated = client.put(
        "/api/admin/system-settings/cr-categories", json={"categories": new_categories}, headers=admin_headers
    )
    assert updated.status_code == 200
    assert "Integration Test Category" in updated.json()

    empty = client.put("/api/admin/system-settings/cr-categories", json={"categories": []}, headers=admin_headers)
    assert empty.status_code == 422

```

```

"""Verifies Module 22 (Final Integration, Security & Quality) spec sections
1 and 2: one real, continuous walk through the ENTIRE lifecycle this app
supports, through the real HTTP API only (this project's established
testing convention - never by calling internal functions directly), with a
version-integrity assertion at every step that produces a version-scoped
artifact:

Register/Login -> Create CR -> Edit CR -> Version created -> Audit
recorded -> AI analysis -> Impact analysis -> Repository analysis -> RAG
evidence -> Risk/security -> Test cases -> Implementation plan ->
Assignment -> Approval request -> Notification -> Review -> Approval ->
Implementation -> Validation -> Closure -> Report

This single test also happens to be the first real exercise of two Module
22 fixes made alongside it:

* the new approval-bypass gate in change_status (app/api/change_requests.py) -
the test explicitly proves a change request CANNOT be marked Approved
while an approval it requested is still Pending (409), then proves it
CAN once that approval is resolved.
* the new recommended_approvals.is_outdated flag (app/schemas/approval.py) -
the test edits the CR after its first analysis and confirms the
recommendation the endpoint returns is flagged outdated, then confirms
it clears again after a fresh analysis.

Two AI-provider-shaped things are monkeypatched, matching this project's
own established conventions for each: 'analysis_engine.get_ai_provider'
for the main analysis completion call (same pattern as
tests/test_module20_phase1.py's own 'FakeProvider'), and
'knowledge_embeddings.get_ai_provider' for embeddings (same pattern as
tests/test_module16_phase3.py's own 'FakeEmbeddingProvider', keyed off a
shared keyword instead of a real embedding model) - so the knowledge-base
evidence step is exercised against a REAL uploaded document and REAL
computed chunk embeddings, not a hand-built fake evidence row.

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

import io

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
import app.services.knowledge_embeddings as knowledge_embeddings
from app.main import app

client = TestClient(app)

# --- Shared fixtures / helpers -----

_KEYWORD = "loyaltyreconciliation" # deliberately unusual - avoids accidentally
# matching content left behind by any other test file sharing the same
# database, since every test in this suite runs against one throwaway
# sqlite file for the whole session (see tests/conftest.py).

def unique_email(tag: str = "m22") -> str:
    return f"{tag}-{uuid.uuid4().hex[:10]}@alight.com"

def _register(name: str) -> tuple[dict, int]:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201, response.text
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

_AI_RESPONSE = {
    "summary": f"Reconciles {_KEYWORD} nightly between the POS and the loyalty service.",
    "classification": {"category": "Integration", "confidence": 0.82, "reason": "New scheduled reconciliation job."},
    "requirements": [
        {
            "category": "functional",
            "description": f"Nightly job reconciles {_KEYWORD} balances between systems.",
            "priority": "high",
        },
    ],
    "affected_components": [
        {
            "name": "Loyalty Service",
            "type": "backend",
            "impact_level": "medium",
            "reason": "New scheduled reconciliation job reads and writes loyalty balances.",
            "confidence": 75,
        }
    ],
    "dependencies": [],
    "risks": [
        {
            "category": "data",
            "description": "A failed reconciliation run could double-count loyalty points.",
            "severity": "high",
            "probability": 0.3,
            "score": 60,
            "mitigation": "Make the job idempotent and log every adjustment made.",
        }
    ],
    "security_analysis": {"concerns": [], "summary": "No new external attack surface identified."},
    "complexity": {"level": "medium", "reasoning": "New scheduled job plus a reconciliation algorithm."},
    "effort": {
        "backend": "3 days",
        "frontend": "Insufficient information.",
        "testing": "1 day",
        "total": "4-5 developer-days",
    },
    "missing_information": [],
    "test_cases": [
        {
            "id": "TC-001",

```

```

        "title": f"{{ KEYWORD}} reconciliation corrects a mismatch",
        "type": "integration",
        "priority": "high",
        "description": "Introduce a loyalty balance mismatch and run the nightly job.",
        "expected_result": "Balances match across both systems after the job runs.",
    }
},
"implementation_plan": [
    {
        "task": "Add nightly reconciliation job",
        "description": "Scheduled job that reconciles loyalty balances and logs adjustments.",
        "component": "Loyalty Service",
        "priority": "high",
        "estimated_effort": "2 days",
    }
],
"recommendation": {"decision": "approve_with_conditions", "reasoning": "Sound approach; needs idempotency review."},
}

class FakeProvider:
    def __init__(self, response_text: str):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def keyword_vector(text: str) -> list[float]:
    """A minimal, deterministic stand-in for a real embedding model (same
    approach as tests/test_module16_phase3.py's own _keyword_vector) - 1.0
    in every dimension if the shared keyword is present, 0.0 otherwise.
    Real enough to exercise cosine-similarity ranking and the min score
    filter in app/services/knowledge_embeddings.py without a real AI call.

    Deliberately 8-dimensional, not 1: this suite runs every test file
    against one shared, persistent database (tests/conf/test.py), so by the
    time this test runs, many other documents' chunks already carry their
    OWN test file's single-dimension [0.0]/[1.0] fake embedding from an
    unrelated keyword. cosine_similarity() scores two vectors of
    different lengths as 0.0 - see app/services/knowledge_embeddings.py -
    so an 8-dimensional vector here can never tie with (or be beaten by) a
    leftover 1-dimensional vector from a different test file, no matter
    how many prior runs have piled up. A same-length collision was
    exactly what caused this test to intermittently retrieve some other
    test's document as "evidence" instead of the one it just uploaded."""
    hit = 1.0 if KEYWORD in text.lower() else 0.0
    return [hit] * 8

class FakeEmbeddingProvider:
    def is_configured(self) -> bool:
        return True

    def embed(self, texts, *, timeout=None):
        return [_keyword_vector(t) for t in texts]

def test_full_lifecycle_end_to_end_with_version_integrity(monkeypatch):
    monkeypatch.setattr(knowledge_embeddings, "get_ai_provider", lambda: _FakeEmbeddingProvider())

    # --- Register/Login -----
    creator_headers, creator_id = _register("E2E Creator")
    approver_headers, approver_id = _register("E2E Approver")

    login = client.post(
        "/auth/login",
        json={"email": _unique_email(), "password": "irrelevant"}, # sanity: unknown login still clean 401
    )
    assert login.status_code == 401

    # --- Create CR -----
    created = client.post(
        "/api/change-requests",
        json={
            "title": f"Nightly {{ KEYWORD}} reconciliation job",
            "description": f"Add a nightly job that reconciles {{ KEYWORD}} balances between the POS and the loyalty service.",
            "business_objective": "Keep loyalty balances accurate across systems.",
            "priority": "medium",
            "requested_by": "Casey Alight",
            "target_system": "Loyalty Service",
        },
        headers=creator_headers,
    ).json()
    cr_id = created["id"]
    assert created["status"] == "pending_analysis"
    assert created["current_version"] == 1

    # --- Upload + embed a real knowledge-base document (for RAG evidence) ---
    upload = client.post(
        "/api/knowledge/documents",
        headers=creator_headers,
        files={
            "file": (
                "loyalty-reconciliation-notes.md",
                io.BytesIO(
                    f"# Loyalty Reconciliation\n\nThe {{ KEYWORD}} process compares point balances "
                    "nightly and reports mismatches for manual review.".encode()
                ),
                "text/markdown",
            )
        },
    )
    assert upload.status_code == 201, upload.text
    document_id = upload.json()["id"]
    embed_response = client.post(f"/api/knowledge/documents/{document_id}/embed", headers=creator_headers)
    assert embed_response.status_code == 200, embed_response.text

    # --- Repository analysis (real scan of this project's own repo) -----
    scan_response = client.post("/api/repository/scan", headers=creator_headers)
    assert scan_response.status_code in (200, 201), scan_response.text

    # --- AI analysis (version 1) -----
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: FakeProvider(json.dumps(AI_RESPONSE)))
    analyze_response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=creator_headers)
    assert analyze_response.status_code == 201, analyze_response.text
    analysis_v1 = analyze_response.json()
    analysis_v1_id = analysis_v1["id"]

    # Version integrity: this analysis, and everything it produced, is
    # tagged with version 1 - the CR hasn't been edited yet.
    assert analysis_v1["change request version"] == 1
    assert analysis_v1["is outdated"] is False
    assert len(analysis_v1["requirements"]) >= 1
    assert len(analysis_v1["risks"]) >= 1
    assert len(analysis_v1["impact assessments"]) >= 7 # the 7 fixed lenses, always filled
    assert len(analysis_v1["security findings"]) >= 9 # the 9 fixed lenses, always filled
    assert len(analysis_v1["test cases"]) >= 1

```

```

assert len(analysis_v1["implementation_tasks"]) >= 1
for tc in analysis_v1["test_cases"]:
    assert tc["analysis_id"] == analysis_v1_id
for task in analysis_v1["implementation_tasks"]:
    assert task["analysis_id"] == analysis_v1_id
for req in analysis_v1["requirements"]:
    assert req["analysis_id"] == analysis_v1_id

# RAG evidence: the uploaded document should have been retrieved and
# recorded as grounding context for this exact analysis (Module 16
# Phase 4/5) - version-tagged the same way as every other artifact.
assert len(analysis_v1["knowledge_evidence"]) >= 1, "expected the uploaded document to be retrieved as evidence"
evidence = analysis_v1["knowledge_evidence"][0]
assert evidence["document_id"] == document_id
assert evidence["is_outdated"] is False

change_request = client.get(f"/api/change-requests/{cr_id}", headers=creator_headers).json()
assert change_request["status"] == "analyzed" # first analysis auto-advances from Pending Analysis
assert change_request["is_analysis_outdated"] is False

# Repository analysis: whatever the real scan/matcher found (if
# anything - matching depends on this repo's actual file names, which
# this test doesn't try to force) must be tagged with the CR's current
# version, never a mismatched one.
findings_v1 = client.get(f"/api/change-requests/{cr_id}/repository-findings", headers=creator_headers)
assert findings_v1.status_code == 200
for finding in findings_v1.json():
    assert finding["change_request_version"] == 1
    assert finding["analysis_id"] == analysis_v1_id

# Recommended approvals should exist and not be flagged outdated yet.
recommended_v1 = client.get(f"/api/change-requests/{cr_id}/approvals/recommended", headers=creator_headers).json()
assert recommended_v1, "a medium-complexity, high-severity-risk CR should recommend at least one approval type"
assert all(r["is_outdated"] is False for r in recommended_v1)

# --- Edit CR -> new version + audit -----
edited = client.put(
    f"/api/change-requests/{cr_id}",
    json={"description": change_request["description"] + " Also covers a manual reconciliation report."},
    headers=creator_headers,
)
assert edited.status_code == 200, edited.text
edited_body = edited.json()
# PUT Returns {change_request, changes, new version} - not a flat CR
# object - so the updated version lives at both of these paths.
assert edited_body["new_version"] == 2
assert edited_body["change_request"]["current_version"] == 2

versions = client.get(f"/api/change-requests/{cr_id}/versions", headers=creator_headers).json()
assert v["version_number"] for v in versions >= {1, 2}

history = client.get(f"/api/change-requests/{cr_id}/history", headers=creator_headers).json()
field_changed_events = [h for h in history if h["action"] == "field_changed"]
assert len(field_changed_events) >= 1
assert field_changed_events[0]["user_id"] == creator_id # audit correctness: real actor, not spoofable

# Module 22 fix in action: the analysis is now outdated (CR moved to
# version 2, analysis #1 still says version 1), and recommended
# approvals now say so too - this is the exact gap this module closed.
stale_cr = client.get(f"/api/change-requests/{cr_id}", headers=creator_headers).json()
assert stale_cr["is_analysis_outdated"] is True
recommended_stale = client.get(
    f"/api/change-requests/{cr_id}/approvals/recommended", headers=creator_headers
).json()
assert recommended_stale and all(r["is_outdated"] is True for r in recommended_stale)

# Downloading the CURRENT report is correctly refused while the
# analysis is outdated (Module 20's own fix, still holding).
stale_report = client.get(f"/api/change-requests/{cr_id}/report", headers=creator_headers)
assert stale_report.status_code == 409

# --- Re-analyze (version 2) -----
reanalyze_response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=creator_headers)
assert reanalyze_response.status_code == 201, reanalyze_response.text
analysis_v2 = reanalyze_response.json()
analysis_v2_id = analysis_v2["id"]
assert analysis_v2_id != analysis_v1_id
assert analysis_v2["change_request_version"] == 2
assert analysis_v2["is_outdated"] is False

fresh_cr = client.get(f"/api/change-requests/{cr_id}", headers=creator_headers).json()
assert fresh_cr["is_analysis_outdated"] is False

recommended_fresh = client.get(
    f"/api/change-requests/{cr_id}/approvals/recommended", headers=creator_headers
).json()
assert recommended_fresh and all(r["is_outdated"] is False for r in recommended_fresh)

# --- Review (Requirements + Security findings) -----
requirement_id = analysis_v2["requirements"][0]["id"]
review = client.patch(
    f"/api/change-requests/{cr_id}/requirements/{requirement_id}/review",
    json={"review_status": "confirmed"},
    headers=creator_headers,
)
assert review.status_code == 200, review.text
assert review.json()["reviewed_by"] == creator_id

security_finding_id = analysis_v2["security_findings"][0]["id"]
finding_update = client.patch(
    f"/api/change-requests/{cr_id}/security-findings/{security_finding_id}/status",
    json={"status": "acknowledged"},
    headers=creator_headers,
)
assert finding_update.status_code == 200, finding_update.text
assert finding_update.json()["status"] == "acknowledged"

reviewed_history = client.get(f"/api/change-requests/{cr_id}/history", headers=creator_headers).json()
assert any(h["action"] == "requirement_reviewed" for h in reviewed_history)

# --- Assignment -----
assignment = client.post(
    f"/api/change-requests/{cr_id}/assignments",
    json={"user_id": approver_id, "role": "reviewer"},
    headers=creator_headers,
)
assert assignment.status_code == 201, assignment.text

# --- Status: Analyzed -> In Review -> Approval Required -----
assert client.put(
    f"/api/change-requests/{cr_id}/status", json={"status": "in_review"}, headers=creator_headers
).status_code == 200
assert client.put(
    f"/api/change-requests/{cr_id}/status", json={"status": "approval_required"}, headers=creator_headers
).status_code == 200

# --- Approval request -> Notification -----
approval_request = client.post(
    f"/api/change-requests/{cr_id}/approvals",
    json={"approval_type": "technical", "approver_user_id": approver_id},
    headers=creator_headers,
)

```

```

)
assert approval_request.status_code == 201, approval_request.text
approval_id = approval_request.json()["id"]

approver_notifications = client.get("/api/notifications", headers=approver_headers).json()
assert any(
    n.get("type") == "approval_requested" and n.get("change_request_id") == cr_id
    for n in approver_notifications
), "the tagged approver should have been notified of the approval request"

# --- Approval bypass gate (Module 22 fix) - proven both ways -----
bypass_attempt = client.put(
    f"/api/change-requests/{cr_id}/status", json={"status": "approved"}, headers=creator_headers
)
assert bypass_attempt.status_code == 409, (
    "a change request must not be approvable while an approval it requested is still Pending"
)

# A non-approver cannot respond on the approver's behalf.
unauthorized_response = client.post(
    f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond",
    json={"status": "approved"},
    headers=creator_headers,
)
assert unauthorized_response.status_code == 403

# --- Approval (the real approver responds) -----
response = client.post(
    f"/api/change-requests/{cr_id}/approvals/{approval_id}/respond",
    json={"status": "approved"},
    headers=approver_headers,
)
assert response.status_code == 200, response.text

requester_notifications = client.get("/api/notifications", headers=creator_headers).json()
assert any(n.get("type") == "approved" and n.get("change_request_id") == cr_id for n in requester_notifications)

# Now that the only requested approval is resolved, the same
# transition that was blocked above succeeds.
approved = client.put(
    f"/api/change-requests/{cr_id}/status", json={"status": "approved"}, headers=creator_headers
)
assert approved.status_code == 200, approved.text
assert approved.json()["status"] == "approved"

# --- Implementation -> Validation -> Closure -----
for target in ("implementation_planned", "in_progress", "implemented", "validated", "closed"):
    step = client.put(f"/api/change-requests/{cr_id}/status", json={"status": target}, headers=creator_headers)
    assert step.status_code == 200, f"transition to {target} failed: {step.text}"
    assert step.json()["status"] == target

# An invalid transition from a terminal state is still rejected.
dead_end = client.put(
    f"/api/change-requests/{cr_id}/status", json={"status": "in_progress"}, headers=creator_headers
)
assert dead_end.status_code == 422

# --- Report (final, current version) -----
final_report = client.get(f"/api/change-requests/{cr_id}/report", headers=creator_headers)
assert final_report.status_code == 200
assert final_report.headers["content-type"] == "application/pdf"
assert len(final_report.content) > 1000 # a real, non-trivial PDF, not an empty stub

final_history = client.get(f"/api/change-requests/{cr_id}/history", headers=creator_headers).json()
assert any(h["action"] == "report generated" for h in final_history)
status_changed_events = [h for h in final_history if h["action"] == "status_changed"]
assert len(status_changed_events) >= 7 # in review, approval required, approved, +5 implementation/close steps
assert all(h["user_id"] == creator_id for h in status_changed_events) # every change made by the real actor

# --- Audit integrity: history has no write route at all -----
# app/api/change_requests.py registers GET .../history only - there is
# no PUT/PATCH/DELETE path for it to even attempt, so this confirms
# the negative directly against the running app rather than asserting
# something about code this test can't otherwise observe.
assert client.put(f"/api/change-requests/{cr_id}/history", headers=creator_headers).status_code == 405
assert client.delete(f"/api/change-requests/{cr_id}/history", headers=creator_headers).status_code == 405

```

tests/test_module24_reports.py

359 lines

```
"""Verifies Module 24 (Reports section):

* A report is never a separate, manually-generated thing - the instant an
  analysis completes for a change request's current version, that
  automatically becomes a REPORT GENERATED event and the report shows up
  on the Reports page with no extra action (confirmed with the project
  owner: "the reports should not be generated separately, once the
  analysis is complete, it should reflect in the reports section"). This
  file tests that behaviour directly - it never posts to a "generate"
  endpoint, because none exists.
* GET /api/reports, GET /api/reports/stats, GET /api/reports/{id},
  GET /api/reports/{id}/download, and
  GET /api/change-requests/{id}/reports all exist and work against real
  data - never hard-coded numbers.
* Reports are NOT a separate database table (see
  app/services/report_registry.py): the automatic event recorded when
  analysis completes and the pre-existing Analysis Dashboard "Download
  Report" button both feed the exact same event stream, and collapse into
  the SAME reports-list row when they're for the same change request +
  version - this is the core architecture claim of this module and gets
  its own test below.
* An unanalyzed change request has no report at all - never a fabricated
  or misleading one. Editing a change request after it was analyzed
  leaves its old report as historical/outdated rather than mutating it or
  silently generating a new "current" one.
* Historical reports (spec section 11) are never mixed with other
  versions' data.
* Search, filters, and permissions all work against real records.

Following this project's established HTTP-only testing convention (see
tests/test_module20_phase1.py), everything here goes through the real
endpoints - never by calling report_registry.py's functions directly.

Run with (from backend/): pytest ../tests
"""
import io
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient
from pypdf import PdfReader

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"m24-{uuid.uuid4().hex[:10]}@alight.com"

def _auth(name: str = "Module24 Reports Tester") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add loyalty points expiry job",
        "description": "Run a nightly job that expires loyalty points older than 12 months.",
        "business objective": "Keep the loyalty ledger accurate and bounded in size.",
        "priority": "medium",
        "requested by": "Casey Alight",
        "target system": "Loyalty Service",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

_OK_AI_RESPONSE = {
    "summary": "Adds a nightly job that expires old loyalty points.",
    "classification": {"category": "Infrastructure", "confidence": 0.75, "reason": "New scheduled job."},
    "requirements": [
        {"category": "functional", "description": "Expire points older than 12 months nightly.", "priority": "high"},
    ],
    "affected components": [],
    "dependencies": [],
    "risks": [],
    "security analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "low", "reasoning": "Single new scheduled job."},
    "effort": {
        "backend": "1 day",
        "frontend": "Insufficient information.",
        "testing": "0.5 day",
        "total": "1-2 developer-days",
    },
    "missing information": [],
    "test cases": [],
    "implementation plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Small, well-scoped, low risk."},
}

class FakeProvider:
    def __init__(self, response_text=None):
        self._text = response_text

    def is_configured(self) -> bool:
        return True

    def complete(self, prompt: str, *, system=None, max_tokens=1024, timeout=None):
        return self._text

def _analyze(headers: dict, cr_id: int, monkeypatch, response_dict=None) -> dict:
    """Runs the real /analyze endpoint - this is now also the only way a
```

```

    report ever comes into existence (see the module docstring above), so
    every test that needs a report just calls this, never a "generate"
    endpoint."""
monkeypatch.setattr(
    analysis_engine, "get_ai_provider", lambda: _FakeProvider(json.dumps(response_dict or _OK_AI_RESPONSE))
)
response = client.post(f"/api/change-requests/{cr_id}/analyze", headers=headers)
assert response.status_code == 201
return response.json()

def _edit(headers: dict, cr_id: int, **fields) -> dict:
    response = client.put(f"/api/change-requests/{cr_id}", json=fields, headers=headers)
    assert response.status_code == 200
    return response.json()

def _pdf_text(content: bytes) -> str:
    reader = PdfReader(io.BytesIO(content))
    raw = "\n".join(page.extract_text() or "" for page in reader.pages)
    return " ".join(raw.split())

# --- Reports appear automatically the moment analysis completes -----

def test_report_appears_automatically_after_analysis(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Report Module Basic Flow CR")

    before_stats = client.get("/api/reports/stats", headers=headers).json()

    # No "generate" call anywhere - analyzing IS what makes the report exist.
    _analyze(headers, created["id"], monkeypatch)

    after_stats = client.get("/api/reports/stats", headers=headers).json()
    assert after_stats["total_reports"] == before_stats["total_reports"] + 1
    assert after_stats["change_requests_with_reports"] == before_stats["change_requests_with_reports"] + 1

    listing = client.get("/api/reports", headers=headers).json()
    matching = [item for item in listing["items"] if item["change_request_id"] == created["id"]]
    assert len(matching) == 1
    report = matching[0]
    assert report["version"] == 1
    assert report["has_analysis"] is True
    assert report["is_current_version"] is True
    assert report["is_historical_pull"] is False

    detail = client.get(f"/api/reports/{report['report_id']}", headers=headers)
    assert detail.status_code == 200
    assert detail.json()["change_request_title"] == "Report Module Basic Flow CR"

    download = client.get(f"/api/reports/{report['report_id']}/download", headers=headers)
    assert download.status_code == 200
    assert download.headers["content-type"] == "application/pdf"
    text = _pdf_text(download.content)
    assert "Report Module Basic Flow CR" in text
    assert "Adds a nightly job that expires old loyalty points." in text

def test_reports_for_change_request_endpoint(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Per-CR Reports Listing")
    _analyze(headers, created["id"], monkeypatch)

    response = client.get(f"/api/change-requests/{created['id']}/reports", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert body["total"] == 1
    assert body["items"][0]["change_request_id"] == created["id"]

def test_reanalyzing_the_same_version_does_not_duplicate_the_report(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Re-run Analysis Without Editing CR")
    _analyze(headers, created["id"], monkeypatch)
    _analyze(headers, created["id"], monkeypatch) # re-analyzed, but still version 1

    response = client.get(f"/api/change-requests/{created['id']}/reports", headers=headers)
    body = response.json()
    assert body["total"] == 1
    assert body["items"][0]["pull_count"] == 2

# --- No report exists before analysis; edits never mutate an old report -----

def test_unanalyzed_change_request_has_no_report(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Never Analyzed CR")

    response = client.get(f"/api/change-requests/{created['id']}/reports", headers=headers)
    assert response.status_code == 200
    assert response.json()["total"] == 0

def test_editing_after_analysis_leaves_old_report_outdated_with_no_new_one(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Edited After Analysis CR")
    _analyze(headers, created["id"], monkeypatch)
    _edit(headers, created["id"], description="A materially different description than before, changing scope.")

    response = client.get(f"/api/change-requests/{created['id']}/reports", headers=headers)
    body = response.json()
    # Editing alone never generates a report for the new version - only a
    # completed analysis does - so the CR still has exactly the one report,
    # now flagged as no longer matching the CR's current version.
    assert body["total"] == 1
    assert body["items"][0]["version"] == 1
    assert body["items"][0]["is_current_version"] is False

# --- Historical reports (spec section 11) -----

def test_historical_report_never_mixes_versions(monkeypatch):
    headers, _ = auth()
    created = _create_change_request(headers, title="Historical Report CR")
    _analyze(headers, created["id"], monkeypatch)
    _edit(headers, created["id"], description="A materially different description, now on version two.")
    _analyze(headers, created["id"], monkeypatch)

    listing = client.get(f"/api/change-requests/{created['id']}/reports", headers=headers).json()
    by_version = {item["version"]: item for item in listing["items"]}
    assert set(by_version) == {1, 2}

    report_v1 = by_version[1]
    assert report_v1["is_current_version"] is False

    report_v2 = by_version[2]
    assert report_v2["is_current_version"] is True

```



```

# Two distinct rows - never collapsed into one just because they're the
# same change request.
assert report_v1["report_id"] != report_v2["report_id"]

download_v1 = client.get(f"/api/reports/{report_v1['report_id']}/download", headers=headers)
text_v1 = pdf_text(download_v1.content)
assert "HISTORICAL REPORT" in text_v1

# --- No duplicate system: the automatic analysis-time event and the ---
# --- long-standing Analysis Dashboard download converge on the same row -----

def test_auto_generated_event_and_dashboard_download_share_one_row(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Shared Audit Trail CR")

    # Analyzing is itself the only "generate" step there is - already
    # records one REPORT GENERATED event.
    _analyze(headers, created["id"], monkeypatch)

    listing = client.get("/api/reports", headers=headers).json()
    matching = [item for item in listing["items"] if item["change_request_id"] == created["id"]]
    assert len(matching) == 1
    assert matching[0]["pull_count"] == 1

    # ...pulled again via the long-standing Analysis Dashboard "Download
    # Report" button - same (change request, version) pair, so it's still
    # exactly one row, now with pull count 2, not a second report entry.
    old_endpoint = client.get(f"/api/change-requests/{created['id']}/report", headers=headers)
    assert old_endpoint.status_code == 200

    listing_after = client.get("/api/reports", headers=headers).json()
    matching_after = [item for item in listing_after["items"] if item["change_request_id"] == created["id"]]
    assert len(matching_after) == 1
    assert matching_after[0]["pull_count"] == 2

# --- Search / filters -----

def test_search_and_filters(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Searchable Widget Rollout CR")
    _analyze(headers, created["id"], monkeypatch)

    by_title = client.get("/api/reports", params={"search": "Searchable Widget"}, headers=headers).json()
    assert any(item["change_request_id"] == created["id"] for item in by_title["items"])

    by_cr_code = client.get(
        "/api/reports", params={"search": f"CR-{created['id']:04d}"}, headers=headers
    ).json()
    assert any(item["change_request_id"] == created["id"] for item in by_cr_code["items"])

    by_version = client.get("/api/reports", params={"version": 1}, headers=headers).json()
    assert all(item["version"] == 1 for item in by_version["items"])

    by_status = client.get("/api/reports", params={"status": "analyzed"}, headers=headers).json()
    assert all(item["cr_status"] == "analyzed" for item in by_status["items"])

    no_match = client.get(
        "/api/reports", params={"search": "definitely-not-a-real-change-request-xyz"}, headers=headers
    ).json()
    assert no_match["total"] == 0

# --- Permissions -----

def test_reports_endpoints_require_authentication(monkeypatch):
    headers, _ = _auth()
    created = _create_change_request(headers, title="Auth Required CR")
    _analyze(headers, created["id"], monkeypatch)
    listing = client.get("/api/reports", headers=headers).json()
    report_id = next(item["report_id"] for item in listing["items"] if item["change_request_id"] == created["id"])

    assert client.get("/api/reports").status_code == 401
    assert client.get("/api/reports/stats").status_code == 401
    assert client.get(f"/api/reports/{report_id}").status_code == 401
    assert client.get(f"/api/reports/{report_id}/download").status_code == 401
    assert client.get(f"/api/change-requests/{created['id']}/reports").status_code == 401

def test_unknown_report_id_is_404(monkeypatch):
    headers, _ = _auth()
    assert client.get("/api/reports/999999999", headers=headers).status_code == 404
    assert client.get("/api/reports/999999999/download", headers=headers).status_code == 404

# --- Empty state -----

def test_empty_reports_list_is_not_an_error(monkeypatch):
    headers, _ = _auth()
    # A brand-new account with no change requests of its own yet still gets
    # a clean, valid (possibly non-empty, since other tests share the same
    # database - see tests/conftest.py) response, never an error.
    response = client.get("/api/reports", headers=headers)
    assert response.status_code == 200
    body = response.json()
    assert isinstance(body["items"], list)
    assert body["total"] == len(body["items"])

```


tests/test_seed_bulk.py

49 lines

```
"""Runs the bulk synthetic-data generator at a small scale as part of the
normal test suite, so a broken generator gets caught by `pytest ../tests`
before anyone runs it for real at full (thousands-of-rows) scale.

Run with (from backend/):  pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.database.seed_bulk import seed_bulk_data
from app.main import app

client = TestClient(app)


def _register_user() -> str:
    email = f"seedcheck-{uuid.uuid4().hex[:10]}@example.com"
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Seed Check", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    return response.json()["access_token"]


def test_seed_bulk_runs_and_dashboard_still_works():
    # Small scale on purpose - this proves the generator's logic (FKs, enum
    # values, chunked commits) is correct without slowing down the normal
    # test run. seed_bulk.py defaults to 3000+ when run directly for real
    # stress-testing, and skips itself if it already ran once before.
    seed_bulk_data(count=15)

    token = _register_user()
    response = client.get("/dashboard/summary", headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 200
    body = response.json()
    # >=15 rather than ==15 because other tests/seeding may add rows too.
    assert body["metrics"]["total_change_requests"] >= 15
```

tests/test_workflow_approvals.py

465 lines

```
"""Verifies Module 12 Phase 4 (Enterprise Workflow - Approvals):

* POST /api/change-requests/{id}/approvals - tag a real person for a
  specific kind of sign-off, permission-gated, duplicate-pending
  rejected, recorded on the audit history
* POST .../approvals/{approval_id}/respond - only the tagged approver
  can respond, a comment is required to reject/request changes, an
  already-resolved approval can't be responded to again
* DELETE .../approvals/{approval_id} - withdraw a still-pending request
  (soft-cancelled, not deleted), permission-gated
* GET .../approvals/recommended - the AI-derived recommendation only
  (never auto-creates an Approval row)
* editing a CR after an approval was requested against it marks that
  approval outdated (is_outdated + an APPROVAL_INVALIDATED history event)

Run with (from backend/): pytest ../tests
"""

import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def unique_email() -> str:
    return f"wfp-{uuid.uuid4().hex[:10]}@example.com"

def auth() -> tuple[dict, int]:
    email = unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Approval Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP login to mobile app",
        "description": "Allow customers to log in using a one-time passcode sent to their registered number.",
        "priority": "medium",
        "requested_by": "Morgan Lee",
        "target_system": "Mobile App",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def request_approval(cr_id: int, headers: dict, approver_id: int, approval_type: str = "security", comment=None):
    body = {"approval_type": approval_type, "approver_user_id": approver_id}
    if comment is not None:
        body["comment"] = comment
    return client.post(f"/api/change-requests/{cr_id}/approvals", json=body, headers=headers)

class FakeProvider:
    """Same stand-in used in tests/test_change_request_editing.py - returns
    canned text instead of calling the real AI provider."""
    def __init__(self, response_text=None, configured=True, raise_exc=None):
        self._text = response_text
        self._configured = configured
        self._raise = raise_exc

    def is_configured(self) -> bool:
        return self._configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

# A minimal, valid AIAnalysisResult - only the approvals behavior is under
# test here, not the analysis content itself. No risks -> risk score falls
# to the 5.0 floor -> the low-risk baseline (Technical) only, plus whatever
# keyword matches the CR's own title/target_system add on top.
_MINIMAL_ANALYSIS_RESPONSE = {
    "summary": "Adds OTP-based login to the mobile app.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.8, "reason": "New auth flow."},
    "requirements": [],
    "affected_components": [],
    "dependencies": [],
    "risks": [],
    "security_analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "medium", "reasoning": "New auth flow, moderate scope."},
    "effort": {"backend": "2 days", "frontend": "2 days", "testing": "1 day", "total": "5 developer-days"},
    "missing_information": [],
    "test_cases": [],
    "implementation_plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Well-scoped."},
}

# --- Requesting an approval -----

def test_request_approval_requires_authentication():
    response = client.post("/api/change-requests/1/approvals", json={"approval_type": "security", "approver_user_id": 1})
    assert response.status_code == 401
```

```

def test_request_approval_404_for_unknown_change_request():
    headers, _ = _auth()
    _, approver_id = _auth()
    response = _request_approval(999999999, headers, approver_id)
    assert response.status_code == 404

def test_creator_can_request_approval_and_it_appears_in_list_and_history():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, approver_id = _auth()

    response = _request_approval(created["id"], headers, approver_id, approval_type="security", comment="Please check rate limiting.")
    assert response.status_code == 201
    body = response.json()
    assert body["approval_type"] == "security"
    assert body["approval_type_label"] == "Security"
    assert body["status"] == "pending"
    assert body["status_label"] == "Pending"
    assert body["approver_id"] == approver_id
    assert body["cr_version"] == 1
    assert body["is_outdated"] is False

    listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=headers).json()
    assert len(listing) == 1
    assert listing[0]["id"] == body["id"]

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    requested_events = [h for h in history if h["action"] == "approval_requested"]
    assert len(requested_events) == 1
    assert "Security approval requested" in requested_events[0]["new_value"]
    assert requested_events[0]["reason"] == "Please check rate limiting."

def test_request_approval_for_unknown_approver_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = _request_approval(created["id"], headers, 999999999)
    assert response.status_code == 404

def test_duplicate_pending_approval_request_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, approver_id = _auth()

    first = _request_approval(created["id"], headers, approver_id, approval_type="qa")
    assert first.status_code == 201

    second = _request_approval(created["id"], headers, approver_id, approval_type="qa")
    assert second.status_code == 409

def test_unrelated_user_cannot_request_approval():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _, approver_id = _auth()

    response = _request_approval(created["id"], other_headers, approver_id)
    assert response.status_code == 403

# --- Responding to an approval -----

def test_approver_can_approve_and_it_appears_in_history():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id, approval_type="technical").json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "approved"
    assert body["status_label"] == "Approved"
    assert body["responded_at"] is not None

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    approved_events = [h for h in history if h["action"] == "approved"]
    assert len(approved_events) == 1
    assert approved_events[0]["new_value"] == "Technical - Approved"

def test_approver_can_reject_with_reason():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id, approval_type="security").json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "rejected", "comment": "The rate limiting design needs more detail."},
        headers=approver_headers,
    )
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "rejected"
    assert body["comment"] == "The rate limiting design needs more detail."

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    rejected_events = [h for h in history if h["action"] == "rejected"]
    assert rejected_events[0]["reason"] == "The rate limiting design needs more detail."

def test_rejecting_without_reason_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "rejected"},
        headers=approver_headers,
    )
    assert response.status_code == 422

def test_requesting_changes_without_reason_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)

```

```

approver_headers, approver_id = _auth()

requested = _request_approval(created["id"], headers, approver_id).json()

response = client.post(
    f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
    json={"status": "changes_requested"},
    headers=approver_headers,
)
assert response.status_code == 422

def test_someone_other_than_the_approver_cannot_respond():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, approver_id = _auth()
    outsider_headers, _ = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()

    # Not even the CR's own creator (who requested the approval) can
    # respond to it on the approver's behalf.
    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=headers,
    )
    assert response.status_code == 403

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=outsider_headers,
    )
    assert response.status_code == 403

def test_responding_to_already_resolved_approval_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()
    first = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )
    assert first.status_code == 200

    second = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "rejected", "comment": "Changed my mind."},
        headers=approver_headers,
    )
    assert second.status_code == 409

# --- Cancelling a pending request -----

def test_creator_can_cancel_pending_approval():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()

    response = client.delete(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}", headers=headers
    )
    assert response.status_code == 204

    listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=headers).json()
    assert listing[0]["status"] == "cancelled"

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert any(h["action"] == "approval_cancelled" for h in history)

def test_cancel_already_resolved_approval_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()
    client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )

    response = client.delete(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}", headers=headers
    )
    assert response.status_code == 409

def test_unrelated_user_cannot_cancel_approval():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, approver_id = _auth()
    outsider_headers, _ = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()

    response = client.delete(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}", headers=outsider_headers
    )
    assert response.status_code == 403

def test_cancel_unknown_approval_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.delete(f"/api/change-requests/{created['id']}/approvals/999999999", headers=headers)
    assert response.status_code == 404

# --- Recommended approval types (AI recommends, humans decide) -----

def test_recommended_approval_types_empty_before_analysis():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.get(f"/api/change-requests/{created['id']}/approvals/recommended", headers=headers)
    assert response.status_code == 200
    assert response.json() == []

```

```

def test recommended_approval_types_reflects_ai_analysis(monkeypatch):
    headers, _ = auth()
    created = create_change_request(headers)
    _patch_provider(monkeypatch, FakeProvider(json.dumps(MINIMAL_ANALYSIS_RESPONSE)))
    analyze = client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)
    assert analyze.status_code == 201

    response = client.get(f"/api/change-requests/{created['id']}/approvals/recommended", headers=headers)
    assert response.status_code == 200
    recommended = {r["approval_type"] for r in response.json()}
    # No risks -> risk score floors at 5.0 -> low-risk baseline (Technical).
    # Target system/title is "Mobile App", one of the customer-facing
    # keywords -> Product is added on top.
    assert recommended == {"technical", "product"}

    # This is a recommendation only - it never creates an Approval row.
    listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=headers).json()
    assert listing == []

# --- Approval invalidation on edit -----

def test editing_change_request_after_approval_requested_marks_it_outdated():
    headers, _ = auth()
    created = create_change_request(headers)
    _, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id, approval_type="dba").json()
    assert requested["cr_version"] == 1
    assert requested["is_outdated"] is False

    edit = client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )
    assert edit.status_code == 200
    assert edit.json()["change_request"]["current_version"] == 2

    listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=headers).json()
    assert listing[0]["is_outdated"] is True
    assert listing[0]["cr_version"] == 1 # unchanged - it was requested against version 1

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert any(h["action"] == "approval_invalidated" for h in history)

def test resolved_approval_is_never_reported_as_outdated():
    headers, _ = auth()
    created = create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = _request_approval(created["id"], headers, approver_id).json()
    client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )

    client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )

    listing = client.get(f"/api/change-requests/{created['id']}/approvals", headers=headers).json()
    assert listing[0]["status"] == "approved"
    assert listing[0]["is_outdated"] is False

# --- List endpoint guard -----

def test_list_approvals_requires_authentication():
    response = client.get("/api/change-requests/1/approvals")
    assert response.status_code == 401

```

tests/test_workflow_comments_notifications.py

582 lines

```
"""Verifies Module 12 Phase 5 (Enterprise Workflow - Comments, Mentions,
Notifications):

* GET/POST .../comments - a flat two-level discussion thread (one level
  of replies only), open to any authenticated user (unlike editing/
  status/approvals, which stay Owner/Technical Lead/Admin-gated)
* PUT .../comments/{id}/resolve - the comment's author or whoever manages
  the CR can mark it resolved/reopened; DELETE - author or admin only
* @mentions parsed from the comment body (app/services/mentions.py)
  notify the mentioned user and record a MENTIONED history event;
  everyone else with a stake in the CR (its creator + assignees) gets a
  plain COMMENT_ADDED notification instead - never both for the same
  comment
* GET/PUT /api/notifications - a user's own inbox, scoped so nobody can
  read or mark another user's notifications
* every other Module 12 Phase 3/4 action that should tell someone
  something (assigned, approval requested/responded/reminded, status
  changed, analysis/approval invalidated by an edit) actually creates a
  Notification row

Run with (from backend/): pytest ../tests
"""
import json
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

import app.services.analysis_engine as analysis_engine
from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"wfcn-{uuid.uuid4().hex[:10]}@example.com"

def _register(name: str) -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": name, "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _auth() -> tuple[dict, int]:
    return _register("Notif Tester")

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP login to mobile app",
        "description": "Allow customers to log in using a one-time passcode sent to their registered number.",
        "priority": "medium",
        "requested by": "Morgan Lee",
        "target_system": "Mobile App",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _comment(cr_id: int, headers: dict, body: str, parent_id=None):
    payload = {"body": body}
    if parent_id is not None:
        payload["parent_id"] = parent_id
    return client.post(f"/api/change-requests/{cr_id}/comments", json=payload, headers=headers)

def _notifications(headers: dict, **params):
    return client.get("/api/notifications", params=params, headers=headers).json()

class FakeProvider:
    """Same stand-in used in tests/test_change_request_editing.py."""

    def __init__(self, response_text=None, configured=True, raise_exc=None):
        self._text = response_text
        self._configured = configured
        self._raise = raise_exc

    def is_configured(self) -> bool:
        return self._configured

    def complete(self, prompt, *, system=None, max_tokens=1024, timeout=None):
        if self._raise is not None:
            raise self._raise
        return self._text

def _patch_provider(monkeypatch, provider) -> None:
    monkeypatch.setattr(analysis_engine, "get_ai_provider", lambda: provider)

_MINIMAL_ANALYSIS_RESPONSE = {
    "summary": "Adds OTP-based login to the mobile app.",
    "classification": {"category": "Feature Enhancement", "confidence": 0.8, "reason": "New auth flow."},
    "requirements": [],
    "affected components": [],
    "dependencies": [],
    "risks": [],
    "security analysis": {"concerns": [], "summary": "No new attack surface identified."},
    "complexity": {"level": "medium", "reasoning": "New auth flow, moderate scope."},
    "effort": {"backend": "2 days", "frontend": "2 days", "testing": "1 day", "total": "5 developer-days"},
    "missing information": [],
    "test cases": [],
    "implementation plan": [],
    "recommendation": {"decision": "approve", "reasoning": "Well-scoped."},
}
```

```

# --- Comments: basic CRUD -----

def test_create_comment_requires_authentication():
    response = client.post("/api/change-requests/1/comments", json={"body": "Looks good."})
    assert response.status_code == 401

def test_create_comment_404_for_unknown_change_request():
    headers, _ = _auth()
    response = _comment(999999999, headers, "Looks good.")
    assert response.status_code == 404

def test_any_authenticated_user_can_comment_and_it_appears_in_list_and_history():
    headers, _ = _auth()
    created = _create_change_request(headers)
    outsider_headers, _ = _auth() # not the creator, not assigned - comments aren't owner-gated

    response = _comment(created["id"], outsider_headers, "Has anyone checked the rate limiting?")
    assert response.status_code == 201
    body = response.json()
    assert body["body"] == "Has anyone checked the rate limiting?"
    assert body["resolved"] is False
    assert body["parent_id"] is None

    listing = client.get(f"/api/change-requests/{created['id']}/comments", headers=headers).json()
    assert len(listing) == 1
    assert listing[0]["id"] == body["id"]

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    comment_events = [h for h in history if h["action"] == "comment_added"]
    assert len(comment_events) == 1
    assert comment_events[0]["new_value"] == "Has anyone checked the rate limiting?"

def test_reply_to_comment_works():
    headers, _ = _auth()
    created = _create_change_request(headers)
    top = _comment(created["id"], headers, "Has anyone checked the rate limiting?").json()

    reply = _comment(created["id"], headers, "Yes, it's in the design doc.", parent_id=top["id"])
    assert reply.status_code == 201
    assert reply.json()["parent_id"] == top["id"]

    listing = client.get(f"/api/change-requests/{created['id']}/comments", headers=headers).json()
    assert len(listing) == 2

def test_reply_to_a_reply_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    top = _comment(created["id"], headers, "Question one.").json()
    reply = _comment(created["id"], headers, "Answer.", parent_id=top["id"]).json()

    second_reply = _comment(created["id"], headers, "Follow-up.", parent_id=reply["id"])
    assert second_reply.status_code == 422

def test_reply_to_unknown_comment_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = _comment(created["id"], headers, "Reply to nothing.", parent_id=999999999)
    assert response.status_code == 404

def test_resolve_comment_by_author():
    headers, _ = _auth()
    created = _create_change_request(headers)
    outsider_headers, _ = _auth()
    comment = _comment(created["id"], outsider_headers, "Needs clarification.").json()

    response = client.put(
        f"/api/change-requests/{created['id']}/comments/{comment['id']}/resolve",
        json={"resolved": True},
        headers=outsider_headers,
    )
    assert response.status_code == 200
    assert response.json()["resolved"] is True

def test_resolve_comment_by_cr_owner():
    headers, _ = _auth()
    created = _create_change_request(headers)
    outsider_headers, _ = _auth()
    comment = _comment(created["id"], outsider_headers, "Needs clarification.").json()

    # The CR's own creator (who didn't write the comment) can still resolve it.
    response = client.put(
        f"/api/change-requests/{created['id']}/comments/{comment['id']}/resolve",
        json={"resolved": True},
        headers=headers,
    )
    assert response.status_code == 200

def test_resolve_comment_unrelated_user_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    author_headers, _ = _auth()
    comment = _comment(created["id"], author_headers, "Needs clarification.").json()
    unrelated_headers, _ = _auth()

    response = client.put(
        f"/api/change-requests/{created['id']}/comments/{comment['id']}/resolve",
        json={"resolved": True},
        headers=unrelated_headers,
    )
    assert response.status_code == 403

def test_delete_own_comment():
    headers, _ = _auth()
    created = _create_change_request(headers)
    comment = _comment(created["id"], headers, "Oops, wrong thread.").json()

    response = client.delete(f"/api/change-requests/{created['id']}/comments/{comment['id']}", headers=headers)
    assert response.status_code == 204

    listing = client.get(f"/api/change-requests/{created['id']}/comments", headers=headers).json()
    assert listing == []

def test_delete_someone_elses_comment_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    author_headers, _ = _auth()
    comment = _comment(created["id"], author_headers, "My comment.").json()

    response = client.delete(f"/api/change-requests/{created['id']}/comments/{comment['id']}", headers=headers)

```

```

assert response.status_code == 403

def test_delete_unknown_comment_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.delete(f"/api/change-requests/{created['id']}/comments/999999999", headers=headers)
    assert response.status_code == 404

# --- Mentions & notification dedup -----

def test_mentioning_a_user_creates_mentioned_notification_and_history_event():
    headers, _ = _auth()
    created = _create_change_request(headers)
    mentioned_headers, mentioned_id = _register("Priya Shah")

    response = _comment(created["id"], headers, "@Priya Shah can you take a look at this?")
    assert response.status_code == 201
    assert response.json()["mentioned_user_ids"] == [mentioned_id]

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    mentioned_events = [h for h in history if h["action"] == "mentioned"]
    assert len(mentioned_events) == 1
    assert "Priya Shah" in mentioned_events[0]["new_value"]

    notifications = _notifications(mentioned_headers)
    mention_notifs = [n for n in notifications if n["type"] == "mentioned"]
    assert len(mention_notifs) == 1
    assert mention_notifs[0]["change_request_id"] == created["id"]
    assert mention_notifs[0]["is_read"] is False

def test_commenting_notifies_creator_but_not_the_comments_own_author():
    headers, creator_id = _auth()
    created = _create_change_request(headers)
    commenter_headers, _ = _auth()

    _comment(created["id"], commenter_headers, "Just a general comment, no mentions.")

    creator_notifs = _notifications(headers)
    assert any(n["type"] == "comment_added" for n in creator_notifs)

    # The commenter themselves never gets notified about their own comment.
    commenter_notifs = _notifications(commenter_headers)
    assert commenter_notifs == []

def test_mentioned_user_does_not_also_get_a_plain_comment_notification():
    headers, _ = _auth()
    created = _create_change_request(headers)
    mentioned_headers, mentioned_id = _register("Devon Cole")
    # Also assign this same person, so without the dedup rule they'd
    # qualify for both a COMMENT_ADDED (as an assignee) and a MENTIONED
    # notification for the same comment.
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": mentioned_id, "role": "reviewer"},
        headers=headers,
    )

    _comment(created["id"], headers, "@Devon Cole please review this.")

    notifications = _notifications(mentioned_headers)
    types = [n["type"] for n in notifications if n["change_request_id"] == created["id"]]
    # The ASSIGNED notification from being assigned above is expected, plus
    # exactly one MENTIONED for the comment - never a COMMENT_ADDED too.
    assert types.count("mentioned") == 1
    assert types.count("comment_added") == 0

# --- Notifications API -----

def test_list_notifications_requires_authentication():
    response = client.get("/api/notifications")
    assert response.status_code == 401

def test_notifications_are_scoped_to_the_current_user():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _comment(created["id"], other_headers, "A comment from someone else.")

    # The creator got a notification; the commenter (who has none of their
    # own) must not see the creator's.
    assert len(_notifications(headers)) >= 1
    assert _notifications(other_headers) == []

def test_mark_notification_read():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _comment(created["id"], other_headers, "Ping.")

    notif = _notifications(headers)[0]
    assert notif["is_read"] is False

    response = client.put(f"/api/notifications/{notif['id']}/read", headers=headers)
    assert response.status_code == 200
    assert response.json()["is_read"] is True

    unread = _notifications(headers, unread_only=True)
    assert unread == []

def test_mark_all_read():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _comment(created["id"], other_headers, "One.")
    _comment(created["id"], other_headers, "Two.")

    response = client.put("/api/notifications/read-all", headers=headers)
    assert response.status_code == 200
    assert response.json()["updated"] >= 2

    assert _notifications(headers, unread_only=True) == []

def test_unread_count():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _comment(created["id"], other_headers, "Ping.")

    response = client.get("/api/notifications/unread-count", headers=headers)

```



```

assert response.status_code == 200
assert response.json()["count"] >= 1

def test_cannot_mark_someone_elses_notification_read():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()
    _comment(created["id"], other_headers, "Ping.")

    notif = _notifications(headers)[0]
    response = client.put(f"/api/notifications/{notif['id']}/read", headers=other_headers)
    assert response.status_code == 404

# --- Notification wiring into Phase 3/4 actions -----

def test_assigning_a_user_creates_assigned_notification():
    headers, _ = _auth()
    created = _create_change_request(headers)
    assignee_headers, assignee_id = _auth()

    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "technical_lead"},
        headers=headers,
    )

    notifs = _notifications(assignee_headers)
    assert any(n["type"] == "assigned" for n in notifs)

def test_requesting_approval_creates_notification_for_approver():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "security", "approver_user_id": approver_id},
        headers=headers,
    )

    notifs = _notifications(approver_headers)
    assert any(n["type"] == "approval_requested" for n in notifs)

def test_responding_to_approval_notifies_requester():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

    requested = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "security", "approver_user_id": approver_id},
        headers=headers,
    ).json()

    client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
        json={"status": "approved"},
        headers=approver_headers,
    )

    notifs = _notifications(headers)
    assert any(n["type"] == "approved" for n in notifs)

def test_changing_status_notifies_assignees_and_creator():
    headers, _ = _auth()
    created = _create_change_request(headers)
    assignee_headers, assignee_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": assignee_id, "role": "reviewer"},
        headers=headers,
    )

    client.put(f"/api/change-requests/{created['id']}/status", json={"status": "analyzed"}, headers=headers)

    notifs = _notifications(assignee_headers)
    assert any(n["type"] == "status_changed" for n in notifs)
    # The person who made the change doesn't notify themselves.
    creator_status_notifs = [n for n in _notifications(headers) if n["type"] == "status_changed"]
    assert creator_status_notifs == []

def test_reminder_creates_notification_and_history_event():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()
    requested = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "qa", "approver_user_id": approver_id},
        headers=headers,
    ).json()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/remind", headers=headers
    )
    assert response.status_code == 204

    notifs = _notifications(approver_headers)
    assert any(n["type"] == "reminder" for n in notifs)

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assert any(h["action"] == "reminder_sent" for h in history)

def test_reminder_requires_permission():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()
    requested = client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "qa", "approver_user_id": approver_id},
        headers=headers,
    ).json()
    outsider_headers, _ = _auth()

    response = client.post(
        f"/api/change-requests/{created['id']}/approvals/{requested['id']}/remind", headers=outsider_headers
    )
    assert response.status_code == 403

def test_reminder_only_for_pending_approval():
    headers, _ = _auth()
    created = _create_change_request(headers)
    approver_headers, approver_id = _auth()

```

```

requested = client.post(
    f"/api/change-requests/{created['id']}/approvals",
    json={"approval_type": "qa", "approver_user_id": approver_id},
    headers=headers,
).json()
client.post(
    f"/api/change-requests/{created['id']}/approvals/{requested['id']}/respond",
    json={"status": "approved"},
    headers=approver_headers,
)

response = client.post(
    f"/api/change-requests/{created['id']}/approvals/{requested['id']}/remind", headers=headers
)
assert response.status_code == 409

def test_editing_after_analysis_notifies_creator_of_outdated_analysis(monkeypatch):
    headers, _ = _auth()
    created = create_change_request(headers)
    patch_provider(monkeypatch, _FakeProvider(json.dumps(MINIMAL_ANALYSIS_RESPONSE)))
    client.post(f"/api/change-requests/{created['id']}/analyze", headers=headers)

    # Reviewer is deliberately NOT enough to edit a CR (workflow rules.
    # can edit change request: creator + Owner/Technical Lead + Admin only,
    # same rule already exercised in test workflow status assignments.py) -
    # this test is about the outdated-analysis notification, so the editor
    # needs a role that can actually get the PUT past permission checks.
    other_headers, other_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "technical_lead"},
        headers=headers,
    )

    edit_response = client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=other_headers,
    )
    assert edit_response.status_code == 200

    notifs = _notifications(headers)
    assert any(n["type"] == "analysis_outdated" for n in notifs)

def test_editing_after_approval_requested_notifies_approver_of_reapproval_needed():
    headers, _ = _auth()
    created = create_change_request(headers)
    approver_headers, approver_id = _auth()
    client.post(
        f"/api/change-requests/{created['id']}/approvals",
        json={"approval_type": "dba", "approver_user_id": approver_id},
        headers=headers,
    )

    client.put(
        f"/api/change-requests/{created['id']}",
        json={"description": "Allow customers to log in using an OTP sent by SMS or email."},
        headers=headers,
    )

    notifs = _notifications(approver_headers)
    assert any(n["type"] == "reapproval_required" for n in notifs)

```

tests/test_workflow_foundation.py

414 lines

```
"""Module 12 (Enterprise Workflow), Phase 1: the foundation - new tables
(versions, audit history, assignments, approvals, comments, notifications),
the expanded status lifecycle, the approval matrix, the "is analysis
outdated" check, and the per-CR permission helpers. No new API endpoints
yet (those come in later phases) - this exercises the models and
app/services/workflow_rules.py directly.

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from datetime import date
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.database.session import SessionLocal
from app.main import app
from app.models.analysis import Analysis
from app.models.approval import Approval
from app.models.change_request import ChangeRequest
from app.models.change_request_assignment import ChangeRequestAssignment
from app.models.change_request_history import ChangeRequestHistory
from app.models.change_request_version import ChangeRequestVersion
from app.models.comment import ChangeRequestComment
from app.models.enums import (
    ApprovalStatus,
    ApprovalType,
    AssignmentRole,
    ChangeRequestStatus,
    ComplexityLevel,
    HistoryAction,
    NotificationType,
    Priority,
    RiskCategory,
    Severity,
    UserRole,
)
from app.models.notification import Notification
from app.models.risk import Risk
from app.models.user import User
from app.services import workflow_rules

client = TestClient(app)

def _unique_email() -> str:
    return f"wf-{uuid.uuid4().hex[:10]}@example.com"

def _auth(role: str = "engineer") -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Workflow Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}"}, body["user"]["id"]

def _create_cr_via_api(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP authentication",
        "description": "Allow customers to log in using OTP sent to their registered mobile number.",
        "priority": "medium",
        "requested_by": "Shivani",
        "target_system": "Mobile App",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

# --- New rows get a real version 1 without needing the backfill -----

def test_new_change_request_gets_current_version_1():
    headers, _ = _auth()
    created = _create_cr_via_api(headers)
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        assert cr.current_version == 1

# --- Backfill: existing rows (current version left NULL, as a pre-Module-12
# row would be) get fixed up by init_db() -----

def test_backfill_gives_pre_existing_rows_version_1_and_a_baseline_version_and_history():
    headers, user_id = _auth()
    created = _create_cr_via_api(headers)

    # Simulate a CR that existed before this column/table existed: null out
    # current version and remove the version/history rows create change request()
    # itself now writes, the way a real pre-Module-12 row would sit in the
    # database before init_db() ever ran the backfill (no version or history
    # rows at all, since those tables didn't exist yet either).
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        cr.current_version = None
        db.query(ChangeRequestVersion).filter(ChangeRequestVersion.change_request_id == cr.id).delete()
        db.query(ChangeRequestHistory).filter(ChangeRequestHistory.change_request_id == cr.id).delete()
        db.commit()

    init_db() # re-run - this is what happens on every backend startup

    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        assert cr.current_version == 1

    version = (
        db.query(ChangeRequestVersion)
        .filter(ChangeRequestVersion.change_request_id == cr.id, ChangeRequestVersion.version_number == 1)
        .first()
    )
}
```

```

    assert version is not None
    assert version.change_summary == "Initial version imported."
    assert '"title"' in version.snapshot

    created_event = (
        db.query(ChangeRequestHistory)
        .filter(
            ChangeRequestHistory.change_request_id == cr.id,
            ChangeRequestHistory.action == HistoryAction.CREATED,
        )
        .first()
    )
    assert created_event is not None
    assert created_event.user_id == user_id

def test_backfill_is_idempotent_no_duplicate_version_rows():
    headers = _auth()
    created = _create_cr_via_api(headers)
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        cr.current_version = None
        db.query(ChangeRequestVersion).filter(ChangeRequestVersion.change_request_id == cr.id).delete()
        db.query(ChangeRequestHistory).filter(ChangeRequestHistory.change_request_id == cr.id).delete()
        db.commit()

    init_db()
    init_db() # running it twice in a row must not double up

    with SessionLocal() as db:
        count = (
            db.query(ChangeRequestVersion)
            .filter(ChangeRequestVersion.change_request_id == created["id"])
            .count()
        )
        assert count == 1

def test_backfill_does_not_duplicate_the_version_and_history_rows_creation_already_wrote():
    """Regression guard: create_change_request() already writes a version-1
    row and a CREATED history event for every brand-new CR. If current version
    alone ever ends up NULL on such a row (without its version/history rows
    being touched), the backfill must recognize they already exist rather
    than blindly inserting a second set."""
    headers, user_id = _auth()
    created = _create_cr_via_api(headers)

    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        cr.current_version = None
        db.commit()

    init_db()

    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        assert cr.current_version == 1

    versions = (
        db.query(ChangeRequestVersion)
        .filter(ChangeRequestVersion.change_request_id == cr.id, ChangeRequestVersion.version_number == 1)
        .all()
    )
    assert len(versions) == 1
    assert versions[0].change_summary == "Change request created." # the original, not overwritten

    created_events = (
        db.query(ChangeRequestHistory)
        .filter(
            ChangeRequestHistory.change_request_id == cr.id,
            ChangeRequestHistory.action == HistoryAction.CREATED,
        )
        .all()
    )
    assert len(created_events) == 1
    assert created_events[0].user_id == user_id

# --- Status transition graph -----

def test_valid_transitions_allowed():
    assert workflow_rules.can_transition(ChangeRequestStatus.IN_REVIEW, ChangeRequestStatus.CHANGES_REQUESTED)
    assert workflow_rules.can_transition(ChangeRequestStatus.IN_REVIEW, ChangeRequestStatus.APPROVAL_REQUIRED)
    assert workflow_rules.can_transition(ChangeRequestStatus.APPROVAL_REQUIRED, ChangeRequestStatus.APPROVED)
    assert workflow_rules.can_transition(ChangeRequestStatus.VALIDATED, ChangeRequestStatus.CLOSED)

def test_invalid_transitions_rejected():
    # Can't skip straight from Draft to Approved.
    assert not workflow_rules.can_transition(ChangeRequestStatus.DRAFT, ChangeRequestStatus.APPROVED)
    # Closed is terminal.
    assert not workflow_rules.can_transition(ChangeRequestStatus.CLOSED, ChangeRequestStatus.IN_PROGRESS)
    # No-op "transition" to the same status isn't a transition.
    assert not workflow_rules.can_transition(ChangeRequestStatus.APPROVED, ChangeRequestStatus.APPROVED)

def test_reason_required_for_sensitive_transitions():
    assert workflow_rules.reason_required(ChangeRequestStatus.CHANGES_REQUESTED)
    assert workflow_rules.reason_required(ChangeRequestStatus.REJECTED)
    assert workflow_rules.reason_required(ChangeRequestStatus.CANCELLED)
    assert not workflow_rules.reason_required(ChangeRequestStatus.APPROVED)

# --- Approval matrix (AI-driven recommendation, never an auto-approval) ---

def _fake_analysis(risk_score: float, category: str = "Feature", security_risk: bool = False) -> Analysis:
    analysis = Analysis(
        change_request_id=0,
        summary="s",
        category=category,
        complexity=ComplexityLevel.MEDIUM,
        risk_score=risk_score,
        confidence_score=90,
    )
    if security_risk:
        analysis.risks = [
            Risk(category=RiskCategory.SECURITY, severity=Severity.HIGH, probability=0.5, score=80, description="d")
        ]
    else:
        analysis.risks = []
    return analysis

def test_low_risk_requires_only_technical_approval():
    cr = ChangeRequest(title="t", description="d", target_system="Reporting Dashboard")
    required = workflow_rules.required_approval_types(cr, _fake_analysis(10, category="Feature"))
    assert required == {ApprovalType.TECHNICAL}

```

```

def test_critical_risk_requires_manager_security_and_director():
    cr = ChangeRequest(title="t", description="d", target_system="Reporting Dashboard")
    required = workflow_rules.required_approval_types(cr, _fake_analysis(90, category="Feature"))
    assert {ApprovalType.ENGINEERING_MANAGER, ApprovalType.SECURITY, ApprovalType.DIRECTOR} <= required

def test_security_finding_always_adds_security_approval_even_at_low_risk():
    cr = ChangeRequest(title="t", description="d", target_system="Reporting Dashboard")
    required = workflow_rules.required_approval_types(cr, _fake_analysis(10, category="Feature", security_risk=True))
    assert ApprovalType.SECURITY in required

def test_customer_facing_target_system_adds_product_approval():
    cr = ChangeRequest(title="t", description="d", target_system="Mobile App")
    required = workflow_rules.required_approval_types(cr, _fake_analysis(10, category="Feature"))
    assert ApprovalType.PRODUCT in required

# --- AI analysis version awareness -----

def test_analysis_outdated_when_cr_version_moves_past_it():
    cr = ChangeRequest(title="t", description="d")
    cr.current_version = 2
    analysis = _fake_analysis(10)
    analysis.change_request_version = 1
    assert workflow_rules.is_analysis_outdated(cr, analysis)

def test_analysis_up_to_date_when_versions_match():
    cr = ChangeRequest(title="t", description="d")
    cr.current_version = 3
    analysis = _fake_analysis(10)
    analysis.change_request_version = 3
    assert not workflow_rules.is_analysis_outdated(cr, analysis)

def test_no_analysis_is_not_reported_as_outdated():
    cr = ChangeRequest(title="t", description="d")
    cr.current_version = 1
    assert not workflow_rules.is_analysis_outdated(cr, None)

# --- Permissions -----

def test_creator_can_edit_own_cr():
    headers, user_id = _auth()
    created = _create_cr_via_api(headers)
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        user = db.get(User, user_id)
        assert workflow_rules.can_edit_change_request(user, cr, [])

def test_unrelated_user_cannot_edit():
    headers, _ = _auth()
    created = _create_cr_via_api(headers)
    other_headers, other_id = _auth()
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        other_user = db.get(User, other_id)
        assert not workflow_rules.can_edit_change_request(other_user, cr, [])

def test_assigned_owner_can_edit_even_if_not_creator():
    headers, _ = _auth()
    created = _create_cr_via_api(headers)
    owner_headers, owner_id = _auth()
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        owner_user = db.get(User, owner_id)
        assignment = ChangeRequestAssignment(
            change_request_id=cr.id, user_id=owner_id, role=AssignmentRole.OWNER, assigned_by=owner_id
        )
        assert workflow_rules.can_edit_change_request(owner_user, cr, [assignment])

def test_admin_can_always_edit():
    headers, _ = _auth()
    created = _create_cr_via_api(headers)
    with SessionLocal() as db:
        cr = db.get(ChangeRequest, created["id"])
        admin = User(name="Admin", email=unique_email(), password_hash="x", role=UserRole.ADMIN)
        db.add(admin)
        db.commit()
        db.refresh(admin)
        assert workflow_rules.can_edit_change_request(admin, cr, [])

# --- New tables round-trip through real relationships -----

def test_history_version_assignment_approval_comment_notification_roundtrip():
    headers, user_id = _auth()
    created = _create_cr_via_api(headers)
    cr_id = created["id"]

    with SessionLocal() as db:
        user = db.get(User, user_id)

        db.add(
            ChangeRequestHistory(
                change_request_id=cr_id,
                user_id=user_id,
                action=HistoryAction.FIELD_CHANGED,
                field_name="priority",
                old_value="medium",
                new_value="high",
                reason=None,
            )
        )
        db.add(
            ChangeRequestVersion(
                change_request_id=cr_id,
                version_number=2,
                changed_by=user_id,
                change_summary="Priority changed from Medium to High",
                snapshot={},
            )
        )
        db.add(
            ChangeRequestAssignment(
                change_request_id=cr_id, user_id=user_id, role=AssignmentRole.SECURITY_REVIEWER, assigned_by=user_id
            )
        )
        approval = Approval(
            change_request_id=cr_id,
            approver_id=user_id,
            approval_type=ApprovalType.SECURITY,
            status=ApprovalStatus.PENDING,

```

```

        requested_by=user_id,
        cr_version=1,
    )
    db.add(approval)
    db.add(ChangeRequestComment(change_request_id=cr_id, user_id=user_id, body="Please define OTP retry limits. "))
    db.add(
        Notification(
            user_id=user_id,
            type=NotificationType.APPROVAL_REQUESTED,
            title="Approval requested",
            message="You were tagged for Security Approval.",
            change_request_id=cr_id,
        )
    )
    db.commit()

cr = db.get(ChangeRequest, cr_id)
assert len(cr.history) >= 1
assert any(v.version number == 2 for v in cr.versions)
assert any(a.role == AssignmentRole.SECURITY REVIEWER for a in cr.assignments)
assert any(a.approval_type == ApprovalType.SECURITY for a in cr.approvals)
assert len(cr.comments) == 1
assert cr.comments[0].body.startswith("Please define")

notification = db.query(Notification).filter(Notification.change_request_id == cr_id).first()
assert notification is not None
assert notification.is_read is False

```

tests/test_workflow_status_assignments.py

299 lines

```
"""Verifies Module 12 Phase 3 (Enterprise Workflow - Status Lifecycle +
Assignments):

* PUT /api/change-requests/{id}/status - only a legal next step is
  accepted, moving into a "something went wrong" status requires a
  reason, only the CR owner/creator/admin can do it, and every change is
  recorded on the audit history
* GET .../{id} exposes status_label + available_transitions computed
  fresh from app/services/workflow_rules.py
* GET/POST/DELETE .../{id}/assignments - assigning/unassigning a team
  member to a role, permission-gated, duplicate-assignment rejected,
  recorded on the audit history
* GET /api/users - the simple directory the assignment picker uses

Run with (from backend/): pytest ../tests
"""
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from app.database.init_db import init_db

init_db()

from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def _unique_email() -> str:
    return f"wfsa-{uuid.uuid4().hex[:10]}@example.com"

def _auth() -> tuple[dict, int]:
    email = _unique_email()
    password = "supersecret123"
    response = client.post(
        "/auth/register",
        json={"name": "Status Tester", "email": email, "password": password, "confirm_password": password},
    )
    assert response.status_code == 201
    body = response.json()
    return {"Authorization": f"Bearer {body['access_token']}", body["user"]["id"]}

def _create_change_request(headers: dict, **overrides) -> dict:
    payload = {
        "title": "Add OTP login to mobile app",
        "description": "Allow customers to log in using a one-time passcode sent to their registered number.",
        "priority": "medium",
        "requested by": "Morgan Lee",
        "target_system": "Mobile App",
    }
    payload.update(overrides)
    response = client.post("/api/change-requests", json=payload, headers=headers)
    assert response.status_code == 201
    return response.json()

def _set_status(cr_id: int, target: str, headers: dict, reason: str | None = None):
    body = {"status": target}
    if reason is not None:
        body["reason"] = reason
    return client.put(f"/api/change-requests/{cr_id}/status", json=body, headers=headers)

# --- Status transitions -----

def test_status_change_requires_authentication():
    response = client.put("/api/change-requests/1/status", json={"status": "analyzed"})
    assert response.status_code == 401

def test_status_change_404_for_unknown_change_request():
    headers, _ = _auth()
    response = _set_status(999999999, "analyzed", headers)
    assert response.status_code == 404

def test_creator_can_advance_status_through_valid_path():
    headers, _ = _auth()
    created = _create_change_request(headers)
    assert created["status"] == "pending_analysis"

    response = _set_status(created["id"], "analyzed", headers)
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "analyzed"
    assert body["status_label"] == "Analyzed"

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    status_events = [h for h in history if h["action"] == "status_changed"]
    assert len(status_events) == 1
    assert status_events[0]["old_value"] == "Pending Analysis"
    assert status_events[0]["new_value"] == "Analyzed"

def test_invalid_transition_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    # Pending Analysis can't jump straight to Approved.
    response = _set_status(created["id"], "approved", headers)
    assert response.status_code == 422

def test_transition_into_reason_required_status_without_reason_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    cr_id = created["id"]
    assert _set_status(cr_id, "analyzed", headers).status_code == 200
    assert _set_status(cr_id, "in_review", headers).status_code == 200

    response = _set_status(cr_id, "changes_requested", headers) # no reason
    assert response.status_code == 422

    response = _set_status(cr_id, "changes_requested", headers, reason="Missing rollback plan.")
    assert response.status_code == 200
    assert response.json()["status"] == "changes_requested"
```

```

history = client.get(f"/api/change-requests/{cr_id}/history", headers=headers).json()
status_events = [h for h in history if h["action"] == "status_changed"]
last = status_events[0] # newest first
assert last["new_value"] == "Changes Requested"
assert last["reason"] == "Missing rollback plan."

def test_unrelated_user_cannot_change_status():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, _ = _auth()

    response = _set_status(created["id"], "analyzed", other_headers)
    assert response.status_code == 403

def test_available_transitions_reflect_current_status():
    headers, _ = _auth()
    created = _create_change_request(headers)

    detail = client.get(f"/api/change-requests/{created['id']}", headers=headers).json()
    transitions = {t["status"]: t["requires_reason"] for t in detail["available_transitions"]}
    assert transitions == {"analyzed": False, "cancelled": True}

# --- Assignments -----

def test_assignments_require_authentication():
    response = client.get("/api/change-requests/1/assignments")
    assert response.status_code == 401
    response = client.post("/api/change-requests/1/assignments", json={"user_id": 1, "role": "owner"})
    assert response.status_code == 401

def test_creator_can_assign_a_user_and_it_appears_in_list_and_history():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, other_id = _auth()

    response = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "technical_lead"},
        headers=headers,
    )
    assert response.status_code == 201
    body = response.json()
    assert body["user_id"] == other_id
    assert body["role"] == "technical_lead"
    assert body["role_label"] == "Technical Lead"

    listing = client.get(f"/api/change-requests/{created['id']}/assignments", headers=headers).json()
    assert len(listing) == 1
    assert listing[0]["user_id"] == other_id

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    assigned_events = [h for h in history if h["action"] == "assigned"]
    assert len(assigned_events) == 1
    assert "Technical Lead" in assigned_events[0]["new_value"]

    # The newly-assigned Technical Lead can now also manage this CR (e.g.
    # change its status), even though they didn't create it.
    assert _set_status(created["id"], "analyzed", other_headers).status_code == 200

def test_duplicate_assignment_is_rejected():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, other_id = _auth()

    first = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "reviewer"},
        headers=headers,
    )
    assert first.status_code == 201

    second = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "reviewer"},
        headers=headers,
    )
    assert second.status_code == 409

def test_assigning_unknown_user_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": 999999999, "role": "reviewer"},
        headers=headers,
    )
    assert response.status_code == 404

def test_unrelated_user_cannot_assign():
    headers, _ = _auth()
    created = _create_change_request(headers)
    other_headers, other_id = _auth()

    response = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "reviewer"},
        headers=other_headers,
    )
    assert response.status_code == 403

def test_delete_assignment_removes_it_and_records_history():
    headers, _ = _auth()
    created = _create_change_request(headers)
    _, other_id = _auth()

    created_assignment = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "reviewer"},
        headers=headers,
    ).json()

    response = client.delete(
        f"/api/change-requests/{created['id']}/assignments/{created_assignment['id']}", headers=headers
    )
    assert response.status_code == 204

    listing = client.get(f"/api/change-requests/{created['id']}/assignments", headers=headers).json()
    assert listing == []

    history = client.get(f"/api/change-requests/{created['id']}/history", headers=headers).json()
    unassigned_events = [h for h in history if h["action"] == "unassigned"]

```



```

assert len(unassigned_events) == 1
assert "Reviewer" in unassigned_events[0]["old_value"]

def test_delete_unknown_assignment_returns_404():
    headers, _ = _auth()
    created = _create_change_request(headers)

    response = client.delete(f"/api/change-requests/{created['id']}/assignments/999999999", headers=headers)
    assert response.status_code == 404

def test_unrelated_user_cannot_delete_assignment():
    headers, _ = _auth()
    created = create_change_request(headers)
    _, other_id = _auth()
    assignment = client.post(
        f"/api/change-requests/{created['id']}/assignments",
        json={"user_id": other_id, "role": "reviewer"},
        headers=headers,
    ).json()

    outsider_headers, _ = _auth()
    response = client.delete(
        f"/api/change-requests/{created['id']}/assignments/{assignment['id']}", headers=outsider_headers
    )
    assert response.status_code == 403

# --- User directory -----

def test_list_users_requires_authentication():
    response = client.get("/api/users")
    assert response.status_code == 401

def test_list_users_returns_registered_users():
    headers, user_id = _auth()
    response = client.get("/api/users", headers=headers)
    assert response.status_code == 200
    ids = {u["id"] for u in response.json()}
    assert user_id in ids

```